

Apache Pulsar

В ДЕЙСТВИИ



Дэвид Хьеррумгор



MANNING



Дэвид Херрумгор

Apache Pulsar в действии

Apache Pulsar in Action

DAVID KJERRUMGAARD

Foreword By Matteo Merli



MANNING
Shelter Island

Apache Pulsar в действии

ДЭВИД ХЬЕРРУМГОР

С предисловием Маттео Мерли



Москва, 2023

УДК 004.43
ББК 32.973.2
X98

Дэвид Хьеррумгор

X98 Apache Pulsar в действии / пер. с англ. А. В. Снастина. – М.: ДМК Пресс, 2023. – 490 с.: ил.

ISBN 978-5-93700-251-8

Книга научит вас создавать масштабируемые системы потокового обмена сообщениями с использованием Pulsar. Вы начнете с быстрого ознакомления с корпоративными системами обмена сообщениями и откроете для себя уникальные преимущества Pulsar. Следуя четким описаниям и выполняя практические примеры, вы будете использовать фреймворк Pulsar Functions для разработки приложения на основе микросервисов.

Издание предназначено для опытных разработчиков на языке Java. Предварительные знания о платформе Apache Pulsar не требуются.

УДК 004.43
ББК 32.973.2

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN (анг.) 978-1-61729-688-8
ISBN (рус.) 978-5-93700-251-8

© 2021 by Manning Publications Co.
© Оформление, издание, перевод,
ДМК Пресс, 2023

Моему отцу, который обещал прочитать эту книгу, даже если не сможет понять в ней ни слова. Возможно, твое сердце переполняется гордостью каждый раз, когда ты говоришь своим друзьям, что твой сын известный автор книг

Моей матери, которая верила в меня, когда никто не верил, и упорно трудилась для того, чтобы я получил любую возможность добиться успеха в жизни. Твой тяжелый труд, вера и стойкость всегда были источником вдохновения для меня. Спасибо, что передала мне эти черты характера, поскольку они были основой всех моих достижений, включая эту книгу

Краткое оглавление

ЧАСТЬ I. Знакомство с Apache Pulsar	29
1 ■ <i>Введение в Apache Pulsar</i>	<i>32</i>
2 ■ <i>Концепции и архитектура Pulsar</i>	<i>77</i>
3 ■ <i>Взаимодействие с Pulsar</i>	<i>115</i>
ЧАСТЬ II. Основы разработки с использованием Apache Pulsar	151
4 ■ <i>Функции Pulsar</i>	<i>153</i>
5 ■ <i>Коннекторы ввода-вывода Pulsar.....</i>	<i>194</i>
6 ■ <i>Обеспечение безопасности Pulsar.....</i>	<i>232</i>
7 ■ <i>Реестр схем</i>	<i>270</i>
ЧАСТЬ III. Практическая разработка приложений с использованием Apache Pulsar	303
8 ■ <i>Паттерны применения Pulsar Functions.....</i>	<i>305</i>
9 ■ <i>Паттерны устойчивости</i>	<i>329</i>
10 ■ <i>Доступ к данным</i>	<i>368</i>
11 ■ <i>Машинное обучение в Pulsar</i>	<i>392</i>
12 ■ <i>Периферийная аналитика</i>	<i>415</i>

Оглавление

Предисловие от издательства	13
Предисловие	14
Предисловие автора	16
Благодарности	19
Об этой книге	21
Об авторе.....	27
Об иллюстрации на обложке	28
ЧАСТЬ I. Знакомство с Apache Pulsar	29
1 <i>Введение в Apache Pulsar</i>	32
1.1. Корпоративные системы обмена сообщениями	33
1.1.1. Основные функциональные возможности	36
1.2. Паттерны потребления сообщений	37
1.2.1. Обмен сообщениями по схеме публикация–подписка ...	37
1.2.2. Очередь сообщений	38
1.3. История развития систем обмена сообщениями	39
1.3.1. Системы обмена сообщениями общего назначения ...	39
1.3.2. Программное обеспечение среднего звена, ориентированное на сообщения	40
1.3.3. Сервисная шина предприятия	42
1.3.4. Распределенные системы обмена сообщениями	45
1.4. Сравнение с Apache Kafka	52
1.4.1. Многоуровневая архитектура	52
1.4.2. Потребление сообщений	55
1.4.3. Постоянство хранения данных	58
1.4.4. Подтверждение приема сообщений	61
1.4.5. Длительность хранения сообщений	64
1.5. Почему необходим именно Pulsar	65
1.5.1. Гарантированная доставка сообщений.....	66
1.5.2. Неограниченная масштабируемость	66
1.5.3. Устойчивость к критическим ошибкам	67
1.5.4. Поддержка миллионов тем.....	69
1.5.5. Георепликация и активная отказоустойчивость	70
1.6. Варианты использования из реальной практики	72
1.6.1. Универсальные системы обмена сообщениями	72
1.6.2. Платформы микросервисов	73
1.6.3. Автомобили с сетевыми функциями	74
1.6.4. Выявление случаев мошенничества.....	74

1.7. Дополнительные информационные ресурсы	75
1.8. Резюме	76

2 Концепции и архитектура Pulsar 77

2.1. Физическая архитектура Pulsar	78
2.1.1. Многоуровневая архитектура Pulsar	79
2.1.2. Уровень обслуживания без сохранения состояния	81
2.1.3. Уровень хранения потоковых данных	84
2.1.4. Хранилище метаданных	89
2.2. Логическая архитектура Pulsar	92
2.2.1. Абоненты, пространства имен и темы	92
2.2.2. Адресация тем в Pulsar	97
2.2.3. Производители, потребители и подписки	98
2.2.4. Типы подписки	99
2.3. Долговременное хранение сообщений и сроки их хранения	104
2.3.1. Долговременное хранение данных	104
2.3.2. Квоты для журналов регистрации	106
2.3.3. Окончание срока хранения сообщений	108
2.3.4. Сравнение журнала регистрации сообщений и определения срока хранения сообщений	109
2.4. Многоуровневое хранилище	110
2.5. Резюме	114

3 Взаимодействие с Pulsar 115

3.1. Начинаем работать с Pulsar	116
3.2. Администрирование Pulsar	117
3.2.1. Создание абонента, пространства имен и темы	118
3.2.2. API администрирования на языке Java	119
3.3. Клиенты Pulsar	120
3.3.1. Клиент Pulsar на языке Java	123
3.3.2. Клиент Pulsar на языке Python	134
3.3.3. Клиент Pulsar на языке Go	138
3.4. Дополнительное администрирование	144
3.4.1. Метрики постоянно хранимой темы	144
3.4.2. Инспекция сообщений	147
3.5. Резюме	148

ЧАСТЬ II. Основы разработки с использованием Apache Pulsar 151

4 Функции Pulsar 153

4.1. Поточковая обработка	153
4.1.1. Обычная пакетная обработка	154

4.1.2. Микропакетная обработка данных	155
4.1.3. Поточковая нативная обработка	155
4.2. Что такое Pulsar Functions	156
4.2.1. Модель программирования	158
4.3. Разработка функций Pulsar	159
4.3.1. Ориентированные на язык функции	159
4.3.2. Pulsar SDK	160
4.3.3. Функции с сохранением состояния	166
4.4. Тестирование функций Pulsar	170
4.4.1. Модульное тестирование	171
4.4.2. Комплексное тестирование	173
4.5. Развертывание функций Pulsar	178
4.5.1. Генерация артефакта развертывания	179
4.5.2. Конфигурация функции	182
4.5.3. Развертывание функции	186
4.5.4. Жизненный цикл развертывания функции	189
4.5.5. Режимы развертывания	190
4.5.6. Поток данных в функции Pulsar	192
4.6. Резюме	193
5 Коннекторы ввода-вывода Pulsar	194
5.1. Что такое коннекторы ввода-вывода Pulsar	195
5.1.1. Коннекторы-приемники	196
5.1.2. Коннекторы-источники	199
5.1.3. Коннекторы типа PushSource	201
5.2. Разработка коннекторов ввода-вывода Pulsar	203
5.2.1. Разработка коннектора-приемника	203
5.2.2. Разработка коннектора PushSource	205
5.3. Тестирование коннекторов ввода-вывода Pulsar	209
5.3.1. Модульное тестирование	209
5.3.2. Комплексное тестирование	211
5.3.3. Упаковка коннекторов ввода-вывода Pulsar	214
5.4. Развертывание коннекторов ввода-вывода Pulsar	215
5.4.1. Создание и удаление коннекторов	215
5.4.2. Отладка развернутых коннекторов	218
5.5. Встроенные коннекторы Pulsar	221
5.5.1. Запуск кластера MongoDB	221
5.5.2. Связывание контейнеров Pulsar и MongoDB	223
5.5.3. Конфигурирование и создание приемника для MongoDB	224
5.6. Администрирование коннекторов ввода-вывода Pulsar	226
5.6.1. Вывод списка коннекторов	226
5.6.2. Мониторинг коннекторов	228
5.7. Резюме	231

6	<i>Обеспечение безопасности Pulsar</i>	232
6.1.	Шифрование на транспортном уровне	233
6.2.	Аутентификация	242
6.2.1.	Аутентификация TLS	243
6.2.2.	Аутентификация JSON Web Token (JWT)	249
6.3.	Авторизация	255
6.3.1.	Роли	256
6.3.2.	Пример сценария	258
6.4.	Шифрование сообщений	264
6.5.	Резюме	269
7	<i>Реестр схем</i>	270
7.1.	Обмен информацией между микросервисами	271
7.1.1.	API микросервисов	272
7.1.2.	Обоснование необходимости реестра схем	274
7.2.	Реестр схем Pulsar	275
7.2.1.	Архитектура	275
7.2.2.	Управление версиями схем	279
7.2.3.	Совместимость схемы	279
7.2.4.	Стратегии проверки совместимости схем	282
7.3.	Использование реестра схем	287
7.3.1.	Моделирование события заказа продуктов в Avro	289
7.3.2.	События производства заказов продуктов	292
7.3.3.	События потребления заказов продуктов	295
7.3.4.	Полный пример	296
7.4.	Развитие схемы	299
7.5.	Резюме	302

ЧАСТЬ III. Практическая разработка приложений с использованием Apache Pulsar 303

8	<i>Паттерны применения Pulsar Functions</i>	305
8.1.	Конвейеры данных	306
8.1.1.	Процедурное программирование	306
8.1.2.	Программирование с использованием потока данных	307
8.2.	Паттерны маршрутизации сообщений	310
8.2.1.	Паттерн «разделитель»	310
8.2.2.	Паттерн «динамический маршрутизатор»	315
8.2.3.	Паттерн «маршрутизатор на основе содержимого»	319
8.3.	Паттерны преобразования сообщений	322
8.3.1.	Паттерн «транслятор сообщений»	322
8.3.2.	Паттерн «улучшение содержимого»	325
8.3.3.	Паттерн «фильтр содержимого»	327
8.4.	Резюме	328

9	<i>Паттерны устойчивости</i>	329
9.1.	Устойчивость Pulsar Functions	331
9.1.1.	Неблагоприятные события	331
9.1.2.	Обнаружение ошибок	337
9.2.	Паттерны проектных решений по обеспечению устойчивости	339
9.2.1.	Паттерн повтора	340
9.2.2.	Паттерн «прерыватель замкнутого цикла»	344
9.2.3.	Паттерн «ограничитель скорости»	350
9.2.4.	Паттерн «ограничитель времени»	354
9.2.5.	Паттерн «кеш»	358
9.2.6.	Паттерн «откат»	360
9.2.7.	Паттерн «обновление идентификационных данных»	362
9.3.	Несколько уровней устойчивости	365
9.4.	Резюме	367
10	<i>Доступ к данным</i>	368
10.1.	Источники данных	369
10.2.	Варианты использования доступа к данным	371
10.2.1.	Проверка устройства	371
10.2.2.	Данные о локации водителя	383
10.3.	Резюме	391
11	<i>Машинное обучение в Pulsar</i>	392
11.1.	Развертывание моделей машинного обучения	393
11.1.1.	Режим пакетной обработки	393
11.1.2.	Режим обработки почти в реальном времени	394
11.2.	Развертывание модели в режиме почти реального времени	395
11.3.	Векторы признаков	397
11.3.1.	Хранилища признаков	398
11.3.2.	Вычисление признаков	400
11.4.	Оценка времени доставки	401
11.4.1.	Экспорт модели машинного обучения	401
11.4.2.	Отображение вектора признаков	404
11.4.3.	Развертывание модели	407
11.5.	Нейронные сети	409
11.5.1.	Тренировка нейронной сети	410
11.5.2.	Развертывание нейронной сети средствами языка Java	412
11.6.	Резюме	414

12 *Периферийная аналитика* 415

12.1. Архитектура промышленного интернета вещей.....	419
12.1.1. Уровень восприятия и реакции.....	419
12.1.2. Уровень передачи данных.....	421
12.1.3. Уровень обработки данных.....	421
12.2. Уровень обработки данных на основе Pulsar.....	422
12.3. Периферийная аналитика.....	425
12.3.1. Телеметрические данные.....	426
12.3.2. Одномерные и многомерные наборы данных.....	428
12.4. Одномерный анализ.....	429
12.4.1. Снижение шума.....	430
12.4.2. Статистический анализ.....	432
12.4.3. Аппроксимация.....	437
12.5. Многомерный анализ.....	440
12.5.1. Создание двунаправленной сетки обмена сообщениями.....	441
12.5.2. Создание многомерного набора данных.....	445
12.6. Послесловие.....	451
12.7. Резюме.....	453

Приложение А. Запуск Pulsar в Kubernetes..... 454

A.1. Создание кластера Kubernetes.....	454
A.1.1. Предварительные условия для установки.....	456
A.1.2. Minikube.....	456
A.2. Pulsar Helm chart.....	458
A.2.1. Что такое Helm.....	459
A.2.2. Pulsar Helm chart.....	461
A.3. Использование Pulsar Helm chart.....	465
A.3.1. Администрирование Pulsar в среде Kubernetes.....	467
A.3.2. Конфигурирование клиентов.....	468

Приложение В. Георепликация..... 469

V.1. Синхронная георепликация.....	469
V.2. Асинхронная георепликация.....	472
V.2.1. Конфигурирование асинхронной георепликации ...	473
V.3. Паттерны асинхронной георепликации.....	477
V.3.1. Георепликация по схеме ведущий–ведущий.....	477
V.3.2. Георепликация по схеме активный–резервный.....	479
V.3.3. Агрегатная георепликация.....	480
Предметный указатель.....	482

Предисловие от издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг – возможно, ошибку в основном тексте или программном коде, – мы будем очень благодарны, если вы сообщите нам о ней. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Если вы найдете какие-либо ошибки в коде, пожалуйста, сообщите о них главному редактору по адресу dmkpress@gmail.com, и мы исправим это в следующих тиражах.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательство «ДМК Пресс» очень серьезно относится к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Предисловие

Книга «Apache Pulsar в действии» – это недостающее руководство, которое проведет вас через все этапы работы с Apache Pulsar. Эту книгу я порекомендовал бы прочесть всем: от разработчиков, начинающих изучать механизм обмена сообщениями по схеме публикация–подписка (pub-sub), до тех, у кого есть опыт обмена сообщениями, и до опытных пользователей Pulsar.

Работа над проектом Apache Pulsar началась в компании Yahoo! приблизительно с 2012 г. во время экспериментов с новой архитектурой, которая должна была решить эксплуатационные задачи на существующих платформах обмена сообщениями. К тому же это было время, когда некоторые значительные сдвиги в мире инфраструктуры данных становились все более заметными. Разработчики приложений начали обращать больше внимания на масштабируемый и надежный механизм обмена сообщениями как на основной компонент для создания следующего поколения программных продуктов. В то же время компании определили крупномасштабные системы анализа потоковых данных в реальном времени как важнейший компонент и преимущество для бизнеса.

Pulsar был спроектирован с нуля с целью установления прочной связи между этими двумя сферами – механизма обмена по схеме публикация–подписка (pub-sub) и анализа потоковых данных, которые весьма часто были изолированы друг от друга. Мы работали над созданием инфраструктуры, представляющей следующее поколение платформ обработки данных в реальном времени, где единственная система могла бы поддерживать все варианты использования на протяжении полного жизненного цикла событий, связанных с данными.

Со временем это представление расширилось, как можно ясно видеть по широкому диапазону компонентов, описанных в этой книге. В проект была добавлена поддержка упрощенной обработки с помощью Pulsar Functions, фреймворка коннекторов Pulsar IO, поддержка схем данных и многие другие функциональные возможности. Не изменилась только конечная цель – создание в высшей степени масштабируемой, гибкой и надежной платформы для обработки данных в реальном времени, позволяющей любому пользователю работать с данными, хранящимися в Pulsar, в наиболее удобной форме.

Я знаком с автором этой книги Дэвидом Херрумгором и работал вместе с ним в течение нескольких лет. За это время я обратил вни-

мание на его глубокую увлеченность работой в сообществе Pulsar. Он всегда готов помочь пользователям решить технические проблемы и затруднения, а также продемонстрировать им все возможности Pulsar для решения конкретной задачи обработки данных.

Особенно важным я считаю тот факт, что в книге органично сочетается теория и абстрактные концепции с ясностью практических пошаговых примеров и что эти примеры основаны на широко известных вариантах использования и паттернах проектирования обмена сообщениями, которые, несомненно, заинтересуют многих читателей. Здесь действительно каждый найдет что-то полезное для себя, и каждый сможет ознакомиться со всеми аспектами и возможностями, которые предлагает Pulsar.

— *Mammeo Мерли (Matteo Merli)*,
технический директор (CTO) Stream Native,
один из создателей и председатель комитета управления
проектом (PMC Chair) Apache Pulsar

Предисловие автора

В 2012 г. команда Yahoo! искала глобальную платформу с георепликацией, которая могла бы обеспечить потоковую обработку всех данных Yahoo! при обмене сообщениями между различными приложениями, например Yahoo Mail и Yahoo Finance. В то время существовали два основных типа систем обработки динамических данных: очереди сообщений, обрабатывающие особо важные бизнес-события в реальном времени, и потоковые системы, которые обрабатывали динамически масштабируемые конвейеры данных. Но при этом не было платформы, предоставляющей одновременно обе функциональные возможности, требуемые для компании Yahoo.

После тщательного исследования положения дел в области обмена сообщениями и потоковой обработки данных стало ясно, что существующие технологии не способны удовлетворить эти потребности, поэтому команда Yahoo! начала работу по созданию объединенной платформы обмена сообщениями и потоковой обработки динамических данных под названием Pulsar. После четырех лет работы в 10 центрах данных, обрабатывающих миллиарды сообщений в день, компания Yahoo! решила открыть исходный код своей платформы обмена сообщениями под защитой лицензии Apache в 2016 г.

Впервые я встретился с Pulsar осенью 2017 г. Тогда я возглавлял группу поддержки профессиональных сервисов в Hortonworks, сосредоточенной на работе с платформой потоковой обработки данных под названием Hortonworks Data Flow (HDF), в которую входили компоненты Apache NiFi, Kafka и Storm. В мои обязанности входил контроль за развертыванием этих технологий в инфраструктуре пользователя и помощь на начальной стадии разработки приложений потоковой обработки данных.

Самым большим затруднением, с которым мы столкнулись при работе с Kafka, была помощь нашим клиентам в правильном администрировании этого средства и специфическом определении необходимого количества разделов для конкретной темы, чтобы достичь надлежащего баланса скорости и эффективности при допущении дальнейшего роста объема данных. Тем из вас, кто знаком с Kafka, печально известен тот факт, что это кажущееся простым решение оказывает серьезное воздействие на масштабируемость тем, а для выполнения процедуры изменения этого значения (даже с 3 на 4) необходим весьма медленный процесс ребалансировки, на

протяжении которого все балансируемые темы становятся недоступными для чтения или записи.

Это требование ребалансировки неизменно вызывало негативную реакцию всех клиентов, использовавших HDF, и вполне справедливо, потому что они воспринимали его как препятствие для масштабирования кластера Kafka при непрерывном росте объемов данных. Только из собственного опыта клиенты узнавали, как трудно изменять в обе стороны масштаб платформы обмена сообщениями. Еще более худшим являлся тот факт, что невозможно было просто «прикрутить» еще несколько узлов для наращивания вычислительной мощности существующего клиентского кластера без соответствующего изменения конфигурации тем, чтобы использовать их для выделения большего количества разделов для существующих тем, чтобы перераспределить данные по только что добавленным узлам. Такая невозможность горизонтального масштабирования мощности потоковой обработки без ручного вмешательства (или необходимости написания огромного объема скриптов) приводила к прямому конфликту с желанием большинства наших клиентов переместить свои платформы обмена сообщениями в облако и воспользоваться всеми преимуществами гибких вычислительных возможностей, предоставляемых облачной средой.

Именно тогда я открыл для себя платформу Apache Pulsar и обнаружил, что ее заявление о том, что она «естественна для облака», особенно привлекательно, потому что она устраняет обе болевые точки масштабируемости. Хотя HDF позволял клиентам быстро приступить к работе, возникали трудности в управлении, да и сама платформа не была предназначена для функционирования в облаке. Я понял, что Apache Pulsar был намного лучшим решением, чем предлагаемое в тот момент нашим клиентам, и попытался убедить свою группу разработчиков рассмотреть возможность замены Kafka на Pulsar в нашем продукте HDF. Я даже зашел так далеко, что написал коннекторы, которые позволили Pulsar работать с компонентом Apache NiFi нашего стека, чтобы облегчить внедрение, но безрезультатно.

Когда в январе 2018 г. ко мне обратились первоначальные разработчики Apache Pulsar и предложили присоединиться к небольшому стартапу под названием Streamlio, я сразу же воспользовался возможностью поработать с ними. В то время Pulsar был пока еще молодым проектом, его только что включили в программу инкубации Apache, и следующие 15 месяцев мы провели, работая над тем, чтобы наш неоперившийся «птенчик» прошел через процесс инкубации и получил статус проекта высшего уровня.

Это было в самый разгар ажиотажа вокруг потоковой передачи данных, и Кафка был доминирующим игроком на этом поле, поэтому му естественно, что все считали эти термины взаимозаменяемыми.

По общему мнению, Kafka был единственной доступной платформой для потоковой передачи данных. На основе своего предыдущего опыта я взял на себя смелость неустанно проповедовать то, что знал как технологически превосходное решение, – одинокий голос вопиющего в пресловутой пустыне.

К весне 2019 г. количество участников и пользователей в сообществе Apache Pulsar существенно увеличилось, но надежной документации по этой технологии катастрофически не хватало. Поэтому, когда мне впервые предложили написать «Apache Pulsar в действии», я сразу же воспринял это как возможность удовлетворить острую потребность для сообщества Pulsar. Хотя мне так и не удалось убедить своих коллег присоединиться ко мне в этом начинании, они были бесценным источником рекомендаций и информации на протяжении всего процесса и использовали эту книгу как средство передачи вам части своих знаний.

Эта книга предназначена для абсолютных новичков в Pulsar и является сочетанием информации, собранной мною во время непосредственного сотрудничества с основателями Pulsar в процессе активной разработки этой платформы, и опыта, накопленного во время работы с организациями, включившими Apache Pulsar в производственный процесс.

Книга предназначена для того, чтобы помочь вам преодолеть препятствия и ловушки, с которыми другие столкнулись во время своих исследований Pulsar. Прежде всего эта книга придаст вам уверенности при разработке приложений потоковой обработки и микросервисов с использованием Pulsar и языка программирования Java. Несмотря на то что я решил использовать Java для большинства примеров кода в книге из-за личного знакомства с этим языком, я также создал аналогичный комплект исходного кода с использованием Python и загрузил его в свою учетную запись GitHub для тех, кто предпочитает писать код на этом языке.

Благодарности

По своей природе мы не можем возвращать блага тем, от кого мы их получаем, или делаем это редко. Но благо, которое мы получаем, должно быть воздано кому-то снова, строка за строкой, дело за делом, цент за центом.

— Ральф Уолдо Эмерсон (Ralph Waldo Emerson)

Я хочу воспользоваться предоставленной возможностью, чтобы поблагодарить всех, кто так или иначе внес свой вклад в создание этой книги, и признать тот факт, что я никогда не смог бы взяться за такой грандиозный проект без тех, кто помог заложить для него основы. В духе Эмерсона, пожалуйста, считайте эту книгу моим способом отплатить за все знания и поддержку, которую вы предоставили мне.

Было бы большим упущением, если бы я не начал этот список с самого первого человека, который познакомил меня с удивительным миром программирования в очень раннем возрасте шести лет: директора начальной школы, мистера Роджерса, который решил посадить меня перед компьютером, а не под арест за невнимательность во время урока математики в первом классе. Вы познакомили меня с чистой творческой радостью программирования и направили меня на путь обучения на протяжении всей жизни.

Я также хочу поблагодарить группу разработчиков Yahoo!, которая создала Pulsar: вы написали потрясающую программу и открыли ее исходный код сообществу, чтобы все мы могли ею наслаждаться. Без вас невозможно было бы написать эту книгу.

Хочу поблагодарить всех своих бывших коллег по Streamlio, особенно Джерри Пенга (Jerry Peng), Айвена Келли (Ivan Kelly), Маттео Мерли (Matteo Merli), Сиджи Гюо (Sijie Guo) и Санжива Кулкарни (Sanjeev Kulkarni), за участие в качестве членов комиссии по управлению проектами Apache PMC – Apache Pulsar, Apache BookKeeper или в обоих проектах. Без вашего руководства и выполнения обязанностей Pulsar не стал бы тем, чем он является сегодня. Также хочу поблагодарить бывшего гендиректора (CEO) Картика Рамасами (Karthik Ramasamy) за помощь в расширении сообщества Apache Pulsar во время совместной работы в Streamlio: я действительно ценю ваше наставничество.

Благодарю всех моих бывших коллег из Splunk за усилия по интеграции Apache Pulsar в столь крупную организацию и за помощь

в продвижении его внедрения внутри организации. После предоставления новой технологии вы сразу взялись за дело и сделали все возможное, чтобы ваши усилия увенчались успехом. Особую благодарность хочу выразить группе создания коннекторов, особенно Аламуси (Alamusi), Гими (Gimi), Алексу (Alex) и Спайку (Spike).

Я также хотел бы поблагодарить рецензентов, которые нашли время в своей весьма насыщенной жизни, чтобы прочитать мою рукопись на разных этапах ее разработки. Ваши положительные отзывы были приятным подтверждением того, что я на правильном пути, и улучшали настроение, когда процесс написания становился слишком утомительным. Ваши отрицательные отзывы всегда были конструктивными и формировали новую точку зрения на материал, которую может создать только свежий взгляд. Эта обратная связь была чрезвычайно ценна для меня и в итоге привела к тому, что книга стала намного лучше, чем она была бы без вашего участия. Спасибо всем вам: Алессандро Кампеис (Alessandro Campeis), Александер Шварц (Alexander Schwartz), Андрес Сакко (Andres Sacco), Энди Кеффалас (Andy Keffalas), Анджело Симоне Скотто (Angelo Simone Scotto), Крис Винер (Chris Viner), Эмануэле Пиччинелли (Emanuele Piccinelli), Эрик Платон (Eric Platon), Джампьеро Гранателла (Giampiero Granatella), Джанлука Ригетто (Gianluca Righetto), Джилберто Таккари (Gilberto Taccari), Хенри Сапутра (Henry Saputra), Игорь Савин (Igor Savin), Джейсон Рендел (Jason Rendel), Джереми Чен (Jeremy Chen), Кабир Ахмед (Kabeer Ahmed), Кент Спиллнер (Kent Spillner), Ричард Тобиас (Richard Tobias), Санкет Наик (Sanket Naik), Сатей Кумар Сах (Satej Kumar Sah), Симоне Сгуазца (Simone Sguazza) и Торстен Вебер (Thorsten Weber).

Спасибо всем онлайн-рецензентам за то, что нашли время, чтобы предоставить мне ценные отзывы через онлайн-форум издательства Manning, особенно Крису Латимеру (Chris Latimer) и его сверхъестественному умению находить все орфографические и грамматические ошибки, которые не смог обнаружить Microsoft Word. Все будущие читатели в долгу перед вами.

Наконец, что не менее важно, я хочу поблагодарить редакторов издательства Manning, особенно Карен Миллер (Karen Miller), Ивана Мартиновича (Ivan Martinović), Адриану Сабо (Adriana Sabo), Алена Куньо (Alain Couniot) и Нинослава Черкеза (Ninoslav Čerkez). Спасибо за сотрудничество и за терпение, когда дела шли плохо. Это был длительный процесс, и я бы не справился без вашей поддержки. Ваш вклад в обеспечение качества этой книги улучшил ее для всех читателей. Также благодарю всех остальных сотрудников Manning, которые работали со мной над изданием и продвижением книги. Это была действительно командная работа.

Об этой книге

Книга «Apache Pulsar в действии» была написана как введение в технологию потоковой обработки данных, чтобы помочь читателям познакомиться с терминологией, семантикой и особенностями, которые необходимо знать для восприятия парадигмы потоковой обработки при переходе от технологии пакетной обработки данных. Она начинается с исторического обзора развития систем передачи сообщений за последние 40 лет и показывает, как Pulsar оказался на вершине этого эволюционного цикла.

После краткого описания основной терминологии в сфере передачи сообщений и обсуждения двух основных паттернов потребления сообщений рассматривается архитектура Pulsar с физической точки зрения, где основное внимание уделяется его проектному решению, наиболее подходящему для облачной среды, а также с точки зрения его логического структурирования данных и поддержки мультиарендности (множественной аренды, многоарендной архитектуры).

В остальной части книги все внимание сосредоточено на том, как можно использовать встроенную вычислительную платформу Pulsar под названием Pulsar Functions для разработки приложений с применением простого API. Этот подход демонстрируется реализацией варианта использования в обработке заказов: на воображаемом предприятии по доставке продуктов питания создается приложение с применением микросервисов исключительно на основе Pulsar Functions с дополнительным развертыванием модели машинного обучения для оценки времени доставки.

Для кого предназначена эта книга

Книга «Apache Pulsar в действии» предназначена главным образом для разработчиков на языке Java, интересующихся технологией обработки потоковых данных, или для разработчиков микросервисов, которые ищут альтернативный фреймворк для порождения (генерации) событий. Группы DevOps, заинтересованные в развертывании и функционировании Pulsar в своих организациях, также найдут эту книгу полезной. Одним из основных критических замечаний по Apache Pulsar является общее отсутствие документации и сообщений в блогах, доступных в интернете, и, хотя без сомнений ожидается, что это изменится в ближайшем будущем, я наде-

уюсь, что книга поможет заполнить этот пробел в промежуточный период и принесет пользу всем, кто хочет узнать больше о потоковой обработке в целом и об Apache Pulsar в частности.

Как организована эта книга: дорожная карта

Книга состоит из 12 глав, разделенных на три части. Часть I начинается с введения в Apache Pulsar и определения его места в 40-летней истории развития систем передачи сообщений, а также сравнения Pulsar с разнообразными платформами обработки сообщений, существующими до него:

- в главе 1 представлена история систем передачи сообщений и определено место Pulsar в 40-летней истории развития технологий передачи сообщений. Также приведен краткий обзор некоторых архитектурных преимуществ Pulsar над другими системами и обоснование выбора Pulsar как единственной платформы обработки сообщений;
- в главе 2 подробно рассматривается многозвенная архитектура Pulsar, обеспечивающая независимое динамическое масштабирование хранилища данных или сервисных уровней. Здесь также описаны некоторые широко применяемые паттерны потребления сообщений, их отличия друг от друга и методы их поддержки в Pulsar;
- в главе 3 демонстрируется взаимодействие с Apache Pulsar из командной строки, а также с использованием его программного интерфейса (API). После изучения этой главы вы будете вполне уверенно запускать локальный экземпляр Apache Pulsar и взаимодействовать с ним.

В части II более глубоко рассматриваются основные варианты использования и функциональные возможности Pulsar, в том числе способы выполнения основных операций обработки сообщений и методы защиты кластера Pulsar, а также более продвинутые функциональные средства, например реестр схем. Кроме того, здесь представлен фреймворк Pulsar Functions с описанием того, как создавать, развертывать и тестировать функции:

- глава 4 представляет собственный потоковый вычислительный фреймворк Pulsar под названием Pulsar Functions с описанием некоторых основных принципов его проектирования и конфигурации, а также показывает, как разрабатывать, тестировать и развертывать функции;
- в главе 5 представлен фреймворк коннекторов Pulsar, предназначенный для перемещения (данных) между Apache Pulsar и внешними системами хранения, такими как реляционные базы данных, хранилища ключ–значение и хранилища blob-

объектов, например S3. Здесь вы узнаете, как разрабатывается коннектор – приводится пошаговое описание процесса разработки;

- глава 6 содержит подробные пошаговые инструкции по обеспечению безопасности кластера Pulsar для гарантии того, что ваши данные защищены как в режиме передачи, так и при хранении;
- в главе 7 рассматривается встроенный реестр схем Pulsar, обосновывается его необходимость, объясняется, как он может помочь в упрощении разработки микросервиса. Также описан процесс эволюции схемы и способ обновления схем, используемых внутри конкретного экземпляра Pulsar Functions.

В части III все внимание сосредоточено на использовании Pulsar Functions для реализации микросервисов и показано, как реализуются разнообразные паттерны проектирования широко применяемых микросервисов с помощью Pulsar Functions. В этой части демонстрируется процесс разработки приложения для предприятия по доставке пищи, чтобы сделать все примеры более реалистичными и применить более сложные варианты использования, включая обеспечение отказоустойчивости, доступ к данным и методику использования Pulsar Functions для развертывания моделей машинного обучения, которые могут работать с данными в реальном времени:

- в главе 8 показано, как реализовать часто применяемые паттерны маршрутизации сообщений, такие как разделение сообщений, маршрутизация на основе содержимого и фильтрация. Также демонстрируется реализация разнообразных паттернов преобразования сообщений, например извлечение значений и перевод сообщения;
- глава 9 подчеркивает важность механизма отказоустойчивости, встроенного в микросервисы, и демонстрирует его реализацию внутри экземпляра Pulsar Functions на основе Java с помощью библиотеки `resiliency4j`. Здесь рассматриваются различные события, которые могут произойти в программе, управляемой событиями, а также разнообразные паттерны, которые можно использовать для защиты сервисов от аварийных сценариев такого рода, чтобы максимизировать время работы приложения;
- в главе 10 рассматривается, как можно обеспечить доступ к данным из различных внешних систем при внутреннем использовании специально созданных функций Pulsar. Показаны разнообразные способы получения информации внутри микросервисов и условия, которые вы должны учитывать с точки зрения вероятных задержек;

- глава 11 представляет полный процесс развертывания разнообразных типов моделей машинного обучения внутри функции Pulsar с использованием различных фреймворков машинного обучения. Также рассматривается весьма важная тема: как передать необходимую информацию в модель, чтобы получить точный прогноз;
- в главе 12 описано использование Pulsar Functions в периферийной вычислительной среде для выполнения анализа в реальном времени данных интернета вещей (IoT). Глава начинается с подробного описания периферийной вычислительной среды и различных уровней ее архитектуры, и только после этого рассматривается, как применить Pulsar Functions для обработки информации на периферии, чтобы отправлять только краткие сводки, а не полный набор данных.

Завершают книгу два приложения, демонстрирующие более продвинутые рабочие сценарии, включающие развертывание в среде Kubernetes и георепликацию:

- в приложении А содержатся пошаговые инструкции, необходимые для развертывания Pulsar в среде Kubernetes с использованием схем Helm, предоставляемых как часть проекта с открытым исходным кодом. Также рассматривается изменение этих схем для соответствия среде, которую вы используете;
- в приложении В описан встроенный в Pulsar механизм георепликации и некоторые часто используемые паттерны репликации, применяемые в современном производстве. Далее подробно описан процесс реализации одного из таких паттернов георепликации в Pulsar.

Примеры исходного кода

Книга содержит множество примеров исходного кода как в форме пронумерованных листингов, так и в виде отдельной строки или нескольких строк в обычном тексте. В обоих случаях исходный код отформатирован с использованием такого шрифта постоянной ширины для отделения его от обычного текста. Иногда код также дополнительно выделяется **полужирным шрифтом**, чтобы специально выделить фрагмент кода, который изменился по сравнению с предыдущими этапами (примерами) в текущей главе, например при добавлении нового функционального средства (свойства) в существующую строку кода.

Во многих случаях изначальный исходный код был переформатирован: добавлены символы перехода на новую строку и изменено выравнивание для адаптации к доступному пространству на странице этой книги. В редких случаях, когда даже такие меры

оказались недостаточными, в листинги включены маркеры продолжения строки (➡). Кроме того, комментарии часто удалялись из исходного кода, если этот код подробно описывается в тексте. Многие листинги предваряются аннотациями к коду, которые особо выделяют наиболее важные концепции.

«Apache Pulsar в действии» – прежде всего книга по программированию, предназначенная для использования в качестве практического руководства для изучения того, как разрабатывать микросервисы с использованием Pulsar Functions. Поэтому я предоставил несколько репозиторий исходного кода, на которые я часто ссылаюсь на протяжении всей книги. Рекомендую загрузить код с соответствующей страницы веб-сайта издательства (<https://www.manning.com/books/apache-pulsar-in-action>) или из моей личной учетной записи на GitHub:

- этот репозиторий GitHub содержит код примеров из глав 3–6 и 8: <https://github.com/david-streamlio/pulsar-in-action>;
- код приложения микросервисов для предприятия по доставке пиццы можно найти в GitHub-репозитории: <https://github.com/david-streamlio/GottaEat>;
- код приложения для анализа интернета вещей (IoT), рассматриваемого в главе 12, размещен здесь: <https://github.com/david-streamlio/Pulsar-Edge-Analytics>;
- для тех, кому необходимы примеры на языке Python, предназначен следующий репозиторий: <https://github.com/david-streamlio/pulsar-in-action-python>.

Прочие онлайн-ресурсы

Нужна дополнительная помощь?

- Веб-сайт проекта Apache Pulsar <https://pulsar.apache.org> – неплохой источник информации о параметрах настройки конфигурации различных компонентов программного обеспечения Apache Pulsar, а также различных практических руководств по реализации конкретных функциональных возможностей этого программного обеспечения. На этом сайте вы всегда найдете самую свежую информацию.
- Канал Apache Pulsar Slack: apache-pulsar.slack.com – действующий форум, где встречаются члены сообщества Apache Pulsar со всего мира для обмена опытом, самыми эффективными практическими методиками и предоставления рекомендаций по устранению проблем для людей, которые впервые столкнулись с неприятными ситуациями в Pulsar. Это отличное место, где можно получить полезный совет, если вы попали в ловушку.

- В моем нынешнем качестве девелопер-адвоката я продолжаю разрабатывать дополнительный образовательный контент, в том числе публикации в блоге и примеры кода, которые вскоре будут доступны в режиме онлайн на веб-сайте моей компании streamnative.io.

Форум обсуждения *liveBook*

Приобретение книги «Apache Pulsar в действии» подразумевает свободный (бесплатный) доступ к веб-форуму издательства Manning Publications, где вы можете добавлять комментарии к этой книге, задавать технические вопросы и отвечать на них, а также получать помощь от автора и других пользователей. Чтобы получить доступ к форуму, перейдите по адресу <https://livebook.manning.com/#!/book/apache-pulsar-in-action/discussion>. О форумах издательства Manning и правилах их модерации можно узнать более подробно здесь: <https://livebook.manning.com/#!/discussion>.

Обязательство издательства Manning перед читателями состоит в том, чтобы предоставить место, где может состояться содержательный диалог между отдельными читателями, а также между читателями и автором. Это не является обязательством какой-либо конкретной степени участия со стороны автора, чей вклад в форум остается добровольным (и неоплачиваемым). Мы предлагаем попробовать задать автору несколько сложных вопросов, чтобы он не потерял интерес к форуму. Форум и архивы предыдущих обсуждений будут доступны на веб-сайте издателя, пока книга находится в печати.

Об авторе



Дэвид Хьеррумгор (David Kjerrumgaard) – участник проекта Apache Pulsar и сотрудник в должности Developer Advocate компании StreamNative, главной обязанностью которого является обучение разработчиков использованию Apache Pulsar. Ранее он был Global Practice Director в компании Hortonworks, где отвечал за разработку наиболее эффективных практических методик и решений для группы профессиональных сервисов, уделяя особое внимание потоковым технологиям, включая Kafka, NiFi и Storm. Дэвид имеет степень бакалавра и магистра по информатике и математике Кентского университета.

Об иллюстрации на обложке

На обложке книги «Apache Pulsar в действии» размещен рисунок «Cosaque», или «Казак», взятый из коллекции изображений одежды из различных стран Жака Грассе де Сен-Совёра (Jacques Grasset de Saint-Sauveur, 1757–1810) под названием «Costumes civils actuels de tous les peuples connus», опубликованной во Франции в 1788 г. Каждая иллюстрация тщательно прорисована и раскрашена от руки. Богатое разнообразие коллекции Грассе де Сен-Совёра живо напоминает нам о том, насколько далекими в культурном отношении были города и регионы мира всего 200 лет назад. Изолированные друг от друга люди говорили на разных диалектах и языках. На улицах или в сельской местности было легко определить, где они живут и какова их профессия или положение в обществе, просто по их одежде.

Наша одежда изменилась с тех пор, и разнообразие регионов, столь богатое в те далекие времена, исчезло. Сейчас трудно отличить жителей разных континентов, не говоря уже о разных городах, регионах или странах. Возможно, мы променяли культурное разнообразие на более разнообразную личную жизнь – и определенно на более разнообразную и динамичную технологическую среду.

В наше время, когда трудно отличить одну книгу по информатике от другой, издательство Manning воздаст должное изобретательности и инициативности компьютерного бизнеса, создавая обложки книг на основе богатого разнообразия региональной культуры двухвековой давности, оживленные иллюстрациями из коллекции Грассе де Сен-Совёра.

Часть I

Знакомство с Apache Pulsar

Корпоративные системы обмена сообщениями (enterprise messaging systems – EMS) предназначены для расширения слабо связанных архитектур, чтобы позволить географически распределенным системам передавать информацию друг другу посредством обмена сообщениями через простой API, поддерживающий две основные операции: публикацию сообщения и подписку на тему (для чтения сообщений). За свою более чем 40-летнюю историю корпоративные системы обмена сообщениями породили несколько важных стилей архитектуры распределенного программного обеспечения:

- программирование вызова удаленной процедуры (remote procedure call – RPC) с использованием таких технологий, как COBRA и Amazon Web Services, которые позволяют программам, написанным на различных языках, напрямую взаимодействовать друг с другом;

- ориентированное на сообщения программное обеспечение среднего звена (messaging-oriented middleware – MOM) для интеграции корпоративных приложений, примером которого является Apache Camel, позволяющее различным системам обмениваться информацией в общем формате сообщений, использующем XML или аналогичный самоописываемый формат;
- сервис-ориентированную архитектуру (service-oriented architecture – SOA), расширяющую стиль модульного программирования по контракту и позволяющую компоновать приложения из сервисов, которые объединяются особым способом для выполнения необходимой бизнес-логики;
- архитектуру, управляемую событиями (event-driven architecture – EDA), расширяющую возможности генерации, обнаружения и ответной реакции на отдельные изменения состояния, обозначаемые как события, с написанием кода, который обнаруживает и реагирует на эти отдельные события. Этот стиль в определенной степени был применен для удовлетворения потребности в обработке непрерывных потоков данных в масштабе интернета, например журнальных записей сервера и цифровых событий, таких как история (маршруты) посещений веб-сайтов.

Корпоративные системы обмена сообщениями (EMS) играют главную роль в каждом из этих стилей архитектуры, поскольку выступают в качестве внутренней технологии, позволяющей распределенным компонентам обмениваться информацией друг с другом, сохраняя промежуточные сообщения и своевременно распространяя их для всех назначенных потребителей. Главное различие между обменом данными через EMS и некоторыми другими механизмами передачи данных исключительно на основе сетевой среды заключается в том, что EMS изначально спроектирована для гарантированной доставки сообщений. Если событие опубликовано в EMS, то оно будет сохранено и передано всем назначенным получателям, в отличие от вызова интернет-микросервисов на основе протокола HTTP, который может потерять событие из-за сбоя сети.

Эти сохраняемые в EMS сообщения также зарекомендовали себя как ценные источники информации для организаций, которые можно анализировать, чтобы извлечь больше пользы для бизнеса. Представьте себе хранилище информации о поведении клиентов, которую предоставляет поток истории посещений ресурсов компании. Обработка этих типов источников данных называется потоковой обработкой, поскольку вы в буквальном смысле обрабатываете неограниченный поток данных. Именно поэтому существует большой интерес к обработке этих потоков с помощью аналитических инструментов, таких как Apache Flink или Spark.

В первой части книги представлен обзор развития корпоративных систем обмена сообщениями, где главное внимание сосредоточено на основных функциональных возможностях, добавленных на каждом этапе развития. Знание этих основ поможет вам лучше понять методы сравнения различных систем обмена сообщениями с учетом сильных и слабых сторон каждого поколения и функций, добавляемых в процессе развития. В итоге, как я надеюсь, вы поймете, почему Apache Pulsar является еще одним шагом вперед в генеалогии EMS и заслуживает особого внимания как весьма важный компонент инфраструктуры вашей компании.

В главе 1 представлено краткое введение в Apache Pulsar и его место в 40-летней истории развития систем обмена сообщениями, а также сравнение с различными более ранними платформами обмена сообщениями. В главе 2 более подробно рассматриваются особенности физической архитектуры Pulsar и возможности многозвенной архитектуры, позволяющей масштабировать уровни хранения и обработки данных независимо друг от друга. Здесь также описаны некоторые часто применяемые паттерны потребления сообщений, различия между ними и способы их поддержки в Pulsar. В главе 3 показано, как можно взаимодействовать с Apache Pulsar из командной строки и с использованием его API. После изучения этой главы вы будете вполне уверенно запускать локальный экземпляр Apache Pulsar и взаимодействовать с ним.

Введение в Apache Pulsar

Темы:

- история развития корпоративных систем обмена сообщениями;
- сравнение Apache Pulsar с существующими корпоративными системами обмена сообщениями;
- отличие сегмент-ориентированного хранилища Pulsar от ориентированной на разделы модели хранения, используемой в Apache Kafka;
- варианты использования из реальной практики, в которых Pulsar применяется для потоковой обработки, и объяснение, почему вы должны рассматривать использование Apache Pulsar.

После завершения разработки компанией Yahoo! в 2013 г. исходный код Pulsar впервые был открыт в 2016 г. и только через 15 месяцев после присоединения к программе инкубации Apache Software Foundation приобрел статус проекта высшего уровня. Apache Pulsar был спроектирован и разработан с нуля для устранения недостатков в существующих на тот момент систем обмена сообщениями с открытым исходным кодом, чтобы обеспечить ранее отсутствующую поддержку многоарендной архитектуры, георепликации и строгие гарантии надежности хранения данных.

Сайт Apache Pulsar определяет это программное обеспечение как распределенную систему обмена сообщениями по схеме публикация–

подписка (pub-sub), обеспечивающую минимальное время задержки между конечными пунктами и при публикации, гарантированную доставку сообщений, хранение данных без потерь и отсутствие специально выделенного сервера, а также предоставляющую упрощенную вычислительную рабочую среду (фреймворк) для обработки потоковых данных. Apache Pulsar предоставляет три главные функции для обработки больших наборов данных:

- обмен сообщениями в реальном времени – позволяет географически распределенным приложениям и системам передавать информацию друг другу в асинхронном режиме посредством обмена сообщениями. Основная цель Pulsar – предоставить эту возможность как можно более широкой группе клиентов с поддержкой многих языков программирования и двоичных протоколов обмена сообщениями;
- обработка (вычисления) в реальном времени – предоставляет возможность выполнять определенные пользователем вычислительные операции с сообщениями, хранящимися непосредственно внутри Pulsar, без необходимости использования внешней вычислительной системы для выполнения основных операций преобразования, таких как обогащение данных, фильтрация и агрегация;
- масштабируемая система хранения – независимый уровень хранения Pulsar и поддержка многоуровневого хранилища обеспечивает сохранение данных сообщения в течение неограниченного времени, т. е. так долго, как необходимо. Не существует никаких физических ограничений по объему данных, которые могут быть сохранены и доступны в Pulsar.

1.1. Корпоративные системы обмена сообщениями

Обмен сообщениями (messaging) – общий термин, используемый для описания передачи данных между производителями и потребителями. Следовательно, существует несколько различных технологий и протоколов, которые развивались в течение многих лет для обеспечения этой функции. Большинство людей хорошо знакомы с такими системами обмена сообщениями, как электронная почта (email), передача текстовых сообщений и приложения оперативной передачи сообщений, включая WhatsApp и Facebook Messenger. Системы обмена сообщениями из этой категории предназначены для передачи текстовых данных и изображений через интернет между двумя и более участниками. Более продвинутые системы оперативной передачи сообщений также поддерживают протокол Voice over IP (VoIP) и функции видеочата. Все эти систе-

мы изначально были предназначены для поддержки межличностного персонального обмена информацией по доступным в текущий момент каналам.

Другая категория систем обмена сообщениями, с которой люди уже знакомы, – потоковые сервисы видео по запросу, например Netflix или Hulu, которые отправляют потоковый видеоконтент одновременно многочисленным подписчикам. Такие сервисы видеопотока являются примерами однонаправленной широковещательной (одно сообщение многим потребителям) передачи данных потребителям, которые подписались на существующий канал, чтобы получать его контент. Хотя такие типы приложений приходят на ум при использовании терминов «системы обмена сообщениями» или «потоковая обработка», мы, с учетом темы этой книги, все же сосредоточимся на корпоративных системах обмена сообщениями.

Корпоративная система обмена сообщениями (enterprise messaging system – EMS) – это программное обеспечение, предоставляющее реализацию различных протоколов передачи сообщений, например DDS (data distribution service – сервис распространения данных), AMQP (advanced message queuing protocol – расширенный протокол организации очереди сообщений), MSMQ (Microsoft message queuing – протокол организации очереди сообщений Microsoft) и пр. Эти протоколы поддерживают отправку и прием сообщений между распределенными системами и приложениями в асинхронном режиме. Но асинхронный обмен данными был возможен не всегда, особенно на самом раннем этапе использования распределенных систем, когда преобладающими были архитектуры клиент–сервер и вызов удаленной процедуры (RPC). Первыми примерами применения RPC стали протокол SOAP (simple object access protocol) и REST (representational state transfer – передача репрезентативного состояния) на основе веб-сервисов, взаимодействующих друг с другом через фиксированные конечные точки. В обеих архитектурах, когда процесс требует взаимодействия с удаленным сервисом, необходимо сначала определить удаленную локацию этого сервиса через сетевой механизм службы обнаружения, а затем удаленно вызвать требуемый метод, используя корректные параметры и типы, как показано на рис. 1.1.

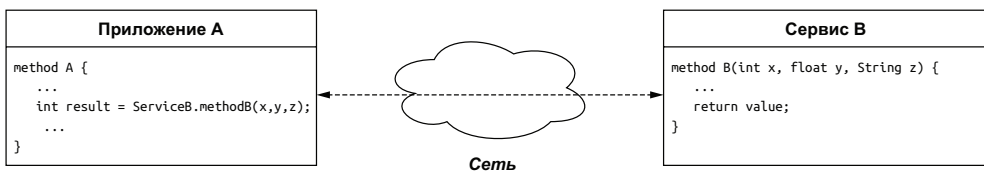


Рис. 1.1. В архитектуре RPC приложение вызывает процедуру сервиса, который работает на другом хосте, поэтому должно ждать возврата из этой процедуры, прежде чем можно будет продолжить обработку

Затем вызывающее приложение должно ждать возврата из вызванной процедуры, прежде чем можно будет продолжить обработку. Принцип синхронизации, присущий этим архитектурам, неизбежно замедлял работу приложений на их основе. Кроме того, существовала вероятность того, что удаленный сервис окажется недоступным в течение некоторого интервала времени, что требовало от разработчика приложения использования методик защитного (безопасного) программирования для определения такого условия и соответствующей реакции.

В отличие от коммуникационных каналов «точка-точка», используемых в программировании RPC, где необходимо ждать, когда вызванная процедура предоставит ответ, EMS позволяет удаленным приложениям и сервисам обмениваться информацией через промежуточный сервис, а не напрямую друг с другом. Вместо обязательного установления прямого сетевого коммуникационного канала между вызывающим и принимающим приложениями, через который происходит обмен параметрами, EMS можно использовать для сохранения этих параметров в форме сообщения, и они будут гарантированно доставлены назначенному получателю для обработки. Это позволяет вызывающей стороне отправлять запрос асинхронно и ждать ответа от сервиса, который она пытается вызвать. Это также позволяет сервису вернуть свой ответ в асинхронном режиме с публикацией результата в EMS с итоговой доставкой исходной вызывающей стороне. Такое разделение операций стимулирует разработку асинхронных приложений, предоставляя стандартизированный, надежный внутренний компонент коммуникационного канала, служащий постоянным буфером для обработки данных, даже если некоторые другие компоненты находятся в режиме офлайн, как можно видеть на рис. 1.2.

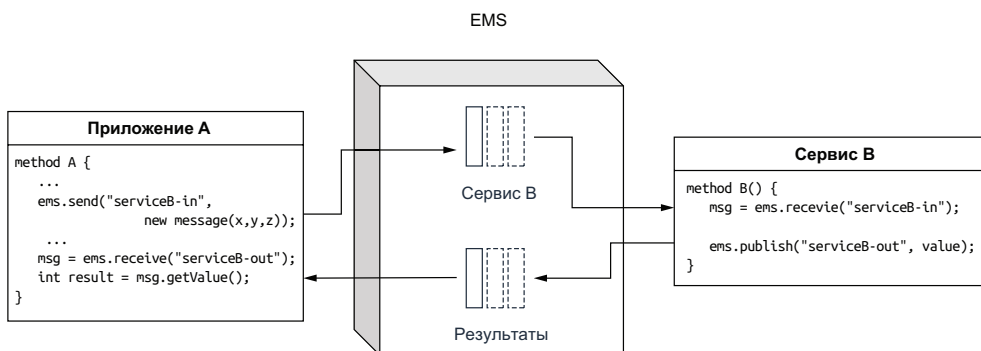


Рис. 1.2. EMS позволяет распределенным приложениям и сервисам обмениваться информацией в асинхронном режиме

EMS обеспечивает создание слабо связанных архитектур, предоставляя независимо разработанные программные компоненты, распределенные по различным системам для обмена данными друг

с другом с использованием структурированных сообщений. Такие схемы сообщений обычно определяются в форматах, независимых от языков программирования, например XML, JSON или Avro IDL, что позволяет разрабатывать компоненты на любом языке, поддерживающем эти форматы.

1.1.1. Основные функциональные возможности

После ознакомления с концепцией корпоративных систем обмена сообщениями и предварительного описания контекста типов задач, который эти системы должны использовать для решения, уточним определение EMS на основе предоставляемых ими функциональных возможностей.

Асинхронный обмен данными

Системы обмена сообщениями позволяют сервисам и приложениям обмениваться данными без блокировок, т. е. от отправителя и получателя сообщения не требуется одновременное взаимодействие с системой обмена сообщениями (или друг с другом). Система обмена сообщениями будет хранить сообщения до тех пор, пока все назначенные получатели не примут их.

Сохранение сообщений

В отличие от механизмов обмена сообщениями на сетевой основе, где сообщения существуют только в сетевой среде, как, например, RPC, сообщения, публикуемые в системе обмена сообщениями, сохраняются на диске до тех пор, пока они не будут доставлены. Недоставленные сообщения могут храниться в течение часов, дней и даже недель, и большинство систем обмена сообщениями позволяют пользователю определять стратегию хранения.

Подтверждение приема

От систем обмена сообщениями требуется сохранение сообщений до тех пор, пока все назначенные получатели не примут их, следовательно, необходим механизм, с помощью которого потребители сообщений могут подтвердить успешную доставку и обработку сообщений. Это позволяет системе обмена сообщениями удалять все успешно доставленные сообщения и повторять операцию доставки тем потребителям, которые пока еще не получили предназначенные для них сообщения.

Потребление сообщений

Очевидно, что система обмена сообщениями практически бесполезна, если не предоставляет механизм, с помощью которого назначенные получатели могут потреблять сообщения. Прежде всего и главным образом EMS обязана гарантировать, что все полученные ею сообщения непременно доставлены адресатам. Нередко сообще-

ния могут быть предназначены для нескольких (многих) потребителей, и EMS должна обеспечивать сохранность и целостность информации о том, какие сообщения уже были доставлены и кому.

1.2. Паттерны потребления сообщений

При использовании EMS вы получаете возможность публикации сообщений в теме или в очереди, но между этими двумя объектами существуют принципиальные различия. Тема (topic) поддерживает одновременное существование многочисленных потребителей одного сообщения. Любое сообщение, опубликованное в теме, автоматически передается (в широковещательном режиме) всем потребителям, подписавшимся (subscribed) на эту тему. Любое количество потребителей может подписаться на любую тему, чтобы получать predанную информацию, – точно так же любое количество пользователей может подписаться на Netflix и получать потоковый контент этого сервиса.

1.2.1. Обмен сообщениями по схеме публикация–подписка

При обмене сообщениями по схеме публикация–подписка производители публикуют сообщения в именованных каналах, называемых темами (topics). После этого потребители могут подписаться на существующие темы, чтобы получать входящие сообщения. Канал сообщений со схемой публикация–подписка (pub-sub) принимает входящие сообщения от нескольких производителей и сохраняет их в точном порядке получения. Но это отличается от очереди сообщений на стороне потребителя, потому что здесь обеспечена поддержка многих потребителей, принимающих каждое сообщение в теме через механизм подписки, как показано на рис. 1.3.

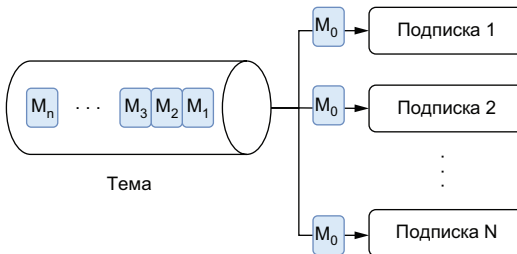


Рис. 1.3. При потреблении сообщений по схеме публикация–подписка каждое сообщение доставляется всем подписавшимся, установленным в этой теме. В данном случае сообщение M_0 было доставлено всем подписавшимся от 1 до N включительно

Системы обмена сообщениями по схеме публикация–подписка идеально подходит для вариантов использования, в которых требуется, чтобы потребители принимали каждое сообщение, или для вариантов, в которых порядок приема и обработки сообщений

чрезвычайно важен для сохранения корректного состояния системы. Рассмотрим вариант сервиса курса акций, который может использоваться большим количеством систем. Здесь важно не только то, чтобы сервисы принимали все сообщения, но не менее важно, чтобы изменения курса поступали в правильном порядке.

1.2.2. Очередь сообщений

С другой стороны, очереди (queues) обеспечивают семантику доставки сообщений по схеме «первым вошел, первым вышел» (first in, first out – FIFO) одному или нескольким конкурирующим потребителям, как показано на рис. 1.4. В очередях сообщения доставляются в порядке их получения, и только один потребитель получает и обрабатывает одно конкретное сообщение, а не все потребители – каждое сообщение. Эта схема хорошо подходит для организации очереди сообщений, представляющих события, активизирующие выполнение некоторой работы, например заявки, поступающие в центр обработки и выполнения заказов для регулирования. В этом сценарии необходимо, чтобы каждый заказ был обработан только один раз.

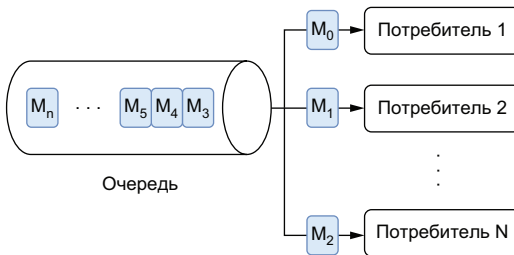


Рис. 1.4. При потреблении сообщений на основе очереди каждое сообщение доставляется исключительно одному потребителю. В данном случае сообщение M_0 было принято потребителем 1, M_1 – потребителем 2 и т. д.

Очереди сообщений могут с легкостью поддерживать более высокие скорости потребления, масштабируя количество потребителей в случае слишком большого числа сообщений, ожидающих обработки. Для уверенности в том, что сообщение обрабатывается ровно один раз, каждое сообщение обязательно должно быть удалено из очереди после успешной обработки и подтверждения от потребителя. Такая гарантия исключительно одноразовой обработки делает очереди сообщений идеальным средством для вариантов использования в очередях рабочих операций.

Если возникает критическая ошибка потребителя (т. е. отсутствие подтверждения приема в течение заданного интервала времени), сообщение будет повторно передано другому потребителю. В таком сценарии это сообщение, вероятнее всего, будет обработано вне существующего порядка. Таким образом, очереди сообщений подходят для тех вариантов использования, в которых чрезвычай-

но важно, чтобы каждое сообщение обрабатывалось один и только один раз, но порядок обработки сообщений не имеет значения.

1.3. История развития систем обмена сообщениями

После того как мы точно определили, из чего состоит EMS и какие основные функциональные возможности предоставляет эта система, я хотел бы привести краткий исторический обзор развития систем обмена сообщениями. Системы обмена сообщениями существовали в течение нескольких десятилетий и эффективно использовались во многих организациях, поэтому Apache Pulsar не является какой-то принципиально новой технологией, появившейся внезапно, скорее это очередной этап в развитии систем обмена сообщениями. Описывая исторический контекст, я надеюсь, что читатели смогут лучше понять суть сравнения Pulsar с существующими системами обмена сообщениями.

1.3.1. Системы обмена сообщениями общего назначения

Прежде чем перейти к специализированным системам обмена сообщениями, необходимо привести упрощенное представление систем обмена сообщениями, чтобы выделить внутренние компоненты, которые имеют все подобные системы. Определение этих основных функциональных средств сформирует основу для сравнения систем обмена сообщениями, существовавших в различное время.

На рис. 1.5 можно видеть, что любая система обмена сообщениями состоит из двух основных уровней, и каждый уровень имеет собственные специальные обязанности, которые мы рассмотрим позже. Мы будем изучать развитие систем обмена сообщениями с учетом этих двух уровней, чтобы правильно классифицировать и сравнивать различные системы, включая Apache Pulsar.

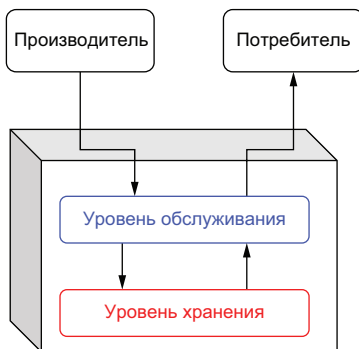


Рис. 1.5. Каждую систему обмена сообщениями можно разделить на два отдельных архитектурных уровня

Уровень обслуживания

Уровень обслуживания – это концептуальный уровень в EMS, который взаимодействует непосредственно с производителями и потребителями сообщений. Его главная задача – принимать входящие сообщения и направлять их в один или несколько пунктов назначения. Таким образом, этот уровень производит обмен данными через один или несколько поддерживаемых протоколов обмена сообщениями, например DDS, AMQP или MSMQ. Следовательно, уровень обслуживания в высшей степени зависит от пропускной способности сети для обмена данными и от ЦП для преобразования протоколов обмена сообщениями.

Уровень хранения

Уровень хранения – это концептуальный уровень в EMS, который отвечает за надежное сохранение и извлечение сообщений. Он взаимодействует непосредственно с уровнем обслуживания для предоставления запрошенных сообщений и обеспечивает правильный порядок сообщений. Следовательно, уровень хранения в высшей степени зависит от дисковой памяти для хранения сообщений.

1.3.2. Программное обеспечение среднего звена, ориентированное на сообщения

Первой категорией систем обмена сообщениями часто называют программное обеспечение среднего (связующего) звена, ориентированное на сообщения (message-oriented middleware – MOM), которое было предназначено для обеспечения обмена данными между процессами и интеграции приложений между распределенными системами, работающими в различных сетевых средах и операционных системах. Одной из наиболее известных реализаций MOM была IBM WebSphere MQ, внедренная в 1993 г.

Самые ранние реализации были спроектированы для развертывания на одном компьютере, который часто размещался глубоко внутри центра данных компании. Такое решение не только создавало единственную критическую точку отказа, но также означало, что масштабируемость системы ограничена возможностями физического аппаратного обеспечения хоста, поскольку этот единственный сервер отвечал за обработку всех клиентских запросов и хранение всех сообщений, как показано на рис. 1.6. Количество производителей и потребителей, одновременно работающих в таких MOM-системах с единственным сервером, было ограничено пропускной способностью сетевой карты, а объем хранимой информации – емкостью физического диска на сервере.

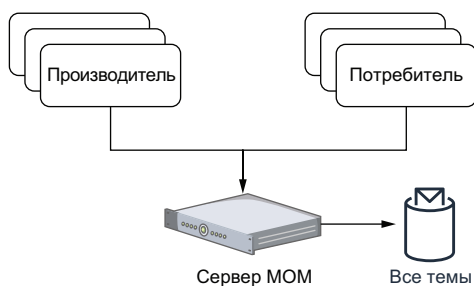


Рис. 1.6. Программное обеспечение среднего (связующего) звена, ориентированное на сообщения, было предназначено для размещения на одном сервере, следовательно, управляло всеми темами сообщений и обрабатывало запросы от всех клиентов

Справедливости ради следует отметить, что эти ограничения были характерны не только для системы IBM, но и для всех систем обмена сообщениями, предназначенных для размещения на одном компьютере, в том числе для RabbitMQ, RocketMQ и многих других. В действительности подобные ограничения существовали не только для систем обмена сообщениями того времени, а распространялись на все типы корпоративного программного обеспечения, спроектированного для работы на одном хосте.

Кластеризация

В конце концов все эти проблемы масштабирования были решены при помощи добавления функций кластеризации в MOM-системы с единственным сервером. Это позволило нескольким экземплярам одиночных сервисов совместно выполнять обработку сообщений и обеспечивать некоторую балансировку загрузки, как показано на рис. 1.7. Но даже несмотря на то, что MOM-системы были кластеризованы, в действительности это всего лишь означало, что каждый экземпляр одиночного сервиса отвечал за обслуживание и хранение сообщений из некоторого подмножества всех тем. В то время аналогичный подход под названием «сегментирование» (sharding) применялся для реляционных баз данных для устранения той же проблемы масштабирования.

Когда в темах возникали «очаги напряженности», незадачливый сервер, которому досталась такая конкретная тема, мог надолго стать узким местом или даже покинуть пул общей емкости хранения. Если на любом сервере кластера возникал критический сбой, то все обслуживаемые им темы становились недоступными. Хотя в такой ситуации сводилось к минимуму воздействие критической ошибки на весь кластер в целом (т. е. кластер продолжал работать), это была единственная критическая точка отказа для всех тем/очередей, которые обслуживал данный сервер.

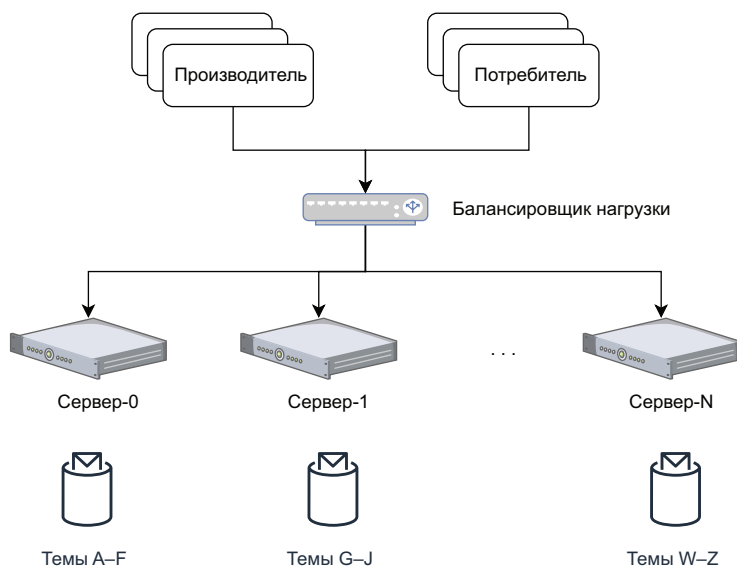


Рис. 1.7. Кластеризация позволяла распределить нагрузку по нескольким серверам. Каждый сервер в кластере отвечал за обработку только части тем

Такое ограничение требовало от организаций выполнения тщательного мониторинга циркулирования сообщений и регулирования распределения тем по аппаратным средствам, чтобы обеспечить равномерное распределение нагрузки в кластере. Но даже при этом оставалась возможность возникновения проблем в отдельной теме. Рассмотрим сценарий работы для крупной финансовой организации, в которой необходима отдельная тема для хранения всей текущей информации по конкретному пакету акций и предоставление этой информации всем настольным приложениям для операций с ценными бумагами внутри этой организации. Огромное количество потребителей и большой объем данных может быстро создать перегрузку единственного сервера, выделенного для обслуживания только этой темы. В таком сценарии необходима возможность распределения нагрузки одной темы по нескольким компьютерам, и, как мы вскоре увидим, именно этим и занимаются распределенные системы обмена сообщениями.

1.3.3. Сервисная шина предприятия

Сервисные шины предприятий (enterprise service buses – ESB) появились в самом начале текущего столетия, когда XML стал наиболее предпочтительным форматом сообщений, применяемым для реализации сервис-ориентированной архитектуры (SOA) приложений, использующих веб-сервисы на основе протокола SOAP. Главной концепцией ESB была шина сообщений (message bus), показанная на рис. 1.8. Шина сообщений служила коммуникацион-

ным каналом между всеми приложениями и сервисами. Такая централизованная архитектура стала прямой противоположностью соединения по схеме точка–точка, ранее используемой другим программным обеспечением промежуточного звена, ориентированным на сообщения.

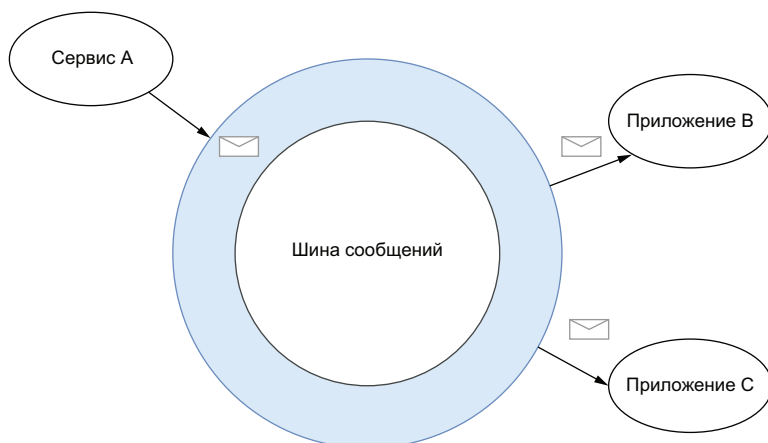


Рис. 1.8. Основная концепция ESB – использование шины сообщений для исключения необходимости обмена данными по схеме точка–точка. Сервис А просто публикует сообщение в шине, и оно автоматически передается в приложения В и С

При использовании ESB каждое приложение или сервис посылает и принимает все сообщения через единый коммуникационный канал без необходимости определения конкретных имен тем, в которых происходит публикация и потребление сообщений. Каждое приложение должно зарегистрироваться в ESB и определить набор правил для идентификации интересующих его сообщений, а ESB обязана обрабатывать всю логику, необходимую для динамического перенаправления сообщений из шины в соответствии с этими правилами. Аналогично теперь от каждого сервиса не требуется предварительное знание назначенной цели (целей) отправки сообщений, сервис может просто публиковать сообщения в шине и позволить ей распределять их.

Рассмотрим сценарий, в котором имеется большой документ в формате XML, содержащий сотни отдельных строк с наименованиями товаров в одном заказе клиента, и необходимо направить только подмножество этих наименований в некоторый сервис на основании определенного критерия, тоже включенного в сообщения (например, условие выбора по категории товара или по отделу). ESB предоставляет возможность извлечения таких отдельных сообщений (на основе результатов работы XQuery) и направления их различным потребителям на основе содержимого самого сообщения.

В дополнение к возможностям динамической маршрутизации шины ESB также сделали первый эволюционный шаг на пути потоковой обработки (stream processing), особо выделяя возможности обработки сообщений внутри самой системы вместо передачи выполнения этой задачи приложениям-потребителям. Большинство шин ESB предоставляло сервисы преобразования сообщений, часто через XSLT или XQuery, которые выполняли преобразование форматов сообщений в интервале между работой передающих и принимающих сервисов. ESB также обеспечивали обогащение данных сообщений и возможности обработки в самой системе обмена сообщениями. До этого обработка выполнялась приложениями, принимающими сообщения. Это был принципиально новый подход к организации систем обмена сообщениями, которые ранее применялись почти исключительно как транспортный механизм.

Можно утверждать, что ESB была первой категорией EMS, представившей третий уровень базовой архитектуры систем обмена сообщениями, показанный на рис. 1.9. В действительности большинство современных ESB поддерживает более продвинутые вычислительные возможности, включая обработку хореографии для управления потоками бизнес-процессов, комплексную обработку событий для их корреляции и сопоставление образцов, а также реализации «из коробки» нескольких паттернов корпоративной интеграции.

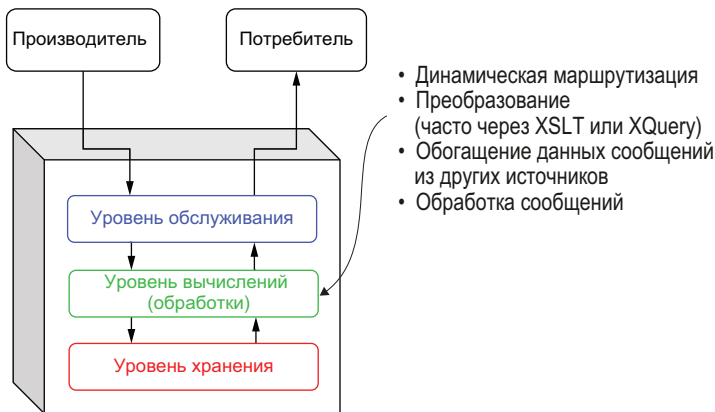


Рис. 1.9. В ESB особенно выделяется динамическая маршрутизация и средства обработки сообщений, представляющие первый случай добавления возможностей потоковой обработки в систему обмена сообщениями. При этом в базовую архитектуру систем обмена сообщениями был введен отдельный новый архитектурный уровень

Шины ESB внесли еще один важный вклад в развитие систем обмена сообщениями: особое внимание было уделено интеграции с внешними системами, благодаря чему системы обмена сообщениями сразу же обеспечили поддержку широкого спектра протоколов, не связан-

ных с передачей сообщений. ESB продолжали полностью поддерживать AMQP и другие протоколы обмена сообщений по схеме pub-sub, но главной отличительной чертой ESB была способность перемещать данные в/из систем, не ориентированных на обмен сообщениями, таких как электронная почта, базы данных и прочие сторонние системы. Для выполнения этих задач ESB предоставляли комплекты разработки ПО (SDK), позволяющие разработчикам реализовать собственные адаптеры для интеграции с произвольно выбираемыми системами.

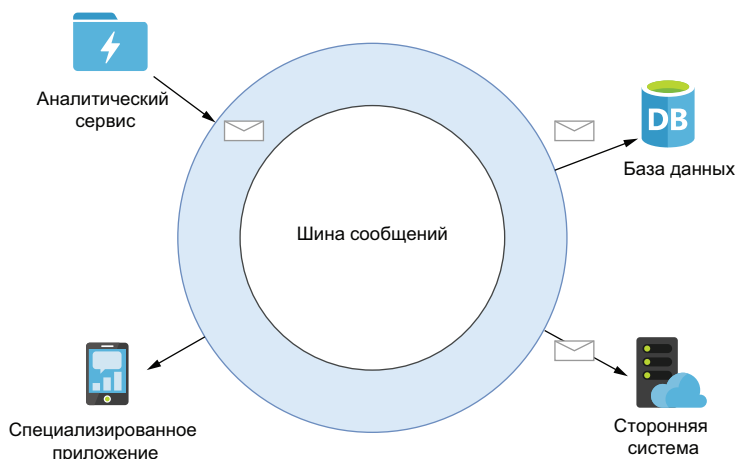


Рис. 1.10. ESB поддерживали интеграцию систем, не связанных с обменом сообщениями, в шину сообщений, тем самым расширяя возможности обмена сообщениями между приложениями предприятия и сторонними приложениями, например базами данных

На рис. 1.10 можно видеть, что такой подход обеспечивал более оперативный обмен данными между системами, упрощая интеграцию разнообразных систем. В этой роли ESB выступала и как инфраструктура передачи сообщений, и как посредник между системами, обеспечивающий преобразование протоколов.

Хотя шины ESB с помощью своих инноваций и расширенных функциональных возможностей, несомненно, способствовали дальнейшему развитию корпоративных систем обмена сообщениями, все же они представляли собой централизованные системы, предназначенные для развертывания на одном хосте. Таким образом, для них были характерны те же самые проблемы масштабируемости, что и для их предшественников MOM.

1.3.4. Распределенные системы обмена сообщениями

Распределенную систему можно описать как группу компьютеров, совместно работающих для поддержки сервиса или функциональной возможности, например файловой системы, хранилища ключ-значение или базы данных, которые функционируют так, как если

бы они работали на одном компьютере для конечного пользователя. Иначе говоря, конечный пользователь ничего не знает о том, что предъявленный ему сервис предоставлен группой совместно работающих компьютеров. Распределенные системы обладают общим состоянием, работают в параллельном режиме и способны противостоять критическим сбоям аппаратуры без какого-либо воздействия на работоспособность всей системы в целом.

Когда начала широко применяться парадигма распределенных вычислений, продвигаемая вычислительным фреймворком Hadoop, ограничения, характерные для одного компьютера, были устранены. Это стало началом эры разработки новых систем, распределяющих обработку и хранение данных по нескольким компьютерам. Одним из самых больших преимуществ распределенных вычислений стала возможность горизонтального масштабирования системы посредством простого добавления новых компьютеров в систему. В отличие от предшественников, ограниченных возможностями физической аппаратуры единственного компьютера, новые разрабатываемые системы теперь могли с легкостью использовать ресурсы сотен компьютеров при относительно небольших накладных расходах.

Как можно видеть на рис. 1.11, системы обмена сообщениями, как и базы данных и вычислительные фреймворки, также приняли на вооружение парадигму распределенных вычислений. Новые системы обмена сообщениями, первой из которых стала Apache Kafka, а недавно и Apache Pulsar, применили модель распределенной обработки данных, чтобы обеспечить масштабируемость и производительность, требуемые современными предприятиями.

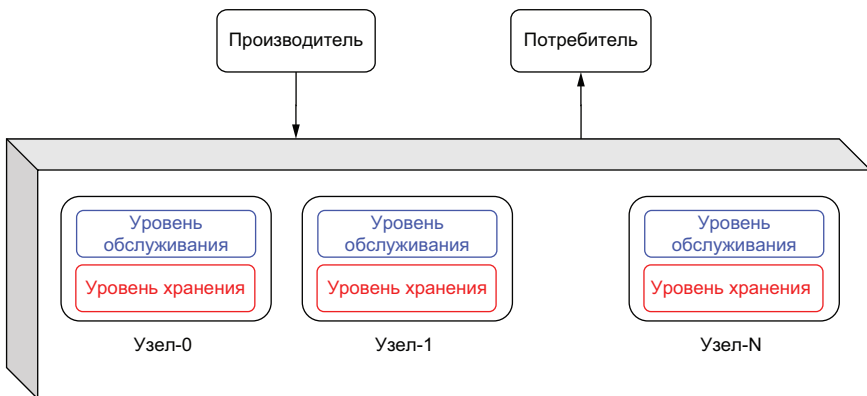


Рис. 1.11. В распределенной системе обмена сообщениями несколько узлов работают вместе и ведут себя как единая логическая система с точки зрения конечного пользователя. Внутри системы функции хранения данных и обработки сообщений распределены по всем узлам

В распределенной системе обмена сообщениями содержимое одной темы распределяется по нескольким компьютерам, чтобы обе-

спечить горизонтальное масштабирование хранения на уровне сообщений, что было невозможно сделать в предшествующих системах обмена сообщениями. Распределение данных по нескольким узлам в кластере также предоставляет некоторые преимущества, включая избыточность и высокую доступность данных, увеличенную емкость хранилища сообщений, ускорение передачи сообщений благодаря увеличению количества брокеров сообщений, а также исключение из системы единственной точки критического сбоя.

Главным архитектурным различием между распределенной системой обмена сообщениями и кластеризованной системой с одним узлом является способ проектирования и организации уровня хранения. В предшествующих системах с одним узлом все данные сообщений в любой конкретной теме хранились на одном компьютере, что позволяло быстро обслуживать данные на локальном диске. Но, как уже было отмечено выше, размер темы был ограничен емкостью локального диска на этом компьютере. В распределенной системе обмена сообщениями данные размещаются на нескольких компьютерах в кластере, что позволяет хранить сообщения в отдельной теме, размер которой превышает емкость устройства хранения на одном компьютере. Основной архитектурной абстракцией, которая делает возможным такое распределение данных, является журнал регистрации записей с упреждением (*write-ahead log*), интерпретирующий содержимое очереди сообщений как единую структуру данных, в которую можно только добавлять записи, т. е. сообщения.

С точки зрения логики на рис. 1.12 можно видеть, что при публикации нового сообщения в теме оно добавляется в конец журнала. Но с физической точки зрения это сообщение может быть записано на любом сервере в кластере.

Этот подход обеспечивает существование систем обмена сообщениями с гораздо более масштабируемой емкостью уровня хранения по сравнению с предыдущими поколениями таких систем. Еще одним преимуществом распределенной архитектуры является обслуживание сообщений в любой конкретной теме более чем одним брокером, что увеличивает эффективность (пропускную способность) производства и потребления сообщений посредством распределения нагрузки по нескольким компьютерам. Например, сообщения, публикуемые в теме, показанной на рис. 1.12, должны обрабатываться тремя отдельными серверами, и каждый записывает предназначенную для него часть на собственный диск. Результатом является более высокая скорость записи, поскольку нагрузка распределяется между несколькими дисками, а не направляется на единственный диск, как это было в предшествующем поколении систем обмена сообщениями. Существуют две различные методики, относящиеся к тому, как данные распределяются по узлам в кластере: хранилище на основе разделов и хранилище на основе сегментов.

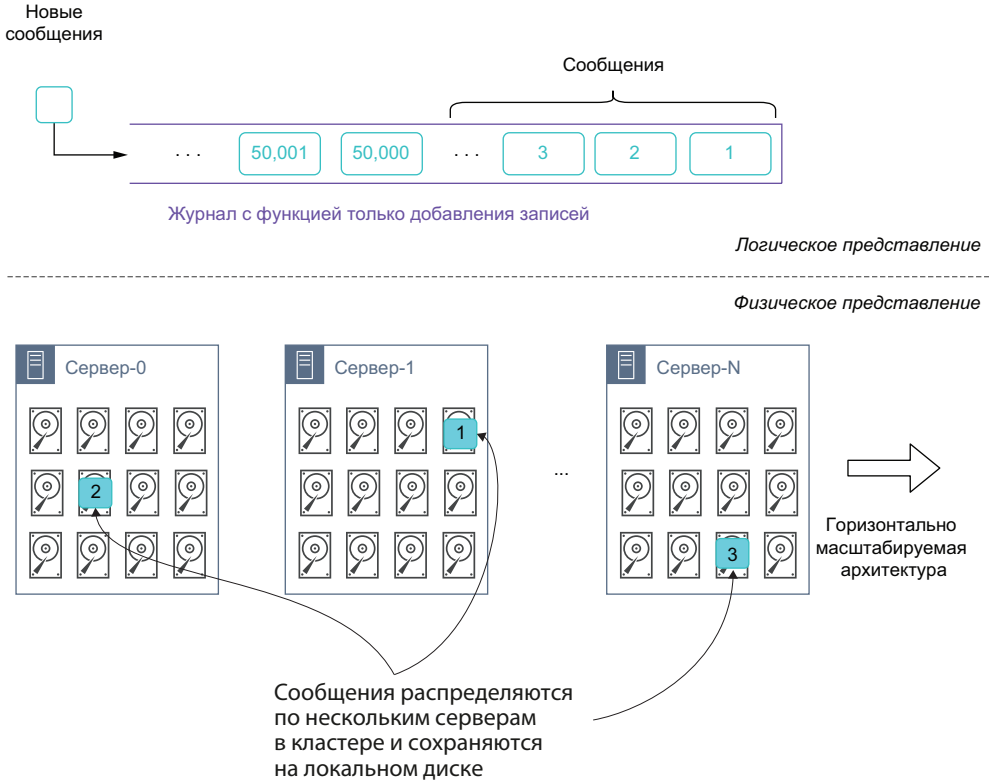


Рис. 1.12. Главная архитектурная концепция, заложенная в основу распределенных систем обмена сообщениями, – журнал, в который можно только добавлять записи (известный также как журнал регистрации записей с упреждением). С логической точки зрения все сообщения в теме хранятся в последовательном порядке, но в действительности сохраняются на нескольких серверах в распределенном режиме

Хранилище на основе разделов в Kafka

При использовании стратегии на основе разделов (partition-based) в системе обмена сообщениями тема разделяется на фиксированное количество групп, называемых разделами (partitions). Данные, публикуемые в этой теме, распределяются по разделам, как показано на рис. 1.13, при этом каждый раздел получает некоторую порцию сообщений, публикуемых в теме. Теперь общая емкость хранилища данных темы равна количеству разделов в теме, умноженному на размер каждого раздела. При достижении этого предельного значения в тему невозможно добавить новые данные. Простое добавление дополнительных брокеров в кластер не устранило эту проблему, так как необходимо также увеличить количество разделов в теме, а это непременно должно быть выполнено вручную. Кроме того, увеличение количества разделов требует еще и выполнения

ребалансирования, которое, как будет показано несколько позже, является дорогостоящим и длительным процессом.

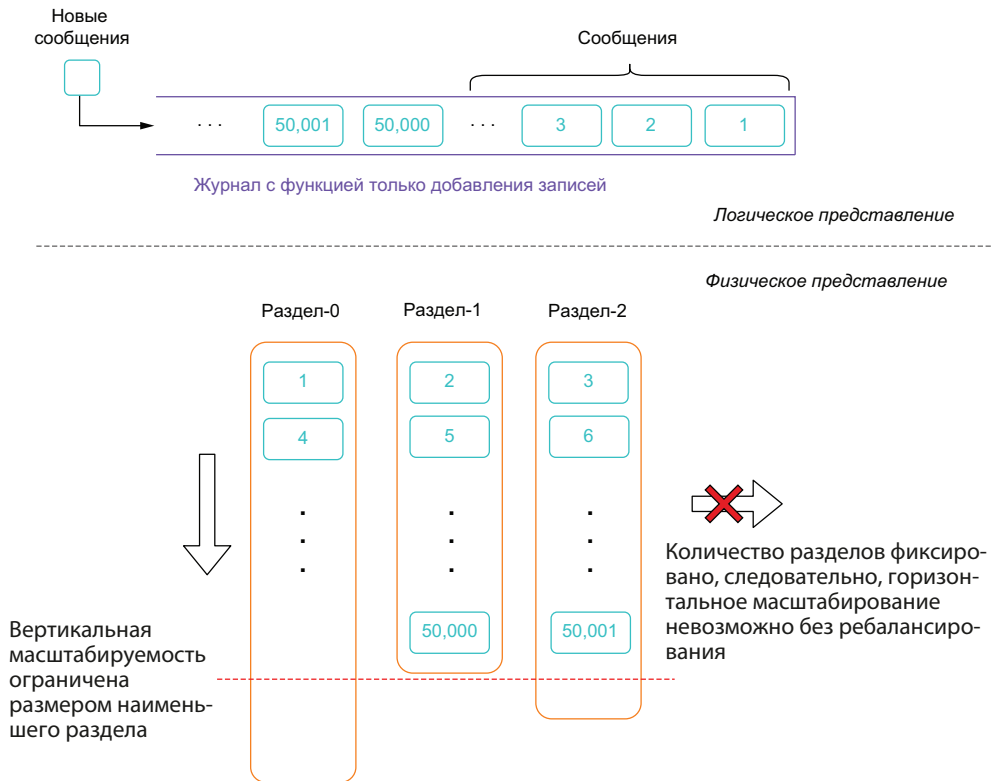


Рис. 1.13. Хранилище сообщений в системе обмена сообщениями на основе разделов

В системе на основе хранилища, ориентированного на разделы, количество разделов задается при создании темы, так как это позволяет системе определить, какие узлы будут отвечать за хранение каждого раздела и т. п. Но предварительное определение количества разделов создает несколько нежелательных побочных эффектов:

- один раздел может храниться только на одном узле в кластере, поэтому размер раздела ограничен объемом свободного дискового пространства на этом узле;
- поскольку данные равномерно распределяются по всем разделам, каждый раздел ограничен наименьшим размером раздела в теме. Например, если тема распределяется по трем узлам с 4 Тб, 2 Тб и 1 Тб свободного пространства на диске соответственно, то раздел на третьем узле может увеличивать свой размер только до 1 Тб, а это, в свою очередь, означает, что все разделы в теме также могут увеличиваться только до 1 Тб;

- хотя это и не является строгим требованием, для каждого раздела обычно несколько раз выполняется репликация на другие узлы для обеспечения избыточности данных. Следовательно, максимальный размер раздела в еще большей степени ограничивается размером наименьшей реплики (точной копии).

Если вы работаете в условиях одного из перечисленных выше ограничений емкости, то единственным средством является увеличение количества разделов в теме. Но процесс увеличения емкости требует ребалансирования всей темы в целом, как показано на рис. 1.14. Во время этого процесса существующие данные темы перераспределяются по всем разделам с учетом свободного дискового пространства на существующих узлах. Таким образом, при добавлении четвертого раздела в существующую тему после завершения процесса ребалансирования каждый раздел должен хранить приблизительно 25 % от общего объема сообщений.

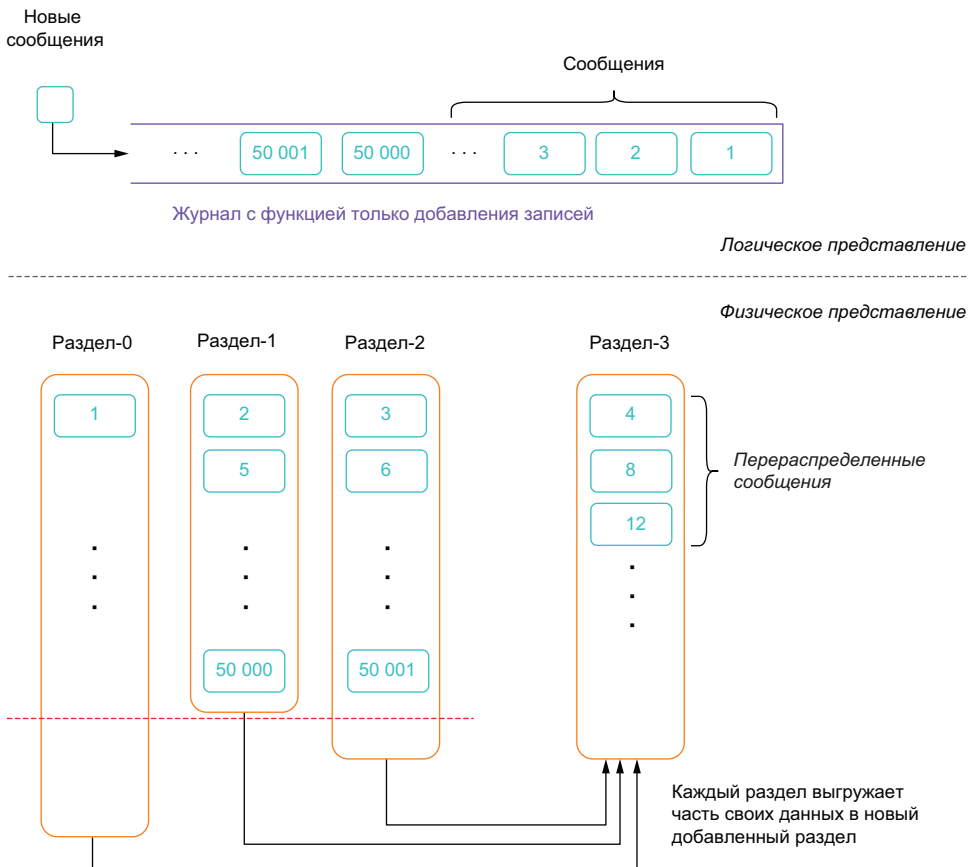


Рис. 1.14. Увеличение емкости хранилища для темы на основе разделов приводит к затратам на ребалансирование

Такое дополнительное копирование данных является дорогостоящей и подверженной ошибкам процедурой, так как потребляет ресурсы пропускной способности сети и дискового ввода-вывода прямо пропорционально размеру темы (например, ребалансирование темы размером в 10 Тб приводит к тому, что 10 Тб данных должны считываться с диска, передаваться по сети и записываться на диск в целевых брокерах). Только после полного завершения процесса ребалансирования можно удалить ранее существующие данные, и тема возобновляет обслуживание клиентов. Таким образом, рекомендуется разумно выбирать размеры разделов, поскольку стоимость ребалансировки невозможно просто не принять во внимание.

Чтобы обеспечить избыточность и обработку критического отказа для данных, можно сконфигурировать репликацию разделов на несколько узлов. Это гарантирует существование нескольких копий данных, доступных на диске, даже если на одном из узлов возникает критический сбой. По умолчанию установлены три реплики, т. е. система будет сохранять три копии каждого сообщения. Хотя это неплохой компромисс с точки зрения пространства для обеспечения избыточности, необходимо учитывать требование к объему такого дополнительного хранилища при определении размера кластера Kafka.

Хранилище на основе сегментов в Pulsar

Основой Pulsar являются проекты Apache BookKeeper для обеспечения постоянного места хранения для сообщений. Логическая модель хранения BookKeeper основана на концепции неограниченного потока записей-элементов, хранимых в форме последовательных журнальных записей. Как можно видеть на рис. 1.15, в BookKeeper каждый журнал (log) разделен на более мелкие фрагменты данных, называемых сегментами (segments), которые, в свою очередь, состоят из нескольких журнальных записей. Затем эти сегменты на уровне хранения записываются на некоторое количество узлов, называемых букмекерами (bookies), для обеспечения избыточности и масштабируемости.

На рис. 1.15 можно видеть, что сегменты могут быть размещены в любой локации уровня хранения, имеющей достаточную емкость диска. Если на уровне хранения емкость становится недостаточной для размещения новых сегментов, то можно легко добавить новые узлы и немедленно использовать их для хранения данных. Одним из главных преимуществ ориентированной на сегменты архитектуры хранения является истинная горизонтальная масштабируемость, поскольку сегменты можно создавать бесконечно и сохранять их в любой локации в отличие от хранилища на основе разделов с искусственными ограничениями вертикального и горизонтального масштабирования, зависящего от количества разделов.

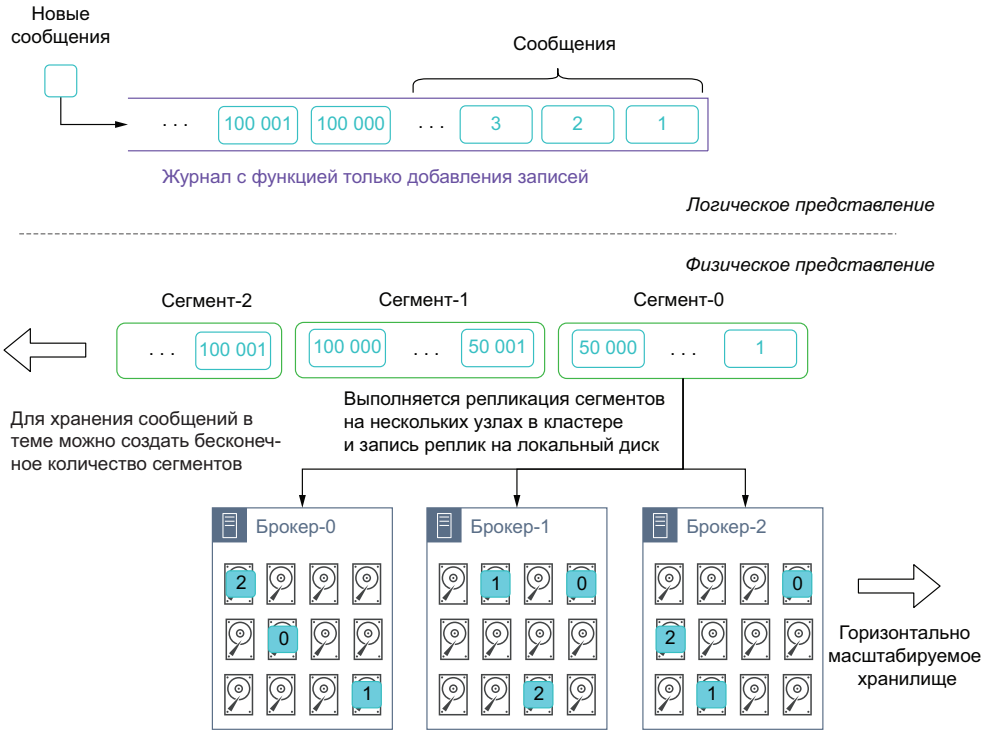


Рис. 1.15. Хранилище сообщений в ориентированной на сегменты системе обмена сообщениями реализовано посредством записи предварительно определенного количества сообщений в «сегмент» с последующим сохранением нескольких реплик этого сегмента на различных узлах на уровне хранения

1.4. Сравнение с Apache Kafka

Apache Kafka и Apache Pulsar – распределенные системы обмена сообщениями, основанные на аналогичных концепциях обработки сообщений. Клиенты взаимодействуют с обеими системами через темы, которые логически интерпретируются как неограниченные только пополняемые потоки данных. Но существует несколько принципиальных различий между Apache Kafka и Apache Pulsar, если рассматривать их с точки зрения масштабируемости, потребления сообщений, долговременности хранения данных и срока хранения сообщений.

1.4.1. Многоуровневая архитектура

Многоуровневая архитектура Apache Pulsar полностью отделяет уровень обслуживания (обработки) сообщений от уровня хранения, позволяя независимо масштабировать эти уровни. Более привычные технологии распределенной обработки сообщений, например Kafka, приняли подход, совмещающий обработку и хра-

нение данных на одних и тех же узлах или экземплярах кластера. Выбор такого проектного решения обеспечивает более простую инфраструктуру и некоторые преимущества в производительности благодаря сокращению объема передаваемых по сети данных, но за счет множества компромиссов, влияющих на масштабируемость, отказоустойчивость и на выполнение конкретных операций.

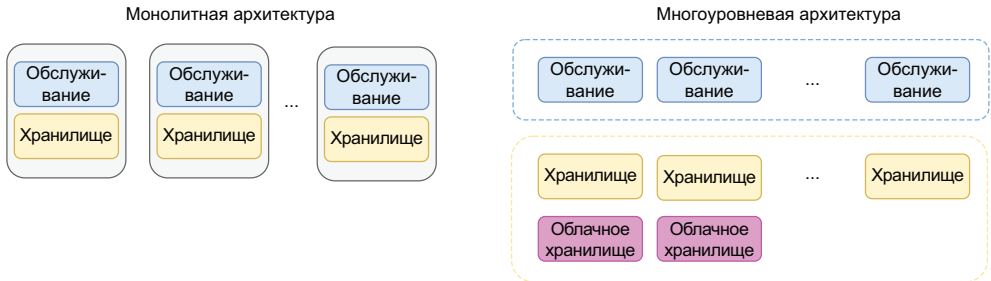


Рис. 1.16. Монолитные распределенные архитектуры совмещают уровни обслуживания (обработки) и хранения, тогда как Pulsar использует многоуровневую архитектуру, отделяющую уровни хранения и обслуживания друг от друга, что обеспечивает возможность их независимого масштабирования

В архитектуре Pulsar принят совершенно другой подход, который играет все большую роль в ряде облачных решений и отчасти стал возможен благодаря значительным улучшениям в пропускной способности сети, которые в наше время стали обычным явлением: принципу разделения вычислений (обработки) и хранения. Архитектура Pulsar разделяет обслуживание (обработку) и хранение данных на отдельные уровни: обслуживание данных выполняется узлами-брокерами (broker nodes) без сохранения состояния, а за хранение данных отвечают узлы-букмекеры (bookie nodes), как показано на рис. 1.16. Такое разделение обеспечивает несколько преимуществ, в том числе динамическую масштабируемость, обновления с нулевым временем простоя и неограниченное увеличение емкости хранилища, и это лишь некоторые из преимуществ. Кроме того, это проектное решение удобно для применения контейнеров, что делает Pulsar идеальной технологией для управления облачной потоковой системой.

Динамическое масштабирование

Рассмотрим вариант существования сервиса, интенсивно использующего ЦП, производительность которого начинает снижаться после превышения определенного порогового значения количества запросов. В таком сценарии необходимо горизонтальное масштабирование инфраструктуры, чтобы обеспечить добавление новых компьютеров и экземпляров приложения для распределения нагрузки, когда уровень использования ЦП выше 90 % на текущем

компьютере. Вместо применения средства мониторинга для оповещения группы DevOps о возникновении этого условия и выполнения процесса добавления вручную более предпочтительной является полная автоматизация такого процесса.

Автоматическое масштабирование – это общепринятая функция всех провайдеров открытых облачных сред, таких как AWS, Microsoft Azure, Google Cloud и Kubernetes. Она позволяет автоматически масштабировать инфраструктуру горизонтально на основе метрик использования ресурсов, например ЦП/памяти, без какого-либо вмешательства человека. Эта функциональная возможность не является исключительной для Pulsar и может применяться любыми другими платформами обработки сообщений для масштабирования в условиях сверхинтенсивного трафика, она наиболее полезна в многозвенной архитектуре, подобной Pulsar, по двум причинам, которые мы обсудим ниже.

В Pulsar брокеры без сохранения состояния на уровне обслуживания также обеспечивают возможность масштабирования инфраструктуры в сторону уменьшения после прохождения пика нагрузки, а это напрямую снижает стоимость использования общедоступной облачной среды. Другие системы обмена сообщениями с монолитной архитектурой не могут выполнять масштабирование с уменьшением узлов, потому что узлы содержат данные на своих жестких дисках. Удаление лишних узлов становится возможным только после полного завершения обработки всех данных или после перемещения данных на другой узел, остающийся в кластере. Такие задачи совсем непросто выполнить в автоматическом режиме.

Во-вторых, в монолитной архитектуре, например в Apache Kafka, брокер может обслуживать запросы только тех данных, которые хранятся на подключенном к нему диске. Эти ограничения делают бесполезным автоматическое масштабирование кластера в ответ на пиковое увеличение трафика, поскольку новые узлы, добавляемые в кластер Kafka, не содержат данные для обслуживания, следовательно, не способны обработать любые входящие запросы на чтение существующих данных из тем. Новые добавленные узлы могут обрабатывать только запросы на запись.

И последнее: в монолитной архитектуре, подобной Apache Kafka, горизонтальное масштабирование осуществляется при помощи добавления новых узлов, которые предоставляют ресурсы и для хранения, и для обслуживания вне зависимости от того, какие метрики вы отслеживаете и реагируете на их изменение. Таким образом, при масштабировании с увеличением емкости обслуживания в ответ на высокую интенсивность использования ЦП вы также увеличиваете емкость хранилища, в чем, возможно, нет необходимости, и наоборот.

Автоматическое восстановление данных

Перед вводом платформы обмена сообщениями в промышленную эксплуатацию необходимо понять, как будет выполняться восстановление из различных сценариев критических сбоев, начиная с простого случая сбоя единственного узла. В многозвенной архитектуре, подобной Pulsar, процесс восстановления чрезвычайно прост. Поскольку узлы-брокеры не сохраняют состояние, их можно заменить, запустив новый экземпляр сервиса вместо сбойного без нарушения обслуживания или выполнения любых других условий замены данных. На уровне хранения несколько реплик данных распределено по различным узлам, которые легко можно заменить новыми узлами в случае критического сбоя. В любом сценарии Pulsar может полагаться на механизмы провайдера облачной среды, такие как автоматическое масштабирование групп, для гарантии того, что минимальное количество узлов всегда находится в рабочем состоянии. Монолитные архитектуры, например Kafka, и в этом случае имеют недостаток из-за того факта, что новые добавляемые в кластер Kafka узлы не содержат никаких данных для обслуживания, следовательно, способны обрабатывать только входящие запросы на запись.

1.4.2. Потребление сообщений

Чтение сообщений из распределенных систем немного отличается от чтения из более старых систем обмена сообщениями, так как распределенные системы обмена сообщениями были предназначены для поддержки большого количества одновременно работающих потребителей. Способ, которым управляются потребляемые данные, в немалой степени зависит от способа их хранения в самой системе, поэтому системы на основе разделов и сегмент-ориентированные системы предоставляют собственные, отличающиеся от всех прочих методы поддержки семантики публикация–подписка (pub-sub) для потребителей.

Потребление сообщений в Kafka

В системе Kafka все потребители входят в так называемую группу потребителей (consumer group), формирующую единого логического подписчика (logical subscriber) на тему. Каждая группа состоит из множества экземпляров потребителей для обеспечения масштабируемости и устойчивости к критическим сбоям, так что если в одном экземпляре возникает критическая ошибка, то все остальные потребители сменяют его. По умолчанию новая группа потребителей создается, когда какое-либо приложение подписывается на тему Kafka. Приложение также может воспользоваться существующей группой потребителей, предъявив соответствующий идентификатор group.id.

В соответствии с документацией «способ потребления в Kafka реализован посредством распределения разделов в журнале по экземплярам потребителей так, чтобы каждый экземпляр являлся исключительным (эксклюзивным) потребителем “результата справедливого распределения ресурсов” разделов в любой момент времени» (<https://docs.confluent.io/5.5.5/kafka/introduction.html>). С точки зрения обычного человека, это означает, что каждый раздел в теме в любой момент времени может иметь только одного потребителя, а разделы равномерно распределяются по всем потребителям в данной группе. Как показано на рис. 1.17, если группа потребителей содержит количество членов, которое меньше числа разделов, то некоторым потребителям будет назначено несколько разделов, но если потребителей больше, чем разделов, то «лишние» потребители будут находиться в состоянии ожидания и захватывать владение ресурсом только в случае отказа (сбоя) одного из активных потребителей.

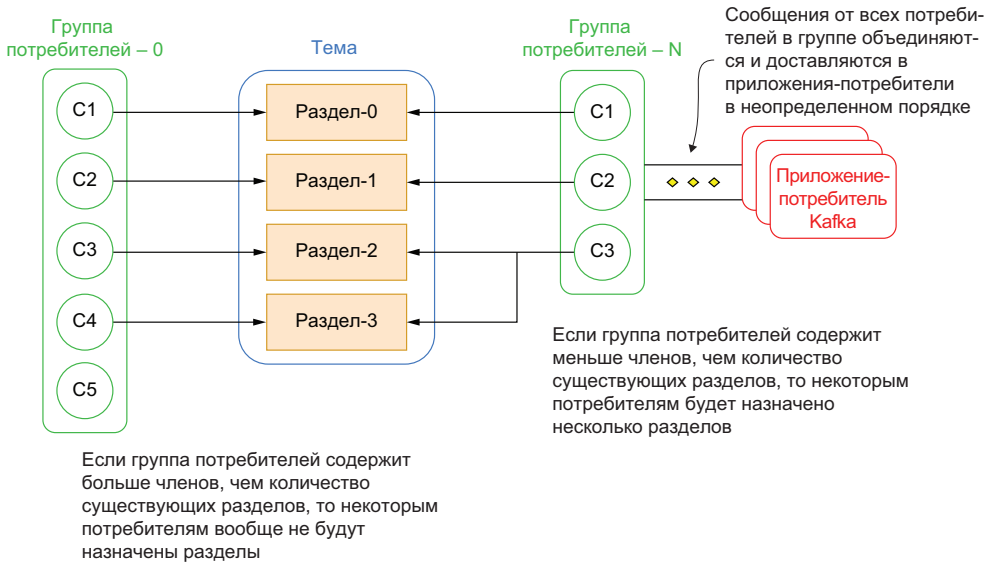


Рис. 1.17. Группы потребителей Kafka тесно связаны с концепцией разделов. Такой подход ограничивает количество одновременно работающих потребителей темы числом ее разделов

Важным побочным эффектом создания эксклюзивных потребителей является тот факт, что внутри группы количество активных потребителей никогда не может превышать число разделов в теме. Это ограничение может стать проблемой, так как единственным способом масштабирования потребления данных из темы Kafka является добавление потребителей в группу этой темы. Это фактически ограничивает степень параллелизма количеством разделов, что, в свою очередь, становится ограничением возможности масштабирования

потребления данных в том случае, когда потребители не могут удерживать равенство с числом производителей темы. К сожалению, единственным средством устранения этой проблемы является увеличение количества разделов темы, что, как уже было отмечено выше, представляет собой сложную, медленную и дорогостоящую операцию.

На рис. 1.17 также можно видеть, что сообщения от отдельных потребителей объединяются и отправляются обратно в клиент Kafka. Таким образом, порядок сообщений не поддерживается группой потребителей. Kafka обеспечивает только общий порядок записей в разделах, но не для различных разделов в теме.

Как было отмечено выше, группы потребителей работают как кластер для обеспечения масштабируемости и устойчивости к критическим сбоям. Это означает, что они динамически адаптируются к добавлению или удалению потребителей в группе. При добавлении нового потребителя в группу он начинает потреблять сообщения из разделов, ранее считываемых другим потребителем. А если потребитель завершает работу в нормальном режиме или из-за критической ошибки, то он покидает группу, и используемые им разделы будут переданы другому потребителю. Такое динамически изменяемое владение разделами в группе потребителей называется ребалансированием (rebalancing), которое может приводить к некоторым нежелательным последствиям, в том числе к потенциальной потере данных, если при перемещении потребителей данные не сохраняются перед началом ребалансирования.

Весьма часто многим приложениям необходимо считывать данные из одной темы. В действительности это одна из главных функций системы обмена сообщениями. Следовательно, темы являются ресурсами, совместно используемыми несколькими (многими) приложениями-потребителями, которые могут иметь совершенно различные потребности. Рассмотрим компанию, предоставляющую финансовые услуги, которая в реальном времени передает в потоковом режиме информацию о котировках фондового рынка в тему «stock quotes», при этом необходимо, чтобы эта информация совместно использовалась всеми подразделениями компании. Некоторые из внутренних приложений, весьма важных для бизнеса, таких как внутренние биржевые торговые площадки, системы алгоритмического трейдинга и предназначенные для клиентов веб-сайты, должны обрабатывать все данные этой темы немедленно после их поступления. Это потребует большого количества разделов для обеспечения необходимой пропускной способности в соответствии с жесткими требованиями SLA (Service Level Agreement – соглашения об уровне обслуживания).

С другой стороны, группа анализа данных может потребовать, чтобы информация из темы о котировках передавалась в некоторые модели машинного обучения для тренировки и проверки на

реальных данных о ценах на фондовом рынке. Для этого необходима обработка записей точно в том порядке, в котором они получены, т. е. нужен особый раздел темы для поддержания общей упорядоченности сообщений.

Группа бизнес-аналитиков разрабатывает отчеты с использованием KSQL¹, объединяющие данные темы о котировках с другими темами на основе конкретного ключа, например биржевого кода, дающего преимущество при наличии темы, разделенной по символу биржевого кода.

Эффективное предоставление данных темы о котировках для этих приложений с совершенно различными паттернами потребления является сложной и даже неразрешимой задачей с учетом зависимости от привязки групп потребителей к номеру раздела – это строго зафиксированное решение, которое весьма трудно изменить. Обычно в подобном сценарии единственная возможность – сопровождение нескольких копий данных в различных темах, сконфигурированных с соответствующим количеством разделов для каждого конкретного приложения.

1.4.3. Постоянство хранения данных

В контексте систем обмена сообщениями термин «постоянство хранения данных» (data durability) обозначает гарантии того, что сообщения, принятые и зарегистрированные в системе, будут сохранены даже в случае критического сбоя системы. В распределенной системе с многими узлами, такой как Pulsar или Kafka, критические ошибки могут возникать на многих уровнях, следовательно, важно понимать, как хранятся данные и какие гарантии постоянства их хранения предоставляет система.

Когда производитель публикует сообщение, из системы обмена сообщениями возвращается подтверждение того, что это сообщение было принято и размещено в соответствующей теме. Подтверждение сообщает производителю о том, что безопасность сообщения обеспечена и он может удалить его, не беспокоясь о том, что оно потеряется. Как мы вскоре увидим, такие гарантии более сильны в Pulsar, чем в Kafka.

Постоянство хранения данных в Kafka

Как уже было отмечено выше, в Apache Kafka принята методика хранения данных сообщений на основе разделов. Для гарантии постоянства хранения данных организована поддержка нескольких реплик (точных копий) разделов в кластере для обеспечения конфигурируемого уровня избыточности данных.

¹ Подмножество языка программирования K, поддерживающее синтаксис SQL. – Прим. перев.

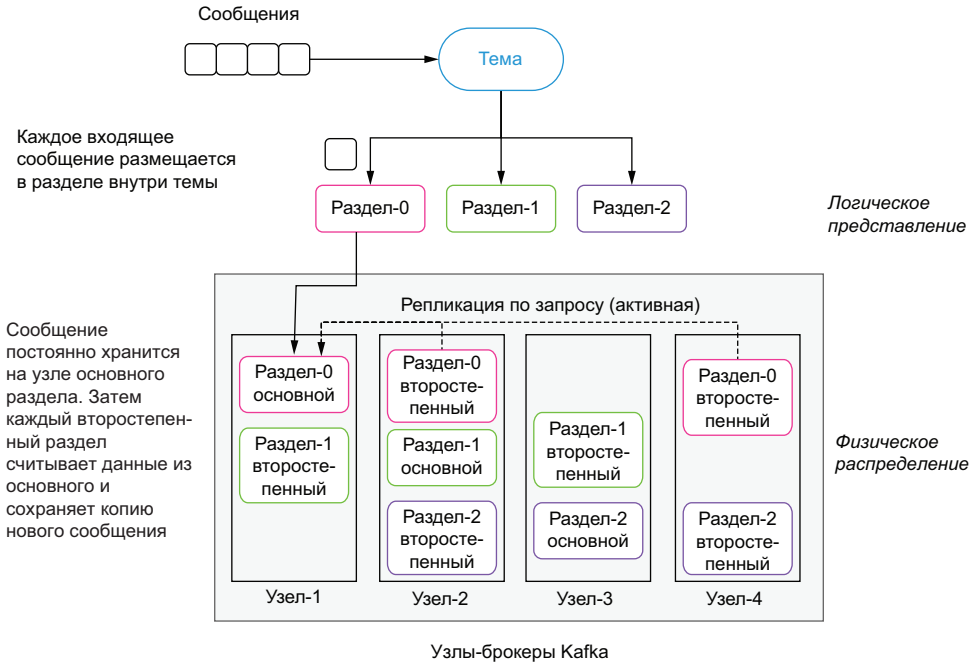


Рис. 1.18. Механизм репликации разделов в Kafka

Когда Kafka принимает входящее сообщение, к нему применяется функция хеширования, чтобы определить, в какой раздел темы должно быть записано новое сообщение. После определения содержимое сообщения записывается в страницу кеша (не дискового) раздела основной реплики (leader replica). После того как сообщение подтверждено основным разделом, каждая второстепенная реплика (follower replica) отвечает за извлечение содержимого сообщения из основного раздела в активной форме (т. е. они действуют как потребители и считывают сообщения из основного раздела), как показано на рис. 1.18. Такой обобщающий подход называют согласованной в конечном счете стратегией (eventually consistent strategy), согласно которой в распределенной системе существует один узел, содержащий самое последнее представление данных, который в итоге обменивается информацией с другими узлами до тех пор, пока на всех узлах не будет получено полностью согласованное представление данных. Хотя такой подход дает преимущество по времени, требуемому для сохранения входящего сообщения, он тем не менее создает две возможности для потери данных: во-первых, в случае внезапного завершения процесса на основном узле из-за сбоя электропитания или по какой-либо другой причине все данные, записанные в страницу кеша, но пока еще не сохраненные постоянно на диске, будут потеряны. Во-вторых, если в текущем основном процессе возникает критический сбой, то один из оставшихся второстепенных узлов выбирается как новый основной.

В таком сценарии восстановления после сбоя все сообщения, подтвержденные предыдущим основным узлом, но пока еще не скопированные в новую выбранную основную реплику, также будут потеряны.

По умолчанию сообщения подтверждаются после того, как основной узел записал их в память. Но это поведение можно заметить на задержку подтверждения до тех пор, пока все реплики не получат копию нового сообщения. Это не влияет на внутренний механизм репликации, в котором второстепенные узлы обязательно должны сами «вытягивать» информацию по сети и отправлять ответ обратно на основной узел. Очевидно, что такое поведение приводит к снижению производительности, которое часто не отмечается в большинстве публикуемых результатов эталонных тестов Kafka, поэтому рекомендуется выполнять собственное тестирование своей конфигурации, чтобы лучше понять, какой уровень производительности следует ожидать.

Еще один побочный эффект такой стратегии репликации – только основная реплика может обслуживать производителей и потребителей, так как только она гарантирует существование самой последней корректной копии данных. Все второстепенные реплики являются пассивными узлами, которые не способны немного разгрузить основной узел при пиковом трафике.

Постоянство хранения данных в Pulsar

Когда Pulsar принимает входящее сообщение, копия сохраняется в памяти, но данные также записываются в журнал с упреждающей записью (write-ahead log – WAL), который принудительно сохраняется на диске перед отправкой подтверждения публикатору сообщения, как показано на рис. 1.19. Такая методика создана по образцу семантики транзакций обычной базы данных – обеспечения атомарности, согласованности, изолированности и долговечности (ACID), гарантирующей, что данные не будут потеряны даже при аппаратном критическом сбое с возвратом в рабочее состояние через некоторое время.

Количество реплик, требуемых для темы, в Pulsar можно сконфигурировать на основе конкретных потребностей в репликации данных, и Pulsar гарантирует, что данные приняты и подтверждены кворумом серверов до отправки подтверждения производителю. Такое проектное решение гарантирует, что данные могут быть потеряны только при наступлении чрезвычайно маловероятного события, когда одновременно возникают критические ошибки на всех узлах-букмекерах, хранящих данные. Именно поэтому рекомендуется распределять узлы-букмекеры по нескольким (географическим) регионам и использовать стратегии устойчивого размещения (rack-aware placement), чтобы обеспечить хранение копии данных в нескольких регионах или центрах данных.

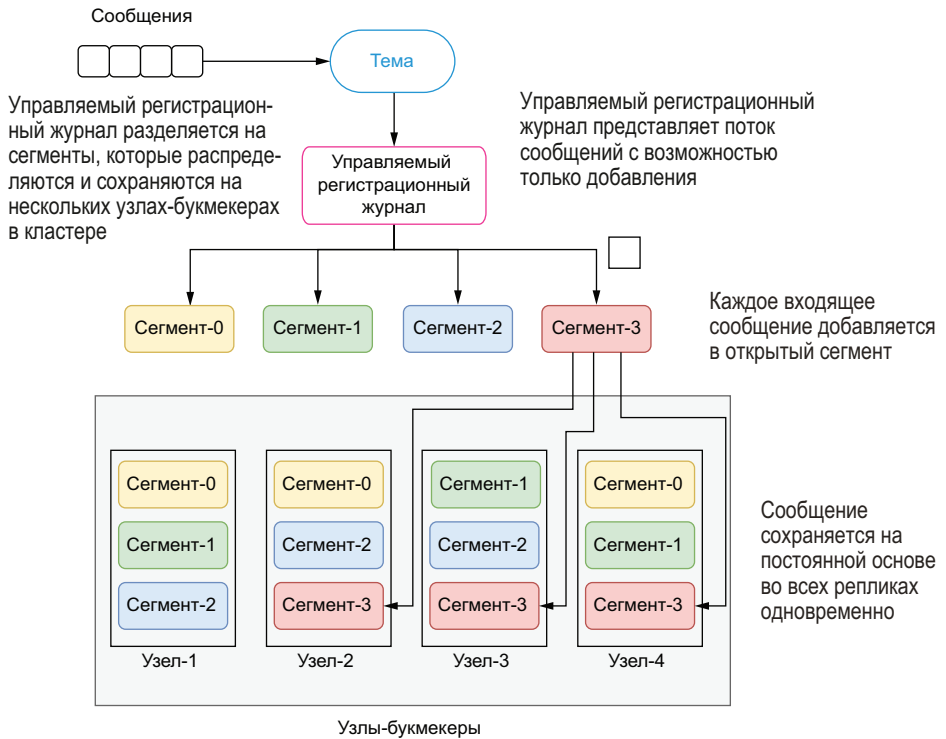


Рис. 1.19. В Pulsar репликация выполняется, когда сообщение впервые записывается в тему

Более важен тот факт, что такое проектное решение устраняет необходимость во второстепенном процессе репликации, который отвечает за синхронизацию данных между репликами, а также исключает все проблемы, связанные с несогласованностью данных, из-за любой задержки в процессе репликации.

1.4.4. Подтверждение приема сообщений

В распределенных системах обмена сообщениями критические сбои являются вполне ожидаемыми событиями. В распределенной системе, подобной Pulsar, критические ошибки могут возникать и в потребителях, читающих сообщения, и в брокерах, обслуживающих сообщения. При возникновении такого критического сбоя самое важное – возобновить потребление в точности с того момента, когда оно было остановлено, немедленно после восстановления, чтобы гарантировать, что сообщения не пропущены и не обработаны повторно. Эту точку, с которой должно возобновляться потребление, часто называют смещением (offset) темы. В Kafka и Pulsar приняты различные подходы к обработке таких смещений, напрямую влияющие на постоянство хранения данных.

Подтверждение приема сообщений в Kafka

Точка возобновления, обозначаемая как смещение потребителя (consumer offset) в Apache Kafka, полностью управляется самим потребителем. Обычно потребитель инкрементирует свое смещение последовательно по мере чтения записей из темы для подтверждения приема сообщения. Но хранение смещения исключительно в памяти потребителя представляет собой опасность. Поэтому значения смещений также сохраняются как сообщения в отдельной теме с названием `__consumer_offsets`. Каждый потребитель передает сообщение, содержащее его текущую позицию, в эту тему через установленные интервалы времени – через каждые 5 с, если вы используете функцию `auto-commit` в Kafka. Такая стратегия лучше, чем хранение смещений только в памяти, но и у этой методики периодических обновлений имеются определенные последствия.

Рассмотрим сценарий с единственным потребителем, где автоматические передачи сообщений совершаются каждые 5 с, а потребитель «умирает» ровно через 3 с после самой последней фиксации значения в теме смещения. В этом случае смещение, считываемое из соответствующей темы, имеет возраст 3 с, поэтому все события, прибывшие в этом трехсекундном интервале, будут обработаны дважды. Хотя имеется возможность сконфигурировать для интервала фиксации меньшее значение и уменьшить интервал, в котором записи будут дублироваться, полностью устранить эту проблему невозможно.

API потребителя Kafka предоставляет метод, позволяющий фиксировать текущее смещение в произвольный момент, что имеет смысл скорее для разработчика приложения, чем для таймера. Таким образом, если вы действительно хотите устранить дублирование обработки сообщений, можно воспользоваться этим API для фиксации смещения после каждого успешно потребленного сообщения. Но такой подход перемещает ответственность за обеспечение точного восстановления смещений на разработчика приложения и создает дополнительную задержку для потребителей сообщений, которые теперь вынуждены фиксировать каждое смещение в соответствующей теме Kafka и ждать подтверждения.

Подтверждение приема сообщений в Pulsar

Apache Pulsar поддерживает регистрационный журнал (ledger) в Apache BookKeeper для каждого подписчика, обозначенный как регистрационный журнал с маркером (cursor ledger) для отслеживания подтверждения приема сообщений. После того как потребитель прочитал и обработал сообщение, он отправляет подтверждение в брокер Pulsar. После приема подтверждения брокер немедленно обновляет регистрационный журнал с маркером для подписки этого потребителя. Поскольку информация хранится

в журнале регистрации в BookKeeper, нам известно, что она была записана на диск (операцией `fsync`) и существует несколько копий на узлах-букмекерах. Сохранение этой информации на диске гарантирует, что потребители не получают сообщение повторно, даже если они завершаются аварийно и возобновляют работу через некоторое время.

В Apache Pulsar существуют два способа подтверждения приема сообщений: выборочный (селективный) и накопительный (кумулятивный). При накопительном подтверждении потребитель должен подтвердить прием только самого последнего принятого сообщения. Все сообщения в разделе темы до идентификатора (ID) самого последнего включительно помечаются как подтвержденные и не будут повторно доставляться потребителю. Накопительное подтверждение так же эффективно, как обновление смещения в Apache Kafka.

Отличительной особенностью Apache Pulsar от Kafka является возможность подтверждения приема сообщений отдельными потребителями (т. е. выборочного (селективного) подтверждения). Эта возможность чрезвычайно важна для поддержки многих потребителей в теме, поскольку обеспечивает повторную доставку сообщений в том случае, когда в одном из потребителей возникает критический сбой.

Еще раз рассмотрим сценарий с критической ошибкой одного потребителя, когда каждый потребитель отдельно подтверждает прием после успешной обработки сообщения. Во время, предшествующее критической ошибке, потребитель пытался обработать некоторые сообщения, тогда как другие были успешно обработаны. На рис. 1.20 показан пример, в котором только два сообщения (4 и 7) были успешно обработаны и подтверждены.

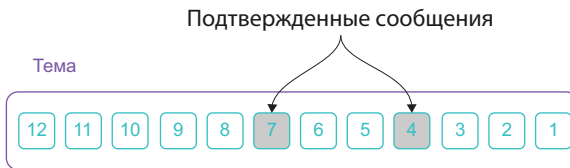


Рис. 1.20. Подтверждение приема отдельных сообщений в Pulsar

С учетом того факта, что концепция смещения Kafka интерпретирует смещения групп потребителей как высшую точку, обозначающую границу, до которой все сообщения считаются подтвержденными, в этом сценарии значение смещения должно было обновиться до семи, так как это наибольший идентификатор подтвержденного сообщения. Когда потребитель Kafka возобновляет работу в этой теме, он должен начать с сообщения 8 и продолжать по возрастанию ID, пропустив сообщения 1–3, 5 и 6, которые в итоге становятся потерянными, поскольку никогда не будут обработаны.

В том же сценарии в Pulsar с выборочными подтверждениями все неподтвержденные сообщения должны быть повторно доставлены, в том числе сообщения 1–3, 5 и 6, при возобновлении работы потребителя, что позволяет избежать потери сообщений из-за ограничений смещения потребителя.

1.4.5. Длительность хранения сообщений

В отличие от более старых систем, таких как ActiveMQ, сообщения не удаляются немедленно из распределенных систем обмена сообщениями после подтверждения их приема всеми потребителями. В старых системах такой подход был принят как способ восстановления емкости диска, когда это возможно, поскольку это ограниченный ресурс. Хотя распределенные системы обмена сообщениями, такие как Kafka и Pulsar, до определенной степени смягчили это ограничение, обеспечивая возможность горизонтального масштабирования хранилища сообщений, в обеих системах остается доступным механизм освобождения дискового пространства. Это важно для точного понимания того, как выполняется автоматизированное удаление сообщений в обеих системах, так как оно может привести к случайной потере данных при неправильной конфигурации.

Длительность хранения сообщений в Kafka

Kafka сохраняет все сообщения, опубликованные в теме, в течение конфигурируемого периода хранения. Например, если для стратегии хранения установлен период, равный семи дням, то в течение семи дней с момента публикации сообщения в теме оно доступно для потребления. По истечении периода хранения сообщение будет удалено для освобождения пространства. Удаление происходит независимо от того, было ли сообщение потреблено и подтверждено. Очевидно, что при этом возникает возможность потери данных, если период хранения меньше времени, необходимого всем потребителям на обработку сообщения, например из-за длительного выхода из строя системы потребления. Другим недостатком такого подхода на основе заданного интервала времени является высокая вероятность того, что сообщения будут храниться намного дольше, чем необходимо (т. е. после того, как они были обработаны всеми соответствующими потребителями). Это приводит к неэффективному использованию емкости хранилища.

Длительность хранения сообщений в Pulsar

В Pulsar после успешной обработки сообщения потребитель должен отправить подтверждение, чтобы соответствующий брокер мог удалить это сообщение. По умолчанию Pulsar немедленно удаляет все сообщения, которые были подтверждены всеми потребителями темы, и сохраняет все неподтвержденные сообщения

в журнале необработанных сообщений. В Pulsar сообщения могут быть удалены только после того, как все подписки полностью завершили их потребление. Pulsar также позволяет сохранять сообщения в течение более длительного времени даже после завершения их потребления всеми подписками, предоставляя возможность конфигурирования периода хранения сообщений. Эта возможность будет более подробно описана в главе 2.

1.5. Почему необходим именно Pulsar

Если вы только начинаете работать с приложениями, обеспечивающими обработку сообщений или потоковых данных, то определенно должны рассмотреть Apache Pulsar как главный компонент своей инфраструктуры обмена сообщениями. Но следует отметить, что существует несколько вариантов выбора технологий, многие из которых прочно укоренились в сообществе разработчиков программного обеспечения. В этом разделе я попытаюсь подробнее описать сценарии, в которых Apache Pulsar превосходит прочие платформы, а также устранить некоторое неправильное понимание функциональности существующих систем и указать конкретные сложности, встречающиеся при использовании этих систем.

В циклах внедрения часто возникают некоторые неправильные представления об укоренившейся технологии, которые сохраняются в сообществе пользователей по многим причинам. Зачастую трудно убедить себя и других в том, что необходимо заменить технологию, лежащую в основе используемой архитектуры. Только при ретроспективном взгляде становится очевидным тот факт, что привычные системы баз данных принципиально непригодны для масштабирования с целью удовлетворения требований, предъявляемых постоянно растущими объемами данных, поэтому необходимо переосмыслить способ хранения и обработки данных с помощью некоторого фреймворка, например Hadoop. Только после того, как мы перевели платформы бизнес-аналитики с обычных хранилищ данных на механизмы SQL на основе Hadoop, такие как Hive, Tez и Impala, стало понятно, что эти инструментальные средства имеют неадекватное время отклика для конечных пользователей, привыкших к получению ответа менее, чем через секунду. Это привело к быстрому внедрению Apache Spark в качестве наиболее предпочтительной технологии для обработки больших данных.

Я хотел выделить эти две новейшие технологии для напоминания о том, что мы не можем позволить нашей привязанности к существующему положению скрыть проблемы, существующие в основных архитектурных системах, а на передний план выдвинуть идею о том, что нам необходимо переосмыслить подход к использованию систем обмена сообщениями, поскольку действующие

технологии в этой сфере, такие как RabbitMQ и Kafka, страдают от принципиальных архитектурных недостатков. Группы разработки Apache Pulsar в компании Yahoo! могли бы просто выбрать вариант применения одного из существующих решений, но после тщательного исследования они решили этого не делать, поскольку требовалась платформа обмена сообщениями, предоставляющая недоступные в существующих монолитных технологиях возможности, которые будут рассматриваться в следующих подразделах.

1.5.1. Гарантированная доставка сообщений

Благодаря встроенного в платформу механизму долговременного хранения данных, который мы уже рассмотрели ранее, Pulsar обеспечивает гарантированную доставку сообщений в приложения. Если сообщение успешно поступает в брокер Pulsar, то оно будет доставлено всем потребителям соответствующей темы. Для обеспечения этой гарантии требуется долговременное хранение неподтвержденных сообщений до тех пор, пока они не будут доставлены и подтверждены потребителями. Такой режим весьма часто называют перманентным (устойчивым) обменом сообщениями. В Pulsar конфигурируемое количество копий всех сообщений сохраняется и синхронизируется на диске.

По умолчанию брокеры сообщений Pulsar гарантируют, что входящие сообщения постоянно хранятся на диске на уровне хранения до получения подтверждения приема каждого сообщения. Сообщения остаются на неограниченно масштабируемом уровне хранения Pulsar до подтверждения их приема, тем самым гарантируя доставку всех сообщений.

1.5.2. Неограниченная масштабируемость

Чтобы лучше понять механизм масштабирования Pulsar, рассмотрим обычный установленный экземпляр Pulsar. На рис. 1.21 можно видеть, что кластер Pulsar состоит из двух уровней: уровня обслуживания без сохранения состояния, сформированного из набора брокеров для обработки запросов клиентов, и уровня постоянного хранения с сохранением состояния, в состав которого входит набор букмекеров для обеспечения долговременного хранения сообщений.

Этот архитектурный паттерн, отделяющий хранилище сообщений от уровня обслуживания (обработки) сообщений, существенно отличается от более привычных систем обмена сообщениями, в которых на протяжении многих лет выбирался вариант совместного размещения этих двух сервисов. Подход с разделением обладает несколькими преимуществами, способствующими масштабируемости. Для начинающих отсутствие сохранения состояния брокеров позволяет динамически увеличивать или уменьшать их количество для соответствия требованиям клиентских приложений.

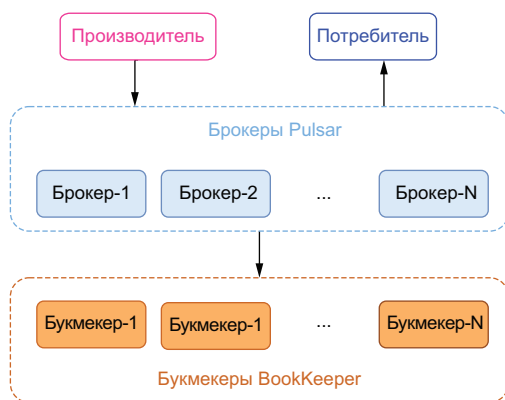


Рис. 1.21. Типовой кластер Pulsar

Беспрерывное расширение кластера

Все букмекеры, добавляемые на уровне хранения, автоматически обнаруживаются брокерами, которые немедленно начинают использовать их для хранения сообщений. Это отличие от Kafka, где требуется ребалансирование тем по разделам для распределения входящих сообщений по новым добавленным брокерам.

Неограниченное хранилище разделов тем

В отличие от Kafka емкость раздела темы не ограничена емкостью какого-либо наименьшего узла. Вместо этого разделы тем можно масштабировать с увеличением общей емкости уровня хранения, который и сам по себе может масштабироваться с помощью простого добавления новых букмекеров. Как уже было отмечено выше, разделы в Kafka имеют несколько ограничений по размеру, а в Pulsar такие ограничения отсутствуют.

Оперативное масштабирование без ребалансирования данных

Поскольку обслуживание и хранение сообщений разделено на два уровня, перемещение раздела темы из одного брокера в другой может происходить почти мгновенно и без какого-либо ребалансирования данных (множественного копирования данных с одного узла на другой). Такая характеристика чрезвычайно важна для многих функций, например для расширения кластера и для быстрой реакции на критические сбои брокеров и букмекеров.

1.5.3. Устойчивость к критическим ошибкам

Многоуровневая архитектура Pulsar также обеспечивает повышенную устойчивость, гарантируя отсутствие единой точки отказа в системе. При изолированном разделении уровней обслуживания и хранения Pulsar способен ограничить воздействие критического сбоя в системе, а процесс восстановления при этом происходит безболезненно.

Эффективный процесс восстановления брокера после критического сбоя

Брокеры формируют уровень обслуживания без сохранения состояния в Apache Pulsar. На уровне обслуживания сохранение состояния отсутствует, потому что в действительности брокеры не хранят локально какие-либо данные сообщений. Такой подход делает Pulsar устойчивым к критическим ошибкам брокеров. Если Pulsar обнаруживает, что брокер вышел из строя, то он может немедленно передать входящих производителей и потребителей другому брокеру. Поскольку данные хранятся на отдельном уровне, нет необходимости в копировании данных для перераспределения, как это делается в Kafka. Pulsar не выполняет дополнительное копирование данных, поэтому восстановление происходит практически мгновенно без снижения уровня доступности каких-либо данных в теме.

Напротив, Kafka направляет все запросы клиентов в основную реплику, поэтому именно она всегда будет содержать самые свежие данные. Основная реплика также отвечает за распространение входящих данных в наборе реплик, поэтому данные в итоге становятся доступными на этих узлах в случае возникновения критического сбоя. Но из-за неизбежной задержки между основной и дополнительными репликами данные могут быть потеряны до копирования.

Эффективный процесс восстановления букмекера после критического сбоя

Уровень постоянного хранения с сохранением состояния, используемый в Pulsar, состоит из букмекеров Apache BookKeeper для поддержки хранилища, ориентированного на сегменты, как уже было отмечено ранее. Когда сообщение публикуется в Pulsar, соответствующие данные сохраняются на диск на постоянной основе во всех N репликах, прежде чем сообщение будет подтверждено. Такое проектное решение гарантирует, что данные останутся доступными на нескольких узлах, следовательно, должно произойти еще $N-1$ критических сбоев в узлах, прежде чем данные будут окончательно потеряны.

Кроме того, уровень хранения Pulsar является самовосстанавливающимся, и если возникает критическая ошибка узла или диска, из-за которой конкретный сегмент содержит неполную реплику, то Apache BookKeeper автоматически определит этот дефект и запланирует восстановление реплики в фоновом режиме. Восстановление реплики в Apache BookKeeper является операцией быстрого восстановления типа «многие-ко-многим» (many-to-many) на уровне сегментов, намного более детализированной, чем простое копирование целого раздела темы, требуемого в Kafka.

1.5.4. Поддержка миллионов тем

Рассмотрим сценарий, в котором необходимо моделировать приложение для работы с некоторым объектом, например с потребителем, и для каждого из таких объектов требуется отдельная тема. Для этого объекта будут публиковаться разнообразные события; потребитель создается, размещается в определенном порядке, вносит оплату, возвращает некоторые элементы, изменяет свой адрес и т. д.

Размещая все эти события в одной теме, вы гарантируете их обработку в правильном хронологическом порядке и можете быстро проинспектировать тему, чтобы определить корректное состояние учетной записи потребителя и т. п. Но в процессе развития вашего бизнеса потребуются поддержка миллионов тем, а ранее существующие системы обмена сообщениями не способны выполнить это требование. Возникают крупные издержки, связанные с наличием множества тем, в том числе увеличение времени задержки между конечными пунктами, рост количества дескрипторов файлов, переполнение памяти и длительное время восстановления после критического сбоя.

В Kafka необходимо соблюдать практическое правило: общее количество разделов тем не должно превышать нескольких сотен, если для вас важна высокая производительность без задержек. Существует несколько руководств по реструктуризации приложений на основе Kafka, чтобы не достичь этого ограничения. Если такое ограничение платформы, влияющее на проектное решение приложения с учетом структуризации тем, нежелательно, то вам следует рассмотреть вариант с использованием Pulsar.

Pulsar предоставляет возможность поддержки до 2.8 млн тем при сохранении стабильного уровня производительности. Главная причина существования возможности такого масштабирования количества тем заключена в том, как организованы внутренние данные на уровне хранения. Если данные тем хранятся в специально выделенных файлах или каталогах, как в ранее существующих системах обмена сообщениями, подобных Kafka, то возможность масштабирования ограничена, потому что операции ввода-вывода будут разбросаны по диску при увеличении количества тем, что приведет к пробуксовке диска и существенному снижению производительности. Для предотвращения такого поведения сообщения из разных тем объединяются, сортируются и сохраняются в более крупных файлах, а затем индексируются в Apache Pulsar. Такой подход ограничивает быстрое размножение маленьких файлов, которое приводит к возникновению проблем с производительностью при увеличении количества тем.

1.5.5. Георепликация и активная отказоустойчивость

Apache Pulsar – система обмена сообщениями с поддержкой синхронной георепликации внутри одного кластера и асинхронной георепликации между несколькими кластерами. Система была развернута глобально в более чем 10 центрах данных компании Yahoo! с 2015 г. с полноценной сетью репликации 10×10 для особо важных сервисов, таких как Yahoo! Mail and Finance.

Георепликация (geo-replication) – это широко известная практическая методика, используемая для обеспечения функций восстановления после катастроф в корпоративных системах посредством распределения копий данных по различным географическим локациям. Такой подход гарантирует, что данные и основанные на них системы смогут противостоять любым непредвиденным катастрофам, например природным. Механизмы георепликации, применяемые в различных системах обработки данных, можно классифицировать как синхронные и асинхронные. Apache Pulsar позволяет с легкостью активизировать механизм асинхронной георепликации, используя всего лишь несколько параметров конфигурации.

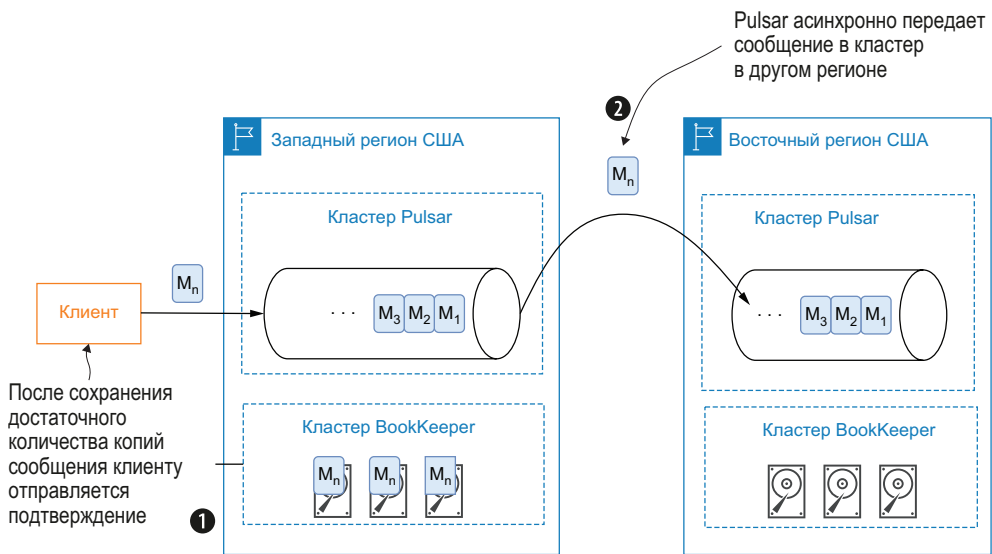


Рис. 1.22. При использовании асинхронной георепликации в Apache Pulsar сообщение сохраняется локально в кластере BookKeeper, работающем в том регионе, где принято это сообщение. Затем это сообщение в фоновом режиме асинхронно передается в кластер Pulsar в другом регионе

При асинхронной георепликации производитель не ждет подтверждения приема сообщения от других центров данных. Вместо этого клиент-производитель получает подтверждение сразу после того, как его сообщение успешно сохранено в локальном кластере BookKeeper. Затем выполняется репликация данных из кластера

источника в другие центры данных в асинхронном режиме, как показано на рис. 1.22.

Асинхронная георепликация обеспечивает более короткую задержку, так как клиент не должен ждать ответов из других центров данных. Но при этом становятся более слабыми гарантии целостности из-за асинхронного режима репликации. Поскольку при асинхронной репликации всегда существует некоторая задержка, некоторая часть данных остается пока еще не скопированной из источника в целевое хранилище.

Синхронная георепликация представляет собой несколько более сложную процедуру для практического применения в Apache Pulsar, чем асинхронная, так как требует некоторого изменения конфигурации вручную для полной гарантии того, что сообщение будет подтверждено только после того, как большинство центров данных представит уведомления о том, что данные этого сообщения успешно сохранены на диске на постоянной основе. Подробности о том, как конкретно можно активизировать механизм синхронной георепликации в Apache Pulsar, приведены в приложении В, но все же я могу отметить, что такая функция существует благодаря двухуровневой архитектуре Pulsar и возможности формирования кластера Apache BookKeeper из локальных и удаленных узлов, расположенных в разных географических регионах, как показано на рис. 1.23.

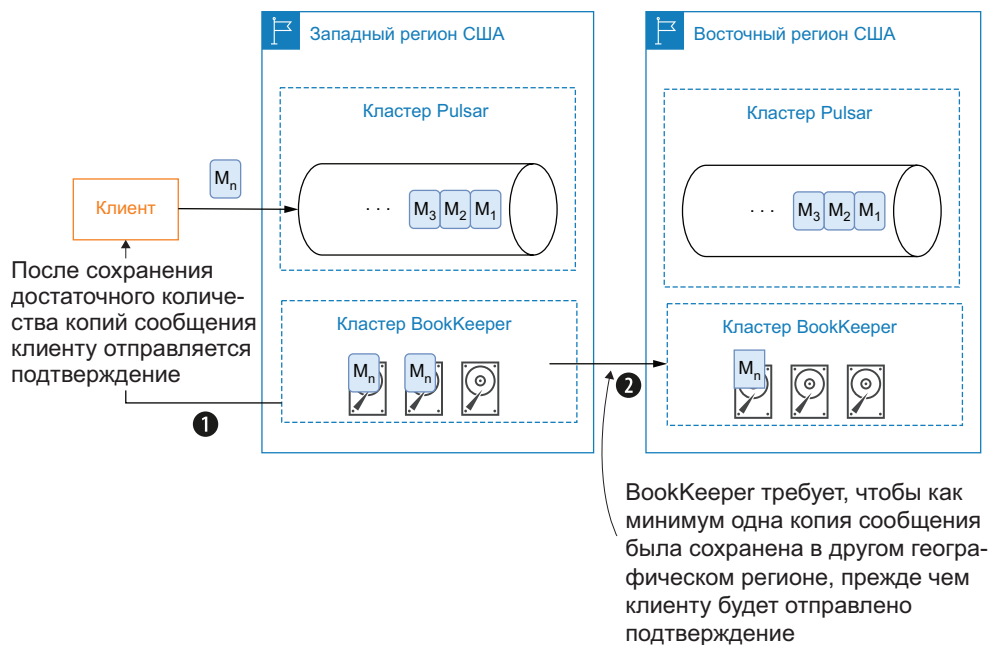


Рис. 1.23. Вы можете воспользоваться возможностью использования в BookKeeper удаленных узлов, чтобы применять синхронную георепликацию для гарантии того, что копия сообщения сохранена в другом географическом регионе

Синхронная георепликация обеспечивает более высокую степень доступности, при этом все физически доступные центры данных формируют глобальный логический экземпляр конкретной системы обработки данных. Приложения могут работать где угодно в любом центре данных и сохранять возможность доступа к данным. При этом также гарантируется полная согласованность данных между различными центрами обработки, на которую могут полагаться приложения без необходимости выполнения каких-либо ручных операций при возникновении критических сбоев в центрах данных.

В отличие от других систем обмена сообщениями, зависящих от внешних сервисов, Pulsar предоставляет георепликацию как встроенную функцию. Пользователи могут с легкостью подключить механизм репликации данных сообщений между кластерами Pulsar в различных географических регионах. После завершения процедуры конфигурации репликации данные непрерывно копируются в удаленные кластеры без какого-либо вмешательства со стороны производителей и потребителей. Более подробно конфигурирование георепликации рассматривается в приложении В.

1.6. Варианты использования из реальной практики

Если вы являетесь менеджером по ПО и ваш продукт требует обработки огромных объемов данных для оперативного предоставления пользователям действительно новых функциональных возможностей или набора данных, то Apache Pulsar – это ключ к раскрытию потенциала обработки данных в режиме реального времени. Главное достоинство Pulsar заключается в том, что существует несколько конкретных сценариев, в которых он может продемонстрировать свое превосходство. Прежде чем мы углубимся в технические подробности, было бы полезно обсудить на высоком уровне некоторые варианты использования, в которых Pulsar уже доказал свою эффективность.

1.6.1. Универсальные системы обмена сообщениями

Вероятно, вам знакомо мнемоническое правило KISS (keep it simple, stupid – «не усложняй, глупый»), часто применяемое, чтобы напомнить архитекторам о том, что самыми лучшими являются простые проектные решения. Систему, в которой применяется лишь несколько технологий, проще развертывать, сопровождать и контролировать. Как уже было отмечено выше, существуют два основных паттерна обмена сообщениями, и до недавнего времени при необходимости поддержки обоих стилей обмена сообщениями в инфраструктуре вам пришлось бы развернуть и сопровождать две совершенно разные системы.

Двуязычная электронная биржа труда [Zhaopin.com](https://zhaopin.com) содержит один из самых больших списков рабочих вакансий в Китае, включающий предложения от местных и зарубежных организаций. У этой компании более 2.2 млн клиентов, а среднее количество просмотров страниц в день – свыше 68 млн. Компания непрерывно развивалась, но вместе с этим увеличивалось и число существенных затруднений в сопровождении двух отдельных систем обмена сообщениями: RabbitMQ для организации очередей и Apache Kafka для схемы pub-sub. После замены обеих систем на единую универсальную платформу на основе Apache Pulsar у компании появилась возможность наполовину сократить эксплуатационные издержки, объем ресурсов инфраструктуры и стоимость постоянной поддержки при сохранении высокой надежности и пропускной способности с минимальными задержками в работе.

1.6.2. Платформы микросервисов

Компания Narvar предоставляет возможности управления каналом поставок (логистической цепочкой), а также платформу обслуживания клиентов для электронного бизнеса по всему миру, включая отслеживание заказов и оповещений, беспрепятственный возврат и работу с клиентами. Платформа Narvar помогает компаниям розничной торговли и крупным брендам обрабатывать данные и события для гарантии своевременного и точного обмена информацией с 400 млн своих клиентов по всему миру.

До появления Apache Pulsar платформа Narvar в течение некоторого времени формировалась с использованием разнообразных технологий обмена сообщениями и обработки данных – от Kafka до Amazon SQS, от Kinesis Streams до Kinesis Firehouse и от RabbitMQ до AWS Lambda. По мере роста трафика стало очевидно, что растущий объем DevOps и поддержки разработчиков, необходимый для сопровождения и масштабирования этих систем, неприемлем. Многие компоненты не были размещены в контейнерах, что усложняло конфигурацию и управление инфраструктурой и требовало частого ручного вмешательства.

Хотя системы, подобные Kafka, надежны, широко распространены и предоставляют открытый исходный код, тем не менее при масштабировании требуют значительных затрат на сопровождение. Для роста пропускной способности требовалось увеличение разделов, точная настройка потребителей и большой объем ручной работы разработчиков и DevOps. Да и облачные решения, такие как Kinesis Streams и Kinesis Firehouse, зависели от конкретной облачной среды, существенно затрудняя отделение процедуры выбора облачных решений от функциональности и применение технологий в других облачных средах, а также поддержку клиентов, работающих в других облачных средах.

Компания Narvar приняла решение о переводе своей платформы на основе микросервисов на Apache Pulsar, потому что, как и Kafka, Pulsar был надежным, независимым от конкретной облачной среды программным обеспечением с открытым исходным кодом. Но, в отличие от Kafka, Pulsar требовал чрезвычайно малых затрат на сопровождение и масштабировался с минимальным ручным вмешательством. Pulsar можно было разместить в контейнере на основе Kubernetes на начальном этапе, тем самым обеспечивая еще большую масштабируемость и удобство сопровождения. Самым важным являлся компонент Pulsar Functions, позволявший Narvar разрабатывать микросервисы, потребляющие и обрабатывающие входящие события непосредственно в самой системе обмена сообщениями, т. е. исключая необходимость в дорогостоящих лямбда-функциях или в создании дополнительных сервисов.

В такой архитектуре каждый микросервис спроектирован как атомарная и самодостаточная часть программного обеспечения. Эти независимые программные компоненты работают как отдельные процессы, распределенные по нескольким серверам. Приложение на основе микросервисов требует организации взаимодействия с несколькими сервисами через определенный тип обмена данными между процессами. Два наиболее часто используемых протокола обмена данными – запрос/ответ HTTP и упрощенный обмен сообщениями. Pulsar являлся идеальным кандидатом для создания упрощенной системы обработки сообщений с поддержкой асинхронного обмена сообщениями, требуемого Narvar.

1.6.3. Автомобили с сетевыми функциями

Крупнейший североамериканский производитель автомобилей создал сервис для машин с сетевыми функциями на основе Apache Pulsar для сбора данных из компьютерных устройств, встроенных в 12 млн транспортных средств, подключенных к сети. Миллиарды фрагментов данных фиксируются ежедневно и используются для обеспечения осмотра и удаленной диагностики в реальном времени по всему миру. Затем эти данные используются для улучшения представления о том, как функционируют транспортные средства, а также для обнаружения потенциальных проблем до их возникновения, чтобы производитель мог заранее предупредить клиентов.

1.6.4. Выявление случаев мошенничества

Крупнейшая китайская платформа мобильных платежей Orange Financial обязана анализировать 50 млн транзакций в день для выявления вероятных случаев финансового мошенничества, защищая 500 млн своих зарегистрированных пользователей. Orange Fi-

пancial ежедневно имеет дело с финансовыми мошенничествами, такими как похищение идентификационных данных клиентов, отмывание денег, незаконный доступ к финансам и мошенничества, совершаемые в торговых точках. Компания использует тысячи моделей обнаружения мошенничества для каждой транзакции, чтобы бороться с этими угрозами в своей системе управления рисками.

Компания искала вариант, который объединял бы хранилище данных, механизм обработки и язык программирования для разработки решений в ее системе управления рисками. С точки зрения конечного пользователя, сканирование с целью обнаружения мошенничества не должно создавать задержки в работе приложений; поэтому компании нужна была платформа, позволяющая обрабатывать данные как можно быстрее. Apache Pulsar обеспечивал доступ к данным транзакций непосредственно на уровне обмена сообщениями и обработку в параллельном режиме с использованием Pulsar Functions, сокращая время задержек, возникавших из-за необходимости перемещения данных во вторую систему для обработки.

Некоторые из рабочих методов обнаружения мошенничества были переданы в фреймворк Pulsar Functions, но компания Orange Financial сохранила возможность применения более сложных алгоритмов выявления случаев мошенничества, разработанных в Spark, используя встроенный в Pulsar коннектор для вычислительного механизма Spark. Это позволило компании выбирать наиболее подходящий фреймворк для моделей в каждом конкретном случае.

1.7. Дополнительные информационные ресурсы

Pulsar поддерживает активное и постоянно растущее сообщество. Из Apache Incubator Pulsar был переведен в группу основных программных продуктов в 2018 г. Текущую версию документации для этого проекта можно найти на официальном сайте: <http://pulsar.apache.org>.

К другим источникам информации об Apache Pulsar относятся блоги (например, <https://streamnative.io/blog>) и учебные руководства (например, <https://www.baeldung.com/apache-pulsar> и <https://medium.com/streamthoughts/introduction-to-apache-pulsar-concepts-architecture-java-clients-71f1a30b75d6>). Наконец, было бы досадным упущением не сообщить о существовании свободного канала [apache-pulsar.slack.com](https://slack.com), который постоянно отслеживается мною и несколькими участниками проекта. Этот активно используемый канал представляет большой объем информации для начинающих и сообщество квалифицированных разработчиков, работающих с Apache Pulsar на ежедневной основе.

1.8. Резюме

- Apache Pulsar – современная система обмена сообщениями, предоставляющая средства потоковой обработки данных с высокой производительностью и привычную организацию очередей сообщений.
- Apache Pulsar предоставляет упрощенный механизм обработки Pulsar Functions, позволяющий разработчикам реализовать простую логику обработки, выполняемую для каждого сообщения, публикуемого в конкретной теме.
- Преимуществом Pulsar является разделение уровней хранения и обслуживания, обеспечивающее неограниченное масштабирование и отсутствие потерь данных.
- Конкретные варианты использования Pulsar в реальной производственной практике: анализ интернета вещей (IoT), обмен данными между микросервисами и универсальные системы обмена сообщениями.



Концепции и архитектура Pulsar

Темы:

- физическая архитектура Pulsar;
- логическая архитектура Pulsar;
- типы потребления сообщений и подписки, предоставляемые Pulsar;
- стратегии долговременного хранения, определения окончания срока хранения и ведения регистрационного журнала.

После получения общего представления о платформе обмена сообщениями Pulsar и сравнения ее с другими системами обмена сообщениями мы подробнее рассмотрим архитектурные характеристики более низкого уровня и познакомимся со специализированной терминологией, используемой для этой платформы. Если вы недостаточно хорошо знакомы с системами обмена сообщениями и распределенными системами, то, вероятно, вам будет трудно понять некоторые концепции и термины Pulsar. Поэтому начнем с обзора физической архитектуры, прежде чем углубиться в логическую организацию работы с сообщениями в Pulsar.

2.1. Физическая архитектура Pulsar

Другие системы обмена сообщениями рассматривают кластер как наивысший уровень с точки зрения администрирования и развертывания, что приводит к необходимости управления и конфигурирования каждого кластера как независимой системы. К счастью, Pulsar предоставляет еще более высокий уровень абстракции, известный как экземпляр Pulsar (Pulsar instance), состоящий из одного или нескольких кластеров Pulsar, работающих вместе как единый комплекс, который можно администрировать из одной локации, как показано на рис. 2.1.

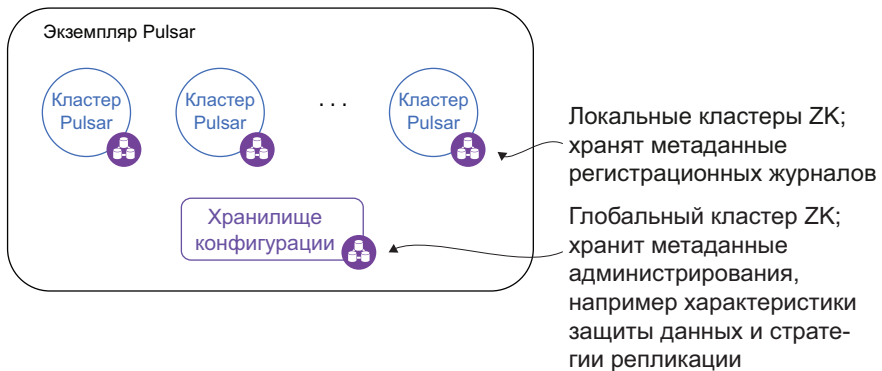


Рис. 2.1. Экземпляр Pulsar может состоять из нескольких географически удаленных друг от друга кластеров

Одно из самых главных обоснований использования экземпляра Pulsar – возможность георепликации. В действительности только кластеры внутри экземпляра можно сконфигурировать для репликации данных между ними.

Экземпляр Pulsar использует общий кластер ZooKeeper (ZK), обозначенный как хранилище конфигурации (configuration store), для хранения информации, относящейся к нескольким кластерам, например стратегию георепликации и стратегии обеспечения безопасности на уровне клиентов. Это позволяет определять такие стратегии и управлять ими в единой локации. Чтобы обеспечить отказоустойчивость хранилища конфигурации, все узлы в группе кластеров ZooKeeper экземпляра Pulsar должны развертываться в разных регионах, гарантируя доступность в случае регионального сбоя.

Важно отметить, что доступность группы ZooKeeper, используемой экземпляром Pulsar для хранилища конфигурации, требуется для работы отдельных кластеров Pulsar, даже если активизирована георепликация. Когда георепликация разрешена, даже если хранилище конфигурации выходит из строя, сообщения, публикуемые

в соответствующих кластерах, будут помещены в буфер локально и перенаправлены в другие регионы, когда группа конфигурации снова продолжит работу.

2.1.1. Многоуровневая архитектура Pulsar

На рис. 2.2 можно видеть, что кластер Pulsar состоит из уровня обслуживания без сохранения состояния с несколькими экземплярами брокеров (broker instances) сообщений Pulsar, уровня хранения с сохранением состояния с несколькими экземплярами букмекеров (bookie instances) и необязательного уровня маршрутизации с несколькими прокси (proxies) Pulsar. При размещении в среде Kubernetes такая разделенная на уровни архитектура позволяет группе DevOps динамически масштабировать в сторону увеличения количество брокеров, букмекеров и прокси при пиковых нагрузках и сокращать их число в менее напряженные периоды, чтобы уменьшить накладные расходы. Трафик сообщений распределяется по всем доступным брокерам равномерно, насколько это возможно, чтобы обеспечить максимальную пропускную способность.

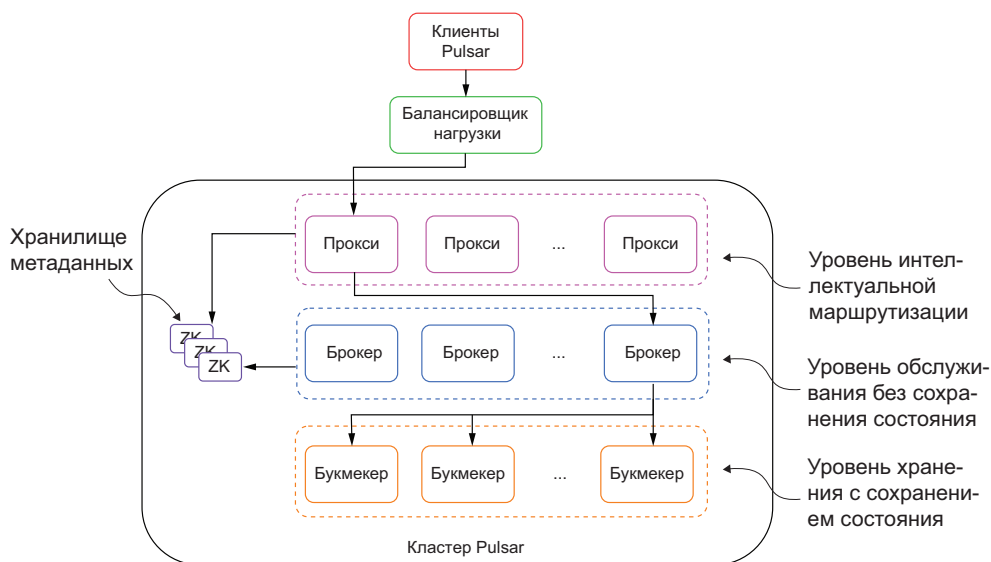


Рис. 2.2. Кластер Pulsar состоит из нескольких уровней: необязательного уровня прокси, обеспечивающего маршрутизацию входящих клиентских запросов в соответствующий брокер сообщений; уровня обслуживания без сохранения состояния, сформированного из нескольких брокеров, обрабатывающих запросы от клиентов; и уровня хранения с сохранением состояния с несколькими букмекерами, хранящими многочисленные копии сообщений

Если клиент получает доступ к теме, которая пока еще не использовалась, то стартует процесс выбора брокера, наиболее под-

ходящего для принятия права владения этой темой. После того как брокер принимает право владения темой, он отвечает за обработку всех запросов в ней, и все клиенты, желающие публиковать или потреблять данные в этой теме, должны взаимодействовать с соответствующим брокером-владельцем. Таким образом, если необходимо публиковать данные в конкретной теме, то вы должны узнать, какой брокер владеет этой темой, и установить соединение с ним. Но информация о распределении брокеров доступна только в хранилище метаданных ZooKeeper, где содержатся сведения об изменениях на основе ребалансирования нагрузки, критических сбоях брокеров и т. п. Следовательно, вы не можете устанавливать соединения непосредственно с брокерами и надеяться, что обмениваетесь данными с тем брокером, который вам нужен. Именно поэтому был создан прокси Pulsar – для работы в качестве посредника для всех брокеров в кластере.

Прокси Pulsar

Если вы размещаете кластер Pulsar в частной и/или виртуальной сетевой среде, такой как Kubernetes, и хотите обеспечить входящие соединения для брокеров Pulsar, то потребуется преобразование внутренних IP-адресов в общедоступные IP-адреса. Хотя такое преобразование можно выполнить, используя обычные технологии и методики балансировки нагрузки, например физические балансировщики нагрузки, виртуальные IP-адреса или систему балансировки нагрузки на основе DNS, распределяющую запросы клиентов по группе брокеров, это не самый лучший подход для обеспечения избыточности и устойчивости к критическим сбоям для клиентов.

Обычная методика балансировки нагрузки неэффективна, поскольку балансировщик не знает, какому брокеру присвоена конкретная тема, поэтому будет направлять запрос случайному брокеру в кластере. Если брокер получает запрос для темы, которую он не обслуживает, то автоматически перенаправляет его соответствующему брокеру для обработки, но при этом непроизводительно затрачивается некоторое время. Поэтому рекомендуется использовать прокси Pulsar, действующий как интеллектуальный балансировщик нагрузки для брокеров.

При использовании прокси Pulsar все соединения, устанавливаемые клиентами, сначала проходят через прокси, а не направляются напрямую к брокерам. Затем прокси использует встроенный механизм обнаружения сервиса Pulsar, чтобы определить, какой брокер управляет темой, к которой пытается получить доступ клиент, и автоматически перенаправляет в нее запрос. Кроме того, прокси кеширует эту информацию в памяти, чтобы еще больше упростить

процесс поиска для следующих запросов. Для обеспечения высокой производительности и устойчивости к сбоям рекомендуется создать более одного прокси позади обычного балансировщика нагрузки. В отличие от брокеров прокси Pulsar способны обработать любой запрос, поэтому выполняют балансирование нагрузки без каких-либо проблем.

2.1.2. Уровень обслуживания без сохранения состояния

Многоуровневое проектное решение Pulsar обеспечивает хранение данных сообщений отдельно от брокеров, а это гарантирует, что каждый брокер всегда может обслуживать данные из любой темы. Это также позволяет кластеру присваивать право владения темой любому брокеру в любой момент времени в отличие от других систем обмена сообщениями, в которых объединен брокер и данные обслуживаемой им темы. Таким образом, мы используем термин «без сохранения состояния» (stateless) для описания уровня обслуживания, потому что в самих брокерах не хранится никакая информация, необходимая для обработки клиентских запросов.

Такое свойство брокеров не только позволяет динамически масштабировать их в сторону увеличения и уменьшения, но также обеспечивает устойчивость кластера к критическим сбоям в нескольких брокерах. Наконец, в Pulsar имеется встроенный механизм планирования нагрузки, выполняющий перераспределение нагрузки по всем активным брокерам в зависимости от постоянно изменяющегося трафика сообщений.

Наборы

Присваивание темы конкретному брокеру выполняется на так называемом уровне наборов (bundles). Все темы в кластере Pulsar распределены по конкретным наборам, и каждый набор присваивается отдельному брокеру, как показано на рис. 2.3. Это помогает обеспечить равномерное распределение всех тем в пространстве имен по всем брокерам.

Количество созданных наборов управляется свойством `defaultNumberOfNamespaceBundles` в конфигурационном файле брокера, для которого по умолчанию установлено значение 4. Вы можете изменить значение этого параметра на уровне каждого конкретного пространства имен при его создании, определяя различные значения в процессе создания пространства имен при помощи API администратора Pulsar. В общем случае рекомендуется устанавливать число наборов, кратное количеству брокеров, чтобы обеспечить равномерное распределение. Например, если имеется три брокера и четыре набора, то одному из брокеров будет присвоено два набора, тогда как другие получат только по одному.

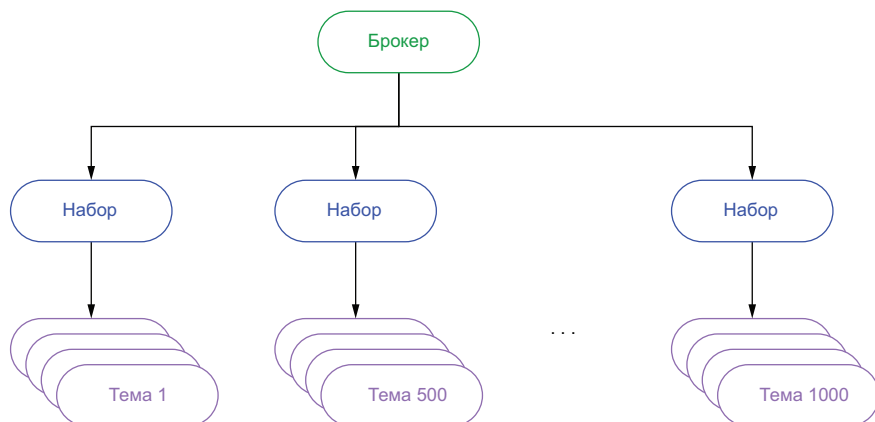


Рис. 2.3. С точки зрения обслуживания каждому брокеру присваивается комплект наборов, содержащих несколько тем. Присваивание набора определяется посредством хеширования имени темы, которое позволяет узнать, к какому набору принадлежит тема, без необходимости хранения этой информации в ZooKeeper

Балансирование нагрузки

Изначально трафик сообщений может быть распределен между активными брокерами настолько равномерно, насколько это возможно, но некоторые факторы могут изменяться с течением времени, и в результате нагрузка становится несбалансированной. Изменения в паттернах трафика сообщений могут привести к тому, что некоторый брокер будет обслуживать несколько тем с интенсивным трафиком, а другие вообще не будут задействованы. Если в существующем наборе превышены некоторые предварительно сконфигурированные пороговые значения, определяемые перечисленными ниже свойствами в конфигурационном файле брокера, то набор будет разделен на два новых набора, один из которых передается новому брокеру:

- `loadBalancerNamespaceBundleMaxTopics;`
- `loadBalancerNamespaceBundleMaxSessions;`
- `loadBalancerNamespaceBundleMaxMsgRate;`
- `loadBalancerNamespaceBundleMaxBandwidthMbytes.`

Этот механизм определяет и корректирует сценарии, где в некоторых наборах возникает повышенная нагрузка по сравнению с другими, и разделяет перегруженные наборы на два новых. Затем один из новых наборов передается другому брокеру в кластере.

Снижение нагрузки

Для брокеров Pulsar имеется еще один механизм, позволяющий обнаружить повышенную нагрузку на конкретный брокер и автоматически снизить ее, передавая некоторые его наборы другому брокеру в том же кластере. Если уровень использования ресурсов

брокера превышает предварительно сконфигурированное пороговое значение, определяемое свойством `loadBalancerBrokerOverloadedThresholdPercentage` в конфигурационном файле брокера, то перегруженный брокер передает один или несколько наборов новому брокеру. Это свойство определяет максимальный процент от общего доступного ресурса ЦП, пропускной способности сети или объема памяти, который может потреблять брокер. Если для любого из этих ресурсов превышено пороговое значение, то срабатывает механизм снижения нагрузки.

Выбранный набор остается неизменным и присваивается другому брокеру. Причина в том, что процесс снижения нагрузки устраняет другую проблему, нежели процесс балансирования нагрузки. При балансировании нагрузки корректируется распределение тем по наборам, потому что в одном из них возникает более интенсивный трафик, чем в других, и мы пытаемся распределить нагрузку по всем наборам.

С другой стороны, механизм снижения нагрузки корректирует распределение наборов по брокерам на основе количественной характеристики ресурсов, требуемых для обслуживания. Даже если каждому брокеру может быть присвоено одинаковое количество наборов, трафик сообщений, обрабатываемый каждым брокером, может оказаться весьма различным, если нагрузка не сбалансирована по наборам.

Для наглядной демонстрации такой ситуации рассмотрим сценарий, в котором существуют 3 брокера и в общей сложности 60 наборов, т. е. каждый брокер обслуживает 20 наборов. Кроме того, 20 наборов в текущий момент обрабатывают 90 % общего трафика сообщений. И если случилось так, что большинство этих наборов присвоено одному брокеру, то, вероятнее всего, ресурсы ЦП, сети и памяти этого брокера почти исчерпаны. Следовательно, передача некоторых перегруженных наборов другому брокеру поможет уменьшить воздействие этой проблемы, тогда как разделение самих наборов должно лишь приблизительно наполовину уменьшить трафик сообщений, оставляя 45 % нагрузки в исходном брокере.

Паттерны доступа к данным

В системе потоковой обработки данных существуют три основных паттерна ввода-вывода: `writes` (запись) – новые данные записываются в систему, `tailing reads` (хвостовое чтение) – потребитель читает самые последние опубликованные сообщения сразу после их публикации, `catch-up reads` (догоняющее чтение) – потребитель читает большое количество сообщений с самого начала темы, чтобы «догнать» последнее сообщение, например если новому потребителю необходим доступ к данным, расположенным в гораздо более ранней точке, чем самое последнее сообщение.

Когда производитель отправляет сообщение в Pulsar, оно немедленно записывается в BookKeeper. После того как BookKeeper подтверждает прием данных, брокер сохраняет копию сообщения в своем локальном кеше, прежде чем передать подтверждение публикации сообщения производителю. Это позволяет брокеру обслуживать хвостовое чтение потребителей напрямую из памяти и избегать задержек, связанных с доступом к диску.

Еще более интересным является паттерн догоняющего чтения (catch-up reads), обеспечивающий доступ к данным на уровне хранения. Когда клиент потребляет некоторое сообщение из Pulsar, оно проходит несколько этапов (шагов), показанных на рис. 2.4. Наиболее часто пример догоняющего чтения встречается, когда потребитель переходит в режим офлайн на достаточно длительный интервал времени, а затем снова начинает потребление, хотя любой сценарий, в котором потребитель не обслуживается непосредственно из кеша, расположенного в памяти брокера, должен считаться догоняющим чтением, например после передачи темы новому брокеру.

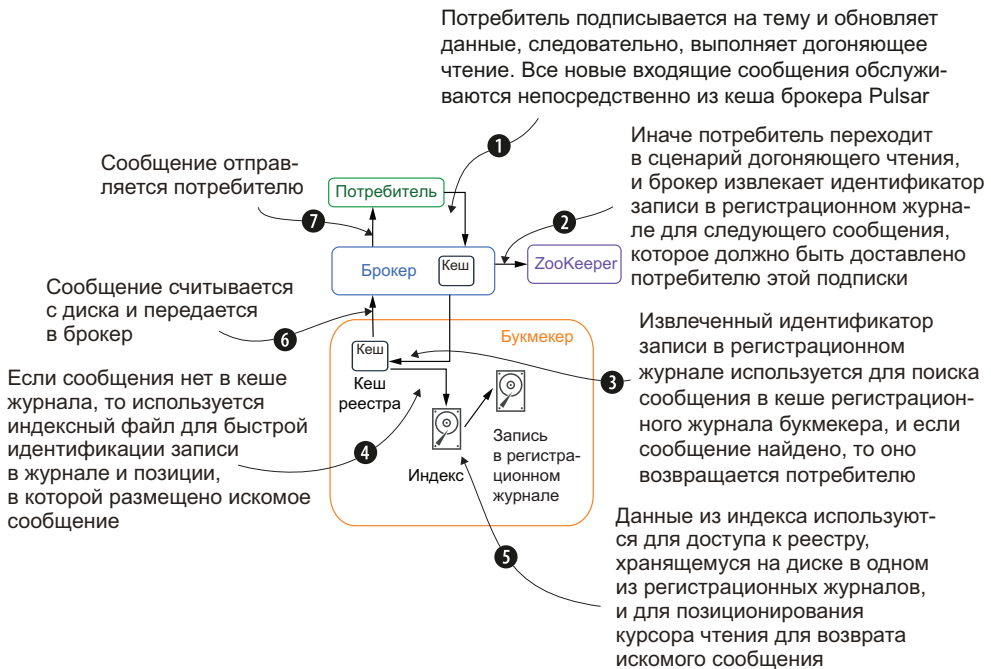


Рис. 2.4. Шаги, выполняемые при потреблении сообщения в Pulsar

2.1.3. Уровень хранения потоковых данных

Pulsar гарантирует доставку для всех потребителей сообщений. Если сообщение успешно прибыло в брокер Pulsar, то можно не сомневаться в том, что оно будет доставлено в указанное место назна-

чения. Для обеспечения этой гарантии все неподтвержденные сообщения должны храниться постоянно до тех пор, пока они не будут доставлены потребителям, которые должны подтвердить факт доставки. Как уже было отмечено выше, Pulsar использует распределенную систему регистрации записи с упреждением (write-ahead log – WAL) Apache BookKeeper для постоянного хранения сообщений. BookKeeper – это сервис, предоставляющий постоянное хранилище потоков журнальных записей в последовательностях, называемых реестрами.

Логическая архитектура хранилища

Темы Pulsar можно воспринимать как бесконечные потоки сообщений, которые сохраняются последовательно в порядке их приема. Входящие сообщения добавляются в конец потока, а потом потребители читают поток на основе паттернов доступа к данным, описанных выше. Хотя такое упрощенное представление помогает быстрее понять обоснование позиции потребителя внутри темы, эта абстракция не может действительно существовать в реальности из-за ограниченного пространства на устройствах хранения. Так или иначе, абстрактную концепцию бесконечного потока мы вынуждены реализовать в физической системе, где эти ограничения существуют.

В Apache Pulsar принят подход к реализации хранилища потоковых данных, абсолютно отличающийся от ранее существующих систем обмена сообщениями, таких как Kafka. В Kafka каждый поток разделен на несколько реплик, каждая из которых полностью хранится на локальном диске брокера. Главное преимущество такого подхода заключается в его простоте и скорости, поскольку все операции записи выполняются последовательно, ограничивая количество перемещений головок диска, требуемое для доступа к данным. Недостатком принятого в Kafka подхода является то, что каждый отдельный брокер должен иметь достаточную емкость хранилища для данных раздела, как было отмечено в главе 1.

Так чем же отличается подход, принятый в Apache Pulsar? Для начинающих каждая тема выглядит не как набор разделов, а как последовательность сегментов. Каждый сегмент может содержать конфигурируемое количество сообщений, по умолчанию – 50 000. Когда сегмент заполняется до отказа, создается следующий для хранения новых сообщений. Таким образом, тему Pulsar можно воспринимать как неограниченный список сегментов, каждый из которых содержит некоторое подмножество сообщений, как показано на рис. 2.5, где отображена не только логическая архитектура уровня хранения потоковых данных, но и ее отображение на внутреннюю физическую реализацию.



Рис. 2.5. Данные в теме Pulsar хранятся как последовательность реестров внутри уровня BookKeeper. Список идентификаторов этих реестров внутри логической конструкции известен как управляемый реестр в ZooKeeper. Каждый реестр содержит 50 000 записей, в которых хранятся копии данных сообщений. Следует отметить, что хранилище persistent://tenant/ns/my-topic будет рассматриваться как концепция несколько позже в этой книге

Тема Pulsar – это не более чем адресуемая конечная точка, используемая для однозначной идентификации конкретной темы в Pulsar, которая является аналогом URL в том смысле, что явно применяется для однозначной идентификации ресурса, с которым клиент пытается установить соединение. Имя темы должно декодироваться брокером Pulsar для определения локации хранилища данных.

Pulsar добавляет дополнительный уровень абстракции поверх реестров BookKeeper под названием «управляемые реестры» (managed ledgers), в которых хранятся идентификаторы реестров, содержащих данные сообщений, публикуемых в теме. Как можно видеть на рис. 2.5, при первоначальной публикации данных в теме А они были записаны в реестр ledger-20. После публикации 50 000 записей в этой теме реестр был закрыт и создан новый (ledger-245), чтобы заменить его. Такой процесс повторяется через каждые 50 000 записей, сохраняющих входные данные, а управляющий реестр хранит эту однозначно определенную последовательность идентификаторов реестров внутри ZooKeeper.

Далее, когда потребитель пытается прочитать данные из темы А, управляемый реестр используется для поиска данных внутри Book-

Keeper и возврата их потребителю. Если потребитель выполняет догоняющее чтение, начиная с самого старого сообщения, то он должен сначала получить все данные из реестра ledger-20, затем ledger-245 и т. д. Проход по этим реестрам от самого старого до самого свежего невидим для конечного пользователя и создает иллюзию единого последовательного потока данных. Управляемые реестры позволяют выполнять эту процедуру и сохраняют порядок реестров BookKeeper для гарантии того, что сообщения считываются в том порядке, в котором они были опубликованы.

Физическая архитектура BookKeeper

В BookKeeper каждый элемент реестра обозначается термином «запись» (entry). Эти записи содержат реальные необработанные байты из входящих сообщений вместе с некоторыми важными метаданными, используемыми для отслеживания и доступа к записям. Самым важным фрагментом метаданных является идентификатор реестра, которому принадлежит запись, и этот идентификатор хранится в локальном экземпляре ZooKeeper, поэтому сообщение можно быстро извлечь из BookKeeper, когда потребитель попытается прочитать его в будущем. Потоки записей журнала хранятся в структурах данных с возможностью только добавления, известных как реестры (ledgers), как показано на рис. 2.6.

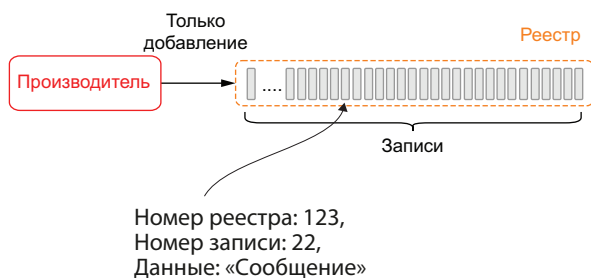


Рис. 2.6. В BookKeeper входящие записи при сохранении объединяются в реестры на серверах, которые называются букмекерами

Реестры обладают семантикой «только добавление» (append-only), означающей, что записи вносятся в реестр последовательно и после этого не могут быть изменены. С практической точки зрения это означает:

- брокер Pulsar сначала создает реестр, затем добавляет в него записи и в итоге закрывает реестр. Все прочие операции запрещены;
- после закрытия реестра в нормальном режиме или после критического сбоя процесса реестр можно открыть в режиме «только для чтения» (read-only);

- если записи в реестре больше не нужны, то весь реестр может быть удален из системы.

Отдельные серверы BookKeeper, которые отвечают за хранение реестров (точнее: фрагментов реестров), называются букмекерами (bookies). При внесении в реестр записи распределяются по подгруппам узлов букмекеров, обозначаемых термином «ансамбль» (ensemble). Размер ансамбля равен коэффициенту репликации (R), определенному для конкретной темы Pulsar, и гарантирует, что будет создано ровно R копий записи, сохраненной на диске, для предотвращения потери данных.

Букмекеры управляют данными с помощью структурированных журналов, реализованных с использованием трех типов файлов: системных журналов (journal), журналов записей (entry logs) и индексных (index) файлов. В системном журнале сохраняются все записи о транзакциях BookKeeper. Перед любым обновлением реестра букмекер обеспечивает запись на диск транзакции, описывающей это изменение, для предотвращения потери данных.

Наконец, производителю отправляется подтверждение, чтобы сообщить ему о том, что его сообщение успешно принято, сохранено и внесено в реестр для последующих обращений к нему

Ответ возвращается в брокер Pulsar, включая идентификатор журнала регистрации, под которым было записано это сообщение.

Когда сообщение окончательно принято, его можно отправить непосредственно всем подписчикам, подключенным к этой теме, и добавить в локальный кеш в брокере

Индекс сообщения обновляется в кеше реестра, размещенном в памяти, выделенной букмекеру, чтобы последующие запросы на чтение можно было обслужить более эффективно

Производитель отправляет сообщение в брокер Pulsar, который перенаправляет сообщение на один из узлов-букмекеров, объявивший себя активным и доступным для сохранения сообщения

Брокер сохраняет пару <Message ID, Ledger ID> в своем локальном ZooKeeper, чтобы отслеживать, где были сохранены сообщения

В букмекере сообщение добавляется в журнал регистрации входящих, а соответствующая ему транзакция – в системный журнал.

Обе эти операции fsync выполняются запись данных на диск для гарантии того, что данные не будут потеряны

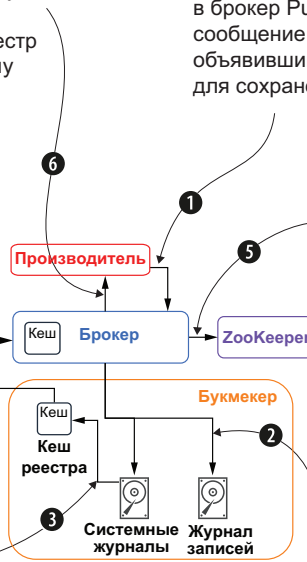


Рис. 2.7. Шаги, обеспечивающие постоянное хранение сообщений в Pulsar

Файл журнала записей (entry log) содержит реальные данные, записанные в BookKeeper. Записи из различных реестров объединяются и записываются последовательно, в то время как их смещения сохраняются как указатели в кеше реестра для быстрого поиска. Для каждого реестра создается индексный файл, содержащий не-

сколько индексов, фиксирующих смещения данных, хранящихся в файлах журналов записей. Индексный файл формируется по образцу индексных файлов в обычных реляционных базах данных и позволяет выполнять быстрый поиск для потребителей реестра. Когда клиент публикует сообщение в Pulsar, оно проходит все шаги, показанные на рис. 2.7, для обеспечения постоянного хранения на диске в реестре BookKeeper.

При распределении записей данных по нескольким файлам на различных дисковых устройствах букмекеры получают возможность защитить результаты операций чтения от задержек, связанных с текущим выполнением операций записи, что позволяет одновременно производить тысячи операций чтения и записи.

2.1.4. Хранилище метаданных

Каждый кластер также имеет собственный локальный ансамбль ZooKeeper, который Pulsar использует для хранения специализированной для кластера конфигурационной информации для абонентов, пространств имен и тем, в том числе стратегий обеспечения безопасности и сроков хранения данных. Это дополняет информацию об управляемом реестре, который рассматривался выше.

Основы организации ZooKeeper

В соответствии с описанием на официальном веб-сайте Apache «ZooKeeper представляет собой централизованный сервис для обслуживания конфигурационной информации, именования, обеспечения распределенной синхронизации и предоставления групповых сервисов» (<https://zookeeper.apache.org>), т. е. источник распределенных данных. ZooKeeper предоставляет децентрализованные локации для хранения информации, что особенно важно в распределенных системах, таких как Pulsar и BookKeeper.

Apache ZooKeeper решает глобальную проблему достижения консенсуса (т. е. соглашения), которую обязана разрешить практически каждая распределенная система. Процессы в распределенной системе требуют согласования по нескольким различным фрагментам информации, например по текущим значениям параметров конфигурации и по владельцам темы. Эта проблема является специфической для распределенных систем из-за того, что существует несколько копий одного компонента, работающих одновременно, при отсутствии реального способа согласования информации между ними. Такая ситуация невозможна в обычных базах данных, потому что в них вводится точка сериализации внутри фреймворка, в которой все вызывающие сервисы должны останавливаться в ожидании снятия единой блокировки в таблице, но такой подход исключает все преимущества распределенной обработки данных.

Наличие доступа к реализации консенсуса позволяет распределенным системам координировать процессы более эффективным способом, предоставляя операцию compare-and-swap (CAS; сравнение с обменом) для реализации распределенных блокировок. Операция CAS сравнивает значение, извлеченное из ZooKeeper, с ожидаемым значением и, только если они одинаковы, обновляет значение. Это гарантирует, что система работает на основании самой свежей информации. Одним из примеров является проверка состояния реестра BookKeeper: он должен быть открыт (open) перед записью данных. Если какой-либо другой процесс закрыл этот реестр, это должно быть отражено в данных ZooKeeper, и текущий процесс необходимо оповестить о невозможности выполнения операции записи. Ну а если процесс закрыл реестр, информацию об этом необходимо отправить в ZooKeeper для передачи всем прочим сервисам, чтобы до попытки записи в реестр они узнали, что он закрыт.

Сам сервис ZooKeeper предоставляет похожий на файловую систему API, чтобы клиенты могли работать с простыми файлами данных (znodes) для хранения информации. Каждый из таких znodes формирует иерархическую структуру, аналогичную файловой системе. В следующих разделах я подробно опишу метаданные, постоянно хранящиеся в ZooKeeper, а также способы и точки их использования, чтобы вы сами могли понять в полной мере, для чего нужен ZooKeeper. Наилучший способ сделать это – воспользоваться инструментальным средством `zookeeper-shell`, которое распространяется вместе с Pulsar, как показано в листинге 2.1, для получения списка всех znodes.

Листинг 2.1. Использование командной оболочки `zookeeper-shell` для получения списка znodes

```
/pulsar/bin/pulsar zookeeper-shell ❶  
ls / ❷  
[admin, bookies, counters, ledgers, loadbalance,  
➔ managed-ledgers, namespace, pulsar, schemas, stream, zookeeper] ❸
```

- ❶ Запуск командной оболочки ZooKeeper.
- ❷ Список узлов-потомков znodes, расположенных ниже уровня корневого узла.
- ❸ Вывод всех znodes, используемых Pulsar.

В листинге 2.1 можно видеть, что в общей сложности существует 11 различных узлов znodes, созданных внутри ZooKeeper для Apache Pulsar и BookKeeper. Они разделяются на четыре категории в зависимости от того, какую информацию содержат и как используются.

Конфигурационные данные

Первая категория информации – конфигурационные данные для абонентов, пространств имен, схем и т. п. Вся эта информация изменяется чрезвычайно медленно, т. е. обновляется только через API администрирования Pulsar, когда пользователь создает новый кластер, абонента, пространство имен, схему или обновляет существующий. Информация включает такие элементы, как стратегии обеспечения безопасности, стратегии определения сроков хранения данных, стратегии репликации, а также схемы. Эта информация хранится в следующих z-узлах: `/admin` и `/schemas`.

Хранилище метаданных

Информация управляемого реестра для всех тем хранится в z-узле `/managed-ledgers`, тогда как z-узел `/ledger` используется BookKeeper для отслеживания всех реестров, хранящихся в текущий момент во всех букмекерах в кластере.

Листинг 2.2. Просмотр управляемого реестра

```
/pulsar/bin/pulsar-managed-ledger-admin print-managed-ledger -  
➔ managedLedgerPath /public/default/persistent/topicA  
➔ --zkServer localhost:2181
```

1

```
ledgerInfo { ledgerId: 20 entries: 50000 size: 3417764 timestamp: 1589590969679}  
ledgerInfo { ledgerId: 245 timestamp: 0}
```

2

- 1 Инструмент управляемый реестр позволяет выполнять поиск реестров по имени темы.
- 2 Эта тема состоит из двух реестров: один содержит 50 000 записей, поэтому закрыт, а второй открыт.

В листинге 2.2 можно видеть, что существует еще одно инструментальное средство под названием `pulsar-managed-ledger-admin`, позволяющее легко получить доступ к информации управляемого реестра, используемой Pulsar для чтения и записи данных в BookKeeper. В рассматриваемом здесь случае данные темы хранятся в двух различных реестрах: `ledgerID-20`, который закрыт, потому что содержит 50 000 записей, и `ledgerID-245`, который в текущий момент открыт, и в нем будут публиковаться входящие данные.

Динамическое согласование между сервисами

Все остальные z-узлы используются для распределенной координации работы систем: `/bookies` обслуживает список букмекеров, зарегистрированных в кластере BookKeeper, а `/namespaces` используется прокси-сервисом, чтобы определить, какой брокер владеет конкретной темой. В листинге 2.3 можно видеть, что иерархия z-узлов `/namespaces` используется для хранения комплекта идентификаторов для каждого пространства имен.

Листинг 2.3. Метаданные, используемые для определения владельца темы

```

/pulsar/bin/pulsar zookeeper-shell
ls /namespace
[customers, public, pulsar]
ls /namespace/customers
[orders]
ls /namespace/customers/orders
[0x40000000_0x80000000]
get /namespace/customers/orders/0x40000000_0x80000000
{"nativeUrl":"pulsar://localhost:6650",
➔ "httpUrl":"http://localhost:8080","disabled":false}

```

1
2
3
4

- ❶ Запуск командной оболочки ZooKeeper.
- ❷ Для каждого абонента существует отдельный z-узел.
- ❸ Для каждого пространства имен существует отдельный z-узел.
- ❹ Для каждого `bundle_id` существует отдельный z-узел.

Вспомним из ранее изученного материала, что имя темы хешируется прокси для определения имени набора (`bundle`) – в рассматриваемом здесь примере это `0x40000000_0x80000000`. Затем прокси запрашивает z-узел `/namespace/{tenant}/{namespace}/{bundle-id}` для извлечения URL брокера, который «владеет» этой темой.

Надеюсь, что это описание помогает вам лучше понять роль ZooKeeper в кластере Pulsar и способ предоставления сервиса, легко доступного через узлы, динамически добавленные в кластер, поскольку можно без затруднений определить конфигурацию кластера и начать обработку клиентских запросов. Пример: возможность нового добавленного брокера начать обслуживание данных из темы через ссылки на данные в z-узле `/managed-ledgers`.

2.2. Логическая архитектура Pulsar

Как и другие системы обмена сообщениями, Pulsar использует концепцию тем для обозначения каналов сообщений для передачи данных между производителями и потребителями. Но способ именования этих тем в Pulsar отличается от всех прочих систем обмена сообщениями. В следующих подразделах мы рассмотрим внутреннюю логическую структуру, которую Pulsar использует для хранения данных и управления темами.

2.2.1. Абоненты, пространства имен и темы

В этом подразделе мы рассмотрим логические конструкции, которые описывают, как данные структурируются и хранятся внутри кластера. Pulsar был спроектирован для работы в качестве систе-

мы с множеством абонентов, что обеспечивает его совместное использование многими подразделениями любой организации с предоставлением каждому подразделению собственных средств защиты и единственную в своем роде среду обмена сообщениями. Такое проектное решение позволяет экземпляру Pulsar эффективно функционировать в качестве «платформы как сервиса» (Platform as a Service – PaaS) для обмена сообщениями в масштабе всей организации в целом. Логическая архитектура Pulsar поддерживает многоарендную архитектуру (архитектуру с множеством абонентов) через иерархию абонентов, пространств имен и тем, как показано на рис. 2.8.

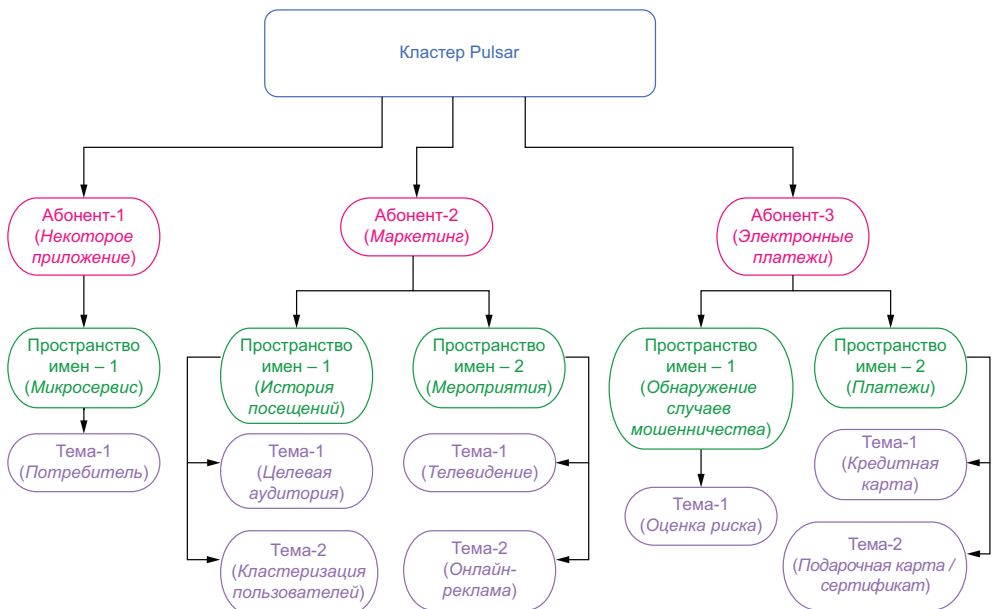


Рис. 2.8. Логическая архитектура Pulsar состоит из абонентов, пространств имен и тем

Абоненты

На вершине иерархии Pulsar находятся абоненты (tenants), которые могут представлять конкретные бизнес-элементы, основные функциональные средства или линию (номенклатуру) продуктов. Абоненты могут быть распределены по кластерам и иметь собственные схемы аутентификации и авторизации, применяемые к ним, т. е. управлять доступом к данным, хранящимся в кластере. К абонентам также относятся административные группы, управляющие квотами хранения, сроками хранения сообщений и стратегиями изоляции.

Пространства имен

Каждый абонент может иметь несколько пространств имен (namespaces) – механизмов логического группирования для администрирования соответствующих тем через стратегии (policies). На уровне пространств имен вы можете установить права доступа, более точно настроить параметры репликации, управлять гео-репликацией данных сообщений между кластерами и устанавливать срок хранения сообщений для всех тем в конкретном пространстве имен.

Рассмотрим подробнее, как можно структурировать пространство имен Pulsar для приложения электронной коммерции. Для обеспечения изоляции чрезвычайно важных входящих данных о платежах и ограничения доступа к ним только для членов финансовой группы можно сконфигурировать отдельного абонента с именем E-payments, как показано на рис. 2.8, и применить стратегию, предоставляющую полные права доступа только для членов финансовой группы, чтобы они получили возможность выполнять аудиторские проверки и обрабатывать транзакции с использованием кредитных карт.

Внутри абонента E-payments можно создать два пространства имен: payments, содержащее данные о входящих платежах, в том числе по кредитным картам и с учетом погашения подарочных сертификатов / карт, а также fraud detection, в котором будут фиксироваться транзакции, помеченные как подозрительные и предназначенные для дальнейшей обработки. При такой схеме развешивания создается возможность ограничить доступ пользовательского приложения правом только на запись в пространство имен payments и предоставить право только на чтение для приложения, выявляющего случаи мошенничества, чтобы оно могло оценивать потенциальные риски.

В пространстве имен fraud detection следует предоставить право доступа на запись приложению, выявляющему случаи мошенничества, чтобы оно могло помещать предположительно мошеннические платежные операции в тему «risk score» (оценка риска). Также можно предоставить право доступа только на чтение того же пространства имен приложению электронной коммерции, чтобы оно могло получать оповещения о любом предполагаемом мошенничестве и реагировать соответствующим образом, например блокировать продажу.

Темы

Темы – это всего лишь тип канала обмена информацией, доступный в Pulsar. Все сообщения записываются в темы и считываются из них. Другие системы обмена сообщениями поддерживают более одного типа канала обмена информацией (например, темы и очереди, различаемые по типу механизма потребления сообщений,

поддерживаемого системой). Как было отмечено в главе 1, очереди поддерживают только механизм потребления «первым вошел, первым вышел» (FIFO), тогда как темы основаны на механизме потребления pub-sub (публикация–подписка) «один-ко-многим». Pulsar не учитывает такие различия, а вместо этого использует разнообразные типы подписки для управления паттерном потребления сообщений.

В Pulsar темы, не размещенные в нескольких разделах, обслуживаются одним брокером, который отвечает за прием и доставку всех сообщений в такой теме. Таким образом, пропускная способность одной темы зависит от вычислительной мощности обслуживающего ее брокера.

Темы, размещенные в нескольких разделах

Pulsar также поддерживает концепцию тем, размещенных в нескольких разделах, которые могут обслуживаться многими брокерами, что обеспечивает гораздо более высокую пропускную способность, так как нагрузка распределяется по группе компьютеров. С точки зрения внутренней организации тема, размещенная в нескольких разделах, реализована как N внутренних тем, где N – количество разделов. Распределение разделов по брокерам Pulsar выполняет автоматически, делая этот процесс фактически невидимым для конечных пользователей.

Реализация тем в нескольких разделах как последовательности отдельных тем позволяет пользователю увеличивать количество разделов без необходимости ребалансирования всей темы в целом. Вместо этого создаются внутренние темы для новых разделов, которые способны немедленно принимать входящие сообщения без какого-либо воздействия на другие внутренние темы (например, для потребителей сохраняется возможность читать/записывать сообщения в существующих разделах без пауз).

С точки зрения потребителя, нет никаких различий между темами в нескольких разделах и обычными темами. Все подписки потребителей работают точно так же, как в не распределенных по разделам темах. Но есть существенное различие в том, что происходит, если сообщение публикуется в теме, размещенной в нескольких разделах. Производитель сообщения отвечает за определение того, в какой внутренней теме публикуется это сообщение в конечном счете. Если сообщение содержит значение в поле ключа метаданных, то производитель хеширует это значение, чтобы определить, в какую тему публикуется сообщение. Такой подход гарантирует, что все сообщения с одинаковым ключом сохраняются в одной теме и размещаются в порядке их публикации.

При публикации сообщения без ключа производитель должен быть сконфигурирован с режимом маршрутизации, определя-

ющим, как распределять сообщения по разделам в теме. Режим маршрутизации по умолчанию называется `RoundRobinPartition`, и, как понятно из его имени, публикуемые сообщения распределяются по разделам по методике круговой циклической (карусельной) диспетчеризации. Эта методика равномерно распределяет сообщения по разделам, обеспечивая максимум пропускной способности. Можно использовать другой режим маршрутизации `SinglePartition`, который случайным образом выбирает один раздел для публикации всех сообщений. Такой подход можно использовать для объединения в группу сообщений от конкретного производителя для сохранения их порядка, если отсутствует значение ключа. Кроме того, можно предоставить собственную реализацию маршрутизации, если необходимо более точное управление распределением сообщения в теме, размещенной в нескольких разделах.

Рассмотрим поток сообщений, изображенный на рис. 2.9, где производитель сконфигурирован с режимом публикации (маршрутизации) `RoundRobinPartition`. В этом сценарии производитель устанавливает соединение с прокси Pulsar и ожидает возврата IP-адреса брокера, связанного с темой, в которую нужно записывать сообщения. В свою очередь прокси обращается в локальное хранилище метаданных за этой информацией и обнаруживает, что тема размещена в нескольких разделах, поэтому требуется преобразование конкретного номера раздела в имя внутренней темы, которую обслуживает этот раздел.

Производитель сконфигурирован для использования конкретного режима маршрутизации (например, `RoundRobinPartition`), применяемого для определения раздела, в который направляется заданное сообщение

Затем производитель публикует это сообщение так же, как и все прочие, указывая имя темы (например, `publish ["partitioned-topic"]`)

Прокси обращается в локальное хранилище метаданных, чтобы определить, какой брокер обслуживает эту тему, и на основании значения номера раздела, предоставленного производителем, узнает адрес брокера, обслуживающего в текущий момент заданный раздел, после чего возвращает его производителю

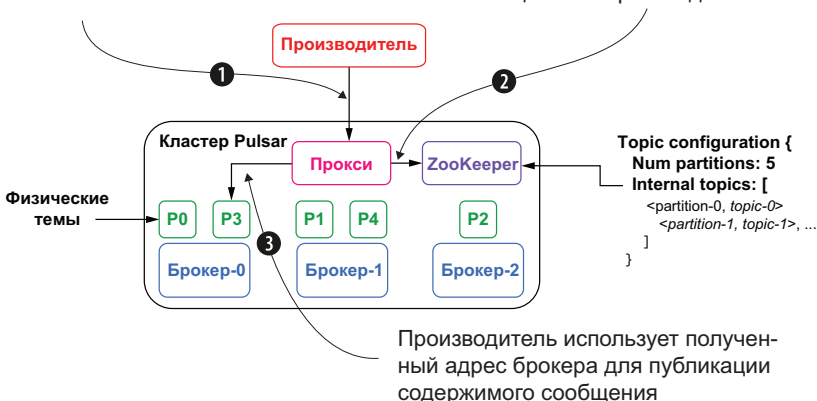


Рис. 2.9. Публикация сообщения в тему, размещенную в нескольких разделах

На рис. 2.9, в соответствии со стратегией круговой циклической (карусельной) диспетчеризации производителя, определено, что сообщение должно быть опубликовано в разделе № 3, реализованном как внутренняя тема р3. Прокси также может определить, что внутренняя тема р3 в текущий момент обслуживается брокером-0. Следовательно, сообщение направляется в этот брокер и записывается в тему р3. Поскольку применяется режим круговой циклической (карусельной) диспетчеризации, следующий вызов от того же производителя приведет к тому, что сообщение будет направлено во внутреннюю тему р4, обслуживаемую брокером-1.

2.2.2. Адресация тем в Pulsar

Иерархическая структура логических уровней Pulsar отображается в соглашении об именовании конечных точек, используемых для доступа к темам в Pulsar. На рис. 2.10 можно видеть, что каждая тема, адресуемая в Pulsar, содержит абонента и пространство имен, к которому она принадлежит. Адрес также содержит префикс персистентности, указывающий, будет ли содержимое сообщения постоянно сохранено в долговременном хранилище, или оно останется только в пространстве памяти букмекера. Если имя темы создается с префиксом `persistent://`, то все принятые, но пока еще не подтвержденные сообщения будут сохранены на нескольких узлах букмекеров, следовательно, обеспечивается возможность их защиты в случае критических сбоев брокера.

`(non-)persistent://tenant/namespace/topic-name`

Рис. 2.10. Схема адресации темы в Pulsar

Pulsar также поддерживает неперсистентные темы, которые оставляют все неподтвержденные сообщения в памяти брокера. Имена неперсистентных тем начинаются с префикса `non-persistent://` для обозначения такого поведения. При использовании неперсистентных тем брокеры немедленно доставляют сообщения всем подписчикам с установленным соединением без постоянного хранения содержимого.

При использовании неперсистентной доставки любая форма критического сбоя брокера или разрыв соединения подписчика с темой приводит к потере всех сообщений, доставляемых в текущий момент из (неперсистентной) темы. Это означает, что подписчики никогда не смогут получить потерянные сообщения, даже если повторно установят соединение. Хотя неперсистентная передача сообщений обычно быстрее персистентной, поскольку отсутствуют задержки, связанные с сохранением данных на диск, она рекомендуется к использованию, только если вы вполне уверены в том, что такой вариант использования может быть устойчивым к потерям сообщений.

2.2.3. Производители, потребители и подписки

Pulsar основан на паттерне «публикация–подписка» (pub-sub). В этом паттерне производители публикуют сообщения в темах. Затем потребители могут подписаться на эти темы, обрабатывать входящие сообщения и отправлять подтверждения после завершения обработки.

Производитель (producer) – это любой процесс, установивший соединение с брокером Pulsar либо напрямую, либо через прокси Pulsar и публикующий сообщения в некоторой теме, а потребитель (consumer) – это любой процесс, установивший соединение с брокером Pulsar для приема сообщений из некоторой темы. Когда потребитель успешно обработал сообщение, он должен отправить подтверждение, чтобы брокер знал, что сообщение принято и обработано. Если такое подтверждение не принято в течение предварительно определенного интервала времени, то брокер повторно доставляет сообщение потребителям данной подписки.

Когда потребитель устанавливает соединение с темой Pulsar, он оформляет то, что называется подпиской (subscription), определяющей, как сообщения будут доставляться в группу из одного или нескольких потребителей. В Pulsar существует четыре доступных режима подписки: эксклюзивная (exclusive), отказоустойчивая (failover), совместно используемая по ключу (key-shared) и совместно используемая (shared). Вне зависимости от типа подписки сообщения доставляются в порядке их приема.

Информация о подписках сохраняется в локальных метаданных Pulsar ZooKeeper и, помимо всего прочего, включает http-адреса всех потребителей. Каждая подписка также содержит связанный с ней курсор, представляющий позицию самого последнего сообщения, которое было потреблено и подтверждено в данной подписке. Для предотвращения повторной доставки сообщений курсоры подписок сохраняются в букмекерах, обеспечивая защиту в случае любых сбоев на уровне брокеров.

Pulsar поддерживает несколько подписок в одной теме, позволяя многочисленным потребителям считывать данные из темы. На рис. 2.11 можно видеть, что в теме существуют две различные подписки: Sub-A и Sub-B. Потребитель-A первым установил соединение с темой и работает в эксклюзивном режиме потребления, означая, что все сообщения в теме будут потреблены потребителем-A. В текущий момент потребитель-A подтвердил только четыре первых сообщения, поэтому для позиции его курсора в подписке Sub-A сейчас установлено значение 5.

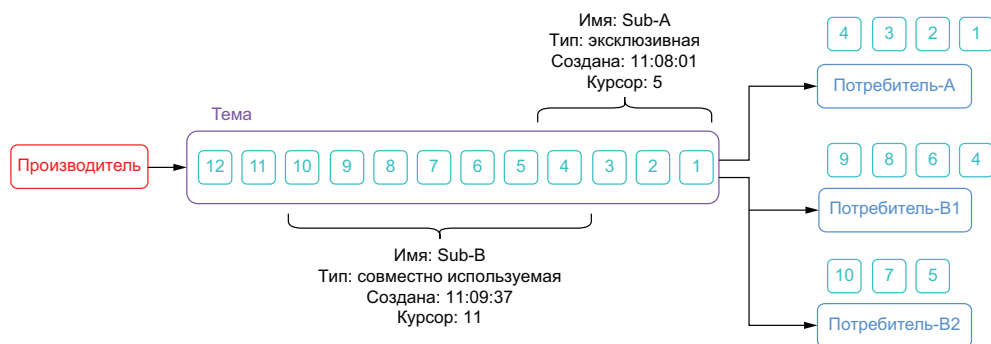


Рис. 2.11. Pulsar поддерживает несколько подписок в одной теме, позволяя многочисленным потребителям считывать одни и те же данные. Потребитель-A обработал первые четыре сообщения в эксклюзивной подписке Sub-A, тогда как сообщения 4–10 распределены между двумя потребителями в совместно используемой подписке Sub-B

Подписка с именем Sub-B создана после того, как были произведены первые три сообщения, поэтому ни одно из этих сообщений не доставлено потребителям данной подписки. Широко распространено заблуждение о том, что любая подписка, созданная в теме, начинает обработку с самого первого сообщения, поэтому я выбрал для демонстрации именно такой случай, чтобы показать, что вы будете принимать только те сообщения, которые публикуются в теме после создания подписки.

На рис. 2.11 также можно видеть, что, поскольку Sub-B работает в совместно используемом режиме, сообщения распределены между всеми потребителями в группе, т. е. каждое сообщение должно быть обработано одним потребителем в группе. Также очевидно, что курсор подписки Sub-B расположен дальше, чем курсор Sub-A, что вполне обычно, когда сообщения распределяются по нескольким потребителям.

2.2.4. Типы подписки

В Pulsar все потребители используют подписки для потребления данных из темы. Подписки (subscriptions) – это просто правила конфигурации, определяющие, как доставляются сообщения потребителям конкретной темы. Подписки Pulsar могут совместно использоваться несколькими приложениями, и в действительности большинство типов подписки спроектировано специально для этого паттерна использования. Pulsar поддерживает четыре различных режима подписки: эксклюзивную (exclusive), отказоустойчивую (failover), совместно используемую по ключу (key-shared) и совместно используемую (shared), как показано на рис. 2.12.

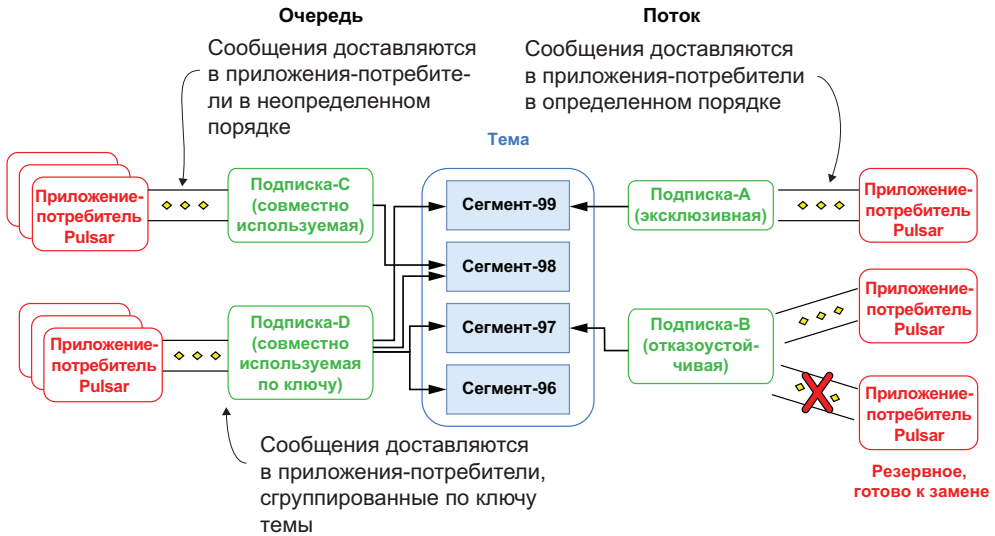


Рис. 2.12. Режимы подписки Pulsar

Каждый тип подписки Pulsar обслуживает отличающийся от других тип варианта использования, поэтому важно понимать это для правильного применения. Вернемся к сценарию, в котором компания, предоставляющая финансовые услуги, в реальном времени направляет поток информации о котировках фондового рынка в тему *stock quotes*, но при этом необходимо совместное использование этой информации всей организацией в целом. Рассмотрим, как каждый из вышеперечисленных режимов подписки можно было бы применить для таких вариантов использования.

Эксклюзивный режим

Эксклюзивная подписка разрешает только одному потребителю доступ к сообщениям конкретной подписки. Если любой другой потребитель пытается подписаться на тему в той же подписке, то будет сгенерировано исключение, и соединение не будет установлено. Этот режим применяется, когда необходимо гарантировать, что каждое сообщение обрабатывается один и только один раз и исключительно известным потребителем.

В примере с организацией, предоставляющей финансовые услуги, группа обработки и анализа данных должна использовать этот тип подписки для передачи данных темы котировок в свои модели машинного обучения, чтобы тренировать и проверять их. Такой режим позволяет обрабатывать записи точно в том порядке, в котором они были приняты, и организовать поток котировок акций в правильной временной последовательности. Для каждой модели потребуется собственная эксклюзивная подписка, как показано на рис. 2.13, чтобы получить отдельную копию данных.

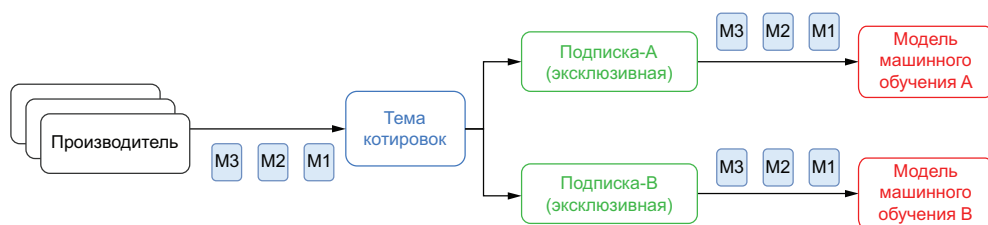


Рис. 2.13. Эксклюзивная подписка позволяет только одному потребителю обрабатывать сообщения

Отказоустойчивые подписки

Отказоустойчивые подписки позволяют нескольким потребителям подключаться к конкретной подписке, но только один потребитель выбирается для приема сообщений. Такая конфигурация позволяет предоставить резервного потребителя для продолжения обработки сообщений в теме, если в первом потребителе возникает критическая ошибка. Если в активном потребителе случается отказ при обработке сообщения, то Pulsar автоматически подключает следующего потребителя в списке и продолжает доставку сообщений.

Этот тип подписки полезен, когда требуется единая семантика обработки с обеспечением высокой доступности потребителей. Это удобно, если необходимо, чтобы приложение продолжало обработку сообщений при системном сбое, и следующий потребитель должен заменить первого при критической ошибке по любой причине. Обычно такие потребители распределяются по разным хостам и/или центрам данных для гарантии того, что приложение сохранит работоспособность при многочисленных авариях. На рис. 2.14 можно видеть, что активным является потребитель-А, тогда как потребитель-В – резервный (готов к замене), он должен быть следующим в процессе приема сообщений, если по какой-либо причине соединение с потребителем-А будет разорвано.

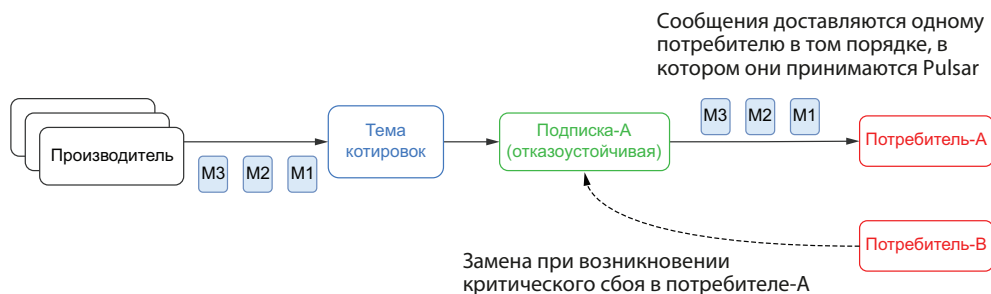


Рис. 2.14. Отказоустойчивая подписка одновременно работает только с одним активным потребителем, но разрешает существование нескольких резервных потребителей

Подходящий пример: если группа обработки и анализа данных все той же компании, предоставляющей финансовые услуги, вернула одну из своих моделей с использованием данных из темы `stock quotes` с генерацией оценок изменения фондового рынка, объединенных с оценками из других моделей для создания обобщенных рекомендаций для группы трейдинга. Был бы опасным подход, при котором существует только один экземпляр этой модели, который постоянно работает для помощи группе трейдинга в принятии решений на основе сгенерированной информации. Но наличие нескольких экземпляров, одновременно работающих и генерирующих рекомендации, может исказить обобщенную рекомендацию.

Совместно используемые подписки

Совместно используемые подписки также позволяют нескольким потребителям подключаться к конкретной подписке, и каждый потребитель может активно принимать сообщения в отличие от отказоустойчивых подписок, которые поддерживают одновременно только одного активного потребителя. Сообщения доставляются по методике круговой циклической (карусельной) диспетчеризации (`robin-round`) всем зарегистрированным потребителям, но любое конкретное сообщение доставляется только одному потребителю, как показано на рис. 2.15.

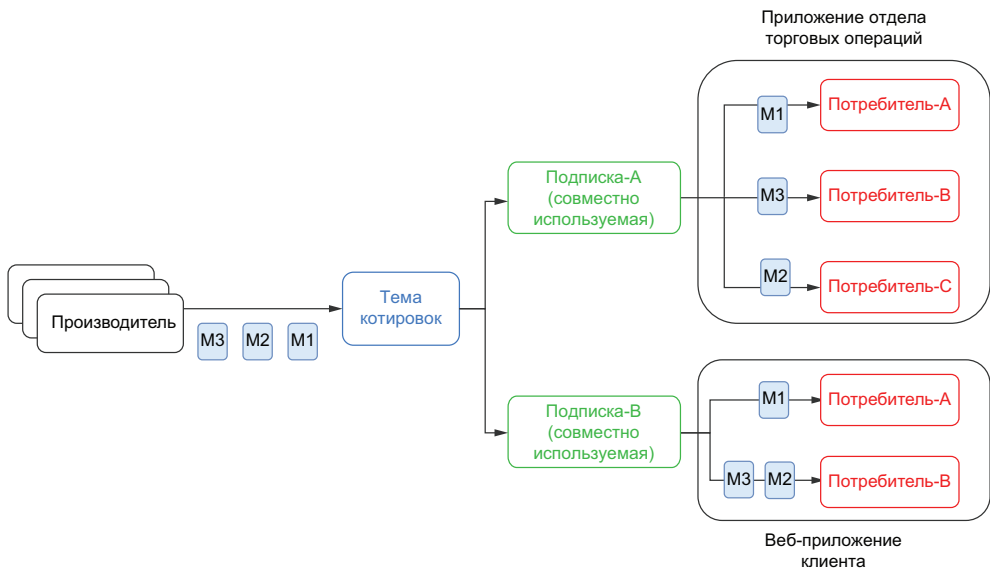


Рис. 2.15. Сообщения распределяются по всем потребителям совместно используемой подписки

Этот тип подписки удобен для реализации очередей рабочих заданий, где порядок сообщений неважен, поскольку позволяет быстро

масштабировать количество потребителей в теме для обработки входящих сообщений. При таком подходе нет ограничений сверху по количеству потребителей в каждой совместно используемой подписке, что обеспечивает расширение потребления посредством увеличения числа потребителей сверх некоторого искусственно заданного граничного значения, определяемого уровнем хранения.

В нашей воображаемой компании, предоставляющей финансовые услуги, самые важные для бизнеса приложения – внутренние платформы трейдинга, системы алгоритмического трейдинга и веб-сайт для взаимодействия с клиентами – должны воспользоваться преимуществами такого типа подписки. Любое приложение должно работать с собственной совместно используемой подпиской, как показано на рис. 2.15, для гарантии того, что каждое из них получит все сообщения, опубликованные в теме котировок.

Совместно используемые по ключу подписки

Подписка, совместно используемая по ключу, также допускает несколько одновременно работающих потребителей, но в отличие от совместно используемой подписки, которая распределяет сообщения по круговой циклической (карусельной) диспетчеризации (round-robin), здесь добавляется вторичный ключ-индекс, гарантирующий, что сообщения с одинаковым ключом будут доставлены одному конкретному потребителю. Эта подписка действует как распределенная опция GROUP BY в языке SQL, где данные с одинаковыми ключами группируются вместе. Это особенно удобно в тех случаях, когда необходимо предварительно рассортировать данные перед потреблением.

Рассмотрим сценарий, в котором группе бизнес-анализа необходимо выполнить некоторый анализ данных в теме котировок. При использовании совместно используемой по ключу подписки обеспечивается полная уверенность в том, что все данные с заданным биржевым кодом будут обработаны одним конкретным потребителем, как показано на рис. 2.16, что упрощает объединение этих данных с другими потоками.

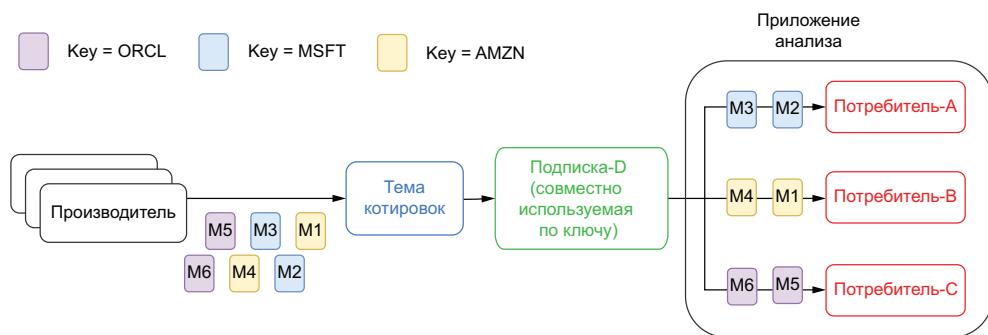


Рис. 2.16. В совместно используемой по ключу подписке сообщения группируются по заданному ключу

Подведем итоги: эксклюзивные и отказоустойчивые подписки позволяют работать только одному потребителю в разделе темы в каждой подписке, и это гарантирует, что сообщения потребляются в порядке их приема. Это наилучший способ потоковой обработки в тех случаях, когда требуется строгий порядок данных.

С другой стороны, совместно используемые подписки позволяют работать нескольким потребителям в одном разделе темы. Каждый потребитель в подписке принимает только часть сообщений, публикуемых в теме. Совместно используемые подписки лучше всего подходят для очередей, где соблюдение строгого порядка не обязательно, но требуется высокая пропускная способность.

2.3. Долговременное хранение сообщений и сроки их хранения

Основная функция Pulsar как системы обмена сообщениями – перемещение данных из точки А в точку В. После доставки данных всем предназначенным получателям возникает предположение о том, что теперь нет необходимости хранить доставленные данные. Следовательно, стратегия долговременного хранения данных по умолчанию точно определяется следующим образом: когда сообщение публикуется в теме Pulsar, оно хранится до тех пор, пока не будет подтверждено всеми потребителями этой темы, после чего сообщение удаляется. Это поведение управляется свойствами `defaultRetentionTimeInMinutes` и `defaultRetentionSizeInMB` в файле конфигурации брокера, для которых по умолчанию установлено значение ноль, означающее, что подтвержденные сообщения не должны храниться постоянно.

2.3.1. Долговременное хранение данных

Но Pulsar также поддерживает стратегии долговременного хранения данных на уровне пространств имен, позволяющие изменить поведение по умолчанию в ситуациях, где необходимо хранить данные темы в течение более длительного периода, например если требуется доступ к данным темы в более поздний момент времени через интерфейс пользователя или SQL.

Эти стратегии долговременного хранения определяют, как долго вы сохраняете сообщения в постоянном хранилище после подтверждения их обработки всеми известными потребителями. Подтвержденные сообщения, не защищенные стратегией долговременного хранения, удаляются. Стратегии долговременного хранения определяются комбинацией граничных значений размера и времени и применяются к каждой теме в заданном пространстве имен. Например, если определено предельное значение раз-

мера 100 Гб, то не более 100 Гб данных будет храниться постоянно в каждой теме указанного пространства имен, но после превышения этого лимита сообщения будут постепенно удаляться (начиная с самых старых) до тех пор, пока общий объем данных не станет ниже предельного значения. Аналогично, если задан лимит времени 24 ч, то подтвержденные сообщения во всех темах заданного пространства имен будут храниться не более 24 ч с того момента, как они были приняты брокером.

Стратегии долговременного хранения требуют определения обоих предельных значений – размера и времени, но применяются они независимо друг от друга. Таким образом, если сообщение нарушило хотя бы один из этих лимитов, то оно будет удалено из темы независимо от того, выполняется ли условие другой стратегии.

Если вы определяете стратегию долговременного хранения с лимитом времени 24 ч и лимитом размера 10 Гб для пространства имен E-payments/refunds, как показано в листинге 2.4, то при достижении любого из этих предельных значений, определенных стратегией, данные удаляются. Таким образом, возможен вариант, при котором сообщение, хранящееся менее 24 ч, должно быть удалено, если общий объем данных превышает 10 Гб.

Листинг 2.4. Различные параметры настройки стратегий долговременного хранения данных Pulsar

```
./bin/pulsar-admin namespaces set-retention E-payments/payments \  
--time 24h \  
--size -1 ❶  
  
./bin/pulsar-admin namespaces set-retention E-payments/fraud-detection \  
--time -1 \  
--size 20G ❷  
  
./bin/pulsar-admin namespaces set-retention E-payments/refunds \  
--time 24h \  
--size 10G ❸  
  
./bin/pulsar-admin namespaces set-retention E-payments/gift-cards \  
--time -1 \  
--size -1 ❹
```

- ❶ Сохраняются все сообщения с «возрастом» менее 24 ч без ограничений по размеру.
- ❷ Сохраняется не более 20 Гб данных сообщений без ограничений по времени.
- ❸ Сохраняется не более 10 Гб данных сообщений с «возрастом» менее 24 ч.
- ❹ Сохраняется неограниченное количество сообщений.

Также возможно установить неограниченный размер или время хранения, определив значение -1 для любого параметра при создании стратегии долговременного хранения, а при установке такого значения для обоих параметров в итоге создается стратегия неограниченного хранения для конкретного пространства имен. Из этого следует, что необходимо весьма осторожно использовать эту стратегию, так как данные никогда не будут удаляться из уровня хранения. Убедитесь в том, что обеспечена достаточная емкость хранилища, и/или сконфигурируйте периодическую выгрузку данных в многоуровневое хранилище.

2.3.2. Квоты для журналов регистрации

«Журнал регистрации» (backlog) – это термин, используемый для всех неподтвержденных сообщений в теме, которые обязательно должны быть сохранены в букмекерах до тех пор, пока не будут доставлены всем назначенным получателям. По умолчанию Pulsar хранит все неподтвержденные сообщения бесконечно. Но при этом на уровне пространств имен поддерживаются стратегии квот для журналов регистрации, позволяющие изменить это поведение, чтобы уменьшить пространство, занимаемое неподтвержденными сообщениями в ситуациях, когда один или несколько потребителей переходят в режим офлайн в течение длительного времени из-за системного сбоя.

Квоты для журналов регистрации предназначены для разрешения весьма особенной ситуации, в которой производители темы отправили больше сообщений, чем способны обработать потребители без возникновения еще большего отставания. В таких условиях, вероятнее всего, потребуется предотвращение роста отставания потребителей до такой степени, что они никогда не смогут догнать производителей. При возникновении подобной ситуации необходимо рассмотреть график обработки данных потребителями и обеспечить, чтобы потребитель отказывался от более старых, неактуальных данных, уделяя все внимание более свежим сообщениям, которые можно продолжать обрабатывать в рамках соглашения о гарантированном уровне обслуживания (service level agreement – SLA). Если данные в теме становятся «несвежими», находясь здесь слишком долго, то реализация квоты для журнала регистрации поможет сосредоточить усилия на обработке только самых последних данных посредством ограничения размера журнала регистрации.

В отличие от стратегий срока хранения данных, описанных в предыдущем подразделе и предназначенных для расширения времени жизни подтвержденных сообщений в теме Pulsar, стратегии квот для журналов регистрации предназначены для сокращения времени жизни неподтвержденных сообщений.

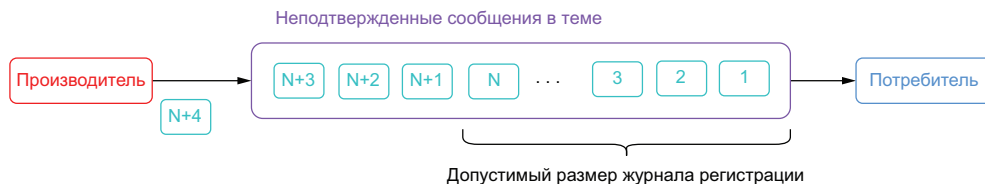


Рис. 2.17. Квота для журнала регистрации Pulsar позволяет определить, какое действие должен выполнить брокер, если объем неподтвержденных сообщений превышает определенное значение размера. Это предотвращает рост журнала регистрации до такой степени, что потребитель обрабатывает малозначительные или вовсе не имеющие ценности данные

Можно ограничить допустимый размер журнала регистрации таких сообщений, сконфигурировав стратегию журнала регистрации, определяющую максимальный разрешенный размер журнала регистрации темы и действие, выполняемое при превышении этого порогового значения, как показано на рис. 2.17. Существуют три возможных варианта такой стратегии, определяющие поведение брокера для приведения к нормальному состоянию:

- брокер может отбросить прибывающие сообщения, отправив производителям исключение, чтобы сообщить о необходимости остановки передачи новых сообщений, посредством определения стратегии регистрации сообщений `producer_request_hold`;
- вместо запроса об остановке передачи производителями брокер принудительно разрывает соединения со всеми существующими производителями, если определена стратегия `producer_exception`;
- если необходимо, чтобы брокер удалил существующие неподтвержденные сообщения из темы, то вы должны определить стратегию `consumer_backlog_eviction`.

Каждый из вышеперечисленных вариантов определяет совершенно различный подход к обработке ситуации, показанной на рис. 2.17. Первый вариант `producer_request_hold` сохраняет соединение производителя, но генерирует исключение для замедления его работы. Такая стратегия применима в сценарии, где необходимо, чтобы клиентское приложение перехватило сгенерированное исключение и повторило отправку сообщения позже. Поэтому лучше всего применять эту стратегию, если нежелательно отбрасывать все сообщения, отправляемые производителем, а клиент должен организовать буферизацию отвергаемых сообщений в течение некоторого интервала времени в ожидании их повторной отправки.

Вторая стратегия `producer_exception` принудительно полностью разрывает соединение с производителем, прекращая публикацию сообщений в теме. При этом требуется код производителя для обнаружения принудительно созданной ситуации и восстановления со-

единения. Такая стратегия создает очевидную возможность потери сообщений, отправляемых клиентами-производителями в то время, когда соединение разорвано. Эту стратегию лучше всего применять, когда известно, что у производителей нет возможности для буферизации сообщений (например, они работают в среде с ограниченными ресурсами, такой как устройство интернета вещей), но нежелательна потеря способности Pulsar принимать сообщения, приводящая к аварийному завершению работы клиентского приложения.

Последняя стратегия `consumer_backlog_eviction` никак не влияет на функциональность производителя, и он продолжает передавать сообщения с текущей скоростью. Но более старые сообщения, которые пока еще не обработаны, будут удаляться, т. е. происходит потеря сообщений.

2.3.3. Окончание срока хранения сообщений

Как уже отмечалось выше, Pulsar хранит все неподтвержденные сообщения бесконечно, и одним из инструментов, который необходимо применять для отмены постоянного хранения таких сообщений, являются квоты для журналов регистрации. Но один из недостатков квот журналов регистрации заключается в том, что они позволяют принять решение о хранении сообщений только на основе размера общего пространства, занимаемого неподтвержденными сообщениями. Как вам известно, одним из главных обоснований ввода квот для журналов регистрации была гарантия того, что потребитель игнорировал устаревшие данные, а все внимание уделял более новым сообщениям. Но было бы больше смысла в том, чтобы существовал способ принудительного удаления исключительно на основе «возраста» самих сообщений. Именно здесь начинают действовать стратегии окончания срока хранения сообщений.

Pulsar поддерживает стратегии времени жизни (*time to live – TTL*) на уровне пространств имен, позволяющие автоматически удалять сообщения, если они остаются неподтвержденными после завершения определенного интервала времени. Окончание срока хранения сообщений полезно в ситуациях, когда для приложения-потребителя более важна обработка самых новых данных, чем их полная хронология. Подходящий пример: данные о местонахождении водителя, предъявляемые пользователям приложения сервиса совместных поездок, когда водитель следует по маршруту. Потребителя больше интересует самая последняя локация водителя, чем все прочие места, в которых он находился несколько минут назад. Таким образом, информация о местонахождении водителя, хранящаяся более пяти минут, становится неактуальной и должна быть удалена, чтобы позволить пользователям обрабатывать только более новые данные, а не пытаться справиться с обработкой сообщений, которые больше не являются полезными.

Листинг 2.5. Настройка стратегий квот журнала регистрации и окончания срока хранения сообщений

```
./bin/pulsar-admin namespaces set-backlog-quota E-payments/payments \
--limit 2G
--policy producer_request_hold ❶
```

```
./bin/pulsar-admin namespaces set-message-ttl E-payments/payments \
--messageTTL 120 ❷
```

❶ Определяет квоту журнала регистрации с ограничением размера 2 Гб и стратегией `producer_request_hold`.

❷ Устанавливает время жизни (TTL) сообщения равным 120 с.

Любое пространство имен может иметь назначенную для него квоту журнала регистрации и стратегию TTL для обеспечения более точного управления сроком хранения неподтвержденных сообщений, расположенных в теме Pulsar, как показано в листинге 2.5.

2.3.4. Сравнение журнала регистрации сообщений и определения срока хранения сообщений

Механизмы длительного хранения данных и определения срока хранения сообщений решают две совершенно различные задачи. На рис. 2.18 можно видеть, что стратегии длительного хранения данных применяются только к подтвержденным сообщениям, т. е. сохраняются только те сообщения, которые соответствуют стратегии длительного хранения. Механизм определения завершения срока хранения применяется только к неподтвержденным сообщениям и управляется параметром настройки TTL, т. е. любые сообщения, которые не обработаны и не подтверждены в течение этого срока, удаляются и не обрабатываются.

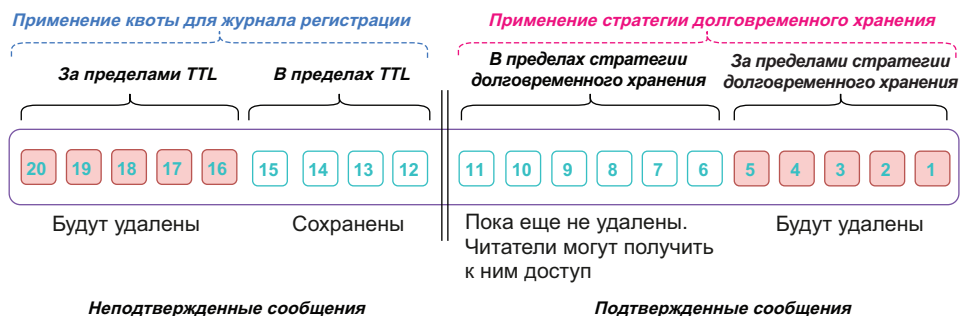


Рис. 2.18. Квота журнала регистрации применяется только к сообщениям, которые не были подтверждены всеми подписками, и она основывается на параметре настройки TTL, тогда как стратегия длительного хранения применяется только к подтвержденным сообщениям и основана на объеме хранимых данных

Стратегии длительного хранения данных могут использоваться в сочетании с многоуровневым хранилищем (tiered storage) для

поддержки неограниченного хранения сообщений особо важных наборов данных, которые необходимо сохранять на неопределенный срок для резервного копирования/восстановления, порождения событий или исследования средствами SQL.

2.4. Многоуровневое хранилище

Функция многоуровневого хранилища (tiered storage) в Pulsar позволяет выгрузить более старые данные темы в более экономичное долговременное хранилище, освобождая дисковое пространство на узлах букмекеров. Для конечного пользователя нет никаких различий между потреблением темы, данные которой хранятся в Apache BookKeeper или в многоуровневом хранилище. Клиенты продолжают производить и потреблять сообщения тем же способом, а внутренний процесс остается невидимым.

Как уже было отмечено ранее, Apache Pulsar хранит темы как упорядоченный список реестров, распределенных по букмекерам на уровне хранения. Поскольку в эти реестры можно лишь добавлять записи, новые сообщения записываются только в самый последний реестр в списке. Все предыдущие реестры опечатаны, поэтому данные в соответствующем сегменте являются неизменяемыми. Так как данные не изменяются, существует возможность простого копирования их в другую систему хранения, например в облачное хранилище.

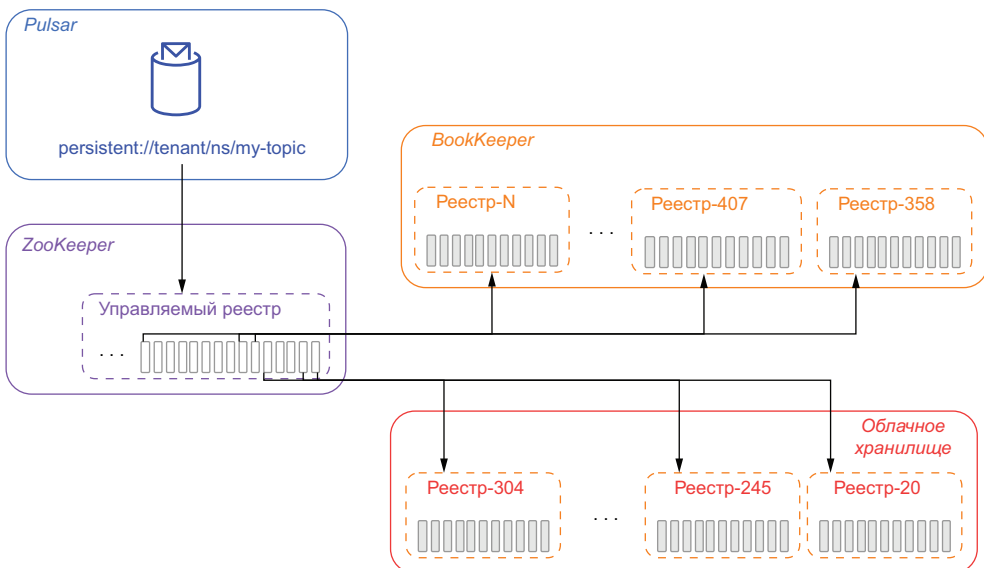


Рис. 2.19. При использовании многоуровневого хранилища реестры, которые были закрыты (и опечатаны), можно скопировать в облачное хранилище и удалить из букмекеров, чтобы освободить пространство. Записи управляемого реестра обновляются для отображения нового местонахождения перемещенных реестров, тем самым обеспечивается возможность продолжения их чтения потребителями темы

После завершения копирования информация в управляемом реестре может быть обновлена для отображения новой локации хранения данных, как показано на рис. 2.19, а исходные копии данных, находящиеся в Apache BookKeeper, можно удалить. При выгрузке во внешнюю систему хранения реестры копируются поочередно, от самого старого до самого нового.

В настоящее время Apache Pulsar поддерживает несколько облачных систем для многоуровневого хранилища, но в этом разделе особое внимание будет уделено системе AWS. В документации Pulsar вы более подробно узнаете, как использовать другие облачные системы хранения.

Конфигурация выгрузки данных в AWS

Первые шаги, которые необходимо выполнить, – создание S3-контейнера, используемого для хранения выгружаемых реестров, и обеспечение для соответствующего аккаунта AWS прав доступа, достаточных для чтения и записи данных в этот контейнер. После выполнения всех условий потребуется еще раз изменить параметры конфигурации брокера, как показано в листинге 2.6.

Листинг 2.6. Конфигурирование многоуровневого хранилища AWS в Pulsar

```
managedLedgerOffloadDriver=aws-s3           ❶  
s3ManagedLedgerOffloadBucket=offload-test-aws ❷  
s3ManagedLedgerOffloadRegion=us-east-1       ❸  
s3ManagedLedgerOffloadRole=<aws role arn>    ❹  
s3ManagedLedgerOffloadRoleSessionName=pulsar-s3-offload ❺
```

- ❶ Определяет тип драйвера выгрузки как AWS S3.
- ❷ Имя S3-контейнера, используемого для хранилища реестров.
- ❸ Регион AWS, в котором расположен контейнер.
- ❹ Если необходимо, чтобы в механизме выгрузки предполагалась роль IAM для выполнения работы, используйте это свойство.
- ❺ Определение имени сеанса, используемое при предполагаемой роли IAM.

Потребуется добавить параметры настройки, специализированные для AWS, чтобы сообщить Pulsar, где сохранять реестры внутри S3. После добавления этих параметров сохраните файл конфигурации и перезапустите брокеры Pulsar, чтобы изменения вступили в силу.

Аутентификация в AWS

Для выгрузки данных из Pulsar в S3 необходима процедура аутентификации в AWS с использованием корректного набора реквизитов. Возможно, вы уже заметили, что Pulsar не предоставляет никаких средств конфигурирования аутентификации для AWS. Вместо этого Pulsar полагается на стандартные механизмы, поддерживаемые

средством `DefaultAWSCredentialsProviderChain`, которое выполняет поиск реквизитов AWS в различных предварительно определенных локациях.

Если брокер работает в экземпляре AWS с экземпляром профиля, предоставляющего требуемые реквизиты, то Pulsar будет использовать эти реквизиты, если не предоставлен какой-либо другой механизм. Иначе можно предоставить собственные реквизиты через переменные среды. Самый простой способ сделать это – отредактировать файл `conf/pulsar_env.sh` и экспортировать переменные среды `AWS_ACCESS_KEY_ID` и `AWS_SECRET_ACCESS_KEY`, добавив соответствующие инструкции, показанные в листинге 2.7.

Листинг 2.7. Предоставление реквизитов для AWS через переменные среды

```
# Добавьте эти инструкции в начало файла pulsar_env.sh
export AWS_ACCESS_KEY_ID=ABC123456789
export AWS_SECRET_ACCESS_KEY=ded7db27a4558e2ea8bbf0bf37ae0e8521618f366c

# или можно выполнить настройку здесь.
PULSAR_EXTRA_OPTS="${PULSAR_EXTRA_OPTS} ${PULSAR_MEM} ${PULSAR_GC}
-Daws.accessKeyId=ABC123456789
-Daws.secretKey=ded7db27a4558e2ea8bbf0bf37ae0e8521618f366c
-Dio.netty.leakDetectionLevel=disabled
-Dio.netty.recycler.maxCapacity.default=1000
-Dio.netty.recycler.linkCapacity=1024"
```

Необходимо использовать один из двух методов, показанных в листинге 2.7. Оба варианта работают одинаково успешно, поэтому выбор за вами. Но оба метода подразумевают потенциальный рис. с точки зрения безопасности, так как реквизиты AWS становятся видимыми в строке описания процесса, если выполнить команду `ps` в ОС Linux. Если вы предпочитаете отказаться от такого сценария, то можете сохранить свои реквизиты в обычной локации для файлов AWS: `~/.aws/credentials` (см. листинг 2.8), а права доступа к этому файлу можно установить только для чтения пользователем, который будет запускать брокер Pulsar (например, `root` – суперпользователь). Но такой подход потребует хранения незашифрованных реквизитов в файле на диске, что также подразумевает потенциальный рис. с точки зрения безопасности, поэтому не рекомендуется для использования в производственной среде.

Листинг 2.8. Содержимое файла `~/.aws/credentials`

```
[default]
aws_access_key_id=ABC123456789
aws_secret_access_key=ded7db27a4558e2ea8bbf0bf37ae0e8521618f366c
```

Конфигурирование автоматической выгрузки данных

Создание конфигурации механизма выгрузки в управляемом реестре еще не означает, что выгрузка действительно произойдет. Остается необходимость в определении стратегии уровня пространства имен для обеспечения автоматической выгрузки данных при превышении определенного порогового значения, которое основано на общем объеме данных в теме Pulsar, содержащихся на уровне хранения BookKeeper.

Листинг 2.9. Конфигурирование автоматической выгрузки данных в многоуровневое хранилище

```
/pulsar/bin/pulsar-admin namespaces set-offload-threshold \  
-size 10GB \  
E-payments/payments
```

Можно определить стратегию, показанную в листинге 2.9, которая устанавливает пороговое значение 10 Гб для всех тем в указанном пространстве имен. После того как объем хранилища темы достигнет значения 10 Гб, срабатывает механизм выгрузки всех закрытых сегментов. Установка порогового значения равным нулю приведет к тому, что брокер будет выгружать реестры настолько энергично, насколько это возможно. Такой режим можно использовать для минимизации объема данных темы, хранящихся в BookKeeper. Отрицательное пороговое значение полностью запрещает автоматическую выгрузку и может использоваться для тем со строго регламентированным временем ответа по SLA, где недопустимы дополнительные задержки, связанные с чтением данных из многоуровневого хранилища.

Многоуровневое хранилище может использоваться при наличии темы, данные которой необходимо хранить в течение весьма длительного интервала времени. Пример: данные о перемещениях по веб-сайту, взаимодействующему с клиентами. Эта информация должна сохраняться достаточно долго на тот случай, если потребуется выполнить анализ поведения пользователей при взаимодействии с клиентами, чтобы определить паттерны их поведения.

Хотя многоуровневое хранилище часто используется в сочетании с темами, стратегии долговременного хранения которых допускают огромные объемы данных, это не является обязательным требованием. В действительности многоуровневые хранилища могут использоваться для любой темы.

2.5. Резюме

Мы рассмотрели логическую структуру адресного пространства Pulsar, обеспечивающую поддержку многоарендной архитектуры (архитектуры с многими абонентами).

- Мы подробно рассмотрели различия между механизмом долговременного хранения сообщений и механизмом определения окончания срока хранения в Pulsar.
- Были описаны подробности низкого уровня, связанные с методами хранения и обслуживания сообщений в Pulsar.

3

Взаимодействие с Pulsar

Темы:

- запуск локального экземпляра Pulsar на компьютере разработчика;
- администрирование кластера Pulsar с использованием его инструментов командной строки;
- взаимодействие с Pulsar с использованием клиентских библиотек на языках Java, Python и Go;
- устранение проблем в Pulsar с использованием его инструментов командной строки.

После подробного изучения общей архитектуры и терминологии Apache Pulsar начнем его практическое использование. Для локальной разработки и тестирования я рекомендую запускать Pulsar внутри контейнера Docker на вашем компьютере, чтобы обеспечить упрощенное начало работы с минимальными затратами времени, усилий и прочих ресурсов. Тем, кто предпочитает использовать полноразмерный кластер Pulsar, следует обратиться к приложению А, чтобы более подробно узнать, как установить и запустить кластер в контейнерной среде, такой как Kubernetes. В этой главе будет подробно и последовательно описан процесс программной отправки и приема сообщений с использованием Java API, начиная с процедуры создания пространства имен Pulsar и темы с применением административных инструментальных средств Pulsar.

3.1. Начинаем работать с Pulsar

Для локальной разработки и тестирования можно запустить Pulsar внутри контейнера Docker на вашем компьютере. Если у вас пока еще не установлен Docker, то следует скачать версию сообщества (<https://www.docker.com/community-edition>) и выполнить инструкции для вашей операционной системы. Если вы не знакомы с контейнерами Docker, сообщая: это проект с открытым исходным кодом для автоматизации развертывания приложений как переносимых самодостаточных образов, которые можно запускать единственной командой. Каждый образ Docker содержит комплект всех отдельных программных компонентов, необходимых для работы всего приложения в целом и объединенных в единый модуль развертывания. Например, образ Docker для простого веб-приложения должен содержать веб-сервер, базу данных и код приложения, короче говоря, все, что требуется для работы приложения. Существует также образ Docker, включающий брокер Pulsar и необходимые компоненты ZooKeeper и BookKeeper.

Разработчики программного обеспечения могут создавать образы Docker и публиковать их в централизованном репозитории Docker Hub. При выгрузке образа можно определить тег, однозначно идентифицирующий его. Это позволит пользователям быстро найти и загрузить требуемую версию образа на свои компьютеры для разработки.

Чтобы начать работу с контейнером Pulsar Docker, просто выполните команду, показанную в листинге 3.1, которая загрузит образ контейнера и запустит все необходимые компоненты. Обратите внимание: в команде задана пара портов (6650 и 8080), которые будут открыты на вашем локальном компьютере. Эти порты необходимы для взаимодействия с кластером Pulsar далее в этой главе.

Листинг 3.1. Запуск Pulsar на настольном компьютере

```
docker pull apache/pulsar-standalone ❶

docker run -d \
  -p 6650:6650 -p 8080:8080 \           ❷
  -v $PWD/data:/pulsar/data \         ❸
  --name pulsar \                      ❹
  apache/pulsar-standalone             ❺
```

- ❶ Скачивание самой последней версии из репозитория DockerHub.
- ❷ Конфигурирование перенаправления для указанных портов.
- ❸ Сохранение данных на локальном диске.
- ❹ Определение имени контейнера.
- ❺ Тег для автономного образа.

Если Pulsar запущен успешно, то вы должны обнаружить сообщения INFO-уровня в файле журнала контейнера Pulsar, свидетельствующие о готовности к работе сервиса обмена сообщениями, как показано в листинге 3.2. К файлу журнала Docker можно получить доступ с помощью команды `docker log`, которая также позволит обнаружить любые проблемы, если контейнер не начал работу.

Листинг 3.2. Проверка работоспособности кластера Pulsar

```
$docker logs pulsar | grep "messaging service is ready"
```

```
20:11:45.834 [main] INFO  org.apache.pulsar.broker.PulsarService -  
➔ messaging service is ready
```

```
20:11:45.855 [main] INFO  org.apache.pulsar.broker.PulsarService -  
➔ messaging service is ready, bootstrap service port = 8080,  
➔ broker url= pulsar://localhost:6650, cluster=standalone
```

Эти сообщения в журнале показывают, что брокер Pulsar активизирован, работает и принимает соединения с портом 6650 на вашем локальном компьютере, предназначенном для разработки. Таким образом, все примеры кода в этой главе будут использовать URL `pulsar://localhost:6650` для отправки и приема данных в брокере Pulsar.

3.2. Администрирование Pulsar

Pulsar предоставляет единый уровень администрирования, позволяющий выполнять функции администратора для всего экземпляра Pulsar в целом, включая все подкластеры, из одной конечной точки. Уровень администрирования Pulsar управляет аутентификацией и авторизацией для всех абонентов, стратегий изоляции ресурсов, квот хранения и многого другого, как показано на рис. 3.1.

Административный интерфейс обеспечивает создание и управление всеми разнообразными компонентами в кластере Pulsar, такими как абоненты, пространства имен и темы, а также конфигурировать для них различные стратегии обеспечения безопасности и долговременного хранения данных. Пользователи могут взаимодействовать с административным интерфейсом с помощью инструмента командной строки `pulsar-admin` или программно через Java API, как показано на рис. 3.1.

При запуске локального автономного кластера Pulsar автоматически создает общедоступного абонента (public tenant) с пространством имен под названием `default`, которое можно использовать для разработки. Но это не соответствует реальному производственному сценарию, поэтому мы подробно рассмотрим процедуру создания абонента и пространства имен.



Рис. 3.1. Представление административного уровня Pulsar

3.2.1. Создание абонента, пространства имен и темы

Pulsar предоставляет интерфейс командной строки (command-line interface – CLI) с доступным инструментальным средством `pulsar-admin`, расположенном в подкаталоге `bin` локации установки Pulsar, а в нашем случае это локация внутри контейнера Docker. Таким образом, для использования инструмента `pulsar-admin` необходимо выполнить команду внутри работающего контейнера Docker. К счастью, Docker предоставляет возможность сделать это с помощью команды `docker exec`. По названию можно понять, что такая команда выполняет заданную инструкцию внутри самого контейнера, а не на локальном компьютере. Начать использование `pulsar-admin` можно, выполнив последовательность команд, показанную в листинге 3.3, после создания темы `persistent://manning/chapter03/example-topic`, с которой мы будем работать в этой главе.

Листинг 3.3. Команды `pulsar-admin`

```
docker exec -it pulsar /pulsar/bin/pulsar-admin clusters list
"standalone" 1
```

```
docker exec -it pulsar /pulsar/bin/pulsar-admin tenants list
"public"
"sample" 2
```

```
docker exec -it pulsar /pulsar/bin/pulsar-admin tenants create manning ❸
```

```
docker exec -it pulsar /pulsar/bin/pulsar-admin tenants list ❹  
"manning"  
"public"  
"sample"
```

```
docker exec -it pulsar /pulsar/bin/pulsar-admin namespaces  
➔ create manning/chapter03 ❺
```

```
docker exec -it pulsar /pulsar/bin/pulsar-admin namespaces list manning  
"manning/chapter03" ❻
```

```
docker exec -it pulsar /pulsar/bin/pulsar-admin topics create ❼  
➔ persistent://manning/chapter03/example-topic ❼
```

```
docker exec -it pulsar /pulsar/bin/pulsar-admin topics list manning/  
➔ chapter03 ❽  
"persistent://manning/chapter03/example-topic" ❽
```

- ❶ Список всех кластеров в этом экземпляре Pulsar.
- ❷ Список всех абонентов в этом экземпляре Pulsar.
- ❸ Создание нового абонента с именем `manning`.
- ❹ Подтверждение создания нового абонента.
- ❺ Создание нового пространства имен `chapter03` в абоненте `manning`.
- ❻ Список пространств имен в абоненте `manning`.
- ❼ Создание новой темы.
- ❽ Список тем в пространстве имен `manning/chapter03`.

Эти команды дают лишь первое представление о том, что можно делать с помощью инструментального средства `pulsar-admin`, поэтому я настоятельно рекомендую обратиться к онлайн-документации (<https://pulsar.apache.org/docs/3.1.x/>), чтобы более подробно узнать о работе с `pulsar-admin` и возможностях этого инструмента. Мы еще вернемся к `pulsar-admin` немного позже в этой главе, чтобы получить некоторые метрики производительности кластера после публикации нескольких сообщений.

3.2.2. API администрирования на языке Java

Другой способ управления экземпляром Pulsar доступен через API администрирования на языке Java, который предоставляет программный интерфейс для выполнения задач администрирования. В листинге 3.4 показано, как создать тему `persistent://manning/chapter03/example-topic`, используя Java API. Этот интерфейс является альтернативой инструменту командной строки и особенно удобен при модульном тестировании, когда необходимо создать и разобрать на части требуемые темы Pulsar программным способом, а не применять внешние инструменты.

Листинг 3.4. Использование API администрирования на языке Java

```

import org.apache.pulsar.client.admin.PulsarAdmin;
import org.apache.pulsar.common.policies.data.TenantInfo;

public class CreateTopic {
    public static void main(String[] args) throws Exception {
        PulsarAdmin admin = PulsarAdmin.builder()
            .serviceHttpUrl("http://localhost:8080")
            .build();

        TenantInfo config = new TenantInfo(
            Stream.of("admin").collect(
                Collectors.toCollection(HashSet::new)),
            Stream.of("standalone").collect(
                Collectors.toCollection(HashSet::new)));

        admin.tenants().createTenant("manning", config);
        admin.namespaces().createNamespace("manning/chapter03");
        admin.topics().createNonPartitionedTopic(
            "persistent://manning/chapter03/example-topic");
    }
}

```

- ❶ Создание клиента-администратора для кластера Pulsar, работающего внутри Docker.
- ❷ Определение ролей администратора для этого абонента.
- ❸ Определение кластеров, с которыми может работать этот абонент.
- ❹ Создание абонента.
- ❺ Создание пространства имен.
- ❻ Создание темы.

3.3. Клиенты Pulsar

Pulsar предоставляет инструментальное средство командной строки `pulsar-client`, позволяющее отправлять и принимать сообщения из темы в работающем кластере Pulsar. Этот инструмент также находится в подкаталоге `bin` локации установки Pulsar, следовательно, снова необходимо использовать команду `docker exec` для взаимодействия с ним.

Поскольку тема уже создана, можно сразу начать с операции подключения к ней потребителя, которая установит подписку и гарантирует, что никакие сообщения не будут потеряны. Это можно сделать, выполнив команды, показанные в листинге 3.5. Потребителем является блокирующий скрипт (blocking script) – он продолжает потребление сообщений из темы до тех пор, пока не будет остановлен вами (с помощью комбинации клавиш **Ctrl+C**).

Листинг 3.5. Запуск потребителя из командной строки

```

$ docker exec -it pulsar /pulsar/bin/pulsar-client consume \
persistent://manning/chapter03/example-topic \
--num-messages 0 \
--subscription-name example-sub \
--subscription-type Exclusive

INFO org.apache.pulsar.client.impl.ConnectionPool - [[id: 0xe410f77d,
➔ L:/127.0.0.1:39276 - R:localhost/127.0.0.1:6650]] Connected to server
18:08:15.819 [pulsar-client-io-1-1] INFO
➔ org.apache.pulsar.client.impl.ConsumerStatsRecorderImpl - Starting Pulsar
➔ consumer perf with config: {
  "topicNames" : [ ],
  "topicsPattern" : null,
  "subscriptionName" : "example-sub",
  "subscriptionType" : "Exclusive",
  "receiverQueueSize" : 1000,
  "acknowledgementsGroupTimeMicros" : 100000,
  "negativeAckRedeliveryDelayMicros" : 60000000,
  "maxTotalReceiverQueueSizeAcrossPartitions" : 50000,
  "consumerName" : "3d7ce",
  "ackTimeoutMillis" : 0,
  "tickDurationMillis" : 1000,
  "priorityLevel" : 0,
  "cryptoFailureAction" : "FAIL",
  "properties" : { },
  "readCompacted" : false,
  "subscriptionInitialPosition" : "Latest",
  "patternAutoDiscoveryPeriod" : 1,
  "regexSubscriptionMode" : "PersistentOnly",
  "deadLetterPolicy" : null,
  "autoUpdatePartitions" : true,
  "replicateSubscriptionState" : false,
  "resetIncludeHead" : false
}
...

18:08:15.980 [pulsar-client-io-1-1] INFO
➔ org.apache.pulsar.client.impl.MultiTopicsConsumerImpl -
➔ [persistent://manning/chapter02/example] [example-sub] Success
➔ subscribe new topic persistent://manning/chapter02/example in topics
➔ consumer, partitions: 2, allTopicPartitionsNumber: 2
18:08:47.644 [pulsar-client-io-1-1] INFO
➔ com.scurrilous.circe.checksum.Crc32cIntChecksum - SSE4.2 CRC32C
➔ provider initialized

```

- 1 Имя темы, из которой выполняется потребление.
- 2 Количество сообщений для потребления. 0 означает потребление без ограничений.
- 3 Специальное имя подписки.
- 4 Тип подписки.

- 5 Подробности конфигурации потребителя.
- 6 Здесь можно видеть имя подписки, определенное в командной строке.
- 7 Здесь можно видеть тип подписки, определенный в командной строке.
- 8 Начало потребления с самого последнего доступного сообщения.

В другом экземпляре командной оболочки запускается производитель командой, показанной в листинге 3.6, для отправки двух сообщений, содержащих текст "Hello Pulsar", в ту же тему, к которой только что был подключен потребитель.

Листинг 3.6. Отправка сообщений с использованием производителя в командной строке Pulsar

```
$ docker exec -it pulsar /pulsar/bin/pulsar-client produce \
persistent://manning/chapter03/example-topic \
--num-produce 2 \
--messages "Hello Pulsar"
18:08:47.106 [pulsar-client-io-1-1] INFO
➔ org.apache.pulsar.client.impl.ConnectionPool - [[id: 0xd47ac4ea,
➔ L:/127.0.0.1:39342 - R:localhost/127.0.0.1:6650]] Connected to server
18:08:47.367 [pulsar-client-io-1-1] INFO
➔ org.apache.pulsar.client.impl.ProducerStatsRecorderImpl - Starting
➔ Pulsar producer perf with config: {
  "topicName" : "persistent://manning/chapter02/example",
  "producerName" : null,
  "sendTimeoutMs" : 30000,
  "blockIfQueueFull" : false,
  "maxPendingMessages" : 1000,
  "maxPendingMessagesAcrossPartitions" : 50000,
  "messageRoutingMode" : "RoundRobinPartition",
  "hashingScheme" : "JavaStringHash",
  "cryptoFailureAction" : "FAIL",
  "batchingMaxPublishDelayMicros" : 1000,
  "batchingMaxMessages" : 1000,
  "batchingEnabled" : true,
  "compressionType" : "NONE",
  "initialSequenceId" : null,
  "autoUpdatePartitions" : true,
  "properties" : { }
}
...
18:08:47.689 [main] INFO org.apache.pulsar.client.cli.PulsarClientTool - 2
➔ messages successfully produced
```

- 1 Имя темы, в которой публикуются сообщения.
- 2 Количество операций отправки сообщения.
- 3 Содержимое сообщения.
- 4 Подробности конфигурации производителя.
- 5 Публикация сообщений.

После выполнения команды запуска производителя из листинга 3.6 вы должны увидеть строки, похожие на код, приведенный в листинге 3.7, в том экземпляре командной оболочки, в котором был запущен потребитель. Это свидетельствует о том, что сообщения были успешно опубликованы производителем и приняты потребителем.

Листинг 3.7. Прием сообщений в экземпляре командной оболочки потребителя

```
----- got message -----  
key:[null], properties:[], content:Hello Pulsar  
----- got message -----  
key:[null], properties:[], content:Hello Pulsar
```

Поздравляю, вы только что успешно отправили свои первые сообщения с использованием Pulsar. После подтверждения того, что наш локальный кластер Pulsar работает и способен передавать и принимать сообщения, рассмотрим несколько более реалистичных примеров с использованием различных языков программирования. Pulsar предоставляет простой, интуитивно понятный API клиента, включающий все подробности обмена данными брокер-клиент для пользователя. Широкая распространенность Pulsar определяет существование нескольких реализаций клиента на различных языках, включая Java, Go, Python и C++, но это не полный список. Это позволяет каждой группе разработчиков в любой организации использовать тот язык, который они предпочитают для реализации своих сервисов.

Хотя существуют существенные различия в функциях, поддерживаемых официальными клиентскими библиотеками Pulsar, в зависимости от выбранного языка программирования (более подробную информацию см. в официальной документации клиента), внутри все они поддерживают прозрачное повторное подключение и/или переключение соединения с брокерами, размещение сообщений в очереди до тех пор, пока они не будут подтверждены брокером, а также эвристики, например повторные попытки установления соединения с отсрочкой. Это позволяет разработчику сосредоточиться на логике обмена сообщениями, а не на обработке в коде приложения исключений, возникающих в соединениях.

3.3.1. Клиент Pulsar на языке Java

В дополнение к API администрирования на языке Java, описанному в предыдущих разделах этой главы, Pulsar также предоставляет клиентский интерфейс на языке Java, который можно использовать для создания производителей, потребителей и читателей сообщений. Самая свежая версия библиотеки клиента Pulsar для

Java доступна в централизованном репозитории Maven. Чтобы воспользоваться последней версией, просто добавьте клиентскую библиотеку Pulsar в конфигурацию сборки, как показано в листинге 3.8. После добавления этой библиотеки в свой проект вы можете начать ее использование для взаимодействия с Pulsar, создавая клиентов, производителей и потребителей непосредственно в коде Java, как будет показано в следующем разделе.

Листинг 3.8. Добавление клиентской библиотеки Pulsar в проект Maven

```
<!-- Inside your pom.xml -->
<properties>
  <pulsar.version>2.7.2</pulsar.version>
</properties>
<dependency>
  <groupId>org.apache.pulsar</groupId>
  <artifactId>pulsar-client</artifactId>
  <version>${pulsar.version}</version>
</dependency>
```

Создание конфигурации клиента Pulsar в коде Java

Если в приложении необходимо создать производителя или потребителя, то сначала потребуется создать объект `PulsarClient`, используя код, подобный показанному в листинге 3.9. В этом объекте определяется URL брокера Pulsar, а также вся прочая информация о конфигурации соединения, которая может потребоваться, например секретные реквизиты.

Листинг 3.9. Создание объекта `PulsarClient` в коде Java

```
PulsarClient client = PulsarClient.builder()
    .serviceUrl("pulsar://localhost:6650")
    .build();
```

❶

❶ URL для установления соединения с брокером Pulsar.

Объект `PulsarClient` обрабатывает все подробности низкого уровня, связанные с созданием соединения с брокером Pulsar, включая автоматическое повторное соединение и обеспечение безопасности, если брокер сконфигурирован с использованием защищенного протокола TLS. Для экземпляров клиентов обеспечивается безопасность потоков, и их можно использовать для создания нескольких производителей и потребителей и управления ими.

Производители Pulsar в коде Java

В Pulsar производители используются для записи сообщений в темы. В листинге 3.10 показано, как можно создать производителя в коде Java с указанием имени темы, в которую будут отправ-

ляться сообщения. Хотя существует несколько параметров конфигурации, определяемых при создании потребителя, обязательным требованием является только имя темы.

Листинг 3.10. Создание производителя Pulsar в коде Java

```
Producer<byte[]> producer = client.newProducer()
    .topic("persistent://manning/chapter03/example-topic")
    .create();
```

Кроме того, существует возможность присоединения метаданных к конкретному сообщению, как показано в листинге 3.11. В этом коде демонстрируется, как определить ключ сообщения, применяемый для маршрутизации в совместно используемой по ключу подписке, вместе с другими свойствами сообщения. Такой возможностью можно воспользоваться для пометки сообщения тегом, содержащим полезную информацию, например время отправки сообщения, отправитель, идентификатор устройства, если сообщение отправлено с встроенного сенсорного датчика, и т. п.

Листинг 3.11. Запись метаданных в сообщения Pulsar

```
Producer<byte[]> producer = client.newProducer()
    .topic("persistent://manning/chapter03/example-topic")
    .create();

producer.newMessage()
    .key("temperture-readings")           ❶
    .value("98.0".getBytes())             ❷
    .property("deviceId", "1234")          ❸
    .property("timestamp", "08/03/2021 14:48:24.1")
    .send();
```

- ❶ Можно определить ключ сообщения.
- ❷ Передача содержимого сообщения как массива байтов.
- ❸ Можно присоединить столько свойств, сколько необходимо.

Значения метаданных, присоединенных к сообщению, будут доступны потребителям, которые в дальнейшем могут использовать эту информацию при выполнении логики обработки. Например, свойство, содержащее значение метки времени, т. е. время отправки сообщения, можно использовать для сортировки входящих сообщений в хронологическом порядке публикации или для корреляции с сообщениями из другой темы.

Потребители Pulsar в коде Java

В Pulsar интерфейс потребителей используется для прослушивания конкретной темы и обработки входящих сообщений. После успешной обработки сообщения должно быть отправлено под-

тверждение обратно в брокер, чтобы оповестить его о том, что мы выполнили обработку сообщения в данной подписке. Это позволяет брокеру узнать, какие сообщения в теме необходимо доставить следующему потребителю в той же подписке. В коде Java можно создать потребителя, определив тему и подписку, как показано в листинге 3.12.

Листинг 3.12. Создание потребителя Pulsar в коде Java

```
Consumer consumer = client.newConsumer()  
    .topic("persistent://manning/chapter03/example-topic") ❶  
    .subscriptionName("my-subscription") ❷  
    .subscribe();
```

- ❶ Определение темы, из которой будет выполняться потребление.
- ❷ Необходимо указать однозначно определяемое имя для подписки.

Метод подписки пытается установить соединение с потребителем темы, использующим конкретную подписку, но это может привести к ошибке, если эта подписка уже существует и не принадлежит к одному из типов совместно используемых (например, вы пытаетесь установить соединение с эксклюзивной подпиской, для которой уже имеется активный потребитель). Если вы устанавливаете соединение с темой в первый раз, используя конкретное имя подписки, то указанная подписка создается автоматически. При создании новой подписки она сначала позиционируется в конце темы по умолчанию, и потребители в ней начинают чтение с первого сообщения, созданного после создания этой подписки. Если вы устанавливаете соединение с уже существующей подпиской, то чтение начнется с самого раннего неподтвержденного сообщения в ней, как показано на рис. 3.2. Такой подход гарантирует, что вы продолжите чтение с того места, где оно было прервано в том случае, если соединение вашего потребителя с темой было неожиданно разорвано.



Рис. 3.2. Потребитель начинает чтение непосредственно после самого позднего подтвержденного сообщения в подписке. Если подписка новая, то потребитель начинает чтение сообщений, добавленных в тему после создания этой подписки

Наиболее часто применяемый паттерн потребления – наличие потребителя, прослушивающего тему в цикле `while`. В листинге 3.13 потребитель непрерывно прослушивает сообщения, выводит содержимое всех принятых сообщений, а затем подтверждает их обработку. Если в логике обработки возникает ошибка, то используется отрицательное подтверждение (`negative acknowledgement`) для повторной доставки сообщения в более позднее время.

Листинг 3.13. Потребление сообщений Pulsar на языке Java

```
while (true) {  
    // Ожидание сообщения.  
    Message msg = consumer.receive();  
  
    try {  
        System.out.println("Message received: " +  
                           new String(msg.getData()));  
        consumer.acknowledge(msg);  
    } catch (Exception e) {  
        consumer.negativeAcknowledge(msg);  
    }  
}
```

- ❶ Ожидание сообщения.
- ❷ Обработка сообщения.
- ❸ Подтверждение сообщения, чтобы брокер мог удалить его.
- ❹ Пометка сообщения для повторной доставки.

Потребитель, показанный в листинге 3.13, обрабатывает сообщения в синхронном режиме, потому что метод `receive()`, применяемый для извлечения сообщений, является блокирующим (т. е. бесконечно ожидает прибытия нового сообщения). Хотя это может оказаться вполне приемлемым для некоторых вариантов использования с небольшим объемом сообщений или для случаев, когда не имеет значения задержка между публикацией и обработкой сообщений, но в общем случае синхронная обработка не является наилучшей методикой. Более эффективный подход – обработка сообщений в асинхронном режиме на основе интерфейса `MessageListener`, предоставляемого Java API, как показано в листинге 3.14.

Листинг 3.14. Асинхронная обработка сообщений на языке Java

```
package com.manning.pulsar.chapter3.consumers;  
  
import java.util.stream.IntStream;  
  
import org.apache.pulsar.client.api.ConsumerBuilder;
```

```

import org.apache.pulsar.client.api.PulsarClient;
import org.apache.pulsar.client.api.PulsarClientException;
import org.apache.pulsar.client.api.SubscriptionType;

public class MessageListenerExample {

    public static void main() throws PulsarClientException {
        PulsarClient client = PulsarClient.builder()
            .serviceUrl(PULSAR_SERVICE_URL)
            .build();

        ConsumerBuilder<byte[]> consumerBuilder =
            client.newConsumer()
                .topic(MY_TOPIC)
                .subscriptionName(SUBSCRIPTION)
                .subscriptionType(SubscriptionType.Shared)
                .messageListener((consumer, msg) -> {
                    try {
                        System.out.println("Message received: " +
                            new String(msg.getData()));
                        consumer.acknowledge(msg);
                    } catch (PulsarClientException e) {
                        // ...
                    }
                })

        IntStream.range(0, 4).forEach(i -> {
            String name = String.format("mq-consumer-%d", i);
            try {
                consumerBuilder
                    .consumerName(name)
                    .subscribe();
            } catch (PulsarClientException e) {
                e.printStackTrace();
            }
        });

        // ...
    }
}

```

- ❶ Клиент, используемый для соединения с Pulsar.
- ❷ Фабрика потребителей, которая будет использоваться для создания экземпляров потребителей в дальнейшем.
- ❸ Бизнес-логика для выполнения после приема сообщения.
- ❹ Создание пяти потребителей в этой теме, в каждом из которых применяется одинаковая реализация `MessageListener`.
- ❺ Установление соединения потребителя с темой для начала приема сообщений.

При использовании интерфейса `MessageListener`, показанного в листинге 3.14, в коде описывается то, что необходимо выполнить после приема сообщения. В данном случае я воспользовался лямбда-функцией Java для обеспечения встроенного (inline) кода, и вы можете видеть, что при этом сохранен доступ к потребителю, который можно использовать для подтверждения сообщения. Применение паттерна `listener` (слушатель) позволяет отделить бизнес-логику от управления потоками, так как потребитель Pulsar автоматически создает пул потоков для запуска экземпляров `MessageListener` и обрабатывает всю логику многопоточности. Объединив все вышеописанное, мы получаем программу на языке Java, показанную в листинге 3.15, которая создает экземпляр клиента Pulsar и использует его для создания производителя и потребителя, обменивающихся сообщениями в теме `my-topic`.

Листинг 3.15. Пара производитель–потребитель Pulsar, бесконечно обменивающаяся сообщениями

```
import org.apache.pulsar.client.api.Consumer;
import org.apache.pulsar.client.api.Message;
import org.apache.pulsar.client.api.Producer;
import org.apache.pulsar.client.api.PulsarClient;
import org.apache.pulsar.client.api.PulsarClientException;

public class BackAndForth {

    public static void main(String[] args) throws Exception {
        BackAndForth sl = new BackAndForth();
        sl.startConsumer();
        sl.startProducer();
    }
    private String serviceUrl = "pulsar://localhost:6650";
    String topic = "persistent://manning/chapter03/example-topic";
    String subscriptionName = "my-sub";

    protected void startProducer() {
        Runnable run = () -> {
            int counter = 0;
            while (true) {
                try {
                    getProducer().newMessage()
                        .value(String.format("{id: %d, time: %tc}",
                            ++counter, new Date()).getBytes())
                        .send();
                    Thread.sleep(1000);
                } catch (final Exception ex) { }
            }
        };
        new Thread(run).start();
    }
}
```

```

protected void startConsumer() {
    Runnable run = () -> {
        while (true) {
            Message<byte[]> msg = null;
            try {
                msg = getConsumer().receive();
                System.out.printf("Message received: %s \n",
                    new String(msg.getData()));
                getConsumer().acknowledge(msg);
            } catch (Exception e) {
                System.err.printf(
                    "Unable to consume message: %s \n", e.getMessage());
                consumer.negativeAcknowledge(msg);
            }
        }
    };
    new Thread(run).start();
}

protected Consumer<byte[]> getConsumer() throws PulsarClientException {
    if (consumer == null) {
        consumer = getClient().newConsumer()
            .topic(topic)
            .subscriptionName(subscriptionName)
            .subscriptionType(SubscriptionType.Shared)
            .subscribe();
    }
    return consumer;
}

protected Producer<byte[]> getProducer() throws PulsarClientException {
    if (producer == null) {
        producer = getClient().newProducer()
            .topic(topic).create();
    }
    return producer;
}

protected PulsarClient getClient() throws PulsarClientException {
    if (client == null) {
        client = PulsarClient.builder()
            .serviceUrl(serviceUrl)
            .build();
    }
    return client;
}
}

```

Здесь можно видеть, что код создает производителя и потребителя в одной теме и одновременно запускает их в отдельных потоках. Если выполнить этот код, то вы должны увидеть вывод результата, похожий на приведенный в листинге 3.16.

Листинг 3.16. Вывод результата работы пары производитель – потребитель Pulsar, бесконечно обменивающейся сообщениями

```
Message received: {id: 1, time: Sun Sep 06 16:24:04 PDT 2020}  
Message received: {id: 2, time: Sun Sep 06 16:24:05 PDT 2020}  
Message received: {id: 3, time: Sun Sep 06 16:24:06 PDT 2020}  
...
```

Обратите внимание: первые два сообщения, отправленные ранее, не включены в вывод, поскольку подписка была создана после публикации этих сообщений. Это прямая противоположность интерфейсу Reader, который мы скоро рассмотрим.

Стратегия недоставленных сообщений

Существует несколько вариантов конфигурации для потребителя Pulsar, описанных в онлайн-документации (<https://pulsar.apache.org/docs/3.1.x/client-libraries-java/#configure-consumer>), но я хотел бы специально выделить конфигурацию `dead-letter-policy` (стратегия недоставленных сообщений), особенно полезную, когда встречаются сообщения, которые невозможно успешно обработать, например при парсинге неструктурированных сообщений из темы. При обычных условиях обработки такие сообщения могут приводить к генерации исключения.

В этот момент есть несколько возможных вариантов: первый – перехватывать любые исключения и просто подтверждать такие сообщения как успешно обработанные, т. е. фактически игнорировать их. Другой вариант – отправлять отрицательное подтверждение для их повторной доставки. Но такой подход в итоге может привести к бесконечному циклу повторной доставки подобных сообщений, если внутреннюю проблему, связанную с ними, устранить невозможно (например, сообщение не поддается парсингу и всегда будет генерировать исключение вне зависимости от того, сколько раз вы будете пытаться его обработать). Третий вариант – перенаправление (маршрутизация) таких проблемных сообщений в отдельную тему недоставленных сообщений (`dead-letter topic`). Это позволяет избежать возникновения бесконечного цикла повторной доставки, но сохраняет проблемные сообщения для дальнейшей обработки и/или исследования в более позднее время.

Листинг 3.17. Конфигурирование стратегии темы недоставленных сообщений в потребителе

```
Consumer consumer = client.newConsumer()  
    .topic("persistent://manning/chapter03/example-topic")  
    .subscriptionName("my-subscription")  
    .deadLetterPolicy(DeadLetterPolicy.builder()  
        .maxRedeliverCount(10)  
        .deadLetterTopic("persistent://manning/chapter03/my-dlq"))  
    .subscribe();
```

①
②

- 1 Установка счетчика максимального количества операций повторной доставки.
- 2 Настройка имени темы недоставленных сообщений.

При конфигурировании стратегии недоставленных сообщений для конкретного потребителя Pulsar требует определить несколько свойств, например счетчик максимального количества операций повторной доставки, при первоначальном создании, как показано в листинге 3.17. После превышения максимального значения этого счетчика, определенного пользователем, сообщение будет передано в тему недоставленных сообщений и подтверждено автоматически. В дальнейшем такие сообщения можно будет исследовать более подробно.

Читатели Pulsar на языке Java

Интерфейс Reader (читатель) позволяет приложениям управлять позициями, с которых они будут потреблять сообщения. Если вы устанавливаете соединение с темой, используя интерфейс читателя, то должны непременно определить, с какого сообщения читатель начнет потребление сообщений с момента установления соединения с темой. Короче говоря, интерфейс читателя предоставляет клиентам Pulsar абстракцию низкого уровня, позволяющую вручную управлять самими позициями в теме, как показано на рис. 3.3.



Рис. 3.3. При установлении соединения с темой интерфейс читателя позволяет начать с самого раннего доступного сообщения, с самого позднего доступного сообщения или с сообщения с предоставленным приложением идентификатором

Интерфейс читателя полезен для таких вариантов, как использование Pulsar для обеспечения семантики однократно эффективной (effectively-once) обработки для системы потоковой обработки данных. Для такого варианта использования чрезвычайно важно, чтобы система потоковой обработки имела возможность «быстрой перемотки» тем к заданному сообщению, чтобы начать с него чтение. Если вы выбираете явное предоставление иденти-

фикатора сообщения, то ваше приложение становится ответственным за знание идентификатора такого сообщения в дальнейшем, возможно, с извлечением его из постоянного хранилища данных или кеша. После создания экземпляра объекта `PulsarClient` можно создать `Reader`, как показано в листинге 3.18.

Листинг 3.18. Создание читателя Pulsar

```
Reader<byte[]> reader = client.newReader()
    .topic("persistent://manning/chapter03/example-topic")      ❶
    .readerName("my-reader")
    .startMessageId(MessageId.earliest)                          ❷
    .create();

while (true) {
    Message msg = reader.readNext();
    System.out.printf("Message received: %s \n", new String(msg.getData()));
}
```

- ❶ Определение темы, из которой необходимо выполнять чтение.
- ❷ Здесь определяется, что необходимо начать чтение с самого раннего сообщения.

После выполнения этого кода вы должны увидеть вывод, похожий на приведенный в листинге 3.19. Чтение должно начинаться с самых первых сообщений, опубликованных в этой теме, т. е. с двух сообщений "Hello Pulsar", которые мы отправили из командной строки.

Листинг 3.19. Вывод читателя, начиная с самого раннего сообщения

```
Message read: Hello Pulsar
Message read: Hello Pulsar
Message read: {id: 1, time: Sun Sep 06 18:11:59 PDT 2020}
Message read: {id: 2, time: Sun Sep 06 18:12:00 PDT 2020}
Message read: {id: 3, time: Sun Sep 06 18:12:01 PDT 2020}
Message read: {id: 4, time: Sun Sep 06 18:12:02 PDT 2020}
Message read: {id: 5, time: Sun Sep 06 18:12:04 PDT 2020}
Message read: {id: 6, time: Sun Sep 06 18:12:05 PDT 2020}
Message read: {id: 7, time: Sun Sep 06 18:12:06 PDT 2020}
```

В примере, показанном в листинге 3.18, читатель создается в заданной теме и выполняет итеративный проход по ней, начиная с самого старого сообщения в этой теме. Для читателя существует несколько вариантов конфигурации, описанных в онлайн-документации (<https://pulsar.apache.org/docs/3.1.x/client-libraries-java/#reader>), но в большинстве случаев вполне достаточно определенных по умолчанию параметров.

3.3.2. Клиент Pulsar на языке Python

Существует также официально поддерживаемый клиент Pulsar для языка программирования Python. Самую последнюю версию клиентской библиотеки Pulsar можно без затруднений установить с помощью менеджера пакетов `pip` командой, показанной в листинге 3.20.

Листинг 3.20. Установка клиентской библиотеки Pulsar на языке Python

```

pip3 install pulsar-client==2.6.3 -user ❶

pip3 list ❷

Package      Version
-----
...
pulsar-client 2.6.3 ❸

```

❶ Установка клиента Pulsar.

❷ Вывод списка всех пакетов.

❸ Подтверждение установки корректной версии клиента Pulsar.

Поскольку версия Python 2.7 уже официально завершила свое существование, я решил использовать версию Python 3 во всех примерах этой главы. После установки клиентских библиотек Pulsar вы можете начать их использование для взаимодействия с Pulsar, создавая производителей и потребителей в коде Python.

Производители Pulsar на языке Python

Если в приложении на языке Python требуется создание производителя или потребителя, то сначала необходимо создать экземпляр объекта клиента, используя код, подобный показанному в листинге 3.21, где предоставляется URL брокера Pulsar. Как и в клиентах на языке Java, объект клиента на языке Python обрабатывает все подробности низкого уровня, связанные с созданием соединения с брокером Pulsar, включая автоматическое восстановление и обеспечение безопасности соединения, если брокер Pulsar сконфигурирован по протоколу TLS. Для экземпляров клиентов обеспечена потокобезопасность, и они могут многократно использоваться для управления несколькими производителями и потребителями.

Листинг 3.21. Создание производителя Pulsar на языке Python

```

import pulsar

client = pulsar.Client('pulsar://localhost:6650') ❶

producer = client.create_producer(
    'persistent://public/default/my-topic',

```

```

        block_if_queue_full=True,
        batching_enabled=True,
        batching_max_publish_delay_ms=10)
    2
    for i in range(10):
        producer.send(('Hello-%d' % i).encode('utf-8'),
        3
            properties=None)
        4
    client.close()
    5

```

- ❶ Создание клиента Pulsar с предоставлением для установления соединения URL брокера Pulsar.
- ❷ Использование клиента Pulsar для создания производителя.
- ❸ Отправка содержимого сообщения.
- ❹ К сообщению можно присоединить свойства, если это необходимо.
- ❺ Закрытие клиента.

В листинге 3.21 можно видеть, что библиотека Python предоставляет несколько различных вариантов конфигурации для создания клиентов, производителей и потребителей, поэтому рекомендуется изучить доступную онлайн-документацию (<https://pulsar.apache.org/api/python/3.2.x/>) для клиента, чтобы узнать больше об этих конфигурациях. В приведенном выше примере мы разрешили объединение сообщений в пакеты на стороне клиента, т. е. вместо обмена отдельными сообщениями с брокером сообщения перед передачей будут объединяться в пакеты. Это позволяет увеличить общую пропускную способность за счет более длительной задержки доставки каждого отдельного сообщения.

Потребители Pulsar на языке Python

В Pulsar интерфейс потребителя используется для прослушивания в конкретной теме и для обработки входящих сообщений. После успешной обработки сообщения в брокер должно быть возвращено подтверждение, оповещающее, что мы завершили обработку сообщения в соответствующей подписке. Это позволяет брокеру точно знать, какое сообщение в теме должно быть доставлено следующему потребителю в той же подписке. На языке Python можно создать потребителя, определив тему и подписку, как показано в листинге 3.22.

Листинг 3.22. Создание потребителя Pulsar на языке Python

```

import pulsar

client = pulsar.Client('pulsar://localhost:6650')
    1

consumer = client.subscribe(
    'persistent://public/default/my-topic',
    2
    'my-subscription',
    3
    4

```

```

consumer_type=pulsar.ConsumerType.Exclusive
initial_position=pulsar.InitialPosition.Latest,
message_listener=None,
negative_ack_redelivery_delay_ms=60000)

while True:
    msg = consumer.receive()
    try:
        print("Received message '%s' id='%s'",
              msg.data().decode('utf-8'), msg.message_id())
        consumer.acknowledge(msg)
    except:
        consumer.negative_acknowledge(msg)

client.close()

```

- ❶ Создание клиента Pulsar с предоставлением для установления соединения URL брокера Pulsar.
- ❷ Использование клиента Pulsar для создания потребителя.
- ❸ Необходимо обязательно определить тему, из которой будет выполняться потребление.
- ❹ Необходимо обязательно определить особое неповторяющееся имя вашей подписки.
- ❺ Ожидание прибытия нового сообщения.
- ❻ После успешной обработки сообщения необходимо подтвердить его.
- ❼ Если возникла ошибка, отправляется отрицательное подтверждение для повторной отправки сообщения.
- ❽ Закрытие клиента.

Метод `subscribe` пытается установить соединение потребителя с темой, используя заданное имя подписки, но при этом возможна ошибка, если такая подписка уже существует и не принадлежит к одному из совместно используемых типов (например, вы пытаетесь установить соединение с эксклюзивной подпиской, для которой уже имеется активный потребитель). Если вы устанавливаете соединение с темой в первый раз, используя конкретное имя подписки, то указанная подписка создается автоматически. При создании новой подписки она сначала позиционируется в конце темы по умолчанию, и потребители в ней начинают чтение с первого сообщения, принятого после создания этой подписки. Если вы устанавливаете соединение с уже существующей подпиской, то чтение начнется с самого раннего неподтвержденного сообщения в ней, как было показано выше на рис. 3.2.

В листинге 3.22 можно видеть, что библиотека Python предоставляет несколько различных вариантов конфигурации для определения подписки, в том числе ее типа, начальной позиции и т. п., поэтому настоятельно рекомендуется изучить доступную онлайн-документацию (<https://pulsar.apache.org/api/python/3.2.x/>) для клиента

на языке Python, чтобы получить самую свежую информацию об этих конфигурациях.

Потребитель, показанный в листинге 3.22, обрабатывает сообщения в синхронном режиме, потому что метод `receive()`, применяемый для извлечения сообщений, является блокирующим (т. е. бесконечно ожидает прибытия нового сообщения). Более эффективный подход – обработка сообщений в асинхронном режиме, как показано в листинге 3.23. Применение паттерна `listener` (слушатель) позволяет отделить бизнес-логику от управления потоками, так как потребитель Pulsar автоматически создает пул потоков для запуска экземпляров слушателей и обрабатывает всю логику многопоточности.

Листинг 3.23. Асинхронная обработка сообщений на языке Python

```
import pulsar

def my_listener(consumer, msg):
    # process message
    print("my_listener read message '%s' id='%s'",
          msg.data().decode('utf-8'), msg.message_id())
    consumer.acknowledge(msg)

client = pulsar.Client('pulsar://localhost:6650')

consumer = client.subscribe(
    'persistent://public/default/my-topic',
    'my-subscription',
    consumer_type=pulsar.ConsumerType.Exclusive,
    initial_position=pulsar.InitialPosition.Latest,
    message_listener=my_listener,
    negative_ack_redelivery_delay_ms=60000)

client.close()
```

- ❶ Функция слушателя должна принимать потребителя и сообщение.
- ❷ Можно получить доступ к содержимому сообщения.
- ❸ Можно использовать потребителя для подтверждения сообщения.
- ❹ Установка слушателя сообщений для конкретного экземпляра потребителя.

Читатели Pulsar на языке Python

Клиент Python также предоставляет интерфейс `Reader` (читатель), позволяющий потребителям управлять позицией, с которой они начнут потреблять сообщения. Если вы устанавливаете соединение с темой, используя интерфейс читателя, то должны непременно определить, с какого сообщения читатель начнет потребление сообщений с момента установления соединения с темой. Если вы выбираете явное предоставление идентификатора сообщения, то ваше приложение становится ответственным за знание идентификатора такого сообщения в дальнейшем и должно сохранять эту

информацию в некотором постоянном хранилище, таком как база данных или кеш. Код в листинге 3.24 устанавливает соединение с темой, начинает чтение с самых ранних доступных сообщений и выводит их содержимое.

Листинг 3.24. Создание читателя Pulsar в коде Python

```
import pulsar

client = pulsar.Client('pulsar://localhost:6650') ❶

reader = client.create_reader(
    'persistent://public/default/my-topic',      ❷
    pulsar.MessageId.earliest)                  ❸

while True:
    msg = reader.read_next()                      ❹
    print("Read message '%s' id='%s'",
          msg.data().decode('utf-8'), msg.message_id())

client.close() ❺
```

- ❶ Создание клиента Pulsar с предоставлением для установления соединения URL брокера Pulsar.
- ❷ Использование клиента Pulsar для создания читателя в заданной теме.
- ❸ Здесь определяется, что необходимо начать чтение с самого раннего сообщения.
- ❹ Чтение сообщений.
- ❺ Заккрытие клиента.

3.3.3. Клиент Pulsar на языке Go

Существует также официально поддерживаемый клиент Pulsar для языка программирования Go¹ (часто используется альтернативное название `golang`), и самую последнюю версию клиентской библиотеки Pulsar можно установить командой `go get -u "github.com/apache/pulsar-client-go/pulsar"`. После установки клиентских библиотек Pulsar вы можете начать их использование для взаимодействия с Pulsar, создавая производителей и потребителей в коде Go.

Создание клиента Pulsar на языке Go

Если в приложении на языке Go требуется создать производителя или потребителя, то сначала необходимо создать экземпляр клиента, используя код, показанный в листинге 3.25. В этом коде предоставляется URL брокера Pulsar вместе с другой информацией о конфигурации соединения, которая может потребоваться, например секретные реквизиты.

¹ Не следует путать с языком Go!, разработанным несколько раньше. – *Прим. перев.*

Листинг 3.25. Создание клиента Pulsar на языке Go

```
import (  
    "log"  
    "time"  
    "github.com/apache/pulsar-client-go/pulsar"  
)  
  
func main() {  
    client, err := pulsar.NewClient(  
        pulsar.ClientOptions{  
            URL: "pulsar://localhost:6650",  
            OperationTimeout: 30 * time.Second,  
            ConnectionTimeout: 30 * time.Second,  
        })  
  
    if err != nil {  
        log.Fatalf("Could not instantiate Pulsar client: %v", err)  
    }  
  
    defer client.Close()  
}
```

- ❶ Импорт библиотеки клиентской библиотеки Pulsar.
- ❷ Создание нового клиента с использованием заданных параметров конфигурации.
- ❸ Параметры конфигурации клиента, включающие URL брокера, тайм-аут соединения и т. п.
- ❹ Проверка возможности установления соединения для этого клиента.

Объект клиента обрабатывает все подробности низкого уровня, связанные с созданием соединения с брокером Pulsar, включая автоматическое восстановление и обеспечение безопасности соединения, если брокер Pulsar сконфигурирован по протоколу TLS. Для экземпляров клиентов обеспечена потокобезопасность, и они могут многократно использоваться для управления несколькими производителями и потребителями. После создания экземпляра клиента его можно использовать для создания производителей, потребителей и читателей.

Производители Pulsar на языке Go

В приведенном ниже листинге 3.26 можно видеть, что после создания объекта клиента он используется для создания производителя в любой теме по вашему выбору. Хотя для производителя Pulsar существует несколько вариантов конфигурации, описанных в онлайн-документации (<https://pkg.go.dev/github.com/apache/pulsar-client-go/pulsar#consumeroptions>), я решил уделить особое внимание конфигурации отложенной доставки сообщений (delayed message delivery), используемой в рассматриваемом здесь примере, кото-

рая позволяет задержать доставку сообщений потребителям темы в течение заданного интервала времени.

Листинг 3.26. Создание производителя Pulsar на языке Go

```
import (
    "context"
    "fmt"
    "log"
    "time"

    "github.com/apache/pulsar-client-go/pulsar"

)

func main() {
    ...
    producer, err := client.CreateProducer(pulsar.ProducerOptions{
        Topic: topicName,
    })

    ctx := context.Background()
    deliveryTime := (time.Minute * time.Duration(1)) +
        (time.Second * time.Duration(30))

    for i := 0; i < 3; i++ {
        msg := pulsar.ProducerMessage{
            Payload: []byte(fmt.Sprintf("Delayed-messageId-%d", i)),
            Key: "message-key",
            Properties: map[string]string{
                "delayed": "90sec",
            },
            EventTime: time.Now(),
            DeliverAfter: deliveryTime,
        }

        messageID, err := producer.Send(ctx, &msg)
        ...
    }
}
```

- ❶ Импортирование клиентской библиотеки Pulsar.
- ❷ Здесь размещается код создания клиента Pulsar.
- ❸ Создание нового производителя для заданной темы.
- ❹ Вычисление времени доставки, требуемого для сообщения.
- ❺ Создание отправляемого сообщения.
- ❻ Клиент на языке Go поддерживает предоставление метаданных в форме ключ-свойства.
- ❼ Предоставление метаданных метки времени события.
- ❽ Определение времени доставки для этого сообщения.
- ❾ Отправка сообщения.

Отложенная доставка сообщений удобна, если вы не хотите, чтобы сообщение обрабатывалось не сразу после получения, а через некоторое время. Рассмотрим сценарий, в котором вы принимаете несколько сообщений, содержащих новые подписки на информационные бюллетени вашей компании с ежедневными специальными предложениями и промоакциями. Вместо того чтобы немедленно рассылать клиентам рекламную листовку предыдущего дня, лучше подождать, пока не станет доступно новое издание. И если ваша маркетинговая группа обязана предоставлять свежую версию информационного бюллетеня каждый день в 9 часов утра, то можно отложить доставку сообщения до 9 часов утра, чтобы гарантировать, что клиенты получают самую свежую версию информационного бюллетеня.

Потребители Pulsar

Как мы уже видели, интерфейс потребителя используется для прослушивания в конкретной теме и для обработки входящих сообщений. После успешной обработки сообщения в брокер должно быть возвращено подтверждение, оповещающее, что мы завершили обработку сообщения в соответствующей подписке. Это позволяет брокеру точно знать, какое сообщение в теме должно быть доставлено следующему потребителю в той же подписке. На языке Go можно создать потребителя, определив тему и подписку, как показано в листинге 3.27.

Потребитель, показанный в листинге 3.27, обрабатывает сообщения в синхронном режиме, потому что метод `receive()`, применяемый для извлечения сообщений, является блокирующим (т. е. бесконечно ожидает прибытия нового сообщения). В отличие от предыдущих двух клиентских библиотек, рассмотренных выше, клиент Go в настоящее время не поддерживает асинхронный режим потребления с использованием паттерна слушателя сообщений. Таким образом, если требуется выполнение асинхронной обработки, то вам придется самому написать всю логику многопоточной обработки¹.

Листинг 3.27. Создание потребителя Pulsar на языке Go

```
import (  
    "context"  
    "fmt"  
    "log"  
    "time"  
  
    "github.com/apache/pulsar-client-go/pulsar"  
)
```

1

¹ В 2023 г. в версии библиотеки 0.10.0 появился тип `ConsumerEventListener` (<https://pkg.go.dev/github.com/apache/pulsar-client-go/pulsar#ConsumerEventListener>). – Прим. перев.

```

func main() {
    ...

    consumer, err := client.Subscribe(pulsar.ConsumerOptions{
        Topic:      topicName,
        SubscriptionName: subscriptionName,
    })

    if err != nil {
        log.Fatal(err)
    }

    for {
        msg, err := consumer.Receive(context.Background())
        if err != nil {
            log.Fatal(err)
            consumer.Nack(msg)
        } else {
            fmt.Printf("Received message : %v\n", string(msg.Payload()))
        }

        consumer.Ack(msg)
    }
}

```

- ❶ Импортирование клиентской библиотеки Pulsar.
- ❷ Здесь размещается код создания клиента Pulsar.
- ❸ Создание нового производителя для заданной темы.
- ❹ Блокирующий вызов для приема входящих сообщений.
- ❺ Отправка отрицательного подтверждения для повторной доставки сообщения.
- ❻ Подтверждение сообщения, чтобы его можно было пометить как обработанное.

Метод `Subscribe` пытается установить соединение потребителя с темой, используя заданное имя подписки, но при этом возможна ошибка, если такая подписка уже существует и не принадлежит к одному из совместно используемых типов (например, вы пытаетесь установить соединение с эксклюзивной подпиской, для которой уже имеется активный потребитель). Если вы устанавливаете соединение с темой в первый раз, используя конкретное имя подписки, то указанная подписка создается автоматически. При создании новой подписки она сначала позиционируется в конце темы по умолчанию, и потребители в ней начинают чтение с первого сообщения, принятого после создания этой подписки. Если вы устанавливаете соединение с уже существующей подпиской, то чтение начнется с самого раннего неподтвержденного сообщения в ней, как было показано выше на рис. 3.2.

В листинге 3.27 можно видеть, что библиотека Python предоставляет несколько различных вариантов конфигурации для определе-

ния подписки, в том числе ее типа, начальной позиции и т. п., поэтому настоятельно рекомендуется изучить доступную онлайн-документацию (<https://pkg.go.dev/github.com/apache/pulsar-client-go/pulsar>) для клиента на языке Go, чтобы получить самую свежую информацию об этих конфигурациях.

Читатели Pulsar

Клиент Go также предоставляет интерфейс `Reader` (читатель), позволяющий потребителям управлять позицией, с которой они начнут потреблять сообщения. Если вы устанавливаете соединение с темой, используя интерфейс читателя, то должны непременно определить, с какого сообщения читатель начнет потребление сообщений с момента установления соединения с темой. Если вы выбираете явное предоставление идентификатора сообщения, то ваше приложение становится ответственным за знание идентификатора такого сообщения в дальнейшем и должно сохранять эту информацию в некотором постоянном хранилище, таком как база данных или кеш. Код в листинге 3.28 устанавливает соединение с темой, начинает чтение с самых ранних доступных сообщений и выводит их содержимое.

Листинг 3.28. Создание читателя Pulsar на языке Go

```
import (  
    "context"  
    "fmt"  
    "log"  
    "time"  
  
    "github.com/apache/pulsar-client-go/pulsar" ❶  
)  
  
func main() {  
    ... ❷  
  
    reader, err := client.CreateReader(pulsar.ReaderOptions{  
        Topic:      topicName,  
        StartMessageID: pulsar.EarliestMessageID(), ❸  
    }) ❹  
  
    for {  
        msg, err := reader.Next(context.Background()) ❺  
        if err != nil {  
            log.Fatal(err)  
        } else {  
            fmt.Printf("Received message : %v\n", string(msg.Payload()))  
        }  
    }  
}
```

- ❶ Импорт библиотеки клиентской библиотеки Pulsar.
- ❷ Здесь размещается код создания клиента Pulsar.
- ❸ Создание нового читателя для заданной темы.
- ❹ Начало чтения с самого раннего доступного сообщения.
- ❺ Чтение следующего сообщения.

3.4. Дополнительное администрирование

Pulsar работает как черный ящик с точки зрения производителя и потребителя (т. е. вы просто устанавливаете соединение с кластером для отправки и приема сообщений). Скрытие подробностей реализации от конечного пользователя является правильным решением, но это может создавать затруднения, когда необходимо устранять проблемы, возникшие непосредственно при доставке сообщений. Например, если потребитель не принимает никакие сообщения, то как можно обнаружить и диагностировать проблему? К счастью, инструмент командной строки `pulsar-admin` предоставляет некоторые средства, позволяющие глубже заглянуть во внутреннюю работу кластера Pulsar.

3.4.1. Метрики постоянно хранимой темы

Во внутренней рабочей среде Pulsar собирает множество метрик уровня темы, которые могут оказаться полезными для диагностики и устранения проблем, возникающих между производителями и потребителями, например если потребитель не принимает никакие сообщения или в ситуации, когда потребители не могут поддерживать темп, заданный производителями, что выражается в росте значения счетчика неподтвержденных сообщений. К статистическим данным темы можно получить доступ с помощью инструмента командной строке `pulsar-admin`, который мы применяли ранее для создания абонента, пространства имен и темы. Необходимо выполнить команду, показанную в листинге 3.29.

Листинг 3.29. Извлечение статистических данных темы Pulsar в командной строке

```
$docker exec -it pulsar /pulsar/bin/pulsar-admin topics stats
➔ persistent://manning/chapter03/example-topic ❶
{
  "msgRateIn" : 137.49506548471038, ❷
  "msgThroughputIn" : 13741.401370605108, ❸
  "msgRateOut" : 97.63210798236112, ❹
  "msgThroughputOut" : 9716.05449008063, ❺
  "bytesInCounter" : 1162174,
  "msgInCounter" : 11538, ❻
  "bytesOutCounter" : 150009,
  "msgOutCounter" : 1500, ❼
}
```

```

"averageMsgSize" : 99.94105113636364,
"msgChunkPublished" : false,
"storageSize" : 1161944,
"backlogSize" : 1161279,
"publishers" : [ {
  "msgRateIn" : 137.49506548471038,
  "msgThroughputIn" : 13741.401370605108,
  "averageMsgSize" : 99.0,
  "chunkedMessageRate" : 0.0,
  "producerId" : 0,
  "metadata" : { },
  "producerName" : "standalone-12-6",
  "connectedSince" : "2020-09-07T20:44:45.514Z",
  "clientVersion" : "2.6.1",
  "address" : "/172.17.0.1:40158"
} ],
"subscriptions" : {
  "my-sub" : {
    "msgRateOut" : 97.63210798236112,
    "msgThroughputOut" : 9716.05449008063,
    "bytesOutCounter" : 150009,
    "msgOutCounter" : 1500,
    "msgRateRedeliver" : 0.0,
    "chuckedMessageRate" : 0,
    "msgBacklog" : 9458,
    "msgBacklogNoDelayed" : 9458,
    "blockedSubscriptionOnUnackedMsgs" : false,
    "msgDelayed" : 0,
    "unackedMessages" : 923,
    "type" : "Shared",
    "msgRateExpired" : 0.0,
    "lastExpireTimestamp" : 0,
    "lastConsumedFlowTimestamp" : 1599511537220,
    "lastConsumedTimestamp" : 1599511537452,
    "lastAckedTimestamp" : 1599511545269,
    "consumers" : [ {
      "msgRateOut" : 97.63210798236112,
      "msgThroughputOut" : 9716.05449008063,
      "bytesOutCounter" : 150009,
      "msgOutCounter" : 1500,
      "msgRateRedeliver" : 0.0,
      "chuckedMessageRate" : 0.0,
      "consumerName" : "5bf2b",
      "availablePermits" : 0,
      "unackedMessages" : 923,
      "avgMessagesPerEntry" : 6,
      "blockedConsumerOnUnackedMsgs" : false,
      "lastAckedTimestamp" : 1599511545269,
      "lastConsumedTimestamp" : 1599511537452,
      "metadata" : { },
      "connectedSince" : "2020-09-07T20:44:45.512Z",
      "clientVersion" : "2.6.1",

```

```

        "address" : "/172.17.0.1:40160"
    } ],
    "isDurable" : true,
    "isReplicated" : false
},
"example-sub" : {
    "msgRateOut" : 0.0,
    "msgThroughputOut" : 0.0,
    "bytesOutCounter" : 0,
    "msgOutCounter" : 0,
    "msgRateRedeliver" : 0.0,
    "chuckedMessageRate" : 0,
    "msgBacklog" : 11528,
    "msgBacklogNoDelayed" : 11528,
    "blockedSubscriptionOnUnackedMsgs" : false,
    "msgDelayed" : 0,
    "unackedMessages" : 0,
    "type" : "Exclusive",
    "msgRateExpired" : 0.0,
    "lastExpireTimestamp" : 0,
    "lastConsumedFlowTimestamp" : 1599509925751,
    "lastConsumedTimestamp" : 0,
    "lastAckedTimestamp" : 0,
    "consumers" : [ ],
    "isDurable" : true,
    "isReplicated" : false
}
},
"replication" : { },
"deduplicationStatus" : "Disabled"
}

```

- ❶ Имя темы, из которой необходимо получить статистические данные.
- ❷ Сумма всех локальных скоростей и скоростей репликации публикаторами, сообщений/с.
- ❸ Сумма всех локальных скоростей и скоростей репликации публикаторами, байт/с.
- ❹ Сумма всех локальных скоростей и скоростей диспетчеризации репликации потребителями, сообщений/с.
- ❺ Сумма всех локальных скоростей и скоростей диспетчеризации репликации, байт/с.
- ❻ Общее количество сообщений, опубликованных в теме.
- ❼ Общее количество сообщений, потребленных из темы.
- ❽ Общий объем дискового пространства, используемого для хранения сообщений темы в байтах.
- ❾ Общая скорость публикации сообщений производителем, сообщений/с.
- ❿ Метка времени первого соединения публикатора (производителя) с темой.
- ⓫ IP-адрес производителя.
- ⓬ Список всех подписок для этой темы.
- ⓭ Общая скорость сообщений, доставленных в эту подписку, байт/с.

- 14 Общее количество сообщений, доставленных в эту подписку.
- 15 Количество пока еще не доставленных сообщений в журнале регистрации этой подписки.
- 16 Количество сообщений, которые уже доставлены, но пока еще не подтверждены.
- 17 Метка времени потребления самого последнего сообщения в этой подписке.
- 18 Метка времени принятия подтверждения самого последнего сообщения в этой подписке.
- 19 Заблокирован или нет потребитель из-за слишком большого количества неподтвержденных сообщений.
- 20 IP-адрес потребителя.
- 0 Признак подписки без активных потребителей.
- 0 Количество сообщений в журнале регистрации подписки.

Здесь можно видеть, что Pulsar собирает обширный набор метрик по каждой постоянно хранящейся теме, который может оказаться весьма полезным при попытке обнаружения и диагностирования проблемы. Возвращаемые метрики включают присоединенных производителей и потребителей, а также все характеристики производства и потребления, данные из журнала регистрации и подписки. Таким образом, если вы пытаетесь определить, почему конкретный потребитель не принимает сообщения, то можно проверить соединение потребителя и скорость потребления сообщений в соответствующей подписке.

Все эти метрики по умолчанию публикуются на платформе мониторинга Prometheus. Их можно с легкостью наблюдать с помощью средства визуализации и анализа Grafana, которое включено в комплект развертывания Pulsar Kubernetes, определенный в Helm chart в составе проекта с открытым исходным кодом. Вы можете сконфигурировать любое средство наблюдения, которое также совместимо с Prometheus.

3.4.2. Инспекция сообщений

Иногда необходимо просмотреть содержимое конкретного сообщения или группы сообщений в теме Pulsar. Рассмотрим сценарий, в котором один из производителей изменяет формат вывода своих сообщений, зашифровывая их содержимое. Потребители, в текущий момент подписанные на эту тему, неожиданно начинают обнаруживать исключения при попытках обработки зашифрованного содержимого, и в результате сообщения не подтверждаются. В итоге такие сообщения накапливаются в теме, поскольку никогда не будут подтверждены. Если это изменение в коде производителя не контролируется вами, то вы не узнаете об истинной причине возникшей внутренней проблемы. К счастью, можно воспользоваться командой `peek-messages` инструмента командной

строки `pulsar-admin` для прямого просмотра байтов сообщения в заданной подписке, как показано в листинге 3.30, демонстрирующем синтаксис выбора последних 10 сообщений из подписки `example-sub` в теме `persistent://manning/chapter03/example-topic`.

Листинг 3.30. Выбор сообщений в теме Pulsar

```
$ docker exec -it pulsar /pulsar/bin/pulsar-admin \
  Topic peek-messages \
  --count 10 \
  --subscription example-sub \
  persistent://manning/chapter03/example-topic

Batch Message ID: 19460:9:0
Tenants:
{
  "X-Pulsar-num-batch-message" : "1",
  "publish-time" : "2020-09-07T20:20:13.136Z"
}

+-----+
| 0 1 2 3 4 5 6 7 8 9 a b c d e f |
+-----+
|00000000| 7b 69 64 3a 20 31 30 2c 20 74 69 6d 65 3a 20 4d |{id: 10, time: M|
|00000010| 6f 6e 20 53 65 70 20 30 37 20 31 33 3a 32 30 3a |on Sep 07 13:20:|
|00000020| 31 33 20 50 44 54 20 32 30 32 30 7d           |13 PDT 2020}|
+-----+
```

- ❶ Запрос последних 10 сообщений.
- ❷ Идентификатор сообщения.
- ❸ Время опубликования сообщения производителем.
- ❹ Содержимое сообщения в форме «сырых» байтов.

Здесь можно видеть, что команда `peek-messages` предоставляет множество подробностей о сообщении, включая его идентификатор, время публикации и содержимое в форме «сырых» байтов (а также в форме строки `String`). Эта информация должна помочь в определении проблемы с содержимым сообщения.

3.5. Резюме

Docker – это контейнерная рабочая среда (фреймворк) с открытым исходным кодом, позволяющая полностью упаковывать приложения в один образ и публиковать этот образ для многократного использования.

- Существует полноценный самодостаточный Docker-образ Pulsar, который можно использовать для запуска Pulsar на локальном компьютере для разработки.

- Pulsar предоставляет инструментальные средства командной строки, которые можно использовать для администрирования абонентов, пространств имен и тем, включая их создание, вывод списка и удаление.
- Pulsar предоставляет клиентские библиотеки для нескольких широко распространенных языков программирования, в том числе Java, Python и Go, что позволяет создавать производителей, потребителей и читателей Pulsar.
- Инструментальные средства командной строки Pulsar можно использовать для извлечения статистических данных о теме, полезных для мониторинга и устранения проблем.

Часть II

Основы разработки с использованием Apache Pulsar

В части II все внимание сосредоточено на встроенной в Pulsar вычислительной среде (фреймворке) без использования сервера под названием Pulsar Functions, а также рассматриваются возможности его применения для обеспечения функций потоковой обработки без необходимости привлечения дополнительного вычислительного фреймворка, такого как Apache Flink или Kafka Streams. Такой тип потоковой обработки без сервера также называют потоковой нативной обработкой (stream-native processing), и она имеет широкий диапазон применения – от ETL в реальном времени и программирования, управляемого событиями, до разработки микросервисов и машинного обучения в реальном времени.

После рассмотрения основ использования фреймворка Pulsar Functions приводится достаточно подробное описание того, как правильно обеспечить безопасность кластера Pulsar для полной

уверенности в том, что ваши данные надежно защищены от посторонних глаз. В последнем разделе части содержится введение в схему реестра Pulsar, которая помогает сохранять информацию о структуре сообщений внутри тем Pulsar в центральной локации.

В главе 4 представлен фреймворк потоковой нативной обработки Pulsar Functions с дополнительной информацией о его проектном решении и конфигурации. Показано, как разрабатывать, тестировать и развертывать отдельные функции. Глава 5 представляет фреймворк коннекторов Pulsar, предназначенный для перемещения данных между Apache Pulsar и внешними системами хранения, такими как реляционные базы данных, хранилища типа ключ-значение или хранилища больших двоичных объектов (blob-объектов). Здесь вы узнаете, как разрабатывается коннектор с пошаговым описанием всех этапов разработки.

В главе 6 приведены пошаговые инструкции по обеспечению безопасности кластера Pulsar для полной уверенности в том, что данные защищены как при передаче, так и при хранении. В последней главе 7 этой части рассматривается встроенная схема реестра, обосновывается ее необходимость и объясняется, как она может упростить разработку микросервиса. Также описан процесс развития схемы и способы обновления схем, используемых в функциях Pulsar.

4

Функции Pulsar

Темы:

- введение в фреймворк Pulsar Functions;
- модель программирования и API Pulsar Functions;
- создание вашей первой функции Pulsar на языке Java;
- конфигурирование, предоставление и мониторинг функции Pulsar.

В предыдущей главе мы рассматривали, как можно работать с Pulsar, используя некоторые из многочисленных клиентских библиотек. В этой главе мы изучим механизм потоковой нативной обработки под названием Pulsar Functions, который существенно упрощает разработку приложений на основе Pulsar. Этот упрощенный фреймворк обработки автоматически обеспечивает создание огромного объема стандартного кода, требуемого для создания и настройки потребителей и производителей Pulsar, позволяя вам сосредоточиться на логике обработки данных, а не на процедурах потребления и обработки сообщений.

4.1. Потоковая обработка

Несмотря на отсутствие официального определения, термин «потоковая обработка» (stream processing) обозначает обработку неограниченных наборов данных, поток которых непрерывно ис-

ходит из некоторой системы-источника. Существует несколько наборов данных, представляющих собой естественные непрерывные потоки, например события сенсорного датчика, активность пользователя на веб-сайте, финансовые операции, которые могут обрабатываться по этой методике.

До появления методики потоковой обработки такие наборы данных сначала должны были сохраняться в базе данных, файловой системе или другом постоянном хранилище, прежде чем можно было начать их анализ. Часто включался дополнительный этап обработки данных, необходимый для извлечения информации, преобразования в корректный формат и загрузки данных в соответствующие системы. Только после завершения процесса ETL (Extracting, Transforming, Loading – извлечение, преобразование, загрузка) данные становились готовыми к анализу с применением традиционных инструментов на основе SQL или других средств. Можете себе представить, насколько существенной была задержка между моментом наступления события и моментом, когда оно становилось доступным для анализа. Главная цель потоковой обработки – минимизировать эту задержку, чтобы можно было принимать самые важные бизнес-решения с учетом самых свежих данных. Существуют три основные методики обработки таких наборов данных – пакетная обработка, микропакетная обработка, потоковая нативная обработка, и в каждой приняты различные подходы относительно того, как и когда обрабатывать эти бесконечные наборы данных.

4.1.1. Обычная пакетная обработка

В течение многих лет подавляющее большинство фреймворков обработки данных было предназначено для пакетной обработки (batch processing). Широко известные хранилища данных Hadoop и Spark – это лишь пара общих примеров систем, обрабатывающих данные в пакетах. Данные часто передаются в такие системы через длинные и сложные конвейеры ETL, которые очищают, готовят и форматируют входящие данные для потребления. Системы обмена сообщениями часто служат не более чем промежуточными буферами, сохраняющими и перенаправляющими данные между различными этапами обработки в конвейере.

Такие сложные конвейеры подготовки данных часто были реализованы с использованием механизмов потоковой обработки, таких как Apache Spark или Flink, предназначенных для эффективной обработки крупных наборов данных в параллельном режиме. Новые входящие элементы данных объединялись, а затем в некоторый момент обрабатывались все вместе как единый пакет. Чтобы увеличить до максимума пропускную способность таких фреймворков, пакетирование данных происходило в течение весьма длительных интервалов времени (часов) или до накопления опре-

деленного объема данных (десятки гигабайт), что создавало искусственную задержку в конвейере обработки данных.

4.1.2. Микропакетная обработка данных

Методика, введенная для устранения задержек в обработке, присущих механизмам обычной пакетной обработки данных, значительно сокращала либо размер пакета, либо интервал обработки. При микропакетной обработке новые входящие элементы данных продолжали объединяться в пакеты, как показано на рис. 4.1, но размер пакетов был существенно уменьшен с помощью сокращения интервала времени до нескольких секунд. Даже если бы обработка могла выполняться более часто, данные продолжали обрабатываться по одному пакету за один раз, и это часто называли микропакетной обработкой (micro-batching), которая использовалась некоторыми фреймворками, например Spark Streaming.

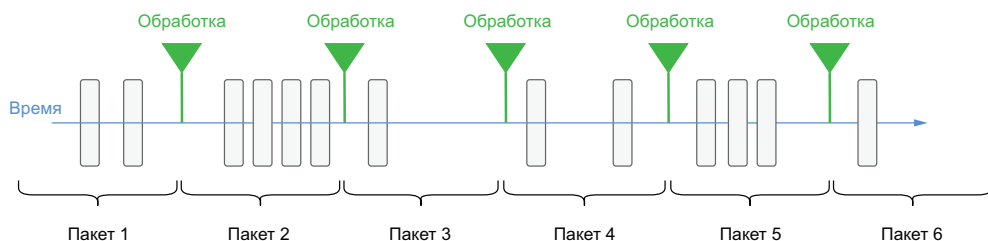


Рис. 4.1. При пакетной обработке сообщения обрабатываются с предварительно определенными интервалами времени и выдерживают постоянный ритм

Хотя эта методика действительно уменьшает задержки при обработке между прибытием элемента данных и началом его обработки, искусственная пауза все же вводится в процесс, который формируется по мере роста сложности конвейера данных. Следовательно, даже процесс микропакетной обработки не может полагаться на постоянные времена ответа и должен учитывать задержки между прибытием данных и началом их обработки. Это делает микропакетную обработку более пригодной для вариантов использования, в которых не требуются самые последние данные и допустима несколько замедленная ответная реакция, тогда как потоковая нативная обработка лучше подходит для вариантов использования, требующих ответной реакции почти в реальном времени, например для выявления случаев мошенничества, формирования цен в реальном времени и системного мониторинга.

4.1.3. Потоковая нативная обработка

При потоковой нативной обработке каждый новый фрагмент данных обрабатывается сразу после прибытия, как показано на рис. 4.2. В отличие от пакетной обработки здесь нет произвольных

интервалов между операциями обработки, и каждый элемент данных обрабатывается отдельно.

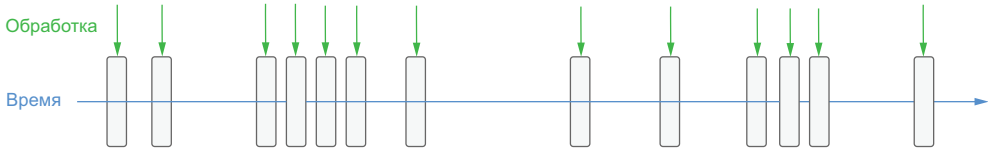


Рис. 4.2. При потоковой обработке процедура обработки запускается в момент прибытия каждого сообщения, поэтому ритм обработки является нерегулярным и непредсказуемым

Хотя может показаться, что различия между потоковой обработкой и микропакетной обработкой относятся лишь ко времени обработки, они воздействуют как на системы обработки данных, так и на приложения, которые от них зависят. Ценность данных для бизнеса быстро снижается после их создания, особенно в таких случаях, как предотвращение мошенничества или обнаружение аномалий. Большие объемы наборов данных, прибывающих с высокой скоростью, в этих вариантах использования часто содержат ценную, но «скоропортящуюся» информацию, на которую необходимо немедленно реагировать. Мошеннические деловые транзакции, такие как перевод денег или загрузка лицензионного программного обеспечения, должны быть выявлены и прекращены до их завершения, иначе невозможно помешать похитителю получить средства незаконным путем. Чтобы максимизировать ценность своих данных для таких вариантов использования, разработчики должны в корне изменить свой подход к обработке данных в реальном времени, сосредоточив внимание на сокращении времени задержки обработки, возникающей на обычных платформах пакетной обработки, и используя более оперативный подход, такой как потоковая нативная обработка.

4.2. Что такое *Pulsar Functions*

В комплект Apache Pulsar включен упрощенный вычислительный механизм под названием Pulsar Functions, предоставляющий разработчикам возможность развертывания реализаций простых функций на языках Java, Python или Golang. Это функциональное средство позволяет пользователям в полной мере воспользоваться преимуществами обработки без сервера, аналогичными предоставляемым AWS Lambda на платформе обмена сообщениями с открытым исходным кодом, а не привязываться к проприетарному API провайдера облачной среды.

Pulsar Functions позволяют применять логику обработки к данным непосредственно во время их перемещения в самой системе

обмена сообщениями. Эти упрощенные процессы обработки выполняются прямо в системе Pulsar и настолько связаны с каждым конкретным сообщением, насколько это вообще возможно. При этом нет необходимости в привлечении сторонних фреймворков обработки, таких как Spark, Flink или Kafka Streams. В отличие от других систем обмена сообщениями, который работают как «тупая труба» для перемещения данных из системы в систему, Pulsar Functions предоставляют возможность выполнения простых вычислений для сообщений, прежде чем они будут направлены потребителям. Pulsar Functions потребляют сообщения из одной или нескольких тем Pulsar, применяют предоставленную пользователем функцию (логику обработки) к каждому входящему сообщению и публикуют результаты в одной или нескольких темах Pulsar, как показано на рис. 4.3.

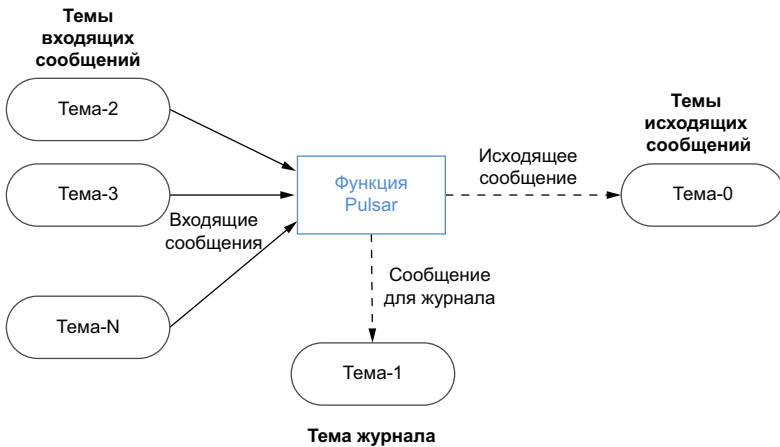


Рис. 4.3. Pulsar Functions выполняют определенный пользователем код для данных, публикуемых в темах Pulsar

Функции Pulsar точнее всего можно охарактеризовать как аналогии лямбда-функций, специально предназначенные для использования Pulsar в качестве внутренней шины передачи сообщений. Причина в том, что они применяют несколько проектных решений системы команд из широко известного фреймворка AWS Lambda, позволяющих выполнять код без наличия контролирующих или управляющих серверов, на которых этот код хранится. Таким образом, обобщающим термином для такой модели программирования является «без сервера» («внесерверная» – serverless).

Фреймворк Pulsar Functions позволяет пользователям разрабатывать самодостаточные фрагменты кода, а затем развертывать их с помощью простого REST-вызова. Pulsar позаботится обо всех внутренних аспектах, требуемых для выполнения пользовательского кода, включая создание потребителей и производителей Pulsar для входящей и исходящей тем этой конкретной функции. Разработ-

чки могут сосредоточиться непосредственно на бизнес-логике и не беспокоиться о стандартном коде, необходимом для передачи сообщений в Pulsar. Короче говоря, фреймворк Pulsar Functions предоставляет полностью готовую к работе вычислительную инфраструктуру в существующем кластере Pulsar.

4.2.1. Модель программирования

Модель программирования, на которой основан фреймворк Pulsar Functions, чрезвычайно проста. Функции Pulsar принимают сообщения из одной или нескольких тем входящих (input) сообщений, и при каждой публикации сообщения в этой теме (темах) выполняется код функции. При активизации код функции выполняет логику обработки для входящего сообщения и записывает свой вывод (необязательный) в тему исходящих сообщений. Хотя для всех функций требуется наличие темы входящих сообщений, они не ограничены строго в производстве любого вывода в тему исходящих сообщений.

Можно сделать тему исходящих сообщений одной функции Pulsar темой входящих сообщений для другой функции, что позволяет в итоге создать направленный ациклический граф (directly acyclic graph – DAG) функций Pulsar, как показано на рис. 4.4. В этом графе каждое ребро представляет поток данных, а каждая вершина – функцию Pulsar, применяющую определенную пользователем логику для обработки данных. Можно до бесконечности комбинировать функции Pulsar в таких DAG'ах, и даже существует возможность написать приложение, полностью состоящее из функций Pulsar и структурированное как DAG, если потребуется.

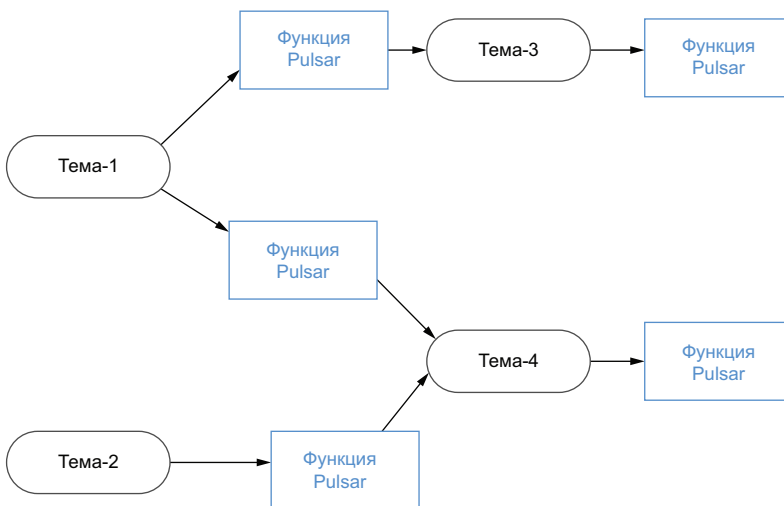


Рис. 4.4. Функции Pulsar можно логически структурировать в сеть обработки

4.3. Разработка функций Pulsar

В настоящее время функции Pulsar можно писать на языках Java, Python и Go. Если вы уже знакомы с одним из этих широко распространенных языков, то сможете достаточно быстро приступить к разработке функций Pulsar.

4.3.1. Ориентированные на язык функции

Pulsar поддерживает свойство, которое обобщенно обозначают термином «ориентированные на язык функции» (language-native functions), означающее, что специализированные для Pulsar библиотеки или какие-либо особые зависимости не требуются. Преимущество ориентированных на язык функций заключается в том, что они не имеют никаких зависимостей, кроме тех, что уже доступны в самом языке программирования. Это делает ориентированные на язык функции чрезвычайно простыми и удобными для разработки. В настоящее время ориентированные на язык функции могут разрабатываться только с использованием Java или Python. Поддержка Golang для этой функциональной возможности пока еще недоступна.

Ориентированные на язык Java функции

Чтобы использовать фрагмент кода Java как ориентированную на язык функцию, необходимо реализовать интерфейс `java.util.Function`, содержащий только один метод `apply`, как показано в листинге 4.1. Эта простейшая функция всего лишь выводит значение любой принятой строки, тем не менее она демонстрирует, насколько просто можно разработать любую функцию, используя только возможности самого языка Java. В метод `apply` можно включить любую разновидность сложной логики, чтобы предоставить возможности для более полноценной потоковой обработки.

Листинг 4.1. Java-ориентированная функция

```
import java.util.Function;

public class EchoFunction implements Function <String, String> { ①

    public String apply(String input) { ②
        return input;
    }
}
```

- ① Определяет, что содержимым входящей темы будет строка, и возвращается также строка.
- ② Определенный в этом интерфейсе единственный метод, выполняемый, когда принимается сообщение.

Ориентированные на язык Python функции

Чтобы использовать фрагмент кода Python как ориентированную на язык функцию, необходимо наличие метода с именем `process`, как показано в листинге 4.2. Этот метод просто добавляет восклицательный знак к значению каждой принятой строки.

Листинг 4.2. Python-ориентированная функция

```
def process(input):  
    return "{}!".format(input)
```

- ❶ Метод, который вызывается, когда прибывает сообщение.
- ❷ Возвращает предоставленные входные данные с восклицательным знаком, добавленным в конце.

Здесь можно видеть, что подход с применением ориентированных на язык функций предоставляет вполне понятный способ написания функций Pulsar без использования каких-либо API. Это идеальная методика разработки простых функций без сохранения состояния.

4.3.2. Pulsar SDK

Другой вариант: разработка функций с применением Pulsar Functions SDK, который использует специализированные для Pulsar библиотеки, предоставляющие широкий спектр функциональности, недоступный в собственных интерфейсах языков, например управление состоянием и пользовательская конфигурация. К дополнительным возможностям и информации можно получить доступ через объект `Context`, определенный внутри SDK и содержащий следующие сведения:

- имя, версию и идентификатор функции Pulsar;
- идентификатор каждого сообщения;
- имя темы, в которую было отправлено сообщение;
- имена всех входящих тем, а также исходящей темы, связанной с данной функцией;
- абонент и пространство имен, связанные с данной функцией;
- объект журнала, используемый данной функцией, который можно использовать для записи журнальных сообщений;
- доступ к произвольным значениям пользовательской конфигурации, предоставляемый через интерфейс командной строки;
- интерфейс для записи метрик.

Реализации Pulsar SDK доступны для языков Java, Python и GoLang, и каждая определяет функциональный интерфейс, который включает объект `Context` как параметр, заполняемый и предоставляемый средой времени выполнения Pulsar Functions.

Функции Java SDK

Чтобы начать разработку функций Pulsar с использованием Java SDK, необходимо добавить в проект зависимость от компонента `pulsar-functions-api`, как показано в листинге 4.3.

Листинг 4.3. Добавление зависимостей Pulsar SDK в файл *pom.xml*

```
<properties>
  <pulsar.version>2.7.2</pulsar.version>
</properties>
...
<dependency>
  <groupId>org.apache.pulsar</groupId>
  <artifactId>pulsar-functions-api</artifactId>
  <version>${pulsar.version}</version>
</dependency>
```

При разработке функции Pulsar на основе этого SDK функция должна реализовать интерфейс `org.apache.pulsar.functions.api.Function`. В листинге 4.4 можно видеть, что этот интерфейс определяет только один метод `process`, обязательный для реализации.

Листинг 4.4. Определение интерфейса функции Pulsar SDK

```
@FunctionalInterface
public interface Function<I, O> {
    O process(I input, Context context) throws Exception;
}
```

Метод `process` вызывается как минимум один раз в зависимости от гарантий обработки, определенных для этой конкретной функции в отношении каждого сообщения, публикуемого во входной теме, указанной в конфигурации данной функции. Прибывающие входные байты сериализуются во входной тип `I` для сообщений на основе JSON, а также в простые типы Java, такие как `String`, `Integer` и `Float`. Если эти типы не соответствуют вашим потребностям, то можно также использовать собственные типы, если для них предоставлена ваша собственная реализация интерфейса `org.apache.pulsar.functions.api.Serde`, или можно зарегистрировать тип входящего сообщения в схеме реестра Pulsar, которая будет подробно рассматриваться в главе 7. В листинге 4.5 показана реализация эхо-функции, демонстрирующая некоторые функциональные возможности Pulsar SDK, например запись в журнал и фиксацию метрик.

Листинг 4.5. Функция Pulsar SDK на языке Java

```
import java.util.stream.Collectors;
import org.apache.pulsar.functions.api.Context;
import org.apache.pulsar.functions.api.Function;
import org.slf4j.Logger;
```

```

public class EchoFunction implements Function<String, String> {

    public String process(String input, Context ctx) {
        Logger LOG = ctx.getLogger();
        String inputTopics =
            ctx.getInputTopics().stream()
                .collect(Collectors.joining(", "));

        String functionName = ctx.getFunctionName();

        String logMessage =
            String.format("A message with a value of \"%s\" +
                \"has arrived on one of the following topics: %s\\n\",
                input, inputTopics);

        LOG.info(logMessage);
        String metricName =
            String.format("function-%s-messages-received", functionName);

        ctx.recordMetric(metricName, 1);
        return input;
    }
}

```

- ❶ Этот класс должен реализовать интерфейс Pulsar Functions.
- ❷ Интерфейс определяет единственный метод с двумя параметрами, включая объект Context.
- ❸ Объект Context используется для доступа к объекту журнала Logger.
- ❹ Объект Context используется для получения списка входящих тем.
- ❺ Объект Context используется для получения имени функции.
- ❻ Генерируется сообщение для записи в журнал.
- ❼ Фиксируется метрика, определенная пользователем.

В Java SDK объект Context обеспечивает доступ к парам ключ-значение, переданным в функцию Pulsar через командную строку (в формате JSON). Такая возможность позволяет писать обобщенные (generic) функции, которые можно использовать многократно, но со слегка измененной конфигурацией. Например, необходимо написать функцию, фильтрующую события на основе регулярного выражения, определенного пользователем. По прибытии события его содержимое сравнивается с определенным конфигурацией регулярным выражением, и возвращаются только записи, соответствующие заданному образцу, а все прочие игнорируются. Такая функция может оказаться полезной, если требуется проверять формат входных данных перед началом их обработки. Пример функции, обеспечивающий доступ к регулярному выражению для проверки пар ключ-значение в объекте Context, показан в листинге 4.6.

Листинг 4.6. Конфигурируемая пользователем функция Pulsar на языке Java

```

import java.util.regex.Pattern;
import org.apache.pulsar.functions.api.Context;
import org.apache.pulsar.functions.api.Function;

public class RegexMatcherFunction implements Function<String, String> {

    public static final String REGEX_CONFIG = "regex-pattern";

    @Override
    public String process(String input, Context ctx) throws Exception {
        Optional<Object> config =
            ctx.getUserConfigValue(REGEX_CONFIG);

        if (config.isPresent() && config.get().getClass()
            .getName().equals(String.class.getName())) {

            Pattern pattern = Pattern.compile(config.get().toString());
            if (pattern.matcher(input).matches()) {
                String metricName =
                    String.format("%s-regex-matches", ctx.getFunctionName());

                ctx.recordMetric(metricName, 1);
                return input;
            }
        }
        return null;
    }
}

```

- ❶ Извлечение паттерна регулярного выражения из конфигураций, предоставляемых пользователем.
- ❷ Если предоставлена строка регулярного выражения, то компилируется соответствующее регулярное выражение.
- ❸ Если входные данные соответствуют регулярному выражению, то разрешается их передача для дальнейшей обработки.
- ❹ Иначе возвращается нулевое значение (`null`).

Функции Pulsar могут публиковать свои результаты в исходящей теме, но это не является обязательным требованием. Также могут существовать функции, которые не всегда возвращают значение, например функция из листинга 4.6, отфильтровывающая не соответствующие условию входные данные. В таком сценарии из функции можно просто возвращать значение `null`.

Функции Python SDK

Чтобы начать разработку функций Pulsar с использованием Python SDK, необходимо добавить зависимость от клиента Pulsar в установленную рабочую среду Python. Самую последнюю версию

клиентской библиотеки Pulsar можно без затруднений установить с помощью менеджера пакетов `pip` и команд, показанных в листинге 4.7. После установки этой библиотеки в локальной среде разработки появится возможность разработки функций Pulsar на языке Python с применением соответствующего SDK.

Листинг 4.7. Добавление зависимостей Pulsar SDK в рабочую среду Python

```
pip3 install pulsar-client==2.6.3 -user ❶
pip3 list ❷

Package      Version
-----
...
pulsar-client 2.6.3 ❸
```

- ❶ Установка клиента Pulsar.
- ❷ Вывод списка всех пакетов.
- ❸ Подтверждение установки корректной версии клиента Pulsar.

Рассмотрим реализацию эхо-функции на языке Python для демонстрации некоторых возможностей SDK в листинге 4.8.

Листинг 4.8. Функция Pulsar SDK на языке Python

```
from pulsar import Function

class EchoFunction(Function):
    def __init__(self):
        pass

    def process(self, input, context):
        logger = context.get_logger()
        evtTime = context.get_message_eventtime()
        msgKey = context.get_message_key();

        logger.info("""A message with a value of {0}, a key of {1},
            and an event time of {2} was received""")
            .format(input, msgKey, evtTime))

        metricName = """function-%s-
            messages-received""".format(context.get_function_name())
        context.record_metric(metricName, 1)

    return input
```

- ❶ Определение функции, требуемое Pulsar SDK.
- ❷ Python SDK предоставляет доступ к объекту журнала.
- ❸ Python SDK предоставляет доступ к метаданным сообщения.

- ❹ Python SDK поддерживает метрики.
- ❺ Возврат эхо-отображения первоначального входного значения.

В Python SDK объект Context предоставляет почти все возможности, что и Java SDK, с двумя существенными исключениями. Первое: начиная с версии 2.6.0, Pulsar Functions на основе Python не поддерживают схемы, которые будут подробно рассматриваться в главе 7. По существу, это означает, что функция Python отвечает за сериализацию и десериализацию байтов сообщения в ожидаемый формат. Вторая возможность, не представленная в Pulsar Functions на основе Python, – доступ к Pulsar Admin API, который, как было отмечено в главе 3, возможен только в Java.

Функции Golang SDK

Чтобы начать разработку функций Pulsar с использованием Golang SDK, необходимо добавить зависимость от клиента Pulsar в установленную рабочую среду Golang. Самую последнюю версию клиентской библиотеки Pulsar можно установить с помощью следующей команды: `go get -u "github.com/apache/pulsar-client-go/pulsar"`.

Рассмотрим реализацию эхо-функции на языке Golang для демонстрации некоторых возможностей SDK в листинге 4.9.

Листинг 4.9. Функция Pulsar SDK на языке Go

```
package main

import (
    "context"
    "fmt"

    "github.com/apache/pulsar/pulsar-function-go/pf"
    log "github.com/apache/pulsar/pulsar-function-go/logutil"
)

func echoFunc(ctx context.Context, in []byte) []byte {
    if fc, ok := pf.FromContext(ctx); ok {
        log.Infof("This input has a length of: %d", len(in))

        fmt.Printf("Fully-qualified function name is:%s\\%s\\%s\\n",
            fc.GetFuncTenant(), fc.GetFuncNamespace(), fc.GetFuncName())
    }
    return in
}

func main() {
    pf.Start(echoFunc)
}
```

- ❶ Импорт библиотеки SDK.
- ❷ Импорт библиотеки функций журналирования.

- ③ Код функции с корректной сигнатурой метода.
- ④ Golang SDK предоставляет доступ к объекту журнала.
- ⑤ Golang SDK предоставляет доступ к метаданным функции.
- ⑥ Возврат эхо-отображения первоначального входного значения.
- ⑦ Регистрация функции `echoFunc` в фреймворке Pulsar Functions.

При написании функций на языке Golang следует помнить, что необходимо предоставить имя функции, которую вы хотите использовать для выполнения реальной логики, в метод `pf.Start()` внутри вызова метода `main()`, как показано в листинге 4.9. Такое действие регистрирует указанную функцию во фреймворке Pulsar Functions и гарантирует, что эта функция является единственной вызываемой в момент прибытия нового сообщения. В рассматриваемом здесь примере функция названа `echoFunc`, но имя может быть любым при условии, что сигнатура метода соответствует любой из поддерживаемых Go сигнатур, перечисленных в листинге 4.10. Все прочие сигнатуры функций не будут приниматься к рассмотрению, следовательно, логика обработки внутри функции Golang не выполняется.

Листинг 4.10. Сигнатуры методов, поддерживаемые в Go

```
func ()  
func () error  
func (input) error  
func () (output, error)  
func (input) (output, error)  
func (context.Context) error  
func (context.Context, input) error  
func (context.Context) (output, error)  
func (context.Context, input) (output, error)
```

В настоящее время существуют некоторые ограничения при использовании SDK для разработки функций Pulsar на языке Go, но ситуация изменяется по мере стабилизации зрелости проекта, поэтому настоятельно рекомендуется внимательно изучать самую свежую версию онлайн-документации, чтобы узнать о самых последних изменениях. Тем не менее на момент написания этой книги функции Golang не поддерживают запись метрик на уровне функций (например, отсутствует метод `recordMetric`, определенный внутри объекта `Context` в Golang SDK). Кроме того, пока не предоставляется возможность реализации функций с сохранением состояния при использовании Golang.

4.3.3. Функции с сохранением состояния

Функции с сохранением состояния используют информацию, собранную из предыдущих сообщений, которые обработаны для генерации выходных данных. Одним из примеров является функ-

ция, которая принимает значения температуры, считывая события с сенсорного датчика IoT (интернета вещей), и вычисляет среднюю температуру сенсора. Для предоставления этой характеристики потребуется вычисление и сохранение текущего среднего значения по ранее прочитанным показаниям температуры.

При использовании Pulsar SDK для разработки функций независимо от применения любого из трех поддерживаемых языков вторым параметром метода `process` является объект `Context`, автоматически предоставляемый фреймворком Pulsar Functions. Если вы пишете на Java или Python, то API объекта `Context` также предоставляет два отдельных механизма для сохранения информации между последовательными вызовами функции Pulsar. Первый механизм – интерфейс отображения (`map`) для сохранения и извлечения пар ключ–значение с помощью методов `putState` и `getState`. Эти методы работают так же, как и любой другой знакомый вам интерфейс отображения, и позволяют сохранять и извлекать значения любого типа, используя в качестве ключей строковые значения.

Второй механизм сохранения состояния, предоставляемый объектом `Context`, – счетчики (`counters`), позволяющие сохранять только числовые значения, используя строки как ключи. Счетчики представляют собой специализированную версию отображения ключ–значение, функционально предназначенную для сохранения исключительно числовых значений. Во внутренней среде счетчики хранятся как 64-битовые двоичные значения с обратным порядком байтов (`big endian`) и могут изменяться только с помощью методов `incrCounter` и `incrCounterAsync`.

В качестве примера рассмотрим функцию, которая принимает значения температуры, считывая события с сенсорного датчика IoT (интернета вещей), и вычисляет среднюю температуру сенсора, чтобы продемонстрировать, как можно использовать сохраняемое состояние в функции Pulsar. Функция, показанная в листинге 4.11, принимает показания сенсорного датчика и сравнивает их со средним значением температуры, вычисленным по предыдущим показаниям, чтобы определить, не требуется ли подать сигнал некоторого типа.

Листинг 4.11. Функция вычисления среднего значения показаний сенсорного датчика

```
public class AvgSensorReadingFunction implements
    Function<Double, Void> {
    private static final String VALUES_KEY = "VALUES";

    @Override
    public Void process(Double input, Context ctx) throws Exception {
        CircularFifoQueue<Double> values = getValues(ctx);
```

1

2

```

    if (Math.abs(input - getAverage(values)) > 10.0) {
        // Срабатывает механизм подачи сигнала.
    }

    values.add(input);
    ctx.putState(VALUE_KEY, serialize(values));
    return null;

private Double getAverage(CircularFifoQueue<Double> values) {
    return StreamSupport.stream(values.spliterator(), false)
        .collect(Collectors.summingDouble(Double::doubleValue))
        / values.size();

private CircularFifoQueue<Double> getValues(Context ctx) {
    if (ctx.getState(VALUE_KEY) == null) {
        return new CircularFifoQueue<Double>(100);
    } else {
        return deserialize(ctx.getState(VALUE_KEY));
    }
}
}
}

```

- 1 Функция принимает значение типа `double` и не производит выходное сообщение.
- 2 Десериализация объекта Java, используемого для хранения предыдущих показаний датчика.
- 3 Если текущее показание существенно отличается, то генерируется сигнал.
- 4 Текущее значение добавляется в список наблюдаемых показаний.
- 5 Обновленный объект Java записывается в хранилище состояний.
- 6 Используется Streams API для вычисления среднего значения.
- 7 Создается экземпляр объекта Java, если он отсутствует в хранилище состояний.
- 8 Преобразование байтов из хранилища состояний в необходимый нам объект Java.

В листинге 4.11 необходимо обратить особое внимание на несколько моментов. Вероятно, вы заметили, что возвращаемый тип функции определен как `Void`. Это означает, что функция не производит выходное значение. Другим моментом является тот факт, что функция полагается на механизм сериализации Java для хранения и извлечения списка последних полученных 100 значений (показаний датчика). Это зависит от реализации сторонней библиотеки поддержки очереди типа FIFO для хранения 100 самых последних показаний для вычисления среднего значения перед сравнением с текущим показанием датчика. Если это показание существенно отличается от среднего значения, то генерируется сигнал. Нако-

нец, самое последнее показание добавляется в очередь FIFO, которая затем сериализуется и записывается в хранилище состояний.

При всех последующих вызовах `AvgSensorReadingFunction` будет извлекать байты из очереди FIFO, выполнять их десериализацию обратно в объект Java и снова использовать для вычисления среднего значения. Этот процесс повторяется бесконечно и сохраняет только самые последние показания для сравнения с трендом (например, с изменением среднего значения показаний датчика). Такой подход диаметрально противоположен предоставляемому Pulsar Functions механизму разделения данных на блоки (windowing), описанному в главе 12. Этот механизм позволяет создать набор из нескольких входных данных перед выполнением функции с учетом интервала времени или фиксированного счетчика. Когда блок заполняется, функция одновременно получает весь список входных данных, тогда как в функции в листинге 4.11 значения передаются по одному, и она должна сопровождать предыдущие значения внутри своего состояния.

У вас может возникнуть вопрос: почему бы просто не использовать встроенный в Pulsar механизм разделения данных на блоки для описанного выше варианта? В данном случае необходимо реагировать на каждое отдельное показание сразу после того, как оно становится доступным, а не ждать накопления достаточного объема показаний. Это позволяет гораздо быстрее обнаружить любую проблему.

А теперь завершим изучение функций с сохранением состояния, рассмотрев подробно интерфейс счетчика, предоставляемый API Context. Удачным примером того, как и когда использовать его функциональность, является функция `WordCount`, сохраняющая количество отдельных слов с применением методов счетчика, предоставленного API объекта Context, как показано в листинге 4.12.

Листинг 4.12. Функция `WordCount`, использующая счетчики сохранения состояния

```
package com.manning.pulsar.chapter4.functions.sdk;

import java.util.Arrays;
import java.util.List;

import org.apache.pulsar.functions.api.Context;
import org.apache.pulsar.functions.api.Function;

public class WordCountFunction implements Function<String, Integer> {
    @Override
    public Integer process(String input, Context context) throws Exception {
        List<String> words = Arrays.asList(input.split("\\."));
        words.forEach(word -> context.incrCounter(word, 1));
        return Integer.valueOf(words.size());
    }
}
```

Логика этой функции очевидна: сначала она разделяет объект входящей строки на несколько слов, используя шаблон регулярно выражения, затем для каждого разделенного слова соответствующий счетчик увеличивается на единицу. Такая функция является верным кандидатом для эффективной однократной обработки семантики с обеспечением точного результата. Если бы вместо этого использовалась обработка семантики типа *at-least-once* (минимум однажды), то в итоге одно сообщение обрабатывалось более одного раза в таком некорректном сценарии, а в результате получились бы удвоенные счетчики многих слов.

4.4. Тестирование функций Pulsar

В этом разделе подробно описывается процесс разработки и тестирования вашей первой функции Pulsar. Мы будем использовать функцию `KeywordFilterFunction`, показанную в листинге 4.13, для демонстрации цикла разработки для функции Pulsar. Эта функция принимает предоставленное пользователем ключевое слово и отфильтровывает любую входящую строку, которая не содержит заданного ключевого слова. Примером практического применения такой функции может послужить сканирование Twitter-канала для поиска твитов, связанных с конкретной темой или содержащих определенную фразу.

Листинг 4.13. Функция `KeywordFilterFunction`

```
package com.manning.pulsar.chapter4.functions.sdk;

import java.util.Arrays;
import java.util.List;
import java.util.Optional;
import org.apache.pulsar.functions.api.Context;
import org.apache.pulsar.functions.api.Function;

public class KeywordFilterFunction
    implements Function<String, String> { ❶

    public static final String KEYWORD_CONFIG = "keyword";
    public static final String IGNORE_CONFIG = "ignore-case";

    @Override
    public String process(String input, Context ctx) {
        Optional<Object> keyword =
            ctx.getUserConfigValue(KEYWORD_CONFIG); ❷
        Optional<Object> ignoreCfg =
            ctx.getUserConfigValue(IGNORE_CONFIG); ❸

        boolean ignoreCase = ignoreCfg.isPresent() ?
            (boolean) ignoreCfg.get(): false;

        List<String> words = Arrays.asList(input.split("\\s")); ❹
```

```

        if (!keyword.isPresent()) {
            return null;
        } else if (ignoreCase && words.stream().anyMatch(
            s -> s.equalsIgnoreCase((String) keyword.get())) {
            return input;
        } else if (words.contains(keyword.get())) {
            return input;
        }
        return null;
    }
}

```

- ❶ Функция принимает строку и возвращает строку.
- ❷ Извлечение ключевого слова из объекта Context.
- ❸ Извлечение параметра настройки «игнорировать регистр букв» из объекта Context.
- ❹ Разделение входящей строки на отдельные слова.
- ❺ При отсутствии заданного ключевого слова соответствия быть не может.
- ❻ Обработка каждого слова без учета регистра букв.
- ❼ Проверка на точное соответствие.

Несмотря на то что это упрощенный код, здесь будет показан полный процесс тестирования, который обычно выполняется при разработке программного обеспечения для промышленного использования. Поскольку это чистый код на языке Java, можно применять любые существующие фреймворки модульного тестирования, например JUnit или TestNG, для проверки логики функции.

4.4.1. Модульное тестирование

Первым этапом должно стать написание набора модульных тестов для некоторых из наиболее обобщенных сценариев, чтобы проверить корректность логики и выдачу точных результатов для различных предложений, отправляемых в функцию. Так как этот код использует Pulsar SDK API, потребуется некоторая библиотека-заглушка, например Mockito, для имитации объекта Context, как показано в листинге 4.14.

Листинг 4.14. Модульные тесты для функции KeywordFilterFunction

```

package com.manning.pulsar.chapter4.functions.sdk;

import static org.mockito.Mockito.*;
import static org.junit.Assert.*;

public class KeywordFilterFunctionTests {
    private KeywordFilterFunction function = new KeywordFilterFunction();

    @Mock
    private Context mockedCtx;

```



```

@Before
public final void setUp() {
    MockitoAnnotations.initMocks(this);
}

@Test
public final void containsKeywordTest() throws Exception {
    when(mockedCtx.getUserConfigValue(
        KeywordFilterFunction.KEYWORD_CONFIG))
        .thenReturn(Optional.of("dog"));

String sentence = "The brown fox jumped over the lazy dog";
String result = function.process(sentence, mockedCtx);
assertNotNull(result);
assertEquals(sentence, result);

@Test
public final void doesNotContainKeywordTest() throws Exception {
    when(mockedCtx.getUserConfigValue(
        KeywordFilterFunction.KEYWORD_CONFIG))
        .thenReturn(Optional.of("cat"));

String sentence = "It was the best of times, it was the worst of times";
String result = function.process(sentence, mockedCtx);
assertNull(result);

@Test
public final void ignoreCaseTest() {
    when(mockedCtx.getUserConfigValue(
        KeywordFilterFunction.KEYWORD_CONFIG))
        .thenReturn(Optional.of("RED"));
    when(mockedCtx.getUserConfigValue(
        KeywordFilterFunction.IGNORE_CONFIG))
        .thenReturn(Optional.of(Boolean.TRUE));

String sentence = "Everyone watched the red sports car drive off.";
String result = function.process(sentence, mockedCtx);
assertNotNull(result);
assertEquals(sentence, result);
}
}

```

- ❶ Конфигурирование ключевого слова *dog*.
- ❷ Ожидается, что входящее предложение будет возвращено, так как оно содержит ключевое слово.
- ❸ Конфигурирование ключевого слова *cat*.
- ❹ Ожидается, что входящее предложение не будет возвращено, так как оно не содержит ключевого слова.
- ❺ Конфигурирование ключевого слова *RED*.
- ❻ Конфигурирование функции для игнорирования регистра букв при фильтрации по ключевому слову.

- 7 Ожидается, что входящее предложение будет возвращено, так как оно содержит вариант ключевого слова в нижнем регистре.

Здесь можно видеть, что созданные модульные тесты покрывают полностью основную функциональность и основаны на имитации объекта Context Pulsar. Этот тип набора тестов точно такой же, как и набор для тестирования любого класса Java, не являющегося функцией Pulsar.

4.4.2. Комплексное тестирование

После получения положительных результатов модульного тестирования необходимо проверить, как функция Pulsar будет выполняться в кластере. Самый простой способ – запустить сервер Pulsar и выполнить функцию локально, используя вспомогательный класс LocalRunner. В этом режиме функция работает как отдельный независимый процесс на компьютере, с которого она была запущена. Это наилучший вариант, если вы разрабатываете и тестируете собственные функции, поскольку он позволяет подключить отладчик к процессу функции на локальном компьютере, как показано на рис. 4.5.

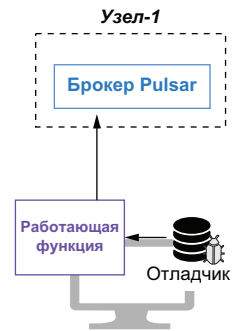


Рис. 4.5. При запуске функции Pulsar с использованием LocalRunner она работает на локальном компьютере, позволяя подключить отладчик и выполнять пошаговый проход по коду

Чтобы воспользоваться LocalRunner, сначала необходимо добавить несколько зависимостей в проект Maven, как показано в листинге 4.15. Это позволит подключить класс LocalRunner, используемый для тестирования функции совместно с работающим кластером Pulsar.

Листинг 4.15. Включение зависимостей для использования LocalRunner

```
<dependencies>
  . . .
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-core</artifactId>
    <version>${jackson.version}</version>
  </dependency>
</dependencies>
```

```

    <groupId>org.apache.pulsar</groupId>
    <artifactId>pulsar-functions-local-runner-original</artifactId>
    <version>${pulsar.version}</version>
  </dependency>
</dependencies>

```

Далее необходимо написать класс для конфигурирования и запуска LocalRunner, как показано в листинге 4.16. Здесь можно видеть, что этот код сначала непременно должен сконфигурировать функцию Pulsar для выполнения в LocalRunner, поэтому в нем определяется адрес реального экземпляра кластера Pulsar, который будет использоваться для тестирования.

Листинг 4.16. Тестирование функции KeywordFilterFunction с использованием LocalRunner

```

public class KeywordFilterFunctionLocalRunnerTest {
    final static String BROKER_URL = "pulsar://localhost:6650";
    final static String IN = "persistent://public/default/raw-feed";
    final static String OUT = "persistent://public/default/filtered-feed";

    private static ExecutorService executor;
    private static LocalRunner localRunner;
    private static PulsarClient client;
    private static Producer<String> producer;
    private static Consumer<String> consumer;
    private static String keyword = "";

    public static void main(String[] args) throws Exception {
        if (args.length > 0) {
            keyword = args[0];
        }
        startLocalRunner();
        init();
        startConsumer();
        sendData();
        shutdown();
    }

    private static void startLocalRunner() throws Exception {
        localRunner = LocalRunner.builder()
            .brokerServiceUrl(BROKER_URL)
            .functionConfig(getFunctionConfig())
            .build();
        localRunner.start(false);
    }

    private static FunctionConfig getFunctionConfig() {
        Map<String, ConsumerConfig> inputSpecs =
            new HashMap<String, ConsumerConfig> ();
    }

```

```

inputSpecs.put(IN, ConsumerConfig.builder()
    .schemaType(Schema.STRING.getSchemaInfo().getName())
    .build());

Map<String, Object> userConfig = new HashMap<String, Object>();
userConfig.put(KeywordFilterFunction.KEYWORD_CONFIG, keyword);
userConfig.put(KeywordFilterFunction.IGNORE_CONFIG, true);

return FunctionConfig.builder()
    .className(KeywordFilterFunction.class.getName())
    .inputs(Collections.singleton(IN))
    .inputSpecs(inputSpecs)
    .output(OUT)
    .name("keyword-filter")
    .tenant("public")
    .namespace("default")
    .runtime(FunctionConfig.Runtime.JAVA)
    .subName("keyword-filter-sub")
    .userConfig(userConfig)
    .build();
}

private static void init() throws PulsarClientException {
    executor = Executors.newFixedThreadPool(2);
    client = PulsarClient.builder()
        .serviceUrl(BROKER_URL)
        .build();
    producer = client.newProducer(Schema.STRING).topic(IN).create();
    consumer = client.newConsumer(Schema.STRING).topic(OUT)
        .subscriptionName("validation-sub").subscribe();
}

private static void startConsumer() {
    Runnable runnableTask = () -> {
        while (true) {
            Message<String> msg = null;
            try {
                msg = consumer.receive();
                System.out.printf("Message received: %s \n", msg.getValue());
                consumer.acknowledge(msg);
            } catch (Exception e) {
                consumer.negativeAcknowledge(msg);
            }
        }
    };
    executor.execute(runnableTask);
}

private static void sendData() throws IOException {
    InputStream inputStream = Thread.currentThread().getContextClassLoader()
        .getResourceAsStream("test-data.txt");

```

```

        InputStreamReader streamReader = new InputStreamReader(inputStream,
            StandardCharsets.UTF_8);
        BufferedReader reader = new BufferedReader(streamReader);
        for (String line; (line = reader.readLine()) != null;) {
            producer.send(line);
        }
    }

    private static void shutdown() throws Exception {
        executor.shutdown();
        localRunner.stop();
        . . .
    }
}

```

16

- 1 Получение предоставленного пользователем ключевого слова.
- 2 URL сервиса для кластера Pulsar, с которым будет работать функция.
- 3 Передача в функцию конфигурации для LocalRunner.
- 4 Запуск LocalRunner и функции.
- 5 Определение: данные во входящей теме должны быть строками.
- 6 Инициализация свойств пользовательской конфигурации ключевым словом, предоставленным пользователем.
- 7 Определение имени класса функции для запуска в локальном режиме.
- 8 Определение входящей темы.
- 9 Передача в функцию свойств конфигурации входящей темы.
- 10 Определение исходящей темы.
- 11 Определение: для этой функции необходимо использовать рабочую среду Java Runtime.
- 12 Передача в функцию свойств пользовательской конфигурации.
- 13 Инициализация производителя и потребителя для тестирования.
- 14 Запуск потребителя в фоновом потоке для чтения сообщений из исходящей темы функции.
- 15 Отправка данных во входящую тему функции.
- 16 Остановка LocalRunner, потребителя и т. д.

Самый простой способ получения доступа к кластеру Pulsar – активизация контейнера Pulsar Docker, как это было сделано в главе 3, т. е. выполнение в окне командной оболочки приведенной ниже команды, которая запустит кластер Pulsar в автономном режиме внутри контейнера. Обратите внимание: мы также монтируем каталог, в котором проект GitHub, связанный с этой книгой, был клонирован на локальном компьютере:

```

$ export GIT_PROJECT=<CLONE_DIR>/pulsar-in-action/chapter4
$ docker run --name pulsar -id \
  -p 6650:6650 -p 8080:8080 \
  -v $GIT_PROJECT:/pulsar/dropbox
  apache/pulsar:pulsar:latest bin/pulsar standalone

```

Обычно имеется возможность запуска теста `LocalRunner` из используемой IDE (интерактивной среды разработки), чтобы подключить отладчик и выполнить пошаговый проход по коду функции для обнаружения и устранения всех возможных ошибок. Но в текущем сценарии я предпочел выполнить тест `LocalRunner` из командной строки. Из этого следует, что необходимо сначала включить тестовый класс, размещенный в каталоге `chapter4/src/main/test` репозитория GitHub этой книги, в JAR-файл вместе со всеми требуемыми зависимостями, включая класс `LocalRunner`, выполнив команду сборки Maven. После завершения сборки можно приступить к выполнению команд `LocalRunner`, показанных в листинге 4.17.

Листинг 4.17. Запуск `LocalRunner` и ввод некоторых данных

```
mvn clean compile test-compile assembly:single ❶
...
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 29.279 s
[INFO] Finished at: 2020-08-15T15:43:58-07:00

java -cp ./target/chapter4-0.0.1-fat-tests.jar
com.manning.pulsar.chapter4.functions.sdk.KeywordFilter
    ➔ FunctionLocalRunnerTest Director ❷

org.apache.pulsar.functions.runtime.thread.ThreadRuntime - ThreadContainer
    ➔ starting function with instance config InstanceConfig(instanceId=0,
    ➔ functionId=0bc39b7d-fb08-4549-a6cf-ab641d583edd,
    ➔ functionVersion=7786da28-
    0bb6-4c11-97d9-3d6140cc4261, functionDetails=tenant: "public"
    namespace: "default"
    name: "keyword-filter"
    className: "com.manning.pulsar.chapter4.functions.
    ➔ sdk.KeywordFilterFunction" ❸
    userConfig: "{\\"keyword\\":\\"Director\\",\\"ignore-case\\":true}"
    autoAck: true
    parallelism: 1
    source { ❹
        typeClassName: "java.lang.String"
        subscriptionName: "keyword-filter-sub"
        inputSpecs {
            key: "persistent://public/default/raw-feed"
            value {
                schemaType: "String"
            }
        }
        cleanupSubscription: true
    }
```

```

sink {
  topic: "persistent://public/default/filtered-feed"
  typeClassName: "java.lang.String"
  forwardSourceMessageProperty: true
}
. . .

```

5

Message received: At the end of the room a loud speaker projected from the
 ➤ wall. The Director walked up to it and pressed a switch. 6

Message received: The Director pushed back the switch. The voice was
 ➤ silent. Only its thin ghost continued to mutter from beneath the
 ➤ eighty pillows.

Message received: Once more the Director touched the switch.

- 1 Создание самодостаточного выполняемого JAR-файла (fat jar), содержащего тест `LocalRunner` и все его зависимости.
- 2 Запуск теста `LocalRunner` и определение ключевого слова `Director` для фильтрации.
- 3 Выходные данные должны показать, что функция была развернута, как ожидалось.
- 4 Этот вывод должен отображать входящую тему функции.
- 5 Этот вывод должен отображать исходящую тему функции.
- 6 Выходные данные должны содержать только предложения с ключевым словом `Director`.

Рассмотрим по шагам, что здесь происходило. Экземпляр `KeywordFilterFunction` был запущен локально в среде JVM на моем ноутбуке и установил соединение с экземпляром Pulsar, работающим в контейнере Docker, который был активизирован ранее. Далее входящая и исходящая темы, определенные в конфигурации функции, были автоматически созданы в этом экземпляре Pulsar. Затем все данные, созданные производителем, работающим внутри метода `sendData` объекта `KeywordFilterFunctionLocalRunnerTest`, были опубликованы в соответствующей теме Pulsar внутри контейнера Docker.

Функция `KeywordFilterFunction` прослушивала ту же самую тему, и логика обработки применялась к каждой строке, опубликованной в этой теме. Только сообщения, содержащие ключевое слово, в данном случае `Director`, записывались в исходящую тему, сконфигурированную для функции. Потребитель, работающий внутри метода `startConsumer` объекта `KeywordFilterFunctionLocalRunnerTest`, считывал сообщения из исходящей темы и направлял их в поток стандартного вывода, чтобы мы могли проверить результаты.

4.5. Развертывание функций Pulsar

После компиляции и тестирования необходимо выполнить развертывание функций Pulsar в производственном кластере. Для этой цели

специально предназначено инструментальное средство командной строки `pulsar-admin functions`, которое позволяет предоставить для функции некоторые свойства конфигурации, в том числе абонента, пространство имен, входящую и исходящую темы. В этом разделе подробно рассматривается процесс конфигурирования и развертывания функции Pulsar с использованием `pulsar-admin functions`.

4.5.1. Генерация артефакта развертывания

При развертывании функции Pulsar первым шагом является генерация артефакта развертывания, содержащего код функции вместе со всеми ее зависимостями. Тип артефакта изменяется в зависимости от языка программирования, используемого для разработки функции.

Функции Java SDK

Для функций на языке Java предпочтительным типом артефакта является файл NAR (NiFi archive), хотя JAR-файлы также приемлемы, если включают все требуемые зависимости. Это условие остается в силе независимо от того, хотите ли вы развернуть простую функцию, написанную только на языке Java, или функцию, разработанную с использованием Pulsar SDK. В любом случае файл архива представляет собой артефакт развертывания. Чтобы упаковать функцию Pulsar как NAR-файл, необходимо включить специальный подключаемый модуль (plugin) в файл `pom.xml`, как показано в листинге 4.18. Этот модуль объединит класс функции со всеми его зависимостями.

Листинг 4.18. Добавление подключаемого модуля NAR Maven в файл `pom.xml`

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.nifi</groupId>
      <artifactId>nifi-nar-maven-plugin</artifactId>
      <version>1.2.0</version>
      <extensions>true</extensions>
      <executions>
        <execution>
          <phase>package</phase>
          <goals>
            <goal>nar</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
    . . .
  </plugins>
</build>
```


После добавления этого подключаемого модуля в файл проекта требуется всего лишь выполнить команду `mvn clean install` для генерации NAR-файла в целевом каталоге проекта. Созданный NAR-файл – это артефакт развертывания для функции Pulsar, написанной на языке Java.

Функции Python SDK

Для функций Pulsar, написанных на языке Python, существуют два варианта развертывания в зависимости от того, использовали вы Pulsar SDK или нет. Если вы обошлись без SDK и хотите развернуть функцию на чистом языке Python, то файл с исходным кодом (например, *my-function.py*) и есть артефакт развертывания, не требующий какой-либо дополнительной упаковки.

Но если необходимо развернуть функцию Pulsar на языке Python, зависимую от пакетов, не входящих в стандартные библиотеки, то сначала придется упаковать все требуемые зависимости в единый артефакт (ZIP-файл), чтобы обеспечить возможность развертывания функции. В каталоге проекта Python обязательно должен существовать файл *requirements.txt*, используемый для сопровождения списка всех зависимостей проекта. Разработчик должен вручную поддерживать актуальность содержимого этого файла. Следует отметить, что `pulsar-client` не требует включения в этот список в качестве зависимости, так как предоставляется фреймворком Pulsar. После подготовки всего необходимого для создания артефакта развертывания на языке Python сначала нужно выполнить команду, показанную в листинге 4.19, чтобы загрузить все зависимости, перечисленные в файле *requirements.txt*, в каталог *deps* проекта Python.

Листинг 4.19. Загрузка зависимостей для функции на языке Python

```
pip download \
--only-binary :all: \
--platform manylinux1_x86_64 \
--python-version 37 \
-r requirements.txt \
-d deps
```

❶
❷
❸
❹
❺
❻

- ❶ Использование менеджера пакетов `pip` для Python.
- ❷ Необходимы только двоичные версии всех зависимостей.
- ❸ Определение операционной системы, управляющей целевой платформой выполнения.
- ❹ Определение версии Python.
- ❺ Относительный путь для файла *requirements.txt*.
- ❻ Целевой каталог загрузки.

После завершения загрузки необходимо создать целевой каталог с требуемым именем пакета (например, *echo-function*). Далее вы должны скопировать полностью оба каталога *src* и *deps* в только что создан-

ный каталог и упаковать его в ZIP-архив командой, показанной последней в листинге 4.20. Полученный ZIP-файл является артефактом развертывания для функции Pulsar, написанной на языке Python.

Листинг 4.20. Упаковка функции Pulsar на языке Python для развертывания

```
mkdir -p /tmp/echo-function ❶

cp -R deps /tmp/echo-function/
cp -R src /tmp/echo-function/ ❷

zip -r /tmp/echo-function.zip /tmp/echo-function ❸
```

- ❶ Создание нового целевого каталога для исходного кода и зависимостей функции.
- ❷ Копирование всех зависимостей и исходного кода из каталога проекта Python.
- ❸ Выполнение команды `zip` для создания ZIP-файла, включающего содержимое целевого каталога.

Функции Golang SDK

Для функций Pulsar, написанных на языке Golang, предпочтительным типом артефакта является единственный двоичный выполняемый файл, содержащий машинный байт-код функции, объединенный с полным кодом поддержки, необходимым для обеспечения выполнения на любом компьютере независимо от существования в системе файлов исходного кода `.go` или даже установленной рабочей среды Go. Для генерации такого двоичного выполняемого файла можно воспользоваться инструментальным комплектом Go, как показано в листинге 4.21.

Листинг 4.21. Упаковка функции Pulsar на языке Go для развертывания

```
cd chapter4 ❶

go build echoFunction.go ❷

go: downloading github.com/apache/pulsar/pulsar-function-go
➔ v0.0.0-20210723210639-251113330b08
go: downloading github.com/sirupsen/logrus v1.4.2
go: downloading github.com/golang/protobuf v1.4.3
go: downloading github.com/apache/pulsar-client-go v0.5.0
... ❸

ls -l ❹

-rwxr-xr-x 1 david staff 23344912 Jul 25 15:23 echoFunction ❺
-rw-r--r-- 1 david staff 538 Jul 25 15:20 echoFunction.go ❻
```

- ❶ Необходимо перейти в каталог проекта.
- ❷ Использование команды сборки кода Go для генерации двоичного выполняемого файла.
- ❸ Go автоматически загружает все требуемые пакеты и включает их в двоичный файл.
- ❹ После завершения сборки следует проверить содержимое текущего каталога.
- ❺ Сгенерированный выполняемый двоичный файл.
- ❻ Файл исходного кода Go, содержащий функцию Pulsar.

Если вы работаете в среде macOS или Linux, то двоичный выполняемый файл будет сгенерирован в том каталоге, где выполнена команда, и ему присваивается имя, соответствующее файлу исходного кода. Это и есть артефакт, который необходимо развернуть для выполнения функции на языке Go в рабочей среде Pulsar.

4.5.2. Конфигурация функции

Теперь мы знаем, как сгенерировать артефакт развертывания, и следующим шагом является конфигурирование и развертывание функций Pulsar. Для всех функций требуется предоставление некоторых основных деталей конфигурации при развертывании в среде Pulsar, например входящие и исходящие темы и т. п. Существуют два способа определения этой конфигурационной информации. Первый способ – аргументы командной строки, передаваемые в инструментальное средство `pulsar-admin functions` с помощью специально предназначенных для функций команд `create` (<https://pulsar.apache.org/docs/en/functions-cli/#create>) и `update` (<https://pulsar.apache.org/docs/en/functions-cli/#update>) (например, `bin/pulsar-admin functions create`). Второй способ – использование единого файла конфигурации.

Вы можете выбрать любой способ по своему усмотрению, но я настоятельно рекомендую использовать второй, так как он не только упрощает процесс разработки, но также позволяет сохранять подробности конфигурации в системе управления версиями вместе с файлами исходного кода. Это обеспечивает доступ к свойствам в любое время и гарантирует, что вы всегда будете выполнять свои функции с корректной конфигурацией. Если выбрать этот подход, то потребуется предоставить только два значения параметров конфигурации для развертывания функций: имя файла артефакта функции, содержащего ее выполняемый код, и имя файла, содержащего все параметры конфигурации. Содержимое файла конфигурации с именем `function-config.yaml` показано в листинге 4.22. Мы используем этот файл для развертывания функции `KeywordFilterFunction` в кластере Pulsar, работающем в контейнере Docker, активизированном в начале текущей главы.

Листинг 4.22. Файл конфигурации для функции KeywordFilterFunction

```

className: com.manning.pulsar.chapter4.functions.sdk.KeywordFilterFunction
tenant: public
namespace: default
name: keyword-filter
inputs:
- persistent://public/default/raw-feed
output: persistent://public/default/filtered-feed
userConfig:
  keyword : Director
  ignore-case: false

#####
# Processing
#####
autoAck: true
logTopic: persistent://public/default/keyword-filter-log
processingGuarantees: ATLEAST_ONCE
retainOrdering: false
timeoutMs: 30000
subName: keyword-filter-sub
cleanupSubscription: true

```

- ❶ С каждой функцией обязательно должен быть связан абонент.
- ❷ С каждой функцией обязательно должно быть связано пространство имен.
- ❸ Для функции можно определить более одной входящей темы.
- ❹ Конфигурация пользователя: отображение ключ–значение.
- ❺ Должна ли функция подтверждать сообщение после его обработки.
- ❻ Семантика доставки, применяемая к функции.
- ❼ Должна ли функция обрабатывать входящие сообщения в порядке их публикации.
- ❽ Имя подписки во входящей теме.

Здесь можно видеть, что файл конфигурации предоставляет все свойства, необходимые для выполнения функции Pulsar, включая имя класса, входящие/исходящие каталоги (темы) и многое другое. Файл также содержит следующие параметры настройки, управляющие тем, как сообщения потребляются функцией:

- `auto_ack` – должно ли сообщение, потребляемое из входящей темы (входящих тем) автоматически подтверждаться фреймворком функции. При установке значения по умолчанию `true` каждое сообщение, при обработке которого не возникло исключение, автоматически подтверждается после завершения выполнения функции. Если установлено значение `false`, то за подтверждение сообщения отвечает код функции;
- `retain-ordering` – должны ли сообщения потребляться точно в том порядке, в котором они появляются во входящей теме.

При установке значения `true` отрицательно подтвержденные сообщения будут повторно доставляться раньше всех необработанных сообщений;

- `processing-guarantees` – гарантии обработки (семантика доставки), применяемые к сообщениям, потребляемым функцией. Возможные значения: `[ATLEAST_ONCE, ATMOST_ONCE, EFFECTIVELY_ONCE]`.

При выполнении приложения потоковой обработки, возможно, потребуется определение семантики доставки для обработки данных в функциях Pulsar. Эти гарантии являются осмысленными, поскольку вы всегда обязаны предполагать возможность критических сбоев в сети или на компьютерах, которые могут привести к потере данных.

В контексте Pulsar Functions эти гарантии обработки определяют, как часто обрабатывается сообщение и как оно обрабатывается в случае возникновения ошибки. Механизм Pulsar Functions поддерживает три семантики доставки сообщений, которые можно применять к любой функции. По умолчанию Pulsar Functions предоставляет гарантии доставки `at-most-once` (максимум однажды), но можно сконфигурировать функцию Pulsar для предоставления любой другой семантики обработки сообщений, которые подробно описаны в следующих подразделах.

Гарантии доставки `at-most-once` (максимум однажды)

Обработка `at-most-once` (максимум однажды) не дает каких-либо гарантий того, что данные обработаны, и не предоставляет дополнительных попыток повторной обработки данных, если они потеряны, прежде чем функция Pulsar смогла их обработать. Каждое сообщение, передаваемое в функцию, либо обрабатывается не более одного раза, либо не обрабатывается вообще.

Как можно видеть на рис. 4.6, если функция Pulsar сконфигурирована для применения обработки `at-most-once`, то сообщение подтверждается сразу же после потребления независимо от того, было ли оно успешно обработано или нет. В этом сценарии сообщение M2 будет обработано следующим в функции, даже если при обработке сообщения M1 возникло исключение.

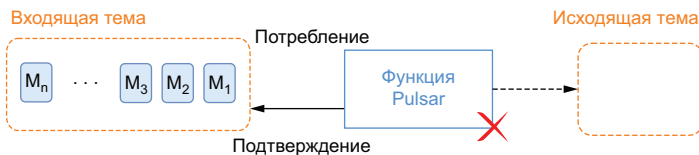


Рис. 4.6. При обработке `at-most-once` входящие сообщения подтверждаются независимо от успешной или ошибочной обработки. Такой подход дает каждому сообщению только один шанс на обработку

Эту гарантию обработки следует использовать только в тех случаях, если приложение может обрабатывать потери данных без отрицательного влияния на их корректность. Один из примеров – функция вычисления среднего значения показаний датчика, например температуры. В течение своего времени жизни функция обрабатывает сотни миллионов отдельных показаний температуры, и потеря нескольких элементов данных никак не повлияет на точность вычисления среднего значения.

Гарантии доставки at-least-once (минимум однажды)

Обработка at-least-once (минимум однажды) гарантирует, что любые данные, опубликованные в одной из входящих тем функции, будут успешно обработаны как минимум один раз. При возникновении ошибки во время обработки сообщение будет автоматически доставлено повторно. Таким образом, существует вероятность того, что любое конкретное сообщение может быть обработано более одного раза.

При этом типе семантики обработки функция Pulsar считывает сообщение из входящей темы, выполняет свою логику и подтверждает сообщение. Если в функции возникает ошибка при подтверждении сообщения, то оно обрабатывается повторно. Процесс повторяется до тех пор, пока функция не подтвердит сообщение. На рис. 4.7 показан сценарий, в котором функция потребляет сообщение, но при обработке возникает ошибка, из-за которой становится невозможной передача подтверждения. В таком сценарии следующим обрабатываемым сообщением должно стать M_1 , и его обработка будет продолжаться до тех пор, пока функция не достигнет успеха.

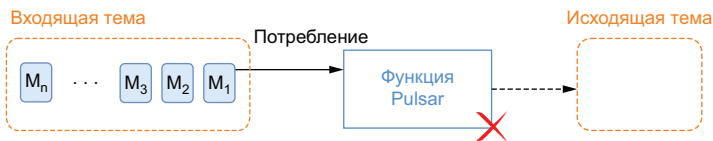


Рис. 4.7. При обработке at-least-once, если в функции возникает ошибка, из-за которой невозможно отправить подтверждение, то же самое сообщение будет обработано повторно

Эту гарантию обработки следует использовать, если приложение может выполнять обработку одних и тех же данных несколько раз без отрицательного влияния на их корректность. Один из сценариев – входящие сообщения представляют записи, которые должны быть обновлены в базе данных. В таком сценарии многократное обновление одних и тех же значений не оказывает отрицательного воздействия на целевую базу данных.

Гарантии доставки *effectively-once* (фактически однажды)

При гарантии *effectively-once* (фактически однажды) сообщение может быть принято и обработано более одного раза, что является общепринятой практикой при наличии критических сбоев. Но самое важное здесь заключается в том, что действительный результат обработки функцией в итоговом состоянии будет таким, как если бы повторно обрабатываемые события наблюдались только один раз. Чаще всего именно такой результат и нужно получить.

На рис. 4.8 показан сценарий, в котором производитель повторно передал сообщение M1 во входящую тему функции. Если сконфигурирована обработка *effectively-once*, то функция проверяет, не было ли ранее обработано это сообщение (на основе свойств, определенных пользователем), и если обработка выполнена, то сообщение игнорируется и отправляется подтверждение, так что повторная обработка не происходит.

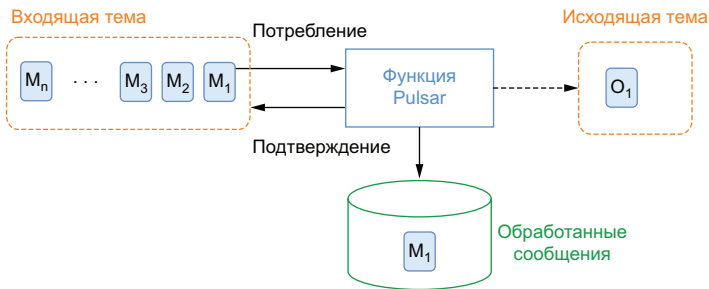


Рис. 4.8. При обработке *effectively-once* дублирующиеся сообщения игнорируются

4.5.3. Развертывание функции

После определения конфигурации, требуемой для использования функции, следующим шагом является ее развертывание в работающем кластере Pulsar. Как было отмечено выше, кластер Pulsar, используемый в примерах этой главы, работает в контейнере Docker и вполне подходит для этой цели. Из командной оболочки *bash* (в среде Linux) можно выполнить команду, показанную в листинге 4.23, для развертывания функции *KeywordFilterFunction* в кластере Pulsar.

Листинг 4.23. Развертывание функции *KeywordFilterFunction*

```

$ docker exec -it pulsar bin/pulsar-admin functions create \
  --jar /pulsar/dropbox/target/chapter4-0.0.1.nar \
  --function-config-file /pulsar/dropbox/src/main/resources/
  function-config.yaml
  
```

1

2

3

- ❶ Используется инструмент `pulsar-admin` внутри контейнера Docker.
- ❷ Определяется артефакт развертывания, который должен быть либо локально доступным файлом, либо URL-адресом.
- ❸ Определяется файл конфигурации функции, который должен быть либо локально доступным файлом, либо URL-адресом.

Если все прошло как ожидалось, то вы должны увидеть вывод этой команды `Created successfully`, означающий, что функция была создана и развернута в кластере Pulsar. Рассмотрим немного подробнее, что произошло и как работает эта команда. В первую очередь следует отметить, что мы использовали инструмент командной строки `pulsar-admin`, существующий в данном случае только внутри контейнера Docker. Следовательно, потребовалась команда `docker exec` для выполнения команды развертывания внутри контейнера. Для команды развертывания были предоставлены два ключа: первый для определения локации артефакта развертывания (в данном случае NAR-файла), второй для определения локации файла конфигурации функции. Поскольку развертывается функция Pulsar на языке Java, для определения локации артефакта использовался ключ `--jar`. Если бы это была функция на языке Python, то вместо него потребовался бы ключ `--py`, а для функции на языке Golang – ключ `--go`. Указание правильного ключа чрезвычайно важно, потому что Pulsar использует эти значения, чтобы определить, какая среда времени выполнения необходима для работы функции (т. е. требуется активизация JVM, интерпретатора Python или среда времени выполнения Go для конкретной развертываемой функции).

Файлы артефакта развертывания и конфигурации функции должны располагаться на том же компьютере, что и инструмент `pulsar-admin`, или должны загружаться через URL. Поэтому каталог `$GIT_PROJECT` был смонтирован в каталог `/pulsar/dropbox` внутри контейнера Docker. Это сделало доступными оба файла локально доступными для инструмента `pulsar-admin`. Хотя это неплохой прием для локальной разработки, всегда следует помнить о том, что в реальном производственном сценарии эти файлы непременно должны быть физически перемещены в кластер Pulsar, лучше всего как часть процесса CI/CD (непрерывной интеграции/непрерывного развертывания). Также можно проверить состояние развернутой функции с помощью команды `function getstatus`, как показано в листинге 4.24. Эта команда предоставляет полезную информацию о функции, в том числе ее состояние, количество обработанных сообщений, значение средней задержки при обработке и все исключения, которые функция, возможно, сгенерировала.

Листинг 4.24. Проверка состояния функции KeywordFilterFunction

```
# docker exec -it pulsar /pulsar/bin/pulsar-admin functions getstatus -
➔ name keyword-filter
{
  "numInstances" : 1,
  "numRunning" : 1,
  "instances" : [ {
    "instanceId" : 0,
    "status" : {
      "running" : true,
      "error" : "",
      "numRestarts" : 0,
      "numReceived" : 0,
      "numSuccessfullyProcessed" : 0,
      "numUserExceptions" : 0,
      "latestUserExceptions" : [ ],
      "numSystemExceptions" : 0,
      "latestSystemExceptions" : [ ],
      "averageLatency" : 0.0,
      "lastInvocationTime" : 0,
      "workerId" : "c-standalone-fw-localhost-8080"
    }
  } ]
}
```

- ❶ Количество требуемых экземпляров функции.
- ❷ Количество действительно работающих экземпляров функции.
- ❸ Индикатор состояния.
- ❹ Если сгенерированы исключения (ошибки), то они описываются здесь.
- ❺ Количество сообщений, принятых функцией.
- ❻ Количество сообщений, успешно обработанных функцией.
- ❼ Среднее значение задержки при обработке сообщений.
- ❽ Время последней обработки сообщения функцией.

Если потребуется отладка работающей функции Pulsar, то команда `function getstatus` является превосходным отправным пунктом. Инструментальное средство `pulsar-admin functions` предоставляет еще одну команду, показывающую все параметры конфигурации функции Pulsar. Проверить конфигурацию функции Pulsar можно командой `function get`, как показано в листинге 4.25.

Листинг 4.25. Проверка конфигурации функции KeywordFilterFunction

```
# docker exec -it pulsar /pulsar/bin/pulsar-admin functions get --name
➔ keyword-filter
{
  "tenant": "public",
  "namespace": "default",
  "name": "keyword-filter",
```

```

"className":
➡ "com.manning.pulsar.chapter4.functions.sdk.KeywordFilterFunction",
"inputSpecs": {
  "persistent://public/default/raw-feed": {
    "isRegexPattern": false,
    "schemaProperties": {}
  }
},
"output": "persistent://public/default/filtered-feed",
"logTopic": "persistent://public/default/keyword-filter-log",
"processingGuarantees": "ATLEAST_ONCE",
"retainOrdering": false,
"forwardSourceMessageProperty": true,
"userConfig": {
  "keyword": "Director",
  "ignore-case": false
},
"runtime": "JAVA",
"autoAck": true,
"subName": "keyword-filter-sub",
"parallelism": 1,
"resources": {
  "cpu": 1.0,
  "ram": 1073741824,
  "disk": 10737418240
},
"timeoutMs": 30000,
"cleanupSubscription": true
}

```

- ➊ Входящая тема (темы).
- ➋ Исходящая тема.
- ➌ Функция гарантирует обработку.
- ➍ Должны ли сообщения обрабатываться в порядке их публикации в теме.
- ➎ Любые значения пользовательской конфигурации.
- ➏ Среда времени выполнения функции (например, Java, Python и т. п.).
- ➐ Должна ли функция автоматически подтверждать сообщения.
- ➑ Количество требуемых экземпляров функции, работающих параллельно.
- ➒ Вычислительные ресурсы, выделенные для функции.

4.5.4. Жизненный цикл развертывания функции

Как мы видели в предыдущем подразделе, существует несколько параметров конфигурации, доступных при создании и обновлении функции. В этом подразделе мы рассмотрим, как используются некоторые из этих параметров в жизненном цикле развертывания функции Pulsar. Когда функция создается впервые, связанный с ней набор библиотек сохраняется в Apache BookKeeper, и к нему может получить доступ любой узел брокера Pulsar в кластере. Такой набор связан с полностью квалифицированным именем функции, которое является сочетанием абонента, пространства имен

и имени самой функции и гарантирует глобальную однозначность во всем кластере Pulsar.

При изначальном создании функции Pulsar последовательно выполняются шаги, показанные на рис. 4.9. После того как функция зарегистрирована и все подробности о ней сохранены в BookKeeper, создаются работники (рабочие процессы) функции на основе предоставленных параметров конфигурации и на режиме развертывания кластера Pulsar (который будет рассматриваться в следующем разделе). Работники функции (function workers) – это экземпляры кода времени выполнения функции Pulsar, которыми могут быть потоки, процессы или поды (pods) Kubernetes. После этого в каждом работнике функции создается потребитель Pulsar и подписка в сконфигурированных входящих темах. Затем работники функции ожидают прибытия входящих сообщений и выполняют для них логику обработки.

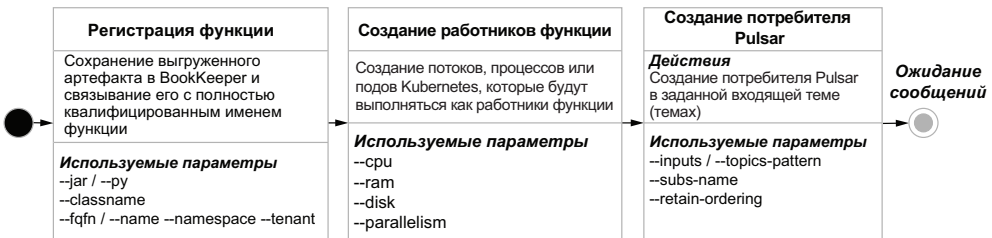


Рис. 4.9. Жизненный цикл развертывания функции Pulsar

4.5.5. Режимы развертывания

Как мы уже видели, для развертывания и управления функциями Pulsar необходим работающий кластер Pulsar, но при этом существует несколько вариантов того, где именно будут работать экземпляры функции Pulsar. Можно выполнять функцию Pulsar на своем локальном компьютере разработки (режим `localrun`), тогда она взаимодействует с брокером через сеть. Так было сделано в варианте тестирования с использованием `LocalRunner`. Для производственных целей потребуется развертывание функций в режиме кластера (`cluster mode`).

Режим кластера

В этом режиме пользователи передают свои функции в работающий кластер, а фреймворк Pulsar позаботится об их распределении по работникам функций для выполнения. Фреймворк Pulsar Functions поддерживает следующие варианты среды времени выполнения при работе в режиме кластера: поток, процесс и Kubernetes, как показано на рис. 4.10. Эти варианты определяют, как сам код функции выполняется внутри работника функции, представляющего собой среду времени выполнения, используемую для управления функциями Pulsar.

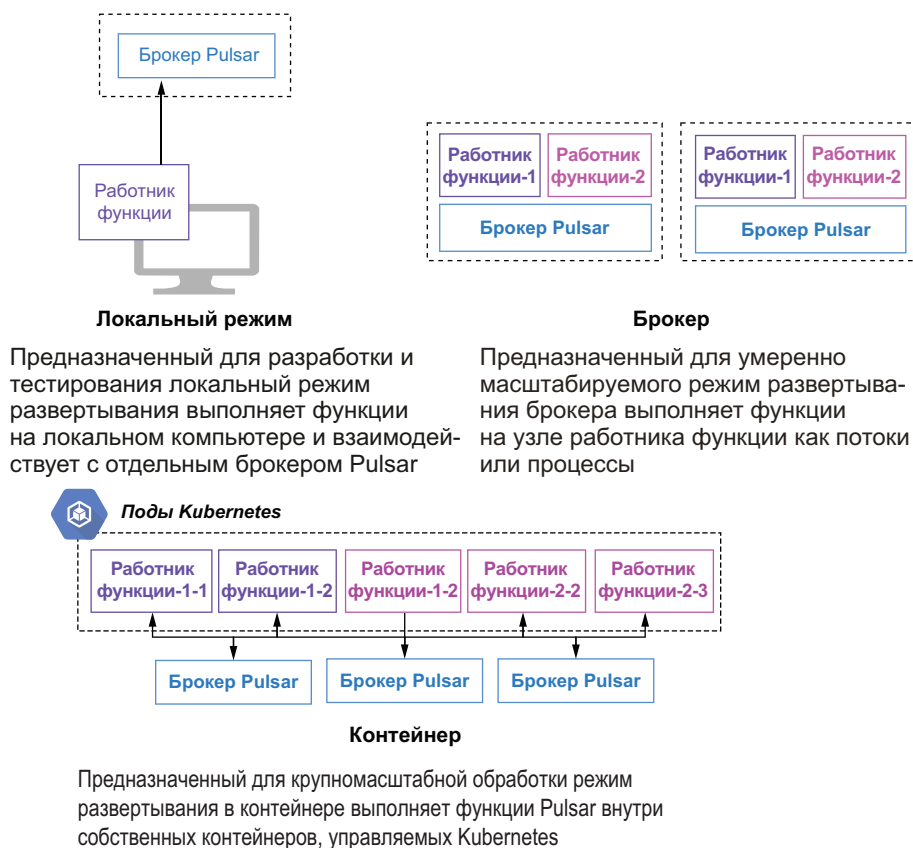


Рис. 4.10. Функция Pulsar может работать как поток, процесс или Kubernetes StatefulSet внутри работников функций Pulsar

В Pulsar существуют два варианта, определяющих, где могут выполняться работники функций. Первый вариант – выполнение внутри самих брокеров Pulsar, что упрощает развертывание. Вторым – запуск процессов работников функций на собственных отдельных узлах для обеспечения улучшенной изоляции ресурсов между ними и брокерами.

Преимущество выполнения работника функции на узле самого брокера как потока или отдельного процесса заключается в том, что он требует меньшего объема аппаратной памяти, поскольку нет необходимости в существовании множества узлов в рабочей среде, а также в сокращении сетевой задержки между процессами функции и брокером, обслуживающим сообщения. Тем не менее выполнение работников функций на отдельных узлах обеспечивает более надежную изоляцию ресурсов и защищает процесс брокера Pulsar от непредвиденного уничтожения и полной потери работоспособности при аварии работника функции.

4.5.6. Поток данных в функции Pulsar

Прежде чем завершить эту главу, я хотел бы уделить немного времени документированию маршрута прохождения отдельного сообщения через функцию Pulsar и связыванию различных этапов этого маршрута с параметрами конфигурации, предоставленными при создании или обновлении функции, чтобы вы лучше понимали, как следует конфигурировать функции. Поток данных внутри функции Pulsar изображен в виде конечного автомата на рис 4.11 вместе со свойствами конфигурации, управляющими поведением функции.

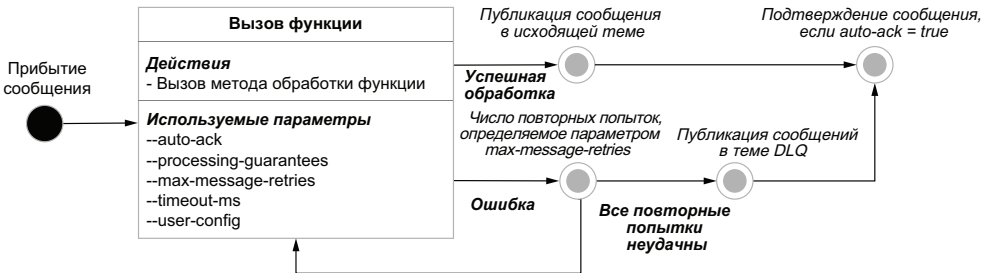


Рис. 4.11. Обычный маршрут прохождения сообщения в функции Pulsar, выполняющейся внутри работника

Когда сообщение прибывает в любую входящую тему, сконфигурированную для функции, вызывается метод обработки этой функции с содержимым сообщения в качестве входного параметра. Если функция может успешно обработать сообщение без возникновения каких-либо исключений времени выполнения, то значение, возвращаемое методом обработки, публикуется в сконфигурированную исходящую тему, если только функция не определена как `void` (т. е. не возвращающая значений), и в этом случае ничего не публикуется.

Но если в функции возникают исключения времени выполнения, то выполняются повторные попытки обработки сообщения, количество которых определено параметром конфигурации `max-message-retries`. Если все эти попытки неудачны, то сообщение перенаправляется в сконфигурированную тему `dead-letter-queue` (если она существует), т. е. сохраняется для дальнейшего исследования. В любом случае сообщение подтверждается как потребленное функцией Pulsar, если для флага `auto-ack` задано значение `true`, что позволяет приступить к обработке следующего сообщения.

4.6. Резюме

Pulsar Functions – это вычислительный фреймворк без сервера, работающий поверх Apache Pulsar и позволяющий определять логику обработки, которая выполняется, когда в тему прибывает новое сообщение.

- Функции Pulsar можно писать на нескольких широко распространенных языках программирования, в том числе на Python, Go и Java, но в оставшейся части книги мы будем работать только с языком Java.
- Функции Pulsar можно конфигурировать, устанавливать и контролировать с помощью интерфейса командной строки Pulsar.
- Функции Pulsar могут быть развернуты для выполнения как потоки, процессы или поды Kubernetes.

5

Коннекторы ввода-вывода Pulsar

Темы:

- введение в фреймворк ввода-вывода Pulsar;
- конфигурирование, развертывание и мониторинг коннекторов ввода-вывода Pulsar;
- написание собственного коннектора ввода-вывода Pulsar на языке Java.

Системы обмена сообщениями гораздо более полезны, если их можно с легкостью использовать для перемещения данных между другими внешними системами, например базами данных, локальными и распределенными файловыми системами или прочими системами обмена сообщениями. Рассмотрим сценарий, в котором необходимо принимать журнальные данные из внешних источников – приложений, платформ и облачных сервисов – и публиковать их для анализа с помощью некоторого механизма поиска. Эта задача легко решается с помощью пары коннекторов ввода-вывода Pulsar – первый должен стать источником, собирающим журнальные записи приложений, а второй – приемником, который записывает отформатированные записи в Elasticsearch.

Pulsar предоставляет набор предварительно встроенных коннекторов, которые можно использовать для взаимодействия с внешними системами, такими как Apache Cassandra, Elasticsearch и HDFS и многими другими. Кроме того, фреймворк Pulsar IO является расширяемым, что позволяет разрабатывать собственные коннекторы для поддержки новых или давно используемых систем, если это необходимо.

5.1. Что такое коннекторы ввода-вывода Pulsar

Фреймворк коннекторов ввода-вывода Pulsar предоставляет разработчикам, инженерам по обработке данных и операторам простой способ перемещения данных с помощью платформы обмена сообщениями Pulsar без необходимости написания какого-либо кода или приобретения навыков эксперта по использованию Pulsar совместно с конкретной внешней системой. С точки зрения реализации коннекторы ввода-вывода Pulsar представляют собой просто специализированные функции Pulsar, предназначенные исключительно для организации взаимодействия с внешними системами через расширяемый API.

Сравните этот подход со сценарием, в котором необходимо реализовать логику взаимодействия с внешней системой, например MongoDB, в классе Java, который использует Java-клиент Pulsar. Вы должны не только хорошо знать интерфейс клиента для MongoDB и Pulsar, но также принять на себя функциональные обязанности по развертыванию и мониторингу отдельного процесса, который сразу же становится весьма важной частью стека приложения. Фреймворк ввода-вывода Pulsar позволяет сделать перемещение данных через Pulsar менее запутанным и сложным.

Коннекторы ввода-вывода Pulsar делятся на два типа: источники (sources), передающие данные из внешних систем в Pulsar, и приемники (sink), отправляющие данные из Pulsar во внешние системы. На рис. 5.1 показаны отношения между источниками, приемниками и Pulsar.

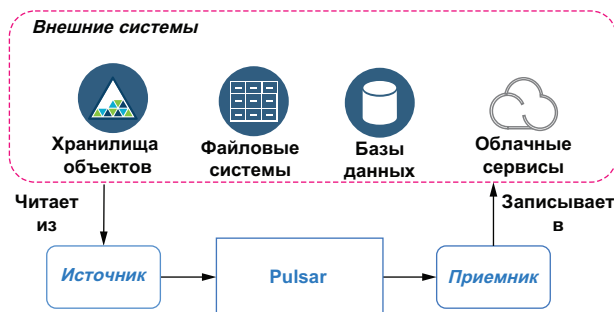


Рис. 5.1. Источники потребляют данные из внешних систем, а приемники записывают данные во внешние системы

5.1.1. Коннекторы-приемники

Хотя фреймворк ввода-вывода Pulsar уже предоставляет набор встроенных коннекторов для некоторых из наиболее широко распространенных систем обработки данных, он был изначально спроектирован с возможностями расширения, позволяя пользователям добавлять новые коннекторы по мере разработки новых систем и API. Модель программирования, лежащая в основе фреймворка Pulsar IO Connectors, чрезвычайно проста, что существенно облегчает процесс разработки. Коннекторы-приемники Pulsar IO могут принимать сообщения из одной или нескольких входящих тем. При каждой публикации сообщения в любой входящей теме вызывается метод приемника Pulsar write.

Реализация метода write отвечает за определение способа обработки содержимого входящего сообщения и свойств операции записи данных во внешнюю систему. Приемник Pulsar, показанный на рис. 5.2, может использовать содержимое сообщения для определения таблицы базы данных, в которую должна вставляться запись, а затем формирует и выполняет соответствующую команду SQL для вставки записи.

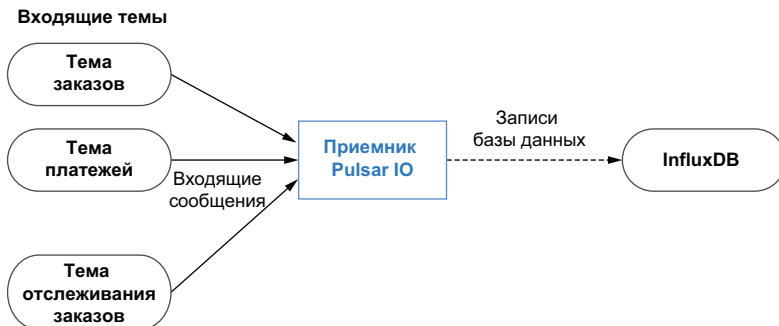


Рис. 5.2. Общая схема модели программирования коннектора-приемника Pulsar IO

Самый простой способ создания специализированного коннектора-приемника – написание класса Java, который реализует интерфейс `org.apache.pulsar.io.core.Sink`, как показано в листинге 5.1. Первым в этом интерфейсе реализован метод `open`, вызываемый только однократно при создании приемника. Этот метод можно использовать для инициализации всех необходимых ресурсов (например, для коннектора с базой данных можно создать JDBC-клиента). Метод `open` также предоставляет один входной параметр `config`, из которого можно извлечь все настройки, предназначенные специально для коннектора (например, URL для соединения с базой данных, имя пользователя и пароль). В дополнение к передаваемому объекту `config` механизм времени выполнения Pulsar также предоставляет для коннектора `SinkContext`, который обеспе-

чивает доступ к ресурсам времени выполнения и во многом похож на объект `Context` из API Pulsar Functions.

Листинг 5.1. Интерфейс приемника Pulsar

```
package org.apache.pulsar.io.core;

public interface Sink<T> extends AutoCloseable {
    /**
     * Открытие коннектора с заданной конфигурацией.
     *
     * @param config -- инициализация конфигурации.
     * @param sinkContext.
     * @генерирует исключения типа Exception IO при открытии коннектора.
     */
    void open(final Map<String, Object> config,
              SinkContext sinkContext) throws Exception;

    /**
     * Запись сообщения в приемник Sink.
     *
     * @param record для записи в приемник.
     * @генерирует исключение Exception.
     */
    void write(Record<T> record) throws Exception;
}
```

В этом интерфейсе также определен метод `write`, отвечающий за потребление сообщений из сконфигурированного для приемника темы-источника Pulsar и за запись данных во внешнюю систему. Метод `write` принимает объект, реализующий интерфейс `org.apache.pulsar.functions.api.Record`, который предоставляет информацию, используемую при обработке входящего сообщения. Следует отметить, что интерфейс приемника расширяет интерфейс `AutoCloseable`, включающий определение метода `close`, который можно использовать для освобождения любых ресурсов, например соединений с базой данных или открытых компонентов записи в файл, до остановки работы коннектора.

Листинг 5.2. Интерфейс записи Record

```
package org.apache.pulsar.functions.api

public interface Record<T> {

    default Optional<String> getTopicName() {
        return Optional.empty();
    }

    default Optional<String> getKey() {
        return Optional.empty();
    }
}
```

```

T getValue(); 3

default Optional<Long> getEventTime() { 4
    return Optional.empty();
}

default Optional<String> getPartitionId() { 5
    return Optional.empty();
}

default Optional<Long> getRecordSequence() { 6
    return Optional.empty();
}

default Map<String, String> getProperties() { 7
    return Collections.emptyMap();
}

default void ack() { 8
}

default void fail() { 9
}

default Optional<String> getDestinationTopic() { 10
    return Optional.empty();
}
}

```

- 1 Если запись взята из темы, то сообщить имя темы.
- 2 Возврат ключа, если с ним связано значение.
- 3 Извлечение реальных данных из записи.
- 4 Извлечение времени события для записи из источника.
- 5 Если запись взята из разделенного на части источника, то возвращается идентификатор раздела. Идентификатор раздела будет использоваться как часть однозначного идентификатора механизмом ввода-вывода Pulsar для обеспечения отсутствия дублирования сообщения и гарантии обработки ровно один раз.
- 6 Если запись взята из последовательного источника, то возвращается последовательность записи. Последовательность записи будет использоваться как часть однозначного идентификатора механизмом ввода-вывода Pulsar для обеспечения отсутствия дублирования сообщения и гарантии обработки ровно один раз.
- 7 Извлечение определенных пользователем свойств, прикрепленных к записи.
- 8 Подтверждение обработки записи.
- 9 Сообщает системе-источнику о том, что при обработке этой записи возникла ошибка.
- 10 Для поддержки маршрутизации сообщений по отдельности (по одному сообщению).

Реализация записи также должна предоставить два метода: `ask` и `fail`, которые будут использоваться коннектором ввода-вывода Pulsar для подтверждения обработки записей и для сообщений о сбоях при попытках обработки ошибочных записей соответственно. Ошибка при подтверждении или некорректные сообщения в коннекторе-источнике приведут к сохранению таких сообщений, что в итоге становится причиной остановки обработки коннектором из-за замедления ответной реакции.

5.1.2. Коннекторы-источники

Коннекторы-источники ввода-вывода Pulsar отвечают за потребление данных из внешних систем и публикацию в сконфигурированной исходящей теме. Существуют два различных типа источников, поддерживаемых фреймворком ввода-вывода Pulsar: первый использует модель на основе извлечения (*pull-based model*). На рис. 5.3 можно видеть, что фреймворк ввода-вывода Pulsar многократно вызывает метод `read()` коннектора Source для извлечения данных из внешнего источника в Pulsar.

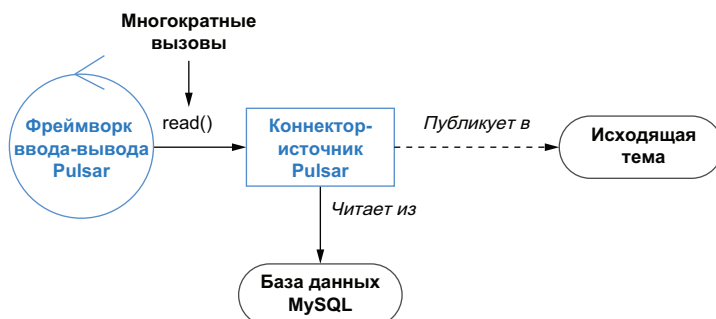


Рис. 5.3. Метод `read()` коннектора Source вызывается многократно для извлечения информации из базы данных и публикации в Pulsar

В рассматриваемом здесь примере логика внутри метода `read()` коннектора должна отвечать за выполнение запросов в базу данных, преобразование извлеченного набора записей в сообщения Pulsar и публикацию этих сообщений в исходящей теме. Такой тип коннектора особенно полезен, если приходится работать со старым приложением регистрации заказов, которое только записывает входящие заказы в базу данных MySQL, и необходимо передавать все новые заказы в другие системы для обработки и анализа в реальном времени.

Простейший способ создания коннектора-источника на основе модели извлечения – написание класса Java, реализующего интерфейс `org.apache.pulsar.io.core.Source`, как показано в листинге 5.3. Первым в этом интерфейсе реализован метод `open`, вызываемый только однократно при создании коннектора-источника. Этот ме-

тод должен использоваться для инициализации всех необходимых ресурсов, например клиента базы данных.

Для метода `open` определен входной параметр `config` типа `Map`, из которого можно извлечь все настройки, специально предназначенные для коннектора, например URL для соединения с базой данных, имя пользователя и пароль. Этот входной параметр содержит все значения, определенные в файле, указанном с помощью ключа `--source-config-file`, вместе со всеми конфигурационными настройками по умолчанию и значениями, предоставленными при помощи разнообразных ключей, используемых при создании или обновлении функции.

Листинг 5.3. Интерфейс источника Pulsar

```
package org.apache.pulsar.io.core;

public interface Source<T> extends AutoCloseable {
    /**
     * Открытие источника с заданной конфигурацией.
     *
     * @param config -- инициализация конфигурации.
     * @param sourceContext.
     * @генерирует исключения типа Exception IO при открытии коннектора.
     */
    void open(final Map<String, Object> config,
              SourceContext sourceContext) throws Exception;

    /**
     * Чтение очередного сообщения из источника.
     * Если источник не содержит новых сообщений, то этот вызов должен
     * блокироваться.
     * @возвращает следующее сообщение из источника. Результат никогда
     * не должен быть нулевым.
     * @генерирует исключение Exception.
     */
    Record<T> read() throws Exception;
}
```

В дополнение к передаваемому объекту `config` механизм времени выполнения Pulsar также предоставляет для коннектора объект `SourceContext`, который во многом похож на объект `Context` из API Pulsar Functions, и предоставляет доступ к ресурсам времени выполнения для таких задач, как сбор метрик, извлечение значений свойств сохраняемого состояния и т. п.

В этом интерфейсе также определен метод `read`, отвечающий за извлечение данных из внешнего источника и за публикацию в целевой теме Pulsar. Реализация этого метода должна обеспечивать его блокировку при отсутствии возвращаемых данных, и он никогда не должен возвращать `null`. Метод `read()` возвращает объект, реализующий интерфейс `org.apache.pulsar.functions.api.Record`, ко-

торый мы видели ранее в листинге 5.2. Следует отметить, что этот интерфейс источника также расширяет интерфейс `AutoCloseable`, включающий определение метода `close`, который должен использоваться для освобождения любых ресурсов, например соединений с базой данных, до остановки работы коннектора.

5.1.3. Коннекторы типа *PushSource*

Для второго типа коннекторов-источников применяется модель на основе продвижения (*push-based model*). Эти коннекторы непрерывно собирают данные и буферизируют их во внутренней блокирующей очереди для итоговой доставки в Pulsar. На рис. 5.4 можно видеть, что коннекторы `PushSource` обычно содержат непрерывно работающий фоновый поток, который собирает информацию из систем-источников и буферизирует собранные данные во внутренней очереди. Фреймворк ввода-вывода Pulsar многократно вызывает метод `read()` коннектора `Source`, и данные из этой внутренней очереди публикуются в Pulsar.

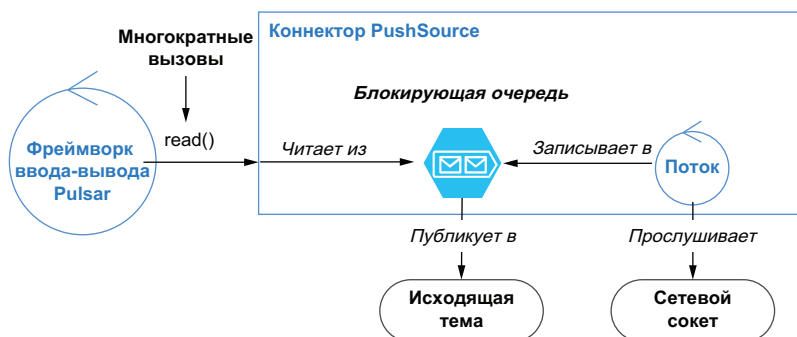


Рис. 5.4. Коннектор `PushSource` содержит постоянно работающий поток, прослушивающий сетевой сокет и продвигающий весь получаемый трафик в блокирующую очередь. При вызове метода `read()` данные извлекаются из блокирующей очереди

В рассматриваемом здесь примере коннектор содержит фоновый поток, который прослушивает весь входящий трафик на сетевом сокете и записывает его во внутреннюю очередь. Этот тип коннектора особенно полезен, если данные потребляются из внешнего источника, который не сохраняет какую-либо информацию и может периодически запрашиваться в любой момент времени в будущем. Сетевой сокет является именно таким примером: если бы поток не был подключен и не прослушивал постоянно сокет, то данные, отправленные через это сетевое соединение, были бы потеряны навсегда. Сравните такой подход с вышеописанным типом коннектора, который отправляет запросы в базу данных. В предыдущем сценарии коннектор может передать запрос в базу данных

после того, как заказ был введен в нее, т. е. сохраняется возможность извлечения данных, потому что база сохранила их.

Простейший способ создания коннектора-источника типа PushSource на основе модели продвижения – написание класса Java, расширяет класс `org.apache.pulsar.io.core.PushSource`, как показано в листинге 5.4. Поскольку этот класс содержит реализацию интерфейса `Source`, создаваемый класс обязательно должен обеспечить реализацию всех методов, которые мы рассмотрели в предыдущем подразделе.

Листинг 5.4. Класс PushSource

```
package org.apache.pulsar.io.core;

/**
 * Интерфейс Pulsar PushSource. PushSource считывает данные из внешних
 * источников (изменения в базах данных, непрерывный поток твитов и т. п.)
 * и публикует их в теме Pulsar. Обоснование названия Push заключается
 * в том, что коннекторы PushSource продвигают данные потребителю,
 * которого они вызывают каждый раз, когда имеются данные
 * для публикации в Pulsar.
 */
public abstract class PushSource<T> implements Source<T> {

    private LinkedBlockingQueue<Record<T>> queue;
    private static final int DEFAULT_QUEUE_LENGTH = 1000;

    public PushSource() {
        this.queue = new LinkedBlockingQueue<>(this.getQueueLength());
    }
    @Override
    public Record<T> read() throws Exception {
        return queue.take();
    }

    /**
     * Присоединение функции потребителя к этому источнику Source.
     * Функция вызывается конкретной реализацией для передачи сообщений
     * каждый раз, когда имеются данные для продвижения в Pulsar.
     *
     * @param record -- очередное сообщение из источника, которое должно быть
     * передано в тему Pulsar.
     */
    public void consume(Record<T> record) {
        try {
            queue.put(record);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }

    /**
     * Получение длины очереди, в которую продвигаются записи.
     */
}
```

```

* Пользователи могут заменить (переписать) этот метод для установки
* собственной длины очереди.
* @возвращает длину очереди.
*/
public int getQueueLength() {
    return DEFAULT_QUEUE_LENGTH;
}

```

4

- ❶ BatchPushSource использует внутреннюю блокирующую очередь для буферизации сообщений.
- ❷ Сообщения считываются из внутренней очереди, которая устанавливает блокировку, если нет доступных данных.
- ❸ Входящие сообщения сохраняются во внутренней очереди, которая при предельном заполнении устанавливает блокировку.
- ❹ Если необходимо увеличить размер внутренней очереди, то вы должны обязательно заменить (переписать) этот метод.

Главной архитектурной характеристикой PushSource является очередь `LinkedBlockingQueue`, используемая для буферизации сообщений перед публикацией в Pulsar. Эта очередь позволяет организовать непрерывно выполняющийся процесс для прослушивания входящих данных и их публикации в Pulsar. Следует также отметить, что требуемый размер внутренней очереди можно ограничить, и это позволяет регулировать потребление памяти коннектором типа PushSource. Когда размер блокирующей очереди достигает установленного предела, в нее больше не записываются данные. В результате фоновый поток блокируется, а это может привести к потерям данных, когда очередь заполнена до отказа. Таким образом, выбор правильного размера очереди чрезвычайно важен.

5.2. Разработка коннекторов ввода-вывода Pulsar

В предыдущем разделе были представлены интерфейсы источника и приемника, предоставляемые фреймворком коннекторов ввода-вывода Pulsar, а также обобщенное описание того, что именно делает каждый метод. В текущем разделе на этой основе будет приведено подробное описание процесса разработки новых коннекторов.

5.2.1. Разработка коннектора-приемника

Начнем с самого простого коннектора-приемника, принимающего непрерывный поток строковых значений и записывающего их в локальный временный файл, как показано в листинге 5.5. Хотя для этого коннектора-приемника имеются некоторые ограничения, в частности невозможность записи бесконечного потока данных в один файл, он служит неплохим примером, который можно использовать для демонстрации процесса разработки коннектора типа Sink.

Листинг 5.5. Коннектор-приемник ввода-вывода Pulsar с записью в локальный файл

```

import org.apache.pulsar.io.core.Sink; ❶

public class LocalFileSink implements Sink<String> {

    private String prefix, suffix;
    private BufferedWriter bw = null;
    private FileWriter fw = null;

    public void open(Map<String, Object> config,
                     SinkContext sinkContext) throws Exception {
        prefix = (String) config.getDefault("filenamePrefix", "test-out");
        suffix = (String) config.getDefault("filenameSuffix", ".tmp"); ❷

        File file = File.createTempFile(prefix, suffix); ❸
        fw = new FileWriter(file.getAbsolutePath(), true); ❹
        bw = new BufferedWriter(fw);
    }

    public void write(Record<String> record) throws Exception {
        try {
            bw.write(record.getValue()); ❺
            bw.flush();
            record.ack(); ❻
        } catch (IOException e) {
            record.fail(); ❼
            throw new RuntimeException(e);
        }
    }

    public void close() throws Exception { ❽
        try {
            if (bw != null)
                bw.close();
            if (fw != null)
                fw.close();
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}

```

- ❶ Импорт интерфейса источника.
- ❷ Извлечение префикса и суффикса имени целевого файла из предоставленных свойств конфигурации.
- ❸ Создание нового файла во временном каталоге.
- ❹ Инициализация файла и буферизованных средств записи.
- ❺ Извлечение значения из входных данных и запись их в открытый файл.
- ❻ Подтверждение успешной обработки сообщения, следовательно, его можно удалить.

- 7 Уведомление о невозможности обработки сообщения, следовательно, его можно сохранить и повторить попытку позже.
- 8 Закрытие обоих потоков открытых файлов для гарантии сброса (записи) данных на диск.

Метод коннектора `open` извлекает свойства конфигурации, предоставленные пользователем, и создает пустой целевой файл во временном каталоге `temp` хоста. Далее переменные уровня экземпляра `FileWriter` и `BufferedWriter` инициализируются для указания на только что созданный целевой файл, тогда как метод `close` будет пытаться закрыть оба эти средства записи при завершении работы коннектора.

Метод `write` приемника вызывается, когда новое сообщение прибывает в какую-либо сконфигурированную входящую тему. Метод добавляет значение записи в целевой файл через метод `write` объекта `BufferedWriter` перед подтверждением успешной обработки сообщения. При наступлении маловероятного события – невозможности записи содержимого сообщения во временный файл генерируется исключение `IOException`, и в приемнике возникает ошибка обработки сообщения до распространения этого исключения.

5.2.2. Разработка коннектора *PushSource*

Теперь напомним специализированный коннектор-источник на основе модели продвижения, который сканирует каталог в поисках новых файлов и публикует их содержимое построчно в Pulsar, как показано в листинге 5.6. Необходимо, чтобы коннектор периодически искал новые файлы и публиковал их содержимое как можно быстрее после записи файла в сканируемый каталог. Это можно сделать, расширив класс `PushSource`, который является специализированной реализацией интерфейса `Source`, предназначенного для использования фонового потока для непрерывного производства записей.

Листинг 5.6. Коннектор *PushSource*

```
import org.apache.pulsar.io.core.PushSource;
import org.apache.pulsar.io.core.SourceContext;

public class DirectorySource extends PushSource<String> {
    private final ScheduledExecutorService scheduler =
        Executors.newScheduledThreadPool(1);

    private DirectoryConsumerThread scanner;

    private Logger log;

    @Override
    public void open(Map<String, Object> config, SourceContext context)
        throws Exception {
        String in = (String) config.getOrDefault("inputDir", ".");
```

1

2

```

String out = (String) config.getDefault("processedDir", ".");
String freq = (String) config.getDefault("frequency", "10");

scanner = new DirectoryConsumerThread(this, in, out, log);
scheduler.scheduleAtFixedRate(scanner, 0, Long.parseLong(freq),
    TimeUnit.MINUTES);
log.info(String.format("Scheduled to run every %s minutes", freq));
}

@Override
public void close() throws Exception {
    log.info("Closing connector");
    scheduler.shutdownNow();
}
}

```

- ❶ Внутренний пул потоков для запуска фонового потока.
- ❷ Получение параметров настройки времени выполнения из переданных свойств конфигурации.
- ❸ Создание фонового потока, передача в него ссылки на коннектор-источник и параметров конфигурации.
- ❹ Запуск фонового потока.

Метод коннектора-источника `open` извлекает свойства конфигурации, предоставленные пользователем и определяющие локальный каталог для чтения из него файлов, а затем запускает фоновый поток типа `DirectoryConsumerThread`, отвечающий за сканирование этого каталога и построчное чтение содержимого каждого файла в нем. Класс фонового потока, показанный в листинге 5.7, в качестве параметра своего метода-конструктора принимает экземпляр коннектора-источника, который в дальнейшем используется для передачи содержимого файлов во внутреннюю блокирующую очередь внутри метода потока `process`.

Листинг 5.7. Класс фонового потока `DirectoryConsumerThread`

```

import org.apache.pulsar.io.core.PushSource;

public class DirectoryConsumerThread extends Thread {
    private final PushSource<String> source;
    private final String baseDir;
    private final String processedDir;

    public DirectoryConsumerThread(PushSource<String> source, String base, String
        ➔ processed, Logger log) {
        this.source = source;
        this.baseDir = base;
        this.processedDir = processed;
        this.log = log;
    }
}

```

```

public void run() {
    log.info("Scanning for files....");
    File[] files = new File(baseDir).listFiles();
    for (int idx = 0; idx < files.length; idx++) {
        consumeFile(files[idx]);
    }
}

private void consumeFile(File file) {
    log.info(String.format("Consuming file %s", file.getName()));
    try (Stream<String> lines = getLines(file)) {
        AtomicInteger counter = new AtomicInteger(0);
        lines.forEach(line ->
            process(line, file.getPath(), counter.incrementAndGet()));

        log.info(String.format("Processed %d lines from %s",
            counter.get(), file.getName()));
        Files.move(file.toPath(), Paths.get(processedDir)
            .resolve(file.toPath().getFileName()), REPLACE_EXISTING);
        log.info(String.format("Moved file %s to %s",
            file.toPath().toString(), processedDir));
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private Stream<String> getLines(File file) throws IOException {
    if (file == null) {
        return null;
    } else {
        return Files.lines(Paths.get(file.getAbsolutePath()),
            Charset.defaultCharset());
    }
}

private void process(String line, String src, int lineNum) {
    source.consume(new FileRecord(line, src, lineNum));
}
}

```

- ❶ Ссылка на коннектор PushSource.
- ❷ Обработка всех файлов в сконфигурированном каталоге base.
- ❸ Разделение каждого файла на отдельные строки.
- ❹ Обработка каждой строки файла по отдельности.
- ❺ После завершения обработки файл перемещается в каталог processed.
- ❻ Разделение заданного файла на поток отдельных строк.
- ❼ Создание новой записи для каждой текстовой строки в файле.

Я также создал новый тип записи FileRecord, показанный в листинге 5.8, для коннектора-источника PushSource. Это позволяет сохранять некоторые дополнительные метаданные о содержимом

файла, включая имя исходного файла и количество строк в нем. Этот тип метаданных можно использовать для последующей обработки записей другими функциями Pulsar, так как появляется возможность сортировать записи по имени файла или типу или обеспечить обработку строк заданного файла в надлежащем порядке (на основе номера строки).

Листинг 5.8. Класс FileRecord

```
import org.apache.pulsar.functions.api.Record;

public class FileRecord implements Record<String> {

    private static final String SOURCE = "Source";
    private static final String LINE = "Line-no";
    private String content;
    private Map<String, String> props;

    public FileRecord(String content, String src, int lineNumber) {
        this.content = content;
        this.props = new HashMap<String, String>();
        this.props.put(SOURCE, srcFileName);
        this.props.put(LINE, lineNumber + "");
    }

    @Override
    public Optional<String> getKey() {
        return Optional.ofNullable(props.get(SOURCE));
    }

    @Override
    public Map<String, String> getProperties() {
        return props;
    }

    @Override
    public String getValue() {
        return content;
    }
}
```

- 1 Действительное содержимое файла для этой конкретной строки.
- 2 Свойства сообщения.
- 3 Использование исходного файла как ключа для подписок на основе ключа и т. п.
- 4 Свойства сообщения предъявляют метаданные.
- 5 Значение сообщения – это само необработанное содержимое файла.

Метод потока `process` вызывает метод `consume()` коннектора `PushSource`, передавая в него содержимое файла. Как можно видеть в листинге 5.8, метод `consume()` в коннекторе `PushSource` просто за-

писывает входящие данные прямо во внутреннюю блокирующую очередь. Это отделяет операцию чтения содержимого файла от вызова фреймворком Pulsar метода `read()` коннектора `PushSource`. Использование фонового потока – это общепринятый паттерн проектирования для коннекторов типа `PushSource`, которые извлекают данные из внешней системы, а затем вызывают в источнике метод потребления для продвижения данных в исходящую тему.

5.3. Тестирование коннекторов ввода-вывода Pulsar

В этом разделе мы подробно рассмотрим процесс разработки и тестирования коннектора Pulsar. Здесь будет использоваться коннектор `DirectorySource`, который был показан в листинге 5.6, для демонстрации жизненного цикла разработки коннектора Pulsar. Этот коннектор принимает предоставленный пользователем каталог и построчно публикует содержимое всех файлов, находящихся в этом каталоге.

Хотя код существенно упрощен, он позволяет подробно рассмотреть процесс тестирования, который обычно используется при разработке коннектора для промышленного применения. Поскольку это чистый код Java, можно воспользоваться любым существующим фреймворком модульного тестирования, например JUnit или TestNG, для проверки логики функции.

5.3.1. Модульное тестирование

Первый этап – написание комплекта модульных тестов, проверяющих некоторые из наиболее общих сценариев, чтобы убедиться в том, что логика корректна и дает точные результаты для разнообразных текстовых фрагментов, передаваемых ей. Поскольку этот код использует Pulsar SDK API, потребуется дополнительная библиотека, например Mockito, для имитации объекта `SourceContext`, как показано в листинге 5.9.

Листинг 5.9. Модульные тесты коннектора `DirectorySource`

```
public class DirectorySourceTest {  
    final static Path SOURCE_DIR =  
        Paths.get(System.getProperty("java.io.tmpdir"), "source");  
    final static Path PROCESSED_DIR =  
        Paths.get(System.getProperty("java.io.tmpdir"), "processed");  
  
    private Path srcPath, processedPath;  
    private DirectorySource spySource;  
  
    @Mock  
    private SourceContext mockedContext;
```

1

2

```

@Mock
private Logger mockedLogger;

@Captor
private ArgumentCaptor<FileRecord> captor; 3

@Before
public final void init() throws IOException {
    MockitoAnnotations.initMocks(this);
    when(mockedContext.getLogger()).thenReturn(mockedLogger);
    FileUtils.deleteDirectory(SOURCE_DIR.toFile());
    FileUtils.deleteDirectory(PROCESSED_DIR.toFile()); 4
    srcPath = Files.createDirectory(SOURCE_DIR,
        PosixFilePermissions.asFileAttribute(
            PosixFilePermissions.fromString("rwxrwxrwx")));
    processedPath = Files.createDirectory(PROCESSED_DIR,
        PosixFilePermissions.asFileAttribute(
            PosixFilePermissions.fromString("rwxrwxrwx"))); 5
    spySource = spy(new DirectorySource()); 6
}

@Test
public final void onelineTest() throws Exception {
    Files.copy(getFile("single-line.txt"), Paths.get(srcPath.toString(),
        "single-line.txt"), COPY_ATTRIBUTES); 7
    Map<String, Object> configs = new HashMap<String, Object>();
    configs.put("inputDir", srcPath.toFile().getAbsolutePath());
    configs.put("processedDir", processedPath.toFile().getAbsolutePath());

    spySource.open(configs, mockedContext); 8
    Thread.sleep(3000);

    Mockito.verify(spySource).consume(captor.capture()); 9
    FileRecord captured = captor.getValue(); 10
    assertNotNull(captured);
    assertEquals("It was the best of times", captured.getValue()); 11
    assertEquals("1", captured.getProperties().get(FileRecord.LINE));
    assertTrue(captured.getProperties().get(FileRecord.SOURCE)
        .contains("single-line.txt")); 12
}

@Test
public final void multilineTest() throws Exception {
    Files.copy(getFile("example-1.txt"), Paths.get(srcPath.toString(),
        "example-1.txt"), COPY_ATTRIBUTES);
    Map<String, Object> configs = new HashMap<String, Object>();
    configs.put("inputDir", srcPath.toFile().getAbsolutePath());
    configs.put("processedDir", processedPath.toFile().getAbsolutePath());

    spySource.open(configs, mockedContext); 13
    Thread.sleep(3000);
    Mockito.verify(spySource, times(113)).consume(captor.capture()); 14
}

```

```

final AtomicInteger counter = new AtomicInteger(0);
captor.getAllValues().forEach(rec -> {
    assertNotNull(rec.getValue());
    assertEquals(counter.incrementAndGet() + "",
        rec.getProperties().get(FileRecord.LINE));
    assertTrue(rec.getProperties().get(FileRecord.SOURCE)
        .contains("example-1.txt"));
});
}

private static Path getFile(String fileName) throws IOException {
    ...
}
}

```

15

- ❶ Использование временного каталога для тестирования.
- ❷ Мы будем наблюдать за коннектором DirectorySource.
- ❸ Класс, который захватывает все элементы данных, записываемые коннектором DirectorySource.
- ❹ Очистка временного каталога перед началом каждого теста.
- ❺ Создание источника и обрабатываемого каталога, используемых во время теста.
- ❻ Создание экземпляра коннектора DirectorySource.
- ❼ Копирование тестового файла в каталог-источник.
- ❽ Запуск коннектора DirectorySource.
- ❾ Проверка публикации отдельной записи.
- ❿ Извлечение опубликованной записи для проверки.
- ⓫ Проверка содержимого извлеченной записи.
- ⓬ Проверка свойств извлеченной записи.
- ⓭ Запуск коннектора DirectorySource.
- ⓮ Проверка: было ли опубликовано ожидаемое количество записей.
- ⓯ Проверка значений и свойств каждой опубликованной записи.

Как можно видеть, эти модульные тесты покрывают всю основную функциональность коннектора и основаны на использовании имитации объекта контекста Pulsar. Этот тип тестового комплекта очень похож на тот, который вы должны написать для проверки любого класса Java, не являющегося функцией Pulsar.

5.3.2. Комплексное тестирование

После получения положительных результатов модульного тестирования необходимо проверить, как коннектор будет работать в кластере Pulsar. Самый простой способ тестирования коннектора – запуск сервера Pulsar и выполнение функции (коннектора) локально с использованием вспомогательного класса LocalRunner. В этом режиме коннектор работает как отдельный независимый процесс на компьютере, где он был активизирован. Этот вариант является самым лучшим для разработки и тестирования собствен-

ных коннекторов, так как он позволяет подключить отладчик к процессу коннектора на локальном компьютере. Чтобы использовать `LocalRunner`, сначала необходимо добавить в проект Maven несколько зависимостей, позволяющих включить класс `LocalRunner` в тестирование коннектора в работающем кластере Pulsar, как показано в листинге 5.10.

Листинг 5.10. Включение зависимостей для класса `LocalRunner`

```
<dependencies>
  . . .
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-core</artifactId>
    <version>2.11.1</version>
  </dependency>
  <dependency>
    <groupId>org.apache.pulsar</groupId>
    <artifactId>pulsar-functions-local-runner-original</artifactId>
    <version>2.6.1</version>
  </dependency>
</dependencies>
```

Далее необходимо написать класс для конфигурирования и запуска `LocalRunner`, как показано в листинг 5.11. Здесь можно видеть, что этот код сначала непременно должен сконфигурировать коннектор Pulsar для выполнения в `LocalRunner` и определить адрес реального экземпляра кластера Pulsar, который будет использоваться для тестирования. Простейшим способом получения доступа к кластеру является запуск контейнера Pulsar Docker с помощью команды в окне командной оболочки `bash`: `docker run -d -p 6650:6650 -p 8080:8080 --name pulsar apachepulsar/pulsar-standalone`. Эта команда запускает кластер Pulsar в автономном режиме внутри контейнера. Обычно тест с использованием `LocalRunner` выполняется из интегрированной среды разработки (IDE), чтобы подключить отладчик и выполнить пошаговый проход по коду функции для выявления и устранения всех возможных ошибок.

Листинг 5.11. Тестирование коннектора `DirectorySource` с использованием `LocalRunner`

```
public class DirectorySourceLocalRunnerTest {
    final static String BROKER_URL = "pulsar://localhost:6650";
    final static String OUT = "persistent://public/default/directory-scan";
    final static Path SOURCE_DIR =
        Paths.get(System.getProperty("java.io.tmpdir"), "source");
    final static Path PROCESSED_DIR =
        Paths.get(System.getProperty("java.io.tmpdir"), "processed");
```

```

private static LocalRunner localRunner;
private static Path srcPath, processedPath;

public static void main(String[] args) throws Exception {
    init();
    startLocalRunner();
    shutdown();
}

private static void startLocalRunner() throws Exception {
    localRunner = LocalRunner.builder()
        .brokerServiceUrl(BROKER_URL)
        .sourceConfig(getSourceConfig())
        .build();
    localRunner.start(false);
}

private static void init() throws IOException {
    Files.deleteIfExists(SOURCE_DIR);
    Files.deleteIfExists(PROCESSED_DIR);
    srcPath = Files.createDirectory(SOURCE_DIR,
        PosixFilePermissions.asFileAttribute(
            PosixFilePermissions.fromString("rwxrwxrwx")));
    processedPath = Files.createDirectory(PROCESSED_DIR,
        PosixFilePermissions.asFileAttribute(
            PosixFilePermissions.fromString("rwxrwxrwx")));

    Files.copy(getFile("example-1.txt"), Paths.get(srcPath.toString(),
        "example-1.txt"), COPY_ATTRIBUTES);

}

private static void shutdown() throws Exception {
    Thread.sleep(30000);
    localRunner.stop();
    System.exit(0);
}

private static SourceConfig getSourceConfig() {
    Map<String, Object> configs = new HashMap<String, Object>();
    configs.put("inputDir", srcPath.toFile().getAbsolutePath());
    configs.put("processedDir", processedPath.toFile().getAbsolutePath());

    return SourceConfig.builder()
        .className(DirectorySource.class.getName())
        .configs(configs)
        .name("directory-source")
        .tenant("public")
        .namespace("default")
        .topicName(OUT)
        .build();
}

```

```
private static Path getFile(String fileName) throws IOException {
    ...
}
}
```

10

- ❶ Использование временного каталога для тестирования.
- ❷ Установление соединения LocalRunner с контейнером Docker.
- ❸ Развертывание коннектора DirectorySource.
- ❹ Создание источника и обрабатываемого каталога, используемых во время тестирования.
- ❺ Копирование тестового файла в каталог-источник.
- ❻ Остановка LocalRunner после 30 с работы.
- ❼ Определение DirectorySource как коннектора, который необходимо запустить.
- ❽ Конфигурирование коннектора DirectorySource.
- ❾ Определение исходящей темы для коннектора-источника.
- ❿ Чтение файла из каталога ресурсов проекта.

5.3.3. Упаковка коннекторов ввода-вывода Pulsar

Поскольку коннекторы ввода-вывода Pulsar являются специализированными функциями Pulsar, вполне ожидаемо, что они должны быть представлены в форме самодостаточных программных пакетов. Следовательно, потребуется упаковка созданного коннектора со всеми его зависимостями как fat-JAR или NAR-файла. NAR – это сокращенное обозначение архивного файла NiFi archive. Это специализированный механизм упаковки, используемый Apache NiFi и обеспечивающий изоляцию Java ClassLoader. Чтобы получить коннектор ввода-вывода Pulsar, упакованный как NAR-файл, требуется всего лишь включить `nifi-nar-maven-plugin` в проект Maven для конкретного коннектора, как показано в листинге 5.12.

Листинг 5.12. Создание пакета NAR

```
<build>
...
<plugin>
  <groupId>org.apache.nifi</groupId>
  <artifactId>nifi-nar-maven-plugin</artifactId>
  <version>1.2.0</version>
  <extensions>true</extensions>
  <executions>
    <execution>
      <phase>package</phase>
      <goals>
        <goal>nar</goal>
      </goals>
    </execution>
  </executions>
</plugin>
</build>
```

Подключаемый модуль сборки в листинге 5.12 используется для генерации NAR-файла, который по умолчанию включает все зависимости проекта в генерируемый архивный файл. Это наиболее предпочтительный способ сборки и развертывания коннекторов ввода-вывода Pulsar на языке Java. С таким подключаемым модулем, добавленным в файл *pom.xml*, необходимо всего лишь выполнить команду `mvn clean install` для создания NAR-файла, который можно использовать для развертывания коннектора в производственном кластере Pulsar. После упаковки коннектора со всеми его зависимостями в NAR-файл следующим этапом является развертывание коннектора в кластере Pulsar.

5.4. Развертывание коннекторов ввода-вывода Pulsar

Как специализированные функции Pulsar, коннекторы ввода-вывода используют ту же самую среду времени выполнения, которая обеспечивает все преимущества фреймворка Pulsar Functions, включая отказоустойчивость, параллельный режим, гибкость, балансировку нагрузки, обновление по запросу и многие другие. С учетом вариантов развертывания можно получить коннектор ввода-вывода Pulsar, работающий на локальном компьютере разработки в режиме `localrun`, внутри рабочих процессов в кластере Pulsar или в режиме кластера. В предыдущем разделе мы использовали `LocalRunner` для работы коннектора в режиме `localrun`. В этом разделе будет подробно рассмотрен процесс выполнения ранее разработанного коннектора `DirectorySource` в режиме кластера.

На рис. 5.5 показано развертывание в режиме кластера в среде Kubernetes, где каждый коннектор работает в собственном контейнере вместе с другими экземплярами функций, не являющихся коннекторами. В режиме кластера коннекторы ввода-вывода Pulsar пользуются свойством отказоустойчивости, предоставленным планировщиком времени выполнения Pulsar Functions для обработки ошибок. Если коннектор работает на компьютере, где возникает критический сбой, то Pulsar автоматически пытается перезапустить задачу на одном из оставшихся в рабочем состоянии узлов в кластере.

5.4.1. Создание и удаление коннекторов

Если вы пока еще не сделали это, выполните команду `mvn clean install`, чтобы создать NAR-файл для коннектора `DirectorySource`. Также необходимо остановить все работающие контейнеры Docker Pulsar, поскольку потребуется запуск нового экземпляра с некоторыми дополнительными параметрами, позволяющими получить доступ к NAR-файлу из контейнера Pulsar Docker, как показано в листинге 5.13.

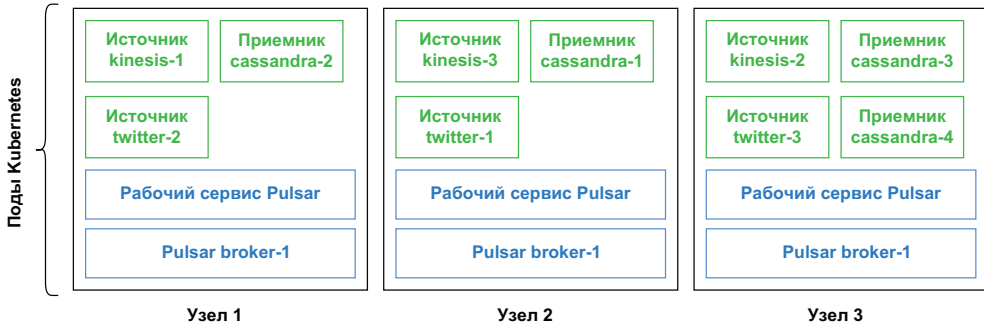


Рис. 5.5. Развертывание коннектора ввода-вывода Pulsar в контейнере Kubernetes

Листинг 5.13. Запуск контейнера Pulsar Docker с монтируемыми дисковыми ресурсами (томами)

```
$ export GIT_PROJECT=<CLONE_DIR>/pulsar-in-action/chapter5 ❶
$ docker run --name pulsar -id \
  -p 6650:6650 -p 8080:8080 \
  -v $GIT_PROJECT:/pulsar/dropbox
  apache/pulsar/pulsar-standalone ❷
```

- ❶ Здесь необходимо указать каталог, в котором вы клонировали репозиторий, связанный с этой книгой.
- ❷ Этот ключ делает доступным каталог проекта внутри контейнера Docker.

В листинге 5.13 можно видеть, что мы добавили ключ `-v` в обычную команду запуска контейнера Pulsar Docker. Этот ключ монтирует локальный каталог, в котором клонирован исходный код для текущей главы на вашем компьютере, в каталог с именем `/pulsar/dropbox` внутри контейнера Docker. Это необходимо для развертывания коннектора, поскольку NAR-файл должен быть доступным физически из кластера Pulsar, чтобы появилась возможность его развернуть. Кроме того, смонтированный каталог будет использоваться для доступа к файлу конфигурации, который обязательно должен быть предоставлен при создании коннектора.

Как уже было отмечено выше, мы всегда предоставляем жестко закодированные значения для свойств конфигурации в модульных и комплексных тестах, но при развертывании в производственной среде необходимо задавать эти значения более динамическим способом. Именно для этого предназначен файл конфигурации, так как он позволяет определять специализированные для конкретного коннектора конфигурации вместе с другими стандартными свойствами коннектора, такими как параллельный режим работы.

Листинг 5.14. Содержимое файла конфигурации для коннектора DirectorySource

```
tenant: public
namespace: default
name: directory-source

className: com.manning.pulsar.chapter5.source.DirectorySource
topicName: "persistent://public/default/directory-scan"
parallelism: 1
processingGuarantees: ATLEAST_ONCE

# Специализированные для этого коннектора параметры конфигурации.
configs:
  inputDir: "/tmp/input"
  processedDir: "/tmp/processed"
  frequency: 10
```

В листинге 5.14 показан файл конфигурации, который будет применяться для развертывания коннектора. Он содержит значения для каталогов источника и обработки, а также несколько свойств, используемых фреймворком Pulsar IO для создания коннектора. Далее файл конфигурации передается в команду `bin/pulsar-admin source create`, когда потребуется создать новый коннектор-источник, как показано в листинге 5.15.

Листинг 5.15. Выходные данные команды create

```
docker exec -it pulsar mkdir -p /tmp/input
docker exec -it pulsar chmod a+w /tmp/input
docker exec -it pulsar mkdir -p /tmp/processed
docker exec -it pulsar chmod a+w /tmp/processed
docker exec -it pulsar cp /pulsar/dropbox/src/test/resources/example-1.txt
➔ /tmp/input ❶

docker exec -it pulsar /pulsar/bin/pulsar-admin source create \
--archive /pulsar/dropbox/target/chapter5-0.0.1.nar \
--source-config-file /pulsar/dropbox/src/main/resources/config.yml
"Created successfully"

docker exec -it pulsar /pulsar/bin/pulsar-admin source list
[
  "directory-source"
]
```

- ❶ Создание входящего и исходящего каталогов внутри контейнера и копирование тестового файла.
- ❷ Формирование команды для создания коннектора с указанием источника.
- ❸ Определяется NAR-файл, содержащий класс коннектора-источника.

- ④ Определяется файл конфигурации для использования.
- ⑤ Это сообщение о том, что источник был создан.
- ⑥ Вывод списка активных коннекторов-источников, подтверждающий, что коннектор действительно был создан.

При создании коннектора вы должны получить сообщение "Created successfully", подтверждающее, что коннектор был успешно создан и запущен. Если такое сообщение не появилось, то потребуется отладка с целью выявления ошибок. Это тема следующего подраздела.

5.4.2. Отладка развернутых коннекторов

Если при развертывании Pulsar IO коннектора возникли какие-либо ошибки или неожиданное поведение, то самым лучшим местом для начала отладки являются файлы журналов на рабочем узле Pulsar, на котором развертывается коннектор. По умолчанию вся информация о запуске коннектора и перехватываемый стандартный поток ошибок `stderr` записывается в файл журнала. Основой имени файла журнала является имя коннектора, и оно соответствует следующему шаблону в производственной среде: `/logs/functions/tenant/namespace/function-name/function-name-instance-id.log`. В автономном режиме, который мы сейчас используем, базовым каталогом становится `/tmp` вместо `/logs`, а остальной путь остается неизменным. Проверим файл журнала для коннектора `DirectorySource`, созданного в предыдущем подразделе. Результат показан в листинге 5.16, где можно увидеть некоторую информацию, полезную для отладки.

Листинг 5.16. Первый раздел файла журнала для коннектора `DirectorySource`

```
cat /tmp/functions/public/default/directory-source/
➔ directory-source-0.log
20:53:30.671 [main] INFO
➔ org.apache.pulsar.functions.runtime.JavaInstanceStarter - JavaInstance
➔ Server started, listening on 36857
20:53:30.676 [main] INFO
➔ org.apache.pulsar.functions.runtime.JavaInstanceStarter -
➔ Starting runtimeSpawner
20:53:30.678 [main] INFO
➔ org.apache.pulsar.functions.runtime.RuntimeSpawner -
➔ public/default/directory-source-0 RuntimeSpawner starting function
20:53:30.689 [main] INFO
➔ org.apache.pulsar.functions.runtime.thread.ThreadRuntime -
➔ ThreadContainer starting function with instance config
➔ InstanceConfig(instanceId=0, functionId=c368b93f-34e9-4bcf-801f-
➔ d097b1c0d173, functionVersion=247cbde2
-b8b4-45bb-a3cb-8926c3b33217, functionDetails=tenant: "public"
namespace: "default"
name: "directory-source"
className: "org.apache.pulsar.functions.api.utils.IdentityFunction"
```

```

autoAck: true
parallelism: 1
source {
  className: "com.manning.pulsar.chapter5.source.DirectorySource"
  configs:
    ➔ "{ \"processedDir\": \" /tmp/processed\", \"inputDir\": \" /tmp/input\",
    ➔ \"frequency\": \"2\" }"
  typeClassName: "java.lang.String"
}
sink {
  topic: "persistent://public/default/directory-scan"
  typeClassName: "java.lang.String"
}
resources {
  cpu: 1.0
  ram: 1073741824
  disk: 10737418240
}
componentType: SOURCE
, maxBufferedTuples=1024, functionAuthenticationSpec=null, port=36857,
➔ clusterName=standalone, maxPendingAsyncRequests=1000)
...
20:53:31.223 [public/default/directory-source-0] INFO
➔ org.apache.pulsar.functions.instance.JavaInstanceRunnable - Initialize
➔ function class loader for function directory-source at function cache
➔ manager, functionClassLoader:
➔ org.apache.pulsar.common.nar.NarClassLoader[/tmp/pulsar-nar/chapter5-
➔ 0.0.1.nar-unpacked]

```

- ❶ Исследование файла журнала для коннектора.
- ❷ Раздел подробностей о конфигурации коннектора.
- ❸ Имя класса коннектора.
- ❹ Пары ключ–значение параметров конфигурации.
- ❺ Исходящая тема.
- ❻ NAR-файл и его версия, используемая для развертывания коннектора.

Первый раздел файла журнала содержит основную информацию о коннекторе, такую как абонент, пространство имен, имя, режим параллельного выполнения, ресурсы и т. д., которую можно использовать, чтобы узнать, правильно ли был сконфигурирован коннектор. Немного ниже в файле журнала вы должны увидеть строку, сообщающую о том, из какого файла артефакта был создан коннектор. Это позволит вам убедиться в том, что был использован корректный файл артефакта.

Листинг 5.17. Последний раздел файла журнала для коннектора DirectorySource

```

org.apache.pulsar.client.impl.ProducerStatsRecorderImpl - Starting
Pulsar producer perf with config: {

```



```

"topicName" : "persistent://public/default/directory-scan",
"producerName" : null,
"sendTimeoutMs" : 0,
...
"multiSchema" : true,
"properties" : {
  "application" : "pulsar-source",
  "id" : "public/default/directory-source",
  "instance_id" : "0"
}
}
20:53:33.704 [public/default/directory-source-0] INFO
➔ org.apache.pulsar.client.impl.ProducerStatsRecorderImpl
➔ - Pulsar client config: {
  "serviceUrl" : "pulsar://localhost:6650",
  "authPluginClassName" : null,
  "authParams" : null,
  ...
  "proxyProtocol" : null
}
20:53:33.726 [public/default/directory-source-0] INFO
➔ org.apache.pulsar.client.impl.ProducerImpl - [persistent://public/
default/directory-scan] [null] Creating producer on cnx [id: 0xbcd9978b,
➔ L:/127.0.0.1:44010 - R:localhost/127.0.0.1:6650]
20:53:33.886 [pulsar-client-io-1-1] INFO
➔ org.apache.pulsar.client.impl.ProducerImpl -
➔ [persistent://public/default/directory-scan] [standalone-0-0]
Created producer on cnx [id: 0xbcd9978b,
➔ L:/127.0.0.1:44010 - R:localhost/127.0.0.1:6650]
20:53:33.983 [public/default/directory-source-0] INFO function-directory-
➔ source - Scheduled to run every 2 minutes
20:53:33.985 [pool-6-thread-1] INFO function-directory-source - Scanning
➔ for files...
20:53:33.987 [pool-6-thread-1] INFO function-directory-source - Processing
➔ file example-1.txt
20:53:33.987 [pool-6-thread-1] INFO function-directory-source - Consuming
➔ file example-1.txt
20:53:34.385 [pool-6-thread-1] INFO function-directory-source - Processed
➔ 113 lines from example-1.txt
20:53:34.385 [pool-6-thread-1] INFO function-directory-source - Moved file
➔ /tmp/input/example-2.txt to /tmp/processed

```

- 1 Производитель Pulsar для этого коннектора-источника.
- 2 Дополнительные свойства коннектора-источника.
- 3 Конфигурация клиента Pulsar, включающая параметры обеспечения безопасности.
- 4 Дополнительные свойства конфигурации клиента Pulsar.
- 5 Журнальные сообщения от коннектора DirectorySource.

Следующий раздел файла журнала, показанный в листинге 5.18, содержит некоторую информацию о производителях и потребителях Pulsar, созданных от имени коннектора и используемых для

публикации и потребления данных в сконфигурированных входящей и исходящей темах. Любая проблема в соединении с любым из этих объектов неизбежно приведет к ошибкам именно здесь. Все сообщения для журнала, добавленные в ваш код, следуют за этим разделом и позволяют проследить текущую работу коннектора или обнаружить любые сгенерированные исключения.

После завершения использования коннектора если он больше не нужен, то можно воспользоваться командой `bin/pulsar-admin source delete` для остановки всех работающих экземпляров этого коннектора. При этом необходимо предоставить только параметры абонента, пространства имен и имени коннектора для его однозначной идентификации при удалении (например, чтобы удалить коннектор-источник из нашего примера, нужно выполнить следующую команду: `bin/pulsar-admin source delete --tenant public --namespace default --name directory-source`).

5.5. Встроенные коннекторы Pulsar

Pulsar предоставляет большой набор разнообразных существующих источников и приемников, обозначенных как встроенные коннекторы, которые позволяют начать использование фреймворка Pulsar IO connectors без необходимости писать какой-либо код. Все встроенные коннекторы Pulsar выпускаются в виде отдельных архивов. Для практического применения требуется просто скопировать в кластер Pulsar архивный NAR-файл необходимого встроенного коннектора, а также простой YAML- или JSON-файл конфигурации, определяющий параметры времени выполнения для соединения с внешней системой. Если Pulsar работает в автономном режиме, например при использовании независимого образа Pulsar в Docker, то все отдельные архивы встроенных коннекторов уже включены как часть дистрибутива образа.

Рассмотрим простой сценарий, в котором применяются встроенные коннекторы для перемещения данных из Pulsar в MongoDB. Хотя этот пример немного упрощен, тем не менее он показывает, насколько простым является использование фреймворка коннекторов, а также помогает продемонстрировать некоторые шаги высокого уровня, требуемые для развертывания и эксплуатации коннекторов Pulsar IO. Первым шагом в этом процессе будет создание экземпляра MongoDB, с которым мы можем взаимодействовать.

5.5.1. Запуск кластера MongoDB

Приведенная ниже команда запускает самую последнюю версию контейнера MongoDB в режиме без соединений. В ней также выполняется отображение портов контейнера в порты хоста, поэтому можно получить доступ к базе данных с локального компьютера,

если потребуется. После запуска этого контейнера мы получаем полнофункциональную развернутую базу данных MongoDB и можем работать с ней:

```
$ docker run -d \  
  -p 27017-27019:27017-27019 \  
  --name mongodb \  
  mongo
```

В текущий момент мы имеем контейнер Docker MongoDB, работающий в режиме без подключений. Далее потребуется выполнение команды `mongo` для запуска командной оболочки клиента MongoDB. В командной оболочке необходимо создать новую базу данных и коллекцию для хранения данных. Далее нужно создать новую базу данных с именем `pulsar_in_action` и определить внутри нее коллекцию, которая будет использоваться для хранения данных. Эти действия выполняются командами, показанными в листинге 5.18.

Листинг 5.18. Создание таблицы базы данных Mongo

```
docker exec -it mongodb mongo ❶  
MongoDB shell version v4.4.1 ❷  
...  
>  
  
>use pulsar_in_action; ❸  
switched to db pulsar_in_action  
  
> db.example.save({ firstname: "John", lastname: "Smith"}) ❹  
WriteResult({ "nInserted" : 1 })  
  
> db.example.find({firstname: "John"}) ❺  
{ "_id" : ObjectId("5f7a53aedccb229a78960d2c"), "firstname" : "John",  
  ➡ "lastname" : "Smith" }
```

- ❶ Запуск интерактивной командной оболочки MongoDB.
- ❷ В выводимой информации вы должны увидеть номер версии командной оболочки MongoDB.
- ❸ Создание базы данных с именем `pulsar_in_action`.
- ❹ Создание коллекции с именем `example` внутри базы данных и определение ее схемы.
- ❺ Запрос в базу данных для подтверждения успешного добавления новой записи.

Теперь у нас есть кластер MongoDB, работающий локально, и созданная база данных, поэтому можно приступить к конфигурированию коннектора-приемника MongoDB. Он будет считывать сообщения из темы Pulsar и записывать их в созданную таблицу MongoDB.

5.5.2. Связывание контейнеров Pulsar и MongoDB

Поскольку мы намерены запустить коннектор Pulsar для MongoDB внутри контейнера Pulsar Docker, должно существовать сетевое соединение между этими двумя контейнерами. Самый простой способ решения этой задачи в среде Docker – использование аргумента командной строки `--link` при запуске контейнера Pulsar. Но мы уже запустили контейнер Pulsar, поэтому сначала нужно остановить его и удалить перед повторным запуском с ключом `--link`. Таким образом, потребуется выполнить все команды, показанные в листинге 5.19, прежде чем продолжить.

Листинг 5.19. Команды для установления соединения между Pulsar и MongoDB

```
$ docker stop pulsar 1

$ docker rm pulsar 2

$ docker run -d \
  -p 6650:6650 -p 8080:8080 \
  -v $PWD/data:/pulsar/data \
  --name pulsar \
  --link mongodb \
  apache/pulsar/pulsar-standalone 3

$ docker exec -it pulsar bash 4

apt-get update && apt-get install vim --fix-missing -y 5
vim /pulsar/examples/mongodb-sink.yml 6
```

- ❶ Останов работающего контейнера Pulsar.
- ❷ Удаление старого контейнера Pulsar, чтобы можно было создать новый с тем же именем.
- ❸ Связывание контейнера MongoDB с контейнером Pulsar.
- ❹ Выполнение команды в новом контейнере Pulsar.
- ❺ В контейнере Pulsar необходимо установить текстовый редактор `vim`, чтобы редактировать файл конфигурации.
- ❻ Запуск текстового редактора в контейнере Pulsar для создания файла конфигурации.

Получив имя контейнера, в котором работает экземпляр MongoDB, в ключе `--link`, Docker создает защищенный сетевой канал между двумя контейнерами, позволяющий контейнеру Pulsar вести диалог с контейнером MongoDB через указанное соединение. Мы увидим это, когда сконфигурируем коннектор-приемник для MongoDB.

5.5.3. Конфигурирование и создание приемника для MongoDB

Конфигурирование Pulsar IO коннекторов выполняется просто. Необходимо всего лишь предоставить файл конфигурации в формате YAML при создании коннектора. Для запуска коннектора-приемника MongoDB нужно подготовить YAML-файл конфигурации, содержащий всю информацию, которая потребуется механизму времени выполнения Pulsar IO для установления соединения с локальным экземпляром MongoDB. Сначала необходимо создать в подкаталоге примеров локальный файл с именем *mongodb-sink.yml* и отредактировать его, наполнив содержимым, показанным в листинге 5.20.

Листинг 5.20. Файл конфигурации коннектора-приемника для MongoDB

```
tenant: "public"
namespace: "default"
name: "mongo-test-sink"
configs:
  mongoUri: "mongodb://mongodb:27017/admin"
  database: "pulsar_in_action"
  collection: "example"
  batchSize: 1
  batchTimeMs: 1000
```

- ❶ Здесь можно использовать имя, заданное в ключе `--link`, вместо имени хоста или IP-адреса.
- ❷ Необходимо обязательно определить имя базы данных Mongo, в которую будет выполняться запись.
- ❸ Необходимо обязательно определить имя коллекции Mongo, в которую будет выполняться запись.

Для получения более подробной информации о конфигурации коннектора-приемника для MongoDB обратитесь к документации (<https://pulsar.apache.org/docs/3.1.x/io-mongo-sink/>). Интерфейс командной строки Pulsar предоставляет команды для запуска и управления Pulsar IO коннекторами, поэтому можно выполнить команду, показанную в листинге 5.21, прямо из командной строки контейнера Pulsar, чтобы запустить коннектор-приемник для MongoDB.

Листинг 5.21. Запуск коннектора-приемника для MongoDB

```
/pulsar/bin/pulsar-admin sink create \
  --sink-type mongo \
  --sink-config-file /pulsar/examples/mongodb-sink.yml \
  --inputs test-mongo
"Created successfully"
```

- ❶ Использование команды создания приемника.
- ❷ Указание о необходимости использования встроенного коннектора-приемника для MongoDB.

- ③ Использование ранее созданного файла конфигурации.
- ④ Определение входящей темы.
- ⑤ Сообщение об успешном создании приемника.

При выполнении этой команды Pulsar создает коннектор-приемник с именем `mongo-test-sink`, который начинает записывать сообщения из темы `test-mongo` в коллекцию `examples` в базе данных `Mongo pulsar_in_action`. Теперь отправим несколько сообщений в тему `test-mongo`, чтобы убедиться в том, что коннектор работает как полагается. Для этого выполним команду, показанную в листинге 5.22, из контейнера Pulsar.

Листинг 5.22. Отправка сообщений во входящую тему коннектора-приемника

```
/pulsar/bin/pulsar-client produce \
-m "{firstname: \"Mary\", lastname: \"Smith\"}" \
-s % \
-n 10 \
test-mongo
```

①
②
③
④
⑤

- ① Команда, производящая сообщения.
- ② Содержимое сообщения, включающее экранируемые кавычки.
- ③ Определение символа-разделителя записей, не являющегося запятой. Иначе будет разделяться содержимое каждого сообщения.
- ④ Определение: это сообщение нужно отправить 10 раз.
- ⑤ Целевая тема.

Теперь можно выполнить запрос в экземпляр MongoDB для проверки корректности работы коннектора. Для этого возвращаемся в командную оболочку MongoDB, открытую ранее для создания базы данных и выполним несколько различных запросов для подтверждения того, что записи действительно были добавлены в таблицу MongoDB, как и ожидалось (см. листинг 5.23).

Листинг 5.23. Запросы в таблицу MongoDB после того, как сообщения были переданы

```
> db.example.find({lastname: "Smith"})
{ "_id" : ObjectId("5f7a53aedccb229a78960d2c"), "firstname" : "John",
  ↳ "lastname" : "Smith" }
{ "_id" : ObjectId("5f7a68bbb94aa03489fa5ca9"), "firstname" : "Mary",
  ↳ "lastname" : "Smith" }
{ "_id" : ObjectId("5f7a68bbb94aa03489fa5caa"), "firstname" : "Mary",
  ↳ "lastname" : "Smith" }
{ "_id" : ObjectId("5f7a68bbb94aa03489fa5cab"), "firstname" : "Mary",
  ↳ "lastname" : "Smith" }
{ "_id" : ObjectId("5f7a68bbb94aa03489fa5cac"), "firstname" : "Mary",
  ↳ "lastname" : "Smith" }
{ "_id" : ObjectId("5f7a68bbb94aa03489fa5cad"), "firstname" : "Mary",
```

①
②
③

```

➡ "lastname" : "Smith" }
{ "_id" : ObjectId("5f7a68bbb94aa03489fa5cae"), "firstname" : "Mary",
➡ "lastname" : "Smith" }
{ "_id" : ObjectId("5f7a68bbb94aa03489fa5caf"), "firstname" : "Mary",
➡ "lastname" : "Smith" }
{ "_id" : ObjectId("5f7a68bbb94aa03489fa5cb0"), "firstname" : "Mary",
➡ "lastname" : "Smith" }
{ "_id" : ObjectId("5f7a68bbb94aa03489fa5cb1"), "firstname" : "Mary",
➡ "lastname" : "Smith" }
{ "_id" : ObjectId("5f7a68bbb94aa03489fa5cb2"), "firstname" : "Mary",
➡ "lastname" : "Smith" }

```

- ❶ Запрос по полю `lastname`.
- ❷ Запись, которую мы опубликовали первой.
- ❸ 10 экземпляров новой записи, созданные из только что отправленных сообщений.

На этом завершается краткое введение во встроенные Ю-коннекторы Pulsar. Теперь вы должны лучше понимать, как конфигурировать и развертывать коннекторы Pulsar и как вообще работает фреймворк коннекторов ввода-вывода.

5.6. Администрирование коннекторов ввода-вывода Pulsar

Инструмент командной строки `pulsar-admin` предоставляет набор команд, позволяющих управлять, контролировать и обновлять коннекторы ввода-вывода Pulsar. Мы рассмотрим некоторые команды, специально предназначенные для коннекторов ввода-вывода Pulsar, а также узнаем, как и когда их следует использовать и какую информацию они предоставляют. Следует отметить, команды для источника и приемника имеют абсолютно одинаковый набор подкоманд, поэтому мы сосредоточимся только на команде для приемника, но все описания также вполне применимы и для источника.

5.6.1. Вывод списка коннекторов

Первой мы рассмотрим команду `pulsar-admin sink list`, возвращающую список всех приемников, работающих в текущий момент в кластере Pulsar. Это удобно, если нужно убедиться в том, что только что созданный коннектор принят и действительно работает. Если бы эта команда была выполнена после развертывания коннектора `mongo-test-sink`, то ее ожидаемый вывод был бы похож на показанный в листинге 5.24.

Листинг 5.24. Вывод команды `list` внутри контейнера Docker

```
docker exec -it pulsar /pulsar/bin/pulsar-admin sink list
[
  "mongo-test-sink"
]
```

Вывод подтверждает, что коннектор `mongo-test-sink` действительно был создан и в текущий момент является единственным коннектором-приемником, работающим в кластере Pulsar. Команду `list` не следует путать с командами `available-sources` и `available-sinks`, которые возвращают список всех встроенных коннекторов, поддерживаемых конкретным кластером Pulsar. По умолчанию встроенные коннекторы включены в автономный контейнер Docker Pulsar, поэтому вывод должен быть таким, как показано в листинге 5.25. Команда `available-sinks` также может помочь в подтверждении успешной установки специализированного коннектора вручную.

Листинг 5.25. Вывод команды `available-sinks` внутри контейнера Docker Pulsar

```
docker exec -it pulsar /pulsar/bin/pulsar-admin sink available-sinks
aerospike
Aerospike database sink
-----
cassandra
Writes data into Cassandra
-----
data-generator
Test data generator source
-----
elastic_search
Writes data into Elastic Search
-----
flume
flume source and sink connector
-----
hbase
Writes data into hbase table
-----
hdfs2
Writes data into HDFS 2.x
-----
hdfs3
Writes data into HDFS 3.x
-----
influxdb
Writes data into InfluxDB database
-----
jdbc-clickhouse
JDBC sink for ClickHouse
```



```

-----
jdbc-mariadb
JDBC sink for MariaDB
-----
jdbc-postgres
JDBC sink for PostgreSQL
-----
jdbc-sqlite
JDBC sink for SQLite
-----
kafka
Kafka source and sink connector
-----
kinesis
Kinesis connectors
-----
mongo
MongoDB source and sink connector
-----
rabbitmq
RabbitMQ source and sink connector
-----
redis
Writes data into Redis
-----
solr
Writes data into solr collection
-----

```

5.6.2. Мониторинг коннекторов

Для мониторинга коннекторов ввода-вывода Pulsar удобна другая команда: `pulsar-admin sink status`. Она возвращает информацию времени выполнения о заданном коннекторе, например количество работающих экземпляров и возникшие в них ошибки.

Листинг 5.26. Вывод команды `pulsar-admin sink status`

```

docker exec -it pulsar /pulsar/bin/pulsar-admin sink status \
--name mongo-test-sink
{
  "numInstances" : 1,
  "numRunning" : 1,
  "instances" : [ {
    "instanceId" : 0,
    "status" : {
      "running" : true,
      "error" : "",
      "numRestarts" : 0,
      "numReadFromPulsar" : 0,
      "numSystemExceptions" : 0,
      "latestSystemExceptions" : [ ],

```

①

②

③

④

⑤

⑥

⑦

```

        "numSinkExceptions" : 0,
        "latestSinkExceptions" : [ ],
        "numWrittenToSink" : 0,
        "lastReceivedTime" : 0,
        "workerId" : "c-standalone-fw-d513daf5b94e-8080"
    }
} ]
}

```

- ❶ Общее количество экземпляров коннектора, которое было затребовано.
- ❷ Общее количество работающих экземпляров коннектора.
- ❸ Массив информации о каждом экземпляре.
- ❹ Текущее состояние коннектора.
- ❺ Любые значимые сообщения об ошибках.
- ❻ Сколько раз выполнялись попытки перезапуска коннектора. Это число увеличивается, когда возникает сбой при запуске коннектора, и он перезапускается.
- ❼ Количество сообщений, потребленных из входящей темы Pulsar этим экземпляром.
- ❽ Последняя метка времени, когда входящее сообщение было потреблено этим экземпляром.

В листинге 5.26 можно видеть, что команда `pulsar-admin sink status` особенно полезна для проверки состояния коннектора сразу после его развертывания для подтверждения его успешного запуска. Команду `pulsar-admin sink get` можно использовать для получения информации о конфигурации коннектора ввода-вывода Pulsar для проверки параметров конфигурации конкретного коннектора, чтобы убедиться в их правильности, как показано в листинге 5.27.

Листинг 5.27. Вывод команды `pulsar-admin sink get`

```

docker exec -it pulsar /pulsar/bin/pulsar-admin sink get \
--name mongo-test-sink
{
  "tenant": "public",
  "namespace": "default",
  "name": "mongo-test-sink",
  "className": "org.apache.pulsar.io.mongodb.MongoSink",
  "inputSpecs": {
    "test-mongo": {
      "isRegexPattern": false
    }
  },
  "configs": {
    "mongoUri": "mongodb://mongodb:27017/admin",
    "database": "pulsar_in_action",
    "collection": "example",
    "batchSize": "1.0",
    "batchTimeMs": "1000.0"
  },
}

```

```

"parallelism": 1,
"processingGuarantees": "ATLEAST_ONCE",
"retainOrdering": false,
"autoAck": true,
"archive": "builtin://mongo"
}

```

6

- ❶ Абонент, пространство имен и имя коннектора.
- ❷ Имя класса реализации коннектора.
- ❸ Входящие темы для коннектора-приемника.
- ❹ Сконфигурирован ли приемник для потребления из нескольких тем на основе некоторого регулярного выражения.
- ❺ Все определенные пользователем свойства конфигурации, предоставленные в файле *sink-config-file*.
- ❻ Все значения по умолчанию для свойств, которые мы не определили.

Еще более полезной делает эту команду тот факт, что ее выводом является конфигурация коннектора в формате JSON, которую можно сохранить в файле, отредактировать и использовать для обновления конфигурации работающего коннектора с помощью команды `update`. Это освобождает вас от необходимости сохранения копии конфигурации, с которой разворачивается конкретный коннектор, так как данные с легкостью извлекаются из работающего коннектора этой командой.

Листинг 5.28. Обновление конфигурации коннектора MongoDB

```

/pulsar/bin/pulsar-admin sink update \
  --sink-type mongo \
  --sink-config-file /pulsar/examples/mongodb-sink.yml \
  --inputs prod-mongo \
  --processing-guarantees EFFECTIVELY_ONCE

```

Команда `pulsar-admin sink update` позволяет динамически изменять параметры конфигурации уже включенного в работу коннектора-приемника без необходимости его удаления и повторного создания. Команда `update` принимает разнообразные ключи командной строки, подробно описанные в официальной документации Apache (<https://pulsar.apache.org/reference/#3.1.x/pulsar-admin/sinks?id=update>), позволяющие изменять практически любое свойство конфигурации коннектора, в том числе архивный файл, если необходимо развернуть новую версию коннектора. Такая возможность делает изменение, тестирование и развертывание гораздо более гибким и оптимизированным процессом. В листинге 5.28 показано, как обновить ранее развернутый коннектор-приемник для MongoDB, чтобы использовать другие темы Pulsar в качестве входящего источника и изменить гарантии обработки.

На этом завершается краткий обзор некоторых команд, доступных для мониторинга и администрирования коннекторов ввода-

вывода Pulsar. Его целью было предоставление общего обзора возможностей, предоставляемых этим фреймворком, чтобы вы смогли начать работу с ним. Я настоятельно рекомендую изучить онлайн-документацию, чтобы узнать все подробности о разнообразных ключах и параметрах для каждой из команд.

5.7. Резюме

- Коннекторы ввода-вывода Pulsar – это расширение фреймворка Pulsar Functions, специально предназначенное для взаимодействия с внешними системами, например с базами данных.
- Коннекторы ввода-вывода Pulsar представлены двумя основными типами, это источники, извлекающие данные из внешних систем в Pulsar, и приемники, публикующие данные из Pulsar во внешние системы.
- Pulsar предоставляет набор встроенных коннекторов, которые можно использовать для взаимодействия с несколькими широко распространенными системами без необходимости написания какого-либо исходного кода.
- Инструмент командной строки Pulsar позволяет администрировать коннекторы ввода-вывода Pulsar, в том числе создавать, удалять и обновлять их.



Обеспечение безопасности Pulsar

Темы:

- шифрование данных, передаваемых в кластер Pulsar и из него;
- обеспечение аутентификации клиента с использованием JSON Web Tokens (JWT);
- шифрование данных, хранящихся в фреймворке Apache Pulsar.

В этой главе рассматривается, как обеспечить безопасность кластера, чтобы защитить его от неавторизованного доступа к данным, проходящим через Apache Pulsar. Хотя описанные в этой главе задачи не имеют значения в среде разработки, они чрезвычайно важны в производственной среде развертывания для снижения риска неавторизованного доступа к секретной информации, предотвращения потерь данных и защиты публичной репутации вашей организации. Современные системы и организации используют различные комбинации средств управления безопасностью и мер защиты, чтобы обеспечить несколько уровней защиты, предотвращающих доступ к данным внутри системы. Это особенно важно для систем и организаций, которые обязаны поддерживать

полное соответствие правилам обеспечения безопасности, например HIPPA, PCI-DSS, GDPR и многие другие.

Pulsar обеспечивает качественную интеграцию с несколькими существующими фреймворками обеспечения безопасности, позволяя использовать эти инструментальные средства для защиты кластера Pulsar на нескольких уровнях, чтобы снизить риск возникновения брешы в одном из механизмов обеспечения безопасности, результатом чего становится общее нарушение защиты. Например, даже если неавторизованный пользователь получит возможность доступа к системе с похищенным паролем, ему потребуется правильный ключ для чтения зашифрованных данных сообщения.

6.1. Шифрование на транспортном уровне

По умолчанию данные, передаваемые между брокером Pulsar и клиентом, отправляются как обычный текст. Это означает, что любые секретные данные, содержащиеся в сообщении, например пароли, номера кредитных карт и социальных страховок, могут быть перехвачены злоумышленниками при передаче по сети. Следовательно, первым уровнем защиты является шифрование данных до передачи их между брокером Pulsar и клиентом.

Pulsar позволяет конфигурировать все каналы обмена информацией для использования TLS (transport layer security) – широко применяемого криптографического протокола, обеспечивающего шифрование данных только при передаче по сети. Именно поэтому его часто называют шифрованием «данных в движении» – данные расшифровываются в принимающей конечной точке, т. е. перестают быть зашифрованными после достижения цели.

Обеспечение шифрования TLS в Pulsar

После краткого описания основ сетевого шифрования данных по протоколу TLS сосредоточимся на том, как можно использовать эту технологию для защиты каналов обмена информацией между брокерами Pulsar и клиентами. Поскольку в документации достаточно подробно описаны шаги, необходимые для обеспечения работы TLS в Pulsar, я решил, что вместо повторного описания этих шагов здесь я включу их в единый скрипт, который можно использовать для автоматизации процесса внутри образа в контейнере Docker.

Затем я более подробно рассмотрю команды, содержащиеся в сценариях, поскольку они связаны с описанием из предыдущего раздела, чтобы вы лучше поняли, почему эти шаги важны и как вы можете изменить их в соответствии со своими потребностями в реальной производственной среде. Если вы заглянете в репозиторий GitHub (<https://github.com/david-streamlio/pulsar-in-action>), связанный с этой книгой, то обнаружите *Dockerfile*, похожий на приведенный в листинге 6.1, в подкаталоге *docker-image/pulsar-standalone*.

Для тех, кто не знаком с Docker, поясняю: *Dockerfile* – это простой текстовый файл, содержащий последовательность инструкций, которые поочередно выполняются клиентом Docker при создании образа. Их можно считать рецептами или схемами создания образов Docker, предоставляющими простой способ автоматизации этого процесса. Теперь, когда нам лучше понятен принцип создания образов Docker, наступило время для создания собственного образа. Целью является создание образа, расширяющего возможности используемого ранее *pulsar-standalone* для включения всех функций обеспечения безопасности, которые будут рассматриваться в этой главе.

Листинг 6.1. Содержимое Dockerfile

```
FROM apache/pulsar/pulsar-standalone:latest ❶
ENV PULSAR_HOME=/pulsar

COPY conf/standalone.conf $PULSAR_HOME/conf/standalone.conf ❷
COPY conf/client.conf $PULSAR_HOME/conf/client.conf ❸

ADD manning $PULSAR_HOME/manning ❹

RUN chmod a+x $PULSAR_HOME/manning/security/*.sh \
    $PULSAR_HOME/manning/security/*/*.sh \
    $PULSAR_HOME/manning/security/authentication/*/*.sh ❺

#####
# Шифрование на транспортном уровне с использованием TLS.
#####
RUN ["/bin/bash", "-c",
    "/pulsar/manning/security/TLS-encryption/enable-tls.sh"] ❻
```

- ❶ Использование существующего образа *pulsar-standalone* как отправного пункта для эффективного расширения его возможностей.
- ❷ Замещение конфигурации брокера другой версией, содержащей обновленные параметры обеспечения безопасности.
- ❸ Замещение конфигурации клиента другой версией, содержащей обновленные идентификационные данные клиента.
- ❹ Копирование содержимого каталога *manning* в подкаталог */pulsar/manning* внутри образа.
- ❺ Предоставление прав на выполнение всем скриптам, которые необходимо выполнить.
- ❻ Выполнение специализированного скрипта для генерации сертификатов, требуемых протоколом TLS.

Начнем с использования незащищенного образа *apache/pulsar/pulsar-standalone:latest* как основы, используя для этого ключевое слово *FROM*. Для тех, кто знаком с объектно ориентированными языками, следует уточнить, что, по существу, это то же самое, что наследование от базового класса. Все сервисы, функции и конфигурации базового

образа автоматически включаются в наш образ, гарантируя, что образы Docker также будут предоставлять полноценную среду Pulsar для тестирования без необходимости повторять те же команды в *Dockerfile*.

После установки переменной среды `PULSAR_HOME` применяется ключевое слово `COPY` для замены файлов конфигурации брокера Pulsar и клиента на другие версии, сконфигурированные для обеспечения безопасности этого экземпляра Pulsar. Это важный шаг, так как после запуска контейнера на основе данного образа невозможно изменить параметры конфигурации и активизировать их. Далее содержимое каталога *manning* добавляется в образ Docker, и выполняется команда, предоставляющая права на выполнение всем *bash*-скриптам, добавленным предыдущей командой, после чего их можно выполнять.

В конце *Dockerfile* находится последовательность *bash*-скриптов, выполняемых для генерации необходимых средств защиты, сертификатов и ключей, требуемых для обеспечения безопасности кластера Pulsar. Рассмотрим подробнее первый скрипт с именем *enable-tls.sh*, выполняющий все шаги для обеспечения шифрования на транспортном уровне по протоколу TLS в кластере Pulsar.

Образ *pulsar-standalone* включает OpenSSL, библиотеку с открытым исходным кодом, которая предоставляет несколько инструментов командной строки для генерации и подписи цифровых сертификатов. Поскольку мы работаем в среде разработки, будем использовать эти инструменты для обеспечения авторизации собственного сертификата, чтобы создать самозаверяющие сертификаты, а не пользоваться надежными сторонними глобальными сервисами авторизации сертификатов (CA) (например, VeriSign, DigiCert) для подписи. В производственной среде всегда следует полагаться на сторонние сервисы CA.

Функционирование в качестве CA означает работу с криптографическими парами секретных ключей и открытых сертификатов. В первую очередь создадим криптографическую пару суперпользователя (*root pair*). Она состоит из ключа суперпользователя (*ca.key.pem*) и сертификата суперпользователя (*ca.cert.pem*). Эта пара образует идентификационные данные CA. Рассмотрим первую часть скрипта *enable-tls.sh*, которая генерирует эту пару. В листинге 6.2 можно видеть, что первая команда в скрипте генерирует ключ суперпользователя, а вторая создает открытый сертификат X.509, используя ключ суперпользователя, сгенерированный на предыдущем шаге.

Листинг 6.2. Часть скрипта *enable-tls.sh*, которая создает CA

```
#!/bin/bash

export CA_HOME=$(pwd)
export CA_PASSWORD=secret-password
```



```

mkdir certs crl newcerts private
chmod 700 private/
touch index.txt index.txt.attr
echo 1000 > serial

# Генерация секретного ключа для сертификата авторизации.
openssl genrsa -aes256 \
    -passout pass:${CA_PASSWORD} \
    -out /pulsar/manning/security/cert-authority/private/ca.key.pem \
    4096

# Ограничение доступа к секретному ключу сертификата авторизации.
chmod 400 /pulsar/manning/security/cert-authority/private/ca.key.pem

# Создание сертификата X.509.
openssl req -config openssl.cnf \
    -key /pulsar/manning/security/cert-authority/private/ca.key.pem \
    -new -x509 \
    -days 7300 \
    -sha256 \
    -extensions v3_ca \
    -out /pulsar/manning/security/cert-authority/certs/ca.cert.pem \
    -subj '/C=US/ST=CA/L=Palo Alto/CN=gottaeat.com' \
    -passin pass:${CA_PASSWORD}

```

- ❶ Эта переменная среды используется для установки пароля для ключа суперпользователя CA.
- ❷ Зашифрование ключа по протоколу AES 256-бит.
- ❸ Генерация секретного ключа, защищенного надежным паролем.
- ❹ Использование 4096 бит для ключа суперпользователя.
- ❺ Каждый, кто владеет ключом суперпользователя и паролем, может создавать надежные сертификаты.
- ❻ Генерация сертификата суперпользователя с применением ключа суперпользователя.
- ❼ Требуется новый сертификат X.509.
- ❽ Для этого сертификата суперпользователя устанавливается длительный срок действия.
- ❾ Определение организации, для которой этот сертификат является действительным.
- ❿ Позволяет предоставить пароль для ключа пользователя из командной строки без промпта.

К этому моменту скрипт сгенерировал защищенный паролем секретный ключ (`ca.key.pem`) и сертификат суперпользователя (`ca.cert.pem`) для внутренней авторизации CA. Скрипт преднамеренно генерирует сертификат суперпользователя для известной локации, поэтому на нее можно ссылаться из файла конфигурации брокера `/pulsar/conf/standalone.conf`. В частности, мы предварительно сконфигурировали свойство `tlsTrustCertsFilePath` для указания на локацию, в которой был сгенерирован сертификат суперпользователя. В производственной среде с применением стороннего

сервиса авторизации CA будет предоставлен сертификат, который можно использовать для аутентификации сертификатов X.509, поэтому потребуется обновление этого свойства, чтобы оно указывало на такой сертификат.

После создания сертификата CA следующим шагом является генерация сертификата для брокера Pulsar и его подпись нашим внутренним сервисом CA. При использовании стороннего сервиса CA запрос необходимо направить в этот сервис и ждать передачи сертификата. Но, поскольку мы работаем с собственным сервисом CA, мы можем создать сертификат самостоятельно, как показано в листинге 6.3.

Листинг 6.3. Часть скрипта `enable-tls.sh`, которая генерирует сертификат брокера Pulsar

```
export BROKER_PASSWORD=my-secret ❶

# Генерация секретного ключа для сертификата сервера.
openssl genrsa -passout pass:${BROKER_PASSWORD} \ ❷
    -out /pulsar/manning/security/cert-authority/broker.key.pem \ ❸
    2048

# Преобразование ключа в формат PEM.
openssl pkcs8 -topk8 -inform PEM -outform PEM \ ❹
    -in /pulsar/manning/security/cert-authority/broker.key.pem \
    -out /pulsar/manning/security/cert-authority/broker.key-pk8.pem \
    -nocrypt

# Генерация запроса сертификата сервера.
openssl req -config /pulsar/manning/security/cert-authority/openssl.cnf \
    -new -sha256 \
    -key /pulsar/manning/security/cert-authority/broker.key.pem \
    -out /pulsar/manning/security/cert-authority/broker.csr.pem \
    -subj '/C=US/ST=CA/L=Palo Alto/O=IT/CN=pulsar.gottaeat.com' \ ❺
    -passin pass:${BROKER_PASSWORD} ❻

# Подпись сертификата сервера сервисом CA.
openssl ca -config /pulsar/manning/security/cert-authority/openssl.cnf \
    -extensions server_cert \ ❼
    -days 1000 -notext -md sha256 -batch \ ❽
    -in /pulsar/manning/security/cert-authority/broker.csr.pem \
    -out /pulsar/manning/security/cert-authority/broker.cert.pem \
    -passin pass:${CA_PASSWORD} ❾
```

- ❶ Эта переменная среды используется для установки пароля для секретного ключа сертификата сервера.
- ❷ Генерация секретного ключа, защищенного надежным паролем.
- ❸ Использование 2048 бит для секретного ключа.
- ❹ Брокер ожидает ключ в формате PKCS 8.
- ❺ Определение организации и имени хоста, для которых этот сертификат является действительным.

- 6 Позволяет предоставить пароль для ключа из командной строки без промпта.
- 7 Определяет, что этот сертификат предназначен для использования сервером.
- 8 Для этого сертификата сервера устанавливается длительный срок действия.
- 9 Необходимо предоставить пароль для секретного ключа, поскольку мы сами работаем как CA.

К этому моменту мы имеем сертификат брокера (`broker.cert.pem`) и связанный с ним секретный ключ (`broker.key-pk8.pem`), которые можно использовать вместе с `ca.cert.pem` для конфигурирования шифрования на транспортном уровне TLS для узлов брокера и прокси. И в этом случае скрипт преднамеренно генерирует сертификат брокера для известной локации, поэтому на нее можно ссылаться из файла конфигурации брокера `/pulsar/conf/standalone.conf`. Рассмотрим все свойства, которые были изменены для обеспечения работы TLS в Pulsar.

Листинг 6.4. Изменения свойств TLS в файле `standalone.conf`

```
#### Для обеспечения шифрования TLS на транспортном уровне. #####
tlsEnabled=true
brokerServicePortTls=6651
webServicePortTls=8443

# Сертификат брокера и связанный с ним секретный ключ.
tlsCertificateFilePath=/pulsar/manning/security/cert-authority/
➔ broker.cert.pem
tlsKeyFilePath=/pulsar/manning/security/cert-authority/broker.key-pk8.pem

# Сертификат CA.
tlsTrustCertsFilePath=/pulsar/manning/security/cert-authority/certs/
➔ ca.cert.pem

# Используется для согласования с TLS, позволяющего определить, какие шифры
# считаются безопасными.
tlsProtocols=TLSv1.2,TLSv1.1
tlsCiphers=TLS_DH_RSA_WITH_AES_256_GCM_SHA384,TLS_DH_RSA_WITH_AES_256_CBC_SHA
```

Поскольку мы разрешили шифрование TLS на транспортном уровне, необходимо также сконфигурировать инструменты командной строки – `pulsar-admin` и `pulsar-perf` – для обеспечения возможности обмена данными с защищенным брокером Pulsar. Для этого нужно изменить свойства в файле `$PULSAR_HOME/conf/client.conf`, как показано в листинге 6.5.

Листинг 6.5. Изменение свойств TLS в файле client.conf

```
#### Для обеспечения шифрования TLS на транспортном уровне. ####
# Использование протоколов и портов TLS.
webServiceUrl=https://pulsar.gottaeat.com:8443/
brokerServiceUrl=pulsar+ssl://pulsar.gottaeat.com:6651/

useTls=true
tlsAllowInsecureConnection=false
tlsEnableHostnameVerification=false
tlsTrustCertsFilePath=pulsar/manning/security/cert-authority/certs/ca.cert.pem
```

Если вы еще не сделали это, то внесите изменения в каталоге, содержащем *Dockerfile*, выполните команду `docker build . -t pia/pulsar-standalone-secure:latest` для создания образа Docker и пометьте его, как показано в листинге 6.6.

Листинг 6.6. Создание образа Docker из Dockerfile

```
$ cd $REPO_HOME/pulsar-in-action/docker-images/pulsar-standalone ①
$ docker build . -t pia/pulsar-standalone-secure:latest ②
Sending build context to Docker daemon 3.345MB
Step 1/7 : FROM apachepulsar/pulsar-standalone:latest
--> 3ed9bffff717 ③
Step 2/7 : ENV PULSAR_HOME=/pulsar
--> Running in cf81f78f5754
Removing intermediate container cf81f78f5754
--> 48ea643513ff
Step 3/7 : COPY conf/standalone.conf $PULSAR_HOME/conf/standalone.conf
--> 6dcf0068eb40
Step 4/7 : COPY conf/client.conf $PULSAR_HOME/conf/client.conf
--> e0f6c81a10c4
Step 5/7 : ADD manning $PULSAR_HOME/manning ④
--> e253e7c6ed8e
Step 6/7 : RUN chmod a+x $PULSAR_HOME/manning/security/*.sh
➔ $PULSAR_HOME/manning/security/*.sh
➔ $PULSAR_HOME/manning/security/authentication/*.sh
--> Running in 42d33f3e738b
Removing intermediate container 42d33f3e738b
--> ddccc85c75f4
Step 7/7 : RUN ["/bin/bash", "-c",
➔ "/pulsar/manning/security/TLS-encryption/enable-tls.sh"] ⑤
--> Running in 5f26f9626a25
Generating RSA private key, 4096 bit long modulus
.....+++++
.....
.....
.....+++++
e is 65537 (0x010001)
Generating RSA private key, 2048 bit long modulus
.....+++++
.....+++++
```

```

e is 65537 (0x010001)
Using configuration from /pulsar/manning/security/cert-authority/openssl.cnf
Check that the request matches the signature
Signature ok
Certificate Details:
    Serial Number: 4096 (0x1000)
    Validity
        Not Before: Jan 13 00:55:03 2020 GMT
        Not After : Oct 9 00:55:03 2022 GMT
    Subject:
        countryName           = US
        stateOrProvinceName   = CA
        organizationName      = gottaeat.com
        commonName            = pulsar.gottaeat.com
    X509v3 extensions:
        X509v3 Basic Constraints:
            CA:FALSE
        Netscape Cert Type:
            SSL Server
        Netscape Comment:
            OpenSSL Generated Server Certificate
        X509v3 Subject Key Identifier:
            DF:75:74:9D:34:C6:0D:F0:9B:E7:CA:07:0A:37:B8:6F:D7:DF:52:0A
        X509v3 Authority Key Identifier:

keyid:92:F0:6D:0F:18:D4:3C:1E:88:B1:33:3A:9D:04:29:C0:FC:81:29:02
    DirName:/C=US/ST=CA/L=Palo Alto/O=gottaeat.com
    serial:93:FD:42:06:D8:E9:C3:89

    X509v3 Key Usage: critical
        Digital Signature, Key Encipherment
    X509v3 Extended Key Usage:
        TLS Web Server Authentication
Certificate is to be certified until Oct 9 00:55:03 2022 GMT (1000 days)

Write out database with 1 new entries
Data Base Updated
Removing intermediate container 5f26f9626a25
--> 0e7995c14208
Successfully built 0e7995c14208
Successfully tagged pia/pulsar-standalone-secure:latest

```

- 1 Переход в каталог, содержащий *Dockerfile*.
- 2 Команда создания образа Docker из *Dockerfile* и его пометки.
- 3 Извлечение образа `arache/pulsar/pulsar-standalone:latest` из общедоступного репозитория.
- 4 Копирование всего содержимого каталога *manning* в образ Docker.
- 5 Выполнение скрипта *enable-tls.sh*.
- 6 Подробности сертификата сервера, генерируемого скриптом *enable-tls.sh*.
- 7 Срок действия этого сертификата.

- ❶ Сообщение о том, что сгенерированный сертификат можно использовать как сертификат сервера.
- ❷ Сообщение об успешном создании, включающее тег (метку), используемый для ссылки на образ.

После того как скрипт *enable-tls.sh* выполнен и все свойства сконфигурированы для указания на корректные значения, образ *pulsar-standalone* будет принимать соединения только по защищенному TLS-каналу. Это можно проверить, используя последовательность команд, показанных в листинге 6.7, для запуска контейнера с новым образом Docker. Обратите внимание: используется ключ *-volume* для создания логической точки монтирования между каталогом *\$HOME* на моем ноутбуке и каталогом внутри контейнера Docker. Это позволяет опубликовать идентификационные данные клиента TLS, которые существуют только внутри контейнера, в локации на моем компьютере, в которой я могу получить к ним доступ. Далее требуется защищенная командная оболочка (*ssh*) внутри контейнера и выполнение скрипта *publish-credentials.sh* в новом созданном сеансе *bash*, чтобы сделать эти идентификационные данные доступными в каталоге *\${HOME}/exchange* на моем локальном компьютере.

Листинг 6.7. Публикация идентификационных данных TLS

```
$ docker run -id --name pulsar -p:6651:6651 -p 8443:8443 \
  -volume=${HOME}/exchange:/pulsar/manning/dropbox \
  -t pia/pulsar-standalone-secure:latest
$ docker exec -it pulsar bash
$ /pulsar/manning/security/publish-credentials.sh
$ exit
```

- ❶ Использование SSL-портов 6651 и 8443.
- ❷ Использование ключа *-volume* для обеспечения копирования файлов из контейнера на локальный компьютер.
- ❸ Определение только что созданного образа.
- ❹ Выполнение скрипта, копирующего сгенерированные идентификационные данные на локальный компьютер.
- ❺ Создание пространства имен по умолчанию для тестирования.
- ❻ Выход из контейнера.

Теперь попытаемся установить соединение с контейнером через защищенный TLS-порт (6651), используя программу на языке Java (см. листинг 6.8), также доступную в репозитории GitHub, связанном с этой книгой.

Листинг 6.8. Использование идентификационных данных TLS для установления соединения с Pulsar

```
import org.apache.pulsar.client.api.*;
public class TlsClient {
    public static void main(String[] args) throws PulsarClientException {
        final String HOME = "/Users/david/exchange";
        final String TOPIC = "persistent://public/default/test-topic";

        PulsarClient client = PulsarClient.builder()
            .serviceUrl("pulsar://localhost:6651/")
            .tlsTrustCertsFilePath(HOME + "/ca.cert.pem")
            .build();

        Producer<byte[]> producer =
            client.newProducer().topic(TOPIC).create();

        for (int idx = 0; idx < 100; idx++) {
            producer.send("Hello TLS".getBytes());
        }
        System.exit(0);
    }
}
```

- ❶ Определение протокола `pulsar+ssl`. Ошибка в этом определении приведет к отказу в установлении соединения.
- ❷ Локация файла, содержащего надежные сертификаты TLS.

На этом завершается процедура конфигурирования шифрования TLS на транспортном уровне для брокера Pulsar. С этого момента весь обмен данными с брокером будет выполняться через SSL, и весь трафик будет зашифрован для предотвращения неавторизованного доступа к данным внутри сообщений, публикуемых и потребляемых из брокера Pulsar. Вы можете поэкспериментировать с этим контейнером, но после того, как завершите свои эксперименты, обязательно выполните команду `docker stop pulsar && docker rm pulsar` для остановки и удаления контейнера. Это необходимо для создания нового образа Docker в следующем разделе, чтобы обеспечить поддержку аутентификации в образе `pulsar-standalone`.

6.2. Аутентификация

Сервис аутентификации предоставляет пользователю способ подтверждения своей идентичности посредством предоставления некоторых идентификационных данных, например имени пользователя и пароля, чтобы доказать, что вы – это действительно вы. Pulsar поддерживает подключаемый в виде модуля механизм, который можно использовать для собственной аутентификации.

В настоящее время Pulsar поддерживает различных провайдеров аутентификации. В этом разделе мы подробно рассмотрим все шаги, требуемые для конфигурирования обоих вариантов аутентификации TLS и JWT.

6.2.1. Аутентификация TLS

При аутентификации TLS клиент использует собственный сертификат. Получение сертификата требует взаимодействия с сервисом CA, генерирующим сертификат, которому может доверять конкретный брокер Pulsar. Чтобы сертификат клиента прошел процесс валидации сервера, обнаруженная цифровая сигнатура должна быть подписана сервисом CA, распознаваемым этим сервером. Поэтому я использовал тот же CA, который сгенерировал сертификат сервера в предыдущем разделе для клиента, чтобы обеспечить доверительное отношение к сертификатам клиентов для аутентификации.

При аутентификации TLS клиент генерирует пару ключей для этой цели и сохраняет секретный ключ из пары в защищенной локации. Затем клиент создает запрос сертификата к надежному сервису CA и получает в ответ цифровой сертификат X.509.

Эти сертификаты клиентов обычно содержат соответствующую информацию, такую как цифровая подпись, дата окончания срока действия, имя клиента, имя CA, состояние аннулирования, номер версии SSL/TLS, серийный номер, общее имя и, возможно, что-то еще – все структурировано с использованием стандарта X.509. Pulsar использует поле общего имени сертификата для установления соответствия клиента некоторой конкретной роли, которая используется, чтобы определить, для выполнения каких действий авторизуется клиент.

Если клиент пытается установить соединение с брокером Pulsar, на котором разрешена аутентификация TLS, то он может передать сертификат клиента для аутентификации как часть процедуры квитирования TLS. При получении такого сертификата брокер Pulsar использует его для идентификации источника и определяет, должен ли клиент получить требуемые права доступа.

Не следует путать сертификаты клиентов с сертификатом сервера, используемым для обеспечения шифрования TLS. Оба сертификата являются цифровыми по стандарту X.509, но это два совершенно различных объекта. Сертификат сервера передается из брокера Pulsar клиенту в начале сеанса и используется клиентом для аутентификации сервера. С другой стороны, сертификат клиента отправляется от клиента брокеру в начале сеанса и используется сервером для аутентификации клиента. Чтобы разрешить аутентификацию на основе TLS, в *Dockerfile* из предыдущего раздела мы добавляем команду, выполняющую еще один скрипт с именем

gen-client-certs.sh. Этот скрипт генерирует TLS-сертификаты клиента, которые можно использовать для аутентификации, как показано в листинге 6.9.

Листинг 6.9. Обновленное содержимое Dockerfile

```
FROM apache/pulsar/pulsar-standalone:latest
ENV PULSAR_HOME=/pulsar

...

RUN ["/bin/bash", "-c",
    "/pulsar/manning/security/TLS-encryption/enable-tls.sh"]

RUN ["/bin/bash", "-c",
    "/pulsar/manning/security/authentication/tls/gen-client-certs.sh"] ❶
# Содержимое то же, что показано в листинге 6.1.
```

❶ Выполнение указанного скрипта для генерации TLS-сертификатов клиента для аутентификации, основанной на ролях.

Рассмотрим подробнее скрипт *gen-client-certs.sh*, показанный в листинге 6.10, чтобы проследить шаги, требуемые для генерации TLS-сертификатов клиента, которые можно использовать для аутентификации в брокере Pulsar.

Листинг 6.10. Содержимое файла gen-client-certs.sh

```
#!/bin/bash

cd /pulsar/manning/security/cert-authority
export CA_HOME=$(pwd)
export CA_PASSWORD=secret-password

function generate_client_cert() {

    local CLIENT_ID=$1 ❶
    local CLIENT_ROLE=$2 ❷
    local CLIENT_PASSWORD=$3 ❸

    # Генерация секретного ключа для сертификата клиента.
    openssl genrsa -passout pass:${CLIENT_PASSWORD} \ ❹
    -out /pulsar/manning/security/authentication/tls/${CLIENT_ID}.key.pem \
    2048

    # Преобразование ключа в формат PEM.
    openssl pkcs8 -topk8 -inform PEM -outform PEM -nocrypt \
    -in /pulsar/manning/security/authentication/tls/${CLIENT_ID}.key.pem \
    -out /pulsar/manning/security/authentication/tls/${CLIENT_ID}-pk8.pem

    # Генерация запроса сертификата клиента.
    openssl req -config /pulsar/manning/security/cert-authority/openssl.cnf \
```

```
-key /pulsar/manning/security/authentication/tls/${CLIENT_ID}.key.pem
-out /pulsar/manning/security/authentication/tls/${CLIENT_ID}.csr.pem \
-subj "/C=US/ST=CA/L=Palo Alto/O=gottaeat.com/CN=${CLIENT_ROLE}" \
-new -sha256 \
-passin pass:${CLIENT_PASSWORD}

# Подпись сертификата сервера сервисом CA.
openssl ca -config /pulsar/manning/security/cert-authority/openssl.cnf \
-extentions usr_cert \
-days 100 -notext -md sha256 -batch \
-in /pulsar/manning/security/authentication/tls/${CLIENT_ID}.csr.pem \
-out /pulsar/manning/security/authentication/tls/${CLIENT_ID}.cert.pem \
-passin pass:${CA_PASSWORD}

# Удаление ключа клиента и запроса сертификата после завершения.
rm -f /pulsar/manning/security/authentication/tls/${CLIENT_ID}.csr.pem
rm -f /pulsar/manning/security/authentication/tls/${CLIENT_ID}.key.pem
}

# Создание сертификата для пользователя adam с правами доступа на уровне
# роли администратора.
generate_client_cert adam admin admin-secret

# Создание сертификата для веб-приложения с правами доступа на уровне
# роли webapp.
generate_client_cert webapp-service webapp webapp-secret

# Создание сертификата для пользователя peggy с правами доступа на уровне
# роли payments.
generate_client_cert peggy payments payment-secret

# Создание сертификата для пользователя david с правами доступа на уровне
# роли driver.
generate_client_cert david driver davids-secret
```

- ❶ Создание локальной переменной `CLIENT_ID` для первого параметра, переданного при вызове этой функции.
- ❷ Создание локальной переменной `CLIENT_ROLE` для второго параметра, переданного при вызове этой функции.
- ❸ Создание локальной переменной `CLIENT_PASSWORD` для третьего параметра, переданного при вызове этой функции.
- ❹ Использование переменной `CLIENT_PASSWORD` для защиты секретного ключа.
- ❺ Pulsar использует значение, связанное с общим именем (CN), для определения роли клиента.
- ❻ Необходимо передать пароль, связанный с ключом клиента, используемого для генерации CSR.
- ❼ Указание на необходимость предоставления сертификата клиента.
- ❽ Необходимо предоставить пароль CA для подтверждения и подписи запроса сертификата клиента.

В производственной среде клиенты должны использовать собственные секретные ключи для генерации запросов сертификатов и передавать их только в CSR-файлах. Поскольку в рассматриваемом здесь примере процесс автоматизирован, я предоставил свободу при генерации как части скрипта для небольшого количества пользователей, каждый из которых имеет роль, отличающуюся от других.

Теперь существует несколько пар секретных ключей и сертификатов клиентов, сгенерированных и подписанных нашим сервисом CA. Их можно использовать для аутентификации в экземпляре `pulsar-standalone`. Свойства, показанные в листинге 6.11, также были добавлены в файл `standalone.conf`, чтобы разрешить аутентификацию на основе TLS в брокере.

Листинг 6.11. Изменения свойств аутентификации TLS в файле `standalone.conf`

```
#### Для разрешения аутентификации TLS.
authenticationEnabled=true
authenticationProviders=org.apache.pulsar.broker.
➔ authentication.AuthenticationProviderTls
```

- ❶ Активизация аутентификации.
- ❷ Определение провайдера аутентификации TLS.

Изменения также требуются и в файле `client.conf`, как показано в листинге 6.12, для предоставления инструментам командной строки Pulsar прав доступа уровня администратора, в том числе возможности определять правила авторизации на уровне пространства имен Pulsar.

Листинг 6.12. Изменения свойств аутентификации TLS в файле `client.conf`

```
## Аутентификация TLS. ##
authPlugin=org.apache.pulsar.client.impl.auth.AuthenticationTls
authParams=tlsCertFile:/pulsar/manning/security/authentication/tls/
➔ admin.cert.pem,tlsKeyFile:/pulsar/manning/security/authentication/tls/
➔ admin-pk8.pem
```

- ❶ Оповещение клиента о необходимости использования аутентификации TLS при установлении соединения с Pulsar.
- ❷ Определение используемого сертификата клиента и связанного с ним секретного ключа.

Теперь необходимо снова создать образ, чтобы включить в него изменения, требуемые для генерации TLS-сертификатов клиента и разрешения аутентификации на основе TLS, выполнив команды, показанные в листинге 6.13.

Листинг 6.13. Создание образа Docker из Dockerfile

```

$ cd $REPO_HOME/pulsar-in-action/docker-images/pulsar-standalone
$ docker build . -t pia/pulsar-standalone-secure:latest
Sending build context to Docker daemon 3.345MB
Step 1/8 : FROM apache/pulsar/pulsar-standalone:latest
--> 3ed9bffff717
...
Step 8/8 : RUN ["/bin/bash", "-c",
    "/pulsar/manning/security/authentication/tls/gen-client-certs.sh"]
--> Running in 5beaabba5865
Generating RSA private key, 2048 bit long modulus
.....+++++
.....+++++
e is 65537 (0x010001)
Using configuration from /pulsar/manning/security/cert-authority/openssl.cnf
Check that the request matches the signature
Signature ok
Certificate Details:
    Serial Number: 4097 (0x1001)
    Validity
        Not Before: Jan 13 02:47:54 2020 GMT
        Not After : Apr 22 02:47:54 2020 GMT
    Subject:
        countryName           = US
        stateOrProvinceName   = CA
        organizationName      = gottaeat.com
        commonName            = admin
    ...

Write out database with 1 new entries
Data Base Updated
Generating RSA private key, 2048 bit long modulus
...
Successfully built 6ef6eddd675e
Successfully tagged pia/pulsar-standalone-secure:latest

```

- 1 Переход в каталог, содержащий *Dockerfile*.
- 2 Команда для создания и пометки образа Docker из *Dockerfile*.
- 3 Обратите внимание: теперь здесь восемь шагов вместо семи.
- 4 Выполнение шагов 2–7.
- 5 Выполнение скрипта *gen-client-certs.sh*.
- 6 Следующая строка файла конфигурации повторяется пять раз – для каждой операции генерации сертификата клиента.
- 7 Серийный номер сертификата индивидуален для каждого клиента.
- 8 Общее имя различно для каждого клиента. Оно используется для связывания клиента с конкретной ролью.
- 9 Здесь должны располагаться еще четыре строки файла конфигурации – по одной для каждого сгенерированного сертификата клиента.
- 10 Сообщение об успешном завершении процесса, включающее тег (метку), используемый для ссылки на этот образ.

Далее необходимо снова выполнить шаги, показанные в листинге 6.7, чтобы запустить новый контейнер на основе обновленного образа Docker, защищенную командную оболочку `ssh` внутри контейнера и выполнить скрипт `publish-credentials.sh` в новом активизированном сеансе `bash`, чтобы создать сертификаты клиентов, доступные в каталоге `${HOME}/exchange` на локальном компьютере. Например, файл сертификата `admin.cert.pem` и связанный с ним файл секретного ключа `admin-pk8.pem` теперь можно использовать для аутентификации в экземпляре `pulsar-standalone`. Далее я попытаюсь выполнить аутентификацию TLS, используя программу на языке Java, показанную в листинге 6.14. Эта программа также доступна в репозитории GitHub, связанном с книгой.

Листинг 6.14. Аутентификация с использованием TLS-сертификатов клиентов

```
import org.apache.pulsar.client.api.*;

public class TlsAuthClient {
    public static void main(String[] args) throws PulsarClientException {
        final String AUTH =
            "org.apache.pulsar.client.impl.auth.AuthenticationTls";
        final String HOME = "/Users/david/exchange";
        final String TOPIC = "persistent://public/default/test-topic";

        PulsarClient client = PulsarClient.builder()
            .serviceUrl("pulsar+ssl://localhost:6651/")
            .tlsTrustCertsFilePath(HOME + "/ca.cert.pem")
            .authentication(AUTH,
                "tlsCertFile:" + HOME + "/admin.cert.pem," +
                "tlsKeyFile:" + HOME + "/admin-pk8.pem")
            .build();

        Producer<byte[]> producer =
            client.newProducer().topic(TOPIC).create();

        for (int idx = 0; idx < 100; idx++) {
            producer.send("Hello TLS Auth".getBytes());
        }
        System.exit(0);
    }
}
```

- ❶ Использование протокола `Pulsar+ssl`. Ошибка в этом определении приводит к отказу в установлении соединения.
- ❷ Локация файла, содержащего надежные TLS-сертификаты.
- ❸ Использование аутентификации TLS.
- ❹ Локация файла сертификата клиента.
- ❺ Секретный ключ, связанный с этим сертификатом клиента.

TLS-аутентификация клиента удобна в тех вариантах, где сервер отслеживает сотни тысяч или даже миллионы клиентов, например в интернете вещей (IoT) или в мобильном приложении с миллионами установленных экземпляров, обменивающихся секретной информацией. Например, промышленное предприятие с сотнями тысяч устройств интернета вещей может создать особый сертификат клиента для каждого устройства, затем установить лимит для соединений с этими устройствами, настроив Pulsar на прием только тех соединений, в которых клиент предоставляет корректный сертификат, подписанный сервисом авторизации компании.

В мобильном приложении, где необходимо защитить секретные данные ваших клиентов, например информацию о кредитной карте, от похищения злоумышленником, получившим несанкционированный доступ к приложению, можно создать особый сертификат для каждого установленного экземпляра и использовать эти сертификаты для проверки, подтверждающей, что запрос пришел от легальной версии мобильного приложения, а не от взломанной версии.

6.2.2. Аутентификация JSON Web Token (JWT)

Pulsar поддерживает аутентификацию клиентов с использованием токенов безопасности (или маркеров доступа – security tokens) на основе JWT – стандартизированного формата, применяемого для создания токенов доступа в формате JSON, подразумевающих одно или несколько заявлений. Заявления (claims) – это фактические утверждения. Во всех JWT можно обнаружить два заявления: iss (issuer – эмитент, автор запроса), идентифицирующее сторону, сформировавшую JWT, и sub, идентифицирующее субъект или сторону, о которой JWT передает информацию.

В Pulsar JWT-токены применяются для аутентификации клиента Pulsar и для связывания его с некоторой ролью, которая затем будет использоваться для определения авторизации клиента на выполнение конкретных действий, например на публикацию или потребление из указанной темы. Обычно администратор кластера Pulsar должен генерировать JWT-токены, содержащие заявление sub, связанное с конкретной ролью (например, администратор, который идентифицирует владельца конкретного токена). Затем администратор должен предоставить JWT-токен клиенту по защищенному каналу обмена данными, после чего клиент может воспользоваться этим токеном для аутентификации в Pulsar и связывания с ролью администратора.

Стандарт JWT определяет структуру токена, состоящую из трех частей: заголовка (header), содержащего основную информацию; полезной нагрузки (payload), содержащей заявления; и подписи (signature), которую можно использовать для проверки подлинности токена. Данные в каждой из этих трех частей зашифровываются

ся отдельно, после чего объединяются с использованием точек для разделения полей. Полученные строки выглядят приблизительно так, как показано на рис. 6.1. Зашифрованный токен можно передавать даже по протоколу HTTP.

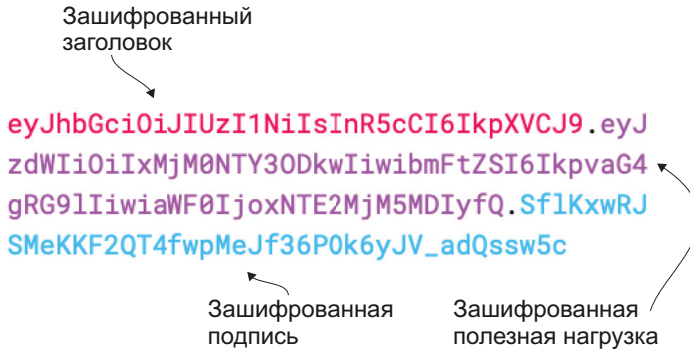


Рис. 6.1. Зашифрованный веб-токен JSON (JWT) и его составные части

Поскольку JWT-токены основаны на открытом стандарте, содержимое любого токена могут прочитать все получившие его в свое распоряжение. Это означает, что любой может попытаться получить доступ в вашу систему с помощью самодельного JWT или измененного перехваченного JWT, в котором отредактировано одно или несколько заявлений для имитации легального пользователя. Здесь начинает действовать подпись, поскольку она предоставляет средство для подтверждения аутентичности всего JWT-токена в целом.

Подпись вычисляется по зашифрованной заголовку и полезной нагрузке с использованием кодирования Base64 URL и объединения этих двух строк с сохранением секретности. Затем полученная строка пропускается через алгоритм криптографического хеширования, определенный в заголовке. Для JWT можно использовать две криптографические схемы: первая обозначается как схема с совместно используемым секретным ключом, в которой участвует сторона, генерирующая подпись, и проверяющая сторона, которая обязательно должна знать секретный ключ. Поскольку обе стороны владеют секретным ключом, каждая может проверить аутентичность JWT, самостоятельно вычисляя подпись и проверяя ее совпадение с подписью в JWT. Любое различие свидетельствует о подделке токена. Эта схема удобна, если требуется, чтобы обе стороны могли обмениваться информацией с обеспечением защиты передаваемых данных. Во второй схеме используется асимметричная пара ключей – открытый и секретный, когда сторона с открытым ключом может проверить аутентичность любого принятого JWT, а сторона с секретным ключом может генерировать надежные JWT-токены.

Существует несколько доступных онлайн-инструментов, позволяющих зашифровывать и расшифровывать JWT-токены, и на рис. 6.2 показано поэлементное сравнение выходных данных одного такого инструмента, использованного для расшифрования двух различных JWT. Слева приведены данные токена, использующего схему с совместно используемым секретным ключом. Здесь можно видеть, что содержимое токена легко можно прочитать с помощью этого инструмента без каких-либо дополнительных идентификационных сведений. Но для проверки подлинности подписи токена требуется секретный ключ, позволяющий узнать, не была ли подделана эта подпись.

JWT с совместно используемым секретным ключом

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": "admin",
  "name": "John Doe",
  "iss": "pulsar-broker.example.com",
  "iat": 1516239022
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256-bit-secret
) ☐ secret base64 encoded
```

Требуется секретный ключ для проверки подлинности подписи

JWT с открытым/секретным ключами

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "RS256"
}
```

PAYLOAD: DATA

```
{
  "sub": "admin",
  "exp": 1611083628
}
```

VERIFY SIGNATURE

```
RSASHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  Public Key or Certificate. Enter it in plain text only if you want to verify a token
),
Private Key. Enter it in plain text only if you want to generate a new token. The key never leaves your browser.
```

Требуется открытый ключ для проверки подлинности подписи

Требуется секретный ключ для генерации токена

Рис. 6.2. Методы верификации для JWT-токенов, подписанных с использованием схемы с совместно используемым секретным ключом и схемы с парой открытый/секретный ключ

Выходные данные, показанные справа на рис. 6.2, получены из JWT, использующего асимметричную схему, содержимое которого также можно легко прочитать без каких-либо дополнительных идентификационных сведений. Но для проверки подлинности

подписи токена требуется открытый ключ. Рассматриваемый инструмент также требует секретный ключ, только если вы хотите использовать его для генерации нового JWT. Надеюсь, это сравнение помогает лучше понять различия между двумя схемами, а также почему для конфигурирования средств аутентификации JWT для брокеров и клиентов Pulsar требуется JWT-токен и связанный с ним ключ.

Наконец, следует отметить, что каждый владелец корректного JWT может использовать его для аутентификации в Pulsar. Из этого следует, что необходимо принять соответствующие превентивные меры по предотвращению попадания сгенерированных JWT в чужие руки. К таким мерам относится передача JWT-токенов только через соединения, зашифрованные по протоколу TLS, и хранение содержимого JWT в защищенной локации на компьютере клиента. Чтобы разрешить аутентификацию JWT, в *Dockerfile* из предыдущего раздела добавляется команда, выполняющая еще один скрипт с именем *gen-tokens.sh* для генерации JWT, который можно использовать для аутентификации, как показано в листинге 6.15.

Листинг 6.15. Обновленное содержимое Dockerfile

```
FROM apache/pulsar/pulsar-standalone:latest
ENV PULSAR_HOME=/pulsar

...

RUN ["/bin/bash", "-c",
    "/pulsar/manning/security/authentication/jwt/gen-tokens.sh"]
```

❶ Содержимое, показанное в листинге 6.9.

❷ Выполняется указанный скрипт для генерации нескольких JWT-токенов для аутентификации на основе ролей.

Рассмотрим подробнее скрипт *gen-tokens.sh*, показанный в листинге 6.16, чтобы точно узнать, какие действия требуются для генерации JWT-токенов, которые можно использовать для аутентификации в брокере Pulsar.

Листинг 6.16. Содержимое файла gen-tokens.sh

```
#!/bin/bash

# Создание пары ключей: открытый/секретный.
/pulsar/bin/pulsar tokens create-key-pair \
    --output-private-key \
        /pulsar/manning/security/authentication/jwt/my-private.key \
    --output-public-key \
        /pulsar/manning/security/authentication/jwt/my-public.key
```

```
# Создание токена для роли администратора.  
/pulsar/bin/pulsar tokens create --expiry-time 1y \  
  --private-key \  
    file:///pulsar/manning/security/authentication/jwt/my-private.key \  
  --subject admin > /pulsar/manning/security/authentication/jwt/admin-token.txt  
...
```

- ❶ Будет использоваться асимметричная схема шифрования, поэтому необходимо создать пару ключей.
- ❷ Создание токена и присваивание ему полномочий администратора.
- ❸ Повторение процесса создания токена для дополнительных ролей.

После выполнения этого скрипта существует несколько JWT-токенов, сгенерированных и сохраненных как текстовые файлы в брокере Pulsar. Токены вместе с открытым ключом должны распределяться между клиентами Pulsar, чтобы они могли использовать их для аутентификации в экземпляре `pulsar-standalone`. Свойства, показанные в листинге 6.17, также необходимо изменить в файле `standalone.conf` для разрешения аутентификации на основе JWT.

Листинг 6.17. Изменение свойств аутентификации JWT в файле `standalone.conf`

```
#### Аутентификация JWT. #####  
authenticationEnabled=true  
authorizationEnabled=true  
authenticationProviders=org.apache.pulsar.broker.authentication.  
➔ AuthenticationProviderTls,org.apache.pulsar.broker.authentication.  
➔ AuthenticationProviderToken  
tokenPublicKey=file:///pulsar/manning/security/authentication/jwt/  
➔ my-public.key
```

- ❶ Это обязательное требование для любого механизма аутентификации в Pulsar.
- ❷ Разрешение авторизации.
- ❸ Добавление провайдера JWT-токенов в список провайдеров аутентификации.
- ❹ Локация файла открытого ключа из JWT-пары ключей, используемой для проверки подлинности подписи.

Если необходимо использовать аутентификацию JWT, то потребуется внести изменения, показанные в листинге 6.18, в файл `client.conf`. Поскольку ранее мы разрешили аутентификацию TLS для клиента, сейчас нужно закомментировать эти свойства, потому что клиент может одновременно использовать только один метод аутентификации.

Листинг 6.18. Изменение свойств аутентификации JWT в файле `client.conf`

```
#### Аутентификация JWT. ####
authPlugin=org.apache.pulsar.client.impl.auth.AuthenticationToken
authParams=file:///pulsar/manning/security/authentication/jwt/admin-token.txt

#### Аутентификация TLS. ####
#authPlugin=org.apache.pulsar.client.impl.auth.AuthenticationTls
#authParams=tlsCertFile:/pulsar/manning/security/authentication/tls/
➔ admin.cert.pem,tlsKeyFile:/pulsar/manning/security/authentication/
➔ tls/admin-pk8.pem
```

1 Закомментированы оба свойства аутентификации TLS.

Теперь заново создадим образ, чтобы включить в него изменения, необходимые для генерации JWT-токенов и разрешения аутентификации на основе JWT, выполнив команды, показанные в листинге 6.19. Перед выполнением этих команд не забудьте остановить и удалить все работающие экземпляры образа Pulsar.

Листинг 6.19. Создание образа Docker из Dockerfile

```
$ cd $REPO_HOME/pulsar-in-action/docker-images/pulsar-standalone
$ docker build . -t pia/pulsar-standalone-secure:latest
Sending build context to Docker daemon 3.345MB
Step 1/9 : FROM apache/pulsar/pulsar-standalone:latest
--> 3ed9bffff717
...
Step 8/9 : RUN ["/bin/bash", "-c", "/pulsar/manning/security/authentication/jwt/
➔ gen-tokens.sh"]
Step 9/9 : CMD ["/pulsar/bin/pulsar", "standalone"]
--> Using cache
--> a229c8eed874
Successfully built a229c8eed874
Successfully tagged pia/pulsar-standalone-secure:latest
```

- 1** Переход в каталог, содержащий *Dockerfile*.
- 2** Команда для создания и пометки образа Docker из *Dockerfile*.
- 3** Обратите внимание: теперь здесь девять шагов вместо восьми.
- 4** Выполнение скрипта *gen-tokens.sh*.
- 5** Сообщение об успешном завершении процесса, включающее тег (метку), используемый для ссылки на этот образ.

Далее необходимо снова выполнить шаги, показанные в листинге 6.7, чтобы запустить новый контейнер на основе обновленного образа Docker, защищенную командную оболочку *ssh* внутри контейнера и выполнить скрипт *publish-credentials.sh* внутри вновь созданного сеанса командной оболочки *bash*, чтобы сделать JWT-токены доступными в каталоге `${HOME}/exchange` на локальном компьютере. После этого мы попытаемся использовать аутентификацию JWT, выполнив

программу на языке Java, показанную в листинге 6.20. Программа также доступна в репозитории GitHub, связанном с этой книгой.

Листинг 6.20. Аутентификация JWT

```
import org.apache.pulsar.client.api.*;

public class JwtAuthClient {
    public static void main(String[] args) throws PulsarClientException {
        final String HOME = "/Users/david/exchange";
        final String TOPIC = "persistent://public/default/test-topic";

        PulsarClient client = PulsarClient.builder()
            .serviceUrl("pulsar+ssl://localhost:6651/")
            .tlsTrustCertsFilePath(HOME + "/ca.cert.pem")
            .authentication(
                AuthenticationFactory.token(() -> {
                    try {
                        return new String(Files.readAllBytes(
                            Paths.get(HOME + "/admin-token.txt")));
                    } catch (IOException e) {
                        return "";
                    }
                })
            ).build();

        Producer<byte[]> producer =
            client.newProducer().topic(TOPIC).create();

        for (int idx = 0; idx < 100; idx++) {
            producer.send("Hello JWT Auth".getBytes());
        }
        System.exit(0);
    }
}
```

На этом завершается процесс конфигурирования аутентификации для брокера Pulsar. С этого момента все клиенты Pulsar обязаны представить корректные идентификационные данные для выполнения аутентификации. В настоящее время реализованы только два из четырех механизмов аутентификации, поддерживаемых Pulsar, – TLS и JWT. Другие методы аутентификации можно активизировать, выполняя действия, подробно описанные в онлайн-документации Pulsar.

6.3. Авторизация

Процесс авторизации начинается только после того, как клиент успешно завершил процедуру аутентификации, выполненную сконфигурированным для Pulsar провайдером аутентификации. Авторизация определяет, обладает ли пользователь достаточными правами для доступа к некоторой конкретной теме. Если разрешена

только аутентификация, то любой пользователь, прошедший эту процедуру, получает право на выполнение любого действия в любом пространстве имен или теме в кластере.

6.3.1. Роли

В Pulsar роли используются для определения набора привилегий, например прав доступа к конкретной группе тем, связанных с группой пользователей. Apache Pulsar использует конфигурируемые провайдеры аутентификации для установления идентичности клиента, а затем присваивает ему токен (маркер) роли (role token). После этого токены роли используются механизмом авторизации, чтобы определить, какие действия разрешено выполнять клиентам.

Токены роли аналогичны физическим ключам, которыми открывают замки. Многие люди могут иметь копию ключа, и замку безразлично, кто его открывает, если это делается правильным ключом. В Pulsar иерархия ролей делится на три категории, и каждая категория обладает собственными функциональными возможностями, свойствами и правилами предполагаемого использования. На рис. 6.3 можно видеть, что иерархия ролей достаточно точно отображает иерархию Pulsar кластер / абонент / пространство имен, т. е. сводится к структурированным данным. Рассмотрим каждую роль более подробно.



Рис. 6.3. Иерархия ролей Pulsar и соответствующие им задачи администрирования

Суперпользователи

По имени можно понять, что суперпользователи могут выполнять любые действия на любом уровне кластера Pulsar. Но наилучшей практической методикой считается делегирование задач администрирования абонентов и нижележащих пространств имен другому человеку, который будет отвечать за управление стратегиями администрирования конкретного абонента (tenant). Таким образом, основной деятельностью суперпользователей является создание администраторов абонентов, и эту задачу можно решить с помощью инструмента командной строки `pulsar-admin`, выполнив команду, аналогичную показанной в листинге 6.21.

Листинг 6.21. Создание абонента Pulsar

```
$PULSAR_HOME/bin/pulsar-admin tenants create tenant-name \ ❶  
--admin-roles tenant-admin-role \ ❷  
--allowed-clusters standalone ❸
```

- ❶ Определение имени абонента.
- ❷ Определение имени роли администратора для этого абонента.
- ❸ Определение кластеров, в которых будет существовать этот абонент.

Это позволяет суперпользователю сосредоточиться на задачах администрирования кластера в целом, таких как создание кластера, управление брокерами и конфигурирование квот ресурсов уровня абонента, обеспечивая равномерное распределение ресурсов кластера между всеми абонентами. Обычно для администрирования абонента назначается пользователь, отвечающий за проектирование схемы размещения тем, требуемых для конкретного абонента, например начальник отдела, менеджер проекта или руководитель группы сопровождения приложения.

Администраторы абонента

Администратор абонента главным образом сосредоточен на проектировании и реализации пространств имен внутри своего абонента. Как было отмечено ранее, пространства имен – это не что иное, как логическая группа тем, для которой определены одинаковые требования стратегии. Стратегии уровня пространства имен применяются к каждой теме в конкретном пространстве имен и позволяют конфигурировать такие характеристики, как стратегии сохранения данных, квоты журналов регистрации и выгрузка данных в многоуровневое хранилище.

Клиенты

Роль клиента может быть присвоена любому приложению или сервису, которому предоставлены права на потребление или производство сообщений в темах кластера Pulsar. Права предоставля-

ются администраторами абонента либо на уровне пространства имен, т. е. применяются для всех тем в конкретном пространстве имен, либо для каждой отдельной темы. В листинге 6.22 (см. ниже в подразделе 6.3.2) показан пример обоих сценариев.

6.3.2. Пример сценария

Предположим, что вы работаете в компании GottaEat, занимающейся доставкой продуктов питания, позволяющей клиентам заказывать продукты из различных сотрудничающих ресторанов и обеспечивающей прямую доставку заказчику. Вместо найма собственного штата водителей компания решила привлечь частных лиц для доставки продуктов, чтобы уменьшить издержки. Компания будет использовать три отдельных мобильных приложения для управления бизнесом: доступное для всех клиентов, с ограниченным доступом для всех сотрудничающих владельцев ресторанов и с ограниченным доступом для всех авторизованных водителей, работающих на доставке.

Вариант использования для размещения заказа

Рассмотрим подробно самый простой вариант использования для компании – от размещения заказа клиентом до назначения задачи доставки водителю – с применением приложения на основе микросервисов, использующего Apache Pulsar для обмена данными в сообщениях. Сосредоточимся на том, как можно структурировать пространства имен внутри абонентов `restaurants` и `driver`, а также на предоставлении необходимых прав доступа. Абонент `microservices` будет использоваться несколькими абонентами, поэтому он создается и управляется группой сопровождения приложения. На рис. 6.4 показан общий поток сообщений для варианта использования при вводе заказа.

В этом варианте использования клиент использует мобильное приложение компании для выбора некоторых наименований продуктов, ввода адреса доставки, предоставления информации об оплате и подтверждения запроса. Информация о заказе публикуется в теме `persistent://microservices/orders/new` мобильным приложением.

Микросервис проверки оплаты потребляет входящий заказ и обменивается данными с системой оплаты кредитными картами для обеспечения платежа за заказанные продукты перед размещением заказа в теме `persistent://microservices/orders/paid`, чтобы сообщить, что по этому заказу оплата выполнена. Сервис также заменяет информацию о кредитной карте на внутренний идентификатор отслеживания, который в дальнейшем используется для приема платежа.

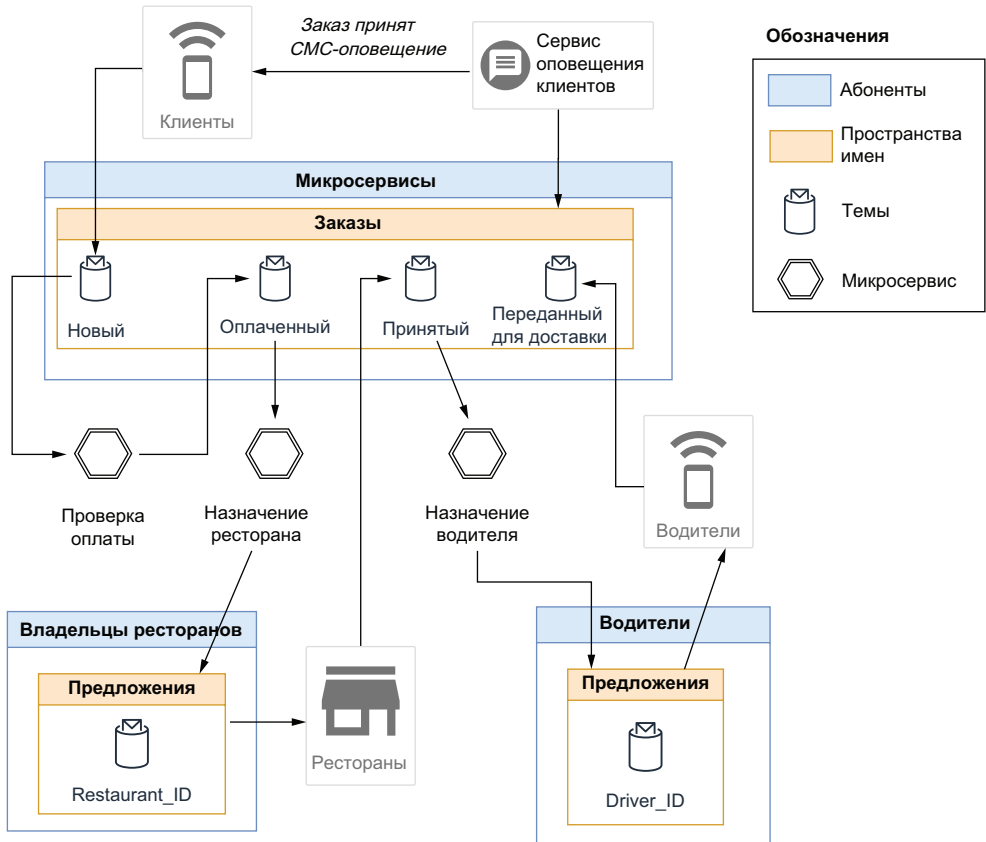


Рис. 6.4. Поток сообщений для варианта использования при размещении заказа клиентом

Микросервис назначения ресторана отвечает за поиск ресторана, который может выполнить заказ, потребляя сообщения из темы `persistent://microservices/orders/paid` и определяя подмножество кандидатов на основе списка заказанных продуктов и адреса доставки. Затем он поочередно отправляет предложение в каждый ресторан-кандидат, публикуя подробности заказа в специализированной для отдельного ресторана теме, к которой только он имеет право доступа (например, `persistent://restaurateurs/offers/restaurant_4827`), чтобы обеспечить выполнение заказа только одним рестораном. Если предложение отвергается или на него нет ответа в течение определенного интервала времени, то оно аннулируется и направляется в следующий по списку ресторан до тех пор, пока предложение не будет принято. После того как ресторан принял заказ через свое мобильное приложение, сообщение, содержащее подробности о ресторане, включая его расположение, публикуется в теме `persistent://microservices/orders/accepted`.

Как только заказ принят для выполнения, микросервис назначения водителя потребляет сообщение из темы `accepted` и пытается найти водителя, который может доставить заказ клиенту. Микросервис составляет список потенциальных кандидатов на основе их местоположения, запланированных маршрутов и т. п. и публикует сообщение в теме, доступ к которой разрешен только конкретному водителю (например, `persistent://drivers/offers/driver_5063`), чтобы обеспечить назначение выполнения заказа только одному водителю. Если предложение отвергается или на него нет ответа в течение определенного интервала времени, то оно аннулируется и направляется следующему по списку водителю до тех пор, пока предложение не будет принято. Если водитель соглашается на доставку, то сообщение, содержащее подробности об этом водителе, публикуется в теме `persistent://microservices/orders/dispatched`, оповещая о том, что доставка текущего заказа запланирована.

Все сообщения, хранящиеся в темах пространства имен `persistent://microservices/orders`, содержат два элемента данных `customer_id` и `order_id`. Сервис оповещения клиентов подписан на все вышеперечисленные темы и при приеме новой записи (сообщения) в любой из этих тем выполняет синтаксический анализ (парсинг) полей идентификаторов и использует полученную информацию для отправки оповещения в мобильное приложение соответствующего клиента, чтобы он мог отслеживать выполнение своего заказа.

Создание абонента

Самой первой задачей суперпользователя Pulsar должно быть создание всех трех абонентов и соответствующих ролей их администраторов. Рассмотрим шаги, требуемые для создания абонентов, необходимых для этого варианта использования, и показанные в листинге 6.22.

Листинг 6.22. Создание абонентов Pulsar

```
$PULSAR_HOME/bin/pulsar-admin tenants create microservices \  
--admin-roles microservices-admin-role  
  
$PULSAR_HOME/bin/pulsar-admin tenants create drivers \  
--admin-roles drivers-admin-role  
  
$PULSAR_HOME/bin/pulsar-admin tenants create restaurateurs \  
--admin-roles restaurateurs-admin-role
```

Поскольку мы делегируем функции администрирования этих абонентов кому-то другому в нашей организации, я кратко опишу характеристики лиц, которые должны заниматься этим, отдельно по каждому абоненту:

- так как абонент микросервисов предназначен для обеспечения совместного использования информации несколькими сервисами и приложениями, явным кандидатом на пост администратора должен быть профессионал в области информационных технологий, ответственный за архитектурные решения во всех отделах, например архитектор корпоративных приложений;
- для абонента владельцев ресторанов лучше всего подходит лицо (лица), занимающееся привлечением новых владельцев, т. е. он (она) будет отвечать за переговоры и соглашения, аттестацию и управление партнерами. В обязанности также может входить создание закрытых тем для отдельных ресторанов как часть процесса интеграции новых партнеров и т. п.;
- аналогично кандидатом в администраторы темы для водителей должно быть лицо (лица), отвечающее за привлечение новых водителей.

Стратегии авторизации

На основе требований для этого варианта использования необходимо тщательно продумать общее проектное решение обеспечения безопасности, аналогичное показанному на рис. 6.5, где определены следующие требования к подсистеме обеспечения безопасности:

- приложение, требующее предоставления конкретных прав доступа;
- метод аутентификации, который будет использоваться;
- темы и пространства имен, требующие предоставления конкретных прав доступа и определения способа предоставления доступа;
- идентификатор роли `roleID`, который будет присваиваться этим правам доступа.

Например, нам известно, что микросервис назначения водителей будет использовать JWT для аутентификации в Pulsar, и ему присваивается токен роли `DA_service`, предоставляющий право потребления в теме `persistent://microservices/orders/accepted` и право производства во всех темах пространства имен `persistent://drivers/offers`. Таким образом, необходимо выполнить команды, показанные в листинге 6.23, чтобы установить стратегии обеспечения безопасности для роли `DA_service`, и аналогичные команды также потребуются для всех остальных сервисов. Следует отметить, что даже при возможности существования нескольких работающих экземпляров сервиса назначения водителей все они могут использовать один и тот же идентификатор роли `RoleID`. Но это неприемлемо для некоторых мобильных приложений, так как мы должны точно знать, какой именно пользователь получает доступ к системе, чтобы применить соответствующие стратегии обеспечения безопасности.

Приложение	Аутентификация	RoleID	Стратегии авторизации
 Микросервис назначения водителей	 JWT	DA_service	microservices/orders/accepted (потребление) drivers/offers/driver_* (публикация)
 Микросервис назначения ресторана	 JWT	RA_service	microservices/orders/paid (потребление) restaurants/offers/restaurant_* (публикация)
 Микросервис проверки платежей	 JWT	PV_service	microservices/orders/new (потребление) microservices/orders/paid (производство)
 Микросервис оповещения клиентов	 JWT	CN_service	microservices/orders/* (потребление)
 Мобильное приложение клиентов	 Сертификат TLS	Customer	microservices/orders/new (публикация)
 Мобильное приложение водителей	 Сертификат TLS	Driver_<id>	drivers/offers/driver_ID (потребление) microservices/orders/dispatched (производство)
 Мобильное приложение ресторанов	 Сертификат TLS	Restaurant_<id>	microservices/orders/accepted (производство) restaurants/offers/restaurant_ID (потребление)

Рис. 6.5. Требования к подсистеме обеспечения безопасности на уровне приложения для варианта использования размещения заказов

Листинг 6.23. Активизация подсистемы обеспечения безопасности для микросервиса назначения водителей

- ```
$PULSAR_HOME/bin/pulsar-admin namespaces create microservice/orders 1
$PULSAR_HOME/bin/pulsar tokens create \ 2
 --private-key file:///path/to/private-secret.key \
 --subject DA_service > DA_service-token.txt 3
$PULSAR_HOME/bin/pulsar-admin namespaces grant-permission \ 4
 drivers/offers --actions produce --role DA_service
$PULSAR_HOME/bin/pulsar-admin topics grant-permission \
 persistent://microservices/orders/accepted \
 --actions produce \
 --role DA_service 5
```
- 1 Создание пространства имен `microservices/orders`. Это нужно сделать только один раз.
  - 2 Создание JWT для роли `DA_service`.
  - 3 Сохранение токена в текстовом файле для совместного использования группой развертывания сервиса.
  - 4 Предоставление роли `DA_service` права на публикацию в любой теме пространства имен `drivers/offers`.

### 5 Предоставление роли `DA_service` права на потребление из одной указанной темы<sup>1</sup>.

JWT-токен должен быть сохранен в файле и совместно использоваться группой, которая будет разворачивать сервис назначения водителей, чтобы сделать токен доступным во время выполнения либо посредством его связывания с этим сервисом, либо обеспечив его размещение в защищенной локации, например в объекте Kubernetes Secrets. В отличие от этого мобильное приложение будет использовать специализированные сертификаты клиентов, генерируемые после того, как водитель или владелец ресторана успешно присоединится к системе и получит неповторяющийся идентификатор, а сгенерированный сертификат клиента будет связан с этим конкретным идентификатором для однозначного определения пользователя при установлении соединения (например, идентификатор `driver_4950` должен получить сертификат клиента, связанный с общим именем `driver_4950`, и ему предоставляется токен для соответствующей роли). Это можно наблюдать в листинге 6.24, где показаны последовательные шаги, выполняемые ролью `driver-admin-role` для добавления нового водителя в кластер Pulsar.

```
openssl req -config file:///path/to/openssl.cnf \
 -key file:///path/to/driver-4950.key.pem -new -sha256 \
 -out file:///path/to/driver-4950.csr.pem \
 -subj "/C=US/ST=CA/L=Palo Alto/O=gottaeat.com/CN=driver_4950" \
 -passin pass:${CLIENT_PASSWORD}

openssl ca -config file:///path/to/openssl.cnf \
 -extensions usr_cert \
 -days 100 -notext -md sha256 -batch \
 -in file:///path/to/driver-4950.csr.pem \
 -out file:///path/to/driver-4950.cert.pem \
 -passin pass:${CA_PASSWORD}

$PULSAR_HOME/bin/pulsar-admin topics grant-permission \
 persistent://drivers/offers/driver_4950 \
 --actions consume \
 --role driver_4950

$PULSAR_HOME/bin/pulsar-admin topics grant-permission \
 persistent://microservices/orders/dispatched \
 --actions produce \
 --role driver_4950
```

### 1 Использование предоставленного водителем секретного ключа для генерации запроса сертификата.

<sup>1</sup> Но в листинге задан ключ `--actions produce`. Вероятно, вместо него должен быть записан ключ `--actions consume`. — *Прим. перев.*

- 2 Использование поля общего имени CN для определения нового идентификатора роли `roleID` для этого водителя.
- 3 Использование CSR для генерации TLS-сертификата клиента для нового водителя.
- 4 Имя файла сертификата клиента, которое будет связано с загружаемым экземпляром мобильного приложения.
- 5 Предоставление роли `driver_4950` права на потребление из специально назначенной ей темы.
- 6 Предоставление роли `driver_4950` права на производство (публикацию) в общей теме `dispatched`.

В листинге 6.24 можно видеть, что TLS-сертификат клиента генерируется специально для нового водителя на основе его идентификатора. Затем этот сертификат связывается с кодом мобильного приложения водителя для обеспечения постоянного использования при аутентификации в Pulsar при установлении соединения с кластером. После этого водителю передается ссылка, которую он может использовать для загрузки и установки экземпляра приложения на свой смартфон.

## 6.4. Шифрование сообщений

Тема новых заказов содержит секретную информацию о кредитных картах, поэтому не должна быть доступной каким-либо потребителям, кроме сервиса проверки платежей. Даже если сконфигурирован механизм TLS-шифрования на транспортном уровне для защиты информации во время передачи в кластер Pulsar, необходимо также обеспечить хранение данных в зашифрованном формате. К счастью, Pulsar предоставляет методики, позволяющие шифровать содержимое сообщений на стороне производителя перед отправкой их в брокер. Содержимое сообщений остается зашифрованным до тех пор, пока потребитель с корректным секретным ключом не расшифрует его для потребления.

Чтобы отправлять и принимать зашифрованные сообщения, в первую очередь необходимо создать асимметричную пару ключей. В образе Docker, используемый в примерах этой главы, включен скрипт `gen-rsa-key.sh`, который можно применять для генерации пары ключей открытый/секретный. В листинге 6.25 показано содержимое этого скрипта, размещенного в каталоге `/pulsar/manning/security/message-encryption`.

### Листинг 6.25. Содержимое скрипта `gen-rsa-key.sh`

```
Генерация секретного ключа.
openssl ecparam -name secp521r1 \
 -genkey \
 -param_enc explicit \
 -out /pulsar/manning/security/encryption/ecdsa_privkey.pem
```

```
Генерация открытого ключа.
openssl ec -pubout \
 -outform pem \
 -in /pulsar/manning/security/encryption/ecdsa_privkey.pem \
 -out /pulsar/manning/security/encryption/ecdsa_pubkey.pem
```

Открытый ключ нужно передать в приложение-производитель, которым в рассматриваемом здесь примере является мобильное приложение клиента. Секретный ключ должен использоваться только сервисом проверки платежей, так как он необходим для потребления зашифрованных сообщений, публикуемых в теме `persistent://microservices/orders/new`. В листинге 6.26 показан пример кода, использующего открытый ключ для зашифрования данных производителем сообщения перед его отправкой.

#### Листинг 6.26. Конфигурация производителя для зашифрования сообщений

```
String pubKeyFile = "path to ecdsa_pubkey.pem";
CryptoKeyReader cryptoReader
 = new RawFileKeyReader(pubKeyFile, null);

Producer<String> producer = client
 .newProducer(Schema.STRING)
 .cryptoKeyReader(cryptoReader)
 .cryptoFailureAction(ProducerCryptoFailureAction.FAIL)
 .addEncryptionKey("new-order-key")
 .topic("persistent://microservices/orders/new")
 .create();
```

- ❶ Клиент должен получить доступ к открытому ключу.
- ❷ Инициализация читателя-шифровальщика для использования открытого ключа.
- ❸ Конфигурирование производителя для использования открытого ключа для зашифрования сообщений с помощью `CryptoReader`.
- ❹ Оповещение производителя о генерации исключения, если сообщение невозможно зашифровать.
- ❺ Определение имени для ключа зашифрования.

В листинге 6.27 показан пример кода для конфигурирования потребителя внутри сервиса проверки платежей, чтобы воспользоваться секретным ключом для расшифрования сообщений, потребляемых из темы `persistent://microservices/orders/new`.

#### Листинг 6.27. Конфигурирование клиента Pulsar для чтения из темы с зашифрованными сообщениями

```
String privKeyFile = "path to ecdsa_privkey.pem";
CryptoKeyReader cryptoReader
 = new RawFileKeyReader(null, privKeyFile);
```

```

ConsumerBuilder<String> builder = client
 .newConsumer(Schema.STRING)
 .consumerName("payment-verification-service")
 .cryptoKeyReader(cryptoReader)
 .cryptoFailureAction(ConsumerCryptoFailureAction.DISCARD)
 .topic("persistent://microservices/orders/new")
 .subscriptionName("my-sub");

```

3  
4

- 1 Клиент должен получить доступ к закрытому ключу.
- 2 Инициализация читателя-шифровальщика для использования закрытого ключа.
- 3 Конфигурирование потребителя для использования закрытого ключа для расшифрования сообщений с помощью `CryptoReader`.
- 4 Оповещение потребителя об отбрасывании сообщения, если его невозможно расшифровать.

После того как мы рассмотрели шаги, требуемые для конфигурирования производителей и потребителей сообщений с поддержкой шифрования, я хотел бы более подробно описать, как все это реализовано внутри Pulsar, и показать некоторые основные проектные решения, о которых вы должны знать, чтобы лучше понять, как применить их в своих приложениях. На рис. 6.6 показаны шаги, выполняемые на стороне производителя при разрешенном шифровании сообщений. При вызове метода производителя `send` с исходными необработанными байтами сообщения производитель сначала выполняет внутреннее обращение в клиентскую библиотеку Pulsar, чтобы получить текущий симметричный AES-ключ шифрования. Ключ шифрования считается текущим, потому что такие ключи автоматически меняются по циклической схеме каждые четыре часа, чтобы ограничить воздействие неавторизованного доступа к ключу, т. е. скомпрометированный ключ влияет только на данные, находящиеся в четырехчасовом окне, а не на всю хронологию темы.

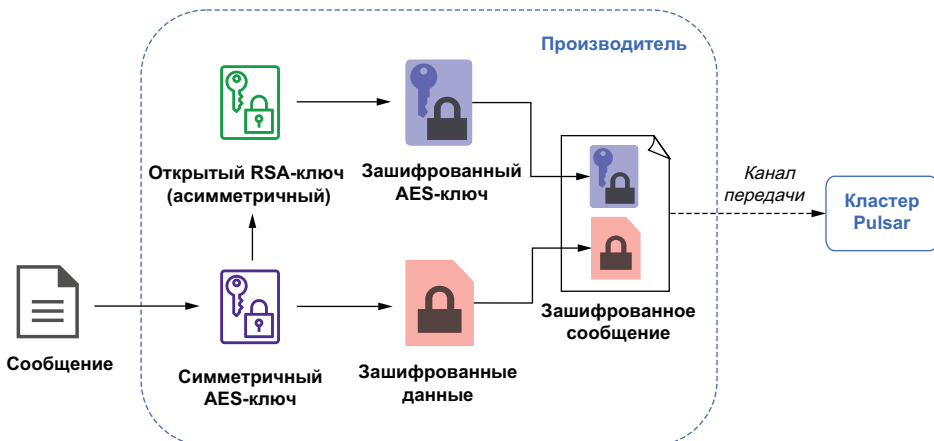


Рис. 6.6. Зашифрование сообщения в производителе Pulsar

Затем текущий AES-ключ используется для зашифрования исходных байтов сообщения, которые после этого размещаются в полезной нагрузке исходящего сообщения. Далее ранее сгенерированный открытый ключ из асимметричной пары RSA применяется для зашифрования самого AES-ключа, и полученный зашифрованный AES-ключ размещается в свойствах заголовка исходящего сообщения.

При зашифровании AES-ключа с использованием открытого ключа из асимметричной пары мы можем сохранять уверенность в том, что только потребитель с соответствующим секретным ключом из той же пары может расшифровать (следовательно, и прочесть) AES-ключ, который применялся для зашифрования полезной нагрузки сообщения. В момент отправки все данные внутри исходящего сообщения зашифрованы: полезная нагрузка с использованием AES-ключа, а заголовок, содержащий сам AES-ключ, с применением открытого RSA-ключа. Далее это сообщение передается по каналу связи в кластер Pulsar для доставки всем назначенным потребителям.

После приема зашифрованного сообщения потребитель выполняет шаги, показанные на рис. 6.7, для расшифрования сообщения и получения доступа к исходным данным. Сначала потребитель считывает заголовки сообщений и ищет в них свойство `encryption_key` для воссоздания AES-ключа. Теперь потребитель обладает AES-ключом и может расшифровать содержимое сообщения, так как это симметричный ключ, т. е. он используется как для зашифрования, так и для расшифрования данных.

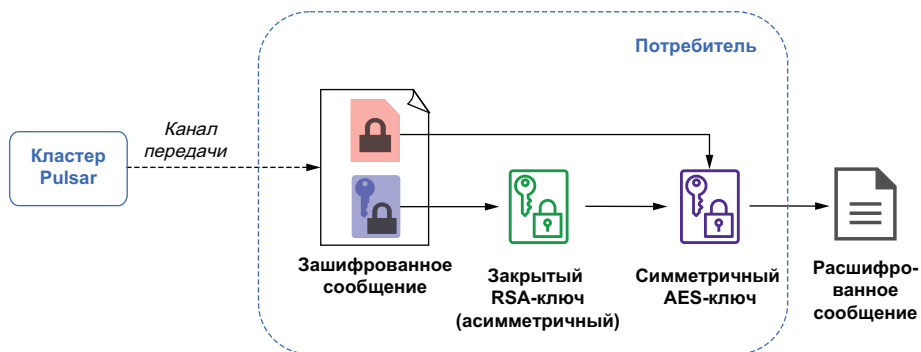


Рис. 6.7. Расшифрование сообщения в потребителе Pulsar

Выбор способа хранения AES-ключа, используемого для шифрования сообщения, вместе с зашифрованным содержимым сообщения освобождает Pulsar от необходимости хранить эти ключи и управлять ими внутри фреймворка. Но если вы потеряли или удалили закрытый RSA-ключ, применяемый для расшифрования AES-ключа, то зашифрованное сообщение потеряно безвозвратно, и Pulsar не сможет его расшифровать. Из этого следует, что наилучшим практическим подходом для хранения и управления пара-



ми RSA-ключей является сторонний сервис обслуживания ключей, например AWS KMS.

После того как я описал возможность потери RSA-ключа, у вас может возникнуть вопрос: что именно происходит в таком сценарии? Очевидно, что подобная ситуация крайне нежелательна, но она может возникнуть, поэтому следует узнать, как можно заставить ваше приложение реагировать и какие варианты существуют на сторонах производителя и потребителя.

Сначала рассмотрим сторону производителя. Здесь существуют только два реальных варианта. Во-первых, можно продолжить отправку сообщений без шифрования. Это может стать приемлемым решением, если ценность данных выше риска их хранения в незашифрованном формате в течение ограниченного интервала времени. Например, в описанном выше сценарии размещения заказов лучше позволить клиентам отправлять заказы, нежели прекратить всю деловую активность из-за потерянного RSA-ключа. Поскольку данные передаются по защищенному TLS-соединению, рис. минимален.

Другой вариант для производителя – определять сообщения как некорректные и генерировать исключение, что является приемлемым решением, если рис. раскрытия данных превышает потенциальную бизнес-ценность при своевременной доставке. Обычно такое решение характерно для некоторого типа приложений, не контактирующих с клиентами, например внутренняя система обработки платежей, для которой не имеется строгого соглашения о качестве обслуживания (SLA). Можно определить, какое действие должен выполнить производитель, используя метод `setCryptoFailureAction()` в его конфигурации, как было показано в листинге 6.26.

Если ошибка возникает при расшифровании на стороне потребителя, то в вашем распоряжении имеются три варианта. Во-первых, можно предпочесть доставку зашифрованного содержимого в приложение-потребитель. Но при этом ответственность за расшифрование принимает само это приложение. Этот вариант приемлем, если логика потребителя не основана на содержимом сообщений, например при перенаправлении сообщения следующим потребителям в цепочке на основе информации в заголовке этого сообщения.

Второй вариант – определение сообщений как некорректных, что заставляет брокер Pulsar повторно доставлять их. Этот вариант удобен при совместно используемой подписке, когда у одной подписки имеется несколько потребителей. При возникновении ошибки вы надеетесь, что проблема с расшифрованием локализуется только в текущем потребителе, например из-за потери закрытого ключа на хосте этого потребителя, и если сообщение повторно доставляется другому потребителю, то, возможно, он успешно расшифрует и обработает его. Но не следует использовать этот ва-

риант для подписки эксклюзивного типа, так как сообщения будут повторно доставляться тому же потребителю снова и снова, а он не сможет их обработать.

Последний вариант – заставить потребителя отбрасывать сообщения, что приемлемо для потребителя эксклюзивной подписки, которому необходим доступ к содержимому сообщения, чтобы выполнить его логику. Как было отмечено ранее, ошибка в сообщении приводит к его бесконечной повторной доставке, и потребитель «зависает», пытаясь повторно обрабатывать одно и то же сообщение. Такой сценарий может полностью остановить потребление сообщений в подписке и постоянно увеличивать размер журнала регистрации сообщений для всей темы в целом. Таким образом, отбрасывание сообщения является единственным способом предотвращения возникновения подобной ситуации, но по цене потери сообщения.

## 6.5. Резюме

- Pulsar поддерживает шифрование по протоколу TLS на транспортном уровне, гарантирующее, что все данные, передаваемые между клиентами и брокером Pulsar, зашифрованы.
- Pulsar поддерживает аутентификацию TLS с использованием сертификатов клиентов, которая позволяет распределять эти идентификационные данные только среди заслуживающих доверия клиентов и ограничить доступ к кластеру только для владельцев подлинных сертификатов клиентов.
- Pulsar позволяет использовать JSON web token (JWT-токены) для аутентификации пользователей и назначения им конкретно определенных ролей.
- После аутентификации пользователю выдается токен роли, используемый для определения ресурсов внутри кластера Pulsar, для чтения и записи которых авторизуется пользователь.
- Pulsar поддерживает шифрование на уровне сообщений для обеспечения защиты данных, хранящихся на локальном диске в букмекерах. Это предотвращает неавторизованный доступ к любым секретным данным, которые могут содержаться внутри сохраненных сообщений.



# *Реестр схем*

---

## **Темы:**

- использование схемы Pulsar для упрощения разработки микросервиса;
- понимание совместимости типов в различных схемах;
- использование класса `LocalRunner` для запуска и отладки своих функций в интегрированной среде разработки (IDE);
- развитие схемы без воздействия на существующих потребителей.

Обычные базы данных используют процесс, обозначенный термином «структурирование данных при записи» (schema-on-write), в котором все столбцы, строки и типы таблицы определяются до того, как любые данные можно будет записать в таблицу. Такой подход гарантирует, что данные соответствуют предварительно определенной спецификации, а клиенты-потребители могут получить доступ к информации о схеме непосредственно из самой базы данных, что позволяет им узнать основную структуру обрабатываемых записей.

Сообщения Apache Pulsar хранятся в виде массивов неструктурированных байтов, а структурирование применяется к ним только при чтении. Эта методика обозначается термином «структурирование данных при чтении» (schema-on-read) и впервые была введена (и стала широко известной) в базах данных Hadoop и NoSQL.

Хотя методика структурирования данных при чтении упрощает потребление и позволяет обрабатывать новые и динамические источники данных на лету, ей присущи некоторые недостатки, такие как отсутствие хранилища метаданных, к которым клиенты могут получить доступ, чтобы определить схему для темы Pulsar, где они выполняют операции потребления.

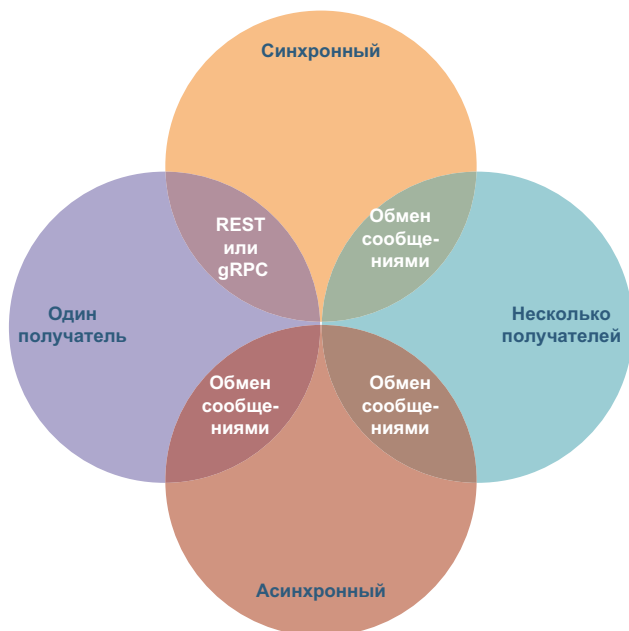
Клиенты Pulsar видят только поток отдельных записей, которые могут иметь любой тип, поэтому требуется эффективный способ, позволяющий определить, как интерпретировать каждую прибывающую запись. Именно здесь начинает действовать реестр схем Pulsar. Это весьма важный компонент стека технологий Pulsar, который отслеживает схемы всех тем в среде Pulsar.

В предыдущей главе мы видели, что группа разработки компании GottaEat, занимающейся доставкой продуктов питания, решила принять архитектурный стиль микросервисов с приложениями, состоящими из набора слабо связанных, независимо разработанных сервисов. В такой архитектуре от разнообразных микросервисов потребуется совместная обработка одних и тех же данных, а для этого они должны знать основную структуру события, включая поля и связанные с ними типы. Иначе потребители событий не смогут выполнять какие-либо осмысленные вычисления с данными событий. В этой главе я продемонстрирую, как реестр схем Pulsar можно использовать для весьма существенного упрощения совместного использования информации всеми группами приложений в GottaEat.

## **7.1. Обмен информацией между микросервисами**

При создании архитектуры микросервисов одной из задач, требующих решения, является обеспечение обмена данными между сервисами. Для этого существует несколько вариантов со своими достоинствами и недостатками. Обычно различные типы механизма обмена данными можно классифицировать по двум факторам. Первый фактор – является ли обмен данными между сервисами синхронным или асинхронным, второй – предназначен ли механизм обмена данными для одного или нескольких (многих) получателей, как показано на рис. 7.1.

Такие протоколы, как HTTP/REST или gRPC, чаще всего используются как механизмы синхронного обмена данными между сервисами на основе схемы запрос/ответ (request/response). При синхронном обмене данными один сервис отправляет запрос и ждет ответ от другого сервиса. Выполнение вызывающего сервиса блокируется и не может продолжить свою задачу до тех пор, пока не будет получен ответ. Этот паттерн обмена данными аналогичен вызовам процедур при программировании обычных приложений, где конкретный метод обращается к внешнему сервису с определенным набором параметров.



**Рис. 7.1.** Факторы обмена данными между микросервисами и соответствующие им протоколы

При асинхронном обмене данными сервис отправляет сообщение, но не обязан ждать ответ. Если необходим асинхронный механизм обмена данными на основе схемы публикация–подписка (publish/subscribe), то наилучшим вариантом выбора являются системы обмена сообщениями, такие как Pulsar. Системы обмена сообщениями также требуются для поддержки обмена данными по схеме публикация–подписка между одним сервисом и несколькими (многими) получателями сообщений. На практике нет необходимости блокировать сервис до тех пор, пока все назначенные получатели подтвердят прием сообщения, так как некоторые из них могут находиться в режиме офлайн. Гораздо лучшим решением является наличие системы, сохраняющей сообщения.

Эти факторы и механизмы обмена данными необходимо знать, чтобы ясно понимать, какие из них вы можете использовать, но это не самое главное при создании микросервисов. Важнее всего возможность интеграции микросервисов с сохранением их независимости, а для этого требуется установление контракта между совместно работающими сервисами вне зависимости от выбранного механизма обмена данными.

### 7.1.1. API микросервисов

Как и в любом проекте разработки программного обеспечения, точные требования помогают группе разработки создавать каче-

ственные программы, а предварительное формирование четко определенных контрактов сервисов позволяет разработчикам микросервиса писать код без необходимости каких-либо предположений о предоставляемых данных или об ожидаемых выходных данных из любого конкретного метода внутри создаваемого сервиса. В этом подразделе рассматривается поддержка контрактов сервисов каждым стилем обмена данными.

### Протоколы REST и gRPC

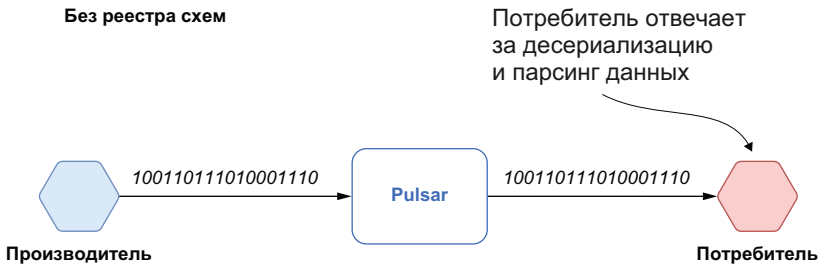
При подробном рассмотрении синхронных механизмов обмена данными между сервисами на основе схемы запрос/ответ обнаруживается одно из самых существенных различий между REST и gRPC – формат полезной нагрузки. Концептуальная модель, используемая gRPC, подразумевает наличие сервисов с четкими определениями интерфейса и структурированными сообщениями для запросов и ответов. Эта модель напрямую передает в язык программирования такие концепции, как интерфейсы, функции, методы и структуры данных. Кроме того, такой подход позволяет gRPC автоматически генерировать необходимые клиентские библиотеки, которые в дальнейшем могут совместно использоваться группами разработки микросервисов и работают как формальный контракт между ними.

Парадигма REST не определяет как обязательную какую-либо структуру, и полезная нагрузка сообщения обычно использует слабо типизированную систему сериализации данных, например JSON. Таким образом, интерфейсы API REST не имеют формального механизма для точного определения формата сообщений, передаваемых между сервисами. Данные передаются в обоих направлениях как необработанные байты, которые обязательно должны быть десериализованы принимающим сервисом (сервисами). Но и в таких сообщениях подразумевается некоторая структура, которой непременно должна соответствовать полезная нагрузка. Таким образом, любые изменения в полезной нагрузке обязательно должны согласовываться между группами, разрабатывающими сервис, чтобы любые изменения, внесенные одной группой, не оказывали воздействия на другие. Например, удаление поля, требуемого другими сервисами. Внесение подобного изменения может нарушить неформальный контракт между сервисами и создать проблемы для потребляющих сервисов, которые зависят от этого конкретного поля для выполнения своей логики обработки.

Такая проблема характерна не только для протокола REST, это побочный эффект в любом используемом протоколе обмена слабо типизированными данными. Следовательно, эта проблема существует в механизме обмена данными на основе сообщений, который использует систему сериализации слабо типизированных данных.

### Протокол обмена сообщениями

На рис. 7.1 можно видеть, что большинство паттернов обмена данными между сервисами могут поддерживаться только протоколом на основе обмена сообщениями. В Pulsar каждое сообщение состоит из двух отдельных частей: полезной нагрузки сообщения, хранящей необработанные байты, и набора определенных пользователем свойств, хранимых в виде пар ключ–значение. Хранение полезной нагрузки в виде необработанных байтов обеспечивает максимальную гибкость, но за счет того, что от каждого потребителя сообщения требуется преобразование этих байтов в формат, ожидаемый потребляющим приложением, как показано на рис. 7.2.



**Рис. 7.2.** Pulsar принимает необработанные байты как входные данные и доставляет те же необработанные байты как выходные данные

Для различных микросервисов потребуется обмен данными через сообщения, чтобы обеспечить эту методику, производитель и потребитель должны согласовать основную структуру сообщений, которыми они обмениваются, в том числе поля и связанные с ними типы. Метаданные, определяющие структуру сообщений, весьма часто называют схемами сообщений (message schemas). Схемы обеспечивают формальное определение способа преобразования необработанных байтов сообщения в более формальный тип структуры (например, способ отображения нулей и единиц, хранящихся в полезной нагрузке сообщения, в тип объекта языка программирования).

Схемы сообщений максимально близки к тому, что мы называем формальным контрактом между сервисами, генерирующими сообщения, и потребляющими сервисами. Удобно мысленно представлять схемы сообщений как прикладные программные интерфейсы (API). Приложения зависят от API и предполагают, что любые изменения в API сохраняют совместимость, чтобы приложения оставались работоспособными.

#### 7.1.2. Обоснование необходимости реестра схем

Реестр схем предоставляет централизованную локацию для хранения информации о схемах, применяемых в вашей организации, что, в свою очередь, существенно упрощает совместное использо-

вание этой информации всеми группами сопровождения приложений. Реестр служит единым правильным источником схем сообщений, применяемых всеми сервисами и группами разработки, способствуя улучшению качества их совместной работы. Наличие внешнего репозитория схем сообщений помогает ответить на следующие вопросы, касающиеся любой конкретной темы:

- если я потребляю сообщения, то как я должен выполнять синтаксический анализ и интерпретацию содержащихся в них данных?
- если я произвожу сообщения, то в каком предполагаемом формате?
- должны ли все сообщения иметь одинаковый формат, или схема изменилась?
- если схема изменилась, то какие измененные форматы сообщений применяются внутри темы?

Наличие централизованного реестра схем в сочетании с согласованным применением схем во всей организации существенно упрощает потребление и анализ данных. Если вы определяете стандартную схему для общего бизнес-объекта, которую будут использовать почти все приложения, например клиент, производитель или обработчик заказов, то все приложения, производящие сообщения, получают возможность генерировать сообщения в этом формате. А потребляющим приложениям не нужно будет выполнять какие-либо преобразования данных для приведения их в соответствие с некоторым другим форматом. С точки зрения анализа данных наличие структуры сообщений, четко определенной в реестре схем, позволяет аналитикам лучше понять структуру данных без лишних вопросов к группам разработки.

## 7.2. Реестр схем Pulsar

Реестр схем Pulsar позволяет производителям и потребителям сообщений согласовывать структуру данных конкретной темы непосредственно через брокер Pulsar без необходимости создания дополнительного уровня обслуживания метаданных. Другие системы обмена сообщениями, например Kafka, требуют наличия отдельного независимого компонента реестра схем.

### 7.2.1. Архитектура

По умолчанию Pulsar использует сервис таблиц Apache BookKeeper для хранения схем, поскольку он предоставляет надежное хранилище с возможностью репликации, гарантирующее, что данные схемы не будут потеряны. Сервис также предоставляет дополнительное преимущество в виде удобного API ключ–значение. Поскольку



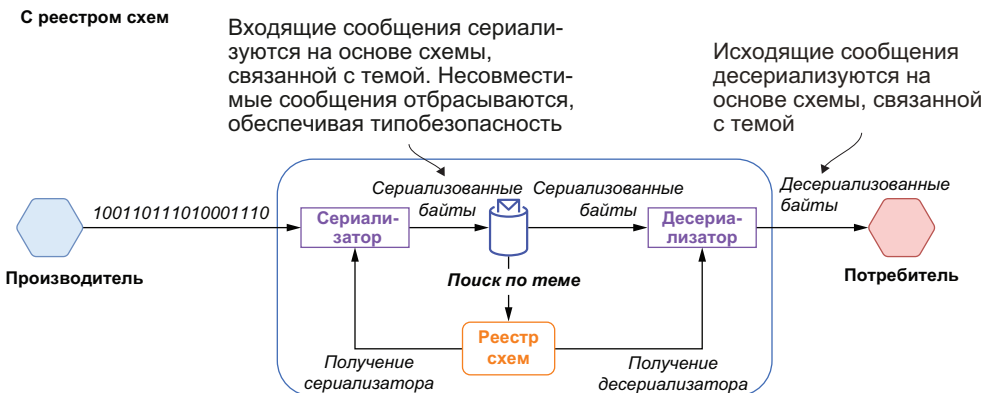
схемы Pulsar применяются на уровне темы, имя темы используется как ключ, а значения представлены структурой данных с именем SchemaInfo, состоящей из полей, показанных в листинге 7.1.

### Листинг 7.1. Пример структуры данных SchemaInfo Pulsar

```
{
 "name": "my-namespace/my-topic", ❶
 "type": "STRING", ❷
 "schema": "", ❸
 "properties": {} ❹
}
```

- ❶ Однозначно определенное имя, совпадающее с именем темы, с которой связана эта схема.
- ❷ Это должен быть один из предварительно определенных типов схемы, например **STRING** или **struct**, если вы используете обобщенную библиотеку сериализации, например Apache Avro или JSON.
- ❸ Если вы используете поддерживаемый тип сериализации, например Avro, то это поле будет содержать сырые данные схемы.
- ❹ Набор свойств, определенных пользователем.

На рис. 7.3 можно видеть, что клиент Pulsar пользуется реестром схем, чтобы получить схему, связанную с темой, с которой он установил соединение, и вызвать соответствующий сериализатор/десериализатор для преобразования байтов в требуемый тип. Это избавляет код потребителя от ответственности за выполнение такого преобразования и позволяет сосредоточиться на бизнес-логике.



**Рис. 7.3.** Реестр схем Pulsar используется для сериализации байтов до публикации в теме и для их десериализации перед доставкой потребителям

Реестр схем использует значения полей `type` и `schema`, чтобы определить, как выполнить сериализацию и десериализацию байтов, содержащихся в теле сообщения. Реестр схем Pulsar поддерживает разнообразные типы для схем, которые можно разделить

на простейшие (primitive) и сложные (complex) типы. Поле `type` используется для определения категории схемы темы.

### Простейшие типы

В настоящее время Pulsar предоставляет несколько простейших типов схем: `BOOLEAN`, `BYTES`, `FLOAT`, `STRING` и `TIMESTAMP`, а также некоторые другие. Если поле `type` содержит имя одного из этих предварительно определенных типов схемы, то байты сообщения будут автоматически сериализованы/десериализованы в соответствующие типы, используемые в языке программирования, как показано в табл. 7.1.

**Таблица 7.1.** Простейшие типы Pulsar

| Простейший тип Pulsar | Тип в языке программирования                                                             |
|-----------------------|------------------------------------------------------------------------------------------|
| BOOLEAN               | Одно из двоичных значений: 0 = false, 1 = true                                           |
| INT8                  | 8-битовое знаковое целое                                                                 |
| INT16                 | 16-битовое знаковое целое                                                                |
| INT32                 | 32-битовое знаковое целое                                                                |
| INT64                 | 64-битовое знаковое целое                                                                |
| FLOAT                 | 32-битовое число с плавающей точкой одинарной точности (IEEE 754)                        |
| DOUBLE                | 64-битовое число с плавающей точкой двойной точности (IEEE 754)                          |
| BYTES                 | Последовательность 8-битовых беззнаковых байтов                                          |
| STRING                | Последовательность символов Unicode                                                      |
| TIMESTAMP             | Число миллисекунд, отсчитываемое от даты 1 января 1970 г., хранящееся как значение INT64 |

Для простейших типов Pulsar не требует и не использует какие-либо данные в поле `schema`, так как схема уже подразумевается, следовательно, в ее определении нет необходимости.

### Сложные типы

Если для сообщений требуется более сложный тип, то необходимо воспользоваться одной из обобщенных библиотек сериализации, поддерживаемых Pulsar, например Avro, JSON или protobuf. Это требование должно быть обозначено пустой строкой в поле `type` объекта `SchemaInfo`, связанного с темой, а поле `schema` должно содержать строку определения схемы в формате JSON в кодировке UTF-8. Рассмотрим, как это применяется для определения схемы Apache Avro, чтобы наглядно продемонстрировать, насколько регистр схем упрощает процесс.

Apache Avro – это формат сериализации с поддержкой сложных типов данных через независимые от языка определения схем в формате JSON. Avro-данные сериализуются в компактный фор-

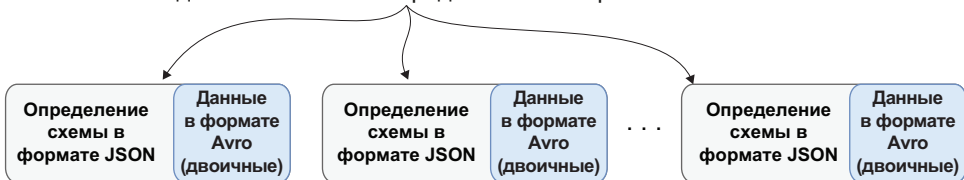
мат двоичных данных, прочитать который можно только с использованием схемы, примененной при записи. Поскольку Avro требует, чтобы читатели имели доступ к исходной схеме записи для получения возможности десериализации двоичных данных, соответствующая схема обычно сохраняется вместе с данными в начале файла. Изначально формат Avro был предназначен для хранения файлов с большим количеством записей одинакового типа, что позволяло сохранить схему один раз и многократно использовать ее при итерационном проходе по записям.

Обычный файл с данными  
в формате Avro



Сообщения с данными в формате Avro

Одна и та же схема передается многократно

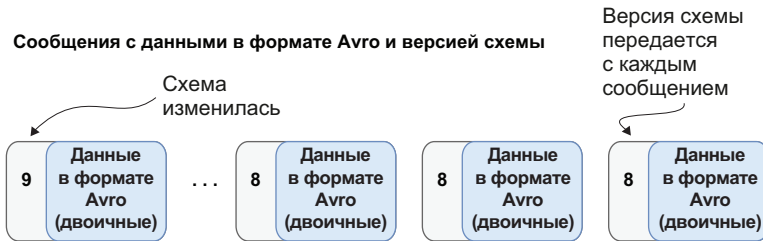


**Рис. 7.4.** Сообщения в формате Avro требуют, чтобы связанная с ними схема была включена в каждое сообщение для обеспечения возможности парсинга двоичного содержимого сообщения

Но при использовании системы обмена сообщениями схема должна передаваться с каждым отдельным сообщением, как показано на рис. 7.4. Включение схемы в каждое сообщение Pulsar, возможно, неэффективно с точки зрения рационального использования памяти, пропускной способности сети и дискового пространства. Но здесь начинает действовать реестр схем Pulsar. Если вы регистрируете типизированного производителя или потребителя, применяющего схему Avro, то представление этой схемы в формате JSON сохраняется в поле `schema` соответствующего объекта `SchemaInfo`. Затем исходные необработанные байты сообщений сериализуются или десериализуются на основе определения схемы Avro, хранящейся в реестре схем, как показано на рис. 7.3. Такой подход устраняет необходимость включения схемы в каждое отдельное сообщение. Кроме того, соответствующий сериализатор или десериализатор кешируется внутри производителей/потребителей и может применяться для всех последующих сообщений до тех пор, пока не встретится другая версия схемы.

### 7.2.2. Управление версиями схем

Каждый объект `SchemaInfo`, хранящийся в теме, имеет связанную с ним версию. Когда производитель публикует сообщение, используя конкретный объект `SchemaInfo`, это сообщение помечается соответствующим номером версии, как показано на рис. 7.5. Запись версии схемы в сообщение позволяет потребителю использовать номер версии для поиска в регистре схем, чтобы выполнить десериализацию сообщения.



**Рис. 7.5.** При использовании регистра схем сообщения в формате Avro должны содержать номер версии схемы, а не полное ее описание. Потребители используют версию схемы, чтобы извлечь правильный десериализатор по номеру версии

Это простой и эффективный способ связывания схемы Avro с каждым сообщением в формате Avro без необходимости присоединения полного описания схемы в формате JSON. Версии схем нумеруются в порядке возрастания (например, v1, v2, ... и т. д.), и, когда первый типизированный по схеме потребитель или производитель устанавливает соединение с темой, соответствующая схема помечается как версия 1. После загрузки исходной схемы брокеры предоставляют информацию о ней для тем, которые они обслуживают, и локально сохраняют ее копию для применения.

В системе обмена сообщениями, такой как Pulsar, сообщения могут сохраняться в течение неограниченного интервала времени. Поэтому некоторым потребителям потребуется обработка таких сообщений с применением более старых версий схемы. Следовательно, регистр схем сохраняет хронологическую последовательность изменяемых версий всех схем, используемых в Pulsar, для обслуживания потребителей подобных «старых» сообщений.

### 7.2.3. Совместимость схемы

Я начал эту главу с обсуждения важности сохранения совместимости сообщений, используемых микросервисами, даже при условии постоянного развития приложений и изменения требований к ним. В Pulsar каждому производителю и потребителю предоставлена свобода использования собственной схемы, так что вполне возможно, что тема может содержать сообщения, соответствующие

щие различным версиям схемы, как показано на рис. 7.5, где в теме существуют сообщения с версиями схемы 8 и 9.

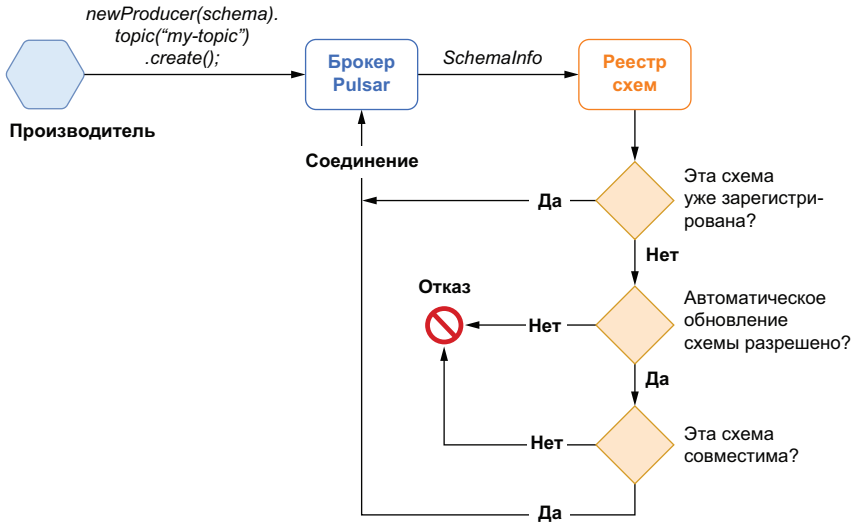
Важно отметить, что реестр схем не обеспечивает применение каждым производителем и потребителем в точности одной и той же схемы, но при этом гарантирует, что используемые схемы совместимы друг с другом. Рассмотрим сценарий, в котором группа разработки изменяет схему для производимых сообщений, добавляя или удаляя некоторое поле или заменяя тип существующего поля со строки на метку времени. Чтобы сохранить неявный контракт производитель–потребитель и избежать непреднамеренного нарушения работы микросервисов потребителя, мы должны гарантировать, что производители публикуют сообщения, содержащие всю информацию, которую требуют потребители. Иначе возникает риск появления сообщений, нарушающих работу существующих приложений из-за удаления необходимого для них поля.

Если реестр схем сконфигурирован для проверки совместимости, то Pulsar выполняет такую проверку, когда производитель устанавливает соединение с темой. Если изменение не нарушает работу потребителей и не приводит к генерации исключения, то оно считается совместимым, и производителю разрешается соединение с темой и создание сообщений с новым типом схемы. Такой подход помогает регулировать деятельность различных групп разработки так, чтобы вносимые изменения не нарушали работу существующих приложений, которые уже потребляют сообщения из тем Pulsar.

### Проверка совместимости схемы производителя

Каждый раз, когда типизированный производитель устанавливает соединение с темой, как показано на рис. 7.6, он передает в брокер копию используемой схемы. Объект `SchemaInfo` создается на основе полученной схемы и передается в реестр схем. Если эта схема уже связана с темой, то производителю разрешается установить соединение, после чего он может продолжить публикацию сообщений с применением заявленной схемы.

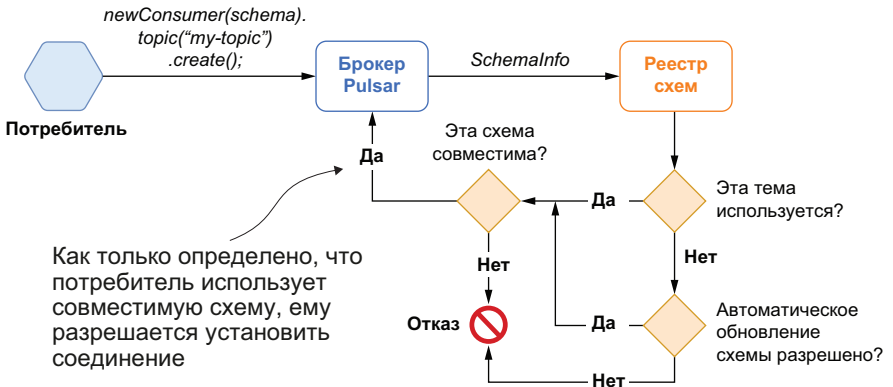
Если схема пока еще не зарегистрирована для данной темы, то реестр схем проверяет параметр настройки стратегии `AutoUpdate` для соответствующего пространства имен, чтобы узнать, разрешено ли производителю регистрировать новые версии схемы в данной теме. Здесь производитель получает отказ, если стратегия запрещает регистрацию новых версий схем. Если обновление схем разрешено, то стратегия обеспечения совместимости выполняет соответствующую проверку, и после ее успешного прохождения схема регистрируется в реестре, а производителю разрешается установить соединение с применением новой версии схемы. Если схема определена как несовместимая, то производитель получает отказ.



**Рис. 7.6.** Логическая блок-схема проверки корректности схемы для типизированного производителя

### Проверка совместимости схемы потребителя

Каждый раз, когда типизированный потребитель устанавливает соединение с темой, как показано на рис. 7.7, он передает в брокер копию используемой схемы. Объект `SchemaInfo` создается на основе полученной схемы и передается в регистр схем.



**Рис. 7.7.** Логическая блок-схема проверки корректности схемы для типизированного потребителя

Если в теме нет активных производителей и потребителей, зарегистрированных схем или каких-либо существующих данных, то считается, что тема не используется (*not in use*). При отсутствии всех перечисленных выше элементов регистр схем проверяет параметр настройки стратегии `AutoUpdate` для соответствующего про-

странства имен, чтобы узнать, разрешено ли потребителю регистрировать новую версию схемы в этой теме. Потребитель получает отказ, если регистрация новых схем запрещена, иначе выполняется проверка стратегии обеспечения совместимости, и после ее успешного прохождения схема регистрируется в реестре, а потребителю разрешается установить соединение с использованием новой версии схемы. Проверка совместимости также выполняется, если тема используется (in use).

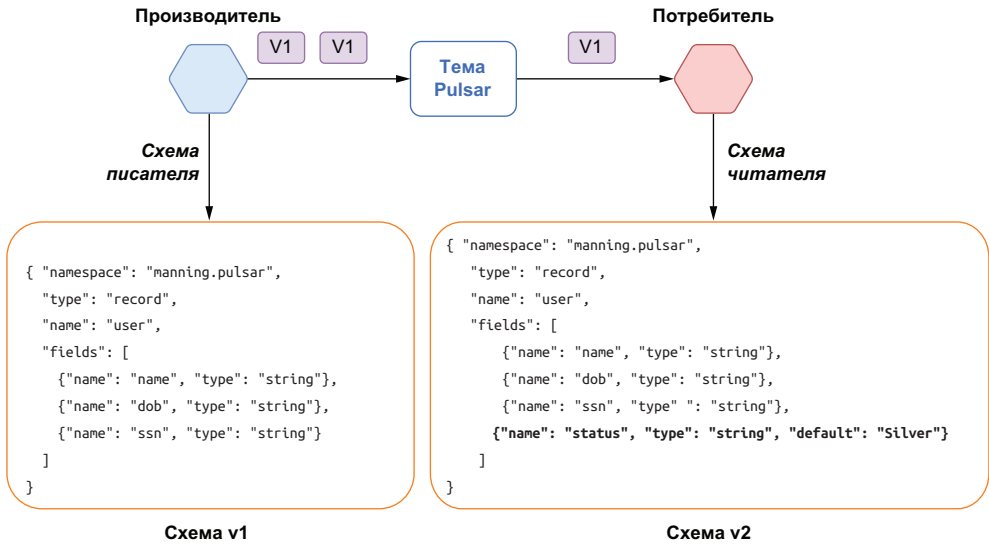
#### **7.2.4. Стратегии проверки совместимости схем**

Реестр схем Pulsar поддерживает шесть различных стратегий обеспечения совместимости, которые можно конфигурировать для каждой темы. Важно отметить, что все проверки совместимости рассматриваются с точки зрения потребителя, даже если выполняется проверка для производителя, поскольку главной целью является предотвращение появления сообщений, которые существующие потребители не могут обработать. Реестр схем использует правила обеспечения совместимости внутренней библиотеки сериализации (например, Avro), чтобы определить, совместима ли новая схема с текущей. По умолчанию для реестра схем Pulsar установлен тип совместимости BACKWARD (обратная), который вместе с другими типами подробно описан в следующих подразделах.

##### **Обратная совместимость**

Обратная совместимость означает, что потребитель может использовать более новую версию схемы, сохраняя возможность чтения сообщений от производителя, применяющего предыдущую версию. Рассмотрим сценарий, в котором производитель и потребитель начинают работу, используя одинаковую версию схемы Avro: v1. Одна из групп, отвечающая за разработку некоторого потребителя, принимает решение о добавлении нового поля с именем `status` в текущую схему для поддержки программы лояльности для новых клиентов с присвоением им различных степеней статуса, например «серебряный», «золотой», «платиновый», как показано на рис. 7.8. В этом случае новая версия схемы v2 должна считаться обратно совместимой, поскольку она определяет значение по умолчанию для нового добавленного поля.

Это позволяет потребителю читать данные, записанные с применением версии v1 схемы, поскольку значение по умолчанию, определенное в новой схеме, будет использоваться для пропущенного поля при десериализации сообщений, сериализованных по версии v1, т. е. в потребляющем приложении все члены получат статус `silver` (серебряный).



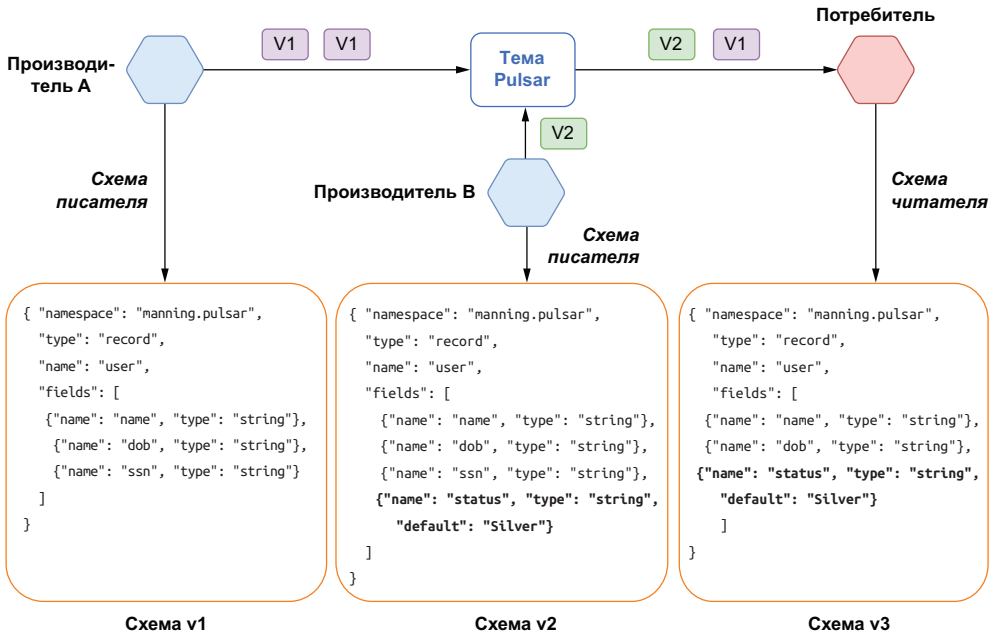
**Рис. 7.8.** Обратные совместимые изменения позволяют потребителю, использующему более новую версию схемы, сохранить возможность обработки сообщений, созданных с применением предыдущей версии этой схемы

Для поддержки этого варианта использования можно использовать стратегию обеспечения совместимости схемы **BACKWARD**. Но при этом поддерживается только вариант, в котором потребители применяют единственную версию схемы раньше, чем производители. Если необходима поддержка производителей, использующих более одной версии схемы позже, чем потребители, то можно воспользоваться стратегией обеспечения совместимости **BACKWARD\_TRANSITIVE**.

Немного усложним вариант использования из предыдущего примера. Теперь у нас есть новый микросервис, добавленный в приложение, отвечающее за определение статуса лояльности клиента и генерирующее сообщения, содержащие поле **status**.

Кроме того, микросервис, который изначально добавил необходимое поле **status**, проходил проверку на безопасность, и при этом было определено, что наличие открытого поля с номером социальной страховки создает слишком большой риск, поэтому оно удалено. На рис. 7.9 можно видеть, что продолжает работу исходный производитель, использующий версию схемы v1, появляется новый микросервис, применяющий версию схемы v2, включающую поле **status**, и остался потребитель, пользующийся третьей версией схемы, в которой удалено поле **ssn**.





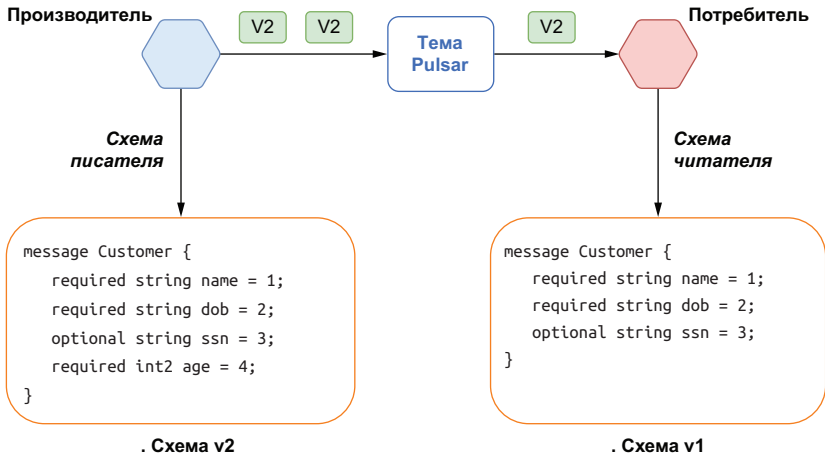
**Рис. 7.9.** Обратные совместимые транзитивные изменения позволяют потребителю, использующему более новую версию схемы, сохранить возможность обработки сообщений, созданных с применением любой предыдущей версии этой схемы

Чтобы отвечать требованиям обратной транзитивной совместимости, новая версия схемы потребителя v3 должна сохранить возможность обработки сообщений от обоих активных производителей (т. е. по схемам v1 и v2). Поскольку версия v3 определяет значение по умолчанию для поля `status`, данные, записанные по схеме v1, могут быть прочитаны потребителем, так как значение по умолчанию в v3 будет использовано для отсутствующего поля при десериализации сообщений, сериализованных по версии v1. Аналогично, поскольку сообщения, сериализованные по версии v2, содержат все поля, требуемые в версии v3, потребитель может просто игнорировать «лишнее» поле `ssn` в этих сообщениях.

### Прямая совместимость

Прямая совместимость (forward compatibility) означает, что потребитель может использовать более старую версию схемы и сохранять возможность чтения сообщений от производителя, применяющего более новую версию. Рассмотрим сценарий, в котором производитель и потребитель начинают работу с одинаковой версией схемы protobuf: v1. Одна из групп, отвечающая за разработку некоторого производителя, принимает решение о добавлении нового поля с именем `age` в текущую схему для поддержки новой

программы маркетинга на основе различных возрастных групп, как показано на рис. 7.10.

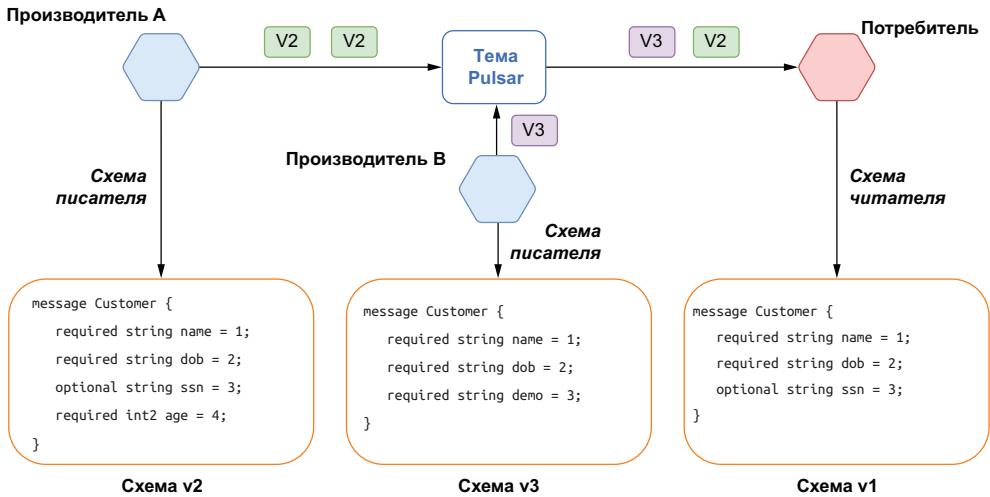


**Рис. 7.10.** Изменения с прямой совместимостью позволяют потребителю, использующему более старую версию схемы, сохранить возможность обработки сообщений, созданных с применением более новой версии этой схемы

В этом варианте новая версия схемы v2 должна считаться обеспечивающей прямую совместимость, так как она просто добавляет новое поле, которого не было в предыдущей версии. Это позволяет потребителю читать данные, записанные с применением версии схемы v2, поскольку новое добавленное поле будет игнорироваться при десериализации сообщений, сериализованных по версии v2. Потребитель может продолжать обработку сообщений, потому что он никак не зависит от нового поля `age`.

Для поддержки этого варианта использования можно использовать стратегию обеспечения совместимости схемы `FORWARD`. Но при этом поддерживается только вариант, в котором потребители применяют единственную версию схемы позже, чем производители. Если необходима поддержка производителей, использующих более одной версии схемы ранее, чем потребители, то можно воспользоваться стратегией обеспечения совместимости `FORWARD_TRANSITIVE`.

Немного усложним вариант использования из предыдущего примера. Теперь у нас есть новый микросервис, добавленный в приложение, отвечающее за определение возраста клиента по полю `age`. Этот микросервис возвращает сообщение, содержащее абсолютно новое поле `demo`, и удаляет «ненужное» поле `ssn`, как показано на рис. 7.11.



**Рис. 7.11.** Прямо совместимые транзитивные изменения позволяют потребителю, использующему более старую версию схемы, сохранить возможность обработки сообщений, созданных с применением любой более новой версии этой схемы

Чтобы отвечать требованиям прямой транзитивной совместимости, версия схемы потребителя v1 должна сохранить возможность обработки сообщений от обоих активных производителей (т. е. по схемам v2 и v3). Поскольку версия v1 определяет, что поле `ssn` не является обязательным, данные, записанные по схеме v3, могут быть прочитаны потребителем, потому что это поле не требуется, и потребитель обязательно должен быть подготовлен для обработки значений `null` для этого поля. Аналогично, поскольку сообщения, сериализованные по версиям v2 и v3, содержат дополнительные поля, не определенные в версии v1, потребитель может просто игнорировать «лишние» поля в этих сообщениях, поскольку они ему не требуются.

### Полная совместимость

Полная совместимость означает, что схема обеспечивает обратную и прямую совместимость одновременно. Данные, сериализованные с применением более старой версии, могут быть десериализованы с использованием новой схемы, а данные, сериализованные с применением новой версии, также могут быть десериализованы при помощи любой предыдущей версии той же схемы.

В некоторых форматах данных, например JSON, не существует полностью совместимых изменений. Каждое изменение является только либо прямо, либо обратно совместимым. Но в других форматах данных, например Avro или protobuf, где можно определять

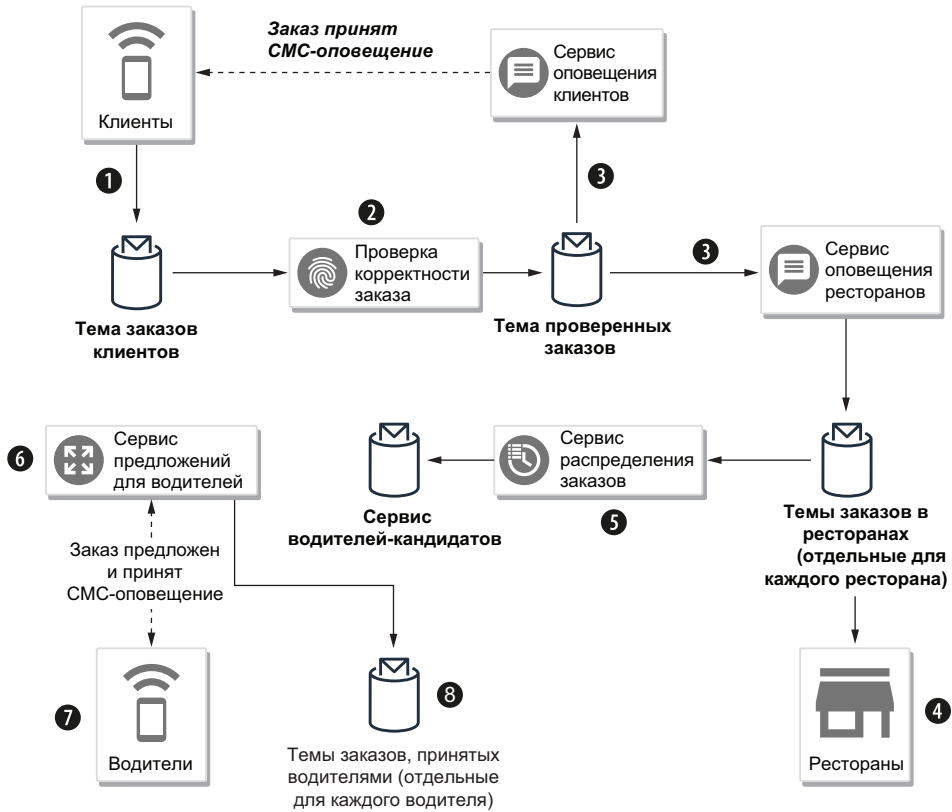
поля со значениями по умолчанию, добавление или удаление поля со значением по умолчанию считается полностью совместимым изменением. Для поддержки этого типа варианта использования можно использовать стратегии обеспечения совместимости схемы FULL или FULL\_TRANSITIVE.

### 7.3. Использование реестра схем

Рассмотрим сценарий для компании доставки продуктов питания GottaEat, который позволяет клиентам находить и заказывать продукты из любого ресторана, включенного в сеть компании, с доставкой в любую локацию по выбору клиента. Заказы можно размещать на сайте компании или через мобильное приложение. Доставка заказа осуществляется с помощью сети независимых водителей, которые должны загрузить специальное мобильное приложение на свои смартфоны. Водители получают оповещения о заказах продуктов, готовых для доставки в своем регионе, и могут принять или отклонить выполнение предложенных заказов.

После того как водитель принял заказ для доставки, соответствующий пункт добавляется в его маршрут, формируемый до вечера (текущего дня), а водителю сообщается направление к ресторану, в котором необходимо забрать заказанные продукты, и локация клиента. Вся эта информация сохраняется в мобильном приложении водителя. Рестораны-участники оповещаются о поступающих заказах и отвечают за просмотр соответствующего списка и за определение временного окна, в пределах которого водитель должен забрать заказанные продукты. Вся эта информация позволяет системе более эффективно планировать работу водителей и исключать случаи слишком раннего прибытия (с потерей времени на ожидание) и опоздания (продукты и блюда теряют свежесть или доставка задерживается).

Как главный архитектор проекта, вы решили, что архитектура микросервисов лучше всего подходит для удовлетворения требований этого бизнеса. Такой подход также позволяет моделировать общую задачу как управляемый событиями процесс, для реализации которого вполне пригоден обмен данными в сообщениях между независимыми микросервисами. Для этой цели вы создали предварительную общую схему проектного решения высокого уровня для варианта регистрации заказов, показанную на рис. 7.12. Теперь необходимо определить, как реализовать это проектное решение, используя Pulsar. Общая блок-схема этого варианта использования описана ниже.



**Рис. 7.12.** Вариант использования регистрации заказов

- Клиенты оформляют заказы, используя веб-сайт компании или мобильное приложение. Заказы публикуются в теме `customer order`.
- Сервис проверки корректности заказов подписывается на тему `customer order` и проверяет заказ, включая обработку предоставленной информации об оплате, например номера кредитной карты или подарочного сертификата, а также получает подтверждение оплаты.
- Проверенные заказы публикуются в теме `validated order` и потребляются сервисом оповещения клиентов (например, клиенту отправляется СМС-сообщение, подтверждающее, что заказ принят и размещен в мобильном приложении) и сервисом оповещения ресторанов, который публикует каждый конкретный заказ в индивидуальной теме `restaurant order`, связанной с заказом (например, `persistent://resturants/orders/<resturant-id>`).
- Рестораны просматривают входящие заказы в своих темах, обновляют состояние заказа с `new` (новый) на `accepted` (принятый) и на основе оценки времени для приготовления продуктов

и блюд определяют временное окно, в котором водитель должен забрать заказанные продукты.

- 5 Сервис распределения заказов отвечает за передачу принятых заказов водителям и использует функцию подписки с применением регулярных выражений Pulsar для потребления из всех индивидуальных тем `restaurant order` (например, `persistent://restaurants/orders/*`) и фильтрует заказы по состоянию `accepted` (принятый). Эту информацию вместе с существующим списком пунктов доставки водителей сервис использует для выбора нескольких кандидатов, чтобы предложить им доставку заказа, а затем публикует сформированный список в теме водителей-кандидатов.
- 6 Сервис водителей-кандидатов потребляет сообщения из темы водителей-кандидатов и направляет оповещение каждому водителю из списка кандидатов, предлагая им выполнить заказ. Если один из водителей принимает заказ, то оповещение отправляется обратно в сервис кандидатов, который, в свою очередь, публикует заказ в индивидуальной теме заказов этого конкретного водителя (например, `persistent://drivers/orders/<drivers-id>`).

Дополнительные варианты использования требуются для формирования маршрута водителя, оповещения клиента о состоянии его заказа и т. д. Но в данный момент все наше внимание сосредоточено на варианте использования регистрации заказов, а также на том, как реестр схем Pulsar упрощает разработку запланированных микросервисов. Рассмотрим подробно структуру проекта GitHub, связанного с этой главой книги. Для текущего подраздела необходимо обратиться к коду в ветви 0.0.1. Это проект Maven с множеством модулей, который содержит три подмодуля, описанных ниже в соответствующих подразделах

### 7.3.1. Моделирование события заказа продуктов в Avro

Первый модуль `domain-schema` содержит все определения схем Avro для варианта использования регистрации заказов GottaEat. Схемы Avro можно определять в чистом формате JSON или в текстовых файлах Avro IDL, но файлы схем необходимо где-то хранить, поэтому рассматриваемый здесь модуль служит именно для этой цели.

Как можно понять из содержимого модуля `domain-schema`, я создал файл определения схемы Avro IDL с именем `src/main/resources/avro/order/food-order.avdl`, содержащий определение схемы, показанное в листинге 7.2. Это файл представляет модель исходных данных для объекта заказа продуктов, который будет использоваться несколькими сервисами, а также для генерации классов Java, применяемых во всех потребляющих микросервисах, написанных на языке Java.

**Листинг 7.2.** Содержимое файла `food-order.avdl`

```

@namespace("com.gottaeat.domain.order") ❶
protocol OrderProtocol {
 import idl "../common/common.avdl";
 import idl "../payment/payment-commons.avdl"; ❷
 import idl "../resturant/resturant.avdl";

 record FoodOrder { ❸
 long order_id;
 long customer_id;
 long resturant_id;
 string time_placed;
 OrderStatus order_status;
 array<OrderDetail> details; ❹
 com.gottaeat.domain.common.Address delivery_location; ❺
 com.gottaeat.domain.payment.CreditCard payment_method;
 float total = 0.0;
 }

 record OrderDetail {
 int quantity;
 float total;
 com.gottaeat.domain.resturant.MenuItem food_item;
 }

 enum OrderStatus {
 NEW, ACCEPTED, READY, DISPATCHED, DELIVERED
 }
}

```

- ❶ Пространство имен для этих типов, соответствующее имени пакета Java.
- ❷ Импорт определений типов Avro из других файлов, позволяющий создавать сложносоставные схемы.
- ❸ Определение записи заказа `FoodOrder`.
- ❹ Каждая запись `FoodOrder` может содержать одно или несколько наименований продуктов и блюд.
- ❺ Использование типа, определенного внутри одного из включаемых определений схемы.

Мы воспользуемся подключаемым модулем Avro для автоматической генерации классов Java на основе определений схем в проекте, добавив конфигурацию, показанную в листинге 7.3, в секцию подключаемых модулей файла Maven `pom.xml`.

**Листинг 7.3.** Конфигурирование подключаемого модуля Avro Maven

```

<plugin>
 <groupId>org.apache.avro</groupId>
 <artifactId>avro-maven-plugin</artifactId>

```

```

<version>1.9.1</version>
<executions>
 <execution>
 <phase>generate-sources</phase> ❶
 <goals>
 <goal>idl-protocol</goal> ❷
 </goals>
 <configuration>
 <sourceDirectory>
 ${project.basedir}/src/main/resources/avro/order ❸
 </sourceDirectory>
 <outputDirectory>
 ${project.basedir}/src/main/java ❹
 </outputDirectory>
 </configuration>
 </execution>
</executions>
</plugin>

```

- ❶ Необходимо генерировать файлы исходного кода на языке Java.
- ❷ Определения в формате IDL.
- ❸ Использование каталога, содержащего файл *food-order.avdl*, как каталога-источника.
- ❹ Локация, в которую выводятся сгенерированные файлы исходного кода.

После определения схем Avro и конфигурирования подключаемого модуля Maven можно выполнить команду, показанную в листинге 7.4, для генерации классов Java в каталоге исходного кода проекта, указанного в листинге 7.3. Эта команда сгенерирует файлы исходного кода классов Java, скомпилирует их и упакует в JAR-файл *domain-schema-0.0.1.jar* перед завершающей публикацией JAR-файла в локальном репозитории Maven.

#### Листинг 7.4. Генерация классов Java из схемы Avro

```

$ cd ./domain-schema
$ mvn install
[INFO] Scanning for projects...
[INFO] -----< com.gottaeat:domain-schema >-----
[INFO] Building domain-schema 0.0.1
[INFO] -----[jar]-----
[INFO]
[INFO] --- avro-maven-plugin:1.9.1:idl-protocol (default) @ domain-schema ---
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile)
[INFO] Compiling 8 source files to domain-schema/target/classes
[INFO]
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ domain-schema ---
[INFO] Building jar: /domain-schema/target/domain-schema-0.0.1.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install)

```



```
[INFO] Installing /domain-schema/target/domain-schema-0.0.1.jar to ..
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

Вы найдете сгенерированные классы в соответствующих подкаталогах внутри каталога исходного кода проекта, как показано в листинге 7.5. Эти файлы слишком велики, чтобы показать их содержимое здесь, но если вы откроете их в любом текстовом редакторе, то увидите, что в них размещены простые «традиционные» объекты Java – POJO, автоматически сгенерированные Авго и содержащие все поля из определений схем вместе с методами сериализации и десериализации объекта в двоичный формат Авго и обратно.

#### Листинг 7.5. Вывод списка всех сгенерированных классов Java

```
ls -R src/main/java/*
gottaeat

src/main/java/com/gottaeat:
domain

src/main/java/com/gottaeat/domain:
common order payment resturant

src/main/java/com/gottaeat/domain/common:
Address.java

src/main/java/com/gottaeat/domain/order:
FoodOrder.java OrderDetail.java OrderProtocol.java OrderStatus.java

src/main/java/com/gottaeat/domain/payment:
CardType.java CreditCard.java

src/main/java/com/gottaeat/domain/resturant:
MenuItem.java
```

К этому моменту мы получили модель предметной области на языке Java для событий заказов продуктов, которая может использоваться другими микросервисами проекта.

### 7.3.2. События производства заказов продуктов

Чтобы не создавать зависимость от мобильного приложения клиентов, разрабатываемого другой группой, мы решили воспользоваться специализированным инструментом для генерации нагрузки для тестирования. Модуль `customer-mobile-app-simulator` содержит коннектор ввода-вывода, предназначенный для имитации мобильного приложения, с помощью которого клиенты будут размещать свои заказы. Коннектор определен внутри класса `CustomerSimulatorSource`, как показано в листинге 7.6.

**Листинг 7.6.** Коннектор ввода-вывода CustomerSimulatorSource

```
import org.apache.pulsar.io.core.Source;
import org.apache.pulsar.io.core.SourceContext;

public class CustomerSimulatorSource implements Source<FoodOrder> { ❶

 private DataGenerator<FoodOrder> generator = new FoodOrderGenerator(); ❷

 @Override
 public void close() throws Exception {
 }

 @Override
 public void open(Map<String, Object> map, SourceContext ctx)
 throws Exception {
 }

 @Override
 public Record<FoodOrder> read() throws Exception {
 Thread.sleep(500); ❸
 return new CustomerRecord<FoodOrder>(generator.generate()); ❹
 }

 static private class CustomerRecord<V> implements Record<FoodOrder> { ❺
 ...
 }
 ... ❻
}
```

- ❶ Реализация интерфейса `Source`, который определяет три замещаемых метода.
- ❷ Класс, производящий случайные заказы продуктов.
- ❸ Пауза в полсекунды между заказами.
- ❹ Публикация новых сгенерированных заказов в исходящей теме.
- ❺ Класс-обертка для передачи объектов `FoodOrder`.
- ❻ Здесь размещен код `LocalRunner`.

Вспомним из главы 5, что из коннектора-источника метод `read` многократно вызывается внутренним фреймворком функций Pulsar и возвращает значение, публикуемое в сконфигурированной исходящей теме. В данном случае возвращаемым значением является заказ продуктов, сформированный случайным образом на основе схем Avro из модуля `domain-schema` и сгенерированный другим классом `FoodOrderGenerator` в том же проекте. Я решил воспользоваться объектом `LocalRunner` для отладки класса `CustomerSimulatorSource`, как показано в листинге 7.7.

**Листинг 7.7.** Использование LocalRunner для отладки коннектора ввода-вывода CustomerSimulatorSource

```

...
public static void main(String[] args) throws Exception {
 SourceConfig sourceConfig = SourceConfig.builder()
 .className(CustomerSimulatorSource.class.getName()) ❶
 .name("mobile-app-simulator") ❷
 .topicName("persistent://orders/inbound/food-orders")
 .schemaType("avro")
 .build();

 // Предполагается, что вы запустили контейнер Docker с ключом
 // --volume=${HOME}/exchange
 String credentials_path = System.getProperty("user.home") +
 File.separator + "exchange" + File.separator;

 LocalRunner localRunner = LocalRunner.builder()
 .brokerServiceUrl("pulsar+ssl://localhost:6651") ❸
 .clientAuthPlugin("org.apache.pulsar.client.impl.auth.AuthenticationTls")
 .clientAuthParams("tlsCertFile:" + credentials_path +
 "admin.cert.pem,tlsKeyFile:" + credentials_path + "admin-pk8.pem")
 .tlsTrustCertFilePath(credentials_path + "ca.cert.pem") ❹
 .sourceConfig(sourceConfig)
 .build();

 localRunner.start(false); ❺
 Thread.sleep(30 * 1000);
 localRunner.stop(); ❻
}

```

- ❶ Определение класса коннектора, который необходимо выполнить.
- ❷ Тема, в которой коннектор будет публиковать сообщения.
- ❸ Определение URL брокера Pulsar, с которым мы будем взаимодействовать.
- ❹ Определение идентификационных данных для TLS-аутентификации, необходимой для установления соединения с Pulsar.
- ❺ Запуск LocalRunner.
- ❻ Остановка LocalRunner.

В листинге 7.7 можно видеть, что LocalRunner сконфигурирован для установления соединения с локально выполняющимся экземпляром образа Docker, который был создан в главе 6. Поэтому в исходном коде присутствуют некоторые параметры конфигурации, обеспечивающие безопасность на основе идентификационных данных, сгенерированных для этого контейнера, например TLS-сертификата клиента и хранилища сертификатов truststore.

### 7.3.3. События потребления заказов продуктов

А теперь рассмотрим модуль `order-validation-service`, содержащий единственную функцию Pulsar с именем `OrderValidationService`, которая в соответствии с целями текущей главы представляет собой всего лишь сокращенную макетную реализацию микросервиса, показанного на рис. 7.12. Эта реализация будет принимать входящие заказы и проверять их корректность, подтверждать оплату и т. п. Со временем в эту реализацию будет добавлена дополнительная логика, но сейчас функция просто записывает все полученные заказы в стандартный поток вывода (`stdout`) перед перенаправлением их в сконфигурированную исходящую тему.

#### Листинг 7.8. Функция `OrderValidationService`

```
import org.apache.pulsar.functions.api.Context;
import org.apache.pulsar.functions.api.Function;
import com.gottaeat.domain.order.FoodOrder;

public class OrderValidationService implements Function<FoodOrder, FoodOrder> {

 @Override
 public FoodOrder process(FoodOrder order, Context ctx) throws Exception {
 System.out.println(order.toString());
 return order;
 }
 ...
}
```

В этот класс также включен метод `main()`, содержащий конфигурацию `LocalRunner` для обеспечения возможности отладки в локальном режиме, как показано в листинге 7.9.

#### Листинг 7.9. Код `LocalRunner` в классе `OrderValidationService`

```
public static void main(String[] args) throws Exception {

 Map<String, ConsumerConfig> inputSpecs =
 new HashMap<String, ConsumerConfig> ();
 inputSpecs.put("persistent://orders/inbound/food-orders",
 ConsumerConfig.builder().schemaType("avro").build()); ❶

 FunctionConfig functionConfig =
 FunctionConfig.builder()
 .className(OrderValidationService.class.getName()) ❷
 .inputs(Collections.singleton(❸
 "persistent://orders/inbound/food-orders"))
 .inputSpecs(inputSpecs)
 .name("order-validation")
 .output("persistent://orders/inbound/valid-food-orders") ❹
 .outputSchemaType("avro")
 .build();
}
```

```

 .runtime(FunctionConfig.Runtime.JAVA)
 .build();

// Предполагается, что вы запустили контейнер Docker с ключом
// --volume=${HOME}/exchange
String credentials_path = System.getProperty("user.home") +
 File.separator + "exchange" + File.separator;

LocalRunner localRunner = LocalRunner.builder()
 .brokerServiceUrl("pulsar+ssl://localhost:6651")
 .clientAuthPlugin("org.apache.pulsar.client.impl.auth.AuthenticationTls")
 .clientAuthParams("tlsCertFile:" + credentials_path +
 "admin.cert.pem,tlsKeyFile:" + credentials_path + "admin-pk8.pem")
 .tlsTrustCertFilePath(credentials_path + "ca.cert.pem")
 .functionConfig(functionConfig)
 .build();

localRunner.start(false);
Thread.sleep(30 * 1000);
localRunner.stop();
}

```

- 1 Будет выполняться потребление сообщений в формате Avro.
- 2 Класс функции, которую необходимо выполнить.
- 3 Тема, из которой эта функция будет потреблять сообщения.
- 4 Тема, в которой эта функция будет публиковать сообщения.
- 5 Определение URL брокера Pulsar, с которым мы будем взаимодействовать.
- 6 Определение идентификационных данных для TLS-аутентификации, необходимой для установления соединения с Pulsar.
- 7 Запуск LocalRunner.
- 8 Остановка LocalRunner.

В листинге 7.8 можно видеть, что LocalRunner сконфигурирован для установления соединения с тем же самым экземпляром Pulsar. Поэтому в исходном коде присутствуют некоторые параметры конфигурации, обеспечивающие безопасность на основе идентификационных данных, сгенерированных для этого контейнера, например TLS-сертификата клиента и хранилища сертификатов truststore.

### 7.3.4. Полный пример

После тщательного разбора кода, содержащегося внутри каждого модуля Maven, переходим к рассмотрению полноценной демонстрации взаимодействия между CustomerSimulatorSource и OrderValidationService. Здесь важно отметить, что для этой первой демонстрации оба класса будут использовать абсолютно одинаковую версию схемы (в рассматриваемом здесь примере: *domain-schema-0.0.1.jar*). Таким образом, схемы производителя и потребителя совместимы друг с другом.

Во-первых, необходимо иметь в своем распоряжении работающий экземпляр образа Docker `pia/pulsar-standalone-secure`, исполняющий роль кластера Pulsar, которым мы воспользуемся для тестирования. Следовательно, потребуется выполнить команды, показанные в листинге 7.10, для запуска экземпляра (если вы пока еще не запустили такой экземпляр), публикации идентификационных данных, необходимых для обоих экземпляров `LocalRunner`, и создания тем, которые будут использоваться.

#### Листинг 7.10. Подготовка кластера Pulsar

```
$ docker run -id --name pulsar --hostname pulsar.gottaeat.com -p:6651:6651
➔ -p 8443:8443 -p 80:80 --volume=${HOME}/exchange:/pulsar/manning/dropbox
➔ -t pia/pulsar-standalone-secure:latest ❶

$ docker exec -it pulsar bash ❷

root@pulsar:/# /pulsar/manning/security/publish-credentials.sh ❸
root@pulsar:/# /pulsar/bin/pulsar-admin tenants create orders ❹
root@pulsar:/# /pulsar/bin/pulsar-admin namespaces create orders/inbound
root@pulsar:/# /pulsar/bin/pulsar-client consume -n 0 -s my-sub
➔ persistent://orders/inbound/food-orders ❺
```

- ❶ Запуск независимого образа Pulsar и назначение тома для совместного использования идентификационных данных.
- ❷ Запуск SSH в только что активизированном контейнере Docker.
- ❸ Публикация всех идентификационных данных в каталоге `${HOME}/exchange` на локальном компьютере.
- ❹ Создание абонента и пространства имен, которые будут использоваться.
- ❺ Запуск потребителя в заданной теме.

Последняя строка в листинге 7.10 запускает потребителя в теме, содержащей события `FoodOrder`, генерируемые объектом `CustomerSimulatorSource`. Эта команда автоматически создает заданную тему и позволяет убедиться в том, что сообщения действительно публикуются. Оставьте открытой эту командную оболочку, чтобы можно было отслеживать входящие сообщения, и переключитесь в свою локальную IDE, используемую для просмотра кода из проекта GitHub, связанного с текущей главой книги. Перемещайтесь по модулю `customer-mobile-app-simulator` и запустите `CustomerSimulatorSource` как приложение Java, которое выполнит код `LocalRunner`, показанный в листинге 7.7.

Если все идет как предполагалось, то вы должны сначала увидеть сообщения Авто, выводимые в окне командной оболочки, как сообщения, доставляемые потребителю. Если взглянуть на пример ожидаемых выходных данных, приведенный ниже, то можно заметить, что полезная нагрузка содержит смесь из читабельного текста и двоичных данных:

```

----- got message -----
????????20200310?AFrench Fries
Large@?@Fountain Drink
Small??709 W 18th StChicagoIL
66012&5555 6666 7777 8888
66011123?A

```

Причина в том, что потребитель, запущенный из командной строки, не имеет связанной с ним схемы. Поэтому исходные байты сообщения, которые в данном случае закодированы в двоичном формате Avro, не десериализуются перед доставкой потребителю, а вместо этого интерпретируются как обычные необработанные байты. Это предоставляет возможность заглянуть внутрь действительного содержимого сообщений, передаваемых от производителя потребителю.

Далее переключаемся обратно в IDE, двигаемся по модулю `order-validation-service` и запускаем `OrderValidationService` как приложение Java, которое выполняет код `LocalRunner`, показанный в листинге 7.9, чтобы активизировать потребителя. Вы должны увидеть сообщения в стандартном потоке вывода (`stdout`), которые содержат данные из заказов продуктов, но теперь в формате JSON вместо двоичных данных Avro, которые ранее появлялись в окне потребителя без схемы. Причина в том, что с этой функцией связана схема, поэтому фреймворк Pulsar автоматически сериализует исходные необработанные байты сообщения в соответствующий класс Java на основе определения схемы Avro:

```

{"order_id": 4, "customer_id": 471, "restaurant_id": 0, "time_placed": "2020-03-14T09:16:13.821", "order_status": "NEW", "details": [{"quantity": 10, "total": 69.899994, "food_item": {"item_id": 3, "item_name": "Fajita", "item_description": "Chicken", "price": 6.99}}], "delivery_location": {"street": "3422 Central Park Ave", "city": "Chicago", "state": "IL", "zip": "66013"}, "payment_method": {"card_type": "VISA", "account_number": "9999 0000 1111 2222", "billing_zip": "66013", "ccv": "555"}, "total": 69.899994}

{"order_id": 5, "customer_id": 152, "restaurant_id": 1, "time_placed": "2020-03-14T09:16:14.327", "order_status": "NEW", "details": [{"quantity": 6, "total": 12.299999, "food_item": {"item_id": 1, "item_name": "Cheeseburger", "item_description": "Single", "price": 2.05}}, {"quantity": 8, "total": 31.6, "food_item": {"item_id": 2, "item_name": "Cheeseburger", "item_description": "Double", "price": 3.95}}], "delivery_location": {"street": "123 Main St", "city": "Chicago", "state": "IL", "zip": "66011"}, "payment_method": {"card_type": "VISA", "account_number": "9999 0000 1111 2222", "billing_zip": "66013", "ccv": "555"}, "total": 43.9}

```

Одной из наилучших функциональных возможностей Avro, предоставляющей великолепное решение для организации обмена данными между сервисами на основе передачи сообщений, явля-

ется поддержка развития схемы (schema evolution). Когда сервис, записывающий сообщения, обновляет свою схему, сервисы, потребляющие эти сообщения, могут продолжать их обработку без необходимости внесения каких-либо изменений в код при условии, что новая схема производителя совместима со старой версией, применяемой потребителями.

Сейчас в рассматриваемом здесь примере производитель и потребитель используют одинаковую версию JAR-файла *domain-schema* (версию 0.0.1), т. е. абсолютно одинаковую версию схемы. Это вполне ожидаемое поведение, но оно не позволяет показать возможность развития схемы Pulsar. В следующем подразделе демонстрируется эта возможность при подробном разборе кода в ветви 0.0.2 проекта GitHub, связанного с текущей главой. Таким образом, потребуется переход к ветви версии 0.0.2, прежде чем начать рассмотрение примеров и, что более важно, оставить в рабочем состоянии контейнер Docker.

## 7.4. Развитие схемы

Во время еженедельной встречи с группой сопровождения мобильного приложения для клиентов получена информация о том, что на начальном этапе тестирования обнаружено несоответствие требованиям. Дело в том, что текущая схема заказа продуктов не поддерживает для клиентов возможность адаптировать заказы в соответствии со своими особыми вкусами. Сейчас клиент не может указать, что ему не нужен лук в гамбургерах или нежелательно дополнительное количество соуса гуакамоле в буррито. Из этого следует, что схему необходимо пересмотреть и в исходный тип `MenuItem` нужно добавить дополнительное поле `customizations`, как показано в листинге 7.11.

### Листинг 7.11. Развитие схемы

```
@namespace("com.gottaeat.domain.restaurant")

protocol RestaurantsProtocol {

 record MenuItem {
 long item_id;
 string item_name;
 string item_description;
 array<string> customizations = [""]; ❶
 float price;
 }
}
```

- ❶ Новое добавленное поле для поддержки более точного указания свойств отдельных заказываемых продуктов с заданным значением по умолчанию.



После внесения этого изменения в схему, расположенную в модуле `domain-schema`, необходимо также обновить версию артефакта до 0.0.2 в файле `pot.xml`, чтобы можно было различать две версии. После внесения всех требуемых изменений в исходный код необходимо выполнить команду, показанную в листинге 7.12, для генерации файлов исходного кода классов Java, их компиляции и упаковки в файл `domain-schema-0.0.2.jar`.

**Листинг 7.12.** Генерация классов Java из обновленной схемы Avro

```
$ cd ./domain-schema
$ mvn install
```

Необходимо убедиться в том, что создаваемая версия действительно имеет номер 0.0.2. Если совершить ошибку при изменении номера версии в файле `pot.xml` и оставить номер 0.0.1, то будет перезаписан существующий JAR-файл со старой версией схемы, и результат получится совсем другой. Если вы случайно перезаписали файл `domain-schema-0.0.01.jar`, то можете удалить новое добавленное поле и пересобрать JAR-файл. Затем верните новое поле, измените номер версии на 0.0.2 и создайте новый JAR-файл. Можно легко проверить, что сгенерированные классы Java основаны на новой версии схемы, если найти поле `customization` в исходном коде класса `src/main/java/com/gottaeat/domain/restaurant/MenuItem.java`.

Далее обновляем номер версии для зависимости `domain-schema` в модуле `customer-mobile-app-simulator`, чтобы использовать обновленную схему, как показано в листинге 7.13.

**Листинг 7.13.** Обновление версии для зависимости `domain-schema`

```
<dependency>
 <groupId>com.gottaeat</groupId>
 <artifactId>domain-schema</artifactId>
 <version>0.0.2</version>
</dependency>
```

Класс `FoodGenerator` был обновлен в ветви версии 0.0.2 для включения уточняющих настроек в заказы продуктов, поэтому потребуется более новая версия JAR-файла, созданного в листинге 7.11. Если при компиляции возникли ошибки, то, вероятнее всего, вы продолжаете ссылаться на более старую версию. Теперь можно обновить зависимости Maven, чтобы обеспечить использование версии 0.0.2 JAR-файла `domain-schema` и снова запустить из `CustomerSimulatorSource LocalRunner`. Обновленная логика в `FoodGenerator` теперь добавляет уточняющие настройки в каждый газированный напиток, чтобы точно определить его тип (например, Кока-Кола, Спрайт и т. д.). Кроме того, случайным образом добавляются различные уточнения к наименованиям других продуктов:

```

----- got message -----
n?.2020-03-14T15:10:16.773?@Fountain Drink
SmallCoca-Cola??123 Main StChicagoIL
66011&1234 5678 9012 3456
66011000?@
----- got message -----
p?.2020-03-14T15:10:17.273?@Fountain Drink
Large
 Sprite@??@BurritBeefSour Cream??@?_A
 FajitaChickenExtra Cheese??@ 844 W Cermak RdChicagoIL
66014&1234 5678 9012 3456
66011000???A

```

- ① Уточнение типа газированного напитка: Coca-Cola.
- ② Уточнение для заказанного продукта: Sour Cream (сметана).

Если снова проверить окно консоли потребителя без схемы, то вы увидите случайную запись с некоторыми уточнениями, как показано в приведенном выше фрагменте кода. Это говорит о том, что мы производим сообщения на основе обновленной схемы версии 0.0.2, как и предполагалось. А теперь выполним `OrderValidationService LocalRunner` в модуле `order-validation-service`, который остался сконфигурированным для использования версии 0.0.1 JAR-файла `domain-schema`, содержащего старую версию схемы.

Так как версия 0.0.2 схемы обратно совместима с версией 0.0.1, применяемой `OrderValidationService`, сохраняется возможность потребления сообщений на основе более новой версии схемы. В приведенных ниже десериализованных сообщениях Avro можно видеть, что, поскольку эти новые сообщения десериализованы с помощью старой схемы, добавленное поле `customizations` игнорируется. Это вполне ожидаемо и никак не влияет на функциональность потребителя, потому что он изначально ничего не знал о таких полях:

```

{"order_id": 55, "customer_id": 73, "restaurant_id": 0, "time_placed": "2020-03-14T15:10:16.773", "order_status": "NEW", "details": [{"quantity": 7, "total": 7.0, "food_item": {"item_id": 10, "item_name": "Fountain Drink", "item_description": "Small", "price": 1.0}}], "delivery_location": {"street": "123 Main St", "city": "Chicago", "state": "IL", "zip": "66011"}, "payment_method": {"card_type": "AMEX", "account_number": "1234 5678 9012 3456", "billing_zip": "66011", "ccv": "000"}, "total": 7.0}

{"order_id": 56, "customer_id": 168, "restaurant_id": 0, "time_placed": "2020-03-14T15:10:17.273", "order_status": "NEW", "details": [{"quantity": 2, "total": 4.0, "food_item": {"item_id": 11, "item_name": "Fountain Drink", "item_description": "Large", "price": 2.0}}, {"quantity": 1, "total": 7.99, "food_item": {"item_id": 1, "item_name": "Burrito", "item_description": "Beef", "price": 7.99}}, {"quantity": 2, "total": 13.98, "food_item": {"item_id": 3, "item_name": "Fajita",

```

```
"item_description": "Chicken", "price": 6.99}}}, "delivery_location":
{ "street": "844 W Cermak Rd", "city": "Chicago", "state": "IL", "zip":
"66014"}, "payment_method": { "card_type": "AMEX", "account_number":
"1234 5678 9012 3456", "billing_zip": "66011", "ccv": "000"}, "total":
25.97}
```

Также следует отметить, что никаких изменений кода не потребовалось для `OrderValidationService`. Таким образом, если бы это была производственная среда, текущий активный экземпляр сервиса мог бы продолжать работать без сбоев даже после внесения изменений в кодовую базу мобильного приложения. Это делает их полностью независимыми друг от друга даже при изменении API (формата сообщения).

## 7.5. Резюме

- Мы рассмотрели различные стили обмена данными между микросервисами и узнали, почему Pulsar лучше всего подходит для асинхронного обмена данными между сервисами на основе схемы публикация–подписка.
- Реестр схем Pulsar позволяет производителям и потребителям сообщений согласовывать структуру данных на уровне темы и обеспечивает совместимость схемы для производителей сообщений.
- Реестр схем Pulsar поддерживает шесть различных стратегий совместимости, включая прямую, обратную и полную, а также механизм проверок каждого типа совместимости с точки зрения потребителя.
- Язык определения интерфейса (IDL) Avro представляет собой превосходный способ моделирования событий, потребляемых в среде Pulsar, поскольку позволяет создать модульную структуру типов и с легкостью обеспечивает их совместное использование несколькими сервисами.
- Реестр схем Pulsar можно сконфигурировать для обеспечения прямой и/или обратной совместимости схемы для темы Pulsar, гарантирующей, что подключенный к ней производитель или потребитель использует схему, совместимую со всеми существующими клиентами.

## *Часть III*

# *Практическая разработка приложений с использованием Apache Pulsar*

**В** этой части мы выходим за рамки теории и упрощенных примеров и более подробно рассматриваем практическое применение функций Pulsar в качестве среды разработки приложений на основе микросервисов, изучая гораздо более реалистичный вариант использования, связанный с вымышленной компанией, занимающейся доставкой продуктов питания, под названием GottaEat. В этой части показано, как реализовать общие шаблоны проектирования как из мира корпоративной интеграции, так и из мира микросервисов, с особым выделением применения различных паттернов, таких как маршрутизация и фильтрация на основе контента, отказоустойчивость и доступ к данным в реальном сценарии.

В главе 8 показано, как реализовать общие паттерны маршрутизации сообщений, такие как разделение сообщений, маршрутизация на основе содержимого и фильтрация. Также демонстрируется, как реализовать различные паттерны преобразования сообщений, такие как извлечение значений и перевод сообщений.

В главе 9 подчеркивается важность механизма обеспечения устойчивости, встраиваемого в конкретные микросервисы, и демонстрируется реализация этого механизма внутри функций Pulsar на основе Java с помощью библиотеки `resilience4j`. Здесь рассматриваются различные сценарии, которые могут возникнуть в программе, основанной на событиях, а также паттерны, которые можно использовать для изоляции микросервисов от таких сценариев сбоев, чтобы максимизировать время безотказной работы приложения.

В главе 10 основное внимание уделено возможности получения доступа к данным из различных внешних систем в функциях Pulsar. Здесь демонстрируются различные методы получения информации в микросервисах и некоторые аспекты, связанные с возможными задержками.

Глава 11 знакомит вас с процессом развертывания различных типов моделей машинного обучения внутри функции Pulsar с использованием различных платформ машинного обучения. Здесь также рассматривается весьма важная задача ввода необходимой информации в модель для получения точного прогноза.

В главе 12 рассматривается использование функций Pulsar в среде периферийных вычислений для выполнения анализа данных интернета вещей в режиме реального времени. Глава начинается с подробного описания среды периферийных вычислений и различных уровней ее архитектуры. Далее демонстрируется практическое применение фреймворка Pulsar Functions для обработки информации на периферии и передачи только сводок, а не всего набора данных.

# Паттерны применения

---

## *Pulsar Functions*

### **Темы:**

- проектирование приложения на основе Pulsar Functions;
- реализация общеизвестных надежных паттернов обмена сообщениями с использованием Pulsar Functions.

В предыдущей главе был представлен воображаемый сервис доставки продуктов питания GottaEat и описан простой вариант использования ввода заказов, в котором клиенты размещают заказы с помощью мобильного приложения этой компании. Как вы помните, первым микросервисом в этом процессе был `OrderValidationService`, отвечающий за проверку корректности заказа перед его передачей водителям для доставки, если заказ правильный, или за оповещение клиента об ошибках в заказе.

Но термин «проверка» (точнее: «валидация» – `validate`) имеет несколько более сложный смысл, нежели функция, позволяющая убедиться в том, что значения в полях соответствуют правильному типу и формату. В рассматриваемом конкретном сценарии заказ считается корректным, только если способ оплаты, предложенный клиентом, является приемлемым, сумма, переводимая из бан-

ка, авторизована, открыт хотя бы один ресторан, который готов предоставить все заказанные наименования продуктов и блюд, и, что наиболее важно, адрес доставки, заявленный клиентом, может быть точно определен парой широта–долгота и названием улицы с номером дома. Если невозможно подтвердить все эти сведения, то заказ считается некорректным, и клиента необходимо оповестить об этом надлежащим образом. Таким образом, `OrderValidationService` – это не просто сервис, который может обеспечить принятие всех вышеперечисленных решений, но также должен согласовывать свои действия с другими системами. Это отличный пример того, как приложение Pulsar может быть скомпоновано из нескольких функций и сервисов меньшего размера.

Для `OrderValidationService` требуется обязательная интеграция с несколькими другими микросервисами и внешними системами для выполнения обработки платежа, геоинформации и размещения полностью проверенного на корректность заказа продуктов. Из этого следует, что лучше поискать готовые решения для задач такого типа, чем заново изобретать колесо. В данном случае великолепным источником является каталог паттернов в книге «Enterprise Integration Patterns» – авторы Грегор Хопе (Gregor Hohpe) и Бобби Вулф (Bobby Woolf) (Addison-Wesley Professional, 2003). Здесь содержится несколько независимых от конкретных технологий, проверенных временем паттернов для решения общеизвестных сложных задач по интеграции предприятий. Паттерны категоризованы по типам решаемых задач и применимы на большинстве платформ интеграции, основанных на обмене сообщениями. В следующих разделах будет показано, как можно реализовать такие паттерны с использованием Pulsar Functions.

## 8.1. Конвейеры данных

Чтобы получить эффективное проектное решение для приложений на основе Pulsar Functions, потребуется знание концепций программирования с использованием сервиса Dataflow и конвейеров данных. Здесь будут описаны на высоком уровне эти модели программирования и показано, что фреймворк Pulsar Functions естественным образом подходит для такого стиля программирования.

### 8.1.1. Процедурное программирование

Обычно компьютерные программы ранее моделировались как набор последовательных операций, и каждая операция зависела от результата (вывода) предыдущей. Такие программы невозможно выполнить в параллельном режиме, потому что они работали с одними и теми же данными, следовательно, были обязаны ждать завершения предыдущей операции перед выполнением следующей.

Рассмотрим логику простого приложения размещения заказов в такой модели программирования. Необходимо написать простую функцию `processOrder`, которая должна выполнять приведенную ниже последовательность шагов (через прямой или косвенный вызов другой функции), чтобы завершить процесс и вернуть номер заказа, обозначающий успешное размещение.

- 1 Поверить в товарных запасах наличие заданного наименования.
- 2 Извлечь информацию о клиенте (адрес доставки, способ оплаты, купоны скидки, степень надежности и т. п.).
- 3 Вычислить цену, включая налог с продажи, стоимость доставки, скидки по купонам, скидки по программе лояльности и т. п.
- 4 Принять платеж от клиента.
- 5 Уменьшить на единицу счетчик этого наименования товара на складе.
- 6 Оповестить центр исполнения о текущем заказе для его обработки и доставки товара.
- 7 Вернуть номер заказа клиенту.

На каждом из перечисленных шагов обрабатывается один и тот же заказ, и каждый шаг зависит от результата, полученного на предыдущем шаге. Например, невозможно принять оплату от клиента до вычисления цены. Таким образом, необходимо ждать завершения предыдущего шага, прежде чем продолжить работу, т. е. невозможно выполнять эти шаги в параллельном режиме.

### 8.1.2. Программирование с использованием потока данных

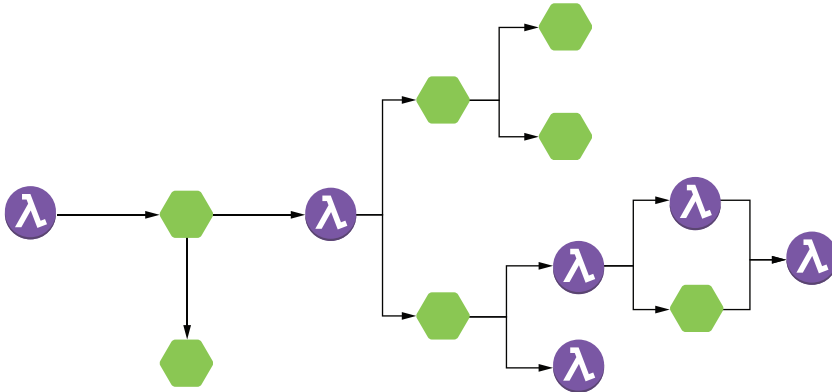
При программировании на основе потока данных внимание сосредоточено на перемещении данных через последовательность независимых функций обработки, соединенных между собой через явно определенные каналы ввода и вывода. Такие предварительно определенные последовательности операций принято обозначать термином «конвейер данных» (*data pipeline*). Приложения на основе *Pulsar Functions* должны моделироваться как конвейеры данных, сущностью которых является перемещение данных через последовательность этапов, каждый из которых обрабатывает эти данные и создает новые выходные данные. Каждый этап в конвейере данных должен иметь возможность обработки, основанной исключительно на содержимом входящего сообщения. Такой подход исключает любые зависимости в процессе обработки и позволяет каждой функции начать выполнение, как только данные становятся доступными.

Общеизвестной аналогией конвейера данных является сборочная линия любого автозавода. Вместо постепенного конструирования автомобиля из отдельных деталей в одной локации каж-



дая машина проходит через последовательность этапов сборки. На каждом этапе к автомобилю добавляется определенная деталь, но такие действия можно выполнять в параллельном, а не последовательном режиме. Таким образом, можно параллельно производить сборку множества машин, существенно увеличивая производительность автозавода.

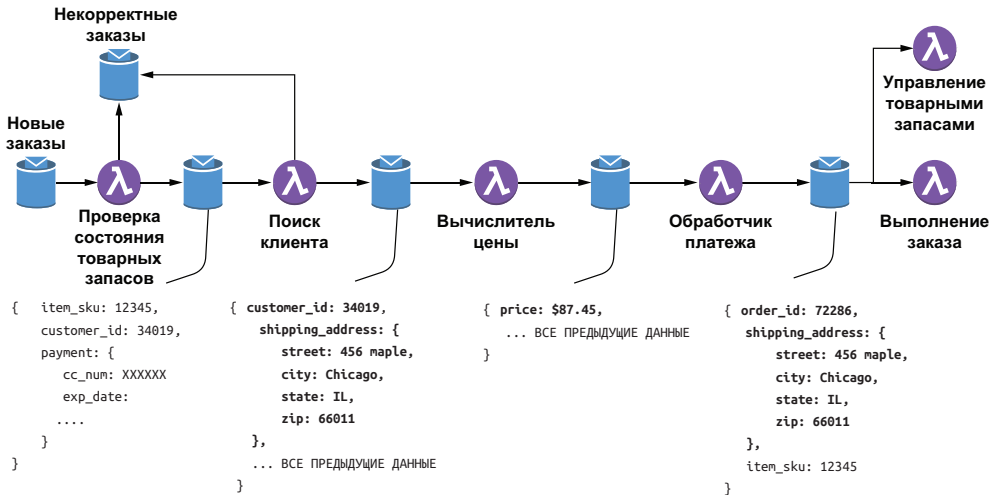
Приложения на основе функций Pulsar должны проектироваться как топологические схемы, состоящие из нескольких отдельных функций Pulsar, выполняющих операции обработки данных и соединенных через входящие и исходящие темы Pulsar. Такие топологические схемы можно считать направленными ациклическими графами (DAG) с функциями/микросервисами, действующими как модули обработки, и ребрами, представляющими парные сопряжения входящих/исходящих тем, используемых для направления данных из одной функции в другую, как показано на рис. 8.1.



**Рис. 8.1.** Наилучшим представлением приложения Pulsar является конвейер данных с направлением потока слева направо через функции и микросервисы для реализации бизнес-логики в форме последовательности шагов

Конвейер данных работает как распределенный фреймворк обработки, где каждая функция конвейера является независимым модулем, который может выполняться немедленно после поступления очередных входных данных. Кроме того, эти слабо связанные компоненты обмениваются данными между собой в асинхронном режиме, что позволяет работать с собственной скоростью и не блокироваться в ожидании ответа от другого компонента. В свою очередь, такой подход позволяет иметь несколько экземпляров любого компонента, работающих параллельно для обеспечения требуемой производительности. Из этого следует, что при проектировании приложения необходимо всегда помнить о сохранении как можно более высокой степени независимости функций и сервисов, чтобы воспользоваться параллельным режимом, когда потребуются. Вернемся к варианту использования размещения заказов, чтобы

продемонстрировать, как можно реализовать его в виде конвейера данных, похожего на показанный на рис. 8.2. Термин «поток данных» (dataflow) подразумевает, что лучше всего сосредоточиться на потоке данных, показанном в нижней части рис. 8.2.



**Рис. 8.2.** Поток данных для варианта использования размещения заказов. Поскольку данные проходят через различные этапы процесса, к сведениям из исходного заказа добавляется дополнительная информация, которая будет использоваться на следующем этапе процесса обработки

На рис. 8.2 можно видеть, что каждый шаг процесса передает исходный заказ, дополненный определенным фрагментом информации, требуемым на следующем шаге (например, идентификатор клиента и адрес доставки добавлены к сообщению сервисом поиска клиента). Поскольку все данные, необходимые для обработки на конкретном этапе, содержатся в сообщении, каждая функция может выполняться немедленно после прибытия очередной порции данных.

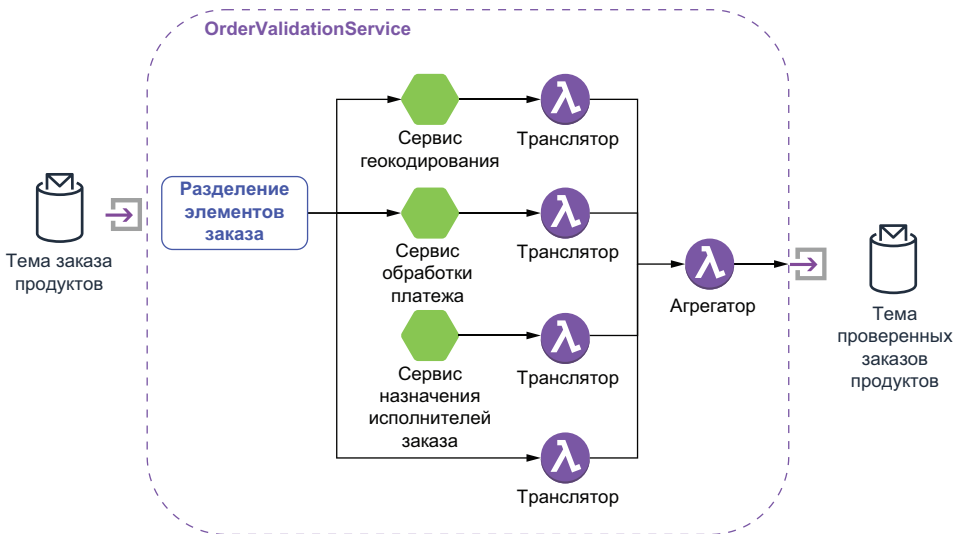
Обработчик платежей удаляет информацию о платеже из сообщения и публикует сообщения, содержащие новый сгенерированный идентификатор заказа, адрес доставки и код конкретного товара (SKU). Эти сообщения потребляются несколькими функциями: сервис управления товарными запасами использует код товара для уменьшения счетчика доступных единиц такого товара на складе, а для функции выполнения заказа необходим код товара и адрес доставки для отправки заказанного товара клиенту. Надеюсь, что эта схема способствует лучшему пониманию того, как должны проектироваться и моделироваться приложения на основе Pulsar Functions, прежде чем мы перейдем к рассмотрению более сложных паттернов проектирования в следующем разделе.

## 8.2. Паттерны маршрутизации сообщений

Маршрутизатор сообщений (message router) – это архитектурный паттерн, используемый для управления потоком данных, проходящим через топологию приложения Pulsar, посредством распределения сообщений по различным темам на основе конкретно заданных условий. Каждый из паттернов, рассматриваемых в этом разделе, предоставляет тщательно проверенные на практике правила для динамической маршрутизации сообщений. Кроме того, будет подробно описано, как можно реализовать паттерны с использованием Pulsar Functions.

### 8.2.1. Паттерн «разделитель»

Функция `OrderValidationService` принимает сообщение, содержащее три взаимосвязанных фрагмента информации, которые необходимо проверить различными способами: адрес доставки, информацию о платеже и сам заказ продуктов. Проверка каждого фрагмента требует взаимодействия с некоторым внешним сервисом, время ответа которого может оказаться слишком длительным. Простейшим подходом к устранению этой проблемы может быть выполнение шагов в последовательном режиме, один за другим. Но такой подход приведет к возникновению слишком большой задержки для каждого входящего заказа. Задержка возникает из-за того, что три вышеперечисленные подзадачи выполняются последовательно, при этом общее время простоя будет равно сумме отдельных задержек.



**Рис. 8.3.** Топологическая схема `OrderValidationService` состоит из нескольких других микросервисов и функций и использует паттерн «разделитель»

Поскольку здесь нет зависимостей между результатами, предоставляемыми этими сервисами промежуточной проверки (например, проверка платежа не зависит от результата геокодирования), более эффективным подходом было бы выполнение этих подзадач в параллельном режиме. При параллельном выполнении общая задержка может быть сокращена до времени задержки в подзадаче с самым длительным временем работы. Для организации такого параллельного выполнения подзадач в функции `OrderValidationService` реализован паттерн «разделитель» (`splitter`) для выделения отдельных элементов сообщения, чтобы их можно было обработать в различных сервисах. На рис. 8.3 можно видеть, что `OrderValidationService` состоит из нескольких меньших функций, реализующих весь процесс проверки в целом.

Предложенное решение также должно стать эффективным с точки зрения использования сетевых ресурсов, поскольку избегает отправки всего заказа продуктов в полном объеме в каждый микросервис, так как для обработки необходима лишь часть сообщения. В коде листинга 8.1 можно видеть, что в каждый промежуточный сервис действительно передается только часть сообщения с идентификатором заказа, который будет использоваться для объединения полученных результатов проверок в конечный итог с помощью функции-агрегатора.

#### Листинг 8.1. Реализация паттерна «разделитель» в `OrderValidationService`

```
public class OrderValidationService implements Function<FoodOrder, Void> {

 private boolean initialized;
 private String geoEncoderTopic, paymentTopic,
 private String restaurantTopic, orderTopic;

 @Override
 public Void process(FoodOrder order, Context ctx) throws Exception {
 if (!initialized) {
 init(ctx);
 }

 ctx.newOutputMessage(geoEncoderTopic, AvroSchema.of(Address.class))
 .property("order-id", order.getMeta().getOrderId() + "")
 .value(order.getDeliveryLocation()).sendAsync();

 ctx.newOutputMessage(paymentTopic, AvroSchema.of(Payment.class))
 .property("order-id", order.getMeta().getOrderId() + "")
 .value(order.getPayment()).sendAsync();

 ctx.newOutputMessage(orderTopic, AvroSchema.of(FoodOrderMeta.class))
 .property("order-id", order.getMeta().getOrderId() + "")
 .value(order.getMeta()).sendAsync();
 }
}
```

```

 ctx.newOutputMessage(resturantTopic, AvroSchema.of(FoodOrder.class))
 .property("order-id", order.getMeta().getOrderId() + "")
 .value(order).sendAsync();
 }

 return null;
}

private void init(Context ctx) {
 geoEncoderTopic = ctx.getUserConfigValue("geo-topic").toString();
 paymentTopic = ctx.getUserConfigValue("payment-topic").toString();
 resturantTopic = ctx.getUserConfigValue("restaurant-topic").toString();
 orderTopic = ctx.getUserConfigValue("aggregator-topic").toString();
 initialized = true;
}

```

6

- ❶ Инициализация всех имен тем, чтобы стало известно, где публиковать сообщения.
- ❷ Добавление идентификатора заказа в свойства сообщения, чтобы появилась возможность объединения результатов.
- ❸ Отправка только элемента адреса доставки из сообщения в сервис геокодирования.
- ❹ Отправка только элемента информации о платеже из сообщения в сервис проверки платежа.
- ❺ Отправка метаданных заказа напрямую в тему агрегатора, так как их обработка не требуется.
- ❻ Отправка только элемента `FoodOrder` из сообщения в сервис назначения исполнителей заказа.

Асинхронный режим обработки отдельных элементов сообщения создает затруднения при объединении результатов. Каждый элемент обрабатывается отдельным сервисом с различным временем ответа (например, сервис геокодирования вызывает веб-сервис, сервис проверки платежа должен обмениваться данными с банком, чтобы обеспечить защиту денежных средств, а каждому ресторану придется отвечать вручную, чтобы принять заказ или отказаться от него). Эти типы подзадач усложняют процесс объединения нескольких различных, хотя и взаимосвязанных сообщений. Для решения этой задачи предназначен паттерн «агрегатор» (`aggregator`).

Агрегатор – это компонент с сохранением состояния, принимающий все сообщения с ответами из вызванных сервисов (т. е. `Geo-Encoder`, `Payment` и т. д.) и объединяющий эти ответы с использованием идентификатора заказа. После формирования полного набора ответов одно объединенное сообщение публикуется в исходящей теме. Если вы выбираете реализацию паттерна «агрегатор», то непременно должны учесть три основных фактора:

- корреляцию – как сообщения связываются (коррелируются) друг с другом;

- завершенность – когда мы готовы к публикации итогового сообщения;
- агрегацию (объединение) – как входящие сообщения объединяются в один итоговый результат.

В рассматриваемом здесь конкретном варианте использования мы решили, что идентификатор заказа применяется как корреляционный идентификатор, что помогает определить, какие ответные сообщения должны объединяться. Результат будет считаться полным только после получения всех трех сообщений для конкретного заказа. Такой подход обозначается как стратегия «wait for all» (ждать всех). Наконец, ответы с результатами будут объединены в один объект типа `ValidatedFoodOrder`.

Рассмотрим подробнее код агрегатора, показанный в листинге 8.2, чтобы увидеть подробности реализации. С учетом строгой типизации функций Pulsar невозможно определить интерфейс для приема нескольких различных типов объектов ответа (например, объекта `AuthorizedPayment` из сервиса `Payment`, типа `Address` из сервиса `GeoEncoder` и т. п.). Поэтому используется функция-транслятор между этими сервисами и `OrderValidationAggregator`. Каждая функция-транслятор выполняет преобразование промежуточных возвращаемых типов, характерных для каждого сервиса, в объект `ValidatedFoodOrder`, который позволяет принимать сообщения от всех сервисов в одной функции Pulsar.

#### Листинг 8.2. Функция-агрегатор в сервисе `OrderValidationService`

```
public class OrderValidationAggregator implements
 Function<ValidatedFoodOrder, Void> {

 @Override
 public Void process(ValidatedFoodOrder in, Context ctx) throws Exception {
 Map<String, String> props = ctx.getCurrentRecord().getProperties();
 String correlationId = props.get("order-id");

 ValidatedFoodOrder order;
 if (ctx.getState(correlationId.toString()) == null) {
 order = new ValidatedFoodOrder();
 } else {
 order = deserialize(ctx.getState(correlationId.toString()));
 }

 updateOrder(order, in);

 if (isComplete(order)) {
 ctx.newOutputMessage(ctx.getOutputTopic(),
 AvroSchema.of(ValidatedFoodOrder.class))
 .properties(props)
 .value(order).sendAsync();
 }
 }
}
```

```

 ctx.putState(correlationId.toString(), null);
 } else {
 ctx.putState(correlationId.toString(), serialize(order));
 }

 return null;
}

private boolean isComplete(ValidatedFoodOrder order) {
 return (order != null && order.getDeliveryLocation() != null
 && order.getFood() != null && order.getPayment() != null
 && order.getMeta() != null);
}

private void updateOrder(ValidatedFoodOrder val,
 ValidatedFoodOrder res) {
 if (res.getDeliveryLocation() != null
 && val.getDeliveryLocation() == null) {
 val.setDeliveryLocation(response.getDeliveryLocation());
 }

 if (resp.getFood() != null && val.getFood() == null) {
 val.setFood(response.getFood());
 }

 if (resp.getMeta() != null && val.getMeta() == null) {
 val.setMeta(response.getMeta());
 }

 if (resp.getPayment() != null && val.getPayment() == null) {
 val.setPayment(response.getPayment());
 }
}

private ByteBuffer serialize(ValidatedFoodOrder order) throws IOException {
 ...
}

private ValidatedFoodOrder deserialize(ByteBuffer buffer) throws IOException,
 ➤ ClassNotFoundException {
 ...
}

```

- ❶ Проверка: получены ли уже некоторые ответы для этого заказа.
- ❷ Если ответы получены, то выполнить десериализацию байтов в объект `ValidatedFoodOrder`.
- ❸ Каждое сообщение будет иметь тип `ValidatedFoodOrder`, но содержать только одно из четырех полей.
- ❹ Проверка: получены ли все четыре сообщения, т. е. все подзадачи выполнены.

- 5 После агрегации текущего заказа можно очистить состояние.
- 6 Если не все сообщения получены, выполнить сериализацию объекта и сохранить его в текущем контексте до прибытия следующего сообщения.
- 7 Объект считается полностью сформированным, только если приняты все четыре сообщения.
- 8 Копирование всех полей, существующих в принятом объекте.
- 9 Вспомогательный метод для преобразования объекта `ValidatedFoodOrder` в `ByteBuffer`.
- 10 Вспомогательный метод для чтения объекта `ValidatedFoodOrder` из `ByteBuffer`.

Важно отметить, что из-за параллелизма, в общем случае присущего архитектурам потоковой обработки, агрегатор может принимать сообщения, соответствующие различным заказам, и далеко не всегда в надлежащем порядке. Поэтому агрегатор поддерживает внутренний список активных заказов, для которых уже получены сообщения. Если для конкретного идентификатора заказа список отсутствует, то предполагается, что получено первое сообщение в этом наборе, и соответствующая запись добавляется во внутренний список. Для такого списка требуется периодическая очистка, чтобы он не увеличивался до бесконечности, поэтому агрегатор обязательно должен очищать его после завершения процедуры агрегации.

### 8.2.2. Паттерн «динамический маршрутизатор»

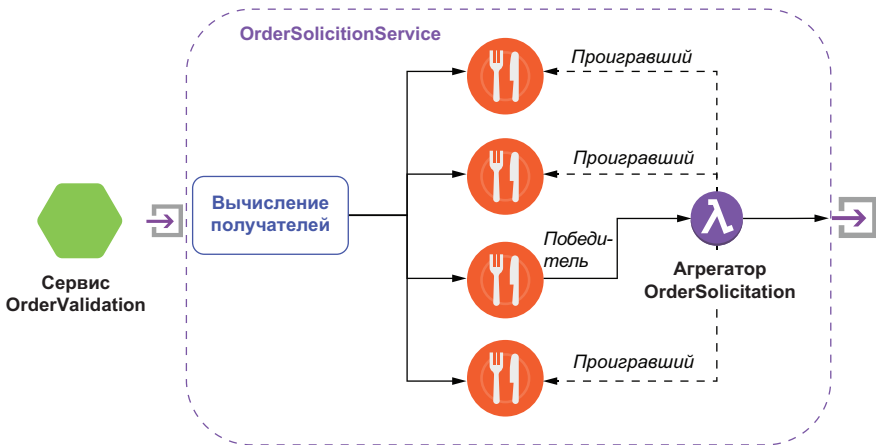
Паттерн «разделитель» удобен, если необходимо обрабатывать различные фрагменты сообщения в параллельном режиме, и вы заранее точно знаете, сколько элементов нужно обработать, а их количество не меняется. Но существуют ситуации, в которых невозможно заранее определить, куда следует направлять сообщения, и вы должны обязательно узнать это на основе содержимого самого сообщения и с учетом некоторых других внешних условий. Примером может служить `OrderSolicitationService` – один из трех микросервисов, вызываемых функцией `OrderValidationService`.

Микросервис `OrderSolicitationService` оповещает подмножество сотрудничающих ресторанов о поступлении заказов на продукты и блюда, которые они могут выполнить, и ждет ответа от каждого ресторана о приеме заказа или отказе. Если заказ принят, то ресторан должен сообщить время, когда можно будет забрать продукты для доставки. Очевидно, что такой список подмножества ресторанов зависит от нескольких факторов. Необходимо организовать маршрутизацию заказов на основе возможности ресторанов предоставить требуемые продукты и блюда (т. е. заказы на бигмаки направляются в Макдоналдс и т. д.). В то же время нежелательна нецеленаправленная широковещательная рассылка заказа в каждый отдельный ресторан Макдоналдс, поэтому список сужается с уче-



том близости ресторанов к адресу доставки. Поскольку такой список составляется в ответ на каждое сообщение (заказ), наилучшим выбором является паттерн «список получателей» (recipient list).

Общий поток данных для `OrderSolicitationService` показан на рис. 8.4, и состоит он из трех отдельных этапов. На первом этапе вычисляется целенаправленный список получателей на основе описанных выше факторов. На втором этапе выполняется итеративный проход по списку получателей, и объект `FoodOrder` отправляется каждому получателю. На последнем третьем этапе сервис ждет ответы от каждого получателя и выбирает победителя для выполнения текущего заказа. После выбора победителя всех прочих получателей оповещают о том, что они проиграли, и текущий заказ продуктов перестает быть доступным.



**Рис. 8.4.** Топологическая схема `OrderSolicitationService`, реализующая паттерн «динамический маршрутизатор»

Практическая реализация такой логики показана в листинге 8.3, и основана она на свойствах сообщений транспортировать метаданные, чрезвычайно важные для агрегатора. В первую очередь в сообщение включается идентификатор заказа, позволяющий определить, с каким объектом `FoodOrder` связан каждый конкретный ответ. Свойство `all-restaurants` используется для обозначения всех кандидатов, выбранных для текущего заказа. Наличие этой информации в сообщении позволяет агрегатору знать все рестораны, которым необходимо отправить сообщение «вы не победили». Последний фрагмент метаданных, содержащийся в свойствах сообщения, — свойство `return-addr`, где хранится имя темы, на которую подписан агрегатор. Это позволяет избежать жесткого кодирования такой информации непосредственно в логике получателя сообщений и дает возможность динамического предоставления необходимых метаданных. В листинге 8.3 показана реализация

паттерна «возвращаемый адрес», описанного в книге «Enterprise Integration Patterns».

**Листинг 8.3.** Реализация паттерна «список получателей» в функции `OrderSolicitationService`

```
public class OrderSolicitationService implements Function<FoodOrder, Void> {
 private String rendezvous = "persistent://restaurants/inbound/accepted";

 @Override
 public Void process(FoodOrder order, Context ctx) throws Exception {

 List<String> cand = getCandidates(order,
 ➔ order.getDeliveryLocation()); ❶

 if (CollectionUtils.isEmpty(cand)) {
 String all = StringUtils.join(cand, ",");
 int delay = 0;
 for (String topic: cand) { ❷
 try {
 ctx.newOutputMessage(topic, AvroSchema.of(FoodOrder.class))
 .property("order-id", order.getMeta().getOrderId() + "") ❸
 .property("all-restaurants", all) ❹
 .property("return-addr", rendezvous) ❺
 .value(order).deliverAfter((delay++ * 10), TimeUnit.SECONDS); ❻
 } catch (PulsarClientException e) {
 e.printStackTrace();
 }
 }
 }
 return null;
 }

 private List<String> getCandidates(FoodOrder order, Address deliveryAddr) {
 ... ❷
 }
}
```

- ❶ Создание списка получателей на основе заказа и адреса доставки.
- ❷ Отправка объекта `FoodOrder` каждому получателю в списке.
- ❸ Использование идентификатора заказа для корреляции.
- ❹ Включение всех ресторанов, чтобы можно было оповестить проигравших.
- ❺ Указание каждому получателю, куда отправлять сообщение с ответом.
- ❻ Регулирование доставки сообщений для минимизации количества отрицательных ответов.
- ❼ Логика для создания списка получателей.

Возвращаемый список получателей составлен в порядке предпочтительных свойств (например, некоторый ресторан ближе всего к адресу доставки, следующий принимал наименьшее участие в де-

ловой активности сегодня и т. п.), и мы используем возможности отложенной доставки сообщений Pulsar для отправки запросов на выполнение с интервалами. Цель такого подхода – минимизировать количество отказов, которые мы вынуждены передавать ресторанам, принявшим `FoodOrder`. Мы не должны затруднять работу ресторанов-партнеров, заваливая их заказами, которые они готовы принять, но в итоге приходится их отменять. Поэтому мы регулируем количество оповещаемых ресторанов и отправляем сообщения медленно, чтобы избежать чрезмерного количества одновременных предложений.

Поскольку `OrderSolicitationService` может отправлять объект `FoodOrder` нескольким получателям, потребуется согласование ответов и передача заказа только одному из ответивших. Для этого доступно множество стратегий, но сейчас мы будем просто принимать первый ответ. Такая логика согласования реализуется с использованием агрегатора, похожего на ранее примененный в `OverValidationService`. В листинге 8.3 можно видеть, что здесь используются свойства сообщений для передачи имени темы, в которую все получатели должны отправлять ответы. Соответствующий агрегатор необходимо сконфигурировать для прослушивания этой темы, чтобы он принимал ответные сообщения и мог оповестить проигравшие рестораны о том, что заказ был передан ресторану-победителю. Затем мобильное приложение ресторатора может отреагировать на оповещение о проигрыше, удалив заказ из рассмотрения.

#### Листинг 8.4. Реализация паттерна «агрегатор» в функции `OrderSolicitationService`

```
public class OrderSolicitationAggregator implements
 Function<SolicitationResponse, Void> {

 @Override
 public Void process(SolicitationResponse response, Context context)
 throws Exception {

 Map<String, String> props = context.getCurrentRecord().getProperties();
 String correlationId = props.get("order-id");
 List<String> bids = Arrays.asList(
 StringUtils.split(props.get("all-restaurants")));

 if (context.getState(correlationId) == null) {
 // Первый ответ побеждает.
 String winner = props.get("restaurant-id");
 bids.remove(winner);
 notifyWinner(winner, context);
 notifyLosers(bids, context);

 // Запись метки времени получения ответа победителя.
 ByteBuffer bb = ByteBuffer.allocate(32);
```

```
 bb.asLongBuffer().put(System.currentTimeMillis());
 context.putState(correlationId, bb);
 }

 return null;
}

private void notifyLosers(List<String> bids, Context context) {
 ...
}

private void notifyWinner(String s, Context context) {
 ...
}
}
```

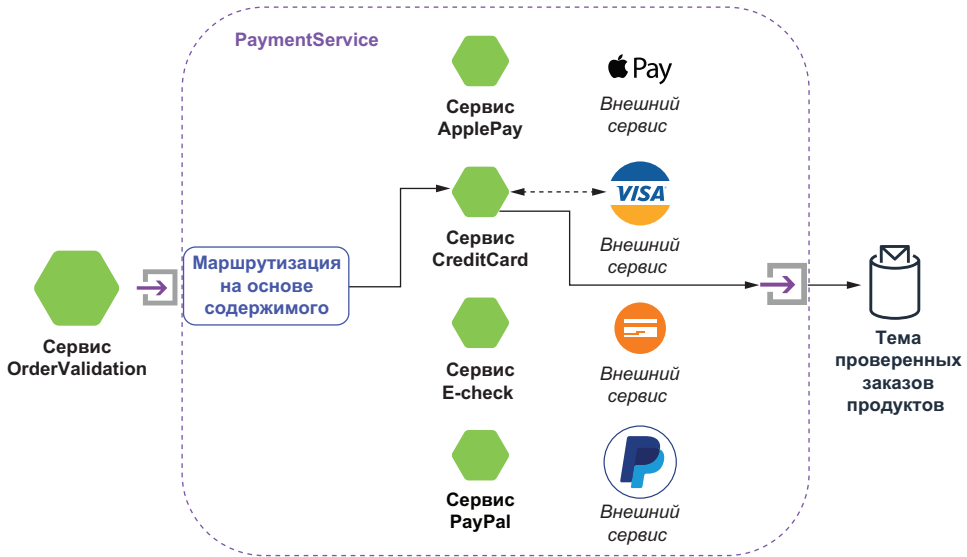
- ❶ Декодирование идентификаторов всех ресторанов-кандидатов.
- ❷ Первый полученный ответ побеждает.
- ❸ Получение идентификатора ресторана-победителя из ответного сообщения.
- ❹ Удаление победителя из списка всех ресторанов (кандидатов).
- ❺ Отправка сообщения ресторану-победителю о том, что заказ передан ему.
- ❻ Отправка сообщений всем проигравшим ресторанам.

В листинге 8.4 можно видеть, что сохраняется корреляция по идентификатору заказа, но критерием завершения становится правило «первый ответивший побеждает» вместо ожидания ответа от всех получателей сообщения, как это делалось в паттерне «разделитель». Несмотря на то что мы делаем все возможное, чтобы предотвратить отправку множества ненужных сообщений, агрегатор все же должен учесть и этот сценарий. Это достигается за счет сохранения метки времени определения победителя для каждого заказа, что позволяет агрегатору игнорировать все последующие ответы по данному заказу, так как точно известно, что конкретный ресторан уже выиграл. Чтобы соответствующая структура данных не становилась слишком большой и не приводила к исчерпанию доступной памяти, в код был добавлен фоновый процесс, который периодически просыпается и удаляет из списка все записи старше конкретной метки времени, соответствующей моменту фиксации победителя.

### 8.2.3. Паттерн «маршрутизатор на основе содержимого»

Маршрутизатор на основе содержимого использует содержимое сообщений, чтобы определить, в какую тему направить то или иное сообщение. Основная концепция заключается в том, что содержимое каждого сообщения проверяется, а затем сообщение перенаправляется в конкретную целевую локацию на основе значений, найденных или не найденных в содержимом. Для варианта использования проверки заказа функция `PaymentService` принимает сообщения с различным содержанием в зависимости от способа платежа, используемого клиентом.

В настоящее время система поддерживает платежи кредитными картами PayPal, Apple Pay и электронными чеками. Каждый из этих методов оплаты обязательно должен проверяться различными внешними системами. Таким образом, главная задача PaymentService заключается в перенаправлении сообщения в соответствующую систему на основе его содержимого. На рис. 8.5 показан сценарий, в котором методом оплаты заказа является кредитная карта, а подробности платежа перенаправляются в CreditCardService.



**Рис. 8.5.** Топологическая схема PaymentService с реализацией паттерна «маршрутизатор на основе содержимого» направляет информацию о платеже в соответствующий сервис в зависимости от метода оплаты, используемого для текущего заказа

Для каждого поддерживаемого способа оплаты существует связанный с ним промежуточный микросервис (например, ApplePayService, CreditCardService и т. д.), конфигурируемый с соответствующими идентификационными данными, конечной точкой и т. д. Затем каждый промежуточный микросервис вызывает требуемый внешний сервис для авторизации платежа, а после приема авторизации перенаправляет ответ в агрегатор сервиса OrderValidationService, где этот ответ объединяется с другими ответами, связанными с текущим заказом. В листинге 8.5 показана реализация паттерна «маршрутизатор на основе содержимого» в функции PaymentService.

**Листинг 8.5.** Реализация паттерна «маршрутизатор на основе содержимого» в функции `PaymentService`

```

public class PaymentService implements Function<Payment, Void> {
 private boolean initialized = false;
 private String applePayTopic, creditCardTopic, echeckTopic, paypalTopic;

 public Void process(Payment pay, Context ctx) throws Exception {
 if (!initialized) {
 init(ctx);
 }

 Class paymentType = pay.getMethodOfPayment().getType().getClass();
 Object payment = pay.getMethodOfPayment().getType();

 if (paymentType == ApplePay.class) {
 ctx.newOutputMessage(applePayTopic, AvroSchema.of(ApplePay.class))
 .properties(ctx.getCurrentRecord().getProperties())
 .value((ApplePay) payment).sendAsync();
 } else if (paymentType == CreditCard.class) {
 ctx.newOutputMessage(creditCardTopic, AvroSchema.of(CreditCard.class))
 .properties(ctx.getCurrentRecord().getProperties())
 .value((CreditCard) payment).sendAsync();
 } else if (paymentType == ElectronicCheck.class) {
 ctx.newOutputMessage(echeckTopic, AvroSchema.of(ElectronicCheck.class))
 .properties(ctx.getCurrentRecord().getProperties())
 .value((ElectronicCheck) payment).sendAsync();
 } else if (paymentType == PayPal.class) {
 ctx.newOutputMessage(paypalTopic, AvroSchema.of(PayPal.class))
 .properties(ctx.getCurrentRecord().getProperties())
 .value((PayPal) payment).sendAsync();
 } else {
 ctx.getCurrentRecord().fail();
 }

 return null;
 }

 private void init(Context ctx) {
 applePayTopic = (String)ctx.getUserConfigValue("apple-pay-topic").get();
 creditCardTopic = (String)ctx.getUserConfigValue("credit-topic").get();
 echeckTopic = (String)ctx.getUserConfigValue("e-check-topic").get();
 paypalTopic = (String)ctx.getUserConfigValue("paypal-topic").get();
 initialized = true;
 }
}

```

- ① Отправка объектов `ApplePay` в сервис `ApplePayService`.
- ② Отправка объектов `CreditCard` в сервис `CreditCardService`.
- ③ Отправка объектов `ElectronicCheck` в сервис `ElectronicCheckService`.
- ④ Отправка объектов `PayPal` в сервис `PayPalService`.
- ⑤ Отказ для любого другого способа оплаты.
- ⑥ Конфигурирование исходящих тем для промежуточных сервисов.

После получения сервисом `CreditCardService` числового кода авторизации для текущей транзакции далее он пересылается в тему `validatedFoodOrder`, так как нам потребуется этот код авторизации для приема денежных средств.

### 8.3. Паттерны преобразования сообщений

К типичным примерам обработки потоковых данных относятся сенсорные датчики интернета вещей, журналы сервера и системы обеспечения безопасности, реклама в реальном времени и данные посещаемости из мобильных приложений и с веб-сайтов. В каждом из этих сценариев имеются источники данных, непрерывно генерирующие тысячи и даже миллионы неструктурированных или частично структурированных элементов данных, – чаще всего простой текст, JSON или XML. Каждый такой элемент данных должен быть преобразован в формат, пригодный для обработки и анализа.

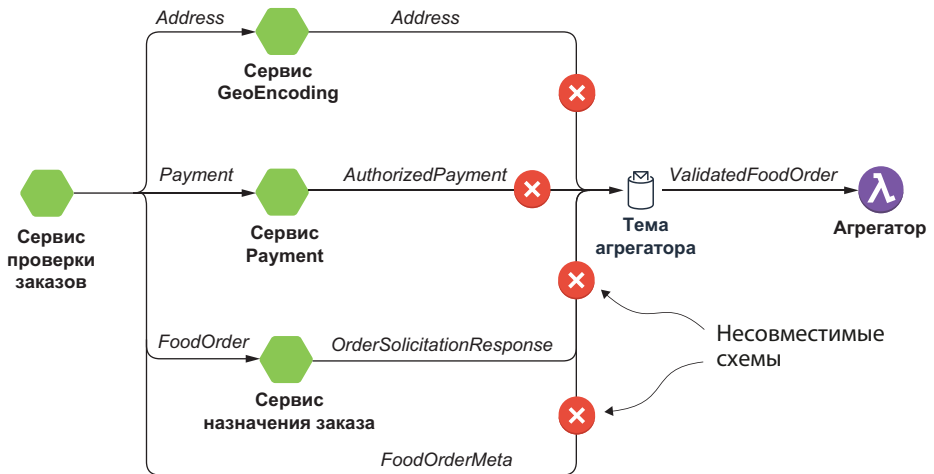
Эта категория обработки является общей на всех платформах потоковой обработки, и подобные задачи преобразования данных аналогичны более привычному типу обработки ETL (*extract, transform, load* – извлечение, преобразование, загрузка), поскольку главная цель – обеспечение нормализации, улучшения и преобразование потребляемых данных в формат, более подходящий для обработки. Паттерны преобразования сообщений используются для работы с содержимым сообщений во время их прохождения по DAG-топологии, чтобы выполнить перечисленные выше типы задач внутри конкретной применяемой архитектуры потоковой обработки. Каждый паттерн, рассматриваемый в этом разделе, предоставляет тщательно проверенные на практике правила для динамического преобразования сообщений.

#### 8.3.1. Паттерн «транслятор сообщений»

Как мы уже видели ранее, `OrderValidationService` выполняет несколько асинхронных вызовов различных сервисов, каждый из которых генерирует сообщения с разными типами схемы. Все эти сообщения необходимо объединить с помощью `OrderValidationAggregator` в один ответ. Но функцию Pulsar можно определить для приема входящих сообщений только одного типа, поэтому невозможно публиковать сообщения разных типов напрямую во входящей теме сервиса, как показано на рис. 8.6, поскольку схемы не являются совместимыми.

Чтобы организовать потребление сообщений с различными схемами, результаты, получаемые от промежуточных микросервисов, необходимо пропустить через функцию транслятора сообщений, выполняющую преобразование ответных сообщений

в тип, соответствующий типу входящей темы `OrderValidationAggregator`, который в данном случае представляет схему, показанную в листинге 8.6. Для использования выбран тип объекта, скомпонованный из типов ответных сообщений. Такой подход позволяет просто копировать ответ от каждого промежуточного сервиса напрямую в соответствующее поле для заполнения в объекте `ValidatedFoodOrder`.



**Рис. 8.6.** Каждый промежуточный микросервис генерирует сообщения с собственным типом схемы, отличающимся от других. Поэтому они не могут напрямую публиковать сообщения во входящей теме сервиса `OrderValidationService`

#### Листинг 8.6. Определение схемы `ValidatedFoodOrder`

```
record ValidatedFoodOrder {
 FoodOrderMeta meta;
 com.gottaeat.domain.restaurant.SolicitationResponse food;
 com.gottaeat.domain.common.Address delivery_location;
 com.gottaeat.domain.payment.AuthorizedPayment payment;
}
```

- ❶ Метаданные `FoodOrderMeta` направлены из `OrderValidationService`.
- ❷ Тип ответа из `OrderSolicitationService`.
- ❸ Тип ответа из `GeoEncodingService`.
- ❹ Тип ответа из `PaymentService`.

Хотя логика для потребления каждого типа ответного сообщения слегка различна, концепция, по существу, одинакова. В листинге 8.7 можно видеть, что логика для обработки сообщений `AuthorizedPayment`, генерируемых `PaymentService`, весьма проста. Создается объект нужного типа, а объект `AuthorizedPayment`, опублико-



ванный сервисом `PaymentService`, полностью копируется в соответствующее поле в объекте-обертке перед отправкой его в агрегатор для потребления.

#### Листинг 8.7. Реализация транслятора `PaymentAdaptor`

```
public class PaymentAdapter implements Function<AuthorizedPayment, Void> {

 @Override
 public Void process(AuthorizedPayment payment,
 ➔ Context ctx) throws Exception {
 ValidatedFoodOrder result = new ValidatedFoodOrder();
 result.setPayment(payment);

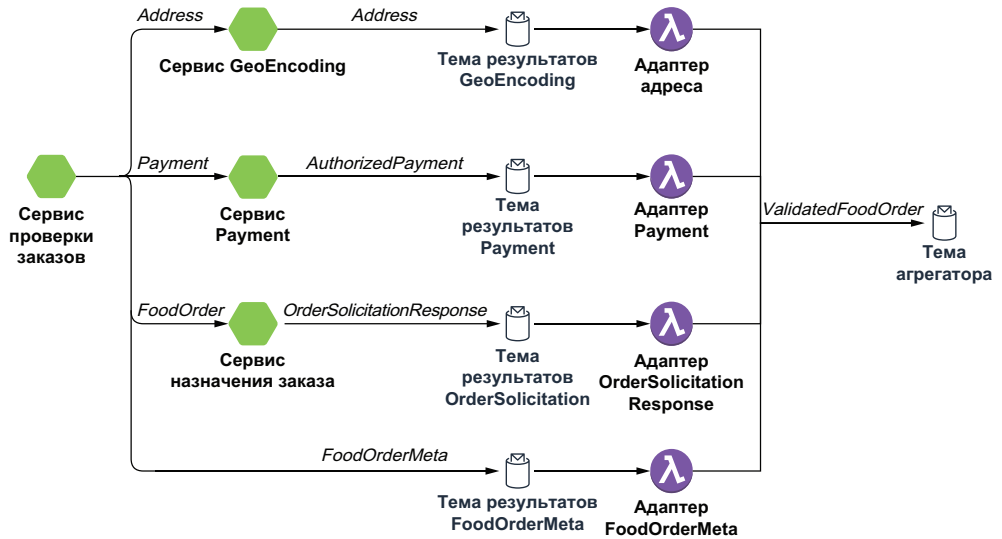
 ctx.newOutputMessage(ctx.getOutputTopic(),
 AvroSchema.of(ValidatedFoodOrder.class))
 .properties(ctx.getCurrentRecord().getProperties())
 .value(result).send();

 return null;
 }
}
```

- ❶ Создание нового объекта-обертки.
- ❷ Обновление поля `payment` данными `AuthorizedPayment`.
- ❸ Публикация сообщения типа `ValidatedFoodOrder` во входящей теме агрегатора.
- ❹ Копирование идентификатора заказа для обеспечения возможности его связывания с другими ответными сообщениями.

Для всех прочих микросервисов также существуют соответствующие адаптеры (трансляторы). После заполнения всех полей объекта полученными значениями заказ считается проверенным и может быть опубликован в теме `validatedFoodOrder` для дальнейшей обработки.

Следует отметить, что каждая функция адаптера должна выполнять потребление из темы, которая содержит ответные сообщения от соответствующего микросервиса, до публикации объектов-обертки во входящей теме сервиса `OrderValidationAggregator`. Следовательно, потребуется создание таких тем ответов и конфигурирование микросервисов для публикации в них, как показано на рис. 8.7. Помимо того, что этот паттерн полезен в ситуациях, когда необходимо объединять результаты, получаемые из нескольких различных функций Pulsar, его также можно использовать для регулирования потребления данных из внешних систем, например из баз данных, с преобразованием в требуемую схему формата для дальнейшей обработки.

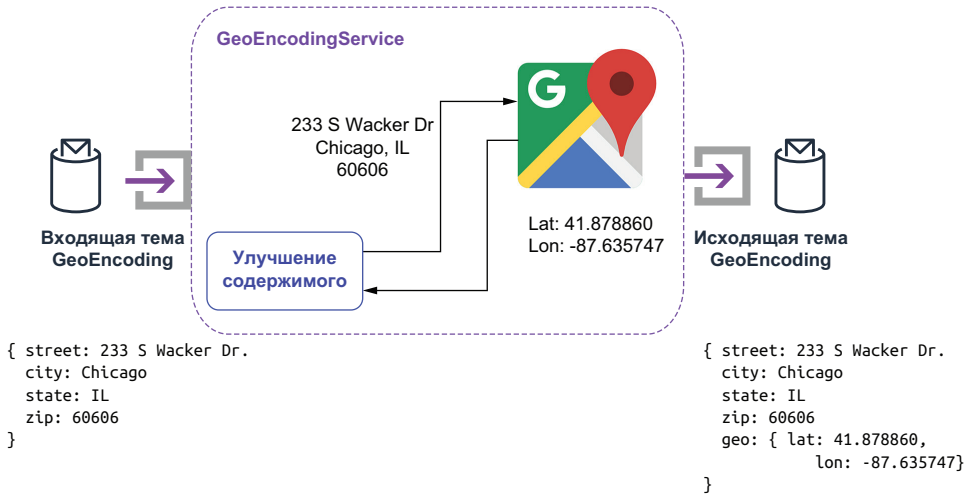


**Рис. 8.7.** Каждый промежуточный микросервис публикует свои ответные сообщения в темах с соответствующим типом схемы. Затем связанные с ними функции-адаптеры потребляют эти сообщения и выполняют преобразование в тип схемы ValidatedFoodOrder, ожидаемый в теме агрегатора

### 8.3.2. Паттерн «улучшение содержимого»

При обработке потоковых событий часто удобно улучшать входящее сообщение, добавляя в него дополнительную информацию, которая требуется для принимающей системы. В рассматриваемом здесь примере событие прихода заказа клиента в `OrderValidationService` содержит необработанный и непроверенный адрес, но для потребляющих микросервисов также требуется геокодированная локация с парой широта–долгота, чтобы обеспечить создание маршрута для водителей, доставляющих заказы.

Этот тип задачи обычно решается с помощью паттерна «улучшение содержимого». Данный термин обозначает процесс, использующий информацию внутри входящего сообщения для дополнения его исходного содержимого новой информацией. В рассматриваемом здесь варианте использования мы будем извлекать данные из внешнего источника и добавлять их в исходящее сообщение, как показано на рис. 8.8. Сервис `GeoEncodingService` принимает подробный адрес доставки, предоставленный клиентом, и передает его в веб-сервис `Google Maps API`. Затем полученные значения широты и долготы, соответствующие этому адресу, включаются в исходящее сообщение.



**Рис. 8.8.** GeoEncodingService реализует паттерн «улучшение содержимого», используя предоставленный адрес доставки для поиска соответствующей широты и долготы, и добавляет эти данные в объект адреса

В листинге 8.8 показана реализация GeoEncoderService, которая вызывает внешний сервис Google Maps и дополняет входящий объект Address связанными с ним значениями широты и долготы.

**Листинг 8.8.** Реализация паттерна «улучшение содержимого» в GeoEncoderService

```

public class GeoEncoderService implements Function<Address, Address> {
 boolean initialized = false;
 String apiKey;
 GeoApiContext geoContext;

 public Void process(Address addr, Context context) throws Exception {
 if (!initialized) {
 init(context);
 }

 Address result = new Address();
 result.setCity(addr.getCity());
 result.setState(addr.getState());
 result.setStreet(addr.getStreet());
 result.setZip(addr.getZip());

 try {
 GeocodingResult[] results =
 GeocodingApi.geocode(geoContext, formatAddress(addr)).await();

 if (results != null && results[0].geometry != null) {
 Geometry geo = results[0].geometry;
 LatLon ll = new LatLon();

```

```
 ll.setLatitude(geo.location.lat);
 ll.setLongitude(geo.location.lng);
 result.setGeo(ll);
 }

 } catch (InterruptedException | IOException | ApiException ex) {
 context.getCurrentRecord().fail();
 context.getLogger().error(ex.getMessage());
 } finally {
 return result;
 }
}

private void init(Context context) {
 apiKey = context.getUserConfigValue("apiKey").toString();
 geoContext = new GeoApiContext.Builder()
 .apiKey(apiKey).maxRetries(3)
 .retryTimeout(3000, TimeUnit.MILLISECONDS).build();
 initialized = true;
}
}
```

Сервис применяет ключ API `apiKey`, предоставляемый конфигурацией для вызова веб-сервиса Google Maps, и использует первый полученный ответ как источник значений широты и долготы. Если обращение к веб-сервису завершилось неудачно, то мы фиксируем ошибку сообщения, чтобы можно было повторить попытку позже.

Хотя использование внешнего сервиса является одним из наиболее часто применяемых вариантов для паттерна «улучшение содержимого», также существуют реализации, которые сами выполняют внутренние вычисления на основе содержимого сообщений, например вычисление размера сообщения или хеша MD5 по его содержимому, и добавляют результат в свойства того же сообщения. Это позволяет потребителю убедиться в том, что содержимое сообщения не было изменено.

Другой часто встречающийся вариант использования – получение текущего времени от операционной системы и включение этой метки времени в сообщение для точного определения времени его приема или обработки. Такая информация полезна для сохранения правильного порядка сообщений, если необходимо обрабатывать сообщения в порядке их поступления, или для определения узких мест в DAG-схеме, если добавлять получаемые метки времени на каждом шаге процесса.

### 8.3.3. Паттерн «фильтр содержимого»

Часто оказывается полезным удаление или маскирование одного или нескольких элементов данных во входящем сообщении для обеспечения безопасности или из каких-либо других соображений.

Паттерн «фильтр содержимого» в сущности является противоположностью паттерна «улучшение содержимого», поскольку предназначен для выполнения преобразования, связанного с уменьшением объема информации.

Рассмотренный в предыдущих разделах сервис `OrderValidation-Service` представляет собой пример фильтра содержимого, который разделяет входящее сообщение `FoodOrder` на более мелкие фрагменты перед перенаправлением их в соответствующие микросервисы для обработки. Эта операция не только минимизирует объем данных, отправляемых в каждый сервис, но также скрывает секретную информацию о платеже от всех прочих сервисов, которым не требуется доступ к такой информации.

Рассмотрим другой сценарий, в котором событие содержит секретный элемент данных, например номер кредитной карты. Фильтр содержимого может обнаруживать такие шаблоны в сообщениях и полностью удалять этот элемент данных, скрывать его с помощью функции одностороннего (необратимого) хеширования, например SHA-256, или зашифровывать это поле данных.

## 8.4. Резюме

- Pulsar Functions – это фреймворк распределенной обработки, который хорошо подходит для программирования Dataflow, когда данные обрабатываются поэтапно, и процедуры обработки могут выполняться в параллельном режиме, как на линии конвейерной сборки.
- Приложения на основе Pulsar Functions можно моделировать как конвейеры данных, в которых функции выполняют вычисления и перенаправляют данных, используя входящие/исходящие темы.
- При проектировании приложения, передающего сообщения, с применением микросервисов часто используются паттерны проектирования, например те, что можно найти в книге «Enterprise Integration Patterns» – авторы Грегор Хопе (Gregor Hohpe) и Бобби Вулф (Bobby Woolf) – и в других источниках.
- Надежные, испытанные паттерны обработки сообщений можно реализовать с использованием Pulsar Functions, что позволит вам применять проверенные временем решения в своих приложениях.

# Паттерны устойчивости



## Темы:

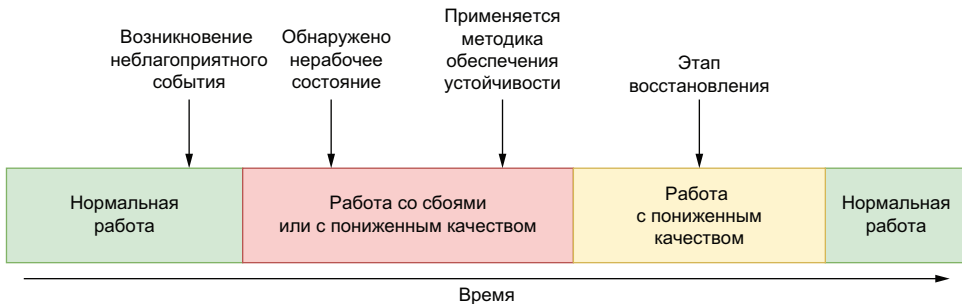
- как сделать приложения на основе Pulsar Functions устойчивыми к неблагоприятным событиям;
- реализация надежных и проверенных паттернов устойчивости с использованием Pulsar Functions.

С учетом микросервисной архитектуры регистрации заказов в предприятии GottaEat ваша главная задача – разработать систему, способную принимать входящие заказы продуктов питания и блюд от клиентов 24 ч в сутки, 7 дней в неделю с обеспечением времени ответа, приемлемого для любого клиента. Система должна быть доступной постоянно, иначе компания не только потеряет доходы и клиентов, но и ее репутация будет безнадежно испорчена. Следовательно, вы должны спроектировать систему так, чтобы она обладала высокой доступностью и устойчивостью для обеспечения непрерывности обслуживания. Каждый желает, чтобы его система была устойчивой, но что в действительности означает этот термин? Устойчивость (resilience) – это способность системы противостоять любым сбоям, вызванным неблагоприятными событиями и условиями, с сохранением приемлемого уровня производительности, определяемого любым количеством числовых

метрик, например доступность, мощность, производительность, надежность, отказоустойчивость и удобство использования.

Устойчивость важна, потому что вне зависимости от того, насколько правильно спроектировано конкретное приложение Pulsar, может возникнуть непредвиденный случай, например отключение электроэнергии или отказ сетевых каналов обмена данными, который нарушит топологическую схему приложения. Здесь неявно подразумевается, что неблагоприятные события и условия непременно будут возникать. В действительности правильным вопросом является не «если», а «когда». Критерий устойчивости определяет, что именно делает программное обеспечение при возникновении неблагоприятных событий, нарушающих работоспособность. Способны ли функции Pulsar обнаруживать такие события и условия? Смогут ли функции отреагировать надлежащим образом после обнаружения подобных нарушений? Обеспечивает ли функция приемлемое восстановление после сбоя?

Система с высокой устойчивостью применяет некоторые методики оперативного обеспечения устойчивости, реагирующие на неблагоприятные события, для динамического обнаружения подобных случаев и ответной реакции на них для автоматического возврата системы в нормальное рабочее состояние, как показано на рис. 9.1. Эти методики особенно полезны в среде потоковой обработки, где любое нарушение обслуживания может привести к тому, что данные не принимаются из источника и начинаются их потери.



**Рис. 9.1.** Устойчивая система автоматически обнаруживает неблагоприятные события или условия и предпринимает превентивные меры для возврата в нормальное рабочее состояние

Очевидно, что главным условием применения методики оперативного обеспечения устойчивости является способность обнаруживать неблагоприятные условия. В этой главе рассматривается, как обнаруживать условия, нарушающие работу, в приложениях Pulsar Functions, а также некоторые методики обеспечения устойчивости, которые можно применять в функциях Pulsar, чтобы сделать их более надежными.

## 9.1. Устойчивость Pulsar Functions

В главе 8 мы узнали, что приложения на основе Pulsar Functions по существу представляют собой топологические схемы, состоящие из нескольких отдельных функций Pulsar, взаимодействующих между собой через входящие/исходящие темы. Эти топологические схемы можно представить в виде направленных ациклических графов (DAG) с данными, проходящими по различным путям в зависимости от значений, содержащихся в сообщениях. С точки зрения устойчивости имеет смысл рассматривать весь DAG в целом как систему, которая должна быть изолирована от воздействия неблагоприятных условий.

### 9.1.1. Неблагоприятные события

Чтобы реализовать общую стратегию обеспечения устойчивости, в первую очередь необходимо определить все неблагоприятные события, которые предположительно могут воздействовать на топологию работающего приложения Pulsar. Вместо попыток составить список всех конкретных возможных событий, которые могут возникнуть в функции Pulsar, мы классифицируем неблагоприятные события по таким категориям, как полный отказ («смерть») функции, замедление работы функции и функция, не продолжающая работу.

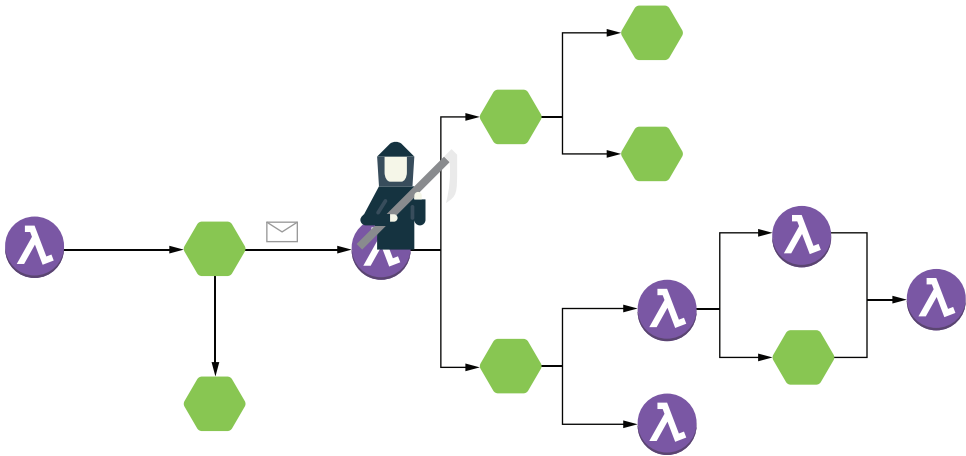
#### Полный отказ функции

Начнем с самого критического события – полного отказа («смерти») функции, возникающего, когда функция Pulsar в топологии приложения внезапно завершает работу. Это условие может быть вызвано любым количеством физических факторов, например критическим сбоем сервера, отключением электроэнергии, или нефизическими факторами, такими как отсутствие свободной памяти в самой функции. Вне зависимости от внутренней причины воздействие на систему может оказаться весьма серьезным, поскольку поток данных, проходящий через DAG, внезапно обрывается.

Если функция, показанная на рис. 9.2, «умирает», то все зависящие от нее потребители прекращают принимать сообщения, и в итоге DAG блокируется в этой точке потока. С точки зрения конечного пользователя, приложение прекратило производство выходных данных, а это в нашем случае означает, что весь процесс регистрации заказов продуктов резко остановился.

К счастью, входящая тема `functions` будет действовать как буфер и сохранять входящие сообщения до достижения предельного их количества, определяемого стратегией хранения сообщений для этой темы. Я отметил только это ограничение, чтобы обратить ваше внимание на важность устранения описанной проблемы как можно быстрее, поскольку может начаться потеря сообщений, если функцию вовремя не перезапустить.





**Рис. 9.2.** Когда функция «умирает» и потребление сообщений останавливается, все приложение в итоге блокируется в этой точке DAG, т. е. останавливается вся последующая обработка

Если вы используете рекомендуемую среду времени выполнения Kubernetes для управления своими функциями Pulsar, то фреймворк Pulsar Functions будет автоматически обнаруживать критический сбой соответствующего пода Kubernetes и перезапускать его, как только получит достаточный объем вычислительных ресурсов в текущей среде Kubernetes. Также необходимо обеспечить правильный мониторинг и подсистему предупреждений и оповещений для обнаружения критического отказа физического хоста и соответствующей реакции как дополнительный уровень обеспечения устойчивости.

### Замедление работы функции

Другое условие, оказывающее отрицательное воздействие на производительность приложения Pulsar, – замедление работы функции (lagging function; дословно: замедленная функция), которое возникает из-за непрерывного прибытия с высокой скоростью сообщений во входящую тему функции. При этом скорость прибытия превышает скорость обработки, поддерживаемую данной функцией. При таких условиях функция все больше и больше опаздывает с обработкой входящих сообщений, что приводит к постоянно увеличивающемуся разрыву между временем готовности сообщения к обработке и временем начала его реальной обработки.

Рассмотрим сценарий, в котором функция *A* публикует 150 событий в секунду во входящей теме другой функции, как показано на рис. 9.3. К сожалению, функция *B* может обрабатывать только 75 событий в секунду, создавая разрыв размером 75 событий в секунду, который буферизуется внутри темы. Если этот дисбаланс при

обработке остается неизменным, то объем сообщений, хранящихся в буфере внутри входящей темы, будет постоянно увеличиваться.



**Рис. 9.3.** Функция В способна обработать только 75 событий в секунду, а функция А производит события со скоростью 150 в секунду. Это приводит к отставанию в размере 75 событий в секунду, т. е. с каждой прошедшей секундой добавляется одна секунда задержки

Сначала эта задержка начинает воздействовать на соглашение о качестве обслуживания (SLA), и в рассматриваемом примере процесс регистрации заказов становится слишком медленным для клиентов, так как они вынуждены ждать, когда их заказы будут обработаны и подтверждены. В конце концов задержка оказывается настолько длительной, что клиенты отменяют свои заказы из-за отсутствия ответа на них в мобильном приложении. Это может создавать ситуации, в которых заказ клиента помещен в очередь и обработан уже после того, как клиент, не дождавшись ответа, решил, что его заказ так и не был принят, и отказался от него – это настоящий кошмар с точки зрения бизнеса, так как нам придется возвращать оплаченные суммы и оповещать ресторан о том, что заказ был отменен.

Чтобы подкрепить эти рассуждения некоторыми числовыми данными, рассмотрим воображаемый сценарий, в котором описанное выше условие возникло в топологии DAG сервиса проверки заказа во время пиковой нагрузки бизнес-активности, например в пятницу около семи часов вечера. Заказы, принимаемые десятью минутами позже, регистрируются после 45 000 ( $75 \text{ событий/с} \times 60 \text{ с} \times 10 \text{ мин}$ ) других заказов, созданных во входящей теме функции В. При скорости обработки 75 сообщений в секунду потребуется 10 мин для обработки уже существующих в теме сообщений, прежде чем будет наконец обработан новый заказ. Таким образом, заказ, размещенный в 7:10, будет обработан не раньше 7:20! Следовательно, для соблюдения соглашения о качестве обслуживания и недопущения отмены заказов из-за серьезного замедления работы функции необходимо протестировать производительность, чтобы определить среднюю скорость обработки каждой функции в сервисе проверки заказов и обеспечить непрерывный мониторинг, чтобы в дальнейшем обнаруживать любой дисбаланс при обработке.

К счастью, если вы используете рекомендуемую среду времени выполнения Kubernetes для управления своими функциями Pulsar, то фреймворк Pulsar Functions позволит регулировать степень рас-

параллеливания для любой функции с помощью команды, показанной в листинге 9.1, которая поможет устранить дисбаланс. Таким образом, средством устранения такого неблагоприятного события является обновление счетчика распараллеливания конкретной (проблемной) функции до приемлемого уровня. Поскольку в приведенном выше вымышленном примере скорости ввода и потребления сообщений относятся как 2:1, потребуется увеличение количества экземпляров функции *B* по отношению к функции *A* как минимум в таком же соотношении, если не больше.

#### Листинг 9.1. Увеличение степени распараллеливания функции

```
$ bin/pulsar-admin functions update \
 --name FunctionA \
 --parallelism 5 \
 ...
```

Хотя настройка отношения количества экземпляров, в точности равная 2:1, теоретически должна устранить проблему, рекомендуется создать один-два дополнительных экземпляра сверх этого отношения на стороне потребителя. Это не только увеличит мощность вычислений, позволяя приложению обрабатывать любой резкий скачок в потоке сообщений, но также сделает функцию устойчивой к отсутствию одного или нескольких экземпляров, прежде чем возникнет какая-либо задержка.

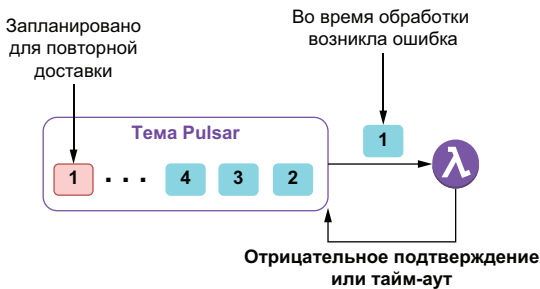
#### Функция, не продолжающая работу

Функция, не продолжающая работу (non-progressing function), отличается от замедленной функции по двум характеристикам. Первая – способность успешно обрабатывать сообщения. В замедленной функции все сообщения обрабатываются успешно, хотя и чрезвычайно медленно, тогда как в функции, не продолжающей работу, некоторые или все сообщения невозможно обработать успешно.

Второй характеристикой является способ устранения возникшей проблемы. Для замедленной функции чаще всего применяется увеличение степени параллельности выполнения экземпляров функции для соответствия объему входящих сообщений. К сожалению, для функции, не продолжающей работу, не существует простого способа устранения проблемы, и единственным решением становится добавление логики обработки непосредственно в функцию Pulsar для выявления и ответной реакции на подобные исключения при обработке. Поэтому вернемся немного назад и рассмотрим ограниченное количество вариантов, имеющихся в нашем распоряжении для регулирования исключений при обработке внутри функции Pulsar.

Можно просто полностью проигнорировать ошибку и явно подтвердить сообщение, оповещая брокер о том, что обработка

выполнена. Очевидно, что это неприемлемое решение для некоторых вариантов использования, например для сервиса проверки платежа в нашем примере, где обязательно требуется авторизованный платеж для продолжения обработки заказа. Другой вариант – отрицательное подтверждение (negative ack) сообщения в блоке catch внутри функции Pulsar, оповещающее брокер о необходимости повторной доставки сообщения несколько позже, но при этом существует вероятность того, что подтверждение вообще не будет передано из функции из-за непрехваченного исключения или сетевого тайм-аута при вызове удаленного сервиса. На рис. 9.4 можно видеть, что в любом случае такие сообщения будут помечены для повторной доставки.



**Рис. 9.4.** Если функция Pulsar отправляет отрицательное подтверждение или не может передать подтверждение в течение интервала времени, определенного конфигурацией, то планируется повторная доставка этого сообщения после минутной паузы

Если в одной теме накапливается все больше и больше таких отрицательно подтвержденных сообщений, то система постепенно замедляет работу, так как выполняются повторные попытки, снова возникают ошибки, и попытки повторяются. При этом вхолостую расходуются циклы обработки на непродуктивные действия, но еще хуже то, что такие сообщения никогда не будут удалены из темы и их воздействие непрерывно – отсюда и термин «функция, не продолжающая работу», поскольку нет никакой возможности продолжить обработку новых сообщений.

Вернемся к сценарию, в котором функция может успешно обрабатывать только 75 событий в секунду и выполняет вызов удаленного сервиса, например сервиса авторизации кредитной карты. Кроме того, конечной точкой, вызываемой этой функцией, в действительности является балансировщик нагрузки, распределяющий вызовы по трем отдельным экземплярам сервиса, но один из них не работает. Каждое третье сообщение сразу же генерирует исключение и подтверждается отрицательно, в результате чего приблизительно для 25 событий в секунду обработка не продолжается. Это приводит к снижению скорости обработки в рассматриваемой

функции с 75 до 50 событий/с. Такое снижение скорости обработки является первым характерным признаком функции, не продолжающей работу.

Увеличение количества экземпляров не решит эту проблему, потому что при этом также пропорционально увеличивается и количество ошибок. Если удвоить степень параллелизма функции для увеличения скорости потребления до 150 событий/с, то в итоге получим 50 ошибочных событий/с, требующих повторной обработки. В действительности не имеет значения, сколько экземпляров мы добавляем, потому что для трети всех сообщений по-прежнему необходима повторная доставка и попытка обработки. Также следует отметить, что для каждого повторно обрабатываемого сообщения возникает еще и неизбежная минутная задержка, оказывающая отрицательное воздействие на предполагаемую скорость ответа приложения.

Теперь рассмотрим воздействие отказа сети на ту же функцию. При каждой операции обработки сообщения выполняется вызов балансировщика нагрузки, но, поскольку сеть не работает, доступ к балансировщику невозможен. В этом случае не только для 100 % сообщений возникает ошибка, но и каждое сообщение ожидает обработки в течение 30 с, так как соответствующее исключение генерируется по истечении сетевого тайм-аута. Воздействие на топологию DAG будет точно таким же, как при «смерти» функции, но, к сожалению, таковым не является. Вместо этого функция в конце концов переходит в состояние «зомби», и даже ее перезапуск не помогает, так как скрытая проблема является внешней относительно такой функции.

Хотя скрытые проблемы, препятствующие обработке сообщений, могут быть совершенно различными, их можно классифицировать по двум основным категориям на основе вероятности их самоустранения: неустойчивым ошибкам (transient faults) и исчезающим ошибкам (non-transitive faults). К неустойчивым ошибкам относится временное отсутствие сетевых каналов связи с компонентами и сервисами, кратковременная недоступность сервиса или тайм-ауты, возникающие, когда вызываемый функцией Pulsar удаленный сервис занят. Такие типы ошибок часто являются самовосстанавливающимися, и если выполняется повторная попытка обработки сообщения после надлежащей задержки, то, вероятнее всего, она будет успешной. Подобный сценарий возникает, если сервис проверки платежей выполняет вызов внешнего сервиса авторизации кредитной карты в тот интервал, когда удаленный сервис перегружен запросами. Существует высокая вероятность того, что следующий вызов будет успешным, поскольку удаленный сервис вполне мог восстановить нормальную работоспособность.

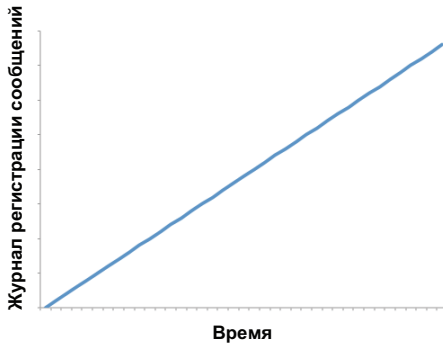
Напротив, неисчезающие ошибки требуют внешнего вмешательства для их устранения, и к этой категории относятся критические отказы аппаратного обеспечения, завершение срока действия секретных идентификационных данных или некорректные данные в сообщении. Рассмотрим сценарий, в котором сервис проверки заказа публикует во входящей теме сервиса проверки платежей сообщение, содержащее некорректную информацию о платеже, например неверный номер кредитной карты. Вне зависимости от того, сколько раз выполняются попытки авторизации для использования такой карты как способа платежа, они всегда будут отвергаться. Другой возможной сценарий – завершился срок действия идентификационных данных для сервиса проверки платежей, следовательно, все попытки авторизации кредитной карты любого клиента завершаются ошибкой.

Часто очень трудно отличить один сценарий от другого, поэтому необходима возможность выявления неисчезающих ошибок, таких как некорректный номер кредитной карты, и отделения их от неустойчивых ошибок, таких как перегруженность удаленного сервиса, чтобы реагировать на них по-разному. К счастью, можно использовать соответствующие типы исключений и другие данные для того, чтобы сделать разумные предположения о природе конкретной ошибки и определить соответствующий комплекс мер по ее устранению.

### 9.1.2. Обнаружение ошибок

Когда возникает необходимость в определении условий возникновения ошибки в топологии Pulsar Functions, требуется оценка скорости прохождения потока данных через всю топологическую схему в целом. Проще говоря, поддерживает ли приложение постоянство объема данных, передаваемых из входящей темы, или этот объем снижается? Поток данных через DAG похож на кровообращение в человеческом теле: он непрерывный и его скорость постоянна. Не должно возникать никаких блокировок, запирающих поток в некоторых конкретных зонах.

Все неблагоприятные события, которые мы рассмотрели к этому моменту, оказывают практически одинаковое воздействие на поток данных: непрерывное увеличение количества необработанных сообщений. В Pulsar главной метрикой, характеризующей такую блокировку, является журнал регистрации сообщений (message backlog). Непрерывное увеличение размера журнала регистрации сообщений во входящей теме приложения Pulsar – это явный признак замедленной или ошибочной операции. Хочу уточнить, я имею в виду не абсолютное количество сообщений в журнале регистрации, а тренд роста их количества за некоторый интервал времени, например в часы пиковой бизнес-деятельности.



**Рис. 9.5.** Условие, при котором размер журнала регистрации сообщений непрерывно возрастает со временем, обозначается термином «замедленная обратная реакция» (backpressure), который означает снижение производительности функции или приложения Pulsar

Если приложение или функция Pulsar не справляется с ростом объема данных во входящей теме, как показано на рис. 9.5, то такое условие обозначается термином «замедленная обратная реакция» (backpressure), который означает недостаток мощности обработки и снижение производительности приложения и, как следствие, несоответствие соглашению о качестве обслуживания (SLA). Термин «замедленная обратная реакция» позаимствован из гидродинамики (backpressure – обратное фильтрационное давление) и используется для обозначения некоторого сопротивления или обратного давления, ограничивающего требуемую скорость прохождения потока данных через топологическую схему, подобно тому, как засор в кухонной раковине препятствует нормальному сливу воды. Аналогично это условие не только изолирует функцию Pulsar, потребляющую сообщения, но также воздействует на всю топологическую схему в целом.

Все неблагоприятные события, которые мы рассмотрели к этому моменту, – полный отказ («смерть») функции, замедление работы функции и функция, не продолжающая работу, – можно обнаружить по признаку «замедленная обратная реакция» во входной теме (темах) функции. Для обнаружения замедленной обратной реакции в функции Pulsar необходим мониторинг следующих метрик уровня темы:

- `pulsar_rate_in` – скорость поступления сообщений в тему, сообщений/с;
- `pulsar_rate_out` – скорость потребления сообщений из темы, сообщений/с;
- `pulsar_storage_backlog_size` – общий размер журнала регистрации сообщений в теме в байтах.

Все эти метрики периодически публикуются в Prometheus, и можно выполнять их мониторинг с помощью любого фрейм-



ворка наблюдения, интегрируемого с этой платформой. Каждое заметное увеличение размера журнала регистрации сообщений в любой входящей теме функции свидетельствует о возникновении одного или нескольких неблагоприятных событий и должно привлечь особое внимание.

## 9.2. Паттерны проектных решений по обеспечению устойчивости

В предыдущем разделе мы рассматривали некоторые варианты обеспечения устойчивости для приложений Pulsar с использованием возможностей, предоставляемых фреймворком Pulsar Functions, например автоматического перезапуска функций после «смерти». Но функциям Pulsar далеко не всегда требуется прямое взаимодействие с внешней системой для выполнения обработки. Косвенное взаимодействие переносит проблемы с устойчивостью внешних систем в приложение Pulsar. Если такие удаленные системы не отвечают на запросы, то результатом становится замедление или полная остановка работы функции в приложении.

Как мы видели в главе 8, процесс проверки заказов на предприятии GottaEat зависит от нескольких сторонних сервисов для приема любых входящих заказов продуктов, и если нет возможности обмена данными с этими внешними системами по какой-либо причине, то вся наша деловая активность полностью остановится. Учитывая, что все виды взаимодействия с внешними сервисами происходят через сетевое соединение, существует очевидная вероятность кратковременных сетевых сбоев, периодов длительных задержек и недоступности сервисов, а также прочих помех. Таким образом, весьма важно, чтобы приложение обеспечивало устойчивость к этим типам ошибок и сбоев, а также способность автоматического восстановления после них. Хотя вы можете попытаться самостоятельно реализовать описанные выше паттерны, это один из тех сценариев, в котором лучше всего воспользоваться сторонней библиотекой, разработанной экспертами по устранению именно таких типов проблем.

Проблемы, возникающие при взаимодействии с удаленными сервисами, являются настолько частыми, что компания Netflix разработала собственную библиотеку обеспечения устойчивости к сбоям Hystrix и открыла ее исходный код в 2013 г. Библиотека содержала несколько паттернов обеспечения устойчивости, предназначенных для устранения конкретных типов проблем. Хотя эта библиотека уже не разрабатывается активно, некоторые ее концепции реализованы в новом проекте с открытым исходным кодом под названием resilience4j. Как мы вскоре увидим, это упрощает применение упомянутых паттернов в функциях Pulsar на языке Java, потому что большая часть логики уже была реализована



в самой библиотеке `resilience4j`, позволяя разработчику сосредоточиться на выборе конкретного паттерна для использования. Таким образом, для включения этой библиотеки в свой проект необходимо просто добавить элемент конфигурации, показанный в листинге 9.2, в раздел зависимостей файла `pom.xml` Maven.

#### Листинг 9.2. Добавление библиотеки `resilience4j` в проект

```
<dependencies>
 <dependency>
 <groupId>io.github.resilience4j</groupId>
 <artifactId>resilience4j-all</artifactId>
 <version>1.7.1</version>
 </dependency>

 ...
</dependencies>
```

В следующих подразделах будут представлены паттерны из этой библиотеки, а также способы их практического применения для обеспечения устойчивости микросервисов на основе `Pulsar Functions`, взаимодействующих с внешними системами, в соответствующих сценариях с возникновением ошибок и сбоев. В дополнение к описанию контекста и задачи, решаемой каждым паттерном, также будут рассматриваться характеристики и условия его реализации, а также примеры, демонстрирующие возможность его применения в каждом конкретном варианте использования. Следует отметить, что все паттерны были спроектированы так, чтобы можно было объединять их друг с другом. Такой подход особенно удобен, если требуется применить несколько паттернов в функции `Pulsar` (например, необходимо использовать паттерны повтора и прерывания замкнутого цикла при взаимодействии с внешним веб-сервисом).

### 9.2.1. Паттерн повтора

При обмене данными с удаленным сервисом может возникать любое количество неустойчивых ошибок, в том числе разрыв сетевого соединения, временная недоступность сервиса и тайм-ауты из-за чрезвычайно высокой загруженности сервиса. Эти типы ошибок в общем случае являются самовосстанавливающимися, и последующие вызовы сервиса, вероятнее всего, окажутся успешными. Если в функции `Pulsar` возникла ошибка такого типа, то паттерн повтора (`retry`) позволит обработать ее одним из трех способов в зависимости от конкретной ошибки. Если обнаруживается, что сбой по своей природе является неисчезающим, например ошибка аутентификации из-за просроченных идентификационных данных, то не следует повторно выполнять вызов, поскольку та же ошибка будет возникать снова и снова.

Если исключение свидетельствует об истечении интервала времени для установления соединения или о том, что запрос был отвергнут из-за высокой загруженности системы, то лучше всего подождать в течение некоторого времени, прежде чем повторять вызов, чтобы предоставить удаленному сервису время на восстановление. Но можно также повторить вызов немедленно, если у вас нет причин считать, что следующий вызов также окажется неудачным. Для реализации этого типа логики в функции требуется обертка вызова удаленного сервиса в декоратор, как показано в листинге 9.3.

### Листинг 9.3. Использование паттерна повтора в функции Pulsar

```
import io.github.resilience4j.retry.Retry;
import io.github.resilience4j.retry.RetryConfig;
import io.github.resilience4j.retry.RetryRegistry;
import io.vavr.CheckedFunction0;
import io.vavr.control.Try;

public class RetryFunction implements Function<String, String> {

 public String apply(String s) {

 RetryConfig config =
 RetryConfig.custom()
 .maxAttempts(2)
 .waitDuration(Duration.ofMillis(1000))
 .retryOnResult(response -> (response == null))
 .retryExceptions(TimeoutException.class)
 .ignoreExceptions(RuntimeException.class)
 .build();

 CheckedFunction0<String> retryableFunction =
 Retry.decorateCheckedSupplier(
 RetryRegistry.of(config).retry("name"),
 () -> {
 HttpGet request =
 new HttpGet("http://www.google.com/search?q=" + s);

 try (CloseableHttpResponse response =
 HttpClient.createDefault().execute(request)) {
 HttpEntity entity = response.getEntity();
 return (entity == null) ? null : EntityUtils.toString(entity);
 }
 });

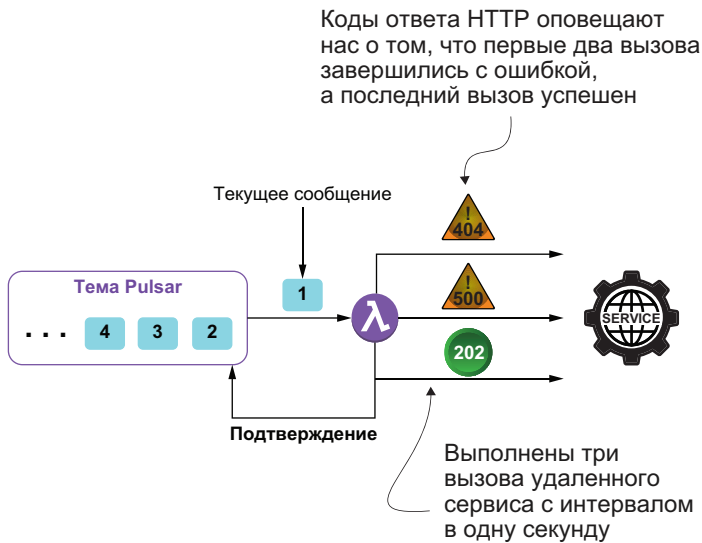
 Try<String> result = Try.of(retryableFunction);
 return result.getOrElse();
 }
}
```

- 1 Требуется несколько классов из библиотеки `resilience4j`.
- 2 Это метод, определенный в функции интерфейса, которая будет вызываться для каждого сообщения.
- 3 Создание собственной специализированной конфигурации повтора.
- 4 Определение не более двух попыток повтора.
- 5 Определение паузы в одну секунду между повторами.
- 6 Выполнение повтора, если возвращен результат `null`.
- 7 Определение списка исключений, интерпретируемых как неустойчивые ошибки, после которых возможен повтор.
- 8 Определение списка исключений, интерпретируемых как исчезающие ошибки, после которых повтор запрещен.
- 9 Дополнение (декорирование) функции только что созданной специализированной конфигурацией повтора.
- 10 Определение лямбда-выражения, которое будет выполняться и повторяться, если это необходимо.
- 11 Выполнение декорированной функции до получения результата или до исчерпания лимита попыток.
- 12 Возврат результата или значения `null`, если ответ не был получен.

Код, показанный в листинге 9.3, просто принимает входящую строку, вызывает API поиска Google с полученной входящей строкой и возвращает результат. Здесь более важен тот факт, что выполняется несколько попыток вызова конечной точки HTTP без необходимости отрицательного подтверждения сообщения и его повторной доставки в эту функцию. Вместо этого сообщение доставляется из темы Pulsar только один раз, и все попытки выполняются в одном вызове внутреннего метода функции Pulsar, как показано на рис. 9.6. Такой подход позволяет избежать бесполезного циклического повторения доставки одного сообщения несколько раз, прежде чем отказаться от обращения к внешней системе.

Первый параметр, передаваемый в метод `Retry.decorateCheckedSupplier`, – объект повтора, ранее сконфигурированный в том же коде, где определяется количество выполняемых попыток, длительность паузы между ними и исключения, сигнализирующие о необходимости повтора вызова функции или о том, что повторы запрещены.

Для тех, кто не знаком с применением лямбда-выражений в коде Java, поясню: реальная вызываемая логика содержится во втором параметре, принимающем определение функции, что обозначено с помощью синтаксиса `()->`, и включает весь код, заключенный в последующие фигурные скобки. Итоговым объектом является декорированная функция, которая затем передается в метод `Try.of`, обрабатывающий все автоматические повторы логики. Термин «декорированная» (*decorated*) взят из широко известного паттерна проектирования «декоратор», обеспечивающего динамическое добавление заданного поведения в объект времени выполнения.



**Рис. 9.6.** При использовании паттерна повтора удаленный сервис вызывается повторно с одним и тем же сообщением. В данном конкретном случае первые два вызова завершаются с ошибкой, но третий успешен, и сообщение подтверждается. Это позволяет избежать отправки отрицательного подтверждения и минутной задержки обработки перед каждой повторной доставкой

Можно было бы реализовать ту же логику, используя сочетание инструкций `try/catch` и счетчик попыток, но подход с декорированной функцией, предоставляемый библиотекой `resilience4j`, является гораздо более изящным решением, которое также позволяет динамически конфигурировать свойства повтора через параметры пользовательской конфигурации, предоставляемые во время развертывания функции Pulsar.

### Проблемы и условия

Хотя это чрезвычайно удобный паттерн для использования при взаимодействии с внешней системой, например веб-сервисом или базой данных, всегда необходимо учитывать следующие факторы при реализации паттерна повтора в функции Pulsar:

- назначайте интервал между повторами в соответствии с бизнес-требованиями конкретного приложения. Чрезмерно агрессивная стратегия повторов с весьма короткими интервалами может создавать дополнительную нагрузку на сервис, который и без того перегружен, и только ухудшит ситуацию. Необходимо принимать к рассмотрению стратегию увеличения интервала между попытками по экспоненте (например, 1 с, 2 с, 4 с, 8 с и т. д.);
- учитывайте важность операции при выборе количества повторных попыток. В некоторых сценариях, возможно, лучше всего быстро зафиксировать ошибку, а не увеличивать задержку

ку при обработке, выполняя многочисленные повторные попытки. Например, в приложении, взаимодействующем с клиентами, лучше сообщить об ошибке после небольшого количества повторов с короткой паузой между ними, чем создавать длительную задержку для конечного пользователя;

- будьте внимательны при использовании паттерна повтора в идемпотентных (не изменяющих объект) операциях, иначе могут возникать неожиданные побочные эффекты (например, вызов внешнего сервиса авторизации кредитной карты, который принят и обработан успешно, но при возврате ответа возникла ошибка). При таких условиях повторная попытка передачи логики отправки запроса может привести к двойной оплате с кредитной карты клиента;
- важно регистрировать в журнале все повторные попытки, чтобы можно было обнаружить все внутренние проблемы при установлении соединения и устранить их как можно быстрее. В дополнение к обычным файлам журналов можно использовать метрики Pulsar для передачи количества повторных попыток администратору.

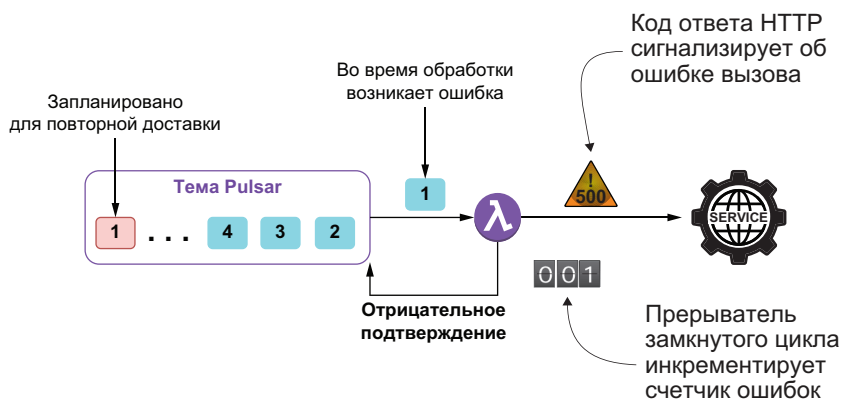
### 9.2.2. Паттерн «прерыватель замкнутого цикла»

Паттерн повтора был предназначен для обработки неустойчивых ошибок, поскольку он позволяет приложению повторять операцию, предполагая, что она в конце концов завершится успешно. С другой стороны, паттерн «прерыватель замкнутого цикла» (circuit breaker) предотвращает выполнение приложением операции, которая, вероятнее всего, ошибочна из-за исчезающего критического сбоя.

Паттерн «прерыватель замкнутого цикла», широко известный по описанию в книге Майкла Ньюгарда (Michael Nygard) «Release It!» (Pragmatic Bookshelf, 2nd ed., 2018), предназначен для предотвращения экстремальной перегрузки удаленного сервиса дополнительными вызовами, когда уже точно известно, что сервис не имеет возможности отвечать. Следовательно, после определенного количества неудачных попыток мы считаем, что сервис недоступен или чрезмерно перегружен, и запрещаем все последующие обращения к нему. Целью является защита удаленного сервиса от еще большей перегруженности непрерывно отправляемыми запросами, которые, как нам уже понятно, вряд ли будут успешными.

Этот паттерн получил название по аналогии с операцией автоматического отключения физической цепи электрического тока в домах (с помощью автоматических предохранителей). Если количество ошибок в течение определенного интервала времени превышает сконфигурированное пороговое значение, то срабатывает прерыватель замкнутого цикла, и все вызовы функции, декорированной этим прерывателем, немедленно отменяются. Пре-

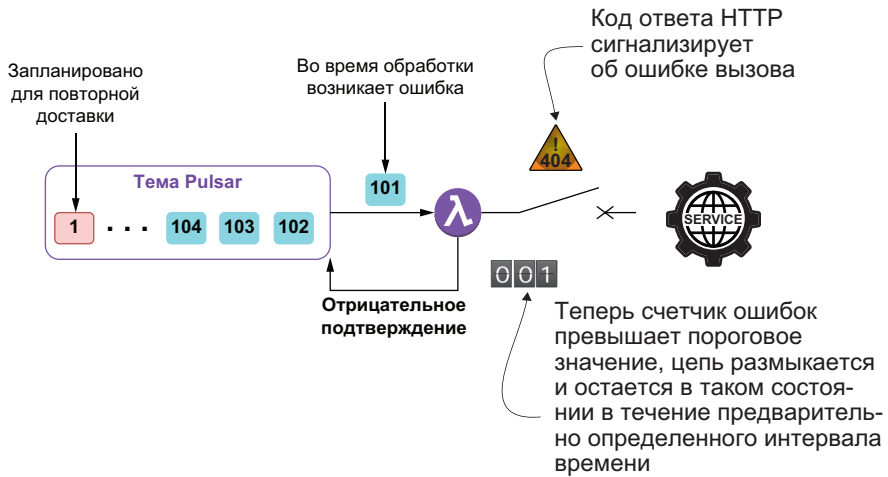
рыватель замкнутого цикла действует как конечный автомат, начинающий работу в замкнутом (close) состоянии, обозначающем возможность прохождения потока данных через прерыватель.



**Рис. 9.7.** Прерыватель замкнутого цикла начинает работу в замкнутом состоянии, т. е. сервис будет вызываться для каждого обрабатываемого сообщения. Вызов сервиса, выполненный при обработке сообщения 1, генерирует исключение, поэтому счетчик ошибок увеличивается на единицу, а сообщение подтверждается отрицательно. Если счетчик превышает сконфигурированное пороговое значение, то прерыватель замкнутого цикла срабатывает и переходит в разомкнутое состояние

На рис. 9.7 можно видеть, что все входящие сообщения приводят к вызову удаленного сервиса. Но прерыватель замкнутого цикла продолжает отслеживать количество вызовов, генерирующих исключения. Если количество исключений превышает сконфигурированное пороговое значение, это является признаком того, что удаленный сервис либо не способен ответить, либо чрезмерно перегружен, чтобы обрабатывать дополнительные запросы. Следовательно, чтобы не создавать дополнительную нагрузку на уже перегруженный сервис, прерыватель замкнутого цикла переходит в разомкнутое (open) состояние, как показано на рис. 9.8. Это похоже на то, что делает электрический предохранитель-автомат, когда обнаруживает бросок электротока и размыкает цепь для изоляции перегруженной розетки электропитания.

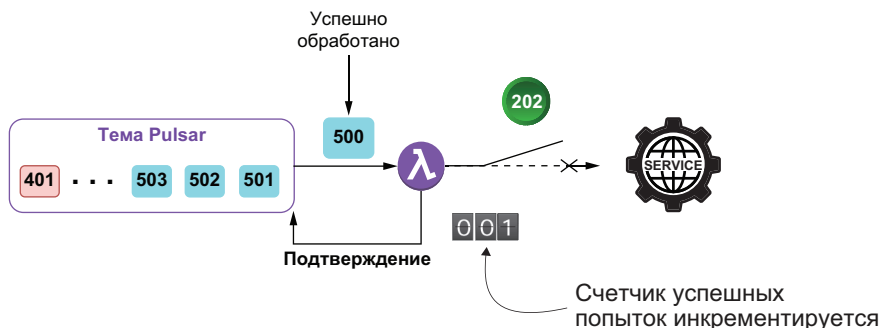
После срабатывания прерывателя замкнутого цикла он остается в разомкнутом состоянии в течение предварительно сконфигурированного интервала времени. Пока этот период не закончится, вызовы сервиса не выполняются. Такой подход предоставляет сервису время для устранения внутренней проблемы, прежде чем разрешить приложению возобновить обращения к этому сервису. После завершения периода размыкания прерыватель переходит в полуразомкнутое (half-open) состояние, в котором разрешается ограниченное количество запросов к удаленному сервису.



**Рис. 9.8.** Когда счетчик ошибок прерывателя замкнутого цикла превышает сконфигурированное пороговое значение, прерыватель переходит в разомкнутое состояние, и все последующие сообщения немедленно подтверждаются отрицательно. Это предотвращает выполнение дополнительных вызовов сервиса, в котором, вероятно, возникла ошибка, и предоставляет ему время на восстановление

Полуразомкнутое состояние предназначено для защиты восстанавливающегося сервиса от новой перегрузки запросами от всех сообщений, накопившихся в журнале регистрации. После полного восстановления сервиса его способность обработки запросов, возможно, будет ограничена в течение некоторого начального периода. Передача ограниченного количества запросов защищает восстанавливающуюся систему от новой перегрузки после восстановления. Прерыватель замкнутого цикла поддерживает счетчик удачных попыток, значение которого увеличивается на единицу для каждого сообщения, которому разрешено вызывать сервис, при успешном завершении этой операции, как показано на рис. 9.9.

Если все запросы успешны, о чем свидетельствует счетчик успешных попыток, предполагается, что сбой, ставший причиной возникновения проблемы, устранен, и сервис готов снова принимать запросы. Поэтому прерыватель замкнутого цикла возвращается в замкнутое состояние и возобновляет операции в обычном режиме. Но если в полуразомкнутом состоянии запрос какого-либо сообщения привел к ошибке, то прерыватель замкнутого цикла немедленно переходит обратно в разомкнутое состояние (без применения порогового значения счетчика ошибок) и перезапускает таймер, чтобы предоставить сервису дополнительное время для восстановления после сбоя.



**Рис. 9.9.** Когда прерыватель замкнутого цикла находится в полуразомкнутом состоянии, разрешены вызовы сервиса ограниченным количеством сообщений. После успешного завершения достаточного количества вызовов прерыватель возвращается в замкнутое состояние. Но если вызов какого-либо сообщения приводит к ошибке, то прерыватель замкнутого цикла немедленно возвращается в разомкнутое состояние

Этот паттерн обеспечивает стабильность работы приложения, пока удаленный сервис восстанавливается после неисчезающей ошибки, немедленно запрещая запросы, которые, вероятнее всего, будут ошибочными и могут привести к каскаду взаимозависимых ошибок во всем приложении. Этот паттерн часто объединяется с паттерном повтора для обработки неустойчивых и исчезающих ошибок в удаленном сервисе. Для использования паттерна «прерыватель замкнутого цикла» в функции Pulsar необходимо обернуть вызов удаленного сервиса в декоратор, как показано в листинге 9.4, где демонстрируется реализация `CreditCardAuthorizationService`, вызываемого из сервиса проверки платежей `GottaEat`, если клиент использует для оплаты кредитную карту.

#### Листинг 9.4. Использование паттерна «прерыватель замкнутого цикла» в функции Pulsar

```
...
import io.github.resilience4j.circuitbreaker.*;
import io.vavr.CheckedFunction0;
import io.vavr.control.Try;

public class CreditCardAuthorizationService
 implements Function<CreditCard, AuthorizedPayment> {

 public AuthorizedPayment process(CreditCard card, Context ctx)
 throws Exception {

 CircuitBreakerConfig config = CircuitBreakerConfig.custom()
 .failureRateThreshold(20)
 .slowCallRateThreshold(50)
 .waitDurationInOpenState(Duration.ofMillis(30000))
```

1

2

3

4



```

 .slowCallDurationThreshold(Duration.ofSeconds(10))
 .permittedNumberOfCallsInHalfOpenState(5)
 .minimumNumberOfCalls(10)
 .slidingWindowType(SlidingWindowType.TIME_BASED)
 .slidingWindowSize(5)
 .ignoreException(e -> e instanceof
 UnsuccessfulCallException &&
 ((UnsuccessfulCallException)e).getCode() == 499)
 .recordExceptions(IOException.class,
 ➔ UnsuccessfulCallException.class)
 .build();

CheckedFunction0<String> cbFunction =
 CircuitBreaker.decorateCheckedSupplier(
 CircuitBreakerRegistry.of(config).circuitBreaker("name"),
 () -> {
 OkHttpClient client = new OkHttpClient();
 StringBuilder sb = new StringBuilder()
 .append("number=").append(card.getAccountNumber())
 .append("&cvc=").append(card.getCcv())
 .append("&exp_month=").append(card.getExpMonth())
 .append("&exp_year=").append(card.getExpYear());

 MediaType mediaType =
 MediaType.parse("application/x-www-form-urlencoded");
 RequestBody body =
 RequestBody.create(sb.toString(), mediaType);

 Request request = new Request.Builder()
 .url("https://noodlio-pay.p.rapidapi.com/tokens/create")
 .post(body)
 .addHeader("x-rapidapi-key", "SIGN-UP-FOR-KEY")
 .addHeader("x-rapidapi-host", "noodlio-pay.p.rapidapi.com")
 .addHeader("content-type", "application/x-www-form-urlencoded")
 .build();
 try (Response response = client.newCall(request).execute()) {
 if (!response.isSuccessful()) {
 throw new UnsuccessfulCallException(response.code());
 }
 return getToken(response.body().string());
 }
 }
);

 Try<String> result = Try.of(cbFunction);
 return authorize(card, result.getOrNull());
}

private String getToken(String json) {
 ...
}

```

```
private AuthorizedPayment authorize(CreditCard card, String token) {
 ...
}
}
```

- ❶ Здесь используются классы из пакета `circuitbreaker`.
- ❷ Количество допустимых ошибок перед переходом в разомкнутое состояние.
- ❸ Количество медленных вызовов перед переходом в разомкнутое состояние.
- ❹ Время пребывания в разомкнутом состоянии до перехода в полуразомкнутое состояние.
- ❺ Любой вызов, продолжающийся более 10 с, считается медленным и инкрементирует счетчик.
- ❻ Количество вызовов, которые можно выполнить в полуразомкнутом состоянии.
- ❼ Минимальное количество вызовов, прежде чем начнется применение счетчика ошибок.
- ❽ Использование временного окна для счетчика ошибок.
- ❾ Использование временного окна длиной в 5 мин перед перезапуском счетчика ошибок.
- ❿ Список исключений, которые не инкрементируют счетчик ошибок.
- ⓫ Список исключений, которые инкрементируют счетчик ошибок.
- ⓬ Использование собственной специализированной конфигурации прерывателя замкнутого цикла.
- ⓭ Предоставление лямбда-выражения, которое будет выполняться, если оно разрешено прерывателем замкнутого цикла.
- ⓮ Выполнение декорированной функции до получения результата или до превышения лимита повторных попыток.
- ⓯ Возврат авторизованного платежа или значения `null`, если ответ не был получен.

Первый параметр, передаваемый в метод `CircuitBreaker.decorateCheckedSupplier`, – объект `CircuitBreaker`, основанный на конфигурации, описанной выше в том же коде и определяющей пороговое значение счетчика ошибок, время пребывания в разомкнутом состоянии и списки исключений, считающихся ошибками вызова и таковыми не являющихся. Дополнительные подробности об этих и других параметрах можно найти в документации библиотеки `resilience4j` (<https://resilience4j.readme.io/docs/circuitbreaker>).

Реальная логика, которая будет вызвана, если прерыватель замкнутого цикла находится в замкнутом состоянии, определена внутри лямбда-выражения, передаваемого как второй параметр. Как можно видеть, логика внутри определения функции отправляет HTTP-запрос стороннему сервису авторизации кредитной карты Noodlio Pay, возвращающему токен авторизации, который в дальнейшем можно использовать для получения оплаты с предостав-

ленной кредитной карты. Итоговым объектом является декорированная функция, которая затем передается в метод `Try.of`, автоматически обрабатывающий всю логику прерывателя замкнутого цикла. При успешном вызове токен авторизации извлекается из объекта ответа `Noodlio Pay` и возвращается в объекте `AuthorizedPayment`.

### Проблемы и условия

Хотя это чрезвычайно удобный паттерн для использования при взаимодействии с внешней системой, например веб-сервисом или базой данных, всегда необходимо учитывать следующие факторы при реализации паттерна «прерыватель замкнутого цикла» в функции `Pulsar`:

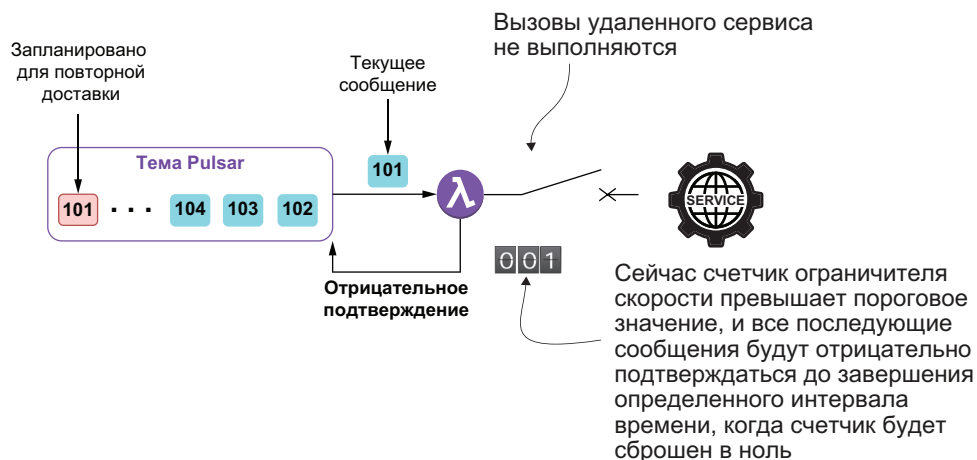
- необходимо регулировать стратегию прерывателя замкнутого цикла на основе степени опасности самих исключений, так как запрос может оказаться ошибочным по многим причинам, а некоторые ошибки могут быть неустойчивыми, тогда как другие – исчезающими. При наличии исчезающей ошибки, возможно, имеет смысл немедленно разомкнуть прерыватель замкнутого цикла, а не ждать определенного количества неудачных запросов;
- не рекомендуется использовать единственный прерыватель замкнутого цикла в функции `Pulsar` для работы с несколькими независимыми провайдерами. Например, сервис проверки платежей `GottaEat` использует четыре различных сервиса в зависимости от способа оплаты, предложенного клиентом. Если бы вызов сервиса оплаты проходил через единственный прерыватель замкнутого цикла, то ошибочные ответы от неработоспособного сервиса могли бы привести к срабатыванию прерывателя и запрету вызовов других трех сервисов, которые, вероятнее всего, остаются в рабочем состоянии;
- прерыватель замкнутого цикла обязательно должен фиксировать в журнале все ошибочные запросы, чтобы предоставить возможность обнаружения внутренних проблем с соединениями и устранения их как можно быстрее. В дополнение к обычным файлам журналов можно использовать метрики `Pulsar` для передачи количества повторных попыток администратору;
- паттерн «прерыватель замкнутого цикла» можно использовать в сочетании с паттерном повтора.

#### 9.2.3. Паттерн «ограничитель скорости»

Паттерн «прерыватель замкнутого цикла» был предназначен для запрещения вызовов сервиса только после фиксации предварительно сконфигурированного количества ошибок, обнаруженных

в течение определенного интервала времени. В отличие от него паттерн «ограничитель скорости» (rate limiter) запрещает вызовы сервиса после фиксации предварительно сконфигурированного количества вызовов, выполненных в течение определенного интервала времени независимо от их успешности. Из названия этого паттерна можно понять, что он используется для снижения частоты обращений к удаленному сервису, поэтому полезен в ситуациях, когда необходимо ограничить количество вызовов за определенный период. Один из примеров – вызов веб-сервиса, скажем Google API, ограничивающего количество возможных обращений с бесплатным ключом API 60 вызовами в минуту.

При использовании паттерна «ограничитель скорости» все входящие сообщения вызывают удаленный сервис не более предварительно сконфигурированного количества раз. Ограничитель скорости отслеживает число обращений к сервису и при достижении лимита запрещает все дополнительные вызовы в течение оставшегося интервала в предварительно сконфигурированном временном окне, как показано на рис. 9.10. Чтобы использовать паттерн «ограничитель скорости» в функции Pulsar, необходимо обернуть вызов удаленного сервиса в декоратор, как показано в листинге 9.5.



**Рис. 9.10.** Если конфигурация ограничителя скорости допускает выполнение 100 вызовов в минуту, то первым 100 сообщениям разрешен вызов сервиса. 101-му сообщению и всем последующим запрещено обращаться к сервису, и вместо этого они подтверждаются отрицательно. По истечении минуты могут быть обработаны следующие 100 сообщений<sup>1</sup>

<sup>1</sup> На рисунке показано значение счетчика 001, хотя в данном случае счетчик должен показывать лимит, равный 100. Видимо, изображение было скопировано с предыдущих рисунков без надлежащих изменений. – Прим. перев.

**Листинг 9.5.** Использование паттерна «ограничитель скорости» в функции Pulsar

```

import io.github.resilience4j.decorators.Decorators;
import io.github.resilience4j.ratelimiter.*;

...
public class GoogleGeoEncodingService implements Function<Address, Void>
{

 public Void process(Address addr, Context ctx) throws Exception {

 if (!initialized) {
 init(ctx);
 }

 CheckedFunction0<String> decoratedFunction =
 Decorators.ofCheckedSupplier(getFunction(addr))
 .withRateLimiter(rateLimiter)
 .decorate();

 LatLon geo = getLocation(
 Try.of(decoratedFunction)
 .onFailure(
 (Throwable t) -> ctx.getLogger().error(t.getMessage())
).getOrNull());

 if (geo != null) {
 addr.setGeo(geo);
 ctx.newOutputMessage(ctx.getOutputTopic(),
 AvroSchema.of(Address.class))
 .properties(ctx.getCurrentRecord().getProperties())
 .value(addr)
 .send();
 } else {
 // Выполнен корректный вызов, но не возвращены корректные геоданные.
 }
 return null;
 }

 private void init(Context ctx) {
 config = RateLimiterConfig.custom()
 .limitRefreshPeriod(Duration.ofMinutes(1))
 .limitForPeriod(60)
 .timeoutDuration(Duration.ofSeconds(1))
 .build();

 rateLimiterRegistry = RateLimiterRegistry.of(config);
 rateLimiter = rateLimiterRegistry.rateLimiter("name");
 initialized = true;
 }

```

```

private CheckedException<String> getFunction(Address addr) {
 CheckedException<String> fn = () -> {
 OkHttpClient client = new OkHttpClient();
 StringBuilder sb = new StringBuilder()
 .append("https://maps.googleapis.com/maps/api/geocode")
 .append("/json?address=")
 .append(URLEncoder.encode(addr.getStreet().toString(),
 StandardCharsets.UTF_8.toString()))
 .append(",")
 .append(URLEncoder.encode(addr.getCity().toString(),
 StandardCharsets.UTF_8.toString()))
 .append(",")
 .append(URLEncoder.encode(addr.getState().toString(),
 StandardCharsets.UTF_8.toString()))
 .append("&key=").append("SIGN-UP-FOR-KEY");

 Request request = new Request.Builder()
 .url(sb.toString())
 .build();

 try (Response response = client.newCall(request).execute()) {
 if (response.isSuccessful()) {
 return response.body().string();
 } else {
 String reason = getErrorStatus(response.body().string());
 if (NON_TRANSIENT_ERRORS.stream().anyMatch(
 s -> reason.contains(s))) {
 throw new NonTransientException();
 } else if (TRANSIENT_ERRORS.stream().anyMatch(
 s -> reason.contains(s))) {
 throw new TransientException();
 }
 return null;
 }
 }
 };
 return fn;
}

private LatLon getLocation(String json) {
 ...
}

```

- ① Инициализация ограничителя скорости.
- ② Декорирование REST-вызова, направляемого в Google Maps API.
- ③ Назначение ограничителя скорости.
- ④ Синтаксический анализ ответа на REST-вызов.
- ⑤ Вызов декорированной функции.
- ⑥ Интервал допустимой скорости равен одной минуте.
- ⑦ Устанавливается лимит, равный 60 вызовам в интервале допустимой скорости.
- ⑧ Пауза в одну секунду между последовательными вызовами.

- 9 Возврат функции, содержащей логику вызова REST API.
- 10 Если получен корректный ответ, то возвращается строка в формате JSON, содержащая широту и долготу.
- 11 Определение типа ошибки по сообщению об ошибке.
- 12 Синтаксический анализ ответа в формате JSON, полученного в результате вызова Google Maps REST API.

Эта функция вызывает Google Maps API и ограничивает количество попыток значением 60 в минуту, чтобы соответствовать правилам использования сервиса Google для бесплатного аккаунта. Если допустимое количество вызовов превышено, то Google в качестве средства защиты заблокирует доступ к API для этого аккаунта на относительно короткий интервал времени. Таким образом, мы приняли превентивные меры, предотвращающие возникновение такого условия.

### Проблемы и условия

Хотя это чрезвычайно удобный паттерн для использования при взаимодействии с внешней системой, например веб-сервисом или базой данных, всегда необходимо учитывать следующие факторы при реализации паттерна «ограничитель скорости» в функции Pulsar:

- этот паттерн почти наверняка снизит пропускную способность функции, и такой недостаток необходимо учитывать при проектировании общей пропускной способности для всего потока данных в целом для полной уверенности в том, что более ранние функции не будут создавать критическую нагрузку на функцию с ограничением скорости обработки;
- этот паттерн должен использоваться для защиты удаленного сервиса от чрезмерной перегрузки вызовами или для ограничения количества вызовов, чтобы избежать превышения квоты, приводящего к полной блокировке со стороны внешнего провайдера.

#### 9.2.4. Паттерн «ограничитель времени»

Паттерн «ограничитель времени» используется для лимитирования времени, предоставленного для вызовов удаленного сервиса до закрытия соединения со стороны клиента. Это позволяет эффективно укорачивать циклы механизма тайм-аутов в удаленном сервисе и позволяет точно определять, когда следует отказаться от удаленного вызова и закрыть соединение со своей (т. е. клиентской) стороны.

Такое поведение удобно, если имеются строгие соглашения о качестве обслуживания (SLA) для всего конвейера данных в целом и выделяется небольшой интервал времени для взаимодействия с удаленным сервисом. Такой подход позволяет функции Pulsar продолжить обработку без ответа от веб-сервиса, а не ждать 30

и более секунд после разрыва соединения по тайм-ауту. Это существенно увеличивает пропускную способность функции, поскольку больше не нужно тратить 30 с на ожидание ответа от неработоспособного сервиса.

Рассмотрим сценарий, в котором вы сначала выполняете вызов к внутреннему кеширующему сервису, например Apache Ignite, чтобы узнать, имеются ли данные, требуемые перед вызовом внешнего сервиса для получения значения. Это делается для ускорения обработки внутри функции, чтобы исключить необходимость слишком затяжного обращения к удаленному сервису. Но при этом возникает риск того, что сам кеширующий сервис окажется недоступным, что приведет к длительной паузе во время выполнения этого вызова. Подобное обстоятельство может свести к нулю все преимущества кеширования. Поэтому вы решаете выделить ограниченный бюджет времени размером в 500 мс для обращения в кеш, чтобы уменьшить возможное отрицательное воздействие неответа кеша на работу функции.

**Листинг 9.6.** Использование паттерна «ограничитель времени» в функции Pulsar

```
import io.github.resilience4j.timelimiter.TimeLimiter;
import io.github.resilience4j.timelimiter.TimeLimiterConfig;
import io.github.resilience4j.timelimiter.TimeLimiterRegistry;

public class LookupService implements Function<Address, Address> {
 private TimeLimiter timeLimiter;
 private IgniteCache<Address, Address> cache;
 private String bypassTopic;
 private boolean initialized = false;

 public Address process(Address addr, Context ctx) throws Exception {
 if (!initialized) {
 init(ctx);
 }

 Address geoEncodedAddr = timeLimiter.executeFutureSupplier(
 () -> CompletableFuture.supplyAsync(() ->
 { return cache.get(addr); })
);

 if (geoEncodedAddr != null) {
 ctx.newOutputMessage(bypassTopic, AvroSchema.of(Address.class))
 .properties(ctx.getCurrentRecord().getProperties())
 .value(geoEncodedAddr)
 .send();
 }

 return null;
 }
}
```



```

private void init(Context ctx) {
 bypassTopic = ctx.getUserConfigValue("bypassTopicName")
 .get().toString();
 TimeLimiterConfig config = TimeLimiterConfig.custom()
 .cancelRunningFuture(true)
 .timeoutDuration(Duration.ofMillis(500))
 .build();

 TimeLimiterRegistry registry = TimeLimiterRegistry.of(config);
 timeLimiter = registry.timeLimiter("my-time-limiter");

 IgniteConfiguration cfg = new IgniteConfiguration();
 cfg.setClientMode(true);
 cfg.setPeerClassLoadingEnabled(true);

 TcpDiscoveryMulticastIpFinder ipFinder =
 new TcpDiscoveryMulticastIpFinder();

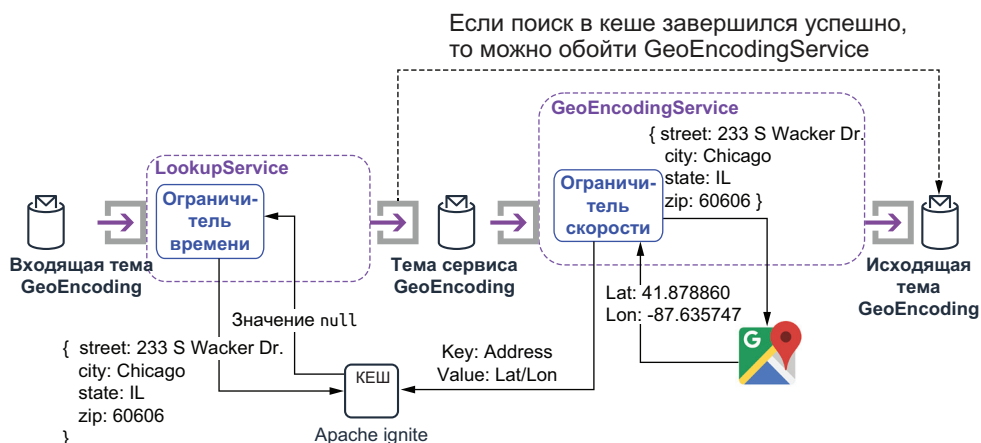
 ipFinder.setAddresses(
 ➔ Collections.singletonList("127.0.0.1:47500..47509"));

 cfg.setDiscoverySpi(new TcpDiscoverySpi().setIpFinder(ipFinder));
 Ignite ignite = Ignition.start(cfg);
 cache = ignite.getOrCreateCache("geo-encoded-addresses");
}
}

```

- ❶ Инициализация ограничителя времени и кеша Apache Ignite.
- ❷ Вызов функции поиска в кеше и ограничение продолжительности этого вызова с помощью ограничителя времени.
- ❸ Поиск в кеше, выполняемый асинхронно.
- ❹ Если обнаружено попадание кеша, то найденное значение публикуется в другой исходящей теме.
- ❺ Тема `bypass` (обход удаленного сервиса) является конфигурируемой.
- ❻ Отмена выполняющихся вызовов, превысивших лимит времени.
- ❼ Установка лимита времени равным 500 мс.
- ❽ Создание ограничителя времени на основе описанной выше конфигурации.
- ❾ Конфигурирование клиента Apache Ignite.
- ❿ Установление соединения с сервисом Apache Ignite.
- ⓫ Извлечение локального кеша, хранящего геокодированные адреса.

Чтобы использовать паттерн «ограничитель времени» в функции Pulsar, необходимо обернуть вызов удаленного сервиса в `CompletableFuture` и выполнить его через метод `executeFutureSupplier` объекта `TimeLimiter`, как показано в листинге 9.6. Эта функция поиска предполагает, что кеш заполняется внутри функции, которая вызывает Google Maps API, что позволяет немного реструктуризировать процесс геокодирования, чтобы поиск выполнялся перед вызовом `GeoEncodingService`, как показано на рис. 9.11.



**Рис. 9.11.** Сервис поиска в кеше можно использовать для предотвращения ненужных обращений к GeoEncodingService, если для заданного адреса уже имеется пара широта–долгота. Кеш заполняется результатами вызовов Google Maps API

И снова этот тип проектного решения предназначен для ускорения процесса геокодирования, следовательно, необходимо ограничить интервал времени, в течение которого мы намерены ожидать ответ от Apache Ignite, прежде чем отменить вызов. Если поиск в кеше завершился успешно, то LookupService опубликует сообщение с результатом непосредственно в той же исходящей теме GeoEncoding, что и сервис GeoEncodingService. Для следующего потребителя не имеет значения, кто опубликовал сообщение, если его содержимое корректно.

### Проблемы и условия

Хотя это чрезвычайно удобный паттерн для использования при взаимодействии с внешней системой, например веб-сервисом или базой данных, всегда необходимо учитывать следующие факторы при реализации паттерна «ограничитель времени» в функции Pulsar:

- регулируйте лимит времени в соответствии с требованиями бизнес-соглашения о качестве обслуживания (SLA) конкретного приложения. Чрезмерно ограниченный лимит времени приведет к тому, что успешные вызовы будут отменяться слишком быстро, и в результате удаленный сервис будет совершать ненужную работу, а в функции Pulsar начнутся потери данных;
- не используйте этот паттерн в идемпотентных операциях, так как при этом могут возникать неожиданные побочные эффекты. Лучше всего применять этот паттерн в вызовах поиска и прочих функциях и сервисах, не изменяющих состояние.

### 9.2.5. Паттерн «кеш»

Если вы предпочитаете отказываться от написания отдельной функции, предназначенной только для выполнения поиска, то библиотека `resilience4j` предоставляет способ декорирования функции с помощью кеша. В листинге 9.7 показано, как декорировать лямбда-выражение Java с помощью абстракции «кеш». Эта абстракция сохраняет результат каждого вызова функции в экземпляре кеша и пытается извлечь предыдущий кешированный результат перед вызовом лямбда-выражения.

**Листинг 9.7.** Использование кеша, обеспечивающего устойчивость, в функции `Pulsar`

```
import javax.cache.CacheManager;
import javax.cache.Caching;
import javax.cache.configuration.MutableConfiguration;
import io.github.resilience4j.cache.Cache;
import io.github.resilience4j.decorators.Decorators;

...
public class GeoEncodingServiceWithCache implements Function<Address, Void> {

 public Void process(Address addr, Context ctx) throws Exception {

 if (!initialized) {
 init(ctx);
 }

 CheckedFunction1<String, String> cachedFunction = Decorators
 .ofCheckedSupplier(getFunction(addr))
 .withCache(cacheContext)
 .decorate();

 LatLon geo = getLocation(
 Try.of(() -> cachedFunction.apply(addr.toString())));

 if (geo != null) {
 addr.setGeo(geo);
 ctx.newOutputMessage(ctx.getOutputTopic(), AvroSchema.of(Address.class))
 .properties(ctx.getCurrentRecord().getProperties())
 .value(addr)
 .send();
 }
 return null;
 }

 private void init(Context ctx) {
 CacheManager cacheManager =
 Caching.getCachingProvider().getCacheManager();
 }
}
```

```
cacheContext = Cache.of(cacheManager.createCache(8
 "addressCache", new MutableConfiguration<>()));
 initialized = true;
}

private CheckedFunction0<String> getFunction(Address addr) {9
 // То же, что и раньше.
}

private LatLon getLocation(String json) {10
 // То же, что и раньше.
}
}
```

- 1 Инициализация кеша.
- 2 Декорирование REST-вызова Google Maps API.
- 3 Назначение кеша.
- 4 Синтаксический анализ ответа из кеша или из REST-вызова.
- 5 Проверка кеша перед вызовом функции.
- 6 Если получены значения широты/долготы, то передать их.
- 7 Использование сконфигурированной реализации кеша.
- 8 Создание кеша адреса.
- 9 Возврат функции, содержащей логику вызова REST API.
- 10 Если получен корректный ответ, то вернуть строку в формате JSON, содержащую широту/долготу.

Необходимо сконфигурировать функцию для использования реализации распределенного кеша, например Ehcache, Caffeine или Apache Ignite. Если операция извлечения из распределенного кеша завершилась с ошибкой, то исключение игнорируется, и вызывается лямбда-выражение для получения требуемого значения. В документации выбранного поставщика услуг вы найдете более подробную информацию о том, как конфигурировать и использовать конкретную реализацию функциональной спецификации JCache.

Сравнивая эту методику с применением отдельного сервиса поиска, можно отметить, что в данном случае вы лишаетесь возможности ограничения времени ожидания завершения обращения к кешу. Поэтому если распределенный кеш неработоспособен, то при каждом обращении вы будете терять 30 с, ожидая сетевого тайм-аута.

### Проблемы и условия

Хотя это чрезвычайно удобный паттерн для использования при взаимодействии с внешней системой, например веб-сервисом или базой данных, всегда необходимо учитывать следующие факторы при реализации паттерна «кеш» в функции Pulsar:

- этот паттерн полезен только при работе с наборами данных, которые по своей природе относительно статичны, например геокодированные адреса. Не пытайтесь кешировать часто из-

меняющиеся данные, так как при этом возникает рис. использования в приложении некорректных данных;

- ограничивайте размер кеша, чтобы избежать переполнения памяти в функции Pulsar. Если необходим кеш большого размера, то следует рассмотреть вариант использования распределенного кеша, например Apache Ignite;
- защитите данные в кеше от устаревания, реализовав стратегию отслеживания возраста данных, чтобы по достижении определенного возраста кеш автоматически очищался.

### 9.2.6. Паттерн «откат»

Паттерн «откат» (fallback) используется для обеспечения функции альтернативным ресурсом в ситуации, когда основной ресурс недоступен. Рассмотрим вариант, в котором вызывается внутренняя база данных через конечную точку балансировщика нагрузки, но в балансировщике возникает критический сбой. Вместо того чтобы позволить сбою в этом отдельном элементе аппаратного обеспечения привести в нерабочее состояние все приложение, можно обойти балансировщик нагрузки и установить прямое соединение с базой данных. В листинге 9.8 показано, как реализовать такую функцию.

**Листинг 9.8.** Использование паттерна «откат» в функции Pulsar

```
import io.github.resilience4j.decorators.Decorators;
import io.vavr.CheckedFunction0;
import io.vavr.control.Try;

public class DatabaseLookup implements Function<String, FoodOrder> {
 private String primary = "jdbc:mysql://load-balancer:3306/food";
 private String backup = "jdbc:mysql://backup:3306/food";
 private String sql = "select * from food_orders where id=?";
 private String user = "";
 private String pass = "";
 private boolean initialized = false;

 public FoodOrder process(String id, Context ctx) throws Exception {
 if (!initialized) {
 init(ctx);
 }

 CheckedFunction0<ResultSet> decoratedFunction =
 Decorators.ofCheckedSupplier(() -> {
 try (Connection con =
 DriverManager.getConnection(primary, user, pass)) {
 PreparedStatement stmt = con.prepareStatement(sql);
 stmt.setLong(1, Long.parseLong(id));
 return stmt.executeQuery();
 }
 })
 }
}
```

```

 .withFallback(SQLException.class, ex -> {
 try (Connection con =
 DriverManager.getConnection(backup, user, pass)) {
 PreparedStatement stmt = con.prepareStatement(sql);
 stmt.setLong(1, Long.parseLong(id));
 return stmt.executeQuery();
 }
 })
 .decorate();

 ResultSet rs = Try.of(decoratedFunction).get();
 return ORMMapping(rs);
 }

 private void init(Context ctx) {
 Driver myDriver;
 try {
 myDriver = (Driver) Class.forName("com.mysql.jdbc.Driver")
 .newInstance();
 DriverManager.registerDriver(myDriver);
 // Настройка локальных переменных по пользовательским свойствам.
 ...
 initialized = true;
 } catch (Throwable t) {
 t.printStackTrace();
 }
 }

 private FoodOrder ORMMapping(ResultSet rs) {
 // Выполнение объектно-реляционного отображения Relational-to-Object.
 }
}

```

- ❶ Основное соединение, установленное через балансировщик нагрузки.
- ❷ Резервное соединение, устанавливаемое напрямую с сервером базы данных.
- ❸ Идентификационные сведения для базы данных, устанавливаемые в методе `init` по пользовательским свойствам.
- ❹ Первый вызов выполняется через балансировщик нагрузки.
- ❺ Если сгенерировано исключение `SQLException`, то запрос повторяется через резервный URL.
- ❻ Успешное получение `ResultSet`.
- ❼ Выполнение объектно-реляционного отображения (ORM) для возврата объекта `FoodOrder`.
- ❽ Загрузка класса драйвера базы данных и его регистрация.

Другой сценарий, в котором возможно применение этого паттерна, – наличие нескольких сервисов авторизации кредитных карт с возможностью выбора одного из них, и если по какой-то причине невозможно установить соединение с основным серви-

сом, то необходимо попытаться соединиться с альтернативным. Обратите внимание: первый параметр метода `withFallback` принимает типы исключений, активизирующие код паттерна «откат», поскольку важно, чтобы соединение со вторым сервисом авторизации устанавливалось в случае недоступности основного сервиса, а не при отказе в обслуживании самой кредитной карты.

### Проблемы и условия

Хотя это чрезвычайно удобный паттерн для использования при взаимодействии с внешней системой, например веб-сервисом или базой данных, всегда необходимо учитывать следующие факторы при реализации паттерна «откат» в функции Pulsar:

- этот паттерн полезен только в тех ситуациях, когда существует альтернативный маршрут к сервису, доступный из функции Pulsar, или резервная копия сервиса, предоставляющая ту же функциональность.

#### 9.2.7. Паттерн «обновление идентификационных данных»

Паттерн «обновление идентификационных данных» (`credentials refresh`) используется для автоматического определения завершения срока действия идентификационных данных в текущем сеансе и необходимости их обновления. Рассмотрим вариант, в котором вы взаимодействуете с сервисом AWS из функции Pulsar, и для аутентификации требуются токены сеанса. Обычно эти токены предназначены для использования в течение короткого интервала времени, следовательно, с ними связан срок действия (например, 60 мин). Таким образом, при взаимодействии с сервисом, требующим предъявления такого токена, необходима стратегия автоматической генерации нового токена после истечения срока действия предыдущего, чтобы сохранить возможность обработки сообщений функцией Pulsar. В листинге 9.9 показано, как реализовать автоматическое определение окончания срока действия токена сеанса и его обновление, используя функциональную библиотеку Vavr для языка Java.

#### Листинг 9.9. Автоматическое обновление идентификационных данных

```
public class PayPalAuthorizationService implements Function<PayPal, String> {
 private String clientId;
 private String secret;
 private String accessToken;
 private String PAYPAL_URL = "https://api.sandbox.paypal.com";

 public String process(PayPal pay, Context ctx) throws Exception {
 return Try.of(getAuthorizationFunction(pay))
 .onFailure(UnauthorizedException.class, refreshToken())
 }
}
```

```
.recover(UnauthorizedException.class,
 (exc) -> Try.of(getAuthorizationFunction(pay)).get())
.getOrNull();
}

private CheckedFunction0<String> getAuthorizationFunction(PayPal pay) {
 CheckedFunction0<String> fn = () -> {
 OkHttpClient client = new OkHttpClient();
 MediaType mediaType =
 MediaType.parse("application/json; charset=utf-8");
 RequestBody body =
 RequestBody.create(buildRequestBody(pay), mediaType);

 Request request =
 new Request.Builder()
 .url("https://api.sandbox.paypal.com/v1/payments/payment")
 .addHeader("Authorization", accessToken)
 .post(body)
 .build();

 try (Response response = client.newCall(request).execute()) {
 if (response.isSuccessful()) {
 return response.body().string();
 } else if (response.code() == 500) {
 throw new UnauthorizedException();
 }
 return null;
 }
 }
};

return fn;
}

private Consumer<UnauthorizedException> refreshToken() {
 Consumer<UnauthorizedException> refresher = (ex) -> {
 OkHttpClient client = new OkHttpClient();
 MediaType mediaType =
 MediaType.parse("application/json; charset=utf-8");
 RequestBody body = RequestBody.create("", mediaType);

 Request request = new Request.Builder().url(PAYPAL_URL +
 "/v1/oauth2/token?grant_type=client_credentials")
 .addHeader("Accept-Language", "en_US")
 .addHeader("Authorization",
 Credentials.basic(clientId, secret))
 .post(body)
 .build();

 try (Response response = client.newCall(request).execute()) {
 if (response.isSuccessful()) {
 parseToken(response.body().string());
 }
 } catch (IOException e) {
```



```
 e.printStackTrace();
 }
 return refresher;
}

private void parseToken(String json) {
 // Парсинг нового токена доступа из объекта ответа.
}

private String buildRequestBody(PayPal pay) {
 // Создание запроса авторизации платежа в формате JSON.
}
}
```

- ❶ Статические идентификационные данные, используемые для получения токена доступа.
- ❷ Локальная копия токена доступа.
- ❸ Первая попытка авторизации платежа с использованием текущего токена доступа.
- ❹ Если сгенерировано исключение `Unauthorized`, то вызывается функция `refreshToken`.
- ❺ Попытка восстановления после исключения `Unauthorized` с помощью повторного вызова метода авторизации.
- ❻ Возврат конечного результата.
- ❼ Создание тела запроса в формате JSON.
- ❽ Предоставление значения текущего токена доступа для авторизации.
- ❾ Авторизация платежа.
- ❿ Генерация исключения `Unauthorized` на основе кода ответа.
- ⓫ Запрос нового токена доступа.
- ⓬ Предоставление статических идентификационных данных при запросе нового токена доступа.
- ⓭ Парсинг нового токена доступа из ответа в формате JSON.

Главное в этом паттерне – обертывание первого обращения к REST API авторизации платежа PayPal в контейнерный тип попытки, представляющий вычисление, результатом которого может быть либо исключение, либо возврат успешно вычисленного значения. Кроме того, это позволяет объединить вычисления в цепочку, т. е. мы получаем возможность обработать исключение более понятным способом. В примере, показанном в листинге 9.9, вся логика «попытка / ошибка / обновление токена / повторная попытка» выполняется всего лишь в пяти строках кода. Можно было бы просто реализовать ту же логику, используя более привычные конструкции `try/catch`, но понять ее было бы гораздо сложнее.

Первый вызов, обернутый в конструкцию `try`, вызывает REST API авторизации платежа PayPal, и если вызов успешный, то возвращается ответ в формате JSON. Более интересный сценарий выполняется, когда при первом вызове возникает ошибка, поскольку

ку функция, расположенная внутри метода `onFailure`, вызывается, если и только если сгенерировано исключение типа `UnauthorizedException`. А это происходит только при возврате из первого вызова кода состояния 500.

Функция, расположенная внутри метода `onFailure`, пытается исправить ситуацию, обновляя токен доступа. Далее используется метод восстановления для выполнения следующего обращения к REST API авторизации платежа PayPal, теперь с обновленным токеном доступа. Таким образом, если первоначальная ошибка возникла из-за окончания срока действия токена сеанса, то такой блок кода попытается автоматически устранить проблему без ручного вмешательства или паузы в работе. Сообщение даже не нужно помечать как ошибочное и повторять его передачу позже, что очень важно, поскольку данная функция взаимодействует непосредственно с клиентским приложением, где время ответа имеет особое значение.

### Проблемы и условия

Хотя это чрезвычайно удобный паттерн для использования при взаимодействии с внешней системой, например веб-сервисом или базой данных, всегда необходимо учитывать следующие факторы при реализации паттерна «обновление идентификационных данных» в функции `Pulsar`:

- этот паттерн можно применять только при взаимодействии с механизмами аутентификации на основе токенов, предоставляющими API обновления токена, доступного для функции `Pulsar`;
- токены, возвращаемые из API обновления, должны сохраняться в защищенной локации, чтобы предотвратить неавторизованный доступ к сервису.

## 9.3. Несколько уровней устойчивости

Вероятно, вы уже заметили, что в предыдущем разделе всегда использовался одновременно только один из всех описанных паттернов. А если в функции `Pulsar` потребуется применить несколько таких паттернов? Возможны ситуации, в которых весьма полезно создать функцию, использующую паттерны «повтор», «кеш» и «прерыватель замкнутого цикла».

К счастью, как было отмечено в начале главы, можно с легкостью декорировать вызов удаленного сервиса любым количеством вышеописанных паттернов из библиотеки `resilience4j` для формирования нескольких уровней устойчивости в функциях `Pulsar`. В листинге 9.10 показано, как просто это делается для функции `GeoEncoding`.

**Листинг 9.10.** Использование нескольких паттернов обеспечения устойчивости в функции Pulsar

```

public class GeoEncodingService implements Function<Address, Void> {
 private boolean initialized = false;
 private Cache<String, String> cacheContext;
 private CircuitBreakerConfig config;
 private CircuitBreakerRegistry registry;
 private CircuitBreaker circuitBreaker;
 private RetryRegistry rertyRegistry;
 private Retry retry;

 public Void process(Address addr, Context ctx) throws Exception {
 if (!initialized) {
 init(ctx);
 }

 CheckedFunction1<String, String> resilientFunction = Decorators
 .ofCheckedSupplier(getFunction(addr))
 .withCache(cacheContext)
 .withCircuitBreaker(circuitBreaker)
 .withRetry(retry)
 .decorate();

 LatLon geo = getLocation(Try.of(() ->
 resilientFunction.apply(addr.toString()))).get();

 if (geo != null) {
 addr.setGeo(geo);
 ctx.newOutputMessage(ctx.getOutputTopic(), AvroSchema.of(Address.class))
 .properties(ctx.getCurrentRecord().getProperties())
 .value(addr)
 .send();
 } else {
 // Выполнен корректный вызов, но не получена корректная геоинформация.
 }

 return null;
 }

 private void init(Context ctx) {
 // Конфигурирование кеша (только один раз).
 CacheManager cacheManager = Caching.getCachingProvider()
 .getCacheManager();

 cacheContext = Cache.of(cacheManager
 .createCache("addressCache", new MutableConfiguration<>()));

 config = CircuitBreakerConfig.custom()
 ...
 cbRegistry = CircuitBreakerRegistry.of(config);
 circuitBreaker = cbRegistry.circuitBreaker(ctx.getFunctionName());
 }
}

```

```
RetryConfig retryConfig = RetryConfig.custom()
...
retryRegistry = RetryRegistry.of(retryConfig);
retry = retryRegistry.retry(ctx.getFunctionName());
initialized = true;
}
}
```

- ❶ Вызов Google Map REST API.
- ❷ Декорирование паттерном «кеш».
- ❸ Декорирование паттерном «прерыватель замкнутого цикла».
- ❹ Декорирование стратегией повторной попытки.
- ❺ Вызов декорированной функции для получения реальных значений широты/долготы.
- ❻ Инициализация конфигураций паттернов обеспечения устойчивости, как в более ранних примерах.

В этой конкретной конфигурации кеш будет проверяться перед вызовом функции. Все ошибочные вызовы будут выполняться повторно на основе определенной здесь конфигурации, и если определенное количество вызовов завершается с ошибкой, то сработает прерыватель замкнутого цикла, запрещая последующие обращения к удаленному сервису в течение достаточного интервала времени, позволяющего сервису восстановиться.

Следует напомнить, что эти паттерны обеспечения устойчивости, включаемые непосредственно в исходный код, также используются в сочетании со средствами обеспечения устойчивости, предоставляемыми фреймворком Pulsar Functions, такими как автоматический перезапуск подов Kubernetes. Применяемые в совокупности все эти средства могут помочь минимизировать время простоя и поддерживать непрерывную работу приложений на основе Pulsar Functions.

## 9.4. Резюме

- Существует несколько различных неблагоприятных событий, которые могут оказывать отрицательное воздействие на функцию Pulsar, и замедленная обратная реакция на сообщения является полезной метрикой для обнаружения ошибок и сбоев.
- Библиотеку `resiliency4j` можно использовать для реализации различных широко известных паттернов обеспечения устойчивости в функциях Pulsar, особенно в тех, которые взаимодействуют с внешними системами.
- Можно использовать несколько паттернов в одной функции Pulsar для усиления ее устойчивости к сбоям и ошибкам.

# 10

## Доступ к данным

### Темы:

- сохранение и извлечение данных с помощью Pulsar Functions;
- использование внутреннего механизма сохранения состояния Pulsar для сохранения и извлечения данных;
- доступ к данным из внешних систем с использованием Pulsar Functions.

До сих пор вся информация, используемая Pulsar Functions, была предоставлена во входящих сообщениях. Это эффективный способ обмена данными, но не менее эффективным и затребованным способом является обмен информацией Pulsar Functions с некоторым другим объектом. Самым большим недостатком такого подхода является создание зависимости от источника сообщений, предоставляющего Pulsar Functions информацию, необходимую для выполнения работы. Это нарушает принцип инкапсуляции объектно ориентированного проектирования, в соответствии с которым внутренняя логика функции не должна быть видимой для внешней окружающей среды. При вышеупомянутом подходе при любых изменениях логики в одной функции могут потребоваться изменения в предшествующих ей функциях, предоставляющих входящие сообщения.

Рассмотрим вариант использования, в котором вы пишете функцию, требующую контактную информацию клиента, в том числе номер мобильного телефона. Вместо передачи сообщения, содержащего всю эту информацию, не проще было бы передавать только лишь идентификатор клиента, который затем можно использовать в функции для запроса к базе данных и извлечения необходимой информации? В действительности это и есть широко применяемый паттерн доступа, если требуемая информация существует во внешнем источнике данных, например в базе данных. Такой подход обеспечивает инкапсуляцию и изолирует изменения в одной функции от прямого воздействия на другие функции на основании того, что каждая функция сама собирает нужную ей информацию вместо передачи во входящем сообщении.

В этой главе подробно рассматривается несколько вариантов использования, требуемых для сохранения и/или извлечения данных из внешних систем, а также демонстрируется, как это делается с помощью Pulsar Functions. Здесь будут описаны разнообразные хранилища данных и различные критерии, используемые для выбора одной из доступных технологий.

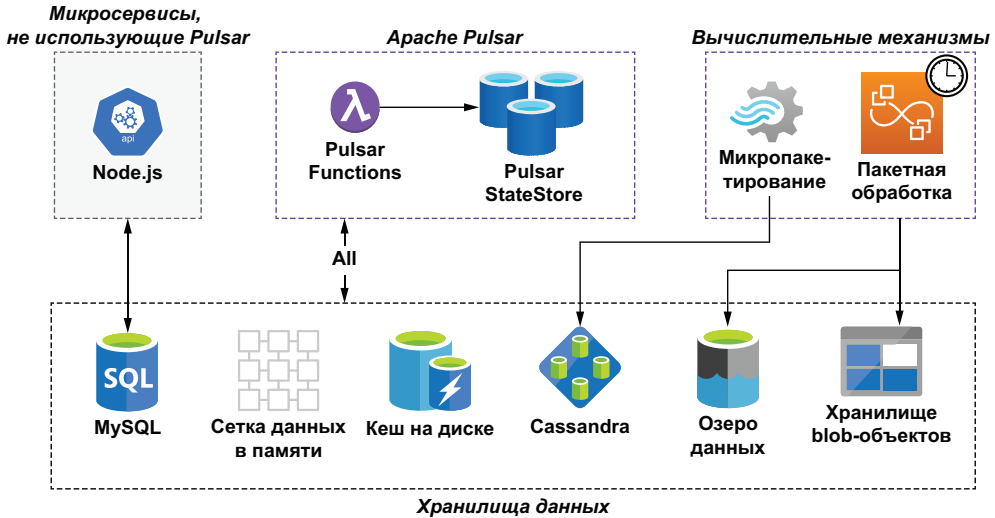
## 10.1. Источники данных

Приложение GottaEat, которое мы разрабатывали до настоящего момента, размещается в контексте гораздо более крупной промышленной архитектуры, состоящей из множества технологий и платформ хранения данных, как показано на рис. 10.1. Эти платформы хранения данных используются многими приложениями и вычислительными механизмами внутри организации GottaEat и действуют как единственный источник истинной информации для всего предприятия в целом.

На рис. 10.1 можно видеть, что фреймворк Pulsar Functions может получить доступ к данным из разнообразных источников – из сеток данных в памяти с минимальной задержкой, из кеша, размещенного на диске, из озер данных с существенной задержкой и из хранилищ blob-объектов<sup>1</sup>. Хранилища данных позволяют получить доступ к информации, предоставляемой другими приложениями и вычислительными механизмами. Например, приложение GottaEat содержит отдельную базу данных MySQL, используемую для хранения информации о клиентах и водителях, – идентификационные данные для регистрации, контактную информацию, домашний адрес и т. п. Такую информацию собирают некоторые микросервисы, не использующие Pulsar, как показано на рис. 10.1. Эти микросервисы предоставляют услуги, не связанные с потоковой обработкой, например регистрация пользователя, обновление аккаунта и т. д.

---

<sup>1</sup> Blob – binary large object – большой двоичный объект.



**Рис. 10.1.** Корпоративная архитектура организации GottaEat состоит из разнообразных вычислительных механизмов и технологий хранения данных. Поэтому чрезвычайно важно, чтобы мы могли получить доступ ко всем этим многочисленным репозиториям данных из функций Pulsar

В корпоративной архитектуре GottaEat существуют распределенные вычислительные механизмы, такие как Apache Spark и Apache Hive, выполняющие пакетную обработку особо крупных наборов данных, хранящихся в озере данных (data lake). Одним из таких наборов являются данные о текущей локации водителя, содержащие координаты широты и долготы каждого водителя, зарегистрированного в приложении. Мобильное приложение GottaEat автоматически записывает данные о локации водителя каждые 30 с и перенаправляет их в Pulsar для сохранения в озере данных. Затем этот набор данных используется для разработки и тренировки моделей машинного обучения компании. При наличии достаточного объема информации модели машинного обучения могут точно прогнозировать разнообразные важнейшие бизнес-метрики, например ожидаемые времена доставки принятых заказов или регионы города, из которых будет сделано большинство заказов в течение следующего часа, чтобы можно было направлять водителей соответствующим образом. Работа вычислительных механизмов планируется на периодической основе для вычисления различных точек данных, требуемых моделями машинного обучения, с последующим сохранением их в базе данных Cassandra, чтобы обеспечить доступ к ним из функций Pulsar, управляющих моделями машинного обучения, для получения прогнозов в реальном времени.

## 10.2. Варианты использования доступа к данным

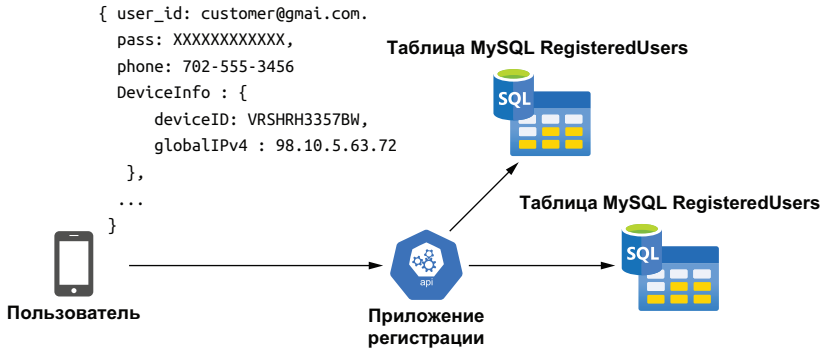
В этом разделе рассматривается организация доступа к данным из разнообразных хранилищ с использованием Pulsar Functions. Прежде чем перейти непосредственно к вариантам использования, необходимо понять, как принимается решение о том, какое хранилище данных является наиболее подходящим для каждого конкретного варианта использования. С точки зрения Pulsar Functions использование того или иного хранилища данных зависит от ряда факторов, включая степень доступности данных и максимальное время обработки, выделенное конкретной функцией Pulsar. Некоторые из функций, например оценка времени доставки, будут напрямую взаимодействовать с клиентом и водителем, поэтому их ответ потребует практически в реальном времени. Для функций таких типов доступ ко всем данным должен быть ограничен источниками с минимальной задержкой для обеспечения предсказуемого времени ответа.

В архитектуре GottaEat существуют четыре хранилища данных, которые можно классифицировать как источники с минимальной задержкой, поскольку они обеспечивают время доступа менее секунды. К ним относятся хранилище состояния Pulsar, поддерживаемое Apache BookKeeper, сетка данных в памяти (IMDG – in-memory data grid), предоставленная Apache Ignite, кеш, размещенный на диске, и распределенная колоночная база данных Apache Cassandra. Сетка хранит все данные в памяти, следовательно, обеспечивает самое быстрое время доступа. Но ее емкость ограничена объемом оперативной памяти, доступной на компьютерах, поддерживающих функции Pulsar. Сетка данных используется для передачи информации между функциями Pulsar, которые не соединены через тему, например при двухфакторной (двухуровневой) аутентификации в варианте использования проверки устройства.

### 10.2.1. Проверка устройства

Когда клиент впервые загружает мобильное приложение GottaEat из App Store и устанавливает его на свой смартфон, требуется создание аккаунта для этого конкретного клиента. Он выбирает способ регистрации, т. е. входа в систему с использованием email или пароля и отправляет свой выбранный вариант вместе с номером мобильного телефона и идентификатором устройства для его однозначной идентификации. На рис. 10.2 можно видеть, что эта информация сохраняется в базе данных MySQL, чтобы использоваться для аутентификации пользователя при последующих входах в систему.



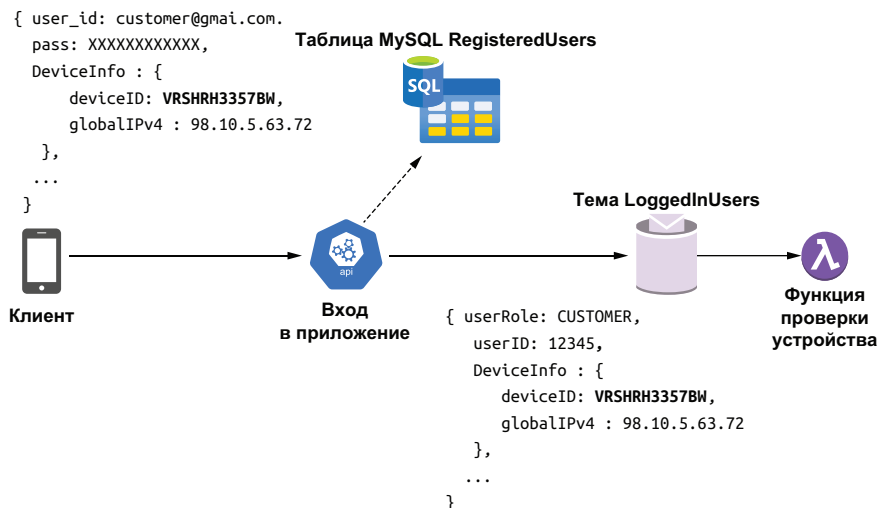


**Рис. 10.2.** Микросервис, не использующий Pulsar, управляет вариантом использования приложения регистрации и сохраняет предоставленные идентификационные данные пользователя и информацию о его устройстве в базе данных MySQL

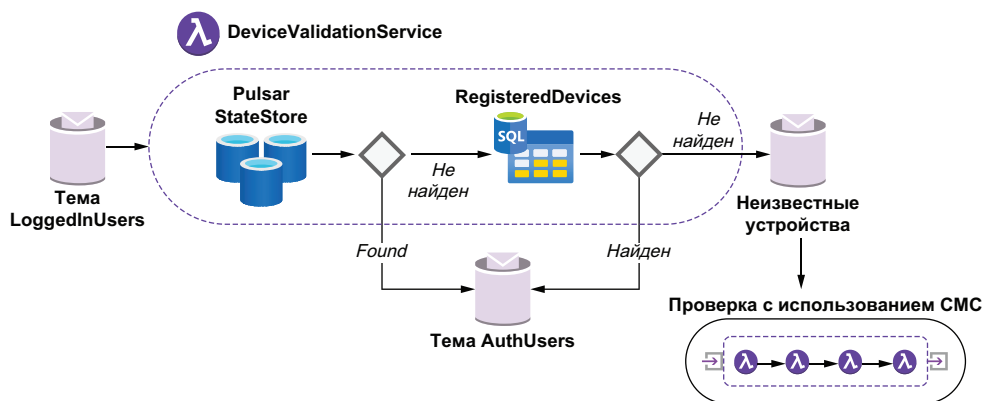
При входе (регистрации) пользователя в приложение GottaEat информация, сохраненная в базе данных, используется для проверки предоставленных идентификационных данных и аутентификации устройства, с которого устанавливается соединение. Поскольку идентификатор устройства был записан при первой регистрации пользователя, можно сравнить его с идентификатором, предоставленным пользователем, пытающимся войти в приложение. Такой подход похож на использование закладки (cookie) в веб-браузере и позволяет подтвердить, что данный пользователь действительно выполняет вход с зарегистрированного устройства. На рис. 10.3 можно видеть, что после проверки совпадения идентификационных данных пользователя со значениями, хранящимися в таблице RegisteredUsers, запись пользователя размещается в теме LoggedInUsers для дальнейшей обработки функцией DeviceValidation, позволяющей убедиться в том, что пользователь действительно использует зарегистрированное ранее устройство.

Такой подход обеспечивает дополнительный уровень защиты и предотвращает передачу мошеннических заказов от похитителей, использующих украденные идентификационные данные клиента, так как при этом требуется предоставление корректного идентификатора устройства для приема заказа. Может возникнуть вопрос: а что произойдет, если клиент просто использует новое или другое свое устройство? Это вполне допустимый сценарий, который необходимо учесть в ситуациях, когда клиент покупает новый телефон или пытается войти в свой аккаунт с другого устройства. В таком сценарии используется двухфакторная аутентификация на основе СМС, т. е. мы отправляем сгенерированный случайным образом пятизначный пин-код на номер мобильного устройства, используемого клиентом для входа, и ожидаем его от-

вета с вводом пин-кода. Это весьма часто применяемая методика для аутентификации пользователей в мобильных приложениях.



**Рис. 10.3.** Идентификатор устройства предоставляется, когда пользователь входит в мобильное приложение. Затем выполняется сравнение с идентификаторами устройств, ранее связанных с этим пользователем



**Рис. 10.4.** В функции DeviceValidationService переданный идентификатор устройства сравнивается с идентификатором устройства, наиболее часто используемого в последнее время клиентом, и если они одинаковы, то пользователь авторизуется. Иначе выполняется поиск в списке известных устройств этого пользователя и при необходимости инициализируется двухфакторная аутентификация

На рис. 10.4 можно видеть, что процесс проверки устройства сначала пытается извлечь идентификатор устройства, наиболее часто используемый в последнее время клиентом, из внутреннего хранилища состояния Pulsar, которое, как мы знаем из главы 4, является хранилищем ключ-значение и может использоваться

функциями Pulsar для сохранения информации о состоянии. Это хранилище использует Apache BookKeeper для гарантии того, что любая записанная в него информация будет постоянно храниться на диске и реплицироваться. Срок хранения таких данных превышает срок жизни любого экземпляра функции Pulsar, считывающего эти данные.

Но за надежность хранения данных в BookKeeper приходится расплачиваться производительностью, так как данные должны быть записаны на диск. Таким образом, даже если хранилище состояния предоставляет ту же семантику ключ–значение, что и другие хранилища, например IMDG, это сопровождается слишком длительной задержкой. Следовательно, использование хранилища состояния не рекомендуется для часто считываемых и записываемых данных. Этот вариант лучше подходит для редких операций чтения/записи, например для фиксации идентификатора устройства, которая требуется только один раз в каждом сеансе пользователя.

В листинге 10.1 показан логический поток в функции DeviceValidationService, когда она пытается найти и извлечь идентификатор устройства из двух различных источников данных. Базовый класс содержит весь исходный код, относящийся к установлению соединения с базой данных MySQL, а также используемый функцией DeviceRegistrationService для обновления той же базы данных, если аутентификация с применением СМС прошла успешно.

#### Листинг 10.1. Функция DeviceValidationService

```
public class DeviceValidationService extends DeviceServiceBase implements
➡ Function<ActiveUser, ActiveUser> {
 ...

 @Override
 public ActiveUser process(ActiveUser u, Context ctx) throws Exception {
 boolean mustRegister = false;

 if (StringUtils.isBlank(u.getDevice().getDeviceId())) {
 mustRegister = true;
 }

 String prev = getPreviousDeviceId(
 u.getUser().getRegisteredUserId(), ctx);

 if (StringUtils.equals(prev, EMPTY_STRING)) {
 mustRegister = true;
 } else if (StringUtils.equals(prev, u.getDevice().getDeviceId())) {
 return u;
 } else if (isRegisteredDevice(u.getUser().getRegisteredUserId(),
 u.getDevice().getDeviceId().toString())) {
 ByteBuffer value = ByteBuffer.allocate(
 u.getDevice().getDeviceId().length());
```

```

 value.put(u.getDevice().getDeviceId().toString().getBytes());
 ctx.putState("UserDevice-" + u.getDevice().getDeviceId(), value); ❸
 }

 if (mustRegister) {
 ctx.newOutputMessage(registrationTopic, Schema.AVRO(ActiveUser.class))
 .value(u)
 .sendAsync(); ❹
 return null;
 } else {
 return u;
 }
}

private String getPreviousDeviceId(long registeredUserId, Context ctx) {
 ByteBuffer buf = ctx.getState("UserDevice-" + registeredUserId); ❺
 return (buf == null) ? EMPTY_STRING :
 new String(buf.asReadOnlyBuffer().array());
}

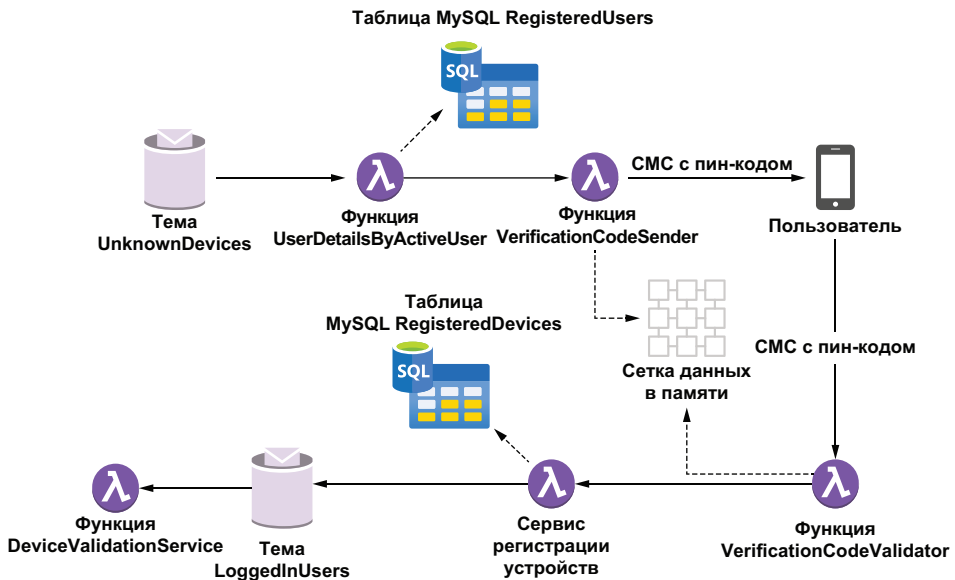
private boolean isRegisteredDevice(long userId, String deviceId) {
 try (Connection con = getDbConnection();
 PreparedStatement ps = con.prepareStatement("select count(*) "
 + "from RegisteredDevice where user_id = ? "
 + "AND device_id = ?")) {
 ps.setLong(1, userId);
 ps.setString(2, deviceId);
 ResultSet rs = ps.executeQuery();
 if (rs != null && rs.next()) {
 return (rs.getInt(1) > 0); ❻
 }
 } catch (ClassNotFoundException | SQLException e) {
 // Это игнорируется.
 }
 return false;
}
...
}

```

- ❶ Если пользователь вообще не предоставил идентификатор устройства.
- ❷ Получить предыдущий идентификатор устройства для этого пользователя из хранилища состояния.
- ❸ Для этого пользователя предыдущий идентификатор устройства отсутствует.
- ❹ Предоставленный идентификатор устройства совпадает с значением из хранилища состояния.
- ❺ Проверка: находится ли идентификатор устройства в списке известных устройств для этого пользователя.
- ❻ Это известное устройство, поэтому необходимо обновить хранилище состояния, записав в него предоставленный идентификатор устройства.

- 7 Это неизвестное устройство, поэтому публикуется сообщение для выполнения проверки с использованием СМС.
- 8 Поиск идентификатора устройства в хранилище состояния с использованием идентификатора пользователя.
- 9 Возврат значения true, если предоставленный идентификатор устройства связан с этим пользователем.

Если устройство невозможно связать с текущим пользователем, то в теме `UnknownDevices` будет опубликовано сообщение, используемое для инициализации процесса проверки с помощью СМС. Возможно, на рис. 10.5 вы заметили, что этот процесс состоит из последовательности функций Pulsar, которые обязательно должны быть согласованы друг с другом для выполнения двухфакторной аутентификации.



**Рис. 10.5.** Процесс проверки с использованием СМС выполняется комплектом функций Pulsar, который отправляет пятизначный числовой код на зарегистрированный номер мобильного телефона текущего пользователя и проверяет этот код, возвращенный пользователем

Первая функция Pulsar в процессе проверки с использованием СМС считывает сообщения из темы `UnknownDevices` и использует зарегистрированный идентификатор пользователя для извлечения номера мобильного телефона, на который будет отправлено СМС с кодом проверки. Это весьма простой и понятный паттерн доступа к данным, в котором используется главный (первичный) ключ для извлечения информации из реляционной базы данных и передачи их в другую функцию. Хотя такой подход обычно считается

антипаттерном, поскольку нарушается принцип инкапсуляции, упомянутый выше, я решил выделить его в отдельную функцию по двум причинам. Во-первых, это способствует многократному использованию такого кода, так как существует множество вариантов использования, в которых потребуется извлечение всей информации о конкретном пользователе, и я предпочел вынести соответствующую логику в отдельный класс, а не распределять ее по нескольким функциям. Такое решение упрощает сопровождение. Вторая причина заключается в том, что в дальнейшем потребуется часть этой информации в сервис `DeviceRegistration`, поэтому было принято решение передавать всю информацию полностью, а не выполнять дважды один и тот же запрос в базу данных, чтобы уменьшить задержку при обработке.

В листинге 10.2 можно видеть, что функция `UserDetailsByActiveUser` предоставляет идентификатор зарегистрированного пользователя в класс `UserDetailsLookup`, который выполняет запрос в базу данных для извлечения требуемых данных и возврата объекта `UserDetails`.

#### Листинг 10.2. Функция `UserDetailsByActiveUser`

```
public class UserDetailsByActiveUser implements
 Function<ActiveUser, UserDetails> {

 private UserDetailsLookup lookupService = new UserDetailsLookup(); ❶

 @Override
 public UserDetails process(ActiveUser input, Context ctx) throws Exception{
 return lookupService.process(
 input.getUser().getRegisteredUserId(),
 ctx); ❷ ❸
 }
}
```

- ❶ Создание экземпляра класса, выполняющего поиск в базе данных.
- ❷ Передача поля главного (первичного) ключа, используемого для поиска.
- ❸ Передача объекта контекста, чтобы класс поиска мог получить доступ к идентификационным сведениям для базы данных и т. п.

Этот паттерн просто извлекает главный (первичный) ключ из сообщения и передает его в другой класс, выполняющий поиск. Его можно применять многократно в любом варианте использования, а в приложении `GottaEat` потоки в различных функциях `Pulsar` применяют его для извлечения информации о пользователе из каждого конкретного заказа продуктов, как показано в листинге 10.3.

**Листинг 10.3.** Функция `UserDetailsByFoodOrder`

```

public class UserDetailsByFoodOrder implements
 Function<FoodOrder, UserDetails> {

 private UserDetailsLookup lookupService = new UserDetailsLookup(); ❶

 @Override
 public UserDetails process(FoodOrder order, Context ctx) throws Exception {
 return lookupService.process(
 order.getMeta().getCustomerId(),
 ctx); ❷
 } ❸
}

```

- ❶ Создание экземпляра класса, выполняющего поиск в базе данных.
- ❷ Передача поля главного (первичного) ключа, используемого для поиска.
- ❸ Передача объекта контекста, чтобы класс поиска мог получить доступ к идентификационным сведениям для базы данных и т. п.

Внутренний класс поиска, используемый для предоставления информации, показан в листинге 10.4. Он содержит логику, необходимую для выполнения запроса к таблице базы данных для извлечения всей информации. Этот паттерн доступа к данным можно использовать для извлечения информации из любой таблицы реляционной базы данных, доступ к которой требуется из какой-либо функции Pulsar.

**Листинг 10.4.** Функция `UserDetailsLookup`

```

public class UserDetailsLookup implements Function<Long, UserDetails> {

 . . .
 private Connection con;
 private PreparedStatement stmt;
 private String dbUrl, dbUser, dbPass, dbDriverClass;

 @Override
 public UserDetails process(Long customerId, Context ctx) throws
Exception {

 if (!isIntialized()) { ❶
 dbUrl = (String) ctx.getUserConfigValue(DB_URL_KEY);
 dbDriverClass = (String) ctx.getUserConfigValue(DB_DRIVER_KEY);
 dbUser = (String) ctx.getSecret(DB_USER_KEY);
 dbPass = (String) ctx.getSecret(DB_PASS_KEY);
 }

 return getUserDetails(customerId);
 }
}

```

```

private UserDetails getUserDetails(Long customerId) {
 UserDetails details = null;
 try (Connection con = getDbConnection();
 PreparedStatement ps = con.prepareStatement("select ru.user_id,
 + " ru.first_name, ru.last_name, ru.email, "
 + "a.address, a.postal_code, a.phone, a.district,"
 + "c.city, c2.country from GottaEat.RegisteredUser ru "
 + "join GottaEat.Address a on a.address_id = c.address_id "
 + "join GottaEat.City c on a.city_id = c.city_id "
 + "join GottaEat.Country c2 on c.country_id = c2.country_id "
 + "where ru.user_id = ?");) {
 2

 ps.setLong(1, customerId);
 3

 try (ResultSet rs = ps.executeQuery()) {
 if (rs != null && rs.next()) {
 details = UserDetails.newBuilder()
 .setEmail(rs.getString("ru.email"))
 .setFirstName(rs.getString("ru.first_name"))
 .setLastName(rs.getString("ru.last_name"))
 .setPhoneNumber(rs.getString("a.phone"))
 .setUserId(rs.getInt("ru.user_id"))
 .build();
 4
 }
 catch (Exception ex) {
 // Игнорируется.
 }
 }
 catch (Exception ex) {
 // Игнорируется.
 }

 return details;
 }
 . . .
}

```

- ❶ Обеспечение наличия всех требуемых свойств из объекта контекста пользователя.
- ❷ Выполнение поиска по заданному идентификатору пользователя.
- ❸ Использование идентификатора пользователя в первом параметре формируемой инструкции.
- ❹ Отображение реляционных данных в объект `UserDetails`.

Функция `UserDetailsByActiveUser` передает объект `UserDetails` в функцию `VerificationCodeSender`, которая, в свою очередь, отправляет контрольный код проверки клиенту и записывает соответствующую транзакцию в сетку данных в памяти (IMDG). Идентификатор транзакции требуется функции `VerificationCodeValidator` для проверки пин-кода, возвращаемого пользователем, но, как можно видеть на рис. 10.5, здесь нет промежуточной темы `Pulsar`,



размещенной между функциями `VerificationCodeSender` и `VerificationCodeValidator`, так что эту информацию невозможно передать в сообщении. Вместо этого приходится полагаться на внешнее хранилище данных IMDG для передачи информации.

Сетка данных в памяти вполне подходит для этой задачи, поскольку время жизни информации короткое, и требуется сохранять ее только до завершения проверки пин-кода. Если пин-код, возвращенный пользователем, правилен, то следующая функция добавит новое аутентифицированное устройство в таблицу `RegisteredDevices` для использования в дальнейшем. После этого пользователь возвращается в тему `LoggedInUser`, чтобы идентификатор проверенного устройства можно было обновить в хранилище состояния.

Поскольку функции `VerificationCodeSender` и `VerificationCodeValidator` используют IMDG для обмена данными друг с другом, я решил, что они будут совместно использовать общий базовый класс, предоставляющий канал связи с совместно используемым кешем, как показано в листинге 10.5.

#### Листинг 10.5. Класс `VerificationCodeBase`

```
public class VerificationCodeBase {

 protected static final String API_HOST = "sms-verify-api.com";

 protected IgniteClient client;
 protected ClientCache<Long, String> cache;
 protected String apiKey;
 protected String datagridUrl;
 protected String cacheName;

 protected boolean isInitalized() {
 return StringUtils.isNotBlank(apiKey);
 }

 protected ClientCache<Long, String> getCache() {
 if (cache == null) {
 cache = getClient().getOrCreateCache(cacheName);
 }
 return cache;
 }

 protected IgniteClient getClient() {
 if (client == null) {
 ClientConfiguration cfg =
 new ClientConfiguration().setAddresses(datagridUrl);
 client = Ignition.startClient(cfg);
 }
 return client;
 }
}
```

- ❶ Имя хоста стороннего сервиса, используемого для выполнения верификации по СМС.
- ❷ Тонкий клиент Apache Ignite.
- ❸ Кеш Apache Ignite, используемый для хранения данных.
- ❹ Имя кеша, к которому получают доступ оба сервиса.
- ❺ Установление соединения тонкого клиента с сеткой данных в памяти.

Функция `VerificationCodeSender` использует номер мобильного телефона из объекта `UserDetails`, предоставленного функцией `UserDetailsByActiveUser`, для вызова стороннего сервиса проверки по СМС, как показано в листинге 10.6.

#### Листинг 10.6. Функция `VerificationCodeSender`

```
public class VerificationCodeSender extends VerificationCodeBase
 implements Function<ActiveUser, Void> {

 private static final String BASE_URL = "https://" + RAPID_API_HOST +
 ➡ "/send-verification-code";
 private static final String REQUEST_ID_HEADER = "x-amzn-requestid";

 @Override
 public Void process(ActiveUser input, Context ctx) throws Exception {
 if (!isInitalized()) {
 apiKey = (String) ctx.getUserConfigValue(API_KEY);
 datagridUrl = ctx.getUserConfigValue(DATAGRID_KEY).toString();
 cacheName = ctx.getUserConfigValue(CACHENAME_KEY).toString();
 }

 OkHttpClient client = new OkHttpClient();
 Request request = new Request.Builder()
 .url(BASE_URL + "?phoneNumber=" + toE164FormattedNumber(
 input.getDetails().getPhoneNumber().toString())
 + "&brand=GottaEat")
 .post(EMPTY_BODY)
 .addHeader("x-rapidapi-key", apiKey)
 .addHeader("x-rapidapi-host", RAPID_API_HOST)
 .build();

 Response response = client.newCall(request).execute();
 if (response.isSuccessful()) {
 String msg = response.message();
 String requestId = response.header(REQUEST_ID_HEADER);
 if (StringUtils.isNotBlank(requestId)) {
 getCache().put(input.getUser().getRegisteredUserId(), requestId)
 }
 }
 return null;
 }
}
```

- 1 URL стороннего сервиса, используемого для отправки СМС с кодом верификации.
- 2 Обеспечение наличия всех требуемых свойств из объекта контекста пользователя.
- 3 Формирование объекта HTTP-запроса к стороннему сервису.
- 4 Отправка объекта запроса в сторонний сервис.
- 5 Проверка успешности выполнения запроса.
- 6 Извлечение идентификатора запроса из объекта ответа.
- 7 Если получен идентификатор запроса, то сохранить его в IMDG.

Если HTTP-запрос выполнен успешно, то сторонний сервис в ответном сообщении предоставляет неповторяющееся значение идентификатора запроса, которое может использоваться функцией `VerificationCodeValidator` для проверки ответа, отправленного пользователем. В листинге 10.6 можно видеть, что идентификатор запроса сохраняется в IMDG с использованием в качестве ключа идентификатор пользователя. Далее, когда пользователь передает ответ с значением пин-кода, мы можем воспользоваться этим значением для его аутентификации, как показано в листинге 10.7.

#### Листинг 10.7. Функция `VerificationCodeValidator`

```
public class VerificationCodeValidator extends VerificationCodeBase
 implements Function<SMSVerificationResponse, Void> { 1

 public static final String VALIDATED_TOPIC_KEY = "";
 private static final String BASE_URL = "https://" + RAPID_API_HOST
 + "/check-verification-code"; 2
 private String valDeviceTopic; 3

 @Override
 public Void process(SMSVerificationResponse input, Context ctx)
 throws Exception {
 if (!isIntialized()) {
 apiKey = (String) ctx.getUserConfigValue(API_KEY).orElse(null);
 valDeviceTopic = ctx.getUserConfigValue(VALIDATED_TOPIC_KEY).toString();
 }

 String requestID = getCache().get(input.getRegisteredUserId()); 4

 if (requestID == null) { 5
 return null;
 }

 OkHttpClient client = new OkHttpClient();
 Request request = new Request.Builder()
 .url(BASE_URL + "?request_id=" + requestID + "&code="
 + input.getResponseCode())
 .post(EMPTY_BODY)
 .addHeader("x-rapidapi-key", apiKey)
```

```

 .addHeader("x-rapidapi-host", RAPID_API_HOST)
 .build();
 6

 Response response = client.newCall(request).execute();
 if (response.isSuccessful()) {
 7
 ctx.newOutputMessage(valDeviceTopic,
 Schema.AVR0(SMSVerificationResponse.class))
 .value(input)
 .sendAsync();
 8
 }
 return null;
 }
}

```

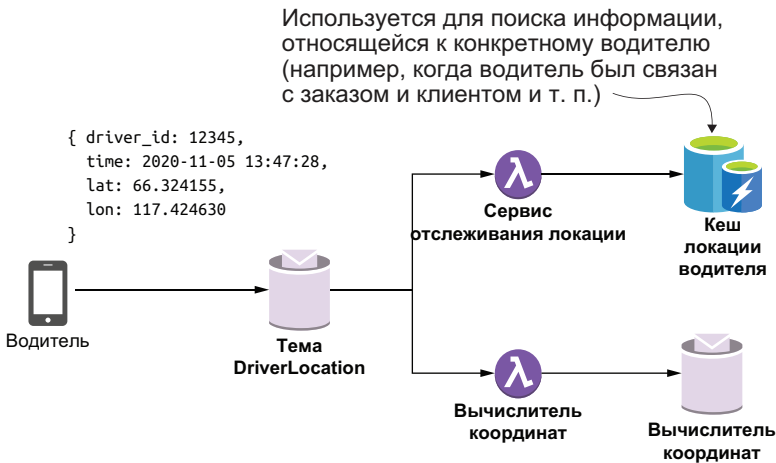
- ❶ Прием ответа с объектом СМС для верификации.
- ❷ URL стороннего сервиса, используемого для выполнения верификации по коду из СМС.
- ❸ Тема Pulsar для публикации в ней проверенного устройства.
- ❹ Получение идентификатора запроса из сетки данных в памяти (IMDG).
- ❺ Если не был найден идентификатор запроса для текущего пользователя, то выполнение функции завершается.
- ❻ Формирование HTTP-запроса к стороннему сервису.
- ❼ Если пользователь передал ответ с правильным пин-кодом, то устройство зафиксировано как корректное.
- ❽ Публикация нового сообщения в теме проверенных устройств.

Этот поток между двумя функциями, включенный в процесс проверки с использованием СМС, показывает, как можно организовать обмен информацией между функциями Pulsar, не соединенными промежуточной темой. Но самым значительным ограничением такой методики является объем данных, который можно сохранить непосредственно в памяти сетки данных. Для более крупных наборов данных, совместно используемых функциями Pulsar, более подходящим решением является кеш, размещенный на жестком диске.

### 10.2.2. Данные о локации водителя

Каждые 30 с сообщение о локации отправляется из мобильного приложения водителя в тему Pulsar. Эти сведения представляют собой самые изменчивые и чрезвычайно важные наборы данных для GottaEat. Переданную информацию необходимо не только сохранить в озере данных для тренировки моделей машинного обучения, она также полезна для нескольких вариантов использования в реальном времени, например для оценки времени прохождения по определенному маршруту в локацию, где заказ берется для доставки или передается клиенту, для определения направлений движения или для предоставления возможности клиенту отслеживать местонахождение своего заказа во время доставки и т. п.

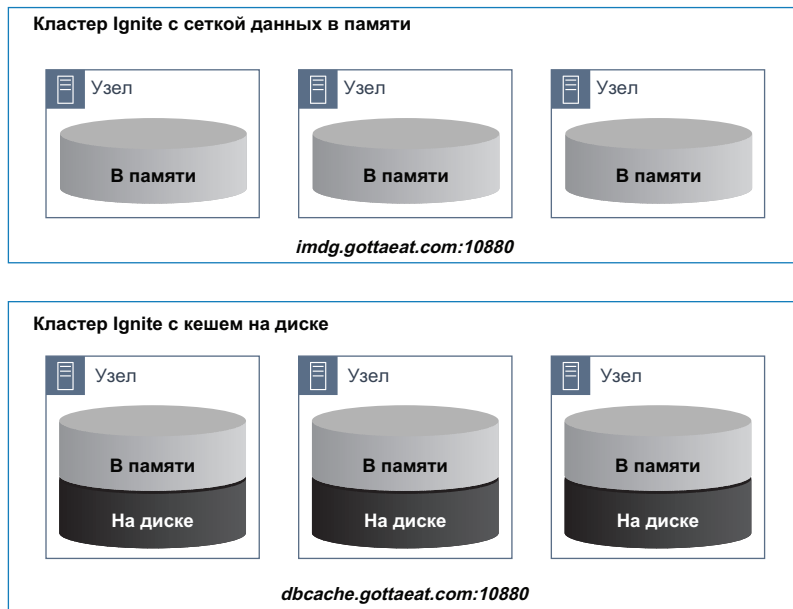
Мобильное приложение GottaEat использует сервисы определения локации, предоставляемые операционной системой смартфона, чтобы получить широту и долготу текущей локации водителя. Эта информация дополняется текущей меткой времени и идентификатором водителя перед публикацией в теме DriverLocation. Информация о местонахождении водителя будет потребляться разнообразными способами, которые, в свою очередь, определяют, как должны храниться, дополняться и сортироваться данные. Один из способов обеспечения доступа к данным о локации – непосредственно по идентификатору водителя, если необходимо найти самую последнюю локацию конкретного водителя.



**Рис. 10.6.** Каждый водитель периодически отправляет информацию о своем местонахождении в тему DriverLocation. Затем эта информация обрабатывается несколькими функциями Pulsar перед записью в постоянное хранилище данных

На рис. 10.6 можно видеть, что функция LocationTrackingService потребляет эти сообщения и использует их для обновления кеша локации водителя, представляющего собой глобальный кеш, хранящий все обновления локации в форме пар ключ–значение, где ключом является идентификатор водителя, а значением – данные о локации. Я решил сохранять эту информацию в кеше, размещенном на жестком диске, чтобы сделать его доступным любой функции Pulsar при необходимости. В отличие от IMDG кеш на диске будет сбрасывать данные, не уместающиеся в памяти, на локальный диск для сохранения, тогда как IMDG просто незаметно отбрасывает «лишние» данные. С учетом предполагаемого объема данных, ожидаемого после начала и продолжения деятельности компании, кеш с использованием жесткого диска является наилучшим вариантом выбора для хранения информации, критичной ко времени, гарантируя, что никакая ее часть не будет потеряна.

Для упрощения мы решили использовать некоторую внутреннюю технологию (Apache Ignite), чтобы одновременно предоставлять IMDG и кеш с использованием жесткого диска для GottaEat. Такое решение не только сокращает количество API, которые должны освоить наши разработчики, но также упрощает работу группы DevOps, потому что от них потребуется развертывание и мониторинг всего лишь двух различных кластеров на основе одинакового программного обеспечения, как показано на рис. 10.7. Единственным различием между этими кластерами является значение одного логического параметра конфигурации с именем `persistenseEnabled`, который разрешает сохранение данных кеша на жестком диске, т. е. позволяет хранить больший объем данных.



**Рис. 10.7.** В архитектуре предприятия GottaEat существуют два различных кластера, использующих Apache Ignite, — один сконфигурирован для постоянного хранения данных кеша на жестком диске, другой не обеспечивает такой возможности

В листинге 10.8 можно видеть, что для доступа к данным в памяти и в кеше на диске используется одинаковый API (т. е. при чтении и записи значений используется единый ключ). Различие состоит только в ожидаемом времени поиска при извлечении данных. При использовании IMDG все данные гарантированно находятся в памяти, следовательно, постоянно обеспечивается быстрый поиск. Но при этом не гарантируется, что данные доступны при попытке их чтения.

Напротив, кеш с использованием диска в памяти хранит только самые свежие данные, а большую часть данных сохраняет на диске по мере роста их объема. С учетом существенного объема данных,

хранящихся на диске, общее время поиска в этом типе кеша будет приблизительно на порядок медленнее. Таким образом, лучше всего использовать этот тип хранилища, если доступность данных важнее скорости поиска.

#### Листинг 10.8. Функция LocationTrackingService

```
public class LocationTrackingService implements
 Function<ActiveUser, ActiveUser> {

 private IgniteClient client;
 private ClientCache<Long, LatLon> cache;

 private String datagridUrl;
 private String locationCacheName;

 @Override
 public ActiveUser process(ActiveUser input, Context ctx) throws Exception {

 if (!initialized()) {
 datagridUrl = ctx.getUserConfigValue(DATAGRID_KEY).toString();
 locationCacheName = ctx.getUserConfigValue(CACHENAME_KEY).toString();
 }

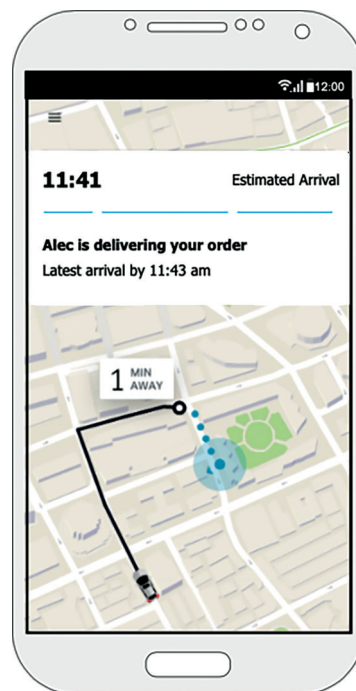
 getCache().put(
 input.getUser().getRegisteredUserId(),
 input.getLocation());

 return input;
 }

 private ClientCache<Long, LatLon> getCache() {
 if (cache == null) {
 cache = getClient().getOrCreateCache(locationCacheName);
 }
 return cache;
 }

 private IgniteClient getClient() {
 if (client == null) {
 ClientConfiguration cfg =
 new ClientConfiguration().setAddresses(datagridUrl);
 client = Ignition.startClient(cfg);
 }
 return client;
 }
}
```

- ❶ Использование идентификатора пользователя в качестве ключа.
- ❷ Добавление локации в кеш с использованием диска.
- ❸ Создание кеша локации.
- ❹ Использование клиента Apache Ignite.

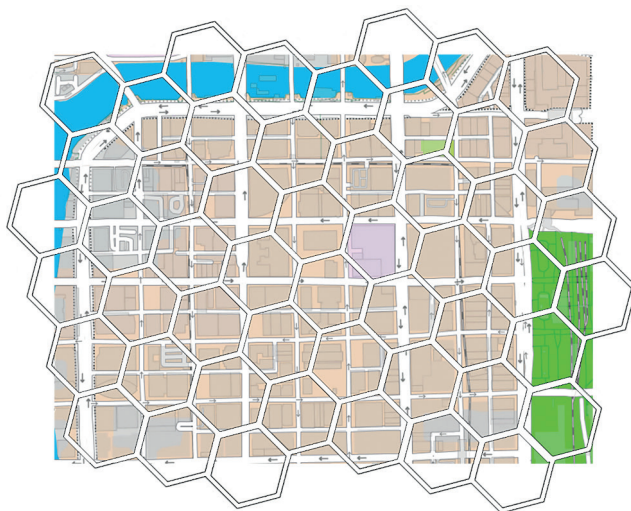


**Рис. 10.8.** Данные о локации водителя используются для обновления карты, отображенной на экране мобильного устройства клиента, и он может наблюдать за текущей локацией водителя с указанием ожидаемого времени доставки

Информация о локации в кеше с использованием диска далее может применяться для определения текущего местонахождения конкретного водителя, связанного с заказом клиента, и передаваться в программное обеспечение карты на мобильном устройстве клиента, чтобы он мог визуально отслеживать состояние своего заказа, как показано на рис. 10.8. Еще один вариант использования данных о локации водителя – определение, какой водитель больше всего подходит для выполнения конкретного текущего заказа. Одним из факторов в таком определении является близость водителя к клиенту и к ресторану, комплектующему продукты по заказу. Теоретически это возможно, но вычисление расстояния между водителем и адресом доставки при каждом приеме заказа требует слишком много времени.

Поэтому необходим способ немедленной идентификации водителей, расположенных достаточно близко к точкам приема/передачи заказа, практически в реальном времени. Один из способов – разделение карты на небольшие логические шестиугольные области, называемые ячейками (cells), как показано на рис. 10.9, с последующим группированием водителей на основе их текущего местонахождения в одной из ячеек. Таким образом, при регистрации заказа нужно только определить, в какой ячейке расположен адрес доставки, а затем в первую очередь выбрать водителей, в текущий момент находящихся в той же ячейке. Если такие водители не найдены, то можно расширить поиск по смежным ячейкам. Сортируя водителей по такой методике, мы получаем возможность выполнить поиск гораздо эффективнее.

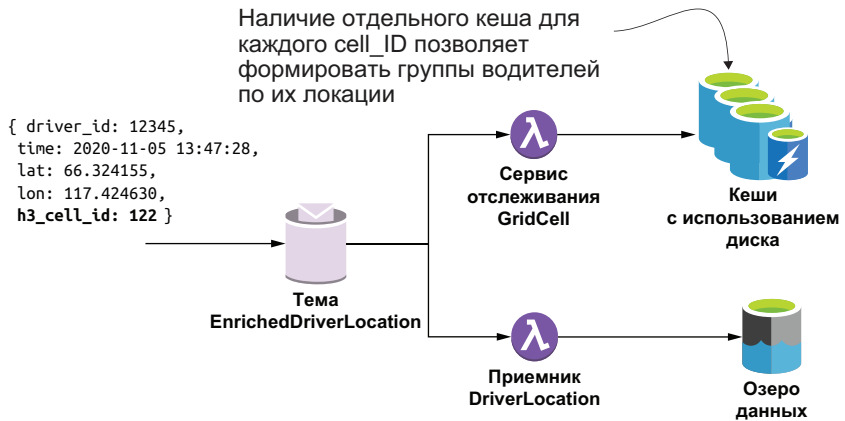




**Рис. 10.9.** Применяется глобальная сетка из шестиугольных ячеек, накладываемая поверх двумерной карты. Каждая пара широта/долгота отображается ровно в одну ячейку этой сетки. Затем водители группируются по ячейкам, в которых они находятся в текущий момент

Другой функцией Pulsar на рис. 10.6 является `GridCellCalculator`, использующая проект с открытым исходным кодом H3 (<https://github.com/uber/h3-java>), разработанный компанией Uber для определения ячейки, в которой находится водитель в текущий момент, и добавления соответствующего идентификатора ячейки в данные о локации перед их публикацией в теме `EnrichedDriverLocation` для дальнейшей обработки. Код этой функции не включен в текущую главу, но если вы хотите рассмотреть реализацию более подробно, то найдете исходный код в соответствующем репозитории, связанном с книгой.

На рис. 10.10 можно видеть, что существует глобальный комплект кешей, использующих жесткий диск (по одному на каждый идентификатор ячейки) для сохранения данных о локациях водителей с конкретным идентификатором ячейки. Предварительная сортировка водителей таким способом позволяет сужать область поиска всего лишь до нескольких ячеек, которые нас интересуют, и игнорировать остальные. В дополнение к ускорению принятия решений о назначении водителей такое группирование водителей по идентификатору ячейки позволяет анализировать географическую информацию для установки динамически меняющихся расценок и принятия решений на уровне всего города в целом, например применение ценообразования с поправочным коэффициентом в зависимости от спроса и стимулирование перемещения в ячейки с недостаточным количеством водителей для соответствия объему заказов и т. д.



**Рис. 10.10.** Сообщения в теме EnrichedDriverLocation дополнены идентификаторами ячеек H3, по которым в дальнейшем формируются группы водителей

В листинге 10.9 можно видеть, что функция GridCellTrackingService использует идентификатор ячейки из каждого входящего сообщения, чтобы определить, в каком кеше публиковать данные о локации. Для кешей с использованием диска применяют следующее соглашение об именовании: drivers-cell-XXX, где XXX – идентификатор ячейки H3. Таким образом, если принято сообщение с идентификатором ячейки 122, то его данные помещаются в кеш drivers-cell-122.

#### Листинг 10.9. Функция GridCellTrackingService

```

import com.gottaeat.domain.driver.DriverGridLocation;
import com.gottaeat.domain.geography.LatLon;

public class GridTrackingService implements Function<DriverGridLocation, Void> {

 static final String DATAGRID_KEY = "datagridUrl";

 private IgniteClient client;
 private String datagridUrl;

 @Override
 public Void process(DriverGridLocation in, Context ctx) throws Exception {
 if (!initialized()) {
 datagridUrl = ctx.getUserConfigValue(DATAGRID_KEY).toString();
 }

 getCache(input.getCellId())
 .put(input.getDriverLocation().getDriverId(),
 input.getDriverLocation().getLocation());
 return null;
 }
}

```

1  
2  
3

```
private ClientCache<Long, LatLon> getCache(int cellID) {
 return getClient().getOrCreateCache("drivers-cell-" + cellID);
}

private IgniteClient getClient() {
 if (client == null) {
 ClientConfiguration cfg =
 new ClientConfiguration().setAddresses(datagridUrl);
 client = Ignition.startClient(cfg);
 }
 return client;
}

private boolean initialized() {
 return StringUtils.isNotBlank(datagridUrl);
}
}
```

- ❶ Выбор кеша для вычисленного конкретного идентификатора ячейки.
- ❷ Использование идентификатора пользователя (водителя) в качестве ключа.
- ❸ Добавление данных о локации в кеш с использованием диска.
- ❹ Возврат или создание кеша для заданного идентификатора ячейки.

Другим потребителем дополненных сообщений с данными о локации является коннектор ввода-вывода Pulsar DriverLocationSink, группирующий сообщения и записывающий их в озеро данных. В рассматриваемом здесь случае конечной целью этого приемника является файловая система HDFS, которая позволяет другим подразделениям организации выполнять анализ данных с использованием различных вычислительных механизмов, например Hive, Storm или Spark.

Поскольку входящие данные имеют более важное значение для моделей оценки времени доставки, приемник можно сконфигурировать для записи данных в особый каталог в файловой системе HDFS. Это позволит выполнять предварительное фильтрование самых свежих данных и группировать их для ускоренной обработки процессом, использующим поступившую информацию для подготовки данных для вектора признаков (характеристик) времени доставки в базе данных Cassandra.

### 10.3. Резюме

- Внутреннее хранилище состояний Pulsar предоставляет удобную локацию для хранения данных, доступ к которым происходит редко, без необходимости их сохранения во внешней системе.
- Из функций Pulsar можно получить доступ к разнообразным внешним источникам данных, в том числе к сеткам данных в памяти, кешам с использованием жесткого диска, реляционным базам данных и т. д.
- При определении используемых систем хранения данных следует учитывать вероятные задержки и функциональные возможности хранилищ. Системы с малым временем задержки обычно больше подходят для систем потоковой обработки.

# *Машинное обучение в Pulsar*

## **Темы:**

- возможность использования Pulsar Functions для поддержки машинного обучения практически в реальном времени;
- разработка и сопровождение набора входных данных, требуемого моделью машинного обучения для формирования прогноза;
- выполнение любой модели с поддержкой PMML в функции Pulsar;
- выполнение моделей без поддержки PMML в функции Pulsar.

Одной из главных целей машинного обучения является извлечение полезной информации из необработанных исходных данных. Полезность информации означает, что ее можно использовать для принятия стратегических управляемых данными решений, приводящих к положительным результатам для бизнеса и клиентов. Например, при каждой передаче заказа клиентом в приложение GottaEat необходимо сообщить ему точную оценку времени доставки. Для этого требуется разработка модели машинного обучения, прогнозирующей время доставки любого заказа на основе ряда факторов, позволяющих принять более точное решение.

Обычно такая полезная информация генерируется моделями машинного обучения, которые разработаны и натренированы (обучены) для приема предварительно определенного набора входных данных, обозначенного термином «вектор признаков» (feature vector; иногда – «вектор характеристик»), и используют эту информацию для принятия решений, например в какое время заказ будет доставлен в локацию клиента. Группа специалистов по исследованию и обработке данных отвечает за разработку и тренировку таких моделей, включая определение векторов признаков. Поскольку эта книга об Apache Pulsar, а не о науке о данных, здесь мы сосредоточимся главным образом на развертывании моделей машинного обучения в фреймворке Pulsar Functions, а не на разработке и тренировке самих моделей.

## **11.1. Развертывание моделей машинного обучения**

Модели машинного обучения разрабатываются на разнообразных языках и с использованием различных комплектов инструментальных средств и обладают собственными достоинствами и недостатками. Вне зависимости от того, как разрабатываются и тренируются эти модели, в итоге они непременно должны быть развернуты в производственной среде для обеспечения реальной пользы для бизнеса. На высоком уровне существуют два режима работы моделей машинного обучения, развернутых в производственной среде: режим пакетной обработки (batch-processing mode) и режим обработки почти в реальном времени (near real-time processing mode).

### **11.1.1. Режим пакетной обработки**

Как следует из названия, выполнение модели в режиме пакетной обработки обозначает процесс, в котором крупный пакет данных передается в модель машинного обучения для одновременного создания нескольких прогнозов в отличие от подхода с прогнозированием в каждом конкретном случае.

Пример: маркетинговые сообщения электронной почты от сайта электронной коммерции, предоставляющие список рекомендуемых покупок на основе вашей истории приобретений в этом магазине и аналогичных покупок других клиентов. Поскольку такие рекомендации основаны на хронологическом и медленно изменяющемся наборе данных (т. е. на истории ваших приобретений), их можно сгенерировать в любое время. Обычно такие маркетинговые письма генерируются один раз в день и доставляются клиентам с учетом их временного пояса. Так как рекомендации создаются для всех клиентов, не отказавшихся от подписки, это самый подходящий кандидат для пакетной обработки моделью машинного обучения. Но если в рекомендациях необходимо учесть самые

последние действия пользователей, например текущее содержимое их корзины, то вы не сможете выполнить модель машинного обучения в пакетном режиме из-за недоступности таких данных. В этом сценарии потребуется развертывание модели машинного обучения в режиме обработки почти в реальном времени.

### 11.1.2. Режим обработки почти в реальном времени

Применение модели машинного обучения для генерации прогнозов в каждом конкретном случае с использованием данных, доступных только во входящей полезной нагрузке, часто обозначают термином «обработка почти в реальном времени» (near real-time processing). Этот тип обработки является значительно более сложным, чем пакетная обработка, в основном из-за ограничений времени задержки, устанавливаемых в системах, обслуживающих процесс создания прогнозов.

Подходящим кандидатом для развертывания в режиме почти реального времени является компонент оценки времени доставки в компании GottaEat, предоставляющий оценку времени доставки для каждого нового зарегистрированного заказа продуктов. Этот сервис не только формирует вектор признаков для модели машинного обучения в зависимости от данных, предоставленных как часть заказа (т. е. адреса доставки), но также должен сгенерировать оценку в течение нескольких сотен миллисекунд. Принимая решение о том, какой из вышеописанных режимов выполнения будет использован для моделей машинного обучения, вы должны учитывать следующие факторы:

- доступность всех данных, требуемых для вектора признаков модели, и место хранения этих данных. Доступны ли данные, необходимые для работы модели, непосредственно в системе, или некоторые из них предоставляются по отдельному запросу;
- насколько быстро точность рекомендаций ухудшается со временем. Можно ли предварительно вычислить рекомендации и многократно использовать их в течение некоторого определенного интервала времени;
- насколько быстро можно извлекать все данные, используемые в векторе признаков. Должны ли они храниться в памяти или в системах, не обеспечивающих доступ в (почти) реальном времени, таких как HDFS, обычные базы данных и т. д.

Возможно, вы уже догадались, что функции Pulsar являются вполне подходящими кандидатами для развертывания моделей машинного обучения в режиме почти реального времени по множеству причин. Во-первых, и это главное, функции выполняются немедленно после приема запроса и получают прямой доступ к данным, переданным как часть запроса, что исключает любые задержки при поиске

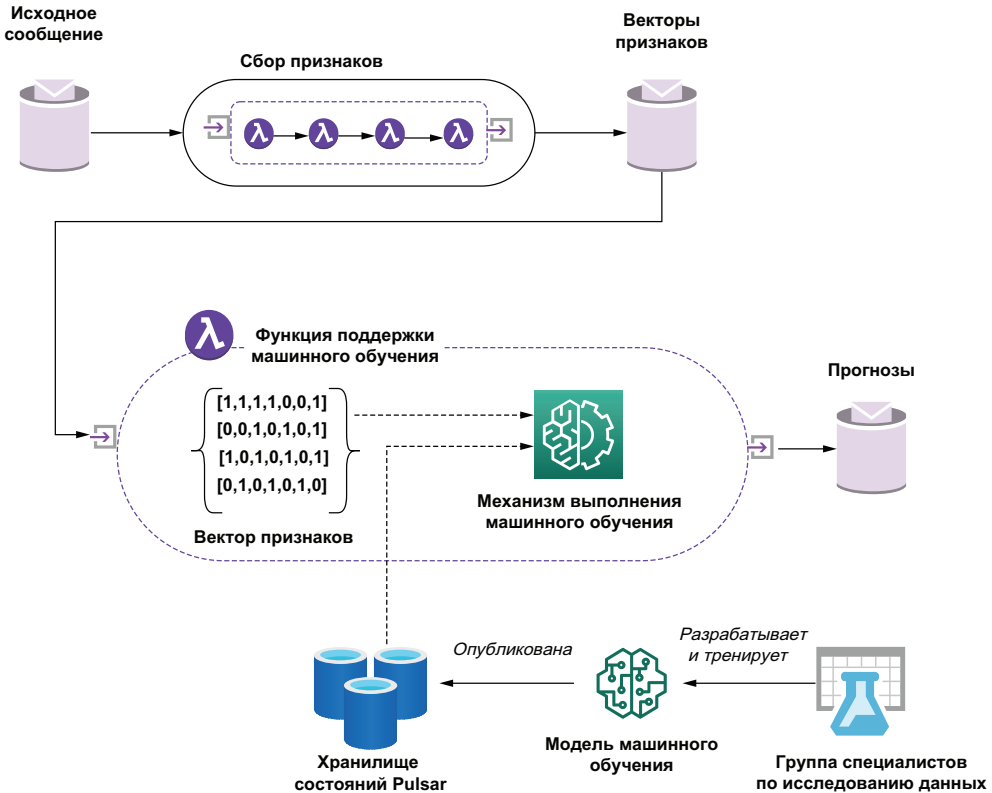
и обработке данных. Во-вторых, как мы узнали из предыдущей главы, можно извлекать данные из разнообразных внешних источников для заполнения вектора признаков любыми требуемыми данными, существующими за пределами принятого запроса. Наконец, не менее важным фактором является гибкость условия развертывания, обеспечиваемая фреймворком Pulsar Functions. Возможность написания функций на языках Java, Python и Go позволяет группе исследования данных разработать модель на языке или в фреймворке по собственному выбору и поддерживать ее развертывание в функции Pulsar с использованием сторонних библиотек, связанных с функцией для обеспечения работы модели во время выполнения.

## **11.2. Развертывание модели в режиме почти реального времени**

Поскольку существует множество фреймворков машинного обучения и все они развиваются весьма быстро, здесь представлено обобщенное решение, обеспечивающее наличие всех важных элементов, требуемых для развертывания модели машинного обучения в функции Pulsar. Мощная поддержка фреймворком Pulsar Functions различных языков программирования позволяет развертывать модели машинного обучения из разнообразных рабочих сред. Здесь представлена сторонняя библиотека, поддерживающая выполнение таких моделей. Например, существование библиотеки Java для TensorFlow позволяет развертывать и выполнять любые модели машинного обучения, разработанные с использованием инструментального комплекта TensorFlow. Аналогично если модель машинного обучения основана на библиотеке pandas, написанной на Python, то можно с легкостью написать на том же языке функцию Pulsar, включающую библиотеку pandas. Вне зависимости от выбранного языка процедура развертывания модели в режиме почти реального времени в функции Pulsar определяется одним и тем же паттерном высокого уровня.

На рис. 11.1 можно видеть, что существует последовательность функций Pulsar (возможно, единственная функция), отвечающая за сбор данных, требуемых для вектора признаков выбранной модели, из внешних источников на основе исходного сообщения. Такой подход работает как некоторая разновидность конвейера улучшения данных, добавляющего в исходные данные из сообщения дополнительные элементы, необходимые для модели машинного обучения. В примере оценки времени доставки можно получить только идентификатор ресторана, занимающегося подготовкой продуктов по заказу. Затем описанный выше конвейер должен отвечать за извлечение географической локации этого ресторана, чтобы эту информацию можно было передать в модель.





**Рис. 11.1.** Исходное сообщение, необходимое для прогноза, дополняется последовательностью вспомогательных функций Pulsar для формирования вектора признаков, предназначенного для модели машинного обучения. Затем функция, обеспечивающая поддержку машинного обучения, заполняет вектор признаков данными из других источников с малым временем задержки перед отправкой вместе с моделью машинного обучения, извлекаемой из внутреннего хранилища состояний Pulsar, в механизм выполнения машинного обучения для обработки

После того как все данные собраны, они передаются в функцию Pulsar, которая вызывает модель машинного обучения, используя стороннюю библиотеку для выполнения этой модели в соответствующем фреймворке (например, TensorFlow и т. п.). Следует отметить, что сама модель машинного обучения извлекается из объекта контекста функции Pulsar, что позволяет развертывать модель независимо от самой функции. Такой подход, отделяющий модель от упакованной и развернутой функции Pulsar, предоставляет возможность динамически менять модели на лету и, что более важно, изменять модели в зависимости от внешних факторов и условий.

Например, возможно, потребуется совершенно другая модель оценки времени доставки для периодов с высокой нагрузкой, скажем для времени ланча и обеда, в отличие от «спокойных» часов. Тогда

потребуется использование внешнего процесса для подстановки соответствующей модели в определенный момент времени. В действительности группа исследования данных могла бы натренировать ту же модель на зависимых от времени наборах данных, чтобы создать различные модели для конкретного времени суток и дней недели. Например, тренировка модели оценки времени доставки с данными, полученными в интервале от 16:00 до 17:00, создает модель, специализированную именно для этого интервала времени.

Затем функция поддержки машинного обучения отправляет заполненный вектор признаков вместе с правильной версией модели машинного обучения, извлеченной из внутреннего хранилища состояния Pulsar, в механизм выполнения машинного обучения (например, в стороннюю библиотеку), которая формирует прогноз и соответствующий уровень его достоверности. Например, механизм может вычислить прогноз значения ожидаемого времени доставки (ETA), равного семи минутам с 84 % достоверности. Далее эту информацию можно использовать для предоставления приблизительного времени доставки клиенту, сделавшему заказ.

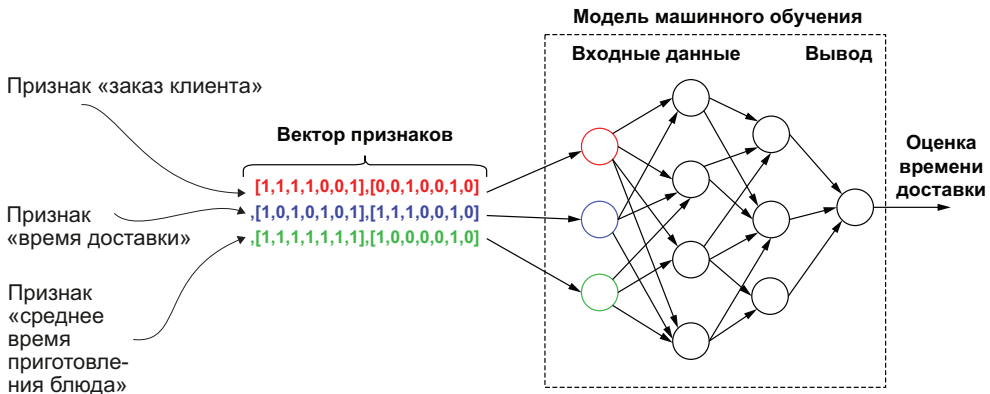
Хотя конкретная функция Pulsar может отличаться по используемому фреймворку машинного обучения, общая схема процесса остается неизменной. Сначала модель машинного обеспечения извлекается из хранилища состояния Pulsar, и ее копия сохраняется в памяти. Далее из входящего сообщения извлекаются предварительно вычисленные входные признаки и отображаются в предполагаемые поля ввода вектора признаков. Вычисляются и/или извлекаются все дополнительные признаки, которые не были заранее вычислены, затем вызывается библиотека машинного обучения для выбранного языка программирования, чтобы выполнить выбранную модель с заданным набором данных и опубликовать результат.

### 11.3. Векторы признаков

В машинном обучении термин «признак» (feature; иногда – «характеристика») означает отдельное числовое или символическое свойство, представляющее особый аспект объекта. Часто признаки объединяют в  $n$ -мерный вектор числовых признаков, или вектор признаков (feature vector), используемый в дальнейшем как входные данные для модели прогноза.

Каждый признак в таком векторе представляет отдельный фрагмент данных, используемых для формирования прогноза. Рассмотрим оценочный признак времени доставки для компании GottaEat. При каждом заказе пользователем продуктов из ресторана модель машинного обучения оценивает срок доставки заказа. Признаки для этой модели включают информацию из запроса (например, время суток или адрес доставки), хронологические признаки

(например, среднее время приготовления продукта/блюда за последние семь дней) и признаки, вычисляемые почти в реальном времени (например, среднее время приготовления продукта/блюда за последний час). Затем признаки передаются в модель машинного обучения для формирования прогноза, как показано на рис. 11.2. Многим алгоритмам машинного обучения требуется числовое представление признаков, поскольку такое представление обеспечивает обработку и статистический анализ, например линейную регрессию.



**Рис. 11.2.** Для модели оценки времени доставки требуется вектор признаков, состоящий из характеристик, взятых из многочисленных источников. Каждый признак представляет собой массив числовых значений 0 или 1

Таким образом, общепринятой практикой считается представление каждого признака числовым значением, которое делает их удобными для разнообразных алгоритмов машинного обучения.

### 11.3.1. Хранилища признаков

Для моделей, предназначенных для развертывания в режиме почти реального времени, назначаются строгие требования по задержкам, и они не могут считаться надежными при вычислениях признаков, для которых требуется доступ к обычным хранилищам данных с высокой скоростью. Невозможно постоянно обеспечивать время ответа меньше секунды при обращении к обычной реляционной базе данных.

Таким образом, требуется создание вспомогательных процессов для предварительного вычисления и индексирования этих признаков, необходимых для модели. После вычисления признаки должны быть сохранены в хранилище данных с малым временем задержки, известном как «хранилище признаков» (feature store), где к ним можно быстро получить доступ за предсказуемое время, как показано на рис. 11.3.

Исходный ключ	Признаки			
<i>restaurant_id</i>	<i>prep_time_last_7</i>	<i>prep_time_last_hour</i>		<i>avg_rating</i>
1	23	21	...	4.3
2	7	9	...	3.9
3	18	16	...	
.				
.				
.				

**Рис. 11.3.** Хранилище характеристик ресторана использует неповторяющееся поле *restaurant\_id* как исходный основной ключ, а каждая строка содержит несколько признаков для каждого ресторана. Некоторые признаки необходимы для модели оценки времени доставки, другие не имеют к ней никакого отношения

Как было отмечено выше, в нашем случае хранилище признаков расположено внутри ориентированной на столбцы базы данных с малым временем задержки, такой как Apache Cassandra, предназначенной для поддержки большого количества столбцов в одной строке без снижения производительности в процессе извлечения данных. Это позволяет хранить признаки, необходимые для различных моделей машинного обучения, в одной таблице (например, поля оценки времени доставки, такие как среднее время приготовления, можно хранить вместе с признаками, используемыми моделью рекомендаций ресторана, такими как средняя оценка (рейтинг) клиентами). Следовательно, нет необходимости создавать отдельную таблицу для каждой применяемой модели (*restaurant\_time\_delivery\_features* и т. д.). Это не только упрощает управление хранилищем признаков, но также обеспечивает использование признаков несколькими моделями машинного обучения.

В большинстве организаций проектированием таблиц в хранилище признаков занимается группа исследования данных, поскольку именно она является основным потребителем данных из этого хранилища. Таким образом, чаще всего нужно всего лишь предоставить модель машинного обучения вместе со списком признаков, требуемых как входные данные для нее, чтобы компания была подготовлена к внедрению модели в производство.

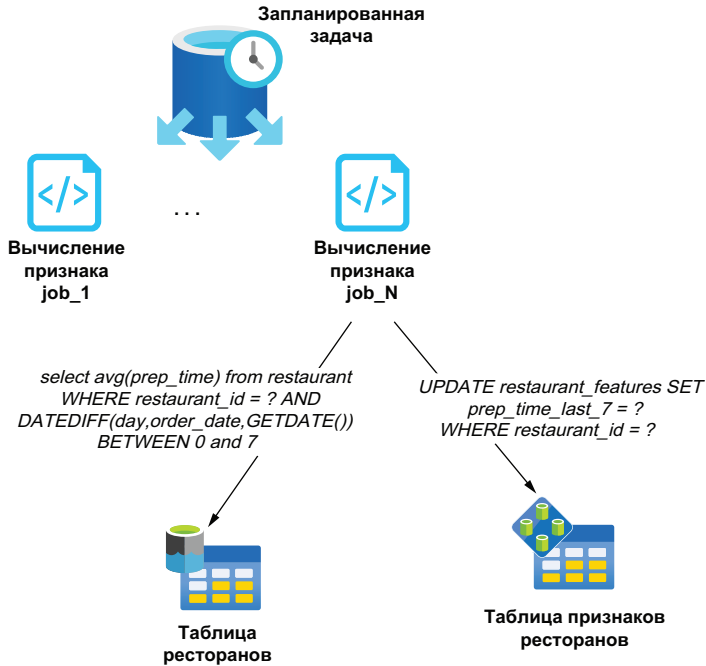
### 11.3.2. Вычисление признаков

Обычно признаки связаны с некоторым типом объекта (ресторан, водитель, клиент и т. д.), потому что каждый признак представляет отношение «принадлежит» (has-a(n); «имеет») для конкретного объекта, например ресторану «принадлежит» среднее время приготовления блюд, равное 23 мин, или водитель «имеет» среднюю оценку клиентов, равную 4.3 звезды. Затем хранилище признаков заполняется с помощью вспомогательного процесса, который вычисляет разнообразные признаки, например среднее время приготовления блюд за последние семь дней для каждого ресторана или среднее время доставки по городу за последний час.

Такие процессы должны планироваться с периодическим выполнением для кэширования результатов в хранилище признаков и обеспечения возможности их извлечения за время меньше секунды, когда поступает заказ. Механизм пакетной обработки больше подходит для выполнения предварительных вычислений признаков, так как он способен эффективно обрабатывать большой объем данных. Например, механизм распределенных запросов, такой как Apache Hive или Spark, может выполнять множество параллельных запросов к большому набору данных, хранящемуся в файловой системе HDFS, используя стандартный синтаксис SQL, как показано на рис. 11.4. Эти задачи способны асинхронно вычислять среднее время приготовления пищи для всех ресторанов в течение заданного интервала времени, и их можно запланировать для ежечасного выполнения, чтобы сохранять актуальность признаков.

На рис. 11.4 можно видеть, что запланированная задача (job) может активизировать множество параллельных подзадач (tasks) для ускоренного выполнения работы. Например, запланированная задача сначала обращается к таблице ресторанов, чтобы точно определить идентификатор конкретного ресторана, затем разделяет работу, предоставляя каждому экземпляру подзадачи вычисления признака некоторое подмножество идентификаторов ресторанов. Такая стратегия «разделяй и властвуй» существенно ускоряет обработку, позволяя выполнить работу в предельно короткий срок.

Не менее важно то, что вы согласовываете свои действия с группой (или несколькими группами), которая должна участвовать в разработке, развертывании и мониторинге этих задач вычисления признаков, поскольку теперь они становятся весьма важными для бизнеса приложениями, работающими постоянно, иначе прогнозы от моделей машинного обучения будут ухудшаться, если данные признаков не обновляются своевременно. Использование устаревших данных в моделях машинного обучения приводит к неточным прогнозам, что снижает уровень доходов и разочаровывает клиентов.



**Рис. 11.4.** Задачи вычисления признаков должны выполняться периодически для обновления значений признаков на основе самых свежих полученных данных. Механизм пакетной обработки, например Apache Hive или Spark, позволяет параллельно выполнять такие задачи

## 11.4. Оценка времени доставки

После достаточно подробного описания теоретических основ можно перейти к рассмотрению процедуры развертывания модели машинного обучения, разработанной и натренированной группой исследования данных предприятия GottaEat. Вполне подходящим кандидатом может стать модель оценки времени доставки, которую мы обсуждаем на протяжении текущей главы. Как и для любой модели машинного обучения, которую необходимо развернуть в производственной среде, группа исследования данных обязана предоставить нам копию тренированной модели для практического использования.

### 11.4.1. Экспорт модели машинного обучения

Группа исследования данных GottaEat использует разнообразные языки программирования и фреймворки для разработки моделей машинного обучения. Одной из самых больших трудностей является поиск единой среды выполнения, поддерживающей модели, разработанные на различных языках.

К счастью, существует несколько проектов, пытающихся стандартизировать форматы моделей машинного обучения с поддержкой

отдельных языков для тренировки и развертывания моделей. Проекты, подобные Predictive Model Markup Language (PMML), позволяют специалистам по исследованию данных и инженерам экспортировать модели машинного обучения, разработанные на разнообразных языках, в независимый от конкретного языка формат XML.

Языки / инструментальные комплекты	Типы моделей
- MatLab     - Python     - Python/PyTorch     - Python/sci-kit-learn     - Python/pandas     - R     - sklearn     - Spark ML     - TensorFlow     - XGboost	- Кластеризация     - Модель общей регрессии     - Модель интеллектуального анализа     - Простейший классификатор Байеса     - Модель ближайших соседей     - Нейронная сеть     - Модель регрессии     - Модель RuleSet     - Карта оценок признаков     - Метод машины опорных векторов     - Модель дерева

**Рис. 11.5.** Список языков программирования, инструментальных комплектов и типов моделей машинного обучения, поддерживаемых PMML. Все поддерживаемые модели, разработанные на перечисленных здесь языках, можно экспортировать в PMML и выполнить в функции Pulsar

На рис. 11.5 можно видеть, что формат PMML можно использовать для представления моделей, разработанных на разнообразных языках программирования и машинного обучения. Следовательно, такой подход можно применить к любому поддерживаемому типу модели, разработанной на одном из поддерживаемых языков и/или фреймворков.

Для конкретной модели оценки времени доставки группа исследования данных использовала язык R для разработки и тренировки этой модели. Но поскольку в фреймворке Pulsar Functions нет прямой поддержки выполнения кода R, группе пришлось сначала преобразовать модель на основе R в формат PMML. Такое преобразование с легкостью выполняется с помощью инструментального комплекта r2pmml, как показано в листинге 11.1.

#### Листинг 11.1. Экспорт модели на языке R в формат PMML

```
// Код разработки модели.
```

```
dte <-(distance ~ ., data = df) ❶
```

```
library(r2pmml) ❷
```

```
r2pmml(dte, "delivery-time-estimation.pmml"); ❸
```

❶ Финализация разработки модели машинного обучения.

❷ Импорт библиотеки, преобразующей модель на языке R в формат PMML.

❸ Выполнение преобразования кода R в формат PMML.

Библиотека `r2pmml` выполняет прямое преобразование объекта модели на языке R в формат PMML и сохраняет результат в локальном файле *delivery-time-estimation.pmml*. Формат файла PMML определяет поля данных для использования в модели, тип выполняемых вычислений (регрессия) и структуру модели. В рассматриваемом здесь примере структурой модели является набор коэффициентов, определенных, как показано в листинге 11.2. Теперь мы имеем спецификацию модели, готовую к промышленной эксплуатации, и применим ее к набору реальных данных для формирования прогнозов времени доставки.

### Листинг 11.2. Модель оценки времени доставки в формате PMML

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<PMML version="4.2" xmlns="http://www.dmg.org/PMML-4_2">
 <Header description="deliver time estimation">
 <Application name="R" version="4.0.3"/>
 <Timestamp>2021-01-18T15:37:26</Timestamp>
 </Header>
 <DataDictionary numberOfFields="4">
 <DataField name="distance" optype="continuous" dataType="double"/>
 <DataField name="prep_last_hour" optype="continuous"
 dataType="double"/>
 <DataField name="prep_last_7" optype="continuous" dataType="double"/>
 ...
 </DataDictionary>
 <RegressionModel functionName="regression">
 <MiningSchema>
 <MiningField name="distance"/>
 ...
 </MiningSchema>
 ...
 <NumericPredicator name="travelTime" coefficient="7.6683E-4"/>
 <NumericPredicator name="avgPreptime" coefficient="-2.0459"/>
 <NumericPredicator name="avgDeliveryTime" coefficient="9.4778E-5"/>
 ..
 </RegressionModel>
</PMML>
```

После экспорта тренированной модели в формат PMML ее копию необходимо опубликовать в хранилище состояния Pulsar, чтобы обеспечить доступ Pulsar Functions к этой модели во время выполнения. Это делается с помощью команды `pulsar-admin putstate`, показанной в листинге 11.3. Команда выгружает заданный файл в указанное пространство имен внутри хранилища состояния Pulsar. Весьма важно отметить, что это должно быть то же пространство имен, которое будет использоваться для развертывания функции Pulsar, применяющей модель, чтобы функция получила доступ для чтения PMML-файла.



**Листинг 11.3.** Выгрузка модели машинного обучения в хранилище состояния Pulsar

```

./bin/pulsar-admin functions putstate \
 --name MyMLEnabledFunction \
 --namespace MyNamespace \
 --tenant MyTenant \
 --state "{ \"key\": \"ML-Model\",
 \"byteValue\": <contents of delivery-time-estimation.pmml >}"

```

- 1 Использование команды `pulsar-admin` для выгрузки PMML-файла.
- 2 Необходимо указать правильный абонент, пространство имен и имя функции.
- 3 Выгрузка содержимого сгенерированного PMML-файла.

Возможность автоматического внесения изменений в модель, находящуюся в промышленной эксплуатации, превосходно вписывается в развивающуюся область операций машинного обучения (ML ops), в которой применяются гибкие принципы непрерывной интеграции, доставки и развертывания к ML-процессу для ускорения и упрощения развертывания моделей машинного обучения. В данном случае мы используем скрипт или инструмент конвейера CI/CD для проверки самой свежей версии модели в системе управления исходным кодом и выгружаем ее в Pulsar, применяя простой скрипт командной оболочки. Кроме того, можно запланировать вспомогательные задачи для автоматической замены различных версий одной модели по определенному условию, например в зависимости от времени суток.

### 11.4.2. Отображение вектора признаков

Группа исследования данных также должна предоставить в наше распоряжение полное определение вектора признаков, требуемого для работы модели машинного обучения, вместе со схемой отображения этих признаков внутри хранилища (или нескольких хранилищ), чтобы можно было извлекать их значения во время выполнения и передавать в модель. Группа разработки GottaEat решила, что проще хранить информацию о размещении вектора признаков в объекте `protobuf`, из-за встроенной в этот протокол поддержки ассоциированных отображений. Это позволяет хранить данные в корректном формате и с легкостью выполнять их сериализацию/десериализацию в формат, совместимый с хранилищем состояния Pulsar (т. е. в массив байтов). Кроме того, протокол `protobuf` не зависит от языков, поэтому его можно использовать с любым языком программирования, поддерживаемым Pulsar Functions, включая Java, Python и Go. Определение объекта `protobuf`, применяемого для хранения информации о размещении признаков, показано в листинге 11.4 и содержит три элемента: имя таблицы для запроса в хранилище признаков, список всех призна-

ков, хранящихся в указанной таблице и необходимых для модели машинного обучения, и ассоциированное отображение, определяющее связи между признаками в хранилище, и поля вектора признаков, передаваемые в модель.

#### Листинг 11.4. Протокол отображения вектора признаков

```
syntax = "proto3";
```

```
message FeatureVectorMapping {
 string featureStoreTable = 1;
 repeated string featureStoreFields = 2;
 map<string, string> fieldMapping = 3;
}
```

- ❶ Имя таблицы в хранилище признаков, содержащей заданные поля.
- ❷ Список всех полей, которые необходимо извлечь из таблицы хранилища признаков.
- ❸ Отображение имени каждого поля в соответствующее имя признака в векторе признаков модели.

Список признаков используется для динамического формирования SQL-запроса с целью извлечения этих признаков. Такой запрос максимально эффективен, поскольку возвращает только те значения, которые действительно необходимы, а не всю строку. Кроме того, тот же список представляет полный перечень всех ключей в структуре данных «отображение» (map), что позволяет выполнить итеративный проход по списку и извлечь все ассоциированные отображения «признак-вектор», содержащиеся в хранилище.

После передачи извлеченной информации в объект protobuf необходимо опубликовать его в хранилище состояния Pulsar, выполнив команду putstate. Имя функции Pulsar должно совпадать с именем в конвейере набора признаков, который будет выполнять поиск признаков для развернутой модели. В примере с моделью оценки времени доставки конвейер извлечения признаков содержит имя функции Pulsar RestaurantFeaturesLookup, выполняющей запрос в таблицу ресторанов хранилища признаков, используя идентификатор ресторана, который будет готовить продукты для клиента.

#### Листинг 11.5. Функция поиска признаков по идентификатору ресторана

```
public class RestaurantFeaturesLookup implements
 Function<FoodOrder, RestaurantFeatures> {

 private CqlSession session;
 private InetAddress node;
 private InetSocketAddress address;
 private SimpleStatement queryStatement;
```

```

@Override
public RestaurantFeatures process(FoodOrder input, Context ctx)
 throws Exception {
 if (!initialized()) {
 hostName = ctx.getUserConfigValue(HOSTNAME_KEY).toString();
 port = (int) ctx.getUserConfigValueOrDefault(PORT_KEY, 9042);
 dbUser = ctx.getSecret(DB_USER_KEY);
 dbPass = ctx.getSecret(DB_PASS_KEY);
 table = ctx.getUserConfigValue(TABLE_NAME_KEY);
 fields = new String(ctx.getState(FIELDS_KEY));
 sql = "select " + fields + " from " + table +
 " where restaurant_id = ?";
 queryStatement = SimpleStatement.newInstance(sql);
 }
 return getRestaurantFeatures(input.getMeta().getRestaurantId());
}

private RestaurantFeatures getRestaurantFeatures (Long id) {
 ResultSet rs = executeStatement(id);

 Row row = rs.one();
 if (row != null) {
 return CustomerFeatures.newBuilder().setCustomerId(customerId)
 .build();
 }
 return null;
}

private ResultSet executeStatement(Long customerId) {
 PreparedStatement pStmt = getSession().prepare(queryStatement);
 return getSession().execute(pStmt.bind(customerId));
}

private CqlSession getSession() {
 if (session == null || session.isClosed()) {
 CqlSessionBuilder builder = CqlSession.builder()
 .addContactPoint(getAddress())
 .withLocalDatacenter("datacenter1")
 .withKeyspace(CqlIdentifier.fromCql("featurestore"));

 session = builder.build();
 }
 return session;
}
}

```

- ❶ Получение имени хоста хранилища признаков из конфигурации.
- ❷ Получение номера порта хранилища признаков из конфигурации.
- ❸ Получение идентификационных данных для хранилища признаков.
- ❹ Получение имени таблицы в хранилище признаков для выполнения запроса.
- ❺ Получение списка всех признаков, которые необходимо извлечь из хранилища.

- 6 Формирование команды SQL, использующей заданные поля и имя таблицы.
- 7 Создание подготовленной инструкции, использующей заданные поля.
- 8 Выполнение подготовленной инструкции для заданного идентификатора.
- 9 Существует только одна запись для каждого заданного идентификатора.
- 10 Отображение полей в итоговую исходящую схему.

Функция `RestaurantFeaturesLookup` устанавливает соединение с хранилищем признаков и извлекает только те признаки, которые были определены группой исследования данных при публикации в хранилище состояния. Далее выполняется отображение их значений в тип итоговой исходящей схемы и публикуется сообщение во входящей теме функции `Pulsar`, которая будет выполнять модель оценки времени доставки.

### 11.4.3. Развертывание модели

Завершающим шагом в процессе развертывания модели машинного обучения в режиме почти реального времени является написание самой функции `Pulsar`. К счастью, после экспорта модели в формат PMML ее можно развернуть для промышленной эксплуатации, используя разнообразные механизмы выполнения, доступные для языка Java. В рассматриваемом здесь примере применяется библиотека с открытым исходным кодом `JPMML`, которую можно включить в зависимости проекта Maven, как показано в листинге 11.6.

#### Листинг 11.6. Зависимость для библиотеки JPMML

```
<dependency>
 <groupId>org.jpmmml</groupId>
 <artifactId>pmml-evaluator</artifactId>
 <version>1.5.15</version>
</dependency>
```

Эта библиотека позволяет импортировать модели в формате PMML и использовать их для формирования прогнозов. После загрузки PMML-модели в класс `evaluator JPMML` она готова к генерации прогнозов времени доставки для поступающих заказов продуктов. В листинге 11.7 можно видеть, что самым первым шагом в этом процессе является извлечение PMML-модели, находящейся в хранилище состояния `Pulsar`, и использование ее для инициализации соответствующего механизма выполнения модели. В данном случае требуется механизм выполнения модели регрессии, так как модель оценки времени доставки использует линейную регрессию.

**Листинг 11.7.** Функция оценки времени доставки

```

import org.dmg.pmml.FieldName;
import org.dmg.pmml.regression.RegressionModel;
import org.jpmmml.evaluator.ModelEvaluator;
import org.jpmmml.evaluator.regression.RegressionModelEvaluator;
import org.jpmmml.model.PMMLUtil;

import com.gottaeat.domain.geography.LatLon;
import com.gottaeat.domain.order.FoodOrderML;

public class DeliveryTimeEstimator implements
 Function<FoodOrderML, FoodOrderML> {

 private IgniteClient client;
 private ClientCache<Long, LatLon> cache;
 private String datagridUrl;
 private String locationCacheName;

 private byte[] mlModel = null;
 private ModelEvaluator<RegressionModel> evaluator;

 @Override
 public FoodOrderML process(FoodOrderML order, Context ctx)
 throws Exception {

 if (initialized()) {
 mlModel = ctx.getState(MODEL_KEY).array();
 evaluator = new RegressionModelEvaluator(
 PMMLUtil.unmarshal(new ByteArrayInputStream(mlModel)));
 }

 HashMap<FieldName, Double> featureVector = new HashMap<>();

 featureVector.put(FieldName.create("avg_prep_last_hour"),
 order.getRestaurantFeatures().getAvgMealPrepLastHour());

 featureVector.put(FieldName.create("avg_prep_last_7days"),
 order.getRestaurantFeatures().getAvgMealPrepLast7Days());

 ...

 featureVector.put(FieldName.create("driver_lat"),
 getCache().get(order.getAssignedDriverId()).getLatitude());

 featureVector.put(FieldName.create("driver_long"),
 getCache().get(order.getAssignedDriverId()).getLongitude());

 Long travel = (Long)evaluator.evaluate(featureVector)
 .get(FieldName.create("travel_time"));

```

```

order.setEstimatedArrival(System.currentTimeMillis() + travel); ❩
 return order;
}
...
}

```

- ❶ Использование механизма выполнения регрессии JPMML для модели машинного обучения, применяющей линейную регрессию.
- ❷ Входящее сообщение содержит подробности заказа и признаки, извлеченные из хранилища.
- ❸ Для модели требуется информация из сетки данных в памяти.
- ❹ Механизм выполнения модели из библиотеки JPMML.
- ❺ Извлечение модели из хранилища состояния Pulsar.
- ❻ Загрузка модели в класс механизма выполнения регрессии.
- ❼ Заполнение вектора признаков значениями, хранящимися во входящем сообщении.
- ❽ Заполнение вектора признаков данными о локации водителя из IMDG.
- ❾ Передача вектора признаков в модель и извлечение прогноза.
- ❩ Вычисление оценочного времени прибытия посредством сложения прогнозируемого времени поездки с текущим временем.

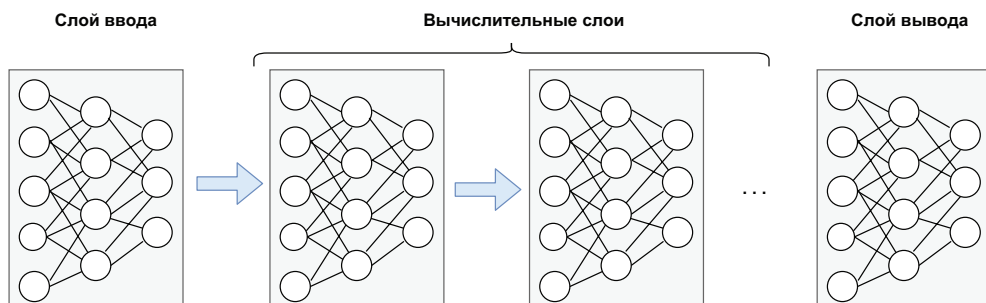
После инициализации механизма вычисления функция `Delivery-TimeEstimator` формирует вектор признаков и заполняет его значениями, содержащимися во входящем сообщении вместе с некоторыми значениями из других источников данных с малым временем задержки, например из сетки данных в памяти. В данном случае для модели требуется текущая локация водителя (широта–долгота), доступная только из IMDG.

## 11.5. Нейронные сети

Хотя формат PMML весьма гибок и поддерживает большое разнообразие языков программирования и моделей машинного обучения, существуют варианты, в которых необходимо развертывание моделей, не поддерживаемых PMML. Модели нейронных сетей (neural net), которые можно использовать для решения различных бизнес-задач, например прогнозирование продаж, проверку данных или обработку естественного языка, не могут быть представлены в формате PMML, следовательно, требуется другая методика их развертывания в режиме почти реального времени.

Создаваемая по образу и подобию человеческого мозга нейронная сеть состоит из тысяч или даже миллионов искусственных нейронов (узлов), связанных между собой. Большинство современных нейросетей организовано в виде нескольких (и даже многочисленных) слоев (уровней) узлов, как показано на рис. 11.6. Каждый узел принимает взвешенные входные данные, которые передаются в функции вычислительных узлов, а результаты вычислений выво-

дятся в следующий слой нейросети. Данные продвигаются по нейросети до тех пор, пока не будет получено единственное значение.



**Рис. 11.6.** Нейронная сеть состоит из слоя ввода, слоя вывода, а также из любого количества скрытых вычислительных слоев. Термин «глубокое обучение» означает нейросеть с глубиной в несколько вычислительных слоев

Основная концепция разделения на слои заключается в том, что каждый дополнительный слой нейросети увеличивает точность самой модели. В действительности термин «глубокое обучение» (deep learning) означает использование нейросетей с глубиной в несколько слоев. В этом разделе рассматривается процесс тренировки и развертывания нейросети с использованием широко распространенного API нейросети высокого уровня под названием Keras, написанного на языке Python.

### 11.5.1. Тренировка нейронной сети

Нейросети предназначены для распознавания определенных шаблонов (паттернов) в данных, и они обучаются этому, анализируя крупные тренировочные наборы данных. Процесс тренировки нейросети подразумевает использование тренировочного набора данных для определения весовых коэффициентов (весов) модели для каждого узла нейросети. Такие тренировочные наборы часто являются помеченными (labeled), чтобы ожидаемые на выходе результаты были известны заранее, а затем веса корректируются до тех пор, пока модель не будет выдавать ожидаемые результаты. Поэтому важно помнить о том, что эти веса весьма важны для производительности и точности моделей.

Первый этап – тренировка модели с использованием библиотеки Keras на языке Python на собственном специализированном наборе данных, как показано в листинге 11.8. Показанные здесь входные данные для модели – это десять двоичных признаков, описывающих различные характеристики заказа продуктов, например средняя стоимость заказа конкретного клиента, расстояние между домашним адресом клиента и адресом доставки заказа и т. п. Выходные данные представлены одной переменной, описывающей вероятность того, что данный заказ является мошенническим.

**Листинг 11.8.** Тренировка нейросети с использованием средств языка Python

```

import keras
from keras import models, layers 1

Определение структуры модели.
model = models.Sequential() 2
model.add(layers.Dense(64, activation='relu', input_shape=(10,)))
...
model.add(layers.Dense(1, activation='sigmoid'))
model.compile(optimizer='rmsprop', loss='binary_crossentropy',
 metrics=[auc])

history = model.fit(x, y, epochs=100, batch_size=100,
 validation_split = .2, verbose=0) 3

model.save("games.h5") 4

```

1 Использование компонентов библиотеки Keras.

2 Определение структуры модели.

3 Компиляция и подгонка модели.

4 Сохранение модели в формате H5.

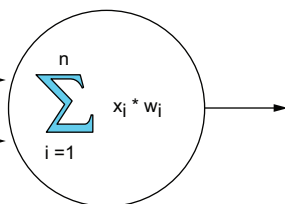
На рис. 11.7 можно видеть, что каждый узел нейросети принимает вектор признаков как входные данные вместе с набором весов, связанных с каждым признаком. Эти веса позволяют назначать большую степень важности для некоторых признаков, по сравнению с другими, при формировании прогноза, например для расстояния между домашним адресом клиента и адресом доставки заказа. Если этот признак является весьма значимой характеристикой мошеннической транзакции, то его вес должен более высоким. Процесс тренировки и подгонки модели формирует набор оптимальных весов для каждого входного узла нейросети.

**Вектор признаков**

$[x_1, x_2, x_3, \dots]$

**Вектор весов**

$[w_1, w_2, w_3, \dots]$



С каждым элементом входных данных  $x_i$  связан вес  $w_i$ , который предварительно вычисляется на этапе тренировки модели

**Рис. 11.7.** Каждый узел нейросети принимает вектор признаков как входные данные вместе с предварительно вычисленным набором весов, связанных с каждым признаком. Веса вычисляются на этапе тренировки модели, и они чрезвычайно важны для точности нейросети



После того как модель правильно натренирована (т. е. вычислены оптимальные веса для каждого входного узла) и готова для развертывания, ее можно сохранить в специализированном для Keras формате под названием H5. Этот формат сохраняет полные модели, включая их архитектуру, веса и конфигурацию тренировки, тогда как модели Keras, экспортируемые в формат JSON, содержат только архитектуру, но не вычисленные веса. Поэтому рекомендуется всегда использовать формат H5 при экспорте нейросетей на основе Keras.

### 11.5.2. Развертывание нейронной сети средствами языка Java

Для выполнения модели на основе Keras в среде времени выполнения Java мы будем использовать библиотеку Deeplearning4J. Эта библиотека предоставляет функциональность для реализации глубокого обучения на языке Java и может загружать и использовать модели, тренированные с применением Keras. Одной из основных концепций, которую необходимо освоить при использовании DL4J, являются тензоры (tensors). В Java нет встроенной библиотеки для эффективной реализации тензоров, поэтому библиотека DL4J включена в зависимости проекта Maven, как показано в листинге 11.9.

**Листинг 11.9.** Зависимости для библиотеки Deeplearning4J

```
<dependency>
 <groupId>org.deeplearning4j</groupId>
 <artifactId>deeplearning4j-core</artifactId>
 <version>1.0.0-M1</version>
</dependency>
<dependency>
 <groupId>org.deeplearning4j</groupId>
 <artifactId>deeplearning4j-modelimport</artifactId>
 <version>1.0.0-M1</version>
</dependency>
<dependency>
 <groupId>org.nd4j</groupId>
 <artifactId>nd4j-native-platform</artifactId>
 <version>1.0.0-M1</version>
</dependency>
```

Теперь мы получаем в свое распоряжение средства библиотеки DL4J и можем начать использование нейросети в режиме почти реального времени, встроив ее в функцию Pulsar, как показано в листинге 11.10.

**Листинг 11.10.** Развертывание нейронной сети в функции Pulsar, написанной на языке Java

```
import org.deeplearning4j.nn.modelimport.keras.KerasModelImport;
import org.deeplearning4j.nn.multilayer.MultiLayerNetwork;
```

```

import org.nd4j.linalg.api.ndarray.INDArray;
import org.nd4j.linalg.factory.Nd4j;

public class FraudDetectionFunction implements
 Function<FraudFeatures, Double> {
 public static final String MODEL_KEY = "key";
 private MultiLayerNetwork model;

 public Double process(FraudFeatures input, Context ctx)
 throws Exception {
 if (model == null) {
 InputStream modelHdf5Stream = new ByteArrayInputStream(
 ctx.getState(MODEL_KEY).array());
 model = KerasModelImport.importKerasSequentialModelAndWeights(
 modelHdf5Stream);

 INDArray features = Nd4j.zeros(10);
 features.putScalar(0, input.getAverageSpend());
 features.putScalar(1, input.getDistanceFromMainAddress());
 features.putScalar(2, input.getCreditCardScore());
 . . .

 return model.output(features).getDouble(0);
 }
 }
}

```

- ❶ Использование средств библиотеки DL4J.
- ❷ Входные данные – набор признаков, используемых для выявления мошенничества.
- ❸ Модель машинного обучения.
- ❹ Извлечение тренированной модели из хранилища состояния.
- ❺ Инициализация модели с использованием файла в формате HDF5.
- ❻ Создание пустого вектора признаков с 10 позициями.
- ❼ Заполнение вектора признаков реальными значениями.
- ❽ Выполнение модели с использованием подготовленного вектора признаков и возврат прогнозируемой вероятности мошенничества.

Объект модели предоставляет методы `predict` и `output`. Метод `predict` возвращает класс прогноза (мошенничество или не мошенничество), а метод `output` – числовое значение, представляющее точное значение вероятности. В данном случае возвращается числовое значение, чтобы можно было использовать его в дальнейших вычислениях (например, для сравнения с конфигурируемым пороговым значением для определения границы, за которой начинается предполагаемое мошенничество).

## 11.6. Резюме

- Pulsar Functions можно использовать для поддержки машинного обучения в режиме почти реального времени на потоковых данных для получения полезной информации.
- Для формирования прогнозов в режиме почти реального времени требуется модель машинного обучения, принимающая предварительно определенный комплект входных данных, известный как набор признаков.
- Признак – это числовое представление отдельного аспекта (характеристики) объекта, например среднее время приготовления пищи в ресторане.
- Большинство признаков в векторе невозможно вычислить, используя данные из единственного сообщения, а кроме того, такое вычисление невозможно выполнить своевременно. Поэтому часто необходим вспомогательный процесс вычислений требуемых значений в фоновом режиме с сохранением их в хранилище с малым временем задержки.
- Язык разметки моделей прогнозов (predictive model markup language – PMML) – стандартный формат для представления моделей машинного обучения, разработанных на разнообразных языках программирования, который делает эти модели переносимыми.
- Существует проект с открытым исходным кодом на языке Java, поддерживающий выполнение моделей в формате PMML и позволяющий с легкостью выполнить любую модель, поддерживаемую PMML, в функции Pulsar.
- Можно использовать другие библиотеки, поддерживающие различные языки программирования, для выполнения моделей, не поддерживаемых PMML, в функциях Pulsar.

# 12

## *Периферийная аналитика*

### **Темы:**

- использование Pulsar для периферийных вычислений;
- использование Pulsar для выполнения периферийной аналитики;
- выявление аномалий на периферии с использованием Pulsar Functions;
- выполнение статистического анализа на периферии с использованием Pulsar Functions.

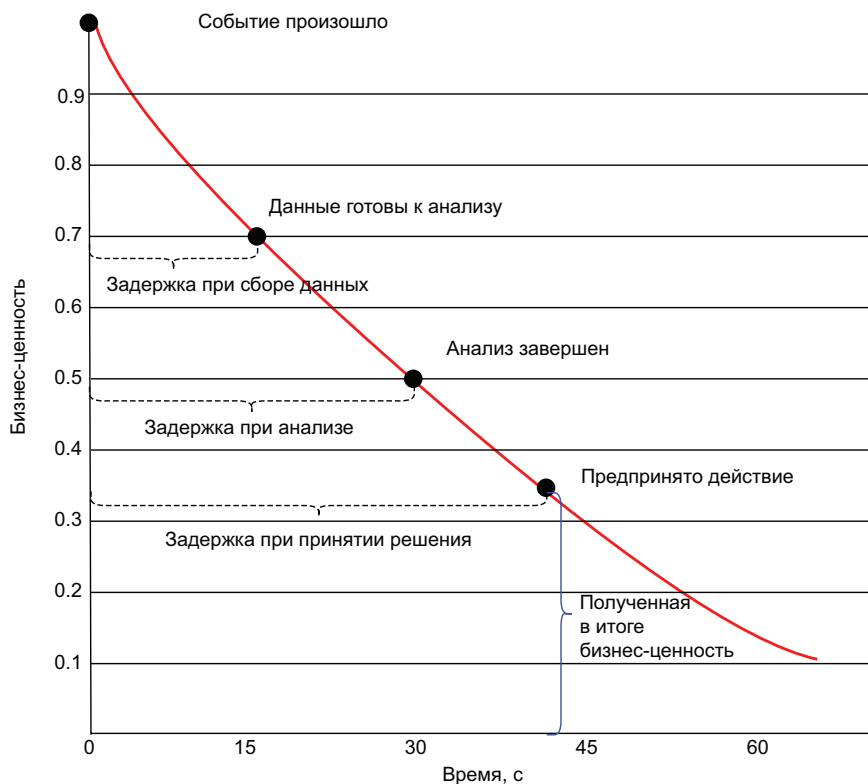
Как и большинство людей, вы, вероятно, встречали термин «интернет вещей» (Internet of Things – IoT) и сразу же представляли себе интеллектуальные терморегуляторы, холодильники с подключением к интернету или персональные помощники, такие как Alexa, Алиса и т. п. Хотя эти ориентированные на потребителя устройства привлекают всеобщее внимание, они являются всего лишь подмножеством IoT, называемым промышленным интернетом вещей (industrial internet of things – IIoT), главная цель которого – использование сенсорных датчиков, соединенных с оборудованием и средствами передвижения в индустриальной, энерге-

тической и транспортной отраслях промышленности. Компании используют информацию, поступающую от датчиков, физически встроенных в промышленное оборудование, для мониторинга, автоматизации и прогнозирования всех типов производственных процессов и выпуска продукции.

Данные, собранные от таких IoT-датчиков, на практике используются несколькими способами, включая мониторинг десятков тысяч километров удаленного промышленного оборудования в энергетике, гарантируя отсутствие вероятных критических сбоев, которые могут привести к катастрофическим последствиям в весьма крупном масштабе. Данные датчиков также можно собирать с нестационарных датчиков промышленного интернета вещей, например в большом парке рефрижераторных прицепов, используемых для распространения вакцины, температуру которой необходимо поддерживать ниже определенной, чтобы вакцина оставалась эффективной. Эти датчики позволяют нам обнаруживать постепенное потепление внутри любой холодильной установки и перенаправлять груз в ближайший сервисный центр для восстановления.

В такой ситуации важно как можно скорее обнаружить изменение температуры внутри холодильных установок, чтобы вовремя отреагировать и сохранить термочувствительный груз. Если бы мы ждали, пока груз прибудет в пункт назначения, прежде чем проверять температуру, было бы слишком поздно, и вакцина стала бы бесполезной. Это явление часто называют уменьшением временной ценности данных (*diminishing time value of data*), поскольку ценность полученной информации достигает наивысшей точки сразу после того, как событие произошло, и со временем она быстро уменьшается. В случае неисправности холодильной установки чем раньше мы сможем отреагировать на эту информацию, тем лучше. Если мы не будем знать о сбое в течение нескольких часов, груз, скорее всего, испортится, и информация лишается практической ценности, потому что будет слишком поздно что-либо предпринимать. Как можно видеть на рис. 12.1, чем дольше время реакции на подобное катастрофическое событие, тем меньшее влияние окажут любые корректирующие действия на систему.

Интервал времени между возникновением события и выполнением соответствующего ответного действия обозначается термином «задержка при принятии решения» (*decision latency*) и состоит из двух компонентов: задержки при сборе данных (*capture latency*), т. е. времени, требуемого для передачи данных в программное обеспечение, выполняющее анализ, и задержки при анализе (*analysis latency*), т. е. времени, затраченного на анализ данных, чтобы определить, какое ответное действие нужно выполнить.



**Рис. 12.1.** Ценность любого фрагмента информации быстро уменьшается во времени, поэтому целью периферийных вычислений является сокращение общей задержки принятия решения посредством устранения задержки при передаче данных от датчика в облако для анализа

С технической точки зрения ПоТ предоставляет те же основные функциональные возможности, что и любое «интеллектуальное» потребительское IoT-устройство, т. е. автоматизированные инструменты и возможности передачи отчетов с физических устройств, которые ранее не обладали такими свойствами. Например, определяющей характеристикой «интеллектуального» терморегулятора является то, что он может сообщать свои текущие показания и настраивать их удаленно через приложение для смартфона. При этом масштаб типичного развертывания ПоТ значительно больше, чем простая система, которая позволяет вам регулировать термостат со смартфона.

Учитывая, что с большой вероятностью миллионы датчиков распределены по одному заводу или большому парку трейлеров и все они каждую секунду выдают новый показатель, можно с легкостью увидеть, что эти наборы данных ПоТ имеют не только большой объем, но и высокую частоту передачи. Общий подход к обработке

таких наборов данных заключается в сборе всех отдельных элементов данных, передаче их в облако и использовании обычных инструментов анализа данных на основе SQL, например Apache Hive, или более привычных крупных хранилищ данных. Такой подход гарантирует, что анализ выполняется на полном наборе данных со всех датчиков, поэтому любые взаимосвязи показаний между датчиками можно наблюдать и использовать для анализа (например, корреляцию между датчиком температуры и общей влажностью растения, полученной от другого датчика, можно отслеживать и анализировать).

Но такая методика имеет некоторые серьезные недостатки, например значительная задержка при принятии решения (время между возникновением события и завершением обработки данных о нем), неэффективность затрат, связанная с необходимостью предоставления достаточной пропускной способности сети и вычислительных ресурсов для обработки таких больших наборов данных, а также стоимость хранения всей этой информации.

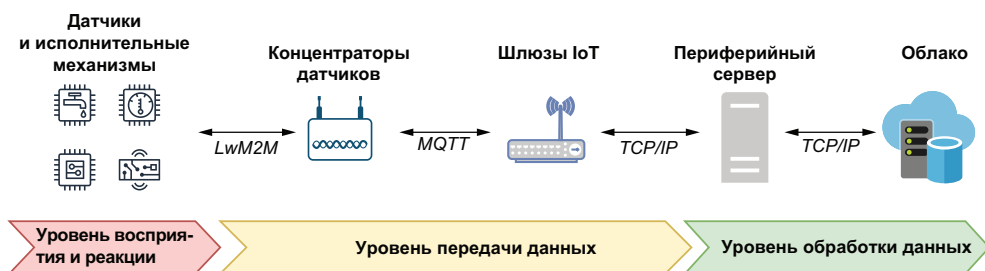
С практической точки зрения количество времени, необходимое для передачи данных с большинства платформ IoT в среду облачных вычислений для анализа, делает практически невозможным реагирование в реальном времени на предположительно катастрофическое событие. Хотя некоторые из наиболее ярких примеров подобных событий включают обнаружение неисправностей в энергетических силовых установках или самолетах до того, как они взорвутся или разобьются, скорость анализа данных в большинстве приложений IoT также имеет решающее значение.

Чтобы преодолеть это ограничение, часть процедур обработки и анализа данных IoT может выполняться в инфраструктуре, которая физически расположена ближе к самому источнику данных. Перенос процедур обработки ближе к источнику данных уменьшает задержку при сборе данных и позволяет приложениям реагировать на данные по мере их получения почти мгновенно вместо ожидания передачи информации через интернет, чтобы начать ее обработку. Эту практическую методику обработки данных на периферии сети, где генерируются данные, а не в централизованной точке сбора данных, такой как центр обработки данных или облако, часто называют периферийными вычислениями (edge computing).

В этой главе демонстрируется, как можно развернуть Pulsar Functions в среде периферийных вычислений для обеспечения обработки и анализа данных почти в реальном времени, чтобы быстрее реагировать на события в среде промышленного интернета вещей (IIoT) и минимизировать задержку при принятии решения между временем обнаружения весьма важного события и выполнением соответствующего ответного действия.

## 12.1. Архитектура промышленного интернета вещей

Платформа промышленного интернета вещей (IIoT) работает как мост между физическим миром промышленного оборудования и цифровым миром автоматизированных систем управления, используемых для мониторинга и реакции на события в физическом мире. Поэтому одной из главных задач любой IIoT-системы является сбор и обработка данных, генерируемых всеми физическими датчиками, распределенными по разнообразным элементам промышленного оборудования. Хотя IIoT-среды отличаются друг от друга по структуре, все они состоят из трех логических архитектурных уровней, показанных на рис. 12.2. Эти уровни играют весьма важную роль в процессе получения и анализа данных.



**Рис. 12.2.** Три логических уровня архитектуры IIoT собирают данные, анализируют их, а затем предоставляют информацию, которую могут использовать люди или независимые системы для принятия соответствующих текущему контексту решений в реальном времени

В IIoT-среде могут существовать миллионы датчиков, контроллеров и исполнительных механизмов, распределенных по различным элементам промышленного оборудования в пределах территории одного предприятия. Все эти датчики и устройства в совокупности формируют то, что обозначается общим термином «уровень восприятия и реакции» (perception and reaction layer), поскольку позволяет воспринимать происходящее в физическом мире.

### 12.1.1. Уровень восприятия и реакции

Уровень восприятия и реакции содержит все компоненты аппаратного оборудования (т. е. элементы интернета вещей). В качестве основы любой IIoT-системы эти соединенные между собой устройства отвечают за восприятие физического состояния промышленного оборудования и окружающей среды, ощущаемое через многочисленные сенсорные датчики, либо встроенные в элементы оборудования, либо размещенные как отдельные независимые объекты. Эти датчики передают в реальном времени непрерывный поток своих показаний.



ний по упрощенным протоколам передачи данных (LwM2M), таким как Bluetooth, Zigbee и Z-Wave, или по протоколам дальнего действия, например MQTT или LoRa, если имеется кабельное соединение.

На практике в большинстве ПоТ-сред требуется несколько сетевых протоколов для поддержки работы разнообразных устройств. Например, датчики на батарейках могут передавать данные только на короткие расстояния, применяя упрощенные протоколы, предназначенные для использования в пределах малого радиуса действия. В общем случае чем больше расстояние, на которое нужно передать сигнал, тем больше энергии потребуется устройству для его передачи. Применение устройств на батарейках, использующих для передачи данных протоколы дальнего действия, нецелесообразно.

Одной из основных обязанностей уровня восприятия и реакции является наблюдение за окружающей средой через датчики с захватом их показаний и передачей на уровень обработки данных. Не менее важной обязанностью этого уровня является реакция на полезную информацию, созданную на уровне обработки, и ее преобразование в немедленное физическое действие при обнаружении потенциально опасного состояния. Способность обнаруживать потенциально опасное состояние почти не увеличивает бизнес-ценность (практическую пользу), если невозможно отреагировать на опасность надлежащим образом.

За несколько десятилетий до появления ПоТ многие крупнейшие производственные предприятия затратили весьма много времени и усилий на создание специализированных программных систем, известных как SCADA (supervisory control and data acquisition – система диспетчерского управления и сбора данных), позволяющих выполнять наблюдение и управление промышленным оборудованием. Такие системы содержали управляющие сети, поддерживающие автоматизированные операции механических и электромеханических устройств, называемых исполнительными механизмами (actuators) и способных выполнять различные рутинные операции, например открытие клапана давления в элементе оборудования или полное отключение электроэнергии для определенного устройства.

Применяя существующую управляющую сеть в подобных системах SCADA, мы получаем возможность программного управления исполнительными механизмами из ПоТ-приложения, просто отправляя правильную команду для активизации конкретного механизма. Если датчики работают как «глаза» на этом уровне, то исполнительные механизмы играют не менее важную роль «рук», позволяющих реагировать на исходные данные. Без таких «рук» не существовало бы возможности как-либо отвечать на катастрофические показания датчиков. Информация, собранная на уровне восприятия и реакции, отправляется через уровень передачи данных, который, как можно понять из его названия, отвечает

за безопасную передачу данных, полученных от датчиков, на уровень централизованной обработки данных.

### 12.1.2. Уровень передачи данных

Показания датчика, созданные на уровне восприятия и реакции и передаваемые по упрощенным протоколам LwM2M, обнаруживаются промежуточными устройствами, известными как концентраторы датчиков (sensor hubs), расположенными на внешней границе уровня передачи данных. Эти специализированные устройства могут принимать показания датчиков, передаваемые по протоколам LwM2M устройствами слабой мощности.

Основная задача концентраторов датчиков заключается в обеспечении своеобразного моста между технологиями передачи данных ближнего и дальнего действия. IoT-устройства на батарейках обмениваются данными с концентраторами датчиков, используя режимы беспроводной передачи ближнего действия, например Bluetooth LE, Zigbee и Z-Wave. При получении таких сообщений концентраторы датчиков немедленно перенаправляют их по протоколам дальнего действия, таким как CoAP, MQTT или LoRa, в устройство под названием IoT-шлюз (IoT gateway). Среди протоколов дальнего действия следует особо выделить MQTT (message queuing telemetry transport – передача телеметрических данных в очереди сообщений), который специализирован для работы в средах с низкой пропускной способностью и с длительными задержками. Эта специализация делает MQTT одним из наиболее часто используемых протоколов в среде ПОТ.

IoT-шлюз – это физическое устройство, работающее как точка соединения между облаком и концентраторами датчиков. Такие устройства предоставляют канал обмена данными между протоколом MQTT и протоколом передачи сообщений Pulsar, а также отвечают за объединение всех входящих показаний датчиков, передаваемых по упрощенному двоичному протоколу MQTT, перед перенаправлением их на уровень обработки данных с применением протокола обмена сообщениями Pulsar.

### 12.1.3. Уровень обработки данных

На рис. 12.2 можно видеть, что уровень обработки данных (processing layer) охватывает два физических уровня: периферийные серверы, расположенные близко к промышленному оборудованию, и корпоративный центр данных или облачную инфраструктуру. Периферийные серверы используются для объединения, фильтрации и назначения приоритетов данным из огромного объема, обычно генерируемого крупномасштабной средой ПОТ. Эти промежуточные операции предназначены для минимизации объема информации, которую необходимо перенаправить в облако. Такая

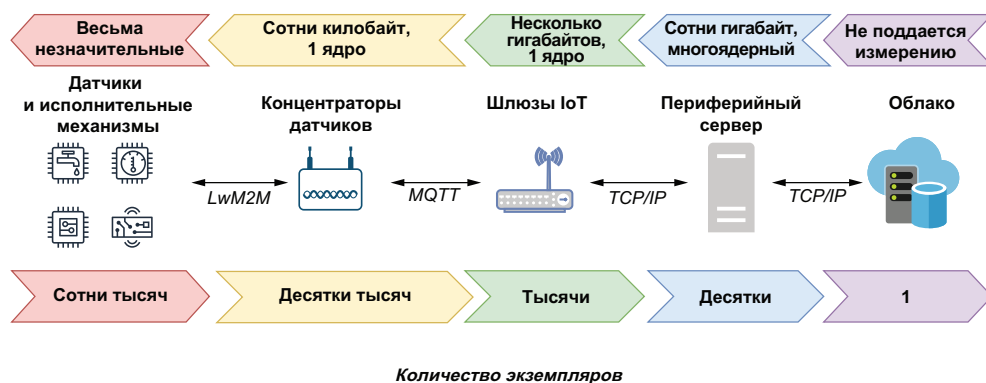
предварительная обработка данных помогает снизить затраты на передачу и сократить время ответной реакции. С точки зрения аппаратного обеспечения уровень периферийной обработки состоит из одного или нескольких обычных компьютеров или серверов, размещенных непосредственно на промышленной территории (т. е. на территории завода). Вычислительная мощность на этом уровне может быть ограничена из-за стесненности физического пространства в конкретной локации, но для этих устройств всегда существует интернет-соединение, позволяющее перенаправлять собранные данные в центр данных компании и/или провайдеру облачной среды для дальнейшего анализа и архивного хранения.

## 12.2. Уровень обработки данных на основе *Pulsar*

После ознакомления с основами архитектуры промышленного интернета вещей (IIoT) перейдем к рассмотрению возможности использования Apache Pulsar для улучшения функций обработки в этой архитектуре посредством расширения уровня обработки данных с приближением его к датчикам и исполнительным механизмам в отличие от обычной схемы IIoT. Сначала рассмотрим вычислительные ресурсы, доступные в каждом физическом устройстве на платформе IIoT. На рис. 12.3 показана обратная зависимость между доступными вычислительными ресурсами и их близостью к промышленному оборудованию. Датчики, исполнительные механизмы и прочие интеллектуальные устройства, размещенные на уровне восприятия и реакции, – это микроконтроллеры с ограниченной памятью и мощностью обработки. В основном они работают на батарейках, следовательно, не рекомендуется выполнять на них какие-либо виды вычислений. Хотя такие беспроводные устройства можно усовершенствовать, снабдив их зарядными блоками, преобразующими энергию окружающей среды в электроэнергию, все же лучше сэкономить их мощность для беспроводного обмена данными с концентраторами датчиков, а не для вычислений.

Концентраторы датчиков обычно располагаются на нескольких более крупных устройствах, именуемых системами на микросхеме (system on a chip – SoC), содержащих некоторые или даже все компоненты обычного компьютера, включая процессор, RAM и внешнее устройство хранения данных, хотя эти компоненты менее мощные. С учетом многочисленности таких устройств их спецификации остаются минимальными, обеспечивая экономическую эффективность их развертывания в больших количествах. Поскольку концентраторы в основном используются для приема и передачи данных, они требуют весьма небольшого ограниченного объема памяти для буферизации сообщений перед перенаправлением в соответствии с требованиями другого протокола.

## Вычислительные ресурсы каждого устройства



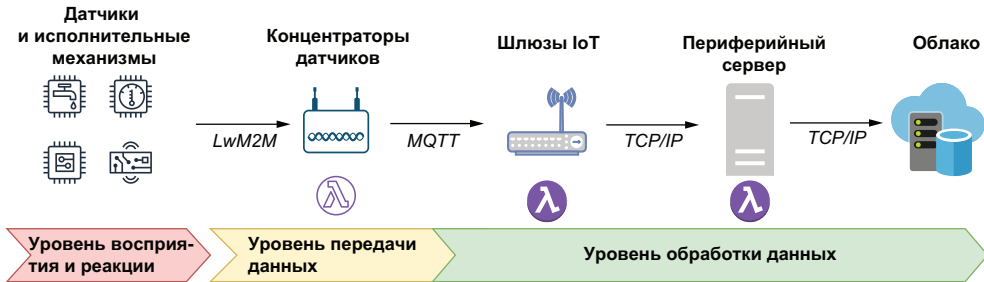
**Рис. 12.3.** Между количеством устройств, развернутых на каждом уровне, и вычислительными ресурсами каждого такого устройства существует обратная зависимость

IoT-шлюзы также размещаются на аппаратных устройствах SoC, а самой широко распространенной платформой для таких устройств является Raspberry Pi, содержащая до 8 Гб оперативной памяти (RAM), четырехъядерный процессор и один слот для карты MicroSD, обеспечивающей хранение до 1 Тб данных. Эти устройства управляются обычными операционными системами, например Linux, поэтому являются идеальными кандидатами для поддержки сложных программных приложений, таких как Pulsar.

Последний физический элемент аппаратного обеспечения – периферийный сервер, являющийся необязательным компонентом в архитектуре IIoT. В зависимости от особенностей промышленного оборудования характеристики этих устройств определяются в диапазоне от нескольких серверов с терабайтной RAM, несколькими ядрами и терабайтами на дисковом устройстве, размещаемых в специально выделенном помещении заводского цеха, до единственного настольного компьютера в отдельной удаленной локации. Как и IoT-шлюзы, устройства на этом уровне управляются обычными операционными системами и напоминают то, что большинство людей воспринимают как «компьютер».

Любое устройство, обладающее достаточными вычислительными ресурсами (8 Гб оперативной памяти и многоядерным процессором x86), работающее под управлением обычной операционной системы и имеющее соединение с интернетом, является потенциальной аппаратной платформой для поддержки работы брокера Pulsar. Внутри архитектуры IIoT это не только периферийные серверы и IoT-шлюзы, но также любой концентратор датчиков, работающий на достаточно хорошо оборудованном SoC-устройстве.

Установка брокера Pulsar на таких устройствах позволяет развертывать непосредственно на них функции Pulsar для выполнения анализа как можно ближе к источникам данных. Такой подход перемещает уровень обработки данных гораздо ближе к источнику показаний датчика, как показано на рис. 12.4. Мы продвигаем границу ближе к промышленному оборудованию, т. е. создаем крупный распределенный вычислительный фреймворк, в котором можно развернуть функции Pulsar в двух уровнях архитектуры для выполнения параллельной обработки данных.



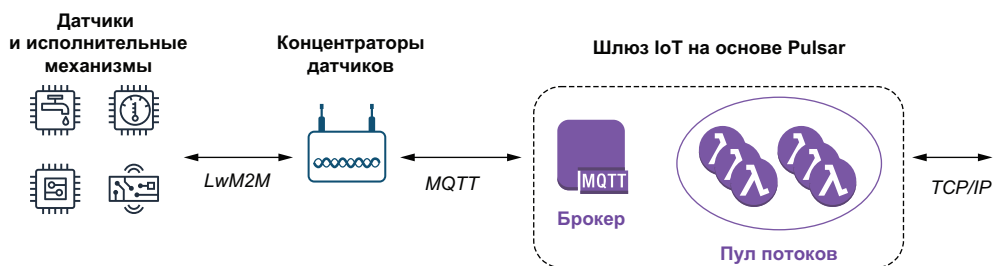
**Рис. 12.4.** При установке брокеров Pulsar на IoT-шлюзах и периферийных серверах можно расширить уровень обработки данных, переместив его ближе к источнику данных, что позволяет реагировать гораздо быстрее

Из этого следует, что можно развертывать функции Pulsar на любом устройстве в среде PoT, способном поддерживать брокер Pulsar, и выполнять сложные задачи анализа на периферии вместо простого сбора событий и перенаправления их для обработки, как это делалось изначально в PoT-среде.

Некоторые провайдеры PoT предоставляют программные пакеты, которые можно использовать для обработки данных на устройствах IoT-шлюзов, но их возможности обычно ограничены самыми простыми функциями, такими как фильтрация и объединение. Кроме того, исходный код этих пакетов закрыт, поэтому не позволяет расширять фреймворк, добавляя собственные функции обработки данных. Применяя Pulsar Functions, вы можете с легкостью создавать и использовать собственные функции для выполнения более сложных задач анализа.

Также следует отметить, что Apache Pulsar обеспечивает поддержку протокола MQTT через подключаемый модуль, что позволяет применять протокол MQTT таким устройствам, как концентраторы датчиков, для публикации сообщений непосредственно в тему в брокере Pulsar. Поэтому Pulsar можно использовать как шлюз IoT без какого-либо другого программного компонента, действующего как брокер сообщений MQTT, и потреблять и обрабатывать показания датчиков прямо в шлюзе, как показано на рис. 12.5. Шлюз IoT на основе Pulsar поддерживает двусторонний

обмен данными по протоколу MQTT, что позволяет функциям Pulsar отправлять сообщения исполнительным механизмам в ответ на обнаруженные предположительно катастрофические события.



**Рис. 12.5.** Полноценную функциональность шлюза IoT можно обеспечить, используя брокер Pulsar с подключенным модулем поддержки протокола MQTT, позволяющим принимать сообщения от концентраторов датчиков, в то время как функции Pulsar можно администрировать через соединение по протоколу TCP/IP

Соединение по протоколу TCP/IP позволяет пользователям не только развертывать, обновлять и администрировать функции Pulsar непосредственно на устройстве шлюза IoT, но также обеспечивает обмен данными брокера Pulsar с уровнем хранения данных на основе Apache BookKeeper, размещенным в удаленной локации. Наконец, не менее важно, что соединение по протоколу TCP/IP позволяет обмениваться данными с другими кластерами Pulsar, включенными в архитектуру ПоТ, в частности с кластерами, развернутыми на периферийных серверах. Это обеспечивает перенаправление данных, сгенерированных на всех шлюзах IoT, в централизованную локацию для дополнительной периферийной обработки перед отправкой в облако для завершающего архивирования.

### 12.3. Периферийная аналитика

Практическую методику выполнения некоторых или всех операций анализа данных в инфраструктуре, расположенной за пределами обычного центра данных или облачной вычислительной среды, часто обозначают термином «периферийная аналитика» (edge analytics) и отличают от обычной аналитики по нескольким основным критериям, о которых следует всегда помнить при проектировании общей стратегии анализа данных. Для начинающих поясню: анализ должен выполняться на потоковых наборах данных, где каждый фрагмент информации предоставляется функции Pulsar только один раз. Поскольку в периферийной среде физическое дисковое пространство весьма ограничено, показания датчиков не сохраняются, следовательно, повторно прочитать их через некоторое время невозможно. Если необходимо определить среднее арифметическое показаний некоторого датчика за прошедший



час, например, в облачной среде, то можно просто выполнить SQL-запрос для вычисления этого значения по ранее сохраненным данным. Но такой вариант невозможен в периферийной аналитике. Еще одно важное различие: чем ближе процесс обработки перемещается к датчикам, тем меньше информации содержится в получаемом наборе данных, из-за чего становится невозможным обнаружение одинаковых закономерностей в показаниях датчиков, если они не размещены в области действия одного шлюза IoT.

### 12.3.1. Телеметрические данные

Чтобы лучше понять смысл термина «периферийная аналитика», а также что именно мы пытаемся сделать, выполняя некоторые операции анализа данных на периферии, лучше всего начать с простого изучения типов данных, обрабатываемых в среде IoT. Самой главной функцией любой системы IoT является сбор данных датчиков для мониторинга и управления промышленной инфраструктурой компании. После сбора данных их можно проанализировать на предмет предполагаемого возникновения событий, на которые потребуются особая ответная реакция.

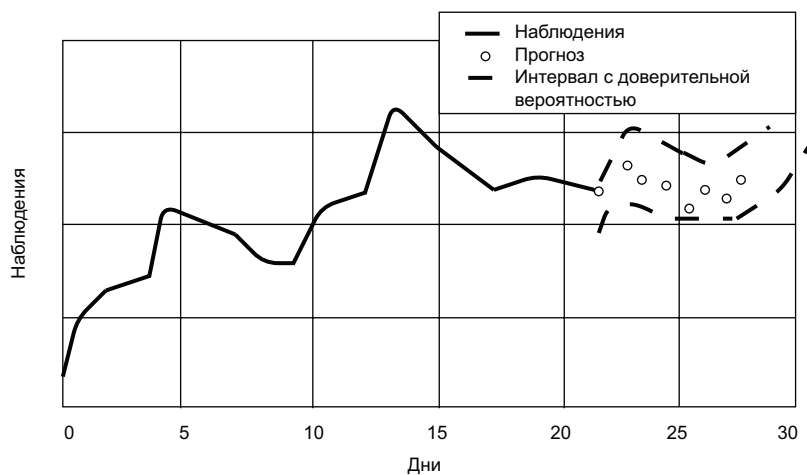
Датчики постоянно передают поток показаний, получаемых при непрерывно повторяющихся во времени измерениях одной и той же переменной характеристики, например датчик, передающий значения температуры конкретного элемента оборудования каждую секунду. Такие последовательности числовых точек данных, передаваемые с фиксированными интервалами в хронологическом порядке, обозначаются термином «временные ряды данных» (time-series data). Весь процесс сбора такой информации в форме регулярно повторяющихся измерений или статистических данных и перенаправление их в удаленные системы часто обозначают термином «телеметрия» (telemetry). Почти как и все временные наборы данных, телеметрические данные чаще всего обладают одной или несколькими характеристиками, среди которых:

- тренд (trend) – во временных рядах данных термином «тренд» обозначается тот факт, что точки данных имеют четко выраженную траекторию в одном направлении (вверх или вниз) на протяжении конкретного отрезка времени. Пример: тренд долговременного роста сетевого трафика;
- циклы (cycles) – повторяющиеся и предсказуемые флуктуации в данных, возникающие с непостоянной частотой, которые из-за этого невозможно связать с конкретным интервалом времени;
- сезонные колебания (seasonality) – если существуют регулярные предсказуемые циклы в данных, которые можно связать с календарем (например, ежедневные, еженедельные и т. д.), то данные обладают характеристикой сезонности. Она отли-

чается от тренда, потому что циклы возникают в коротком интервале времени и обычно управляются внешними факторами, например пик сетевого трафика на широко известном сайте электронной торговли приходится на день Cyber Monday;

- шум (noise) – так обозначается случайность некоторых точек данных, которые невозможно связать с любыми явными трендами. Шум возникает несистематически и на короткое время, и его необходимо отфильтровать, чтобы минимизировать негативное воздействие на прогнозы. Рассмотрим датчик температуры, постоянно передающий значение 200 °F в течение прошедшего часа. Любое показание датчика, существенно отклоняющееся от предыдущих принятых значений, вероятнее всего, является просто шумом, поэтому должно игнорироваться. Например, единственное показание 25 °F с большой вероятностью является неточным измерением, которое следует игнорировать.

В контексте IoT периферийная аналитика используется для определения перечисленных выше характеристик данных, чтобы в дальнейшем их можно было использовать для прогнозирования будущих значений на основе предшествующих наблюдений, как показано на рис. 12.6. Затем реальные наблюдаемые значения можно сравнить с прогнозируемыми, чтобы определить, требуется ли выполнить некоторые конкретные действия. Например, если в показаниях давления наблюдается тренд снижения, то, возможно, это признак потери давления в трубопроводе, и бригада ремонтников должна его проверить.

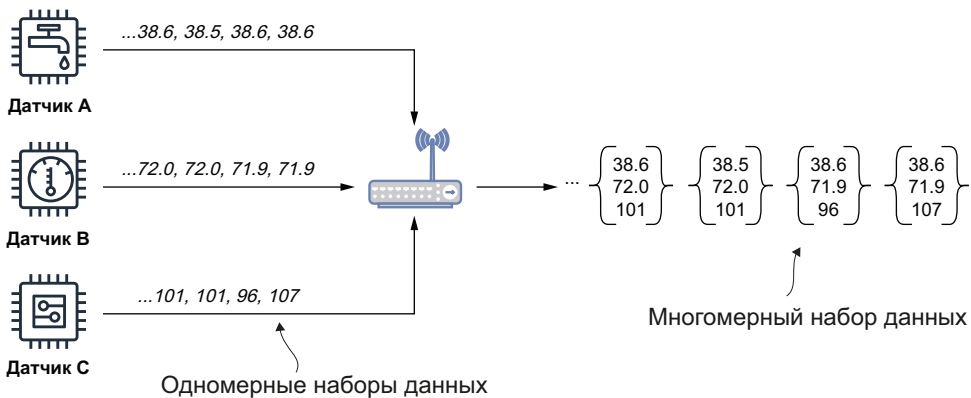


**Рис. 12.6.** Прогнозирование временных рядов – это использование временных рядов данных для прогнозирования будущих значений на основе предшествующих наблюдений



### 12.3.2. Одномерные и многомерные наборы данных

Другой характеристикой наборов телеметрических данных является количество переменных, отслеживаемых в процессе их обработки. Наиболее частый сценарий: данные содержат набор наблюдаемых значений одной переменной (например, показания одного и того же датчика). Для этого типа данных существует более формальный термин – «одномерный (univariate) набор», и для наших учебных примеров мы будем считать, что все наборы данных, собираемые на уровне шлюза IoT, являются одномерными. Вероятно, вы уже поняли, что любой набор данных, полученный при отслеживании более одной переменной, называется многомерным (multivariate) и позволяет наблюдать за взаимосвязью между показаниями нескольких датчиков в течение одного и того же интервала времени. Такие наборы данных в каждый конкретный момент времени содержат не одно, а несколько значений. В архитектуре ПоТ на основе Pulsar многомерные наборы данных генерируются с использованием функций Pulsar посредством объединения нескольких одномерных наборов данных, как показано на рис. 12.7, где показания каждого датчика, получаемые одновременно, объединяются в кортеж из трех значений.



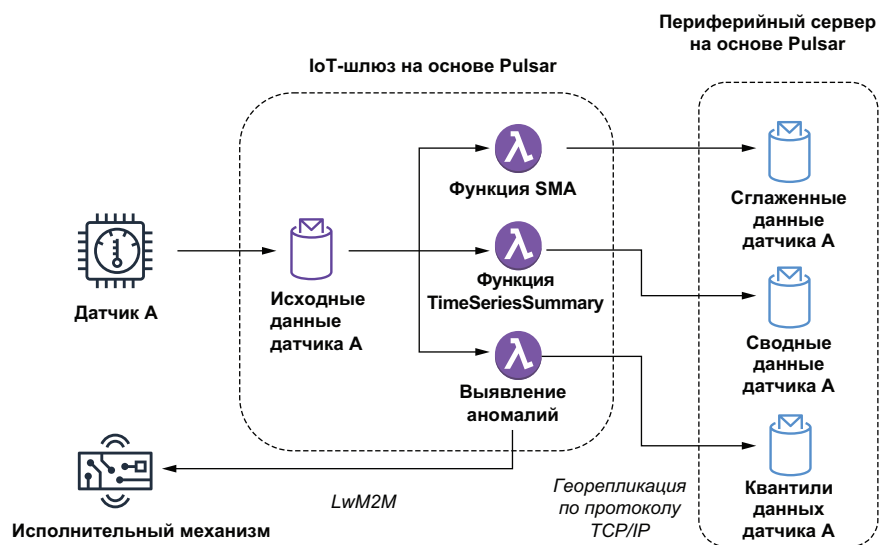
**Рис. 12.7.** Каждый датчик передает последовательность показаний с предварительно определенным интервалом. После приема в шлюзе IoT эти одномерные наборы данных объединяются в один многомерный набор, содержащий значения всех трех датчиков в каждый конкретный момент времени. Это позволяет обнаруживать закономерности и взаимосвязи между показаниями трех датчиков

Многомерные наборы данных можно использовать для выполнения более сложного и точного анализа, объединяя данные из нескольких источников, являющихся тесно связанными. Поскольку нет никаких ограничений на количество переменных, объединяемых в наборы данных, можно группировать столько показателей, сколько необходимо. В действительности многомерные наборы данных

вполне подходят в качестве хранилищ признаков для любой модели машинного обучения, которую требуется развернуть на периферии. Как было отмечено в главе 11, хранилища признаков содержат набор предварительно вычисленных значений, необходимых для моделей машинного обучения. Такие наборы признаков заполняются внешними процессами, использующими хронологию данных для вычисления требуемых значений. В среде IoT на основе Pulsar многомерные наборы данных заполняются из одномерных наборов, собранных на периферии, обеспечивая снабжение моделей машинного обучения самыми свежими данными для формирования прогнозов.

## 12.4. Одномерный анализ

Бесконечный поток показаний датчика – это основа периферийной аналитики. Такие одномерные наборы данных представляют исходные необработанные данные для выполнения анализа IoT-характеристик как единого целого. Поэтому лучше всего начать с рассмотрения типа анализа, который может быть выполнен с такими одномерными наборами. Общая схема различных типов аналитической обработки, которые выполняются чаще всего, представлена на рис. 12.8.



**Рис. 12.8.** Датчик публикует показания в теме `SensorARowData`, используемой как входные данные для трех функций Pulsar. Две функции SMA и `TimeSeriesSummary` используют эти значения для вычисления статистических характеристик по исходным данным перед публикацией в локальных темах, сконфигурированных для георепликации в кластере Pulsar, работающем на периферийном сервере. Третья функция определяет, является ли показание датчика аномальным, и должна активизировать исполнительный механизм для ответного действия при любом обнаруженном предположительно катастрофическом событии

Обработка выполняется с использованием функций Pulsar, развернутых на IoT-шлюзе, ближайшем к источнику данных. Поэтому весьма важно, чтобы эти функции минимизировали объем памяти и ресурсы процессора, требуемые для выполнения анализа.

### 12.4.1. Снижение шума

Выше было отмечено, что возможно возникновение шума (т. е. случайных точек) в показаниях датчика. Для улучшения прогнозируемых значений важным этапом предварительной обработки является снижение шума в одномерных наборах данных. Широко известной методикой сглаживания подобных отклонений в данных является простое вычисление математического среднего значения по точкам данных в предварительно определенном временном окне для получения требуемого значения. Затем это вычисленное среднее смещенное значение сохраняется вместо исходных значений, по которым выполнялось вычисление. Это позволяет минимизировать воздействие любого отдельного показания датчика на передаваемое значение (например, если 99 показаний датчика содержат одно и то же значение 70, а единственное показание выдает значение 100, то использование вычисленного среднего значения 70.3 эффективно сглаживает все данные и предоставляет значение, которое выглядит более достоверным показанием счетчика в этом интервале времени).

Существует множество различных моделей смещения среднего значения, но для краткости в этом разделе будет рассматриваться только методика простого скользящего среднего (simple moving average – SMA). Простое скользящее среднее значение вычисляется по сохраненному самым последним подмножеству временного ряда данных, например по последним 100 показаниям датчика. Когда поступают новые показания, вычисленное значение используется для замены самого старого значения в наборе. После добавления самого нового значения математическое среднее всех остальных значений вычисляется и возвращается.

#### Листинг 12.1. Функция Pulsar для вычисления простого скользящего среднего

```
public class SimpleMovingAverageFunction implements Function<Double, Void> {

 private CircularFifoQueue<Double> values;
 private PulsarClient client;
 private String remotePulsarUrl, topicName;
 private boolean initialized;
 private Producer<Double> producer;

 @Override
 public Void process(Double input, Context ctx) throws Exception {
```

```

 if (!initialized) {
 initialize(ctx);
 }

 values.add(input);
 double average = values.stream()
 .mapToDouble(i->i).average().getAsDouble();
 publish(average);
 return null;
 }

 private void publish(double average) {
 try {
 getProducer().send(average);
 } catch (PulsarClientException e) {
 e.printStackTrace();
 }
 }

 private PulsarClient getEdgePulsarClient() throws PulsarClientException {
 if (client == null) {
 client = PulsarClient.builder().serviceUrl(remotePulsarUrl).build();
 }
 return client;
 }

 private Producer<Double> getProducer() throws PulsarClientException {
 if (producer == null) {
 producer = getEdgePulsarClient()
 .newProducer(Schema.DOUBLE)
 .topic(topicName).create();
 }
 return producer;
 }

 private void initialize(Context ctx) {
 initialized = true;
 Integer size = (Integer) ctx.getUserConfigValueOrDefault("size", 100);
 values = new CircularFifoQueue<Double> (size);
 remotePulsarUrl = ctx.getUserConfigValue("pulsarUrl").get().toString();
 topicName = ctx.getUserConfigValue("topicName").get().toString();
 }
}

```

- ❶ Циклический буфер значений с автоматическим удалением самого старого элемента.
- ❷ Клиент кластера Pulsar, работающего на периферийных серверах.
- ❸ Добавление очередного показания датчика в список значений.
- ❹ Вычисление простого скользящего среднего.
- ❺ Публикация вычисленного значения в теме на периферийном сервере.

К счастью, реализовать вычисление смещенных средних значений с помощью функций Pulsar достаточно просто. В листинге 12.1 можно видеть, что основной особенностью реализации является использование циклического буфера для хранения последних  $n$  значений, требуемых для вычисления простого скользящего среднего. Далее можно использовать эту функцию для предварительной обработки отдельных показаний датчика и публиковать вычисленное SMA-значение вместо исходного. Это обеспечивает выполнение дальнейших этапов анализа с меньшим уровнем шума.

### 12.4.2. Статистический анализ

Смещенные средние значения – это не единственные информативные статистические характеристики, которые можно вычислить по одномерным наборам данных. В действительности достаточно просто вычисляется практически любая характеристика, часто используемая при статистическом анализе с помощью функции Pulsar. Например, рассмотрим код, показанный в листинге 12.2, который вычисляет следующие статистические характеристики в одной функции: среднее геометрическое значение, дисперсию генеральной совокупности, коэффициент эксцесса распределения, среднее квадратическое отклонение, асимметрию распределения и стандартное отклонение.

**Листинг 12.2.** Функция Pulsar для вычисления нескольких статистических характеристик

```
import org.apache.commons.math3.stat.descriptive.DescriptiveStatistics;
import org.apache.commons.math3.stat.descriptive.SummaryStatistics;
import org.apache.commons.math3.stat.regression.*;
import org.apache.pulsar.functions.api.Context;
import org.apache.pulsar.functions.api.Function;

public class TimeSeriesSummaryFunction implements
 Function<Collection<Double>, SensorSummary> {

 @Override
 public SensorSummary process(Collection<Double> input, Context context)
 throws Exception {

 double[][] data = convertToDoubleArray(input);
 SimpleRegression reg = calcSimpleRegression(data);
 SummaryStatistics stats = calcSummaryStatistics(data);
 DescriptiveStatistics dstats = calcDescriptiveStatistics(data);
 double rmse = calculateRSME(data, reg.getSlope(), reg.getIntercept());

 SensorSummary summary =
 SensorSummary.newBuilder()
 .setStats(TimeSeriesSummary.newBuilder()
 .setGeometricMean(stats.getGeometricMean())
```

```

 .setKurtosis(dstats.getKurtosis())
 .setMax(stats.getMax())
 .setMean(stats.getMean())
 .setMin(stats.getMin())
 .setPopulationVariance(stats.getPopulationVariance())
 .setRmse(rmse)
 .setSkewness(dstats.getSkewness())
 .setStandardDeviation(dstats.getStandardDeviation())
 .setVariance(dstats.getVariance())
 .build())
 .build();

 return summary;
}

private SimpleRegression calcSimpleRegression(double[][] input) {
 SimpleRegression reg = new SimpleRegression();
 reg.addData(input);
 return reg;
}

private SummaryStatistics calcSummaryStatistics(double[][] input) {
 SummaryStatistics stats = new SummaryStatistics();
 for(int i = 0; i < input.length; i++) {
 stats.addValue(input[i][1]);
 }
 return stats;
}

private DescriptiveStatistics calcDescriptiveStatistics(double[][] in)
{
 DescriptiveStatistics dstats = new DescriptiveStatistics();
 for(int i = 0; i < in.length; i++) {
 dstats.addValue(in[i][1]);
 }
 return dstats;
}

private double calculateRSME(double[][] input,
 double slope, double intercept) {
 double sumError = 0.0;
 for (int i = 0; i < input.length; i++) {
 double actual = input[i][1];
 double indep = input[i][0];
 double predicted = slope*indep + intercept;
 sumError += Math.pow((predicted - actual),2.0);
 }
 return Math.sqrt(sumError/input.length);
}

private double[][] convertToDoubleArray(Collection<Double> in)
 throws Exception {

```

```

double[][] newIn = new double[in.size()][2];
int i = 0;
for (Double d : in) {
 newIn[i][0] = i;
 newIn[i][1] = d;
 i++;
}
return newIn;
}
}

```

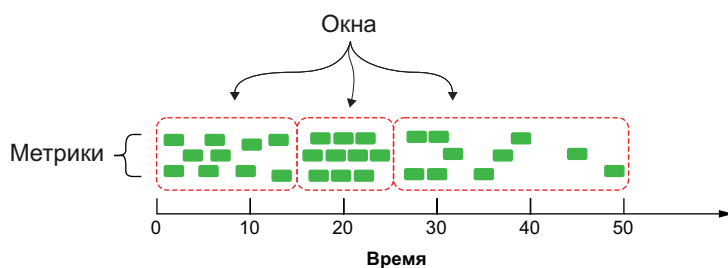
- ❶ Эта функция зависит от внешних библиотек для выполнения статистических вычислений.
- ❷ Входной набор данных содержит все показания датчика в заданном временном окне.
- ❸ Преобразование данных из исходного набора в двумерный массив.
- ❹ Использование различных вычисляемых статистических характеристик для заполнения итоговой возвращаемой структуры данных.

Очевидно, что эти статистические характеристики невозможно вычислить по единственной точке во временном ряде данных, для этого требуется обработка более крупного набора элементов данных. Попытка вычисления среднего геометрического значения по единственному числу не имеет никакого смысла. Поэтому необходимо определить стратегию разделения бесконечного потока данных измерений на конечные наборы данных, которые называют окнами (windows), чтобы использовать их для вычисления статистических характеристик. Но как определить границы окна данных? В фреймворке Pulsar Functions существуют две стратегии, используемые для управления границами окон:

- стратегия триггера (trigger policy) – управление происходит при выполнении кода функции. Это правила, которые фреймворк Apache Pulsar Functions применяет для оповещения кода, что наступило время для обработки всех данных, накопленных в окне;
- стратегия вытеснения (eviction policy) – управляет объемом данных, сохраняемых в окне. Это правила, позволяющие определить, должен ли элемент данных сохраниться в окне, или необходимо его вытеснить.

Обе стратегии управляются временем или объемом данных в окне и могут быть определены в единицах времени или длины (в количестве элементов данных). Рассмотрим подробнее различия между этими двумя стратегиями и как они работают в тесном взаимодействии друг с другом. Существует множество разнообразных методик создания окон, но наиболее часто применяются «переворачивающееся окно» (tumbling window) и «скользящее окно» (sliding window).

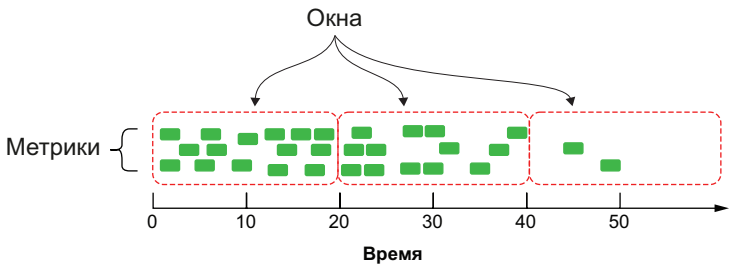
Переворачивающиеся окна – это непрерывные непересекающиеся окна, имеющие фиксированный размер, например 100 элементов, или формирующиеся через фиксированные интервалы времени, например через каждые пять минут. Стратегия вытеснения для переворачивающихся окон всегда запрещена, чтобы обеспечить заполнение окна до отказа. Поэтому требуется определить только стратегию триггера, которую необходимо применять на основе счетчика элементов или контроля времени. На рис. 12.9 показано поведение переворачивающегося окна с применением стратегии триггера на основе размера в 10 элементов. Это означает, что в момент, когда в окне находятся 10 элементов, будет выполнен код функции Pulsar, и окно будет очищено. Это поведение не зависит от времени – не имеет значения, достигнет счетчик значения 10 в течение пяти секунд или пяти часов, – важен только момент появления в счетчике заданного значения длины окна. Поэтому первое окно заполняется приблизительно за 15 с, тогда как для заполнения последнего требуется 25 с.



**Рис. 12.9.** При использовании триггера на основе счетчика каждое переворачивающееся окно будет содержать абсолютно одинаковое количество измеряемых характеристик, но время его заполнения может быть различным

Скользящее окно использует сочетание стратегии триггера, определяющей интервал сдвига, и стратегии вытеснения, ограничивающей количество элементов данных в окне для обработки. На рис. 12.10 показано поведение скользящего окна со стратегией вытеснения, определяющей время хранения, равное 20 с, т. е. любые данные старше 20 с не будут сохраняться и использоваться в вычислениях. Для стратегии триггера также сконфигурирован интервал 20 с, т. е. через каждые 20 с будет выполняться код соответствующей функции Pulsar, и мы получим доступ ко всем данным в текущем окне для выполнения вычислений.





**Рис. 12.10.** При использовании скользящего окна каждое окно будет существовать в течение абсолютно одинакового интервала времени и, вероятнее всего, содержать различное количество метрик

Поэтому в сценарии, показанном на рис. 12.10, первое окно содержит 15 событий, а последнее – только два. В рассматриваемом здесь примере для обеих стратегий триггера и вытеснения определены параметры времени, но только для одной из них возможно определение параметра длины. Кроме того, длина окна и интервал сдвига не обязательно должны иметь одинаковое значение. Например, если необходимо выполнить вычисление простого скользящего среднего (SMA) с применением этой методики, это можно сделать, установив длину окна равной 100, а интервал сдвига равным 1.

После вычисления этих статистических сводок их можно отправить на периферийные серверы для сравнения статистических характеристик, вычисленных для аналогичного оборудования в промышленной инфраструктуре, чтобы обнаружить потенциальные проблемы, или можно перенаправить сводки в базу данных в облаке для долговременного хранения и анализа в дальнейшем. Выполнение вычислений перед сохранением статистики в базе данных позволяет избежать повторных вычислений на этапе анализа, которые могут оказаться весьма затратными операциями. Фреймворк Pulsar Functions предоставляет четыре различные конфигурации параметров окон, показанные в табл. 12.1 и позволяющие реализовать все четыре варианта, описанные в этом подразделе, с применением соответствующей комбинации параметров.

**Таблица 12.1.** Параметры конфигурации окон для функций Pulsar

Переворачивающееся окно на основе времени	--windowLengthDurationMS===xxx
Переворачивающееся окно на основе длины	--windowLengthCount===xxx
Скользящее окно на основе времени	--windowLengthDurationMS===xxx --slidingIntervalDurationMs===xxx
Скользящее окно на основе длины	--windowLengthCount===xxx --slidingIntervalCount===xxx

Реализация любого из вышеперечисленных типов окон в функциях Pulsar осуществляется просто и требует только определения в качестве типа входных данных `java.util.Collection`. Таким образом, если необходимо выполнить функцию Pulsar, показанную в листинге 12.2, для вычисления статистических характеристик в скользящем окне данных, то достаточно всего лишь указать эту функцию в команде, приведенной в листинге 12.3, а фреймворк Pulsar Functions сделает всю остальную работу.

**Листинг 12.3.** Выполнение вычисления статистических характеристик в скользящем окне данных

```
$ bin/pulsar-admin functions create \
 --jar edge-analytics-functions-1.0.0.nar \
 --classname com.manning.pulsar.iot.analytics.TimeSeriesSummaryFunction \
 --windowLengthDurationMS==20000 \ ①
 --slidingIntervalDurationMs=20000. ②
```

① Определение стратегии вытеснения: 20 с.

② Определение стратегии триггера: 20 с.

Это упрощает применение любой из двух методик формирования окон без необходимости написания значительного объема повторяющегося кода для выполнения сбора и сохранения отдельных событий. Таким образом, вы можете полностью сосредоточиться исключительно на реализации требуемой бизнес-логики.

### 12.4.3. Аппроксимация

При анализе потоковых данных существуют определенные типы запросов, которые невозможно вычислить на периферии, поскольку для них требуется огромный объем вычислительных ресурсов и времени для получения точных результатов. Примеры: различия в счетчиках, квантили, наиболее часто встречающиеся элементы, соединения, вычисление матриц и анализ графов. Обычно эти типы вычислений вообще не выполняются в потоковых наборах данных. Но если достаточно получить приблизительные результаты, то для этого существует особая категория потоковых алгоритмов для предоставления приблизительных (аппроксимированных) значений, оценок и выборок данных для статистического анализа, когда поток событий слишком велик для сохранения его памяти или данные перемещаются слишком быстро.

Такие потоковые алгоритмы используют небольшие структуры данных – скетчи (sketches), обычно имеющие размер в несколько килобайт для хранения информации. Алгоритмы на основе скетчей выполняют обработку «в одно касание» (one-touch processing), т. е. они должны прочитать каждый элемент в потоке только один раз. Оба вышеописанных свойства делают эти

алгоритмы идеальными кандидатами для развертывания на периферийных устройствах. Существуют четыре семейства скетч-алгоритмов, каждое из которых специально предназначено для решения отдельного типа задач:

- скетчи мощности множества (количества элементов) (cardinality sketches) – предоставляют приблизительные счетчики для каждого отдельного значения в потоке, например количество просмотров страниц в некотором множестве различных веб-страниц за определенный интервал времени;
- скетчи часто встречающихся элементов (frequent item sketch) – предоставляют список наиболее часто наблюдаемых значений в потоке, например первые 10 веб-страниц, наиболее часто просматриваемые в течение определенного интервала времени;
- скетчи выборок (sampling sketches) – используют семейство вероятностных алгоритмов резервуарной выборки (reservoir sampling)<sup>1</sup> для предоставления универсальной случайной выборки данных из потока, которую можно использовать для анализа;
- скетчи квантилей (quantile sketches) – предоставляют частотную гистограмму, содержащую информацию о распределении значений потока данных, которую можно использовать для выявления аномалий.

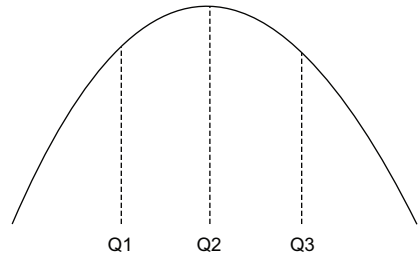
Существует библиотека с открытым исходным кодом Apache DataSketches, содержащая реализации этих алгоритмов на языке Java, которая упрощает их практическое применение в функциях Pulsar. Один из примеров применения показан в листинге 12.4 – использование скетча квантилей для выявления аномалий в показаниях датчика.

### Квантили

Термин «квантиль» (quantile) является производным от слова quantity (количество) и обозначает просто группы равного размера каких-либо элементов, обычно это распределение вероятностей или последовательность наблюдаемых значений. Вы уже знакомы с некоторыми из наиболее часто используемых квантилей, которые более привычны под названиями половина, треть, четверть и т. д. В статистике и теории вероятностей квантили более формально определяются как точки отсечения, которые делят распределение вероятностей на непрерывные смежные интервалы с равным количеством элементов.

<sup>1</sup> «Википедия» сообщает, что «нет устоявшегося русского перевода». – *Прим. перев.*

**Рис. 12.11.** Набор данных, разделенный на три секции равного размера – квантили. Значения, меньшие или равные  $Q_1$ , считаются частью первого квантиля. Значения между  $Q_1$  и  $Q_2$  – часть второго квантиля. Значения, большие или равные  $Q_3$ , – часть третьего квантиля



Например, на рис. 12.11 показаны три значения  $Q_1$ ,  $Q_2$  и  $Q_3$ , разделяющие набор данных на секции с равными весами. Области под графиком между каждой из этих точек являются квантилями, в данном случае третями, содержащими равное количество точек данных.

В нашем варианте использования обрабатывается бесконечный поток измеряемых значений, которые используются для динамически формируемого квантиля. Это позволяет принимать решения исключительно на основе наблюдаемых в реальном времени значений, чтобы определить, является ли некоторое значение аномальным, сравнивая его с предыдущими значениями той же метрики. Когда новые показания метрики приходят в функцию Pulsar, сначала они добавляются в квантиль для обновления модели распределения, затем вычисляется ранг (rank) этого показателя метрики. Ранг точнее всего описывается как пропорция значений в распределении, по отношению к которым рассматриваемое значение является большим или равным. Например, если очередное показание больше 79 % предыдущих наблюдаемых значений, то его ранг равен 79. Ранжирование значений помогает определить, является ли рассматриваемое показание нормальным, и мы принимаем решение об использовании конфигурируемого значения для установления порогового значения, которое считается аномальным.

#### Листинг 12.4. Функция Pulsar для выявления аномалий

```
import org.apache.datasketches.quantiles.DoublesSketch;
import org.apache.datasketches.quantiles.
[CA]UpdateDoublesSketch;
...

public class AnomalyDetector implements Function<Double, Void> {

 private UpdateDoublesSketch sketch;
 private double alertThreshold;
 private boolean initialized = false;

 @Override
 public Void process(Double input, Context ctx) throws Exception {
 if (!initialized) {
 init(ctx);
 }
 }
}
```

```

 sketch.update(input); 2

 if (sketch.getRank(input) >= alertThreshold) { 3
 react(); 4
 }
 return null;
}

protected void init(Context ctx) {
 sketch = DoublesSketch.builder().build();
 alertThreshold = (double) ctx.getUserConfigValue ("threshold");
 initialized = true;
}

protected void react() { 5
 // Конкретная реализация.
}
}

```

- 1 Эта функция использует библиотеку `DataSketches` для выполнения статистических вычислений.
- 2 Добавление показателя метрики в скетч.
- 3 Получение ранга показателя метрики и сравнение его с пороговым значением.
- 4 Если метрика больше сконфигурированного порогового значения, то выполняется ответное действие.
- 5 Реализация может быть различной на основе протокола LwM2M, используемого исполнительным механизмом.

Логика этой функции проста. Сначала обновляется скетч квантиля посредством добавления в него элемента данных. Затем запрашивается относительный ранг этого значения в общем распределении. Далее определяется, является ли это значение выбросом, с помощью сравнения ранга показателя метрики с предварительно сконфигурированным пороговым значением. Если обнаружен выброс, то используется клиент с поддержкой протокола LwM2M для передачи сообщения исполнительному механизму на уровне восприятия и реакции для выполнения некоторого типа защитного действия, например отключение машины или открытие запорного клапана давления. Логика функции `react` в листинге 12.4 может быть различной, но основанной на применении протокола LwM2M и командах, которые требуется передать в ответ на событие.

## 12.5. Многомерный анализ

К текущему моменту мы реализовали разнообразные аналитические методики для данных, получаемых от одного датчика. Такой одномерный анализ позволяет выполнять выявление аномалий и исследование тренда для единственного датчика, но более ин-

тересный анализ можно выполнять только после объединения данных от нескольких датчиков, как было показано на рис. 12.7. В этом разделе рассматриваются шаги, необходимые для объединения данных, собираемых из различных IoT-шлюзов, их анализа и ответной реакции на новые обстоятельства.

### 12.5.1. Создание двунаправленной сетки обмена сообщениями

Создание фреймворка обмена сообщениями, который можно использовать для передачи сообщений из IoT-шлюзов на периферийные серверы, подразумевает начальный этап конфигурации для добавления всех кластеров Pulsar в один и тот же экземпляр Pulsar. Как было отмечено в главе 2, экземпляр Pulsar может содержать несколько кластеров Pulsar. Конфигурирование кластера как части экземпляра Pulsar также является необходимым предварительным условием для обеспечения георепликации данных между кластерами Pulsar, которая, как можно видеть на рис. 12.8, представляет собой наиболее предпочтительный механизм для перенаправления вычисляемых статистических данных датчика из IoT-шлюзов на периферийный сервер.

Первый этап конфигурации – добавление каждого отдельного кластера Pulsar на основе IoT-шлюза в один экземпляр Pulsar, а также кластера Pulsar, работающего на периферийном сервере. Это легко выполнить, используя команду `pulsar-admin` или REST API, как показано в листинге 12.5, где выполняется команда для добавления одного кластера IoT-шлюза в экземпляр Pulsar.

#### Листинг 12.5. Добавление кластера на основе IoT-шлюза в экземпляр Pulsar

```
$ pulsar-admin clusters create iot-gateway-1 \
 --broker-url http://<IoT-Gateway-IP>:6650 \
 --url http://<IoT-Gateway-IP>:8080

$ pulsar-admin clusters list
```

- ❶ Каждое имя должно быть неповторяющимся.
- ❷ URL-адрес брокера на шлюзе, доступном по протоколу TCP.
- ❸ URL сервиса для шлюза.
- ❹ Проверка списка кластеров, добавленных в список.

Эту команду необходимо выполнить по одному разу для каждого кластера Pulsar на IoT-шлюзе в среде IIoT. После добавления кластера в экземпляр Pulsar появляется возможность асинхронной доставки сообщений через механизм георепликации Pulsar без необходимости написания дополнительного кода для репликации данных.

Следующий этап организации георепликации между IoT-шлюзами и периферийными сервисами – определение абонента, который

можно использовать для двунаправленного обмена данными. Это легко сделать, используя команду `pulsar-admin` или REST API, как показано в листинге 12.6, где выполняется команда создания нового абонента Pulsar, доступного для всех кластеров на IoT-шлюзах, а также для кластеров, работающих на периферийных серверах.

#### Листинг 12.6. Создание абонента с возможностью георепликации

```
$ pulsar-admin tenants create iiot-analytics-tenant \
 --allowed-clusters iot-gateway-1, iot-gateway-2, ... \
 --admin-roles analytics-role
```

1  
2

- 1 Предоставление полного списка всех созданных кластеров на IoT-шлюзах.
- 2 Определение роли администратора для этого пространства имен.

Последний этап процесса конфигурации – создание пространства имен, которое можно использовать для георепликации данных между IoT-шлюзами и периферийными серверами. Это легко сделать, используя команду `pulsar-admin` или REST API, как показано в листинге 12.7, где выполняется команда создания нового пространства имен Pulsar, доступного из всех кластеров на IoT-шлюзах, а также для кластеров, работающих на периферийных серверах. Обратите внимание: это реплицируемое пространство имен должно находиться в ранее созданном абоненте. (Более подробную информацию о конфигурации георепликации между кластерами Pulsar можно найти в приложении В.)

#### Листинг 12.7. Создание пространства имен для георепликации

```
$ pulsar-admin namespaces create \
 iiot-analytics-tenant/analytics-namespace \
 --clusters iot-gateway-1, iot-gateway-2, ...
```

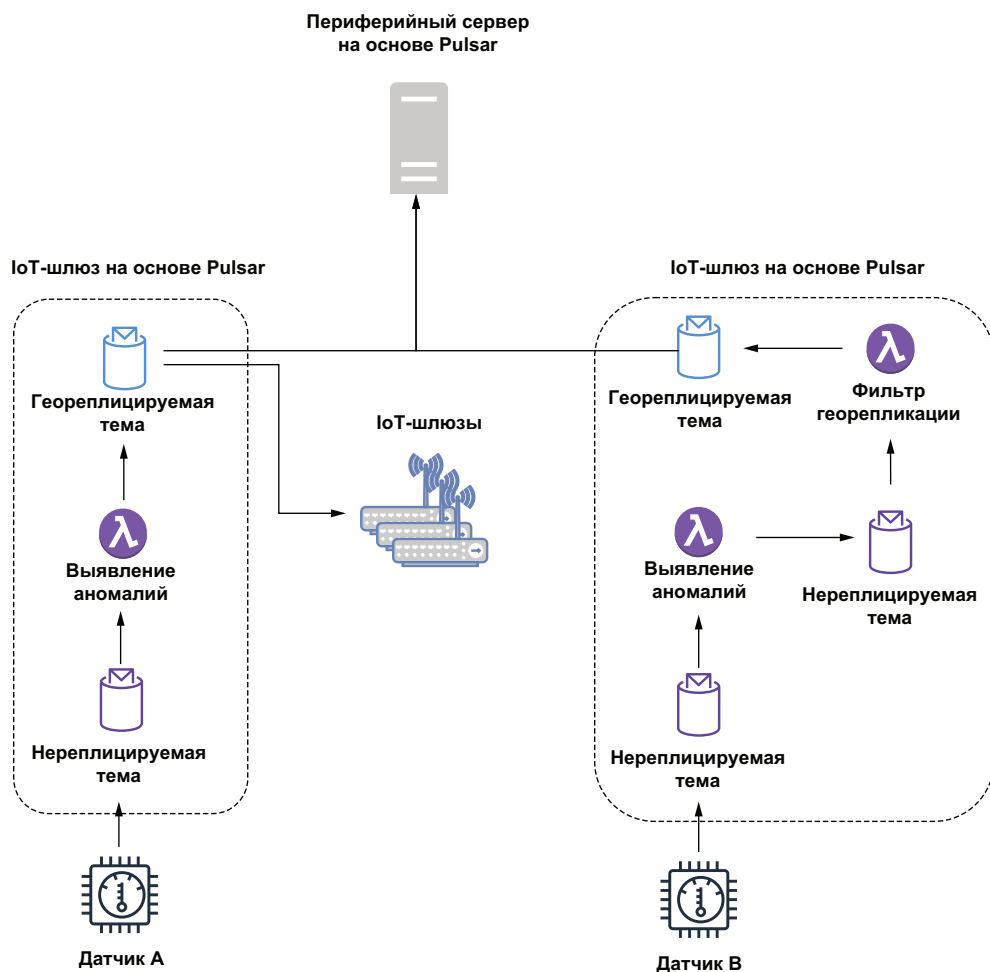
1

- 1 Предоставление полного списка всех созданных кластеров на IoT-шлюзах.

После создания пространства имен для георепликации все темы, создаваемые в нем производителями или потребителями, автоматически реплицируются во всех кластерах. Поэтому любые сообщения, публикуемые в темах геореплицируемого пространства имен на IoT-шлюзах, будут автоматически передаваться в асинхронном режиме на периферийный сервер. К сожалению, результатом этого также может стать избыточная репликация на всех прочих IoT-шлюзах в инфраструктуре ПОТ, что совершенно не входит в наши планы. Такая репликация между шлюзами не только снижает доступную пропускную способность сети, но также нецелесообразно расходует дисковое пространство на самих шлюзах, поскольку они вынуждены хранить все принятые сообщения. Так как эти сообще-

ния никогда не будут потребляться на других шлюзах, их хранение не имеет никакого смысла.

Таким образом, необходимо обеспечить выборочную репликацию, гарантирующую, что исходящие сообщения, предназначенные для потребления исключительно периферийным сервером, реплицируются только на нем, но не на всех IoT-шлюзах. Это можно сделать, написав простую функцию Pulsar, потребляющую данные из локальной темы без георепликации на конкретном шлюзе и публикующую их в геореплицируемой теме с ограничением репликации только на периферийном сервере, как показано в правой части рис. 12.12.



**Рис. 12.12.** Без использования фильтра георепликации данные датчика А автоматически реплицируются на всех кластерах IoT-шлюзов, а также на периферийном сервере, тогда как данные датчика В реплицируются только в кластере Pulsar, работающем на периферийном сервере



В листинге 12.8 можно видеть, что функция Pulsar, предназначенная для перенаправления сообщений, должна использовать существующий Java API производителя для ограничения репликации сообщений только в периферийном кластере, получая в итоге поток сообщений, показанный в правой части рис. 12.12, где сообщения сначала передаются в локальную тему перед перенаправлением в геореплицируемую тему, но только на периферийном сервере, а не во всех кластерах.

**Листинг 12.8.** Функция фильтрации геореплицируемых сообщений

```
public class GeoReplicationFilterFunction implements Function<byte[],Void> {

 private boolean initialized = false;
 private List<String> restrictReplicationTo;
 private Producer<byte[]> producer;
 private PulsarClient client;
 private String serviceUrl;
 private String topicName;

 @Override
 public Void process(byte[] input, Context ctx) throws Exception {
 if (!initialized) {
 init(ctx);
 }
 getProducer().newMessage()
 .value(input)
 .replicationClusters(restrictReplicationTo)
 .send();
 return null;
 }

 private void init(Context ctx) {
 serviceUrl = "pulsar://localhost:6650";
 topicName = ctx.getUserConfigValue("replicated-topic").get().toString();
 restrictReplicationTo = Arrays.asList(
 ctx.getUserConfigValue("edge").get().toString());
 initialized = true;
 }

 private Producer<byte[]> getProducer() throws PulsarClientException {
 if (producer == null) {
 producer = getClient().newProducer()
 .topic(topicName)
 .create();
 }
 return producer;
 }
}
```

```
private PulsarClient getClient() throws PulsarClientException {
 if (client == null) {
 client = PulsarClient.builder().serviceUrl(serviceUrl).build();
 }
 return client;
}
}
```

- ❶ Создание нового сообщения с использованием входящих байтов.
- ❷ Ограничение кластеров, на которые будет выполняться репликация этого сообщения.
- ❸ Публикация в локальной геореплицируемой теме.
- ❹ Целевая тема должна находиться в реплицируемом пространстве имен.
- ❺ Здесь должно быть имя кластера Pulsar, работающего на периферийном сервере.

Функция Pulsar выполняет запись в геореплицируемую тему на локальном компьютере, чтобы позволить механизму георепликации Pulsar управлять перенаправлением сообщения, а не отправлять его напрямую на периферийный сервер, чтобы избежать синхронных вызовов через сеть для каждого сообщения. После рассмотрения методики прямого направления в сетке обмена сообщениями сосредоточимся на обратном направлении в той же сетке, т. е. на доставке полезной информации, полученной на периферийных серверах в результате анализа данных от многочисленных датчиков, обратно в функции Pulsar, работающие на IoT-шлюзах. В действительности этот процесс весьма прост, так как периферийный сервер просто публикует данные в геореплицируемой теме, а заинтересованные стороны должны подписаться на эту тему для доставки им сообщений. Несмотря на то что тема сконфигурирована как геореплицируемая, сообщения не будут опрашиваться на те IoT-шлюзы, которые не являются активными потребителями этой темы на соответствующем узле шлюза.

### 12.5.2. Создание многомерного набора данных

Рассмотрим сценарий, в котором существует множество термодатчиков, измеряющих температуру воздуха в центре данных. Вместо сравнения текущего показания отдельного датчика с предыдущими его измерениями необходимо обеспечить сравнение со всеми другими термодатчиками, установленными в центре данных. Очевидно, что это более полезное сравнение, особенно в том случае, если показания постепенно (но не внезапно) смещаются выше или ниже обычного уровня, например если один из элементов оборудования медленно перегревается, а показания температуры постепенно выходят из безопасного диапазона на более опасный уровень. В таком сценарии может не существовать единственного датчика, показания которого имеют достаточно большое значение, чтобы считаться аномалией, но при сравнении с другими аналогичными датчиками все его показания выглядят завышенными.

Для выполнения сравнения в гораздо более обширном наборе предположительно потребуется объединение данных от сотен различных датчиков. Поэтому сначала необходимо собрать все данные в одной локации для вычисления статистических характеристик по группе датчиков, а не по отдельным датчикам. К счастью, скетчи данных, используемые в функции `AnomalyDetector`, можно с легкостью объединить для вычисления этих типов статистических характеристик. Скетч данных, сгенерированный из последовательности показаний от одного датчика, можно использовать для определения ранга текущего показания любого конкретного датчика по отношению ко всем его показаниям, зафиксированным в этом скетче на текущий момент. Но если объединяются 100 скетчей, то можно использовать итоговый скетч для определения ранга текущего показания любого конкретного датчика по отношению ко всем показаниям 100 датчиков. Это гораздо более полезное значение, которое должно способствовать устранению возникшей проблемы, поскольку мы сравниваем показания каждого датчика со всеми остальными, установленными на предприятии.

Для этого сначала необходимо изменить существующую функцию `AnomalyDetector`, как показано в листинге 12.9, для периодической отправки копии ее скетчей на периферийный сервер, чтобы можно было объединять их во всех экземплярах другой функции `BiDirectionalAnomalyDetector`, работающих на различных IoT-шлюзах.

#### Листинг 12.9. Обновление функции `AnomalyDetector`

```
public class BiDirectionalAnomalyDetector implements Function<Double,
Void> {

 private boolean initialized = false;
 private long publishInterval;
 private String reportingTopic;
 private String alertThresholdTopic;
 private double alertThreshold;
 private String remotePulsarUrl;
 private PulsarClient client;
 private Producer<byte[]> producer;
 private Consumer<Double> consumer;
 private ExecutorService service = Executors.newFixedThreadPool(1);
 private ScheduledExecutorService executor =
 Executors.newScheduledThreadPool(1);
 private UpdateDoublesSketch sketch;

 @Override
 public Void process(Double input, Context ctx) throws Exception {
 if (!initialized) {
 init(ctx);
 launchReportingThread();
 launchFeedbackConsumer();
 }
 }
}
```

```

synchronized(sketch) {
 getSketch().update(input);
 if (getSketch().getRank(input) >= alertThreshold) {
 react();
 }
}
return null;
}

private void launchReportingThread() {
 Runnable task = () -> {
 synchronized(sketch) {
 try {
 if (getSketch() != null) {

getProducer().newMessage().value(getSketch().toArray()).send();
 sketch.reset();
 }
 } catch (final PulsarClientException ex) { /* Обработчик. */ }
 }
 };
 executor.scheduleAtFixedRate(task,
 publishInterval, publishInterval, TimeUnit.MINUTES);
}

private void launchFeedbackConsumer() {
 Runnable task = () -> {
 Message<Double> msg;
 try {
 while ((msg = getConsumer().receive()) != null) {
 alertThreshold = msg.getValue();
 getConsumer().acknowledge(msg);
 }
 } catch (PulsarClientException ex) { /* Обработчик. */ }
 };
 service.execute(task);
}

private UpdateDoublesSketch getSketch() {
 if (sketch == null) {
 sketch = DoublesSketch.builder().build();
 }
 return sketch;
}

private Producer<byte[]> getProducer() throws PulsarClientException {
 if (producer == null) {
 producer = getEdgePulsarClient().newProducer(Schema.BYTES)
 .topic(reportingTopic).create();
 }
 return producer;
}

```

```

private Consumer<Double> getConsumer() throws PulsarClientException {
 if (consumer == null) {
 consumer = getEdgePulsarClient().newConsumer(Schema.DOUBLE)
 .topic(alertThresholdTopic).subscribe();
 }
 return consumer;
}

private void react() {
 // Конкретная реализация.
}

private PulsarClient getEdgePulsarClient() throws PulsarClientException {
 ...
}

protected void init(Context ctx) {
 ...
}
}

```

- ❶ Свойство, которое определяет, с каким интервалом (в мин) публикуются скетчи локальных данных.
- ❷ Свойство, определяющее имя темы, используемой для отправки скетчей локальных данных на периферийный сервер.
- ❸ Свойство, определяющее имя темы, используемой для приема обновленного сигнального порогового значения с периферийного сервера.
- ❹ Вычисляемое сигнальное пороговое значение, предоставляемое периферийным сервером.
- ❺ URL брокера Pulsar, работающего на периферийном сервере.
- ❻ Производитель для отправки скетчей данных на периферийный сервер.
- ❼ Потребитель для приема обновленного сигнального порогового значения от периферийного сервера.
- ❽ Локальный пул потоков, в котором поток потребителя может выполняться в фоновом режиме.
- ❾ Локальный пул потоков, который вызывает поток публикации через фиксированные интервалы времени (например, через каждые пять минут).
- ❿ Запуск потока публикации скетча данных только один раз.
- ⓫ Запуск потока потребления сигнального порогового значения только один раз.
- ⓬ Создание исключаяющей блокировки скетча данных при записи данных.
- ⓭ Создание исключаяющей блокировки скетча данных при публикации данных.
- ⓮ Очистка всех данных в скетче.
- ⓯ Планирование задачи публикации для выполнения через фиксированные интервалы времени, определенные свойствами конфигурации.
- ⓰ Ожидание входящих сообщений с сигнальным пороговым значением.
- ⓱ Обновление сигнального порогового значения с использованием предоставленных данных.
- ⓲ Запуск потока потребления сигнального порогового значения в фоновом режиме.

Функция `BiDirectionalAnomalyDetector` постоянно прослушивает входящие показания датчика и добавляет их в локальный объект скетча данных перед сравнением с пороговым значением аномалии, чтобы определить, нужно ли немедленно выполнить некоторое ответное действие. Но при этом также создаются два дополнительных фоновых потока для обмена данными с функцией `SketchConsolidator`, работающей на периферийном сервере. Эта функция основана на методе `Java ScheduledThreadExecutor` для гарантии того, что локальные скетчи данных публикуются с периодическими интервалами (например, через каждые пять минут). Второй поток используется для непрерывного мониторинга в теме обратной связи всех изменений сигнального порогового значения. Далее необходимо создать новую функцию `Pulsar`, подобную показанной в листинге 12.10, которая будет работать на периферийных серверах, принимать входящие скетчи и объединять их, чтобы получить укрупненный, более точный скетч, содержащий данные от всех датчиков одинакового типа.

#### Листинг 12.10. Функция объединения скетчей данных

```
import org.apache.datasketches.memory.Memory;
import org.apache.datasketches.quantiles.DoublesSketch;
import org.apache.datasketches.quantiles.DoublesUnion;
import org.apache.pulsar.functions.api.Context;
import org.apache.pulsar.functions.api.Function;

public class SketchConsolidator implements Function<byte[], Double> {
 private DoublesSketch consolidated;

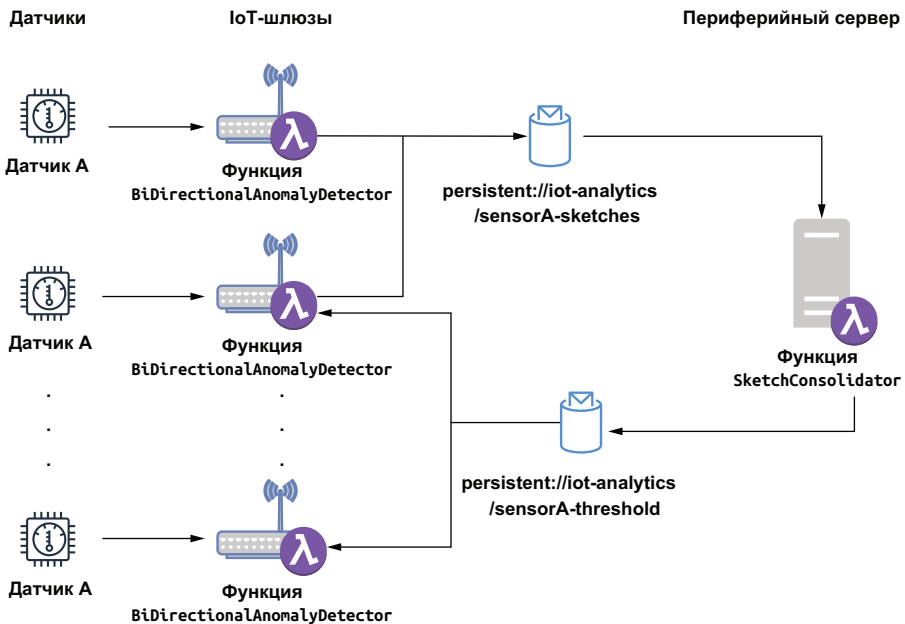
 @Override
 public Double process(byte[] bytes, Context ctx) throws Exception {
 DoublesSketch iotGatewaySketch =
 DoublesSketch.wrap(Memory.wrap(bytes));

 DoublesUnion union = DoublesUnion.builder().build();
 union.update(iotGatewaySketch);
 union.update(consolidated);
 consolidated = union.getResult();
 return consolidated.getQuantile(0.99);
 }
}
```

- ① Скетч, содержащий данные от всех датчиков.
- ② Преобразование входящих байтов в скетч данных.
- ③ Создание нового объекта, используемого для объединения нескольких скетчей данных.
- ④ Добавление входящего скетча данных в объект `union`.
- ⑤ Добавление существующего объединенного скетча данных в объект `union`.
- ⑥ Обновление объединенного скетча данных для равенства результату объединения.
- ⑦ Публикация нового вычисленного порогового значения для 99-го процента.

Взаимодействие между этими двумя функциями показано на рис. 12.13, где изображены оба канала обмена данными: первый используется для отправки данных из функций `BiDirectionalAnomalyDetector`, работающих на всех IoT-шлюзах, в функцию `SketchConsolidator`, выполняющуюся на периферийном сервере. Второй канал предназначен для передачи нового вычисленного порогового значения из функции `SketchConsolidator` обратно в экземпляры функции `BiDirectionalAnomalyDetector` на IoT-шлюзах.

Функция `SketchConsolidator` должна быть сконфигурирована для прослушивания геореплицируемой темы, в которой все функции `BiDirectionalAnomalyDetector` будут публиковать свои скетчи. В листинге 12.10 можно видеть, что после объединения скетчей данных мы используем новый созданный объект для определения точной характеристики, представляющей пороговое значение 99-го процентиля всех показаний счетчиков. Затем это значение возвращается во все экземпляры функции `BiDirectionalAnomalyDetector` вместо всего скетча (для экономии памяти и пропускной способности), чтобы они могли использовать новую вычисленную характеристику как сигнальное пороговое значение вместо вычисленного локально.



**Рис. 12.13.** Копия функции `BiDirectionalAnomalyDetector` запускается на каждом IoT-шлюзе и публикует локально вычисленные скетчи данных в одной геореплицируемой теме. Функция `SketchConsolidator` потребляет эти скетчи и объединяет их перед публикацией граничного значения для 99-го процентиля в другой геореплицированной теме. Функции `BiDirectionalAnomalyDetector` потребляют сообщения из этой темы и используют опубликованные данные как новое аномальное пороговое значение для показаний счетчика

## 12.6. Послесловие

Завершая эту последнюю главу книги, я надеюсь, что вам понравилось ее читать и вы нашли ее информативной, поучительной и заставляющей задуматься. Было приятно общаться со многими из вас на протяжении всего процесса MEAP через дискуссионный онлайн-форум, и я ценю все предоставленные вами отзывы. Приятно осознавать то, что многие из вас считают эту книгу полезной, но более важно то, как именно вы намереваетесь использовать Apache Pulsar для использования возможностей потоковой передачи данных в своих организациях.

Как и все современные технологии, Apache Pulsar будет продолжать быстро развиваться благодаря растущему и динамичному сообществу разработчиков и пользователей. В действительности с тех пор, как я начал писать эту книгу, было добавлено несколько новых функциональных возможностей, таких как поддержка транзакций. Широкое применение Pulsar в различных компаниях и отраслях промышленности является свидетельством технологической мощи этого фреймворка. Но такое стремительное развитие неизбежно приведет к тому, что содержание книги со временем будет становиться все более устаревшим, поэтому рекомендуется обратиться к следующим ресурсам для получения самой актуальной информации и описания новых функциональных возможностей. Это:

- страница проекта Apache Pulsar (<https://pulsar.apache.org/>) и соответствующей документации (<https://pulsar.apache.org/docs/en/standalone/>);
- slack-канал Apache Pulsar [apache-pulsar.slack.com](https://apache-pulsar.slack.com), который я и некоторые участники проекта просматривают ежедневно. Этот интенсивно используемый канал содержит обилие информации для начинающих, а также представляет мощное сообщество разработчиков, активно и постоянно применяющих Apache Pulsar;
- несколько блогов, в том числе на веб-сайте StreamNative (<https://streamnative.io/en/blog/>), которые ведут многие участники проекта Apache Pulsar.

Тем из вас, кто хочет применить потоковые вычисления в своей организации или ищет возможность использования архитектуры микросервисов на основе сообщений для будущих приложений, чтобы воспользоваться преимуществами развивающейся парадигмы облачных вычислений, позвольте предложить следующий совет, как убедить руководство вашей компании рассмотреть возможность внедрения Apache Pulsar: сосредоточьтесь на преимуществах использования Pulsar, например на том факте, что это облачная технология, предназначенная для независимого масштабиро-



вания уровней вычислений и хранения, что приводит к более эффективному использованию дорогостоящих облачных ресурсов. Еще одним преимуществом является то, что Pulsar может служить как системой обмена сообщениями на основе очередей, например RabbitMQ, так и платформой потокового обмена сообщениями, например Kafka, в одной системе.

Другой подход – использование Apache Pulsar в качестве базовой технологии для внедрения внутри организации совершенно новой функциональности, такой как микросервисы. Можно начать разработку приложений для микросервисов в своей организации, используя функции Apache Pulsar в качестве базовой технологии. Команда разработчиков получит преимущество от простой модели программирования, которую предоставляет Pulsar Functions, без необходимости использования собственного API. Если ваша компания использует одну из технологий вычислений без сервера от облачных провайдеров, например AWS Lambda, то вы можете особо выделить тот факт, что фреймворк Pulsar Functions обеспечивает ту же функциональность всего лишь за часть стоимости и без привязки к конкретному провайдеру. Кроме того, весь процесс разработки и тестирования приложений можно бесплатно выполнить локально, а не на дорогостоящих вычислительных ресурсах AWS.

Если в вашей организации уже применяется проверенная технология, такая как Apache Kafka, то было бы разумно прислушаться к высказыванию Марка Твена: «Легче обмануть людей, чем убедить их, что их обманули». Эта фраза кратко отражает нежелание людей смириться с тем, что они, возможно, сделали не самый удачный выбор. Поэтому вместо того, чтобы вести разговор как соперничество между существующей технологией и Pulsar, чтобы убедить руководство организации в том, что они приняли неправильное решение, вам следует сосредоточиться на тех преимуществах, которые Pulsar предоставляет вашей организации и которые не может обеспечить другая технология. Таким образом, Pulsar можно рассматривать как вспомогательную технологию, которая дополнит инфраструктуру организации, а не как заменяющую технологию, которая потребует значительных изменений во всей инфраструктуре. Предложение стратегии полной замены встретит большое сопротивление со стороны тех, кто выбрал проверенную технологию, и тех, кто вложил много времени и усилий в разработку решений на ее основе. У вас будет больше шансов на успех, если вы сосредоточитесь на сильных сторонах Pulsar (о которых говорилось в главе 1), а не на слабых сторонах существующей технологии.

Еще раз благодарю читателей за интерес к Apache Pulsar. Я надеюсь, что эта книга способствовала тому, чтобы вы начали использовать Apache Pulsar в некоторых своих проектах, и с нетерпением жду возможности пообщаться с вами на одном из многочисленных форумов сообщества открытого исходного кода Pulsar.

## 12.7. Резюме

- Интервал времени между возникновением события и ответной реакцией на него называется временной ценностью данных. Эта временная ценность со временем быстро снижается, поэтому важно как можно быстрее выполнить ответное действие.
- Pulsar Functions можно использовать для предоставления аналитики почти в реальном времени по данным IoT в среде периферийных вычислений, состоящей преимущественно из устройств с ограниченными ресурсами, такими как IoT-шлюзы.
- Выполнение функций Pulsar непосредственно на IoT-шлюзах maximizes временную ценность данных.
- Механизм георепликации Pulsar можно использовать для создания двунаправленной сети обмена данными между кластерами Pulsar, работающими на IoT-шлюзах, и кластерами Pulsar на периферийных серверах.

# Приложение А.

## Запуск Pulsar в Kubernetes

---

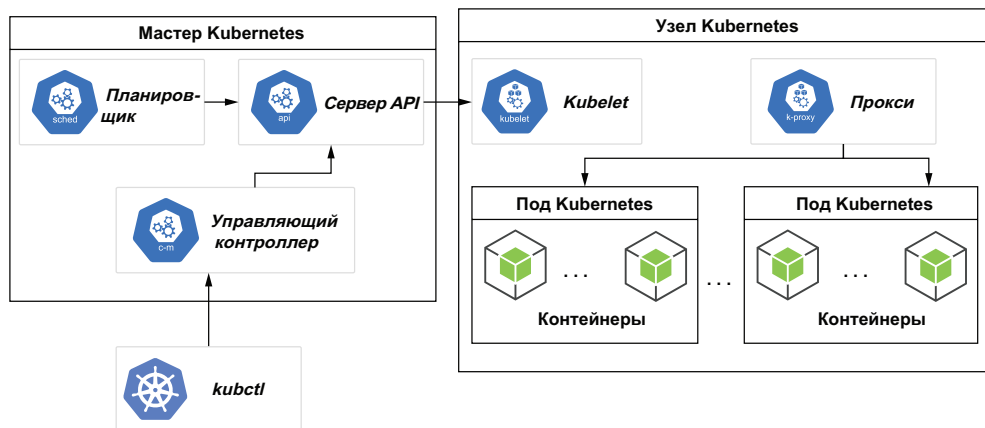
Kubernetes – популярная платформа с открытым исходным кодом для развертывания и запуска контейнерных приложений в крупном масштабе. Эта платформа создана в компании Google как решение для управления ее обширной инфраструктурой путем автоматизации многих ручных процессов, связанных с развертыванием, управлением и масштабированием приложений на многочисленных хостах. Для получения более подробной информации о Kubernetes настоятельно рекомендуется книга «Kubernetes in Action», автор Марко Лукша (Marko Lukša) (Manning, 2017 г.).

Основная цель Kubernetes – планирование работы контейнеров в кластере физических или виртуальных машин на основе доступных вычислительных ресурсов и требований к ресурсам каждого контейнера. Контейнер – это просто готовый к запуску программный пакет, содержащий все необходимое для запуска приложения. Как мы видели в главе 3, Docker – одна из самых популярных контейнерных технологий. Поэтому, разумеется, Kubernetes можно использовать для планирования и запуска контейнеров Docker, включая созданные проектом Apache Pulsar. Это позволяет запускать кластер Pulsar и все его компоненты, такие как Zookeeper, BookKeeper и Pulsar Proxy, полностью в кластере Kubernetes. В этом приложении подробно описан процесс планирования и запуска.

### **А.1. Создание кластера Kubernetes**

Кластер – это основа для запуска контейнерных приложений. В Kubernetes кластер состоит как минимум из одного главного кластера (cluster master) и нескольких рабочих компьютеров, называе-

мых узлами (nodes), как показано на рисунке А.1. На главном компьютере кластера размещается уровень управления Kubernetes, который выполняет все административные функции кластера, а узлы – это компьютеры, на которых будут размещаться сами контейнеры.



**Рис. А.1.** Главный узел используется для управления всеми узлами Kubernetes в пуле узлов. На каждом узле Kubernetes может размещаться несколько модулей, которые, в свою очередь, могут содержать один или несколько контейнеров приложений

Вычислительные ресурсы всех узлов регистрируются мастером кластера и образуют пул ресурсов, из которого извлекаются все контейнеры. Например, если в некотором конкретном контейнере размещено приложение базы данных, для которого требуется 8 Гб ОЗУ и четыре ядра процессора, то мастеру кластера придется найти узел с достаточными ресурсами для удовлетворения этого запроса и запустить на нем контейнер. Затем эти заявленные ресурсы будут вычтены из пула ресурсов, чтобы указать, что они уже переданы в контейнер. После исчерпания пула ресурсов кластера невозможно разместить новые контейнеры до тех пор, пока не станут доступны требуемые ресурсы.

С помощью Kubernetes ресурсы можно с легкостью наращивать, добавляя в кластер дополнительные узлы. Это позволяет эффективно масштабировать кластер в соответствии с потребностями. Эта функция настолько привлекательна, что почти все провайдеры облачных сервисов предлагают тот или иной вариант Kubernetes для размещения приложений. Кроме того, существует крупная реализация Kubernetes с открытым исходным кодом, известная как OpenShift, которая позволяет размещать кластер Kubernetes на вашем собственном физическом оборудовании. Хотя оба эти варианта являются неплохим выбором для производственных приложений, они создают относительно высокий барьер для локального развития. Большинство людей не хотят платить за хостинг крупного кластера Kubernetes, предназначенного только лишь

для разработки или тестирования, поэтому `minikube` является широко распространенным вариантом для разработчиков. Pulsar был разработан специально для работы в контейнерной среде, такой как Kubernetes, где вы можете легко увеличивать или уменьшать количество контейнеров брокеров Pulsar и/или букмекеров BookKeeper в зависимости от конкретных требований.

### A.1.1. Предварительные условия для установки

Для применения Kubernetes необходимо установить инструментальное средство командной строки `kubectl`, которое позволяет выполнять команды в кластерах Kubernetes. Этот инструмент также потребуется для развертывания приложений, для наблюдения и управления ресурсами кластера, а также для просмотра журналов. Чтобы установить `kubectl`, необходимо скачать его (<https://kubernetes.io/docs/tasks/tools/#before-you-begin>) и выполнить инструкции для вашей операционной системы.

#### Листинг А.1. Установка `kubectl` на MacBook

```
brew install kubectl
. . .
==> Downloading https://homebrew.bintray.com/bottles/
 └─ kubernetes-cli-1.19.1.catalina.bottle.tar.gz
==> Pouring kubernetes-cli-1.19.1.catalina.bottle.tar.gz
==> Caveats
Bash completion has been installed to:
 /usr/local/etc/bash_completion.d
zsh completions have been installed to:
 /usr/local/share/zsh/site-functions
==> Summary
 /usr/local/Cellar/kubernetes-cli/1.19.1: 231 files, 49MB
```

- ❶ Использование Homebrew для установки `kubectl`.
- ❷ Загрузка и установка версии 1.19.1.

Если вы работаете на Mac, то можете воспользоваться менеджером пакетов Homebrew для установки `kubectl` единственной командой, как показано в листинге А.1. В другой операционной системе следует обратиться к онлайн-документации для получения инструкций по установке в конкретной ОС. Необходимо обязательно использовать версию `kubectl`, миноре версии которой отличается не больше, чем на единицу от версии кластера. То есть лучше всего использовать самую последнюю версию `kubectl`, чтобы избежать проблем с совместимостью.

### A.1.2. Minikube

После установки `kubectl` следующий шаг – создание кластера Kubernetes для размещения кластера Pulsar. Хотя все основные

облачные провайдеры предоставляют среды Kubernetes, хорошо подходящие для промышленного применения, в этом приложении я буду использовать более дешевый альтернативный вариант под названием `minikube`, позволяющий запустить кластер Kubernetes на компьютере разработки.

Инструментальное средство `minikube` запускает кластер Kubernetes с одним узлом на персональном компьютере. Такой вариант вполне подходит для выполнения повседневных задач разработки, требующих доступа к контейнерному приложению, например к Pulsar. Это неплохой выбор, если необходимо разработать и протестировать приложение в среде Kubernetes для ознакомления с Kubernetes API.

#### Листинг А.2. Установка `minikube` на MacBook

```
brew install minikube ❶
...
==> Downloading https://homebrew.bintray.com/bottles/minikube-
 ➡ 1.13.0.catalina.bottle.tar.gz ❷
Already downloaded: /Users/david/Library/Caches/Homebrew/downloads/
➡ b4e7b1579cd54deea3070d595b60b315ff7244ada9358412c87ecfd061819d9b--
➡ minikube-1.13.0.catalina.bottle.tar.gz
==> Pouring minikube-1.13.0.catalina.bottle.tar.gz
==> Caveats
Bash completion has been installed to:
 /usr/local/etc/bash_completion.d

zsh completions have been installed to:
 /usr/local/share/zsh/site-functions
==> Summary
 /usr/local/Cellar/minikube/1.13.0: 8 files, 62.2MB
```

- ❶ Использование Homebrew для установки `minikube`.
- ❷ Загрузка и установка версии 1.13.0.

Чтобы установить `minikube`, необходимо скачать его (<https://minikube.sigs.k8s.io/docs/start/>) и выполнить инструкции для вашей операционной системы. Если вы работаете на Mac, то можете воспользоваться менеджером пакетов Homebrew для установки `minikube` единственной командой, как показано в листинге А.2. В другой операционной системе следует обратиться к онлайн-вой документации для получения инструкций по установке в конкретной ОС. После установки `minikube` следующий шаг – создание кластера Kubernetes командами, показанными в листинге А.3. Первая команда создает кластер и определяет ресурсы, предоставляемые конкретным ноутбуком (в данном случае – ноутбуком автора) для пула ресурсов кластера.

**Листинг А.3.** Создание кластера Kubernetes с использованием minikube

```
minikube start \
 --memory=8192 \
 --cpus=4 \
 --kubernetes-version=v1.19.0

kubectl config use-context minikube
```

- ❶ Резервирование 8 Гб оперативной памяти для кластера.
- ❷ Резервирование четырех ядер процессора для кластера.
- ❸ Определение используемой версии Kubernetes.
- ❹ Настройка kubectl для использования minikube.

Чтобы инструмент kubectl мог найти кластер Kubernetes и получить доступ к нему, сначала необходимо сконфигурировать kubectl для указания на требуемый кластер, с которым он будет взаимодействовать. Эта связь управляется файлом *kubeconfig*, создаваемым автоматически при развертывании кластера minikube и находящимся в каталоге *~/.kube/config*. Можно воспользоваться командой `kubectl config use-context <cluster-name>`, показанной в листинге А.3, для конфигурирования kubectl, чтобы указать на новый созданный кластер minikube. Правильность конфигурации можно проверить командой `kubectl cluster-info`, которая возвращает основную информацию о кластере Kubernetes.

## А.2. Pulsar Helm chart

После установки и запуска кластера Kubernetes можно развернуть в нем контейнерное приложение. Это можно сделать с помощью конфигурационного файла развертывания, содержащего всю информацию, необходимую для создания всех контейнеров, требуемых приложением. Такие конфигурационные файлы развертывания представляют собой просто YAML-файлы с особой структурой, показанной в листинге А.4, где приведена конфигурация для одного веб-сервера на основе Nginx, прослушивающего порт 80 в ожидании входящих запросов.

**Листинг А.4.** Конфигурационный файл Kubernetes для развертывания приложения

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: mysite
 labels:
 name: mysite
spec:
```

```
replicas: 1
template:
 metadata:
 labels:
 app: mysite
 spec:
 containers:
 - name: mysite
 image: nginx
 resources:
 limits:
 memory: "128Mi"
 cpu: "500m"
 ports:
 - containerPort: 80
```

- ❶ Определение версии API для этого конфигурационного файла.
- ❷ Указание типа ресурса, определяемого в этом конфигурационном файле.
- ❸ Имя приложения.
- ❹ Количество создаваемых подов.
- ❺ Определение всех контейнеров внутри каждого пода.
- ❻ Имя используемого образа Docker.
- ❼ Ресурсы, необходимые для контейнера Nginx.
- ❽ Объявленный порт для контейнера.

После создания такого файла можно выполнить команду `kubectl apply -f filename` для развертывания указанного приложения в созданном кластере Kubernetes. Хотя описанный подход относительно прост, тем не менее он несколько утомителен при создании и редактировании вручную всех файлов конфигурации. Можно видеть, что файл развертывания для простого приложения с одним контейнером содержит 22 строки кода YAML. Можно представить, каким большим и сложным окажется файл развертывания для такого крупного приложения, как Pulsar, для которого потребуются многочисленные экземпляры нескольких контейнеров (брокеров, букмекеров, ZooKeeper и т. д.).

Развертывание контейнеров крупномасштабных приложений с оркестровкой Kubernetes – сложный процесс. Разработчики могут использовать неправильные данные для файлов конфигурации или не иметь опыта развертывания приложений по шаблонам YAML. Поэтому было создано инструментальное средство Helm для упрощения процесса развертывания контейнерных приложений в среде Kubernetes.

### A.2.1. Что такое Helm

Helm – это менеджер пакетов для Kubernetes, позволяющий разработчикам с легкостью упаковывать, конфигурировать и развертывать приложения и сервисы в кластерах Kubernetes. Helm похож



на менеджеры пакетов Linux, такие как YUM или APT, поскольку все они позволяют развернуть программный пакет со всеми его зависимостями одной простой командой.

Мы будем использовать Helm для установки кластера Pulsar, поэтому необходимо установить этот менеджер пакетов. Если вы работаете на Mac, то можете воспользоваться менеджером пакетов Homebrew для установки Helm единственной командой, показанной в листинге А.5. В другой операционной системе следует обратиться к онлайн-документации (<https://helm.sh/docs/intro/install/>) для получения инструкций по установке в конкретной ОС.

#### Листинг А.5. Установка Helm на MacBook

```
brew install helm ❶
...
==> Downloading https://homebrew.bintray.com/bottles/
 ➡ helm-3.3.1.catalina.bottle.tar.gz ❷
Already downloaded: /Users/david/Library/Caches/Homebrew/downloads/
 ➡ 77e13146a8989356ceaba3a19f6ee6a342427d88975394c91a263ae1c35a3eb6--helm-
 ➡ 3.3.1.catalina.bottle.tar.gz
==> Pouring helm-3.3.1.catalina.bottle.tar.gz
==> Caveats
Bash completion has been installed to:
 /usr/local/etc/bash_completion.d

zsh completions have been installed to:
 /usr/local/share/zsh/site-functions
==> Summary
 /usr/local/Cellar/helm/3.3.1: 56 files, 40.3MB
```

❶ Использование Homebrew для установки Helm.

❷ Загрузка и установка версии 3.3.1.

Helm позволяет разместить приложения Kubernetes в пакетах с предварительно сконфигурированными ресурсами Kubernetes. Такие пакеты называются *charts*<sup>1</sup>. Helm chart предоставляет возможность развертывания и удаления приложений буквально «одной кнопкой», упрощая разработку и развертывание приложений Kubernetes для разработчиков, не имеющих опыта работы с контейнерами и микросервисами.

#### Структура Helm chart

Helm chart – это просто набор файлов в некотором каталоге. Имя этого каталога используется как имя для chart. Внутри каталога Helm chart содержится файл *chart.yaml*, описывающий саму струк-

<sup>1</sup> В русскоязычных публикациях и даже в документации используется исключительно термин «charts», т. е. достойного варианта перевода пока не найдено. – *Прим. перев.*

туру chart, файл *values.yaml* и один или несколько файлов манифеста, расположенных в подкаталоге шаблонов (*templates*) chart, как показано в листинге А.6.

#### Листинг А.6. Схема каталога Helm chart

```
package-name/
 charts/
 templates/ ❶
 Chart.yaml ❷
 values.yaml ❸
 requirements.yaml ❹
```

- ❶ Подкаталог файлов манифеста.
- ❷ Файл, описывающий саму структуру chart.
- ❸ Значения по умолчанию, используемые в шаблонах.
- ❹ Необязательный список зависимостей.

Helm chart использует YAML-шаблоны для конфигурации приложения вместе с отдельным файлом *values.yaml*, содержащим все значения, вставляемые в шаблоны YAML во время установки. По существу, Helm charts можно считать файлами Kubernetes, которые можно параметризовать.

Когда chart готов для развертывания, можно воспользоваться командой `helm package <chart-name>` для создания tar-архива, содержащего все перечисленные выше файлы. После упаковки можно использовать Helm chart, выполнив команду `helm install` с предоставлением требуемых значений параметров конфигурации во внешнем файле значений или как аргументов для этой команды. Переданные значения применяются при создании приложения Kubernetes командой `helm install <chartname>`.

### А.2.2. Pulsar Helm chart

В предыдущем подразделе были описаны Helm charts и их использование для развертывания приложений. К счастью, существует Helm chart для Apache Pulsar, включенный в дистрибутивный комплект с открытым исходным кодом, и вы можете с легкостью получить доступ к chart, клонируя репозиторий командой `git`, как показано в листинге А.7.

#### Листинг А.7. Загрузка Pulsar Helm chart

```
git clone https://github.com/apache/pulsar-helm-chart ❶

cd pulsar-helm-chart ❷
```

- ❶ Клонирование репозитория Helm chart.
- ❷ Переход в каталог, в который был клонирован репозиторий.

После клонирования репозитория можно просмотреть содержимое Helm chart в соответствующем подкаталоге, как показано в листинге А.8. Структура этого подкаталога соответствует структуре каталога Helm chart, показанной выше в листинге А.6, с файлами *Chart.yaml* и *values.yaml* на основном уровне и файлами шаблонов в подкаталоге *templates*.

#### Листинг А.8. Содержимое каталога Pulsar Helm chart

```
ls ./charts/pulsar/ ❶
Chart.yaml templates values.yaml

ls ./charts/pulsar/templates/*.yaml ❷
./charts/pulsar/templates/autorecovery-configmap.yaml
./charts/pulsar/templates/autorecovery-service.yaml
./charts/pulsar/templates/autorecovery-statefulset.yaml
./charts/pulsar/templates/bookkeeper-cluster-initialize.yaml
./charts/pulsar/templates/bookkeeper-configmap.yaml
./charts/pulsar/templates/bookkeeper-pdb.yaml
./charts/pulsar/templates/bookkeeper-podmonitor.yaml
./charts/pulsar/templates/bookkeeper-service.yaml
./charts/pulsar/templates/bookkeeper-statefulset.yaml
./charts/pulsar/templates/bookkeeper-storageclass.yaml
./charts/pulsar/templates/broker-cluster-role-binding.yaml
./charts/pulsar/templates/broker-configmap.yaml
./charts/pulsar/templates/broker-pdb.yaml
./charts/pulsar/templates/broker-podmonitor.yaml
./charts/pulsar/templates/broker-rbac.yaml
./charts/pulsar/templates/broker-service-account.yaml
./charts/pulsar/templates/broker-service.yaml
./charts/pulsar/templates/broker-statefulset.yaml
./charts/pulsar/templates/dashboard-deployment.yaml
./charts/pulsar/templates/dashboard-ingress.yaml
./charts/pulsar/templates/dashboard-service.yaml
./charts/pulsar/templates/function-worker-configmap.yaml
./charts/pulsar/templates/grafana-admin-secret.yaml
./charts/pulsar/templates/grafana-configmap.yaml
./charts/pulsar/templates/grafana-deployment.yaml
./charts/pulsar/templates/grafana-ingress.yaml
./charts/pulsar/templates/grafana-service.yaml
./charts/pulsar/templates/keytool.yaml
./charts/pulsar/templates/namespace.yaml
./charts/pulsar/templates/prometheus-configmap.yaml
./charts/pulsar/templates/prometheus-deployment.yaml
./charts/pulsar/templates/prometheus-pvc.yaml
./charts/pulsar/templates/prometheus-rbac.yaml
./charts/pulsar/templates/prometheus-service.yaml
./charts/pulsar/templates/prometheus-storageclass.yaml
./charts/pulsar/templates/proxy-configmap.yaml
./charts/pulsar/templates/proxy-ingress.yaml
./charts/pulsar/templates/proxy-pdb.yaml
./charts/pulsar/templates/proxy-podmonitor.yaml
```

```
./charts/pulsar/templates/proxy-service.yaml
./charts/pulsar/templates/proxy-statefulset.yaml
./charts/pulsar/templates/pulsar-cluster-initialize.yaml
./charts/pulsar/templates/pulsar-manager-admin-secret.yaml
./charts/pulsar/templates/pulsar-manager-configmap.yaml
./charts/pulsar/templates/pulsar-manager-deployment.yaml
./charts/pulsar/templates/pulsar-manager-ingress.yaml
./charts/pulsar/templates/pulsar-manager-service.yaml
./charts/pulsar/templates/tls-cert-internal-issuer.yaml
./charts/pulsar/templates/tls-certs-internal.yaml
./charts/pulsar/templates/toolset-configmap.yaml
./charts/pulsar/templates/toolset-service.yaml
./charts/pulsar/templates/toolset-statefulset.yaml
./charts/pulsar/templates/zookeeper-configmap.yaml
./charts/pulsar/templates/zookeeper-pdb.yaml
./charts/pulsar/templates/zookeeper-podmonitor.yaml
./charts/pulsar/templates/zookeeper-service.yaml
./charts/pulsar/templates/zookeeper-statefulset.yaml
./charts/pulsar/templates/zookeeper-storageclass.yaml
```

- ❶ Просмотр структуры сгенерированного каталога Pulsar Helm chart.
- ❷ Список всех сгенерированных шаблонов.

В листинге А.8 можно видеть, что несколько шаблонов содержат всю логику chart. Рассмотрим шаблоны, связанные с брокерами Pulsar, чтобы лучше понять их подробности.

#### Листинг А.9. Файл конфигурации развертывания брокера Pulsar

```
cat ./charts/pulsar/templates/broker-service.yaml ❶
...

{{- if .Values.components.broker }}
apiVersion: v1
kind: Service
metadata:
 name: "{{ template "pulsar.fullname" . }}-{{ .Values.broker.component }}"
 namespace: {{ .Values.namespace }}
 labels:
 {{- include "pulsar.standardLabels" . | nindent 4 }}
 component: {{ .Values.broker.component }}
 annotations:
 {{ toYaml .Values.broker.service.annotations | indent 4 }}
spec:
 ports:
 # prometheus needs to access /metrics endpoint
 - name: http
 port: {{ .Values.broker.ports.http }} ❷
 {{- if or (not .Values.tls.enabled) (not .Values.tls.broker.enabled) }}
 - name: pulsar
 port: {{ .Values.broker.ports.pulsar }} ❸
 {{- end }}
```

```

{{- if and .Values.tls.enabled .Values.tls.broker.enabled }} ❷
- name: https
 port: {{ .Values.broker.ports.https }} ❸
- name: pulsarssl
 port: {{ .Values.broker.ports.pulsarssl }} ❹
{{- end }}
clusterIP: None
selector:
 app: {{ template "pulsar.name" . }}
 release: {{ .Release.Name }}
 component: {{ .Values.broker.component }}
{{- end }}

```

- ❶ Файл, содержащий определение сервиса Pulsar Broker.
- ❷ Используемый порт HTTP.
- ❸ Используемый порт данных.
- ❹ Должен ли брокер использовать протокол TLS.
- ❺ Используемый защищенный порт HTTPS.
- ❻ Используемый защищенный порт данных.

В листинге А.9 можно видеть, что файл определения брокера Pulsar зависит от конкретных значений параметров конфигурации. Вероятно, вы уже поняли, что эти значения предоставляются в файле *values.yaml*, сгенерированном при выполнении скрипта, создающего Pulsar Helm chart. В листинге А.10 показан соответствующий раздел файла *values.yaml*, содержащий определения для брокера Pulsar.

#### Листинг А.10. Значения для брокера Pulsar в файле values.yaml

```

Pulsar: Broker cluster
templates/broker-statefulset.yaml
##
broker:
 # Использование имени компонента, совпадающего с конфигурацией grafana,
 # чтобы метрики правильно отображались в панели управления grafana.
 component: broker
 replicaCount: 3
 # Если использование Prometheus-Operator разрешает PodMonitor
 # для обнаружения целей, то Prometheus-Operator не добавляет цели
 # на основе аннотаций k8s.
 podMonitor:
 enabled: false
 interval: 10s
 scrapeTimeout: 10s
 ports:
 http: 8080
 https: 8443
 pulsar: 6650
 pulsarssl: 6651
 # nodeSelector:
 # cloud.google.com/gke-nodepool: default-pool
 ...
 resources:

```

```

requests:
 memory: 512Mi
 cpu: 0.2
Broker configmap
templates/broker-configmap.yaml
##
configData:
 PULSAR_MEM: >
 -Xms128m -Xmx256m -XX:MaxDirectMemorySize=256m
 PULSAR_GC: >
 -XX:+UseG1GC
 -XX:MaxGCPauseMillis=10
 -Dio.netty.leakDetectionLevel=disabled
 -Dio.netty.recycler.linkCapacity=1024
 -XX:+ParallelRefProcEnabled
 -XX:+UnlockExperimentalVMOptions
 -XX:+DoEscapeAnalysis
 -XX:ParallelGCThreads=4
 -XX:ConcGCThreads=4
 -XX:G1NewSizePercent=50
 -XX:+DisableExplicitGC
 -XX:-ResizePLAB
 -XX:+ExitOnOutOfMemoryError
 -XX:+PerfDisableSharedMem
 managedLedgerDefaultEnsembleSize: "2"
 managedLedgerDefaultWriteQuorum: "2"
 managedLedgerDefaultAckQuorum: "2"

```

- ❶ Определение общего количества экземпляров брокера: три.
- ❷ Раздел определения различных значений портов.
- ❸ Раздел определения ресурсов пода.
- ❹ Отображение значений конфигурации для связанного с этим шаблоном брокера.
- ❺ Настройки памяти JVM для подов брокера.
- ❻ Настройки механизма сборки мусора JVM для подов брокера.
- ❼ Общий размер для журнала регистрации Pulsar.
- ❽ Размер кворума записи для журнала регистрации Pulsar.
- ❾ Кворум подтверждения для журнала регистрации Pulsar.

В листинге А.9 можно видеть, что значения этих параметров невелики с точки зрения ресурсов. Причина в том, что по умолчанию Pulsar Helm chart специально спроектирован для развертывания на основе minikube. При необходимости вы можете изменить эти значения.

### А.3. Использование Pulsar Helm chart

После загрузки и изучения Pulsar Helm chart следующий шаг – использование его для создания кластера Pulsar. Первым этапом этого процесса является добавление Pulsar Helm chart в локальный репозиторий и его инициализация, как показано в листинге А.11. Это позволит локальному клиенту Helm найти и загрузить Pulsar Helm chart.

**Листинг А.11.** Добавление Pulsar Helm chart в локальный репозиторий Helm

```

helm repo add apache https://pulsar.apache.org/charts ❶

./scripts/pulsar/prepare_helm_release.sh \
--create-namespace \
--namespace pulsar \
--release pulsar-mini ❷
namespace/pulsar created ❸
generate the token keys for the pulsar cluster ❹
The private key and public key are generated to /var/folders/zw/
➤ x39hv0dd7133w9v9cgnt1lvr0000gn/T/tmp.QT3EjywR and
➤ /var/folders/zw/x39hv0dd7133w9v9cgnt1lvr0000gn/T/tmp.YkhbAyG
➤ successfully.
secret/pulsar-mini-token-asymmetric-key created
generate the tokens for the super-users: proxy-admin,broker-admin,admin ❺
generate the token for proxy-admin
secret/pulsar-mini-token-proxy-admin created
generate the token for broker-admin
secret/pulsar-mini-token-broker-admin created
generate the token for admin
secret/pulsar-mini-token-admin created ❻

The jwt token secret keys are generated under: ❼
- 'pulsar-mini-token-asymmetric-key'

The jwt tokens for superusers are generated and stored as below: ❽
- 'proxy-admin':secret('pulsar-mini-token-proxy-admin')
- 'broker-admin':secret('pulsar-mini-token-broker-admin')
- 'admin':secret('pulsar-mini-token-admin')

```

- ❶ Добавление репозитория Pulsar Helm в локальный репозиторий Helm.
- ❷ Оповещение Helm о необходимости создания пространства имен Kubernetes.
- ❸ Имя создаваемого пространства имен Kubernetes.
- ❹ Имя релиза Pulsar.
- ❺ Генерация файлов открытого и секретного токена.
- ❻ Генерация токенов для различных пользователей с правами администратора.
- ❼ Генерация секретного токена JWT.
- ❽ Генерация токенов доступа JWT.

Завершающий шаг процесса – использование Helm для установки кластера Pulsar, как показано в листинге А.12. Важно определить параметр `initialize=true` при первой установке релиза Pulsar, так как это гарантирует правильность инициализации метаданных кластера для BookKeeper и Pulsar.

**Листинг А.12.** Установка Pulsar с использованием Helm chart

```

helm install \
--set initialize=true \
--values examples/values-minikube.yaml \
pulsar-mini \
apache/pulsar

kubectrl get pods -n pulsar -o name
pod/pulsar-mini-bookie-0
pod/pulsar-mini-bookie-init-94r5z
pod/pulsar-mini-broker-0
pod/pulsar-mini-grafana-6746b4bf69-bjtff
pod/pulsar-mini-prometheus-5556dbb8b8-m8287
pod/pulsar-mini-proxy-0
pod/pulsar-mini-pulsar-init-dmztl
pod/pulsar-mini-pulsar-manager-6c6889dff-q9t5q
pod/pulsar-mini-toolset-0
pod/pulsar-mini-zookeeper-0

```

- ① Требование инициализации метаданных кластера.
- ② Использование значений из указанного файла.
- ③ Особое неповторяющееся имя для этого кластера.
- ④ Используемый Helm chart.
- ⑤ Список всех подов, созданных для этого кластера.

После завершения процесса установки можно воспользоваться инструментом `kubectrl` для вывода списка всех подов, созданных для этого кластера Pulsar и проверки готовности к работе всех необходимых сервисов, для получения IP-адресов и т. д.

**А.3.1. Администрирование Pulsar в среде Kubernetes**

После развертывания кластера Pulsar в среде Kubernetes одной из первых задач является принятие решения о том, как будет осуществляться администрирование этого кластера. К счастью, Pulsar Helm chart создает под `pulsar-mini-toolset-0`, содержащий инструмент командной строки `pulsar-admin`, уже сконфигурированный для взаимодействия с развернутым кластером Pulsar. Поэтому для администрирования кластера требуется только использование команды `kubectrl exec` для доступа к поду и выполнение команд непосредственно в кластере, как показано в листинге А.13.

**Листинг А.13.** Администрирование Pulsar в среде Kubernetes

```

kubectrl exec -it -n pulsar pulsar-mini-toolset-0 /bin/bash

bin/pulsar-admin tenants create manning

bin/pulsar-admin tenants list

"manning"
"public"
"pulsar"

```



Поскольку инструмент `pulsar-admin` одинаково работает и для кластера Kubernetes, и для независимого контейнера Docker, команды `docker exec` и `kubectl exec` могут использоваться как равноценно заменяющие друг друга на протяжении всей книги, если вы предпочли выполнять примеры с применением Kubernetes вместо Docker. Более подробную информацию о практическом применении инструмента `pulsar-admin` можно найти в документации.

### А.3.2. Конфигурирование клиентов

Главное затруднение при установлении соединения с кластером Pulsar в среде Kubernetes – поиск портов, которые прослушивает данный конкретный кластер. По умолчанию порт двоичных данных 6650 и порт администрирования HTTP 8080 не предъявлены за пределами среды Kubernetes. Поэтому сначала необходимо определить, на какие порты узла отображены требуемые стандартные порты.

По умолчанию Pulsar Helm chart предъявляет кластер Pulsar через балансировщик нагрузки Kubernetes. В `minikube` можно воспользоваться командой, показанной в листинге А.14, для проверки прокси-сервиса. Вывод этой команды сообщит нам, на какие порты узла перенаправлен порт двоичных данных и HTTP-порт кластера Pulsar. Номер порта после 80: – это HTTP-порт, а номер порта после 6650: – порт двоичных данных.

**Листинг А.14.** Определение портов клиента Pulsar

```
$kubectl get services -n pulsar | grep pulsar-mini-proxy ❶

pulsar-mini-proxy LoadBalancer 10.110.67.72 <pending>
➡ 80:30210/TCP,6650:32208/TCP 4h16m ❷

$minikube service pulsar-mini-proxy -n pulsar --url ❸
http://192.168.64.3:30210 ❹
http://192.168.64.3:32208 ❺
```

- ❶ Команда определения отображения портов.
- ❷ Вывод сообщает: порт 80 отображен в порт 30210, а порт 6650 – в порт 32208.
- ❸ Команда поиска IP-адреса предъявленных портов в `minikube`.
- ❹ URL прокси по протоколу HTTP.
- ❺ URL для приема двоичных данных по протоколу HTTP.

Теперь мы получили URL сервисов, необходимые для установления соединений клиентов с кластером Pulsar, работающим внутри `minikube`, и их можно использовать вместе с требуемыми токенами безопасности, сгенерированными ранее при конфигурировании клиентов Pulsar для взаимодействия с этим кластером.

# Приложение В.

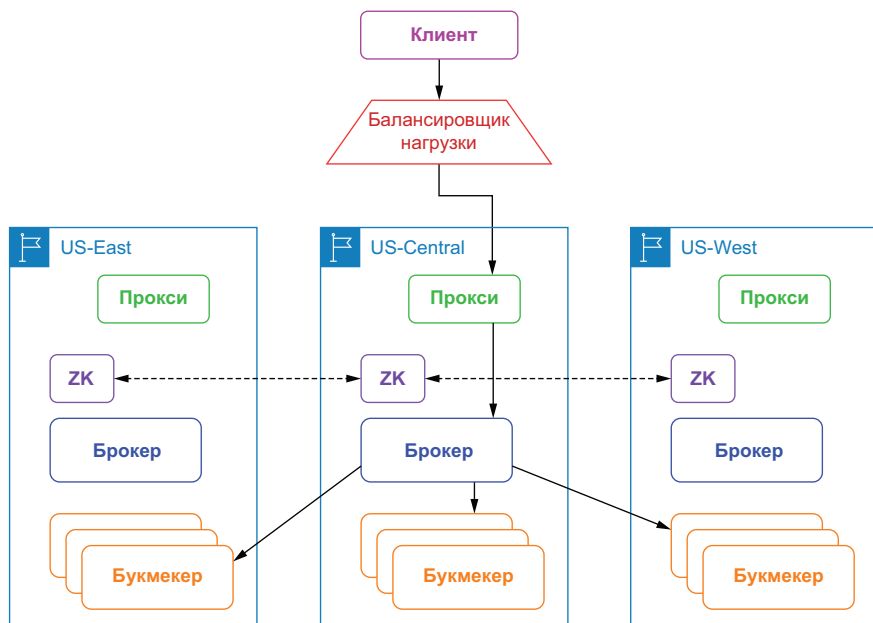
## Георепликация

---

Георепликация – это широко распространенный механизм, используемый для аварийного восстановления в вариантах развертывания с несколькими центрами обработки данных. В отличие от других систем обмена сообщениями по схеме pub-sub, которые требуют дополнительных процессов для зеркалирования сообщений между центрами обработки данных, георепликация автоматически выполняется брокерами Pulsar и может быть включена, отключена или динамически изменена во время выполнения. Обычные механизмы георепликации, как правило, делятся на две категории: синхронные и асинхронные. Apache Pulsar поставляется с репликацией из нескольких центров обработки данных в качестве интегрированной функции, которая поддерживает обе эти стратегии георепликации. В следующих примерах предполагается, что мы выполняем развертывание экземпляра Pulsar в трех регионах облачных провайдеров: Западе США (US-West), Центре США (US-Central) и Востоке США (US-East).

### **В.1. Синхронная георепликация**

Синхронно геореплицируемый вариант установки Pulsar состоит из кластера букмекеров, работающих в нескольких регионах, кластера брокеров, также распределенных по всем регионам, и одной глобальной установки ZooKeeper для формирования единого глобального логического экземпляра во всех доступных регионах, как показано на рис. В.1. Глобальный «растянутый» ансамбль ZooKeeper имеет чрезвычайно важное значение для поддержки этого подхода, поскольку используется для хранения управляемых реестров.



**Рис. В.1.** Клиенты получают доступ к синхронно геореплицируемому кластеру через единственный балансировщик нагрузки, который перенаправляет запрос на публикацию в один из прокси-серверов Pulsar. Прокси-сервер направляет запрос брокеру, владеющему темой, который затем публикует данные в регионах в соответствии с конфигурацией стратегии размещения

В случае синхронной георепликации, когда клиент отправляет запрос на запись в кластер Pulsar в одной географической локации, данные записываются в нескольких букмекерах в разных географических локациях в рамках одного вызова. Подтверждение этого запроса на запись отправляется клиенту только после получения от сконфигурированного количества центров обработки данных подтверждений сохранения переданных данных. Хотя такой подход обеспечивает наивысший уровень гарантий сохранности данных, тем не менее он связан с накладными расходами при задержках в сети между центрами обработки данных при передаче каждого сообщения.

В действительности синхронная георепликация выполняется Apache BookKeeper на уровне хранения Pulsar и основана на стратегии размещения для распределения данных по нескольким центрам обработки данных и обеспечения ограничений доступности. Можно активизировать стратегию размещения с учетом физического оборудования или региона в зависимости от того, работаете ли вы непосредственно с аппаратурой или в облачной среде соответственно, изменив файл конфигурации брокера (*broker.conf*), как показано в листинге В.1.

**Листинг В.1.** Обеспечение стратегии *region-aware*

```
Установите значение true, если кластер распределен по стойкам в пределах
одного центра обработки данных или по нескольким локациям в одном регионе.
bookkeeperClientRackawarePolicyEnabled=true
Установите значение true, если кластер распределен по нескольким
центрам данных или по регионам облачного провайдера.
bookkeeperClientRegionawarePolicyEnabled=true
```

Например, когда вы разрешаете стратегию размещения с учетом региона, BookKeeper будет выбирать букмекеров из разных регионов при формировании нового ансамбля букмекеров. Это гарантирует, что данные из тем будут равномерно распределены по всем доступным регионам. Обратите внимание: только один из этих параметров будет учитываться во время выполнения, причем приоритет будет иметь информация о регионе, если для обоих параметров установлено значение *true*.

Использование одного кластера ZooKeeper для реализации синхронной георепликации также требует некоторых дополнительных изменений конфигурации, чтобы географически распределенные компоненты брокера и букмекера могли работать вместе как единый кластер. Настройка ZooKeeper для такого сценария включает добавление параметра *server.N* в файл *conf/zookeeper.conf* для каждого узла в кластере ZooKeeper, где *N* – количество узлов ZooKeeper, как показано в листинге В.2, где используется один узел ZooKeeper на регион.

**Листинг В.2.** Конфигурация одного узла ZooKeeper для синхронной георепликации

```
server.1=zk1.us-west.example.com:2888:3888
server.2=zk1.us-central.example.com:2888:3888
server.3=zk1.us-east.example.com:2888:3888
```

Кроме изменения файла *conf/zookeeper.conf* в каталоге *conf* каждого установленного экземпляра Pulsar, также потребуется изменить свойство *zkServers* в файле *conf/bookkeeper.conf*, чтобы получить список всех серверов ZooKeeper, как показано в листинге В.3.

**Листинг В.3.** Конфигурация BookKeeper для синхронной георепликации

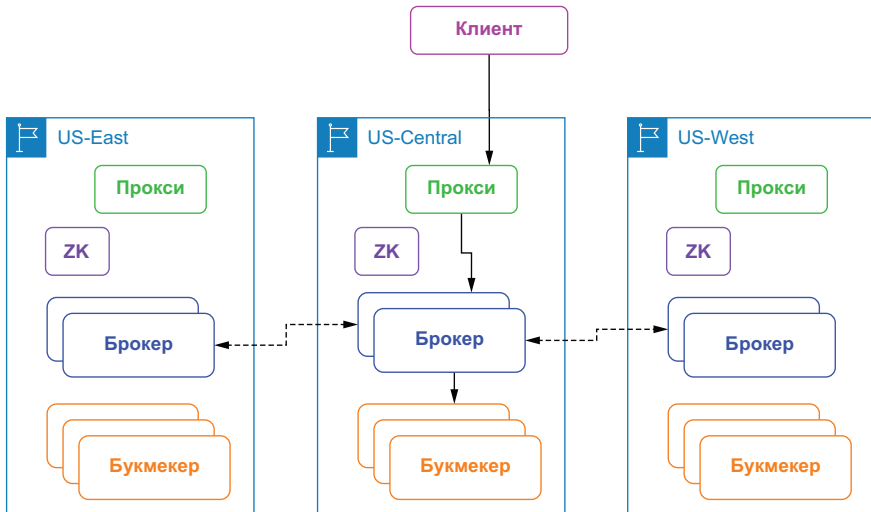
```
zkServers=zk1.us-west.example.com:2181,zk1.us-central.example.com:2181,
zk1.us-east.example.com:2181
```

Аналогичным образом необходимо обновить свойство *ZookeeperServers* в файлах *conf/discovery.conf* и *conf/proxy.conf*, чтобы оно также представляло собой список серверов ZooKeeper, разделенных запятыми, поскольку и прокси-сервер Pulsar, и механизм обнаружения сервисов зависят от ZooKeeper, поэтому требуется предоставить им актуальные метаданные о кластере Pulsar.

Синхронная георепликация обеспечивает более надежные гарантии согласованности данных, чем асинхронная репликация, поскольку данные всегда синхронизируются между центрами обработки данных, что упрощает работу приложений независимо от того, где публикуются сообщения. Синхронный геореплицируемый кластер Pulsar может продолжать функционировать как обычно, даже если весь центр обработки данных выйдет из строя, при этом сбой будет полностью прозрачен для приложений, которые обращаются к кластеру через балансировщик нагрузки. Это делает синхронную георепликацию наиболее подходящей для особо важных вариантов использования, для которых не критична несколько более длительная задержка при публикации.

## В.2. Асинхронная георепликация

Вариант установки Pulsar с асинхронной георепликацией состоит из двух и более независимых кластеров Pulsar, работающих в различных регионах. Каждый кластер Pulsar содержит собственный набор брокеров, букмекеров и узлов ZooKeeper, полностью изолированных друг от друга. При асинхронной георепликации сообщения, производимые в теме Pulsar, сначала существуют в локальном кластере, а затем выполняется асинхронная репликация в удаленные кластеры. Процесс репликации осуществляется через каналы связи между брокерами, как показано на рис. В.2.



**Рис. В.2.** Клиенты получают доступ к синхронно геореплицируемому кластеру через ближайший прокси, который перенаправляет запрос в брокер, владеющий темой. Затем этот брокер публикует полученные данные в букмекерах в том же регионе. Далее брокер выполняет репликацию полученных данных в брокеры других регионов

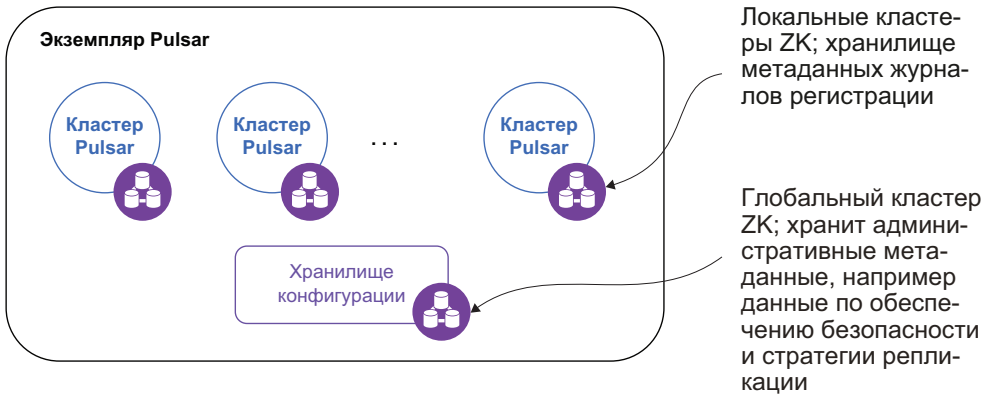
При асинхронной георепликации производитель сообщений не ждет подтверждений от нескольких кластеров Pulsar. Вместо этого производитель получает ответ сразу после того, как ближайший кластер успешно сохранил данные. Затем выполняется репликация данных в другие кластеры Pulsar асинхронно в фоновом режиме. При обычных условиях сообщения реплицируются за тот же интервал времени, который необходим для их распределения по локальным потребителям.

Хотя асинхронная георепликация обеспечивает более короткую задержку, так как клиент не должен ждать ответ из других центров данных, тем не менее при этом не гарантируется полная согласованность данных из-за асинхронности репликации. С учетом неизбежной задержки при асинхронной репликации всегда существует некоторое количество данных, которые не были реплицированы из источника в целевой пункт в любой рассматриваемый момент времени. Таким образом, если вы выбираете для реализации этот паттерн, то приложение должно обеспечивать устойчивость к некоторым потерям данных при обмене за счет уменьшения задержки при публикации. Обычно задержка при двухточечной репликации ограничена временем на передачу и подтверждение приема в сети (network round-trip time – RTT) между удаленными регионами.

Следует отметить, что асинхронная георепликация допустима на основе абонентов Pulsar, а не на основе кластеров, что позволяет конфигурировать репликацию только для тех тем, для которых она действительно необходима. Поэтому каждый отдел или группа может управлять собственными стратегиями репликации данных. Асинхронная георепликация регулируется на уровне пространства имен, обеспечивая более точное управление наборами данных, подлежащих репликации. Это особенно удобно в случаях, когда данным запрещено покидать пределы конкретного региона по существующим законам и/или из соображений обеспечения безопасности.

### **В.2.1. Конфигурирование асинхронной георепликации**

В главе 2 отмечалось, что экземпляр Pulsar состоит из одного или нескольких кластеров Pulsar, совместно работающих как единое целое, которое можно администрировать из одной локации, как показано на рис. В.3. В действительности одним из самых значительных обоснований использования экземпляра Pulsar является возможность георепликации, и только кластеры в одном экземпляре можно сконфигурировать для репликации данных между ними. Из этого следует, что для обеспечения георепликации требуется сначала создать экземпляр Pulsar.



**Рис. В.3.** Экземпляр Pulsar может состоять из нескольких кластеров, находящихся в различных географических регионах

Экземпляр Pulsar использует общий (глобальный) кластер ZooKeeper под названием «хранилище конфигурации» (configuration store) для долговременного хранения информации, относящейся к нескольким кластерам, например стратегии георепликации и обеспечения безопасности на уровне абонентов. Это позволяет определять и управлять такими стратегиями в одной локации. Несмотря на доступность онлайн-документации, мы подробно рассмотрим некоторые шаги этого процесса в следующем разделе.

Следует отметить, что общий для всего экземпляра кластер ZooKeeper необходимо развернуть так, чтобы он был абсолютно независим от отдельных кластеров Pulsar, чтобы в случае критического сбоя в части ансамбля экземпляров ZooKeeper кластеры могли продолжить работу без простоев.

### Развертывание хранилища конфигурации

Помимо установки отдельных кластеров, создание многокластерного экземпляра Pulsar предполагает развертывание отдельного кворума ZooKeeper, который будет использоваться в качестве хранилища конфигурации. Это хранилище конфигурации должно быть реализовано с собственным выделенным кворумом ZooKeeper, распределенным как минимум по трем регионам. Учитывая весьма низкую ожидаемую нагрузку на серверы хранилища конфигурации, можно использовать те же хосты, которые выделены для локального кворума ZooKeeper, но вам придется сделать это либо в форме отдельных процессов ZooKeeper, либо в форме подов Kubernetes, в зависимости от конкретной среды развертывания. Также придется использовать другой порт TCP, чтобы избежать конфликтов портов.

**Листинг В.4.** Конфигурация ZooKeeper для кворума хранилища конфигурации

```
tickTime=2000
dataDir=/var/lib/zookeeper
clientPort=2185
initLimit=5
syncLimit=2
server.1=zk2.us-west.example.com:2185:2186
server.2=zk2.us-central.example.com:2185:2186
server.3=zk2.us-east.example.com:2185:2186
```

1

2

3

- 1 Использование другой локации для хранения журнала транзакций.
- 2 Использование порта, отличающегося от порта для локального экземпляра ZK.
- 3 Кворум состоит из серверов из трех различных регионов, прослушивающих один и тот же порт.

Настройка отдельного кворума ZooKeeper выполняется достаточно просто и подробно документирована на соответствующей странице Apache ZooKeeper. Каждый сервер ZooKeeper содержит единственный JAR-файл, поэтому установка заключается в скачивании этого файла, его распаковке и создании файла конфигурации. По умолчанию место расположения файла конфигурации: *conf/zoo.cfg*. Все серверы в новом кворуме ZooKeeper должны иметь абсолютно одинаковый файл конфигурации, показанный в листинге В.4.

**Инициализация метаданных кластера**

После настройки и запуска вторичного кворума ZooKeeper следующий шаг – заполнение хранилища конфигурации информацией обо всех кластерах, включаемых в экземпляр Pulsar. Эти метаданные можно инициализировать с помощью команды *initialize-cluster-metadata*, показанной в листинге В.5.

**Листинг В.5.** Инициализация метаданных кластеров

```
$ /pulsar/bin/pulsar initialize-cluster-metadata \
--cluster us-west \
--zookeeper zk1.us-west.example.com:2181 \
--configuration-store zk1.us-west.example.com:2184 \
--web-service-url http://pulsar.us-west.example.com:8080/ \
--web-service-url-tls https://pulsar.us-west.example.com:8443/ \
--broker-service-url pulsar://pulsar.us-west.example.com:6650/ \
--broker-service-url-tls pulsar+ssl://pulsar.us-west.example.com:6651/
```

1

2

3

- 1 Имя кластера, который будет использоваться при настройке репликации.
- 2 Строка для установления соединения с локальным ZK.
- 3 Строка для установления соединения для хранилища конфигурации.



Команда в листинге В.5 связывает все URL для установления соединения с заданным именем кластера и сохраняет эту информацию в хранилище конфигурации. Сохраненная информация используется, когда активизируется репликация для установления соединений с брокерами, необходимых для обмена данными между ними (например, репликация данных из US-West в US-East). Требуется выполнить эту команду для каждого кластера Pulsar, добавляемого в экземпляр.

### Конфигурирование сервисов для использования хранилища конфигурации

После заполнения хранилища конфигурации всеми метаданными, относящимися к кластерам Pulsar в текущем экземпляре, необходимо внести изменения в несколько файлов конфигурации каждого кластера для обеспечения георепликации. Поскольку георепликация осуществляется через каналы обмена данными между брокерами, самым важным является файл конфигурации *conf/broker.conf*, показанный в листинге В.6.

#### Листинг В.6. Обновление broker.conf для обеспечения асинхронной георепликации

```
Локальные серверы ZooKeeper.
zookeeperServers=zk1.us-west.example.com:2181,zk2.us-
➔ west.example.com:2181,zk3.us-west.example.com:2181 ❶

Конфигурация строки соединения с кворумом хранилища.
configurationStoreServers=zk2.us-west.example.com:2185,zk2.us-
➔ central.example.com:2185,zk2.us-east.example.com:2185 ❷

clusterName=us-west ❸
```

- ❶ Использование локального кворума ZK, как и ранее.
- ❷ Использование второго кворума ZK для хранилища конфигурации.
- ❸ Определение имени кластера, которому принадлежит данный брокер.

Убедитесь в том, что вы установили параметр `ZookeeperServers` для отображения локального кворума, а параметр `ConfigurationStoreServers` – для отображения кворума хранилища конфигурации. Также необходимо указать имя кластера, которому принадлежит брокер, в параметре `clusterName`, но будьте внимательны – используйте значение, указанное в команде `initialize-cluster-metadata`. Убедитесь в том, что порты брокера и веб-сервиса также соответствуют значениям, которые были указаны в команде `initialize-cluster-metadata`. В противном случае процесс репликации завершится неудачно, поскольку брокер-источник попытается установить связь через неправильный порт.

Если вы используете механизм обнаружения сервисов, включенный в Pulsar, то потребуется изменить несколько параметров в файле конфигурации *conf/discovery.conf*. В частности, необходимо

установить для параметра `ZooKeeperServers` строку установления соединения с кворумом `ZooKeeper` кластера, а для параметра `ConfigurationStoreServers` – строку установления соединения с кворумом хранилища конфигурации, используя те же значения, которые указаны в файле конфигурации брокера. После завершения обновления всех необходимых файлов конфигурации эти сервисы необходимо перезапустить, чтобы новые свойства вступили в силу.

## В.3. Паттерны асинхронной георепликации

При асинхронной георепликации Pulsar предоставляет абонентам достаточно высокую степень гибкости для специализации стратегии репликации. Это означает, что приложению разрешается настраивать репликацию по схеме ведущих–ведущий (active-active) с полностью ячеистой структурой, репликацию по схеме активный–резервный (active-standby) и агрегатную репликацию между несколькими центрами данных. Кратко рассмотрим, как реализуется каждый из этих паттернов с использованием Pulsar.

### В.3.1. Георепликация по схеме ведущих–ведущий

Асинхронная георепликация в Pulsar управляется на уровне каждого абонента. Это означает, что георепликацию между кластерами можно разрешить только в том случае, если создан абонент, обеспечивающий доступ ко всем задействованным кластерам. Чтобы настроить мультиактивную георепликацию, необходимо указать, к каким кластерам абонент имеет доступ, с помощью инструмента командной строки `pulsar-admin`, как видно в листинге В.7, где показана команда создания нового абонента и предоставления ему разрешения на доступ только к кластерам US-East и US-West.

**Листинг В.7.** Предоставление абоненту прав доступа к указанным кластерам

```
$ /pulsar/bin/pulsar-admin tenants create customers \ ❶
 --allowed-clusters us-west,us-east \ ❷
 --admin-roles test-admin-role
```

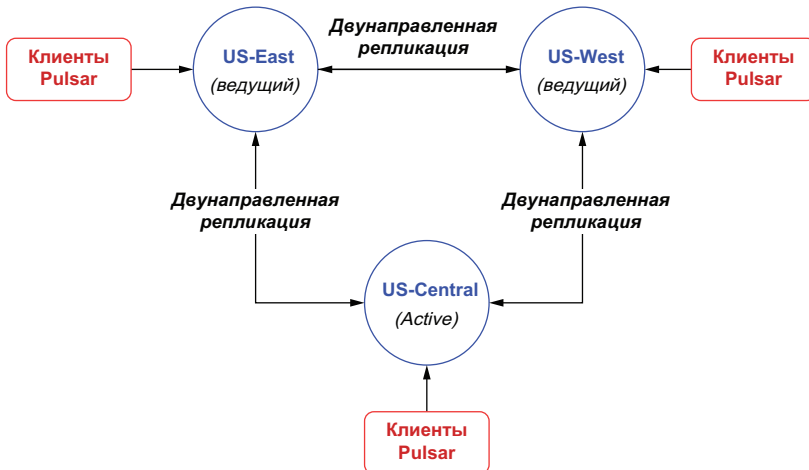
- ❶ Создание нового абонента с именем `customers`.
- ❷ Предоставление этому абоненту права доступа только к этим двум кластерам.

После создания абонента необходимо сконфигурировать георепликацию на уровне пространства имен. Поэтому сначала потребуется создать пространство имен с помощью инструмента командной строки `pulsar-admin`, затем связать это пространство имен с кластером или с несколькими кластерами, выполнив команду `set-clusters`, показанную в листинге В.8.

**Листинг В.8.** Связывание пространства имен с кластером

```
$ /pulsar/bin/pulsar-admin namespaces create customers/orders
$ /pulsar/bin/pulsar-admin namespaces set-clusters customers/orders \
 --clusters us-west,us-east,us-central
```

По умолчанию после завершения конфигурирования репликации между двумя или несколькими кластерами, как показано в листинге В.8, все сообщения, опубликованные в темах в пространстве имен в одном кластере, асинхронно реплицируются во все остальные кластеры в заданном списке. Таким образом, поведением по умолчанию является полносвязная ячеистая структура репликации всех тем в пространстве имен с публикацией сообщений в нескольких направлениях, как показано на рис. В.4. Если у вас есть только два кластера, поведение по умолчанию можно рассматривать как конфигурацию кластера «ведущий–ведущий», в которой данные доступны в обоих кластерах для обслуживания клиентов, а в случае сбоя одного кластера все клиенты могут быть перенаправлены к сохранившему работоспособность кластеру без перерыва.

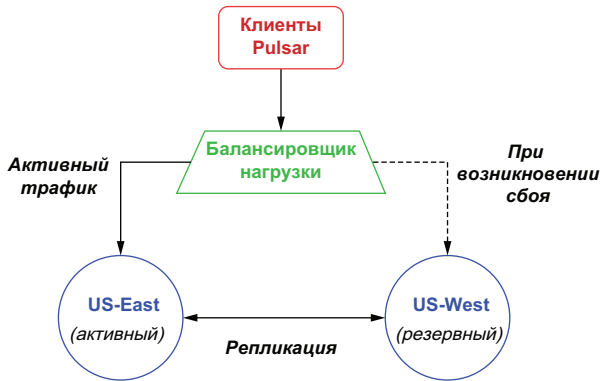


**Рис. В.4.** Поведением по умолчанию является полносвязная ячеистая структура репликации между кластерами. Сообщения, публикуемые в теме в реплицируемом пространстве имен кластера US-East, будут перенаправляться в кластеры US-East и US-Central

Помимо схемы георепликации с полносвязной ячеистой структурой (ведущий–ведущий) существует несколько других паттернов репликации, которые также можно использовать. Для восстановления после катастроф часто применяется паттерн репликации по схеме активный–резервный (active-standby).

### В.3.2. Георепликация по схеме активный–резервный

В этом случае требуется сохранение актуальной копии кластера в другой географической локации, чтобы можно было возобновить работу в случае сбоя с наименьшими потерями данных или минимальным временем восстановления. Поскольку Pulsar не предоставляет средств для определения односторонней репликации пространств имен, единственный способ создания такой конфигурации – ограничение клиентов единственным кластером, известным как активный кластер, с обеспечением аварийного переключения на резервный кластер только при сбое. Обычно это можно сделать с помощью балансировщика нагрузки или другого механизма сетевого уровня, который обеспечивает прозрачность перехода для клиентов, как показано на рис. В.5. Клиенты Pulsar публикуют сообщения в активном кластере, а затем сообщения реплицируются в резервный кластер для безопасного хранения.



**Рис. В.5.** Асинхронную георепликацию можно использовать для реализации сценария активный–резервный, в котором все данные в текущем пространстве имен перенаправляются в кластер, используемый только в случае возникновения критического сбоя

Вероятно, вы заметили, что репликация данных Pulsar по-прежнему будет выполняться двунаправленно, а это означает, что кластер US-West попытается отправить данные, полученные в момент сбоя, в кластер US-East. При этом может возникнуть проблема, если сбой связан с одним или несколькими компонентами кластера Pulsar или сеть кластера US-East недоступна. Поэтому вам следует рассмотреть возможность добавления кода выборочной репликации в приложения-производители Pulsar, чтобы предотвратить попытки кластера US-West реплицировать сообщения в кластер US-East, который, вероятнее всего, полностью неработоспособен.

Можно выборочно ограничить репликацию, явно указав список репликации для сообщений на уровне приложения. Код в листинге В.9 показывает пример создания сообщения, которое будет репли-

цироваться только в кластер US-West, что является именно тем поведением, которое необходимо в этом сценарии активный-резервный.

#### Листинг В.9. Выборочная репликация для каждого сообщения

```
List<String> restrictDatacenters = Lists.newArrayList("us-west");

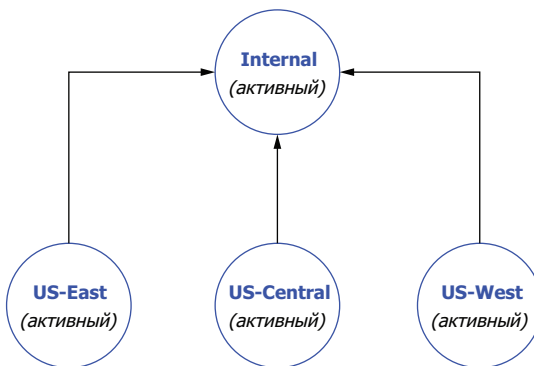
Message message = MessageBuilder.create()
 ...
 .setReplicationClusters(restrictDatacenters)
 .build();

producer.send(message);
```

Иногда необходимо собрать сообщения из нескольких кластеров в одной локации в целях агрегирования. Один из примеров: сбор всех платежных данных, поступивших из нескольких географических регионов, для обработки и сохранения.

### В.3.3. Агрегатная георепликация

Предположим, что имеются три кластера, каждый из которых активно обслуживает клиентов GottaEat в своих регионах, и четвертый кластер Pulsar с именем `internal`, который полностью изолирован от веб-среды и доступен только сотрудникам компании. Этот кластер используется для агрегирования данных от всех кластеров, обслуживающих клиентов, как показано на рис. В.6. Чтобы реализовать агрегатную георепликацию для четырех обслуживающих кластеров, необходимо выполнить команды, показанные в листинге В.10, которые сначала создают абонент `E-payments`, затем предоставляют доступ ко всем кластерам.



**Рис. В.6.** Конфигурация агрегатной георепликации для сбора сообщений от трех кластеров Pulsar, обслуживающих клиентов, во внутреннем кластере Pulsar для объединения и анализа

Далее необходимо создать пространство имен для каждого кластера обслуживания клиентов (например, `E-payments/us-east-`

payments). Запрещается использовать такое имя, как E-payments/payments, поскольку это приведет к полной репликации по схеме «полносвязная ячеистая структура», если вы попытаетесь его задействовать, так как каждый кластер будет содержать такое пространство имен. Таким образом, для нормальной работы каждого кластера требуется собственное пространство имен.

#### Листинг В.10. Агрегатная георепликация

```
/pulsar/bin/pulsar-admin tenants create E-payments \ ❶
--allowed-clusters us-west,us-east,us-central,internal

/pulsar/bin/pulsar-admin namespaces create E-payments/us-east-payments ❷
/pulsar/bin/pulsar-admin namespaces create E-payments/us-west-payments
/pulsar/bin/pulsar-admin namespaces create E-payments/us-central-payments

/pulsar/bin/pulsar-admin namespaces set-clusters \ ❸
E-payments/us-east-payments --clusters us-east,internal

/pulsar/bin/pulsar-admin namespaces set-clusters \ ❹
E-payments/us-west-payments --clusters us-west,internal

/pulsar/bin/pulsar-admin namespaces set-clusters \ ❺
E-payments/us-central-payments --clusters us-central,internal
```

- ❶ Создание глобального абонента для платежей (payments).
- ❷ Создание пространств имен, предназначенных для каждого кластера.
- ❸ Конфигурирование кластера US-East для внутренней репликации.
- ❹ Конфигурирование кластера US-West для внутренней репликации.
- ❺ Конфигурирование кластера US-Central для внутренней репликации.

Если вы решили реализовать этот паттерн и намерены выполнять одинаковые копии приложения во всех кластерах, обслуживающих клиентов, то следует обеспечить присваивание конфигурируемого имени темы, чтобы приложение, работающее в кластере US-East, знало о публикации сообщений в темах, расположенных в пространстве имен us-east-payments. Иначе репликация не будет работать.

# Предметный указатель

---

## А

- Авторизация
  - пример сценария 258
  - стратегия 261
- Администрирование Pulsar 117
  - API на языке Java 119
  - метрики уровня темы, постоянное хранение 144
  - просмотр сообщений 147
  - создание
    - абонента 118
    - пространства имен 118
    - темы 118
- Анализ
  - статистический 432
- Атомарность, согласованность, изолированность и долговечность (ACID) 60

## Б

- Блокирующий скрипт 120

## В

- Вектор признаков 397
  - вычисление 400
  - хранилище 398
- Временной ряд данных 426
- Время на передачу и подтверждение приема в сети 473

## Г

- Георепликация 70, 469
  - асинхронная 70, 472
    - паттерн агрегирования 480
    - паттерн «активный-резервный» 479

- паттерн «ведущий-ведущий» 477

- синхронная 71, 469

- Глубокое обучение 410

## Д

- Доступ к данным
  - паттерн ввода-вывода 83
    - catch-up read (догоняющее чтение) 83
    - tailing read (хвостовое чтение) 83
    - write (запись) 83

## Ж

- Журнал регистрации (backlog) 106
  - квота 106

## З

- Задержка при анализе 416
- Задержка при принятии решения 416
- Задержка при сборе данных 416
- Замедленная обратная реакция (backpressure) 338

## И

- Исполнительный механизм 420

## К

- Квантиль 438
- Конвейер данных 307
- Коннектор ввода-вывода 195
  - PushSource 201
    - администрирование 226
    - вывод списка 226
    - мониторинг 228
  - встроенный 221
    - приемник для MongoDB 224

источник (source) 195, 199  
    модель на основе извлечения 199  
    пример разработки 205  
приемник (sink) 195  
    пример разработки 203  
развертывание 215  
    в режиме localrun 215  
    в режиме кластера 215  
    отладка 218  
тестирование 209  
    комплексное 211  
    модульное 209  
упаковка 214  
Концентратор датчиков 421, 422  
Корпоративная система обмена  
    сообщениями 34  
    асинхронный режим 35  
    подтверждение приема 36  
    потребление сообщений 36  
    сохранение сообщений 36

## М

Маршрутизатор сообщений 310  
Методика простого скользящего  
    среднего 430  
Микропакетная обработка 155  
Многоуровневое хранилище 110  
Модель машинного обучения 393  
    вектор признаков 397  
    развертывание 407  
    режим развертывания 393  
        обработка почти в реальном  
        времени 394, 395  
    пакетная обработка 393  
Модель программирования  
    без сервера 157

## Н

Направленный ациклический граф 158, 308  
Неблагоприятное событие  
    замедление работы функции 332  
    полный отказ функции 331  
    функция, не продолжающая работу 334  
Нейронная сеть 409  
    развертывание средствами  
        языка Java 412  
    тренировка 410

## О

Обмен сообщениями 33  
Обработка в одно касание 437  
Ошибка  
    неисчезающая 336  
    неустойчивая 336

## П

Пакетная обработка 154  
Паттерн обеспечения устойчивости  
    кеш 358  
    обновление идентификационных  
        данных 362  
    ограничитель времени 354  
    ограничитель скорости 351  
    откат 360  
    повтор 340  
    прерыватель замкнутого цикла 344  
Периферийная аналитика 425  
Периферийные вычисления 418  
Периферийный анализ  
    многомерный 441  
        создание двунаправленной сетки  
        обмена сообщениями 441  
        создание многомерного набора  
        данных 445  
    одномерный 429  
        аппроксимация 437  
        снижение шума 430  
        статистический анализ 432  
Постоянство хранения данных 58  
Потоковая обработка 153  
    нативная 155  
Программное обеспечение среднего  
    звена, ориентированное на  
    сообщения 40  
Промышленный интернет  
    вещей (IIoT) 419  
    уровень восприятия и реакции 419  
    уровень обработки данных 421  
        на основе Pulsar 422  
    уровень передачи данных 421  
Публикация–подписка, паттерн 98

## Р

Ранг метрики 439  
Распределенная система обмена  
    сообщениями 32, 46  
Реестр схем Pulsar  
    Avro 289  
    пример использования 287  
    стратегия проверки совместимости 282  
        обратная 282  
        полная 286  
        прямая 284

## С

Сервис авторизации  
    сертификатов (CA) 235  
Сервисная шина предприятия 42



Сервис-ориентированная архитектура (SOA) приложений 42  
 Система на микросхеме 422  
 Система обмена сообщениями  
   очередь 38  
   публикация–подписка 37  
   распределенная 46  
     журнал регистрации записей с упреждением 47  
 сервисная шина предприятия 42  
 тема 37  
 уровень  
   обслуживания 40  
   хранения 40  
 хранилище на основе разделов Kafka 48  
 хранилище на основе сегментов Pulsar 51  
 Скetch  
   выборки (sampling sketches) 438  
   квантилей (quantile sketches) 438  
   мощности множества (количества элементов) (cardinality sketches) 438  
   часто встречающихся элементов (frequent item sketch) 438  
 Скetch данных 437, 446  
 Смещение среднего значения 430  
 Стратегия недоставленных сообщений 132

## Т

Телеметрические данные 426  
   набор  
     многомерный 428  
     одномерный 428  
   сезонные колебания 426  
   тренд 426  
   цикличность 426  
   шум 427  
 Телеметрия 426  
 Токен безопасности 249  
 Уменьшение временной ценности данных 416  
 Устойчивость 329  
 Функция с сохранением состояния 166  
 Хранилище конфигурации 78, 474  
 Экземпляр Pulsar 78

## А

Actuator 420  
 AMQP, протокол 34  
 Analysis latency 416  
 AnomalyDetector, функция 446  
 Apache Avro, формат сериализации 277

Apache DataSketches, библиотека 438  
 Apache Pulsar  
   многоуровневая архитектура 52  
   узел-брокер (broker node) 53  
   узел-букмейкер (bookie node) 53  
   распределенная система обмена сообщениями 32  
   сравнение с Apache Kafka 52  
 apply, метод реализации  
   Pulsar Functions 159  
 available-sinks, команда 227  
 available-sources, команда 227  
 AWS\_ACCESS\_KEY\_ID, переменная среды 112  
 AWS\_SECRET\_ACCESS\_KEY, переменная среды 112

## В

BookKeeper 87  
   управляемый реестр 86  
   хранилище метаданных 91

## С

Capture latency 416  
 CircuitBreaker, объект 349  
 Circuit breaker, паттерн 344  
 Cluster master 454  
 clusterName, параметр  
   георепликации 476  
 Compare-and-swap (CAS сравнение с обменом), операция 90  
 Configuration store 78, 474  
 ConfigurationStoreServers, параметр  
   конфигурации георепликации 476  
 Consumer 98  
 consumer\_backlog\_eviction, стратегия  
   регистрации сообщений 107  
 Context, объект 160, 167  
 Credentials refresh, паттерн 362  
 CustomerSimulatorSource, класс  
   коннектора ввода-вывода 292

## Д

DAG, directly acyclic graph 158  
 Data durability 58  
 Data pipeline 307  
 DDS, протокол 34  
 Dead-letter policy 131  
 Decision latency 416  
 Deep learning 410  
 Deeplearning4j, библиотека 412  
 defaultNumberOfNamespaceBundles, свойство 81  
 defaultRetentionSizeInMB, свойство 104

defaultRetentionTimeInMinutes,  
свойство 104  
Diminishing time value of data 416  
DirectoryConsumerThread, класс  
фонового потока 206  
docker exec, команда 118  
docker log, команда 117

**E**

Edge analytics 425  
Edge computing 418  
EMS, enterprise messaging system 34  
ESB, enterprise service bus 42

**F**

Fallback, паттерн 360  
Feature vector 397  
function getstatus, команда проверки  
Pulsar Functions 188

**G**

Geo-replication 70  
getState, метод извлечения состояния 167  
Golang SDK 165  
gRPC, протокол 273

**H**

Helm 459  
Helm chart 460  
структура 460

**I**

IMDG, in-memory data grid 371  
initialize-cluster-metadata, команда 475  
IoT gateway 421  
IoT-шлюз 421

**J**

Java SDK 161  
java.util.Function, интерфейс 159  
JPMML, библиотека 407  
JWT (JSON Web Token)  
аутентификация 249

**K**

Kafka 52  
длительность хранения сообщений 64  
подтверждение приема сообщений 62  
смещение потребителя (consumer  
offset) 62  
постоянство хранения данных 58  
второстепенная реплика 59  
основная реплика 59  
потребление сообщений 55  
логический подписчик 55  
ребалансирование 57

Keras, библиотека Python 410  
формат H5 412  
KeywordFilterFunction, функция  
(пример для тестирования) 170  
kubectrl, инструментальное средство  
управления кластерами  
Kubernetes 456  
Kubernetes 454  
Pulsar Helm chart 458  
кластер 454  
главный 454  
узел 455

**L**

LinkedBlockingQueue, очередь 203  
listener, паттерн 129  
loadBalancerBrokerOverloadedThresholdP  
ercentage, свойство 83  
LocalRunner, вспомогательный класс для  
тестирования Pulsar Functions 173

**M**

MessageListener, интерфейс 127  
minikube, вариант кластера Kubernetes  
на одном компьютере 457  
MOM, message-oriented middleware 40  
кластеризация 41  
сегментирование (sharding) 41  
MQTT (message queuing telemetry  
transport передача телеметрических  
данных в очереди сообщений),  
протокол 421  
MSMQ, протокол 34

**N**

NAR (NiFi archive) файл 179  
Near real-time processing 394  
Neural net 409  
Non-transitive fault 336

**O**

One-touch processing 437  
org.apache.pulsar.functions.api.Function,  
интерфейс 161  
org.apache.pulsar.functions.api.Record,  
интерфейс 197  
org.apache.pulsar.functions.api.SerDe,  
интерфейс 161  
org.apache.pulsar.io.core.PushSource,  
класс 202  
org.apache.pulsar.io.core.Sink,  
интерфейс 196  
org.apache.pulsar.io.core.Source,  
интерфейс 199

**P**

- peek-messages, команда 147
- pf.Start(), метод для реализации Pulsar Functions на языке Go 166
- Predictive Model Markup Language (PMML) 402
- process, метод реализации Pulsar Functions на языке Python 160
- Producer 98
- producer\_exception, стратегия регистрации сообщений 107
- producer\_request\_hold, стратегия регистрации сообщений 107
- Pull-based model 199
- Pulsar
  - абонент (tenant) 93
  - автоматическое восстановление данных 55
  - автоматическое снижение нагрузки 82
  - архитектура
    - логическая 92
    - многоуровневая 79
    - физическая 78
  - балансирование нагрузки 82
  - брокер 79
  - букмекер 79, 88
  - гарантированная доставка сообщений 66
  - георепликация 70
  - динамическое масштабирование 54
  - длительность хранения сообщений 64
  - долговременное хранение данных 104
  - кластер 79
  - конфигурационные данные 91
  - логическая архитектура хранилища потоковых данных 85
  - BookKeeper 86
  - многоуровневое хранилище 110
  - AWS 111
  - набор (bundle) 81
  - неограниченное масштабирование 66
  - обеспечение безопасности 233
    - авторизация 255
    - аутентификация 242
    - шифрование передаваемых данных 233
    - шифрование хранимых сообщений 264
  - обеспечение устойчивости
    - несколько уровней (паттернов) 365
  - подписка 99
  - отказоустойчивая 101
  - совместно используемая 102
  - совместно используемая по ключу 103
  - эксклюзивная 100
- подтверждение приема сообщений 62
- выборочное (селективное) 63
- накопительное 63
- регистрационный журнал с маркером (cursor ledger) 62
- постоянство хранения данных
  - журнал с упреждающей записью (write-ahead log, WAL) 60
- потребитель 98
- производитель 98
- прокси 79, 80
- пространство имен (namespace) 94
- расширение кластера без ребалансирования 67
- реестр схем 271
- архитектура 275
- обмен данными между сервисами 271
- проверка совместимости 280
- роль 256
  - администратор абонента 257
  - клиент 257
  - суперпользователь 257
- срок хранения сообщений 108
- стратегия времени жизни (time to live TTL) 108
- тема 86
  - адресация 97
  - подписка 98
  - размещение в нескольких разделах 95
- тема (topic) 94
- уровень обслуживания без сохранения состояния 81
- уровень хранения потоковых данных 85
  - распределенная система регистрации записи с упреждением (write-ahead log WAL) 85
- устойчивость к критическим ошибкам 67
- pulsar-admin, средство администрирования 117
- pulsar-admin functions, средство развертывания Pulsar Functions 179
- pulsar-admin sink get, команда 229
- pulsar-admin sink list, команда 226
- pulsar-admin sink status, команда 228
- pulsar-admin sink update, команда 230

- pulsar-client, инструментальное средство 120
  - PulsarClient, объект 124
  - Pulsar Functions 156
    - аналог лямбда-функций 157
    - доступ к данным
      - вариант использования 371
      - получение локации водителя 383
      - проверка устройства 371
    - источник данных 369
    - комплексное 173
    - модель программирования без сервера 157
    - направленный ациклический граф 331
    - окно данных
      - стратегия вытеснения (eviction policy) 434
      - стратегия триггера (trigger policy) 434
    - ориентированные на язык функции 159
    - паттерн
      - агрегатор (aggregator) 312
      - маршрутизатор на основе содержимого 321
      - разделитель (splitter) 311
      - список получателей (recipient list) 316
      - транслятор сообщений 322
      - улучшение содержимого 325
      - фильтр содержимого 328
    - развертывание 178
      - auto\_ack, параметр конфигурации 183
      - processing-guarantees, параметр конфигурации 184
      - retain-ordering, параметр конфигурации 183
      - гарантия доставки at-least-once (минимум однажды) 185
      - гарантия доставки at-most-once (максимум однажды) 184
      - гарантия доставки effectively-once (фактически однажды) 186
      - генерация артефакта 179
      - режим localrun 190
      - режим кластера (cluster mode) 190
    - тестирование 170
      - модульное 171
    - устойчивость 331
    - функция с сохранением состояния 167
  - Pulsar Functions SDK 160
  - Pulsar Helm chart 461
  - Pulsar instance 78
  - Pulsar IO connectors 221
  - Pulsar IO Connectors 196
  - pulsar-managed-ledger-admin, утилита 91
  - Push-based model 201
  - PushSource, класс 205
  - putState, метод сохранения состояния 167
  - Python SDK 163
- Q**
- Quantile 438
- R**
- reader, интерфейс 132
  - receive(), метод 127
  - Resilience 329
  - REST, протокол 34, 273
  - Retry, паттерн 340
  - RoundRobinPartition, режим маршрутизации 96
  - RPC, вызов удаленной процедуры 34
  - RTT, network round-trip time 473
- S**
- SCADA (supervisory control and data acquisition система диспетчерского управления и сбора данных) 420
  - ScheduledThreadPoolExecutor, метод Java 449
  - SchemaInfo, объект 277, 280
  - SchemaInfo, структура данных для реестра схем 276
  - Security token 249
  - Sensor hub 421
  - set-clusters, команда 477
  - setCryptoFailureAction(), метод 268
  - SinglePartition, режим маршрутизации 96
  - SinkContext, объект 196
  - SketchConsolidator, функция 449
  - SMA, simple moving average 430
  - SOAP, протокол 34
  - SoC, system on a chip 422
  - SourceContext, объект 200
  - Stream processing 153
  - Subscription 99
- T**
- Telemetry 426
  - Tiered storage 110
  - Time-series data 426
  - TLS (transport layer security), криптографический протокол передачи данных 233

TLS-аутентификация 243

Transient fault 336

## V

Vavr, библиотека Java 362

## W

Write-ahead log 47

## Z

zkServers, свойство конфигурации  
георепликации 471

ZooKeeper 87

хранение метаданных 89

ZookeeperServers, свойство конфигурации  
георепликации 471, 477

zookeeper-shell, утилита 90

Книги издательства «ДМК Пресс» можно заказать  
в торгово-издательском холдинге «КТК Галактика» наложенным платежом,  
выслав открытку или письмо по почтовому адресу:

115487, г. Москва, пр. Андропова д. 38 оф. 10.

При оформлении заказа следует указать адрес (полностью),  
по которому должны быть высланы книги;

фамилию, имя и отчество получателя.

Желательно также указать свой телефон и электронный адрес.

Эти книги вы можете заказать и в интернет-магазине:

**[www.galaktika-dmk.com](http://www.galaktika-dmk.com).**

Оптовые закупки: тел. (499) 782-38-89.

Электронный адрес: **[books@aliants-kniga.ru](mailto:books@aliants-kniga.ru)**.

**Дэвид Хьеррумгор**

## **Apache Pulsar в действии**

Главный редактор *Мовчан Д. А.*

*[dmkpress@gmail.com](mailto:dmkpress@gmail.com)*

Зам. главного редактора *Сенченкова Е. А.*

Перевод *Снастин А. В.*

Корректор *Абросимова Л. А.*

Верстка *Луценко С. В.*

Дизайн обложки *Мовчан А. Г.*

Формат 70×100 1/16.

Гарнитура «NewBaskervilleC». Печать цифровая.

Усл. печ. л. 39,81. Тираж 100 экз.

Веб-сайт издательства: *[www.dmkpress.com](http://www.dmkpress.com)*

Надежный обмен сообщениями между серверами является сердцем распределенного приложения. Apache Pulsar — это гибкая платформа обмена сообщениями в реальном времени, созданная для работы в Kubernetes и обеспечивающая масштабируемость и отказоустойчивость, необходимые для облачных систем. Pulsar поддерживает как потоковую передачу, так и организацию очередей сообщений, и, в отличие от других решений, он может обмениваться данными по нескольким протоколам, включая MQTT, AMQP и двоичный протокол Kafka.

Книга научит вас создавать масштабируемые системы потокового обмена сообщениями с использованием Pulsar. Вы начнете с быстрого ознакомления с корпоративными системами обмена сообщениями и откроете для себя уникальные преимущества Pulsar. Следуя четким описаниям и выполняя практические примеры, вы будете использовать фреймворк Pulsar Functions для разработки приложения на основе микросервисов.

*Книга предназначена для опытных разработчиков на языке Java. Предварительные знания о платформе Apache Pulsar не требуются.*

#### Основные темы:

- публикация сообщений из Pulsar в сторонние репозитории данных и на другие платформы;
- проектирование и разработка функций Apache Pulsar;
- создание управляемого событиями приложения обработки заказов на доставку продуктов питания.

Дэвид Хьеррумгор — участник проекта Apache Pulsar. В настоящее время он является разработчиком в компании StreamNative, где создает наиболее эффективные решения на основе Pulsar.

«В книге с удивительной легкостью сочетается теория и абстрактные концепции с ясностью пошаговых примеров. Рекомендую всем!»

*Маттео Мерли, один из основателей Apache Pulsar*

«Книга дает читателям представление о том, как работает «магия»... Определенно рекомендуется к изучению».

*Генри Сануипа, Splunk*

«Исчерпывающая, практическая и очень занимательная книга».

*Сатей Кумар Саху, Honeywell*

«Лучшая книга для того, чтобы начать работать с Pulsar».

*Эммануэль Пиччинелли, Empirix*

Интернет-магазин: [www.dmkpress.com](http://www.dmkpress.com)

Оптовая продажа: КТК «Галактика»  
[books@aliants-kniga.ru](mailto:books@aliants-kniga.ru)



ISBN 978-5-93700-251-8



9 785937 002518 >